Landmark XVI-Q — Q11 (P.6 evanescent counterterm: BRST-trivial directions as sigma-images in the gauge-fixed BV variables, honest partial)

One-loop κ^3: evanescent three-point defect σ_0-exact at pole level, X non-unique (ghost-equation compatible); trivial span conditional, 43 directions unclassified, operator basis and p22 consistency open. O.1 CLOSED (scoped, κ^2 two-point) unchanged

2026-10-12

Contents

1 Q11: BRST-trivial directions as σ-images in the gauge-fixed BV variables **3**

Q10 layer **4**

2 Q10: erratum on the trivial counts; anatomy of the family vectors **4**

Q09 layer **5**

3 Q09: review fixes; derived BRST-trivial directions of X **5**

Q08 layer **6**

4 Q08: structure of the κ^3 evanescent counterterm **6**

Q07 layer **7**

5 Q07: κ^3 three-point rows — h h c, evanescent counterterms, pole-level σ_0 closure **7**

Q06 layer **8**

6 Q06: P.6 continued (honest partial) **8**

Q05 layer **10**

7 Q05: P.6 continued (one-loop κ^3 three-point sector) — honest partial **10**

Q04 layer **11**

8 Q04: P.6 entry — one-loop κ^3 three-point sector (honest partial) **11**
8.1 Inventory and vertices (Constructed) . . . 11
8.2 Identity at κ^3 (ledger Constructed) . . . 11
8.3 Evaluation (Partial) . . . 11
8.4 Next step . . . 11
8.5 Q03 review findings (low), folded in . . . 12

Q03 layer **13**

9 Q03: correction release (no new science) 13
9.1 O.1 status . . . 13
9.2 Correction 1: counterterm domain of O.1 item 5 (medium) . . . 13
9.3 Correction 2: prescription; XVI-O closure reason 8 (medium) . . . 13
9.4 What O.1 CLOSED (scoped) does not imply . . . 13
9.5 Caveats . . . 13
9.6 Correction 3: gate and guard (low) . . . 13

Q02 layer 14

10 Q02 scope and outcome 14

11 Bulk graphs 14

12 Bulk retained identity, counterterms, closure 15

13 O.1 walk (verbatim ledger) 15

14 Guard, gate and records 16

Q01 layer 17

15 XVI-Q Q01: scope and registration 17

16 O.2: the classical κ^2 master $2(S_0, S_2) + (S_1, S_1) = 0$ on S_{inv} 17

17 d guard (carried) 18

18 Status and Q02 plan 18

19 Appendix: certificates, status, intake 19

20 Appendix: source listings 81

1 Q11: BRST-trivial directions as σ-images in the gauge-fixed BV variables

Construction

The gauge fixing is the canonical shift by $\Psi = \bar{c} \cdot F(h)$ with B integrated out. So the antifield of h enters only through $\tilde{K} = K + \bar{c} \cdot F'(h)$, and the Q08 ghost equation ($\rho = -2$) is this statement on the amplitudes. A local ghost-number -1 functional must therefore be built from $\tilde{K}$: $Y_F = \tilde{K} \cdot F(h, h)$ has the $\bar{c}$ partner $\rho\, T(K \cdot F)$, and the Q08 P family is absorbed into it. $Y_G = c^* \cdot G(c, h)$ gives X_{Khc} ($c^* \to D^T \tilde{K}$), its $\bar{c}$ partner, and the $c^* cc$ piece ($h \to s_0 h$, antisymmetrised in the two ghosts). Q08 lacked that last piece. $\sigma = (S_0, \cdot)$ is applied with the same channel operators that build the rows, with the antibracket sign -1 between antifield-type and field-type pieces. Every generator gets one test: rows, Bose and ghost equation. No per-generator fit is made.

Results

Q08 F pieces violate only the $h \cdot h \cdot c$ row. 12 generators fail through the antifield-type piece alone, because the $\bar{c}$ partner and the E channel are missing. With $\tilde{K}$: 71 of 194 nonzero F images and 8 of 33 G images lie in N, all of them also satisfying the ghost equation. The remaining failures are all in the $h \cdot h \cdot c$ row (for G, also in the ghost rows). A (λ, μ) scan, used only as a diagnostic, shows that no single normalisation works for every generator. Failures occur with and without trace monomials, so the η-trace hypothesis is not the (sole) cause. Conclusion: on the evanescent sector of the $h \cdot h \cdot c$ row, the row operators do not compose to zero with this σ. A candidate cause is the 4D-momentum rule $T(p, \cdot) = 0$ for hat components, under which $s_0 \hat{h} = 0$. This is OPEN.

Classification (conditional)

$\dim N_g = 104 = 46$ (σ span) $+ 24$ (blind) $- 9$ (overlap) $+ 43$ (unclassified). The 46 directions are trivial only if this σ is the correct linearised Slavnov–Taylor operator on the evanescent sector. No direction is certified physical. The invariant evanescent basis and the p22 κ^2 consistency check remain OPEN.

Disclosed revision

Q11-R1: the Q10 layer gate leaves code/p6q06/q07/q08/q09/q10/q11/ and tools/*q11* to the Q11 gate and treats the guard_scan3 file count as a lower bound; it is pinned by q11_expected.json. The OpenAI catalogue and the En05–En07 sheets are not premises.

Q10 layer

2 Q10: erratum on the trivial counts; anatomy of the family vectors

New manuscripts (intake, context only)

The companion source (cheungngo/enter-companion, head bb663a2, 2026-10-11) adds En05-A/B/C, En06-A/B and En07-A (partitioned Enter dynamics, operational causality, thermodynamic access) and OpenAI separation edges. The OpenAI catalogue is linked only by separations ("not a premise and not a declared path"). None of these has BRST, dimensional-regularisation, evanescent or counterterm content; nothing is imported as a premise. XV-B (modified in the same commit) stays unopened. Record: intake/new_manuscripts_q10.json.

Erratum Q10-E1 (medium)

Withdrawn: the Q08 counts "≥ 97 in the trivial span at the best fitted λ", "≥ 55 trivial" and "≥ 36 unexplained" as a count of non-trivial directions; the Q09 counts "F verified on 139/204" and "derived trivial span 59 (27 in the ghost-equation subspace)" as counts of BRST-trivial directions. Reason (Q10_family_anatomy): of the 204 F generators, 15 vanish after the h-leg symmetrisation; all 124 nonzero generators that pass have *each piece separately* in N, so the λ test is vacuous on them, and 62 of them are supported only on blind coordinates. 64 of the 65 failures admit no λ at all. For G, 28/33 admit no λ (5 need $i/2$); for P, 132/134 admit no λ (2 need -2). The Q08 hand-built pieces are not σ-images of the stated Y for most generators. The defect lies in the construction, not the normalisation, so no normalisation (derived $-1, -1, +1$ or fitted $-1, i/2, -2$) can repair it.

Blind directions and the restated bound

42 X coordinates (30 in the $K{\cdot}c{\cdot}h$ sector, 12 in the c^*cc sector) enter no three-point row and no Bose condition. 24 of these survive the ghost equation. They are free at this order and unconstrained by the rows. $\dim N - \dim(N \cap$ span of all Q08 pieces$) = 152 - 116 = 36$ is exact linear algebra, but the spanning pieces are not verified σ-images. So 36 bounds only the directions outside that span. It is neither a lower bound on physical directions nor an upper bound on trivial ones.

Open

The F trace-type failures (the η-trace hypothesis is neither confirmed nor excluded); a correct construction of $\sigma(Y)$ for $Y = K{\cdot}F$, $c^*{\cdot}G$, $\bar{c}{\cdot}\Phi$ (or a proof of absence); classification of N; the invariant evanescent operator basis and X in it; consistency with the p22 κ^2 evanescent counterterms.

Disclosed revision

Q10-R1: the Q09 layer gate leaves code/p6q06/q07/q08/q09/q10/ and tools/*q10* to the Q10 gate; it treats the guard_scan3 file count as a lower bound (Q10 files are scanned, not exempted); it is pinned by q10_expected.json. The Q05–Q08 gates are unchanged.

Q09 layer

3 Q09: review fixes; derived BRST-trivial directions of X

Q07 review fixes

(i) Status relabelled: *evanescent κ^3 three-point defect σ_0-exact at pole level; X non-unique (ghost-equation compatible).* This supersedes the Q07/Q08 wording "σ_0 closure CLOSED at pole level"; the frozen Q07/Q08 certificates are left unchanged. (ii) The Q04 independent rebuild report is bound by its sha256 in the Q09 gate. (iii) A new static scan module (guard_scan3) flags operator.methodcaller in any form and coefficient/exponent walks aimed at d (coeff, as_coeff_exponent, as_independent, collect, as_poly, degree, and dictionary walks of the power/coefficient dictionaries keyed by d). All 10 probes are flagged; code/, tools/ and tests/ are clean. The Q05 scan is unchanged; the runtime guard stays primary.

(a) Derived normalisations

For $Y = K \cdot F(h,h)$ the components of $\sigma Y = (S_0, Y)$ are produced by the same operators that build the rows (X_{hhh} by the $\delta S_0/\delta h$ contraction, X_{Khc} by $h \to s_0 h(c)$). These operators commute, so $\sigma^2 = 0$ fixes the relative sign $\lambda_F = -1$; likewise $\lambda_P = -1$ for $Y = \bar{c} \cdot \Phi$, and $\lambda_G = +1$ for $Y = c^* \cdot G$. Check: at $\lambda_F = -1$, 139 of 204 generators lie in the kernel. All 65 failures carry a K- or h-trace monomial, which points to the η-trace convention in the probe contraction (open). The Q08 G and P constructions do not close at the derived values (0/33, 0/134); they were assembled by hand and are open. The Q08 fitted counts are superseded.

(b)–(d)

(b) Derived trivial span: 59 directions, 27 of them inside the 104-dimensional ghost-equation subspace. The rest are not classified. (c) No invariant evanescent operator basis was constructed: open. (d) Consistency with the p22 κ^2 evanescent counterterms: open.

Disclosed revision

Q09-R1: the Q08 gate leaves code/p6q06/q07/q08/q09/ and tools/*q09* to the Q09 gate; it is pinned by the Q09 expected-values file. The Q05–Q07 gates and guard_scan2 are unchanged.

Q08 layer

4 Q08: structure of the κ^3 evanescent counterterm

Scope

Pole level, one loop, κ^3, three-point rows (c$\phi\phi$, cc$\bar{c}$, ccK, hhc) as in Q07. Unknowns: the local evanescent counterterm X of Q07 (821 coordinates: X_{Kch} 134, $X_{\bar{c}ch}$ 81, X_{c^*cc} 18, Bose-symmetric X_{hhh} 588).

1. Ghost equation

The loop poles obey $\Gamma_{\bar{c}ch} = \rho\,\Gamma_{Kch}[K \to T(\bar{c})]$, $T(\bar{c}) = \mathrm{sym}(k, \bar{c}) - \frac{1}{2}(k \cdot \bar{c})\eta$, with one constant $\rho = -2$ (graph by graph: G04/G03 and G09/G07). Imposing $X_{\bar{c}ch} = -2X_{Kch}[K \to T]$ keeps the system solvable; a particular solution is certified ($Ax = b$ verified). Free directions drop from 152 to 104. The η-trace is taken at the $d \to 4$ residue level (hat trace 4, tilde trace 0).

2–3. Free directions and trivial families

Kernel N of the rows: dim 152 (104 with the ghost equation). Single-sector blind directions: 69 in X_{Kch} alone, 12 in X_{c^*cc} alone, 26 in X_{hhh} alone (σ_0-closed evanescent cubic structures, candidates for cubic parts of invariant evanescent operators), none in $X_{\bar{c}ch}$ alone. BRST-trivial candidates $\sigma(Y)$, gh -1: $Y = K \cdot F(h, h)$ (field redefinition of h), $Y = c^* \cdot G(c, h)$, $Y = \bar{c} \cdot \Phi(h, h)$ (gauge fermion). The relative normalisation of the two pieces of each family was fitted (best uniform λ: -1, $i/2$, -2); with it at least 97 directions of N (55 inside the ghost-equation subspace) are trivial, at most 116. At least 36 directions are not shown trivial. Whether they are physical ambiguities (invariant evanescent operators) is not settled, and no explicit operator basis was constructed.

4. Consistency with p22

Open: the cubic completion of the p22 κ^2 evanescent operators was not built.

Outcome

P.6 Partial. Q07's pole-level σ_0 closure on the three-point rows now holds with a ghost-equation compatible X. Not implied: four-point rows, finite parts, uniqueness/physicality of X, p22 consistency.

Disclosed revision of a frozen layer

Q08-R1: the Q07 gate (code/p6q06/q07/release_q07.py) leaves files under code/p6q06/q07/q08/ and tools/*q08* to the Q08 gate; it is pinned by the Q08 expected-values file. The Q05 and Q06 gates are unchanged.

Q07 layer

5 Q07: κ^3 three-point rows — h h c, evanescent counterterms, pole-level σ_0 closure

Scope

One loop, κ^3, UV-pole level, declared XVI-P prescription (DRED, opaque d, MS-bar, Feynman $+i0$). Rows of $(S, \Gamma_1^{\text{pole}})$ with three external legs: $c\phi\phi$ (closed, Q05), $cc\bar{c}$ and ccK (4D part closed, Q06), and hhc (this release). Four-point and higher κ^3 rows and finite parts are not addressed.

Row h h c

Channels: (A) Γ_{hhh} (G01+G02+G08+G14+G15) with one leg replaced by $s_0 h(c)$; (B) own Γ_{hh} pole (graviton, ghost and scalar loops) with one leg $s_1 h = \frac{1}{2} L_c h$; (C) $\delta S_{EH,3}/\delta h \cdot \Gamma_{Kc}$; (D) $\delta S_2/\delta h$ (EH + gauge fixing) $\cdot \Gamma_{Khc}$; (E) $s\bar{c} = \sigma_F F(h)$: $F \cdot \Gamma_{\bar{c}ch}$ (the channel missing from the Q04 ledger, erratum Q06-E2). The tree row $(S,S)/2|_{hhc}$ closes exactly with $\sigma_F = i$ and requires channel E. The scalar loops G14, G15 and the scalar part of Γ_{hh} were recomputed with the Q05 engine; the matter sector alone closes exactly (incl. evanescent terms). Result: the 4D part of the row vanishes identically with all channels at $+1$ (the unconstrained fit has the unique solution all-equal), every single-channel deletion and a flip of σ_F re-open it; an explicit evanescent remainder (248 terms, every term containing a $\tilde{g}$ contraction) is left.

Evanescent counterterms

(i) The naive p22-type choice (subtract the $\tilde{g}$ part of every loop input) is *not* σ_0-consistent: tree vertices regenerate $\tilde{g}$ structures from 4D loop parts (22 / 26 terms left on ccK / $cc\bar{c}$). (ii) Joint test: $CT = -\Gamma^{\text{pole}} + X$ with X a generic local, purely evanescent κ^3 functional (monomials with at least one $\tilde{g}$) for Γ_{Kch}, $\Gamma_{\bar{c}ch}$, Γ_{c^*cc} and a Bose-symmetric Γ_{hhh}; the same X_{Kch}, $X_{\bar{c}ch}$ enter the ghost rows and hhc. The linear system over $\mathbb{Q}(i)$ (821 unknowns) is solvable; a particular solution ($X_{c^*cc} = 0$) is certified and verified by substitution; 152 parameters stay free (non-unique). Not tested: ghost-equation compatibility of X, four-point rows.

Outcome

σ_0 closure at κ^3: CLOSED at pole level on the one-loop three-point rows in the enlarged (evanescent) counterterm domain. P.6 remains Partial: CT-domain reduction to an invariant operator basis, four-point rows, finite parts, ghost-equation compatibility and uniqueness of X are open. O.1 (scoped, κ^2 two-point) unchanged; bulk symmetry restoration not claimed.

Disclosed revision of a frozen layer

Q07-R1: the Q06 gate (code/p6q06/release_q06.py) leaves files under code/p6q06/q07/ and tools/*q07* to the Q07 gate and treats the pinned Q05 guard-scan file count as a lower bound (as Q06-R1 did for the Q05 gate). The Q05 gate is byte-identical to Q06. The revised Q06 gate is pinned by the Q07 expected-values file.

Q06 layer

6 Q06: P.6 continued (honest partial)

Scope and standing constraints

One-loop κ^3 three-point sector at UV-pole level, XVI-P declared prescription (DRED, opaque d via qguard, MS-bar, Feynman $+i0$), own Feynman rules of Q05. XVI-O/N/P read-only; O.1 remains CLOSED (scoped, κ^2 two-point) and is not affected; bulk symmetry restoration is not claimed; insertion frozen; no Missing is ever set to zero.

1. Ghost rows $c\,c\,\bar{c}$ and $c\,c\,K$: diagnosis and fix

Q05 reported both rows OPEN: the assembled one-loop row did not vanish and no rescaling of the four channels closed it. Q06 splits every channel by orientation of the two identical ghosts and fits all six coefficients freely on the 4D (non-evanescent) part. The fit has a unique solution (up to scale), every coefficient ± 1: the h channel and both c channels enter as assembled in Q05, but the K channel $\delta S_1/\delta h|_{Xc_j} \cdot \Gamma_{Kc_i}$ enters with the *opposite* orientation sign. Reason: in the K channel the external c_i sits inside Γ_{Kc_i}, to the right of c_j, while in the h channel the order is reversed; bringing the monomial to the row ordering $c_1 c_2$ costs one Grassmann transposition. The tree row has no K channel, so the tree calibration ($\sigma_c = -1$ for $\bar{c}$, $+1$ for K) is unchanged. With this fix the 4D part of both rows vanishes exactly, and every single-channel mutation (drop $h_{1,2}$, $K_{1,2}$, c_1, c_2, flip σ_c, restore the Q05 K sign) leaves a nonzero 4D residual.

Second route for G10. The $c^* cc$ triangle was recomputed with a re-routed loop momentum (graviton carries $l - p_1$) and the two c slots of V_{c^*cc} entered in the opposite order (Grassmann sign -1): bit-identical to the Q05 pole. G10 was therefore not the defect.

What remains. A purely evanescent remainder (22 terms for ccK, 24 for $cc\bar{c}$; every term carries one T between external polarisations). Because the row is linear and its 4D part vanishes, local evanescent counterterms equal to minus the evanescent pole parts (p22 prescription) remove it; whether those counterterms are s_0-consistent on the full κ^3 evanescent domain is OPEN; it is recorded explicitly (sha256 in the certificate) and is not called an obstruction.

2. hhh graphs G01 and G02

A sparse-ring fast path (same algorithm as the Expr path, partial products truncated in the Taylor parameter at the pole order) reproduces G05 and G06 bit-identically and makes G01 (V_3^3 triangle) and G02 feasible. G02 needs the quartic vertex: V_4 was derived from S_{EH} by expanding $\sqrt{-g}\, g^{mn} g^{ac} g^{bd}$ to second order. No non-scaleless XVI-P graph contains V_4 (its tadpole is scaleless), so V_4 is validated by the exact Einstein–Hilbert gauge identity $\delta_0 S_4 + \delta_1 S_3 = 0$ ($\delta_0 h = \partial\xi + \partial\xi$, $\delta_1 h = L_\xi h$), which vanishes identically; the same identity at $n = 2$ (V_3 vs V_2) is the machinery control. G02 (three channels) and G01 are Bose-symmetric under $(a_1, p_1) \leftrightarrow (a_2, p_2)$, and G02's channels map onto each other. All 15 P.6 graphs now have pole values (G14/G15 from Q04).

3. Row $h\,h\,c$, κ^3 defect, counterterms, σ_0 closure

Not assembled: Missing. The channel ledger is recorded, including a channel absent from the Q04 ledger: with b eliminated, $s\bar{c} \propto F(h)$, so $F(h) \cdot \Gamma_{\bar{c}ch}$ (G04+G09) enters (erratum Q06-E2). The κ^3 counterterm-domain reduction, defect, counterterm solve and σ_0 closure are not established.

Errata

Q06-E1 (low): Q05's ghost rows were OPEN because of the K-channel Grassmann sign in the assembly, not because of the graphs; corrected here, Q05 record kept. Q06-E2 (low): Q04 h h c ledger lacks the $F(h) \cdot \Gamma_{\bar{c}ch}$ channel.

Disclosed revision of a frozen layer

Q06-R1: the Q05 gate (code/p6q05/release_q05.py) now treats the pinned Q05 guard-scan file count as a lower bound (the recursive scan grows when Q06 adds code) and leaves Q06-owned files to the Q06 gate; Q05_guard_scan.json was regenerated (26 to 33 files, still clean). The revised Q05 gate is re-pinned by the Q06 expected-values file.

Status

P.6 Partial. execution_pass true; claim_status partial; scope: pole level, one loop, κ^3. Next: evanescent s_0-exactness of the ghost-row remainder; h h c row; κ^3 counterterm domain, defect, counterterm solve and σ_0 closure.

Q05 layer

7 Q05: P.6 continued (one-loop κ^3 three-point sector) — honest partial

Scope. Pole level, one loop, κ^3, under the XVI-P declared prescription (DRED with opaque d guarded by `qguard`, MS-bar, Feynman $+i0$, evanescent structures kept as tilde metrics). O.1 stays CLOSED (scoped, κ^2 two-point) and is not changed; `bulk_symmetry_restored` stays false.

Own gravity and ghost Feynman rules

Derived in `code/p6q05/gravity.py` from the XVI-P declared action: $S_{EH} = -(2/\kappa^2)\int L_{\Gamma\Gamma}$, the gauge fixing obtained by eliminating b ($+D_\mu D^\mu$), $S_{gh} = \int i\bar{c}_\mu D^\mu[(\partial c + \partial c) + \kappa L_c h]$, $K\,sh$ with $sh = \frac{1}{2}L_c(\eta + \kappa h)$, and $sc = \frac{\kappa}{2}c\cdot\partial c$, $s\phi = \frac{\kappa}{2}c\cdot\partial\phi$ (XVI-P P13 s-images). Checks: the EH quadratic form is gauge invariant. EH plus gauge fixing equals $k^2(I - \frac{1}{2}J)$ (XVI-P $K = -\Box P$), so the graviton propagator is $i/k^2\,(I - \frac{1}{2}J)$ with $\mathrm{tr}\,\eta = 4$ (Q4S). Validation against pinned XVI-P two-point poles, all structures including evanescent ones, constant ratio (own/XVI-P): D01 (V3×V3) $= I$, D03 (ghost loop) $= I$, D06 $(\bar{c}c) = I$, D13 $(K'\cdot c) = -I$. I is the Q04 convention factor (D11). **Disclosure:** D13 has the opposite overall sign: the orientation of the $K'\cdot c$ slot differs between the two codes. Own rows use own D13 consistently. The ghost equation holds at two points ($D06 = -2\,D13[K \to T]$) and graph by graph at three points ($G04 = -2\,G03[K \to T]$, $G09 = -2\,G07[K \to T]$).

Graphs (UV pole residues, own rules)

Evaluated: G03–G13; nonzero poles are certified in `Q05_P6_graphs.json`. Q04 carried G14, G15 and the scaleless T1–T3. **Missing** (never set to zero): G01 ($V3^3$, implemented, evaluation did not finish within the release budget) and G02 (needs V4, not constructed).

Identity rows at the pole

Antifield-channel signs are calibrated on the classical $(S_0, S_1) = 0$ (tree level) and used unchanged at one loop. **Row $c\phi\phi$: closed.** The residual is identically 0. It needs the channel $\delta S_1/\delta h|_{\phi\phi}\cdot\Gamma_{Kc}$, which the Q04 ledger omitted (erratum Q05-E1). Detected mutations: flipped σ, dropped K-channel, dropped G06/G12/G13. G05's s_0-projection vanishes identically, so dropping G05 is undetectable in this row; that is recorded, not a failure. **Rows $cc\bar{c}$, ccK: OPEN.** The tree identities close with one constant σ_c. The one-loop rows do not close, and no rescaling of the four channels closes them. They are *not* reported as an obstruction. Since the ghost equation holds, the defect is located in G10 (one sign bug already fixed, erratum Q05-E4), in the K/c channels, or in the Grassmann bookkeeping of the two identical c legs. **Row hhc: Missing** (needs G01, G02).

Counterterms and σ_0 closure

In the $h\phi\phi$ and $\phi^* c\phi$ sector, the MS-bar counterterm is minus the certified poles. Its consistency in row $c\phi\phi$ is exactly the closed row. Every other κ^3 sector is Missing or Open. The κ^3 counterterm-domain reduction is not done. **σ_0 closure at κ^3 is not established.** Finite parts are Missing.

Q04 review folded in

The static scan is hardened (`guard_scan2`: recursive over `code/`, flags all 7 reviewer evasion forms). The claim is downgraded to a *best-effort policy scan, not a security boundary.* The Q05 gate rejects any unpinned file in `code/**`, `tools/` or `tests/`. The `scalar_loop3.py` docstring (G13/G14 should read G14/G15) is erratum Q05-E2; the file is pinned by the Q04 layer gate and left unchanged. Pinned Q04 tools are left unchanged, and Q05 adds `build_q05.py`, `source_set_q05.py`, `rebuild_q05.py` and `make_expected_q05.py`.

Next step

Independently check G10 and the c-channel, then close or diagnose the ghost rows. Evaluate G01 (lower-cost contraction) and construct V4 for G02, which gives row hhc. Then reduce the κ^3 counterterm domain and attempt σ_0 closure.

Q04 layer

8 Q04: P.6 entry — one-loop κ^3 three-point sector (honest partial)

P.6 is the one-loop κ^3 three-point sector. It lies outside the O.1 scope (one-loop κ^2 two-point), and **O.1 CLOSED (scoped) is unchanged**. Q04 releases the stages listed below. Anything not evaluated is marked *Missing*, never set to zero.

8.1 Inventory and vertices (Constructed)

An independent enumerator (one-loop 1PI graphs as cycles of at most 3 vertices; each vertex of an $E = 3$ graph carries at least one external leg) is run on the p14r loop-capable vertices plus the κ^3 vertices. It reproduces XVI-P P23's 15 loop graphs exactly (hhh 5, h$\phi\phi$ 4, $\bar{c}ch$ 2, Kch 2, ccc^* 1, $\phi^* c\phi$ 1). It adds three κ^3 tadpoles: V5_grav $\to$ hhh, and V_hhh$\phi\phi$ with an hh line $\to$ h$\phi\phi$ or a $\phi\phi$ line $\to$ hhh. All three are scaleless (massless self-contraction; $\int d^d l\, l^{2k}/l^2 = 0$, no log member), so they are derived 0.

Power counting: at one loop $I = V$, so $\sum_v(\text{legs}_v - 2) = E$ and the κ power equals E (checked on every enumerated graph). Hence the one-loop κ^3 one-point and two-point sectors are empty.

κ^3 vertices: the ghost, antifield and gauge-fixing vertices are exact at their order. $sh = \frac{1}{2}L_c(\eta + \kappa h)$ is affine in h, Ψ is linear in h, and sc, $s\phi$ are exact at κ. The only new vertices are therefore V5_grav and V_hhh$\phi\phi$. The cubic term of $\sqrt{-g}g^{\mu\nu}$ is derived in closed form and agrees with an independent exact determinant/inverse series; a mutation is detected. V5_grav is not constructed, because it enters only the scaleless tadpole.

8.2 Identity at κ^3 (ledger Constructed)

$(S, \Gamma_1) = A_1$ at κ^3 is $(S_0, \Gamma_1^{[3]}) + (S_1, \Gamma_1^{[2]}) + (S_2, \Gamma_1^{[1]}) + (S_3, \Gamma_1^{[0]})$. The last two terms vanish ($\Gamma_1^{[1]} = 0$ by D14; $\Gamma_1^{[0]}$ is field independent). The ghost-number-1 three-field rows are enumerated mechanically: 14 rows. Four are odd under ϕ-parity (zero), six contain auxiliary or nonminimal fields, and four need P.6 inputs (chh, $c\phi\phi$, $cc\bar{c}$, ccK); each needed input is recorded. The κ^3 counterterm domain is given only as structural monomial counts (4D versus evanescent); its BRST/σ_0 reduction is Missing.

8.3 Evaluation (Partial)

Own UV-pole engine (`code/q04/uvpole.py`). It uses its own Feynman rules, symbolic opaque d through qguard, and the DRED hat/tilde split (QdS momenta, Q4S vertex indices); MS-bar residues are taken only through `dred.substitute_d`. The UV pole is extracted by an exact Taylor (IR-rearranged) expansion. *Validation:* the engine's scalar bubble equals $i\times$ XVI-P's P18 D11 pole on all 14 structures, including the 8 evanescent ones. A vertex mutation breaks the agreement.

Evaluated: P6-G14 (scalar triangle) and P6-G15 (scalar bubble), the scalar-loop contributions to hhh, at the UV-pole level (exact residue; Bose symmetric). **Matter Ward identity at three points**: diffeomorphism invariance of the scalar-loop $W[g]$, i.e. the matter part of row chh, with Γ_2 the validated D11. The pole residue satisfies it exactly, including the evanescent structures. A wrong Lie-derivative sign and dropping the $\sqrt{-g}g^{-1}$ quadratic-vertex bubbles are both detected. The d-dependent coefficient differs from zero by $O(d-4)\times U$, which is finite and not evaluated.

Missing: 13 graphs, namely every graph with a graviton or ghost loop, V3/V4_grav, the ghost/antifield vertices or mixed lines (P6-G01..G13). Also Missing: their finite parts, the κ^3 defect assembly, the counterterm solve and σ_0 closure.

8.4 Next step

Implement V3_grav (own rules, validated against XVI-P D01) and the ghost vertex (validated against D06/D13). Evaluate G01–G13 at the pole level, starting with the rows $c\phi\phi$ (G05, G06, G11, G12, G13) and $cc\bar{c}/ccK$ (G03, G04, G07, G09, G10). Assemble the projected κ^3 defect, reduce the counterterm domain, solve, and check σ_0 closure.

8.5 Q03 review findings (low), folded in

(i) `tools/source_set.py`, which decides which files count as source, is now pinned in the gate together with `build.py` and `rebuild_from_source.py`; the gate also re-hashes the Q03 independent report. (ii) The d guard cannot see code that leaves sympy symbolics. This covers a pure-Python term walk that identifies d by name (comparing with `== d` is already refused by the p22 guard), a `str`/`srepr` rename followed by a re-parse, and raw dense-list evaluation. The probes in `certificates/Q04_guard_scan.json` show these *do* bypass the runtime guard, and they cannot be closed at runtime. The class is closed by policy: an AST static scan of every package file (`code/q0*/*.py`; the guard probe files are exempt) forbids the escape primitives, and the gate requires the scan to be clean.

Q03 layer

9 Q03: correction release (no new science)

Q03 corrects the O.1 closure record after the Q02 independent review (report `reproducibility/independent_rebuild_report_q`
sha256 `e4b54702bc76...`, reviewed source zip `378651a3...`, tree `4b551b98...`; decision: CONFIRM CLOSED (scoped)). The Q02 certificates are a frozen layer; the authoritative record is `certificates/Q03_O1_record.json`.

9.1 O.1 status

`O1_closure` = CLOSED (scoped), `O1_closure_confirmed` = true, bound to the report sha256 and the reviewed tree (the gate re-hashes the report). Scope (verbatim from the review): one-loop, order κ^2, bulk two-point retained sector of $S_{\rm inv}$ ($h, \phi, c, \bar{c}, b$ + antifields; gh0 seeds), $m = 0$, frozen gauge fermion, under XVI-P's declared prescription (DRED: QdS loop momenta, Q4S vertex indices, MS-bar) including the p22 evanescent counterterm contract revision.

9.2 Correction 1: counterterm domain of O.1 item 5 (medium)

The counterterm domain used is the *enlarged* domain: the 11 p14r 4D directions over $\mathbb{Q}(i)$ *plus* the 10 p22 evanescent operators ($K' \cdot c$ E2, coefficient $-1/16$; $\bar{c}c$ SE, $1/8$; eight $h \cdot h$ evanescent structures), 21 directions in all. With QdS (d-dimensional) external momenta the local divergent part contains evanescent poles that only the p22 operators cancel. "ST4a in the declared p14r domain" on its own holds only for 4D externals. σ_0 closure including the evanescent operators was verified in the p20/p22 independent reviews.

9.3 Correction 2: prescription; XVI-O closure reason 8 (medium)

The closure uses XVI-P's prescription, not XVI-O's. XVI-O's o70-sync D11 matter-loop kernel counterterm (closure reason 8) is superseded on evanescent legs: both XVI-O kernel continuations fail there (p22). Reason 8 is therefore recorded as NOT ANSWERED in XVI-O's terms; reasons 1–7 are answered as in Q02, and reason 7 now names the enlarged domain.

9.4 What O.1 CLOSED (scoped) does not imply

Not the functional/all-n-point ST4b; not bulk symmetry restoration (`bulk_symmetry_restored` = false); no change to XVI-O's own ledger (read-only, O.1 there stays NOT_CLOSED); not the three-point sector / P.6 (κ^3) or higher orders; not the insertion (frozen); not finite amplitudes with ϵ-scalar externals.

9.5 Caveats

(i) Both independent reviews ran on the same box as the author (shared Python/sympy and TeX Live). (ii) The D01/D03 pole values were never recomputed from independent Feynman rules; they are cross-checked only through the exact cancellation of the $c \cdot h$ defect in d. (iii) Full σ_0 closure and the $c \cdot h$ defect rely on the earlier p19–p22 independent checks. (iv) D06 is accepted through its exact equality with XVI-P P15 Σ.

9.6 Correction 3: gate and guard (low)

The gate now re-hashes every pinned provenance file (all 33, and it rejects any unpinned provenance file), every pinned code/conventions file and the bound independent report at gate time. It also checks that every file it reads is in the source set or is a build output. The d guard now refuses value assignment (`evaluate`/`subs`/`__call__`/`compose`) on `PolyRing`/`FracField` elements whose generators include d when called directly. All 8 reviewer forms raise, and the 18 earlier forms still raise. Disclosed residual: raw dense-list evaluation carries no symbol information and is not guarded; no package code uses it.

Q02 layer

10 Q02 scope and outcome

Q02 evaluates the bulk O.5 graphs D04, D05a and D06, reconciles D12, states XVI-O's ST4a/ST4b on the full one-loop κ^2 bulk two-point sector of $S_{\rm inv}$ ($m = 0$), solves the bulk counterterms, checks full σ_0 closure, and walks the O.1 checklist. XVI-O, XVI-N and XVI-P are read-only and hash-pinned. The prescription is XVI-P's declared one: DRED with QdS loop momenta and Q4S vertex indices; d opaque behind the XVI-Q guard; MS-bar; Feynman $+i0$; the p22 evanescent counterterms. Normalisation: $i\Gamma_1 = \sum$ graphs. Coefficients are given per κ^2, in units of $M_B = \hat{I}(1,1) = 1/\epsilon + 2 - L + O(\epsilon)$, with $L = \log(-p^2/\mu^2 - i0)$.

11 Bulk graphs

graph	content	value	evidence
D04	$\phi\phi$, two $V_{h\phi\phi}$, h+ϕ lines	0 exactly in d	two routes + hand contraction
D05a	$\phi\phi$, $hh\phi\phi$, h loop	0 (derived)	numerator l-independent, $\omega = 2$, no log r
D06	$\bar{c}c$, two $V_{\bar{c}ch}$, h+c lines	$\delta^\rho_\nu p^4$: $-\frac{4d-7}{8(d-1)}$; $p^\rho p_\nu p^2$: $\frac{3(d-2)}{8(d-1)}$; $\tilde{G}^\rho{}_\nu p^4$: $-\frac{3}{8(d-1)}$	= XVI-P P15 Σ exactly
D12	hh, $hh\phi\phi$, ϕ loop	0 (derived)	$\omega \in \{2, 4\}$, no log member

D04: mechanism. The D04 bubble at $p \neq 0$ is *not* scaleless, so its zero needs an explicit reason.

Take the vertex $V_{ab}(p,q) = p_a q_b + p_b q_a - \eta_{ab}\, p\cdot q$ with $q = l - p$, and the graviton numerator $P = \frac{1}{2}(\eta\eta + \eta\eta) - c\,\eta\eta$, where the vertex-index trace is $\eta \cdot \eta = n$. Then

$$V \cdot P \cdot V' = 2p^2q^2 + (n-2)(p\cdot q)^2\,[1 - c(n-2)].$$

The $(p\cdot q)^2$ term is the coupling to the trace mode. It cancels exactly when $c = 1/(n-2)$, which holds for the de Donder propagator in DRED ($n = 4$, $c = \frac{1}{2}$, the declared prescription) and in CDR ($n = d$, $c = 1/(d-2)$). What remains is $2p^2(l-p)^2$. This cancels the scalar propagator and leaves $p^2 \int 1/l^2$, which is scaleless with $\omega = 2$ and no logarithmic member, so D04 = 0 exactly in d.

D04 = 0 is therefore a statement about de Donder gauge, minimal coupling and $m = 0$, off shell. It is not a general property of the bubble, and no free-EOM equivalence is used.

D04: routes.

- Engine route 1: Feynman parameters, with no scaleless rule.
- Engine route 2: an invariant fit identical to $\frac{1}{2}p^2(l-p)^2$, followed by IBP.
- Route 4: fully independent of the engine. It uses its own covariant algebra and its own Feynman-parameter integration with symbolic D for arbitrary propagator powers.

In route 4, both the DRED and CDR cases give 0. The mutations, as ratios to the master, are:

- mixed scheme ($n = 4$, $c = 1/(D-2)$): $\frac{D-4}{2(D-2)}p^4$, i.e. $O(\epsilon)\times$pole;
- vertex trace normalisation $\frac{1}{2}$: $p^4/4$;
- non-de Donder gauge pieces of Fröb's propagator: $T^{(4)}$ gives $(D-1)p^4$ and $T^{(5)}$ gives $\frac{(D-2)(D-1)}{8}p^4$;
- non-minimal coupling ξ: $-\xi p^4$, which is linear in ξ;
- engine mutation dropping the trace term of the propagator: $-i/8$.

The gauge piece $T^{(3)}$ gives 0, and this is expected rather than a failure: $l^a V_{ab} = p_b q^2 + q_b p^2$, so every $T^{(3)}$ term carries q^2 and is scaleless. It is recorded as a Ward-identity zero, not counted as a detector.

D04: literature. M. B. Fröb, CQG 35 (2018) 055006 (arXiv:1710.00839), works in flat space, in a general gauge, in n dimensions. The de Donder part of the graviton-loop scalar self-energy (his eq. inv2pf_termsumint_grav) has an integrand bracket proportional to m^2 or ξ, and "vanishes identically" for $\xi = 0$, $m = 0$, in accordance with Kahya and Woodard (2007). For $m = 0$ his ξ^2 terms vanish, and the gauge-parameter pieces contribute. **All three statements agree** with our results. The 1PI self-mass is gauge dependent; D04 = 0 holds in the declared gauge only.

E7. The symmetry factor of D04 is 1. XVI-O's "$\frac{1}{2}$" was marked Unverified and never used for a value. XVI-P never evaluated D04, and its D11, which has two identical ϕ lines, keeps $\frac{1}{2}$. No earlier value changes.

D05a, D12. These are genuine one-propagator tadpoles: no external momentum flows through the loop line. They are pure power divergences ($\omega = 2$, and $2, 4$ for D12) with no logarithmic member, so there is no pole on either side, and the zero is not a cancellation between UV and IR poles. Fröb also notes that graviton tadpoles vanish in dimensional regularisation around flat space.

D06. This is the full $\bar{c}c$ two-point function. MS-bar poles: $-3/8$, $1/4$, $-1/8$. Local finite parts: $-2/3$, $5/12$, $-1/3$. Coefficients of L: $3/8$, $-1/4$, $1/8$. It is identical in d to the ghost self-energy Σ of XVI-P P15, for which the ghost equation against D13 is verified.

D12 reconciliation. XVI-O never labels D12 "Missing". The word survives only as stale p14r text in XVI-P's P23 O.5 line. XVI-P p15 derived D12 = 0, and Q02 confirms this independently. The value is derived, never set by convention; this is erratum E6.

12 Bulk retained identity, counterterms, closure

XVI-O's definitions, quoted verbatim:

- ST4a: $\Gamma_{1,\mathrm{local}} = \sum z_i C_i$ (gh 0);
- ST4b: $P_{\mathrm{ret}}(S, C) = -P_{\mathrm{ret}} A_1$ (gh 1);
- retained identity: $(S, \Gamma_1) = \mathcal{A}_1$ at one loop and order κ^2 in the retained sector.

Rows. All 23 one- and two-field monomials of ghost number 0 and 1 are classified, each with its own justification. Two independent one-loop enumerations agree with XVI-P: $E = 2$ gives 9 graphs and $E = 1$ gives 3.

Counterterms (ST4a). The MS-bar counterterm, $-$pole, in the 11-direction declared p14r domain over $\mathbb{Q}(i)$:

- $h{\cdot}h$: $\frac{49i}{480}, \frac{41i}{160}, -\frac{49i}{240}, -\frac{41i}{80}, \frac{43i}{120}$;
- $c{\cdot}\bar{c}$: $-\frac{3}{8}, \frac{1}{4}$, in the route-G orientation; these equal the D06 poles up to the orientation sign -1;
- $c{\cdot}K'$: $0, -\frac{3}{8}, \frac{1}{4}$;
- $\phi{\cdot}\phi$: 0. P19 carried this coordinate as a placeholder; it is now earned.

The p22 evanescent counterterms ($K'c$ E2: $-\frac{1}{16}$; $\bar{c}c$ SE: $\frac{1}{8}$; hh sum) subtract the evanescent poles only. They lie outside the p14r 4D domain as a disclosed contract revision. They are not needed for the identity: the ch defect is zero with $x = 0$.

ST4b and closure.

- $c \cdot h$: the defect is zero exactly in d (P19).
- $c \cdot b$: this is the ghost equation, and its Σ equals D06.
- $c \cdot \bar{c}^*$: unreachable.
- $c \cdot \phi$: zero by parity.

Full σ_0 closure of the complete MS-bar counterterm holds in every gh+1 sector. The σ_0 rank on $\phi{\cdot}\phi$ is 0, and the cohomology of the domain is 0. **Outcome: cancellable, with no residual.**

13 O.1 walk (verbatim ledger)

1. *State retained quantum identity*: earned. XVI-O status was Constructed_as_statement.
2. *List amplitudes constrained by identity*: earned. XVI-O status was "manually_specified_partially_checked". Completeness now rests on two mechanical enumerations.
3. $S^{[2]}_{\mathrm{inv}}$ *classical consistency*: earned in Q01.
4. *Evaluate inventory graphs*: earned. All 12 graphs have exact values.

5. *Remainder cancellation / CT solution*: earned, as above.

6. *Enlarge target*: not triggered. At one loop the κ^2 part of (S, Γ_1) is $(S_0, \Gamma_1[\kappa^2]) + (S_1, \Gamma_1[\kappa])$. $\Gamma_1[\kappa]$ is one-point only and is 0, and Γ_1 with three fields starts at κ^3.

All eight closure-check reasons are answered. Enlarge rule 3 is respected: no XVI-M graph is used or identified with an action graph. Downgrade rule 2 is respected: no amplitude is omitted without its own justification.

O1_closure = CLOSED_PENDING_INDEPENDENT_REVIEW (confirmed: false). The scope is the one-loop κ^2 bulk two-point retained sector of S_{inv} at $m = 0$, with the frozen gauge fermion, under the declared prescription, with external momenta in QdS. The functional ST4b (all n-point) and bulk symmetry restoration are *not* claimed.

14 Guard, gate and records

- The guard refuses any `Poly` that has d as a generator. d inside the coefficient domain is admitted, as used by the XVI-P integral library. `apart` and `apart_list` are removed from the allow-list. 18 bypass forms $\times$ 4 expressions are probed, and all of them raise.

- The gate compares every Q01/Q02 certificate value path by path against `q02_expected.json`, whose sha256 is pinned in the gate. It independently checks the O.2(c) cells (exact set, all true, $\kappa^0..\kappa^2$, including the build input), each per-file pin, the conventions files and the errata file, and fails closed.

- `tools/build.py` no longer syncs by default; a release opts in with `--sync`.

- The registration record, errata E2 (severity low) and E4 (unpinned `p23_conventions`) are corrected, and the XVI-N label note (r10 file, r12 content) is added.

Q01 layer

15 XVI-Q Q01: scope and registration

XVI-Q is a new landmark (registered locally: `registration/local_registration.json`; not published). Its aim is to import XVI-P's verified ℓ_4 to discharge O.2, to state the bulk identity, and to attempt O.1 closure. XVI-O, XVI-N and XVI-P are read-only. They are imported by sha256 (`intake/imports.json`; XVI-P frozen at p23, source zip `3f4a88db...`). Q01 covers the intake and O.2. O.1 stays NOT_CLOSED, bulk symmetry is not claimed, and the insertion stays frozen.

O.1 target (XVI-O ledger, verbatim). "one-loop_order_kappa2_bulk_two_point_on_S_inv": bulk two-point (graviton, scalar, ghost), including the Einstein–scalar hh channels and the vertex-derived gh= 0 BV/source tree seeds; baseline $S_{\rm inv}$; frozen gauge fermion; $m=0$. Closure requires a *verified* retained quantum identity with O.2, O.4, O.5, O.6 and O.7 earned.

Errata from the p23 review (intake).

- E1 (medium): the p23 O.5 line "all $E=2$ graphs evaluated" overclaims. D04 and D05a were never evaluated, D06 only as the ghost self-energy, and D12 is Missing. Bulk O.5 is open.
- E2: the stated reason "grav ℓ_4 Incomplete" is stale.
- E3: guard bypasses (fixed here).
- E4: gate value gaps (fixed here: every number is value-checked).
- E5: scaleless 5-leg tadpoles are missing from the κ^3 inventory.

16 O.2: the classical κ^2 master $2(S_0,S_2)+(S_1,S_1)=0$ on $S_{\rm inv}$

(a) Full ℓ_4 — Constructed. XVI-P's p23 stages, re-executed from the pinned zip, reproduce the $\ell_{2,3,4}$ exports byte-identically. The ℓ_4 export (3360 terms) equals, term by term, an *independent* reconstruction from the $\Gamma\Gamma$ Lagrangian $\sqrt{-g}\,g^{mn}(\Gamma^a_{mb}\Gamma^b_{na}-\Gamma^a_{mn}\Gamma^b_{ab})$. That reconstruction uses exact ϵ-grading, g^{-1} and $\sqrt{-g}$ to second order in H, and a sparse ring over $\mathbb{Q}$. Its only normalisation, -2, is fixed by XVI-O's conformal probe and coincides with XVI-O's convention $\ell_4=-2[\epsilon^4](\sqrt{-q}R)$, since $L_{\Gamma\Gamma}=\sqrt{-g}R+\partial B$. A one-term mutation is detected.

(b) Explicit $O(\kappa^2)$ antibracket expansion — Constructed. With $S_{\rm EH}=\sum_n \kappa^{n-2}\int \ell_n$, $s=s_0+\kappa s_1$ ($s_2=0$, derived), and $S_{af}\ni\int h^{*mn}(sh)_{mn}$, the field projection is

$$(S,S)|_{af0,\rm grav}=2\int\sum_{m\le n}\frac{\delta S_{\rm EH}}{\delta h_{mn}}(sh)_{mn}.$$

Its κ^2 coefficient is $2\int W_2$ with $W_2=\sum E_{mn}(\ell_4)(s_0h)_{mn}+E_{mn}(\ell_3)(s_1h)_{mn}$. This is the component form of $2(S_0,S_2)+(S_1,S_1)$. W_2 is linear in c, so $\int W_2=0$ for all c iff $E_{c^\mu}(W_2)=0$. On an independent XVI-Q jet engine, W_2 has 15428 terms and $E_{c^\mu}(W_2)=0$ exactly for $\mu=0..3$; the κ^0 and κ^1 regressions also vanish. Doubling one ℓ_4 term, or dropping s_1, breaks the result.

(c) Ghost/antifield sector — Constructed (re-executed evidence). In XVI-P's exact P13 master (re-executed; content-identical to p23), every κ^2 cell vanishes: af0 ghost/gauge fixing (transformed), and af1 and af2 in both invariant and Ψ-transformed variables. s_2 is *derived* zero in all sectors, from the affine g, the linear Lie derivative, and the ghost law; it is not invented. This answers XVI-O's "absent under inherited s". s is nilpotent exactly. Seven mutations each produce residuals. Caveat: Q01 has no second, independent engine for this sector.

Outcome. O.2 at κ^2 is **Constructed in XVI-Q**; XVI-O's own record is unchanged (read-only). The scope is classical, $S_{\rm inv}$, $m=0$: not quantum, not the prepared S_D, not the insertion.

17 d guard (carried)

The XVI-P p23 dred is imported unchanged, with one added rule. Polynomial construction (`Poly.new`, a class attribute) and any internal rename of d are admitted only when the *entry* sympy function called from user code is an allow-listed rational normaliser; otherwise they are denied. This closes the p23 bypasses: `sympy.polys.rem`, pre-import capture, `reduced`, `groebner`, the Dummy rename and `rep.eval`. Probe: 16 forms $\times$ 4 expressions all raise, and ordinary algebra is unchanged.

18 Status and Q02 plan

After Q01: O.2 Constructed; O.4 inventory (with E1); O.5 bulk open; O.6/O.7 bulk open; enlarge rule not triggered at κ^2. Hence O.1 is NOT_CLOSED. Q02 will:

1. evaluate the bulk O.5 graphs D04, D05a and D06 (full $c\bar{c}$ two-point), and settle D12 (Missing, never zero);
2. state and verify the bulk retained identity ST4a/ST4b on the full two-point sector;
3. solve for the bulk counterterms and attempt O.1.

19 Appendix: certificates, status, intake

RELEASE_STATUS.json


```
{
 "landmark": "XVI-Q",
 "release": "Q11",
 "status": "released (honest partial: sigma-images in Ktilde variables, conditional classification)",
 "P6_outcome": "Partial",
 "sigma0_status_kappa3": "evanescent kappa^3 three-point defect sigma0-exact at pole level; X non-unique (ghost-eq compatible)",
 "O1_closure": "CLOSED (scoped)",
 "O1_closure_confirmed": true,
 "hard_stops": {
  "bulk_symmetry_restored": false,
  "insertion": "frozen",
  "XV_B_opened": false,
  "XIV_J_opened": false,
  "Route_B_discharged": false,
  "Missing_never_zero": true
 },
 "independent_rebuild_q05": "not requested",
 "independent_rebuild_q06": "not requested",
 "independent_rebuild_q07": "PASSED (reviewer; report sha 03f5393f)",
 "independent_rebuild_q08": "PASSED on execution (reviewer; report sha cc67001e)",
 "independent_rebuild_q09": "not requested",
 "independent_rebuild_q10": "not requested",
 "independent_rebuild_q11": "not requested",
 "goals": {
  "1_sigma_images": "Partial: genuine Ktilde construction built (F, G; P absorbed as the cbar partner of F); 71/194 F and 8/33 G in N; Q08 F defect
diagnosed (missing cbar partner / E channel); remaining failures OPEN (not trace-driven)",
  "2_classification": "Partial (conditional): N_g 104 = sigma span 46 + blind 24 - overlap 9 + 43 unclassified",
  "3_invariant_basis": "OPEN: not constructed",
  "3_p22_consistency": "OPEN: not reached"
 },
 "execution_pass": true,
 "claim_status": "partial",
 "gate": "code/p6q06/q07/q08/q09/q10/q11/release_q11.py"
}
```


certificates/Q11_sigma_images.json


```
{
 "construction": "Y built from Ktilde = K + cbar.F'(h) (canonical shift by Psi = cbar.F, B integrated out); the cbar partner of every K-functional is
rho*T(.), rho = -2 (Q08 ghost equation). sigma = (S0, .) applied with the row channel operators (D, A, E), antibracket sign -1 between antifield-type
and field-type pieces; G family includes the c*cc piece (h -> s0 h, antisymmetrised in the two c's).",
 "Q08_F_defect_diagnosis": {
  "blocks_violated_by_Q08_pieces": {
   "F": {
    "ccK: ccb: hhc: bose:": 124,
    "ccK: ccb: hhc:PQ bose:": 53,
    "ccK: ccb: hhc:P bose:": 12
   },
   "G": {
    "ccK:Q ccb:P hhc:P bose:": 13,
    "ccK:Q ccb:P hhc:PQ bose:": 6,
    "ccK: ccb: hhc:PQ bose:": 8,
    "ccK: ccb: hhc:P bose:": 6
   },
   "P": {
    "ccK: ccb: hhc:PQ bose:": 122,
    "ccK: ccb: hhc:P bose:": 12
   }
  },
  "reading": "the Q08 F pieces violate only the h.h.c row; 12 generators fail through the antifield-type piece alone, i.e. Q08 F omitted the cbar
partner (Ktilde) and the E channel"
 },
 "results_at_derived_signs": {
  "F": {
   "n": 204,
   "leak": 0,
   "zero_vector": 10,
   "derived_minus1": {
    "in_N": 71,
    "in_N_and_ghost_eq": 71,
    "fail_blocks": {
     "hhc": 123
    }
   },
   "plus1": {
    "in_N": 49,
    "in_N_and_ghost_eq": 49,
    "fail_blocks": {
```

```
    "hhc": 145
   }
  }
 },
 "G": {
  "n": 33,
  "leak": 0,
  "zero_vector": 0,
  "derived_minus1": {
   "in_N": 8,
   "in_N_and_ghost_eq": 8,
   "fail_blocks": {
    "ccK ccb hhc": 13,
    "hhc": 12
   }
  },
  "plus1": {
   "in_N": 5,
   "in_N_and_ghost_eq": 5,
   "fail_blocks": {
    "ccK ccb hhc": 16,
    "ccK ccb": 3,
    "hhc": 9
   }
  }
 }
},
"lam_mu_scan_diagnostic": {
 "any": 41,
 "[('free', '0')]": 92,
 "[('-1', '1')]": 21,
 "[('free', '1')]": 18,
 "[('0', '0'), ('free', '0')]": 1,
 "none": 24,
 "[('-2', '0')]": 1,
 "[('-1', '1'), ('-2', '-1')]": 1,
 "[('-1', '0'), ('free', '0')]": 2,
 "[('-2', '0'), ('free', '0')]": 1,
 "[('-2', 'free'), ('free', '0')]": 2
},
"trace_correlation": {
 "no_trace/evanescent: Ktilde=in K-only=out": 6,
 "no_trace/evanescent: Ktilde=out K-only=in": 58,
 "no_trace/evanescent: Ktilde=out K-only=out": 11,
 "trace/evanescent: Ktilde=in K-only=in": 41,
 "trace/evanescent: Ktilde=in K-only=out": 34,
 "trace/evanescent: Ktilde=out K-only=in": 40,
 "trace/evanescent: Ktilde=out K-only=out": 14
},
"reading": "With the Ktilde construction 71 of 194 nonzero F images and 8 of 33 G images lie in N (all of them also satisfy the ghost equation). The remaining failures are all in the h.h.c row (G also in the ghost rows). No single normalisation works for every generator (lam/mu scan), and failures occur with and without trace monomials, so the eta-trace hypothesis is NOT the (sole) cause. The row operators as inserted on evanescent X do not compose to zero with this sigma: sigma^2 = 0 is not realised on the evanescent sector of the h.h.c row. Diagnosis: OPEN (candidate: the 4D-momentum rule T(p, .) = 0 for hat components, under which s0 h_hat = 0, versus the d-dimensional gauge structure of the kinetic operator)."
}
```


certificates/Q11_classification.json


```
{
 "N_g": 104,
 "sigma_span_passing": 46,
 "blind_in_N_g": 24,
 "sigma_cap_blind": 9,
 "sigma_plus_blind": 61,
 "unclassified": 43,
 "status": "CONDITIONAL: the 46 directions are sigma-images of local Ktilde-functionals that lie in N_g, hence trivial IF this sigma is the correct linearised Slavnov-Taylor operator on the evanescent sector; that is exactly what the failures above put in doubt. 24 blind directions (9 overlapping the sigma span). 43 directions are unclassified: NOT shown to be physical.",
 "physical_evanescent_directions": "not established (no direction certified physical)"
}
```


certificates/Q11_P6_status.json


```
{
 "landmark": "XVI-Q",
 "release": "Q11",
 "P6_outcome": "Partial",
 "sigma0_status_kappa3": "evanescent kappa^3 three-point defect sigma0-exact at pole level; X non-unique (ghost-eq compatible)",
 "goals": {
  "1_sigma_images": "Partial: genuine Ktilde construction built (F, G; P absorbed as the cbar partner of F); 71/194 F and 8/33 G in N; Q08 F defect diagnosed (missing cbar partner / E channel); remaining failures OPEN (not trace-driven)",
  "2_classification": "Partial (conditional): N_g 104 = sigma span 46 + blind 24 - overlap 9 + 43 unclassified",
```

```
  "3_invariant_basis": "OPEN: not constructed",
  "3_p22_consistency": "OPEN: not reached"
 },
 "not_implied": [
  "uniqueness or physicality of X",
  "an unconditional count of BRST-trivial directions",
  "any physical evanescent direction",
  "p22 consistency",
  "bulk symmetry restoration",
  "any change to the O.1 record"
 ],
 "disclosed_revisions": {
  "Q11-R1": "disclosed scope revision of the frozen Q10 layer gate (code/p6q06/q07/q08/q09/q10/release_q10.py): files under
code/p6q06/q07/q08/q09/q10/q11/ and tools/*q11* are left to the Q11 gate; and the Q10 gate treats n_files_scanned of Q09_guard_scan3 as a lower bound
(new Q11 files are scanned, not exempted); release_q10.py is pinned by q11_expected.json. Q05-Q09 gates untouched."
 },
 "premises_note": "OpenAI catalogue and En05-En07 sheets are not premises (Q10 intake)",
 "O1_closure_unchanged": "CLOSED (scoped)",
 "bulk_symmetry_restored": false,
 "insertion": "frozen",
 "flags": {
  "execution_pass": true,
  "claim_status": "partial",
  "scope": "pole level, one loop, kappa^3, three-point rows"
 },
 "seconds": 400.6
}
```


ledgers/Q11_milestones.json


```
{
 "landmark": "XVI-Q",
 "release": "Q11",
 "P6_outcome": "Partial",
 "sigma0_status_kappa3": "evanescent kappa^3 three-point defect sigma0-exact at pole level; X non-unique (ghost-eq compatible)",
 "goals": {
  "1_sigma_images": "Partial: genuine Ktilde construction built (F, G; P absorbed as the cbar partner of F); 71/194 F and 8/33 G in N; Q08 F defect
diagnosed (missing cbar partner / E channel); remaining failures OPEN (not trace-driven)",
  "2_classification": "Partial (conditional): N_g 104 = sigma span 46 + blind 24 - overlap 9 + 43 unclassified",
  "3_invariant_basis": "OPEN: not constructed",
  "3_p22_consistency": "OPEN: not reached"
 },
 "not_implied": [
  "uniqueness or physicality of X",
  "an unconditional count of BRST-trivial directions",
  "any physical evanescent direction",
  "p22 consistency",
  "bulk symmetry restoration",
  "any change to the O.1 record"
 ],
 "disclosed_revisions": {
  "Q11-R1": "disclosed scope revision of the frozen Q10 layer gate (code/p6q06/q07/q08/q09/q10/release_q10.py): files under
code/p6q06/q07/q08/q09/q10/q11/ and tools/*q11* are left to the Q11 gate; and the Q10 gate treats n_files_scanned of Q09_guard_scan3 as a lower bound
(new Q11 files are scanned, not exempted); release_q10.py is pinned by q11_expected.json. Q05-Q09 gates untouched."
 },
 "premises_note": "OpenAI catalogue and En05-En07 sheets are not premises (Q10 intake)",
 "O1_closure_unchanged": "CLOSED (scoped)",
 "bulk_symmetry_restored": false,
 "insertion": "frozen",
 "flags": {
  "execution_pass": true,
  "claim_status": "partial",
  "scope": "pole level, one loop, kappa^3, three-point rows"
 }
}
```


certificates/Q10_P6_status.json


```
{
 "landmark": "XVI-Q",
 "release": "Q10",
 "P6_outcome": "Partial",
 "sigma0_status_kappa3": "evanescent kappa^3 three-point defect sigma0-exact at pole level; X non-unique (ghost-eq compatible)",
 "goals": {
  "erratum_Q08_counts": "DONE: Q10-E1 withdraws the Q08 >=97/>=55/>=36 trivial counts and the Q09 139 / 59 counts",
  "K_sector_blind_directions": "DONE: 42 blind coordinates (K 30, S 12); 62 of the 124 passing F generators live only there",
  "bound_36_restated": "DONE: exact, but only a bound on N outside the (unverified) family span",
  "F_trace_failures": "OPEN: 64/65 failures admit no lambda; the defect is in the Q08 construction; the eta-trace hypothesis is neither confirmed nor
excluded",
  "G_P_families": "OPEN: Q08 constructions shown not to be sigma-images (no lambda for 28/33, 132/134); not rebuilt and not proven absent",
  "classification_remaining": "OPEN",
  "invariant_evanescent_basis": "OPEN: not constructed",
```

```
  "p22_kappa2_consistency": "OPEN: not reached"
 },
 "new_manuscripts": "En05-A/B/C, En06-A/B, En07-A and the OpenAI separation edges read; none imported",
 "not_implied": [
  "uniqueness or physicality of X",
  "any count of BRST-trivial directions",
  "kappa^3 four-point / higher rows",
  "p22 consistency",
  "bulk symmetry restoration",
  "any change to the 0.1 record"
 ],
 "disclosed_revisions": {
  "Q10-R1": "disclosed scope revision of the frozen Q09 layer gate (code/p6q06/q07/q08/q09/release_q09.py): files under code/p6q06/q07/q08/q09/q10/
and tools/*q10* are left to the Q10 gate; and the Q09 gate treats n_files_scanned of Q09_guard_scan3 as a lower bound (new Q10 files are scanned, not
exempted); release_q09.py is pinned by q10_expected.json. Q05-Q08 gates untouched."
 },
 "errata": [
  "Q10-E1"
 ],
 "O1_closure_unchanged": "CLOSED (scoped)",
 "bulk_symmetry_restored": false,
 "flags": {
  "execution_pass": true,
  "claim_status": "partial",
  "scope": "pole level, one loop, kappa^3, three-point rows"
 },
 "seconds": 17.0
}
```


certificates/Q10_family_anatomy.json


```
{
 "source": "frozen Q08 linear system and family vectors (build/logs/q08_triv.pkl, regenerated by q08_certs)",
 "blind_coordinates": {
  "n": 42,
  "by_sector": {
   "K": 30,
   "C": 0,
   "S": 12,
   "H": 0
  },
  "dim_inside_ghost_eq": 24,
  "meaning": "X coordinates on which no three-point row and no Bose condition depends (zero columns of A): free at this order, unconstrained by the
rows"
 },
 "families": {
  "F": {
   "n": 204,
   "in_N": 124,
   "in_N_blind": 62,
   "in_N_nonblind": 62,
   "fail": 65,
   "fail_kpure": 0,
   "zero_vector": 15,
   "lam_any": 124,
   "lam_unique": {
    "-2": 1
   },
   "lam_none": 64
  },
  "G": {
   "n": 33,
   "in_N": 0,
   "in_N_blind": 0,
   "in_N_nonblind": 0,
   "fail": 33,
   "fail_kpure": 0,
   "zero_vector": 0,
   "lam_any": 0,
   "lam_unique": {
    "I/2": 5
   },
   "lam_none": 28
  },
  "P": {
   "n": 134,
   "in_N": 0,
   "in_N_blind": 0,
   "in_N_nonblind": 0,
   "fail": 134,
   "fail_kpure": 0,
   "zero_vector": 0,
   "lam_any": 0,
```

```
   "lam_unique": {
    "-2": 2
   },
   "lam_none": 132
  }
 },
 "dim_N": 152,
 "dim_N_with_ghost_eq": 104,
 "dim_span_passing": 59,
 "dim_span_passing_mod_blind": 33,
 "dim_span_passing_inside_ghost_eq": 27,
 "dim_span_passing_plus_blind": 75,
 "reading": "F: of 204 generators 15 vanish after the h-leg symmetrisation; for ALL 124 nonzero generators that pass, each piece is separately in N
(lambda arbitrary), so the lambda test is vacuous on them; 62 of them are supported only on blind coordinates. 64 of the 65 failures admit NO lambda
at all (1 needs lambda=-2). G: 28/33 admit no lambda, 5 need lambda=i/2. P: 132/134 admit no lambda, 2 need lambda=-2. The Q08 hand-built pieces are
therefore not sigma-images of the stated Y for most generators: the defect is in the construction, not in the normalisation, and no single
normalisation (derived or fitted) can repair it."
}
```


certificates/Q10_erratum_Q08_counts.json


```
{
 "id": "Q10-E1",
 "severity": "medium",
 "withdrawn": {
  "Q08": [
   "'>=97 directions of N inside the trivial span at the best fitted lambda'",
   "'>=55 trivial directions'",
   "'>=36 unexplained directions' as a count of NON-trivial directions"
  ],
  "Q09": [
   "'F verified on 139/204 generators' (15 of the 139 are zero vectors and the other 124 pass for every lambda)",
   "'derived trivial span 59 (27 inside the ghost-equation subspace)' as a count of BRST-trivial directions"
  ]
 },
 "reason": "F: of 204 generators 15 vanish after the h-leg symmetrisation; for ALL 124 nonzero generators that pass, each piece is separately in N
(lambda arbitrary), so the lambda test is vacuous on them; 62 of them are supported only on blind coordinates. 64 of the 65 failures admit NO lambda
at all (1 needs lambda=-2). G: 28/33 admit no lambda, 5 need lambda=i/2. P: 132/134 admit no lambda, 2 need lambda=-2. The Q08 hand-built pieces are
therefore not sigma-images of the stated Y for most generators: the defect is in the construction, not in the normalisation, and no single
normalisation (derived or fitted) can repair it.",
 "restated_bound": {
  "statement": "dim N - dim(N cap span{all Q08 family pieces}) = 152 - 116 = 36",
  "Q08_value_recorded": 36,
  "caveat": "lambda-independent and exact as linear algebra, but the spanning pieces are NOT verified sigma-images (anatomy above), so 36 is a lower
bound on the directions of N outside that span only; it is neither a lower bound on the non-trivial (physical) directions nor an upper bound on the
trivial ones"
 },
 "what_remains_established": [
  "dim N = 152, dim N with ghost equation = 104 (Q08, linear algebra)",
  "blind coordinates 42 (K 30, S 12); 24 inside the ghost-equation subspace"
 ],
 "frozen_layers": "Q08/Q09 certificates unchanged; this erratum supersedes the listed statements",
 "Q09_superseded_values": {
  "F_n_in_N": 139,
  "dim_trivial_span_derived": 59
 }
}
```


certificates/Q10_new_manuscripts.json


```
{
 "intake_file": "intake/new_manuscripts_q10.json",
 "sha256": "65ea1f6427bd120f52a6f28f882da2a0057a40cbb87e194af74ab35681634a3d",
 "head_commit": "bb663a20def2692236482a22b21d168d0d3aea76 (2026-10-11T17:00:06Z)",
 "ids": [
  "En05-A",
  "En05-B",
  "En05-C",
  "En06-A",
  "En06-B",
  "En07-A"
 ],
 "openai": "OpenAI-authored external mathematics catalogue, linked ONLY by separation edges (to I ... IX-A etc.); boundary verbatim: 'This edge is a
separation. It is not a",
 "premises_imported": [],
 "relevance_to_Q10": "none (no BRST / DRED / evanescent / counterterm content); cited as context only",
 "XV_B_opened": false
}
```

ledgers/Q10_milestones.json


```
{
 "landmark": "XVI-Q",
 "release": "Q10",
 "P6_outcome": "Partial",
 "sigma0_status_kappa3": "evanescent kappa^3 three-point defect sigma0-exact at pole level; X non-unique (ghost-eq compatible)",
 "goals": {
  "erratum_Q08_counts": "DONE: Q10-E1 withdraws the Q08 >=97/>=55/>=36 trivial counts and the Q09 139 / 59 counts",
  "K_sector_blind_directions": "DONE: 42 blind coordinates (K 30, S 12); 62 of the 124 passing F generators live only there",
  "bound_36_restated": "DONE: exact, but only a bound on N outside the (unverified) family span",
  "F_trace_failures": "OPEN: 64/65 failures admit no lambda; the defect is in the Q08 construction; the eta-trace hypothesis is neither confirmed nor
excluded",
  "G_P_families": "OPEN: Q08 constructions shown not to be sigma-images (no lambda for 28/33, 132/134); not rebuilt and not proven absent",
  "classification_remaining": "OPEN",
  "invariant_evanescent_basis": "OPEN: not constructed",
  "p22_kappa2_consistency": "OPEN: not reached"
 },
 "new_manuscripts": "En05-A/B/C, En06-A/B, En07-A and the OpenAI separation edges read; none imported",
 "not_implied": [
  "uniqueness or physicality of X",
  "any count of BRST-trivial directions",
  "kappa^3 four-point / higher rows",
  "p22 consistency",
  "bulk symmetry restoration",
  "any change to the O.1 record"
 ],
 "disclosed_revisions": {
  "Q10-R1": "disclosed scope revision of the frozen Q09 layer gate (code/p6q06/q07/q08/q09/release_q09.py): files under code/p6q06/q07/q08/q09/q10/
and tools/*q10* are left to the Q10 gate; and the Q09 gate treats n_files_scanned of Q09_guard_scan3 as a lower bound (new Q10 files are scanned, not
exempted); release_q09.py is pinned by q10_expected.json. Q05-Q08 gates untouched."
 },
 "errata": [
  "Q10-E1"
 ],
 "O1_closure_unchanged": "CLOSED (scoped)",
 "bulk_symmetry_restored": false,
 "flags": {
  "execution_pass": true,
  "claim_status": "partial",
  "scope": "pole level, one loop, kappa^3, three-point rows"
 }
}
```


certificates/Q09_P6_status.json


```
{
 "landmark": "XVI-Q",
 "release": "Q09",
 "P6_outcome": "Partial",
 "sigma0_status_kappa3": "evanescent kappa^3 three-point defect sigma0-exact at pole level; X non-unique (ghost-eq compatible)",
 "relabel_note": "supersedes the Q07/Q08 wording 'sigma0 closure CLOSED at pole level' (Q07 review, medium); frozen Q07/Q08 certificates left
unchanged",
 "goals": {
  "a_derived_normalisations": "Partial: lambda_F = lambda_P = -1, lambda_G = +1 derived; F verified on 139/204 generators (failures all trace-type,
OPEN); G, P constructions not closed (OPEN)",
  "b_classification": "Partial: derived trivial span 59 (27 inside the ghost-equation subspace of dim 104); the remaining directions are NOT
classified (trivial vs physical)",
  "c_invariant_operator_basis": "OPEN: not constructed; X not expressed in an operator basis",
  "d_p22_consistency": "OPEN: not reached"
 },
 "review_fixes": {
  "sigma0_relabel": true,
  "Q04_report_bound": true,
  "scan_extension": true
 },
 "not_implied": [
  "uniqueness or physicality of X",
  "kappa^3 four-point / higher rows",
  "finite parts",
  "p22 consistency",
  "bulk symmetry restoration",
  "any change to the O.1 record"
 ],
 "disclosed_revisions": {
  "Q09-R1": "disclosed scope revision of the frozen Q08 layer gate (code/p6q06/q07/q08/release_q08.py): files under code/p6q06/q07/q08/q09/ and
tools/*q09* are left to the Q09 gate; release_q08.py is pinned by q09_expected.json. Q05-Q07 gates untouched. guard_scan2 (Q05) untouched; the
extension is a new Q09 module."
 },
 "O1_closure_unchanged": "CLOSED (scoped)",
 "bulk_symmetry_restored": false,
 "flags": {
  "execution_pass": true,
```

```
  "claim_status": "partial",
  "scope": "pole level, one loop, kappa^3, three-point rows"
 },
 "seconds": 5.5
}
```


certificates/Q09_trivial_families.json


```
{
 "derivation": "sigma(Y) = (S0, Y). Y = K.F: X_hhh = D-op(F), X_Khc = A-op(F); the row operators are the same A-op and D-op and commute, so sigma^2 =
0 fixes lambda_F = -1 (relative sign of the field / antifield derivatives of a fermionic Y); likewise lambda_P = -1 with the E-op; lambda_G = +1 (both
pieces are dY/dc* . dS0/dc).",
 "families": {
  "F": {
   "lambda_derived": "-1",
   "n_generators": 204,
   "n_in_N": 139,
   "n_fail": 65,
   "n_fail_with_trace_monomials": 65
  },
  "G": {
   "lambda_derived": "1",
   "n_generators": 33,
   "n_in_N": 0,
   "n_fail": 33,
   "n_fail_with_trace_monomials": 13
  },
  "P": {
   "lambda_derived": "-1",
   "n_generators": 134,
   "n_in_N": 0,
   "n_fail": 134,
   "n_fail_with_trace_monomials": 98
  }
 },
 "dim_trivial_span_derived": 59,
 "dim_trivial_span_derived_inside_ghost_eq": 27,
 "Q08_fitted_counts_superseded": {
  "fitted_lower_bound_in_N": 97,
  "status": "fitted, not derivation-backed; superseded by the derived counts"
 },
 "diagnosis": "F at the derived lambda: 139/204 generators in N; every one of the 65 failures carries a trace monomial (K or h trace), pointing to the
eta-trace / d-convention inside the probe contraction rather than to the derivation. The Q08 G and P constructions are NOT closed at the derived
lambda (0/33, 0/134): their pieces were built by hand and are inconsistent with the row operators. Both are OPEN."
}
```


certificates/Q09_guard_scan3.json


```
{
 "scope": "every .py under code/, tools/, tests/ (recursive) minus declared probe files",
 "n_files_scanned": 104,
 "extension_hits": {},
 "extension_clean": true,
 "probes_flagged": {
  "methodcaller_attr": true,
  "methodcaller_alias": true,
  "methodcaller_getattr": true,
  "coeff_d": true,
  "coeff_exponent_d": true,
  "independent_d": true,
  "degree_d": true,
  "powers_dict_subscript": true,
  "powers_dict_get": true,
  "coefficients_dict_get": true
 },
 "all_probes_flagged": true,
 "base_scan2_clean": true,
 "base_scan2_evasions_flagged": true
}
```


certificates/Q09_q04_report_binding.json


```
{
 "file": "reproducibility/independent_rebuild_report_q04.json",
 "sha256": "bc2b16d99f01cf172a5c67f6ee1abb5fdb7b9b5c30e6ad3e510c241ffa442a45",
 "pinned": "bc2b16d99f01cf172a5c67f6ee1abb5fdb7b9b5c30e6ad3e510c241ffa442a45",
 "bound": true
}
```

ledgers/Q09_milestones.json


```
{
 "landmark": "XVI-Q",
 "release": "Q09",
 "P6_outcome": "Partial",
 "sigma0_status_kappa3": "evanescent kappa^3 three-point defect sigma0-exact at pole level; X non-unique (ghost-eq compatible)",
 "relabel_note": "supersedes the Q07/Q08 wording 'sigma0 closure CLOSED at pole level' (Q07 review, medium); frozen Q07/Q08 certificates left
unchanged",
 "goals": {
  "a_derived_normalisations": "Partial: lambda_F = lambda_P = -1, lambda_G = +1 derived; F verified on 139/204 generators (failures all trace-type,
OPEN); G, P constructions not closed (OPEN)",
  "b_classification": "Partial: derived trivial span 59 (27 inside the ghost-equation subspace of dim 104); the remaining directions are NOT
classified (trivial vs physical)",
  "c_invariant_operator_basis": "OPEN: not constructed; X not expressed in an operator basis",
  "d_p22_consistency": "OPEN: not reached"
 },
 "review_fixes": {
  "sigma0_relabel": true,
  "Q04_report_bound": true,
  "scan_extension": true
 },
 "not_implied": [
  "uniqueness or physicality of X",
  "kappa^3 four-point / higher rows",
  "finite parts",
  "p22 consistency",
  "bulk symmetry restoration",
  "any change to the O.1 record"
 ],
 "disclosed_revisions": {
  "Q09-R1": "disclosed scope revision of the frozen Q08 layer gate (code/p6q06/q07/q08/release_q08.py): files under code/p6q06/q07/q08/q09/ and
tools/*q09* are left to the Q09 gate; release_q08.py is pinned by q09_expected.json. Q05-Q07 gates untouched. guard_scan2 (Q05) untouched; the
extension is a new Q09 module."
 },
 "O1_closure_unchanged": "CLOSED (scoped)",
 "bulk_symmetry_restored": false,
 "flags": {
  "execution_pass": true,
  "claim_status": "partial",
  "scope": "pole level, one loop, kappa^3, three-point rows"
 }
}
```


certificates/Q08_P6_status.json


```
{
 "landmark": "XVI-Q",
 "release": "Q08",
 "P6_outcome": "Partial",
 "goals": {
  "1_ghost_equation_compatibility": "ESTABLISHED: X can be chosen ghost-equation compatible (rho = -2 from the loops); free directions 152 -> 104",
  "2_invariant_operator_basis": "Partial: no explicit invariant-operator (R^2/Riemann^2-type evanescent) basis constructed; three-point cohomology-
type count only (26 hhh-only closed evanescent cubic structures); X not expressed in an operator basis",
  "3_free_parameters": "Partial: >= 97 (fitted lambda) of 152 in BRST-trivial family span, <= 116; >= 36 uncharacterised; physical vs trivial NOT
settled",
  "4_p22_consistency": "OPEN: not reached (cubic completion of the p22 kappa^2 evanescent operators not constructed)"
 },
 "sigma0_closure_kappa3": "unchanged from Q07: CLOSED at pole level on the one-loop three-point rows (enlarged evanescent CT domain); now also with a
ghost-equation-compatible X",
 "not_implied": [
  "kappa^3 four-point / higher rows",
  "finite parts",
  "uniqueness of X",
  "physical nature of the remaining free directions",
  "consistency with p22 operators",
  "bulk symmetry restoration",
  "any change to the O.1 record"
 ],
 "disclosed_revisions": {
  "Q08-R1": "disclosed scope revision of the frozen Q07 layer gate (code/p6q06/q07/release_q07.py): files under code/p6q06/q07/q08/ and tools/*q08*
are left to the Q08 gate. release_q07.py is pinned by q08_expected.json. Q05 and Q06 gates untouched."
 },
 "O1_closure_unchanged": "CLOSED (scoped)",
 "bulk_symmetry_restored": false,
 "flags": {
  "execution_pass": true,
  "claim_status": "partial",
  "scope": "pole level, one loop, kappa^3, three-point rows"
 },
 "seconds": 186.9
}
```

certificates/Q08_ghost_equation.json


```
{
 "ghost_equation": "Gamma_{cbar c h} = rho Gamma_{K c h}[K -> T(cbar) = sym(k,cbar) - (1/2)(k.cbar) eta]",
 "loop_poles": {
  "rho_total": "-2",
  "rho_G04_G03": "-2",
  "rho_G09_G07": "-2"
 },
 "loops_satisfy_with_rho_minus_2": true,
 "constraint_imposed": "X_{cbar ch} = -2 X_{Kch}[K -> T(cbar)]",
 "solvable_with_ghost_equation": true,
 "particular_solution": {
  "verified_A_x_eq_b": true,
  "n_nonzero": 525,
  "sha256": "ad67cf90b540f88ac13a91da394589b885b4ef87796629ddcd4cdf114699eac9"
 },
 "dim_free_without_ghost_eq": 152,
 "dim_free_with_ghost_eq": 104,
 "trace_convention": "eta-trace at d4-residue level: hat trace 4, tilde trace 0 (geq.py); validated by the loop poles obeying the map with one
constant"
}
```


certificates/Q08_free_directions.json


```
{
 "homogeneous_space": {
  "dim_N": 152,
  "dim_N_with_ghost_eq": 104,
  "rank_rows_bose": 669,
  "n_unknowns": 821
 },
 "single_sector_blind_directions": {
  "dim_N_Kch_only": 69,
  "dim_N_cbar_ch_only": 0,
  "dim_N_cstar_cc_only": 12,
  "dim_N_hhh_only": 26,
  "dim_trivial_best_lambda_cap_N_and_ghost_eq": 55
 },
 "trivial_families": {
  "F": {
   "n_generators": 204,
   "dim_span_both_pieces": 121,
   "dim_N_cap_span_both_pieces_upper_bound": 67,
   "best_uniform_lambda": "-1",
   "dim_N_cap_family_at_best_lambda": 65,
   "scan": {
    "0": 2,
    "1": 61,
    "-1": 65,
    "2": 61,
    "-2": 63,
    "1/2": 61,
    "-1/2": 61,
    "I": 61,
    "-I": 61,
    "2*I": 61,
    "-2*I": 61,
    "I/2": 61,
    "-I/2": 61,
    "4": 61,
    "-4": 61
   }
  },
  "G": {
   "n_generators": 33,
   "dim_span_both_pieces": 66,
   "dim_N_cap_span_both_pieces_upper_bound": 24,
   "best_uniform_lambda": "I/2",
   "dim_N_cap_family_at_best_lambda": 11,
   "scan": {
    "0": 0,
    "1": 0,
    "-1": 0,
    "2": 0,
    "-2": 0,
    "1/2": 0,
    "-1/2": 0,
    "I": 0,
    "-I": 0,
    "2*I": 0,
    "-2*I": 0,
    "I/2": 11,
```

```
    "-I/2": 0,
    "4": 0,
    "-4": 0
   }
  },
  "P": {
   "n_generators": 134,
   "dim_span_both_pieces": 109,
   "dim_N_cap_span_both_pieces_upper_bound": 13,
   "best_uniform_lambda": "-2",
   "dim_N_cap_family_at_best_lambda": 12,
   "scan": {
    "0": 0,
    "1": 0,
    "-1": 0,
    "2": 0,
    "-2": 12,
    "1/2": 0,
    "-1/2": 0,
    "I": 0,
    "-I": 0,
    "2*I": 0,
    "-2*I": 0,
    "I/2": 0,
    "-I/2": 0,
    "4": 0,
    "-4": 0
   }
  }
 },
 "dim_N_cap_all_families_upper_bound": 116,
 "dim_N_cap_all_families_best_lambda": 97,
 "dim_trivial_best_lambda_within_ghost_eq_subspace": 55,
 "reading": "Of the 152 free directions (104 with the ghost equation), at least 97 (55 within the ghost-equation subspace) lie in the span of fitted
BRST-trivial families (field redefinitions h -> h + F(h,h), c -> c + G(c,h), gauge-fermion change cbar.Phi(h,h)); at most 116 can. The families are
only partly closed in N with one uniform lambda (F 65/121, G 11/66, P 12/109): the relative normalisations are FITTED, not derived, and the families
are not complete at three points. Remaining >= 36 directions are not shown trivial: 26 hhh-only sigma0-closed evanescent cubic structures (candidate
invariant evanescent operators) and blind directions of the three-point rows (69 Kch-only, 12 c*cc-only) constrained only by the ghost equation /
four-point rows."
}
```


ledgers/Q08_milestones.json


```
{
 "landmark": "XVI-Q",
 "release": "Q08",
 "P6_outcome": "Partial",
 "goals": {
  "1_ghost_equation_compatibility": "ESTABLISHED: X can be chosen ghost-equation compatible (rho = -2 from the loops); free directions 152 -> 104",
  "2_invariant_operator_basis": "Partial: no explicit invariant-operator (R^2/Riemann^2-type evanescent) basis constructed; three-point cohomology-
type count only (26 hhh-only closed evanescent cubic structures); X not expressed in an operator basis",
  "3_free_parameters": "Partial: >= 97 (fitted lambda) of 152 in BRST-trivial family span, <= 116; >= 36 uncharacterised; physical vs trivial NOT
settled",
  "4_p22_consistency": "OPEN: not reached (cubic completion of the p22 kappa^2 evanescent operators not constructed)"
 },
 "sigma0_closure_kappa3": "unchanged from Q07: CLOSED at pole level on the one-loop three-point rows (enlarged evanescent CT domain); now also with a
ghost-equation-compatible X",
 "not_implied": [
  "kappa^3 four-point / higher rows",
  "finite parts",
  "uniqueness of X",
  "physical nature of the remaining free directions",
  "consistency with p22 operators",
  "bulk symmetry restoration",
  "any change to the O.1 record"
 ],
 "disclosed_revisions": {
  "Q08-R1": "disclosed scope revision of the frozen Q07 layer gate (code/p6q06/q07/release_q07.py): files under code/p6q06/q07/q08/ and tools/*q08*
are left to the Q08 gate. release_q07.py is pinned by q08_expected.json. Q05 and Q06 gates untouched."
 },
 "O1_closure_unchanged": "CLOSED (scoped)",
 "bulk_symmetry_restored": false,
 "flags": {
  "execution_pass": true,
  "claim_status": "partial",
  "scope": "pole level, one loop, kappa^3, three-point rows"
 }
}
```


certificates/Q07_P6_status.json


```
{
```

```
 "landmark": "XVI-Q",
 "release": "Q07",
 "P6_outcome": "Partial",
 "stages": {
  "row_c_phi_phi": "closed at pole level (Q05)",
  "row_c_c_cbar": "4D closed (Q06); evanescent remainder cancellable by local evanescent CT (Q07)",
  "row_c_c_K": "4D closed (Q06); evanescent remainder cancellable by local evanescent CT (Q07)",
  "row_h_h_c": "4D closed (Q07, all channels +1, sigma_F tree-calibrated); evanescent remainder cancellable by local evanescent CT (Q07)",
  "graphs_missing": [],
  "graphs_derived_zero": [
   "T1",
   "T2",
   "T3"
  ]
 },
 "kappa3_CT_domain_reduction": "Partial: three-point kappa^3 CT = -Gamma_pole (4D part, sigma0-closed on all four rows) + X (local evanescent,
existence shown, non-unique); reduction to a basis of local invariant operators / field redefinitions not performed",
 "kappa3_defect": "zero on the 4D part of all four three-point rows; evanescent part sigma0-exact (pole level)",
 "kappa3_CT_solve": "solved at pole level (particular solution certified, Q07_evanescent_CT.json)",
 "sigma0_closure_kappa3": "CLOSED at pole level on the kappa^3 one-loop THREE-POINT rows (c.phi.phi, c.c.cbar, c.c.K, h.h.c) in the enlarged
(evanescent) CT domain",
 "not_implied": [
  "kappa^3 four-point / higher rows",
  "finite parts",
  "ghost-equation compatibility of X",
  "uniqueness of X",
  "bulk symmetry restoration",
  "any change to the O.1 record"
 ],
 "errata": {
  "Q06-E2": "addressed: the F(h).Gamma_{cbar c h} channel is required (tree: sigma_F = i; mutation drop_E detected)"
 },
 "disclosed_revisions": {
  "Q07-R1": "disclosed scope revision of the frozen Q06 layer gate (code/p6q06/release_q06.py): files under code/p6q06/q07/ and tools/*q07* are left
to the Q07 gate, and the pinned Q05_guard_scan file count is a lower bound (as Q06-R1 did for the Q05 gate). Q05 gate untouched. release_q06.py is
pinned by q07_expected.json."
 },
 "O1_closure_unchanged": "CLOSED (scoped)",
 "bulk_symmetry_restored": false,
 "flags": {
  "execution_pass": true,
  "claim_status": "partial",
  "scope": "pole level, one loop, kappa^3, three-point rows"
 },
 "seconds": 651.7
}
```


certificates/Q07_hhc_row.json


```
{
 "row": "h.h.c: c(gc1) p1, h(a2) p2, h(a3) p3; (S, Gamma_1^pole) at kappa^3",
 "channels": {
  "A": "Gamma_hhh (G01+G02+G08+G14+G15) at h(p1) -> s0 h(c)",
  "B": "own Gamma_hh (graviton + ghost + scalar loops) with one leg s1 h = (1/2) L_c h",
  "C": "delta S_EH3/delta h|_{h2 h3} . Gamma_{Kc} (D13 own)",
  "D": "delta S_2/delta h|_{h_i} (EH + gauge fixing) . Gamma_{K h_j c} (G03+G07)",
  "E": "s cbar = sigma_F F(h_i): F . Gamma_{cbar c h_j} (G04+G09) -- erratum Q06-E2 channel"
 },
 "tree_calibration": {
  "sigma_F": "I",
  "tree_row_closes": true,
  "tree_row_without_E_nonzero": true
 },
 "scalar_loops_own": {
  "G14": {
   "n_terms": 368,
   "sha256": "cda2509533470c2f11288abff30d1c90a9909e0042ffa0480d62ae2ef5ee5a17"
  },
  "G15": {
   "n_terms": 493,
   "sha256": "cad4b5800d77028d80b1e0dc0830404aa661aebb5cab714f3f0c069305143e12"
  }
 },
 "matter_sector_closes_exactly": true,
 "channel_terms": {
  "A": 433,
  "B2_grav": 50,
  "B2_ghost": 50,
  "B2_scalar": 40,
  "D2": 191,
  "E2": 1,
  "B3_grav": 229,
  "B3_ghost": 229,
```

```
 "B3_scalar": 179,
 "D3": 423,
 "E3": 1,
 "C": 142
},
"residual_4D": "0",
"residual_4D_zero": true,
"unconstrained_fit_4D": {
 "groups": [
  "A",
  "B",
  "C",
  "D",
  "E"
 ],
 "solution": [
  "y4",
  "y4",
  "y4",
  "y4",
  "y4"
 ]
},
"mutations_detected": {
 "drop_A": true,
 "drop_B": true,
 "drop_C": true,
 "drop_D": true,
 "drop_E": true,
 "flip_sigma_F": true
},
"evanescent_remainder": "-I*H_a2_a2*H_a3_p1*H_p1_p1**2*T_a3_gc1/48 - I*H_a2_a2*H_a3_p1*H_p1_p1*H_p1_p2*T_a3_gc1/48 +
11*I*H_a2_a2*H_a3_p1*H_p1_p1*H_p2_p2*T_a3_gc1/48 + I*H_a2_a2*H_a3_p1*H_p1_p2**2*T_a3_gc1/24 - 5*I*H_a2_a2*H_a3_p1*H_p1_p2*H_p2_p2*T_a3_gc1/48 -
7*I*H_a2_a2*H_a3_p1*H_p2_p2**2*T_a3_gc1/48 - 5*I*H_a2_a2*H_a3_p2*H_p1_p1**2*T_a3_gc1/48 - I*H_a2_a2*H_a3_p2*H_p1_p1*H_p1_p2*T_a3_gc1/6 +
7*I*H_a2_a2*H_a3_p2*H_p1_p1*H_p2_p2*T_a3_gc1/24 + I*H_a2_a2*H_a3_p2*H_p1_p2**2*T_a3_gc1/3 + 25*I*H_a2_a2*H_a3_p2*H_p1_p2*H_p2_p2*T_a3_gc1/48 +
I*H_a2_a2*H_a3_p2*H_p2_p2**2*T_a3_gc1/8 + I*H_a2_a2*H_gc1_p1*H_p1_p1**2*T_a3_a3/48 + I*H_a2_a2*H_gc1_p1*H_p1_p1*H_p1_p2*T_a3_a3/48 +
11*I*H_a2_a2*H_gc1_p1*H_p1_p1*H_p2_p2*T_a3_a3/48 - I*H_a2_a2*H_gc1_p1*H_p1_p2**2*T_a3_a3/24 + 3*I*H_a2_a2*H_gc1_p1*H_p1_p2*H_p2_p2*T_a3_a3/4 +
19*I*H_a2_a2*H_gc1_p1*H_p2_p2**2*T_a3_a3/48 + 3*I*H_a2_a2*H_gc1_p2*H_p1_p1**2*T_a3_a3/16 + I*H_a2_a2*H_gc1_p2*H_p1_p1*H_p1_p2*T_a3_a3/4 -
I*H_a2_a2*H_gc1_p2*H_p1_p2**2*T_a3_a3/4 - I*H_a2_a2*H_gc1_p2*H_p1_p2*H_p2_p2*T_a3_a3/24 + I*H_a2_a2*H_gc1_p2*H_p2_p2**2*T_a3_a3/12 +
I*H_a2_a3*H_a2_p1*H_p1_p1*H_p1_p2*T_a3_gc1/24 + I*H_a2_a3*H_a2_p1*H_p1_p1*H_p2_p2*T_a3_gc1/6 + I*H_a2_a3*H_a2_p1*H_p1_p2**2*T_a3_gc1/12 +
5*I*H_a2_a3*H_a2_p1*H_p1_p2*H_p2_p2*T_a3_gc1/8 + 7*I*H_a2_a3*H_a2_p1*H_p2_p2**2*T_a3_gc1/24 + I*H_a2_a3*H_a2_p2*H_p1_p1**2*T_a3_gc1/6 +
I*H_a2_a3*H_a2_p2*H_p1_p1*H_p1_p2*T_a3_gc1/4 + I*H_a2_a3*H_a2_p2*H_p1_p1*H_p2_p2*T_a3_gc1/6 - 5*I*H_a2_a3*H_a2_p2*H_p1_p2**2*T_a3_gc1/12 -
5*I*H_a2_a3*H_a2_p2*H_p1_p2*H_p2_p2*T_a3_gc1/24 - I*H_a2_a3*H_a3_p1*H_p1_p1*H_p1_p2*T_a2_gc1/8 - I*H_a2_a3*H_a3_p1*H_p1_p1*H_p2_p2*T_a2_gc1/4 +
I*H_a2_a3*H_a3_p1*H_p1_p2*H_p2_p2*T_a2_gc1/3 + 7*I*H_a2_a3*H_a3_p1*H_p2_p2**2*T_a2_gc1/24 + I*H_a2_a3*H_a3_p2*H_p1_p1**2*T_a2_gc1/8 +
I*H_a2_a3*H_a3_p2*H_p1_p1*H_p1_p2*T_a2_gc1/8 - 3*I*H_a2_a3*H_a3_p2*H_p1_p1*H_p2_p2*T_a2_gc1/8 - 5*I*H_a2_a3*H_a3_p2*H_p1_p2*H_p2_p2*T_a2_gc1/24 -
I*H_a2_a3*H_gc1_p1*H_p1_p1**2*T_a2_a3/24 + I*H_a2_a3*H_gc1_p1*H_p1_p1*H_p1_p2*T_a2_a3/24 - I*H_a2_a3*H_gc1_p1*H_p1_p1*H_p2_p2*T_a2_a3/4 +
I*H_a2_a3*H_gc1_p1*H_p1_p2**2*T_a2_a3/4 - I*H_a2_a3*H_gc1_p1*H_p1_p2*H_p2_p2*T_a2_a3 - 7*I*H_a2_a3*H_gc1_p1*H_p2_p2**2*T_a2_a3/12 -
3*I*H_a2_a3*H_gc1_p2*H_p1_p1**2*T_a2_a3/8 - 7*I*H_a2_a3*H_gc1_p2*H_p1_p1*H_p1_p2*T_a2_a3/12 + I*H_a2_a3*H_gc1_p2*H_p1_p1*H_p2_p2*T_a2_a3/6 +
I*H_a2_a3*H_gc1_p2*H_p1_p2**2*T_a2_a3/3 + I*H_a2_a3*H_gc1_p2*H_p1_p2*H_p2_p2*T_a2_a3/3 - I*H_a2_gc1*H_a2_p1*H_p1_p1**2*T_a3_a3/6 -
23*I*H_a2_gc1*H_a2_p1*H_p1_p1*H_p1_p2*T_a3_a3/48 - I*H_a2_gc1*H_a2_p1*H_p1_p1*H_p2_p2*T_a3_a3/2 - 7*I*H_a2_gc1*H_a2_p1*H_p1_p2**2*T_a3_a3/24 -
13*I*H_a2_gc1*H_a2_p1*H_p1_p2*H_p2_p2*T_a3_a3/16 - I*H_a2_gc1*H_a2_p1*H_p2_p2**2*T_a3_a3/3 - 13*I*H_a2_gc1*H_a2_p2*H_p1_p1**2*T_a3_a3/48 -
3*I*H_a2_gc1*H_a2_p2*H_p1_p1*H_p1_p2*T_a3_a3/8 - 13*I*H_a2_gc1*H_a2_p2*H_p1_p1*H_p2_p2*T_a3_a3/48 + I*H_a2_gc1*H_a2_p2*H_p1_p2**2*T_a3_a3/3 +
I*H_a2_gc1*H_a2_p2*H_p1_p2*H_p2_p2*T_a3_a3/6 + I*H_a2_gc1*H_a3_p1*H_p1_p1**2*T_a2_a3/8 + 5*I*H_a2_gc1*H_a3_p1*H_p1_p1*H_p1_p2*T_a2_a3/12 +
I*H_a2_gc1*H_a3_p1*H_p1_p1*H_p2_p2*T_a2_a3/4 + I*H_a2_gc1*H_a3_p1*H_p1_p2**2*T_a2_a3/3 + 5*I*H_a2_gc1*H_a3_p1*H_p1_p2*H_p2_p2*T_a2_a3/8 +
I*H_a2_gc1*H_a3_p1*H_p2_p2**2*T_a2_a3/4 + I*H_a2_gc1*H_a3_p2*H_p1_p1**2*T_a2_a3/3 + 5*I*H_a2_gc1*H_a3_p2*H_p1_p1*H_p1_p2*T_a2_a3/8 +
I*H_a2_gc1*H_a3_p2*H_p1_p1*H_p2_p2*T_a2_a3/6 - I*H_a2_gc1*H_a3_p2*H_p1_p2**2*T_a2_a3/12 - I*H_a2_gc1*H_a3_p2*H_p1_p2*H_p2_p2*T_a2_a3/24 -
I*H_a2_p1**2*H_a3_p1*H_p1_p1*T_a3_gc1/24 - I*H_a2_p1**2*H_a3_p1*H_p1_p2*T_a3_gc1/12 - I*H_a2_p1**2*H_a3_p1*H_p2_p2*T_a3_gc1/24 -
I*H_a2_p1**2*H_a3_p2*H_p1_p1*T_a3_gc1/24 - I*H_a2_p1**2*H_a3_p2*H_p1_p2*T_a3_gc1/12 - I*H_a2_p1**2*H_a3_p2*H_p2_p2*T_a3_gc1/24 +
I*H_a2_p1**2*H_gc1_p1*H_p1_p1*T_a3_a3/24 + I*H_a2_p1**2*H_gc1_p1*H_p1_p2*T_a3_a3/12 + I*H_a2_p1**2*H_gc1_p1*H_p2_p2*T_a3_a3/24 -
I*H_a2_p1**2*H_gc1_p2*H_p1_p1*T_a3_a3/48 - I*H_a2_p1**2*H_gc1_p2*H_p1_p2*T_a3_a3/24 - I*H_a2_p1**2*H_gc1_p2*H_p2_p2*T_a3_a3/48 -
I*H_a2_p1*H_a2_p2*H_a3_p1*H_p1_p1*T_a3_gc1/12 - I*H_a2_p1*H_a2_p2*H_a3_p1*H_p1_p2*T_a3_gc1/6 - I*H_a2_p1*H_a2_p2*H_a3_p1*H_p2_p2*T_a3_gc1/12 -
I*H_a2_p1*H_a2_p2*H_a3_p2*H_p1_p1*T_a3_gc1/12 - 5*I*H_a2_p1*H_a2_p2*H_a3_p2*H_p1_p2*T_a3_gc1/12 - 5*I*H_a2_p1*H_a2_p2*H_a3_p2*H_p2_p2*T_a3_gc1/24 +
5*I*H_a2_p1*H_a2_p2*H_gc1_p1*H_p1_p1*T_a3_a3/48 + 5*I*H_a2_p1*H_a2_p2*H_gc1_p1*H_p1_p2*T_a3_a3/24 + 5*I*H_a2_p1*H_a2_p2*H_gc1_p1*H_p2_p2*T_a3_a3/48 +
I*H_a2_p1*H_a2_p2*H_gc1_p2*H_p1_p1*T_a3_a3/6 + I*H_a2_p1*H_a2_p2*H_gc1_p2*H_p1_p2*T_a3_a3/3 + I*H_a2_p1*H_a2_p2*H_gc1_p2*H_p2_p2*T_a3_a3/6 -
I*H_a2_p1*H_a3_a3*H_p1_p1**2*T_a2_gc1/16 - I*H_a2_p1*H_a3_a3*H_p1_p1*H_p1_p2*T_a2_gc1/16 + I*H_a2_p1*H_a3_a3*H_p1_p1*H_p2_p2*T_a2_gc1/48 -
I*H_a2_p1*H_a3_a3*H_p1_p2**2*T_a2_gc1/8 - 11*I*H_a2_p1*H_a3_a3*H_p1_p2*H_p2_p2*T_a2_gc1/24 - 13*I*H_a2_p1*H_a3_a3*H_p2_p2**2*T_a2_gc1/48 +
I*H_a2_p1*H_a3_gc1*H_p1_p1**2*T_a2_a3/12 + 5*I*H_a2_p1*H_a3_gc1*H_p1_p1*H_p1_p2*T_a2_a3/24 - I*H_a2_p1*H_a3_gc1*H_p1_p1*H_p2_p2*T_a2_a3/12 +
I*H_a2_p1*H_a3_gc1*H_p1_p2**2*T_a2_a3/12 + I*H_a2_p1*H_a3_gc1*H_p1_p2*H_p2_p2*T_a2_a3/3 + I*H_a2_p1*H_a3_gc1*H_p2_p2**2*T_a2_a3/4 -
I*H_a2_p1*H_a3_p1**2*H_p2_p2*T_a2_gc1/24 - I*H_a2_p1*H_a3_p1*H_a3_p2*H_p1_p1*T_a2_gc1/8 + I*H_a2_p1*H_a3_p1*H_a3_p2*H_p2_p2*T_a2_gc1/24 -
I*H_a2_p1*H_a3_p1*H_gc1_p1*H_p1_p1*T_a2_a3/12 - I*H_a2_p1*H_a3_p1*H_gc1_p1*H_p1_p2*T_a2_a3/6 - I*H_a2_p1*H_a3_p1*H_gc1_p2*H_p1_p1*T_a2_a3/24 -
I*H_a2_p1*H_a3_p1*H_gc1_p2*H_p1_p2*T_a2_a3/12 - 3*I*H_a2_p1*H_a3_p1*H_gc1_p2*H_p2_p2*T_a2_a3/8 - I*H_a2_p1*H_a3_p2**2*H_p1_p1*T_a2_gc1/8 +
I*H_a2_p1*H_a3_p2**2*H_p2_p2*T_a2_gc1/12 - I*H_a2_p1*H_a3_p2*H_gc1_p1*H_p1_p1*T_a2_a3/6 - I*H_a2_p1*H_a3_p2*H_gc1_p1*H_p1_p2*T_a2_a3/3 -
I*H_a2_p1*H_a3_p2*H_gc1_p1*H_p2_p2*T_a2_a3/6 - I*H_a2_p1*H_a3_p2*H_gc1_p2*H_p1_p1*T_a2_a3/24 - I*H_a2_p1*H_a3_p2*H_gc1_p2*H_p1_p2*T_a2_a3/12 -
3*I*H_a2_p1*H_a3_p2*H_gc1_p2*H_p2_p2*T_a2_a3/8 + I*H_a2_p1*H_p1_p1**2*T_a2_a3*T_a3_gc1/8 - 5*I*H_a2_p1*H_p1_p1**2*T_a2_gc1*T_a3_a3/24 +
3*I*H_a2_p1*H_p1_p1*H_p1_p2*T_a2_a3*T_a3_gc1/8 - 23*I*H_a2_p1*H_p1_p1*H_p1_p2*T_a2_gc1*T_a3_a3/48 + I*H_a2_p1*H_p1_p1*H_p2_p2*T_a2_a3*T_a3_gc1/8 -
11*I*H_a2_p1*H_p1_p1*H_p2_p2*T_a2_gc1*T_a3_a3/24 + I*H_a2_p1*H_p1_p2**2*T_a2_a3*T_a3_gc1/4 - 3*I*H_a2_p1*H_p1_p2**2*T_a2_gc1*T_a3_a3/8 +
25*I*H_a2_p1*H_p1_p2*H_p2_p2*T_a2_a3*T_a3_gc1/24 - 59*I*H_a2_p1*H_p1_p2*H_p2_p2*T_a2_gc1*T_a3_a3/48 + 13*I*H_a2_p1*H_p2_p2**2*T_a2_a3*T_a3_gc1/24 -
29*I*H_a2_p1*H_p2_p2**2*T_a2_gc1*T_a3_a3/48 - I*H_a2_p2**2*H_a3_p1*H_p1_p1*T_a3_gc1/8 - I*H_a2_p2**2*H_a3_p2*H_p1_p1*T_a3_gc1/12 -
I*H_a2_p2**2*H_a3_p2*H_p1_p2*T_a3_gc1/6 - I*H_a2_p2**2*H_a3_p2*H_p2_p2*T_a3_gc1/12 - I*H_a2_p2**2*H_gc1_p1*H_p1_p1*T_a3_a3/4 -
I*H_a2_p2**2*H_gc1_p1*H_p1_p2*T_a3_a3/2 - I*H_a2_p2**2*H_gc1_p1*H_p2_p2*T_a3_a3/4 - I*H_a2_p2**2*H_gc1_p2*H_p1_p1*T_a3_a3/8 -
I*H_a2_p2**2*H_gc1_p2*H_p1_p2*T_a3_a3/4 - I*H_a2_p2**2*H_gc1_p2*H_p2_p2*T_a3_a3/8 - 7*I*H_a2_p2*H_a3_a3*H_p1_p1**2*T_a2_gc1/24 -
17*I*H_a2_p2*H_a3_a3*H_p1_p1*H_p1_p2*T_a2_gc1/48 - I*H_a2_p2*H_a3_a3*H_p1_p1*H_p2_p2*T_a2_gc1/48 + 5*I*H_a2_p2*H_a3_a3*H_p1_p2**2*T_a2_gc1/24 +
I*H_a2_p2*H_a3_a3*H_p1_p2*H_p2_p2*T_a2_gc1/48 - I*H_a2_p2*H_a3_a3*H_p2_p2**2*T_a2_gc1/8 + I*H_a2_p2*H_a3_gc1*H_p1_p1**2*T_a2_a3/6 +
I*H_a2_p2*H_a3_gc1*H_p1_p1*H_p1_p2*T_a2_a3/3 - 5*I*H_a2_p2*H_a3_gc1*H_p1_p1*H_p2_p2*T_a2_a3/24 - I*H_a2_p2*H_a3_gc1*H_p1_p2*H_p2_p2*T_a2_a3/24 +
```

```
I*H_a2_p2*H_a3_p1**2*H_p1_p1*T_a2_gc1/8 - I*H_a2_p2*H_a3_p1**2*H_p2_p2*T_a2_gc1/12 + I*H_a2_p2*H_a3_p1*H_a3_p2*H_p1_p1*T_a2_gc1/8 -
I*H_a2_p2*H_a3_p1*H_a3_p2*H_p2_p2*T_a2_gc1/24 + 5*I*H_a2_p2*H_a3_p1*H_gc1_p1*H_p2_p2*T_a2_a3/12 - 7*I*H_a2_p2*H_a3_p1*H_gc1_p2*H_p1_p1*T_a2_a3/24 -
7*I*H_a2_p2*H_a3_p1*H_gc1_p2*H_p1_p2*T_a2_a3/12 - 3*I*H_a2_p2*H_a3_p1*H_gc1_p2*H_p2_p2*T_a2_a3/8 + I*H_a2_p2*H_a3_p2**2*H_p2_p2*T_a2_gc1/12 +
I*H_a2_p2*H_a3_p2*H_gc1_p1*H_p1_p1*T_a2_a3/3 + 2*I*H_a2_p2*H_a3_p2*H_gc1_p1*H_p1_p2*T_a2_a3/3 + 5*I*H_a2_p2*H_a3_p2*H_gc1_p1*H_p2_p2*T_a2_a3/8 +
I*H_a2_p2*H_a3_p2*H_gc1_p2*H_p1_p1*T_a2_a3/24 + I*H_a2_p2*H_a3_p2*H_gc1_p2*H_p1_p2*T_a2_a3/12 + I*H_a2_p2*H_p1_p1**2*T_a2_a3*T_a3_gc1/3 -
9*I*H_a2_p2*H_p1_p1**2*T_a2_gc1*T_a3_a3/16 + 7*I*H_a2_p2*H_p1_p1*H_p1_p2*T_a2_a3*T_a3_gc1/12 - 35*I*H_a2_p2*H_p1_p1*H_p1_p2*T_a2_gc1*T_a3_a3/48 -
I*H_a2_p2*H_p1_p1*H_p2_p2*T_a2_a3*T_a3_gc1/24 - 7*I*H_a2_p2*H_p1_p1*H_p2_p2*T_a2_gc1*T_a3_a3/24 - 5*I*H_a2_p2*H_p1_p2**2*T_a2_a3*T_a3_gc1/12 +
13*I*H_a2_p2*H_p1_p2**2*T_a2_gc1*T_a3_a3/24 - I*H_a2_p2*H_p1_p2*H_p2_p2*T_a2_a3*T_a3_gc1/4 + 3*I*H_a2_p2*H_p1_p2*H_p2_p2*T_a2_gc1*T_a3_a3/16 -
I*H_a2_p2*H_p2_p2**2*T_a2_gc1*T_a3_a3/8 + I*H_a3_a3*H_gc1_p1*H_p1_p1**2*T_a2_a2/48 - I*H_a3_a3*H_gc1_p1*H_p1_p1*H_p1_p2*T_a2_a2/48 +
I*H_a3_a3*H_gc1_p1*H_p1_p1*H_p2_p2*T_a2_a2/16 - I*H_a3_a3*H_gc1_p1*H_p1_p2**2*T_a2_a2/8 + 11*I*H_a3_a3*H_gc1_p1*H_p1_p2*H_p2_p2*T_a2_a2/24 +
5*I*H_a3_a3*H_gc1_p1*H_p2_p2**2*T_a2_a2/16 + 3*I*H_a3_a3*H_gc1_p2*H_p1_p1**2*T_a2_a2/16 + 7*I*H_a3_a3*H_gc1_p2*H_p1_p1*H_p1_p2*T_a2_a2/24 -
5*I*H_a3_a3*H_gc1_p2*H_p1_p1*H_p2_p2*T_a2_a2/24 - I*H_a3_a3*H_gc1_p2*H_p1_p2**2*T_a2_a2/6 - 3*I*H_a3_a3*H_gc1_p2*H_p1_p2*H_p2_p2*T_a2_a2/8 -
I*H_a3_a3*H_gc1_p2*H_p2_p2**2*T_a2_a2/12 + I*H_a3_gc1*H_a3_p1*H_p1_p1*H_p2_p2*T_a2_a2/12 - 17*I*H_a3_gc1*H_a3_p1*H_p1_p2*H_p2_p2*T_a2_a2/48 -
I*H_a3_gc1*H_a3_p1*H_p2_p2**2*T_a2_a2/3 + 7*I*H_a3_gc1*H_a3_p2*H_p1_p1*H_p2_p2*T_a2_a2/16 + I*H_a3_gc1*H_a3_p2*H_p1_p2*H_p2_p2*T_a2_a2/6 +
5*I*H_a3_p1**2*H_gc1_p2*H_p2_p2*T_a2_a2/16 - 3*I*H_a3_p1*H_a3_p2*H_gc1_p1*H_p2_p2*T_a2_a2/16 + 5*I*H_a3_p1*H_a3_p2*H_gc1_p2*H_p2_p2*T_a2_a2/12 -
I*H_a3_p1*H_p1_p1**2*T_a2_a2*T_a3_gc1/24 + I*H_a3_p1*H_p1_p1**2*T_a2_a3*T_a2_gc1/12 - I*H_a3_p1*H_p1_p1*H_p1_p2*T_a2_a2*T_a3_gc1/12 +
I*H_a3_p1*H_p1_p1*H_p1_p2*T_a2_a3*T_a2_gc1/6 + 7*I*H_a3_p1*H_p1_p1*H_p2_p2*T_a2_a2*T_a3_gc1/24 - I*H_a3_p1*H_p1_p1*H_p2_p2*T_a2_a3*T_a2_gc1/24 +
I*H_a3_p1*H_p1_p2**2*T_a2_a3*T_a2_gc1/4 - I*H_a3_p1*H_p1_p2*H_p2_p2*T_a2_a2*T_a3_gc1/2 + 7*I*H_a3_p1*H_p1_p2*H_p2_p2*T_a2_a3*T_a2_gc1/8 -
23*I*H_a3_p1*H_p2_p2**2*T_a2_a2*T_a3_gc1/48 + 13*I*H_a3_p1*H_p2_p2**2*T_a2_a3*T_a2_gc1/24 - I*H_a3_p2**2*H_gc1_p1*H_p2_p2*T_a2_a2/8 +
I*H_a3_p2**2*H_gc1_p2*H_p2_p2*T_a2_a2/8 - 5*I*H_a3_p2*H_p1_p1**2*T_a2_a2*T_a3_gc1/48 + 11*I*H_a3_p2*H_p1_p1**2*T_a2_a3*T_a2_gc1/24 -
I*H_a3_p2*H_p1_p1*H_p1_p2*T_a2_a2*T_a3_gc1/6 + 3*I*H_a3_p2*H_p1_p1*H_p1_p2*T_a2_a3*T_a2_gc1/4 + 35*I*H_a3_p2*H_p1_p1*H_p2_p2*T_a2_a2*T_a3_gc1/48 -
5*I*H_a3_p2*H_p1_p1*H_p2_p2*T_a2_a3*T_a2_gc1/24 + I*H_a3_p2*H_p1_p2**2*T_a2_a2*T_a3_gc1/3 - I*H_a3_p2*H_p1_p2**2*T_a2_a3*T_a2_gc1/12 +
11*I*H_a3_p2*H_p1_p2*H_p2_p2*T_a2_a2*T_a3_gc1/16 - I*H_a3_p2*H_p1_p2*H_p2_p2*T_a2_a3*T_a2_gc1/4 + I*H_a3_p2*H_p2_p2**2*T_a2_a2*T_a3_gc1/8 +
I*H_gc1_p1*H_p1_p1**2*T_a2_a2*T_a3_a3/24 - I*H_gc1_p1*H_p1_p1**2*T_a2_a3**2/24 + I*H_gc1_p1*H_p1_p1*H_p1_p2*T_a2_a3**2/24 +
7*I*H_gc1_p1*H_p1_p1*H_p2_p2*T_a2_a2*T_a3_a3/24 - I*H_gc1_p1*H_p1_p1*H_p2_p2*T_a2_a3**2/4 - I*H_gc1_p1*H_p1_p2**2*T_a2_a2*T_a3_a3/6 +
I*H_gc1_p1*H_p1_p2**2*T_a2_a3**2/4 + 29*I*H_gc1_p1*H_p1_p2*H_p2_p2*T_a2_a2*T_a3_a3/24 - I*H_gc1_p1*H_p1_p2*H_p2_p2*T_a2_a3**2 +
17*I*H_gc1_p1*H_p2_p2**2*T_a2_a2*T_a3_a3/24 - 7*I*H_gc1_p1*H_p2_p2**2*T_a2_a3**2/12 + 3*I*H_gc1_p2*H_p1_p1**2*T_a2_a2*T_a3_a3/8 -
3*I*H_gc1_p2*H_p1_p1**2*T_a2_a3**2/8 + 13*I*H_gc1_p2*H_p1_p1*H_p1_p2*T_a2_a2*T_a3_a3/24 - 7*I*H_gc1_p2*H_p1_p1*H_p1_p2*T_a2_a3**2/12 -
5*I*H_gc1_p2*H_p1_p1*H_p2_p2*T_a2_a2*T_a3_a3/24 + I*H_gc1_p2*H_p1_p1*H_p2_p2*T_a2_a3**2/6 - 5*I*H_gc1_p2*H_p1_p2**2*T_a2_a2*T_a3_a3/12 +
I*H_gc1_p2*H_p1_p2**2*T_a2_a3**2/3 - 5*I*H_gc1_p2*H_p1_p2*H_p2_p2*T_a2_a2*T_a3_a3/12 + I*H_gc1_p2*H_p1_p2*H_p2_p2*T_a2_a3**2/3",
 "evanescent_remainder_sha256": "e6f3c0773410c2ebf3c8c7cfbc585b53f38f2c80070b0efb8669d98f2a227591",
 "evanescent_remainder_n_terms": 248,
 "evanescent_remainder_every_term_has_T": true
}
```


certificates/Q07_evanescent_CT.json


```
{
 "question": "sigma0-consistency of evanescent kappa^3 counterterms removing the evanescent remainders",
 "naive_p22_type": {
  "definition": "CT_ev = -(T-part of every loop input), i.e. rows on T-projected inputs",
  "ccK": {
   "zero": false,
   "n_T_terms_left": 22
  },
  "ccb": {
   "zero": false,
   "n_T_terms_left": 26
  },
  "verdict": "NOT sigma0-consistent: tree vertices regenerate T structures from the 4D loop parts"
 },
 "joint_local_CT": {
  "definition": "CT = -Gamma_pole + X, X = generic local evanescent (>= one T) kappa^3 monomials for Gamma_{Kch}, Gamma_{cbar ch}, Gamma_{c*cc},
Gamma_hhh (Bose symmetric); the same X_{Kch}, X_{cbar ch} enter c.c.K/c.c.cbar and h.h.c; kappa^2 poles unmodified",
  "unknowns": {
   "Gamma_Kch": 134,
   "Gamma_cbar_ch": 81,
   "Gamma_cstar_cc": 18,
   "Gamma_hhh": 588
  },
  "n_equations": 1521,
  "solvable": true,
  "n_free": 152,
  "particular_CT_Gamma_Kch": "I*H_a3_a3*H_aK_p1*H_p1_p2*T_aK_gc1/24 + I*H_a3_a3*H_aK_p1*H_p2_p2*T_aK_gc1/48 + I*H_a3_a3*H_aK_p2*H_p1_p1*T_aK_gc1/16 -
I*H_a3_a3*H_aK_p2*H_p2_p2*T_aK_gc1/24 + I*H_a3_aK*H_a3_p1*H_p1_p2*T_aK_gc1/6 + I*H_a3_aK*H_a3_p1*H_p2_p2*T_aK_gc1/6 -
I*H_a3_aK*H_a3_p2*H_p1_p1*T_aK_gc1/6 - I*H_a3_aK*H_a3_p2*H_p1_p2*T_aK_gc1/6 - I*H_a3_aK*H_aK_p1*H_p1_p1*T_a3_gc1/16 -
2*I*H_a3_aK*H_aK_p1*H_p1_p2*T_a3_gc1/3 - 5*I*H_a3_aK*H_aK_p1*H_p2_p2*T_a3_gc1/12 + I*H_a3_aK*H_aK_p2*H_p1_p1*T_a3_gc1/24 +
I*H_a3_aK*H_aK_p2*H_p1_p2*T_a3_gc1/4 + I*H_a3_aK*H_gc1_p2*H_p1_p1*T_a3_aK/6 + I*H_a3_aK*H_gc1_p2*H_p1_p2*T_a3_aK/4 +
I*H_a3_aK*H_gc1_p2*H_p2_p2*T_a3_aK/24 - I*H_a3_gc1*H_aK_p1*H_p1_p2*T_a3_aK/4 - I*H_a3_gc1*H_aK_p1*H_p2_p2*T_a3_aK/24 +
11*I*H_a3_p1*H_aK_aK*H_p1_p1*T_a3_gc1/64 - 11*I*H_a3_p1*H_aK_aK*H_p1_p2*T_a3_gc1/96 - 13*I*H_a3_p1*H_aK_aK*H_p2_p2*T_a3_gc1/96 +
3*I*H_a3_p1*H_aK_p1**2*T_a3_gc1/16 + 7*I*H_a3_p1*H_aK_p1*H_aK_p2*T_a3_gc1/48 - 17*I*H_a3_p1*H_aK_p2**2*T_a3_gc1/48 +
55*I*H_a3_p2*H_aK_aK*H_p1_p1*T_a3_gc1/192 + 23*I*H_a3_p2*H_aK_p1**2*T_a3_gc1/48 + 11*I*H_a3_p2*H_aK_p1*H_aK_p2*T_a3_gc1/16 +
I*H_aK_aK*H_gc1_p1*H_p1_p1*T_a3_a3/32 + 19*I*H_aK_aK*H_gc1_p1*H_p1_p2*T_a3_a3/48 + 7*I*H_aK_aK*H_gc1_p1*H_p2_p2*T_a3_a3/32 -
13*I*H_aK_aK*H_gc1_p2*H_p1_p1*T_a3_a3/48 - I*H_aK_aK*H_gc1_p2*H_p1_p2*T_a3_a3/16 + I*H_aK_aK*H_gc1_p2*H_p2_p2*T_a3_a3/48 +
3*I*H_aK_gc1*H_aK_p1*H_p1_p1*T_a3_a3/32 + 3*I*H_aK_gc1*H_aK_p1*H_p1_p2*T_a3_a3/16 + I*H_aK_gc1*H_aK_p1*H_p2_p2*T_a3_a3/12 -
3*I*H_aK_gc1*H_aK_p2*H_p1_p1*T_a3_a3/64 + I*H_aK_gc1*H_aK_p2*H_p1_p2*T_a3_a3/8 + I*H_aK_gc1*H_aK_p2*H_p2_p2*T_a3_a3/8 +
I*H_aK_p1**2*H_gc1_p1*T_a3_a3/32 + I*H_aK_p1**2*H_gc1_p2*T_a3_a3/24 - I*H_aK_p1*H_aK_p2*H_gc1_p2*T_a3_a3/12 + 13*I*H_aK_p2**2*H_gc1_p1*T_a3_a3/48 +
I*H_aK_p2**2*H_gc1_p2*T_a3_a3/8",
  "particular_CT_Gamma_Kch_n_terms": 44,
  "particular_CT_Gamma_cbar_ch": "-11*I*H_a3_a3*H_p1_p1**2*T_gb2_gc1/64 - 15*I*H_a3_a3*H_p1_p1*H_p1_p2*T_gb2_gc1/32 -
61*I*H_a3_a3*H_p1_p1*H_p2_p2*T_gb2_gc1/192 + I*H_a3_a3*H_p1_p2**2*T_gb2_gc1/16 + 3*I*H_a3_a3*H_p1_p2*H_p2_p2*T_gb2_gc1/16 +
I*H_a3_a3*H_p2_p2**2*T_gb2_gc1/8 - 5*I*H_a3_gb2*H_p1_p1**2*T_a3_gc1/48 - 25*I*H_a3_gb2*H_p1_p1*H_p1_p2*T_a3_gc1/96 -
25*I*H_a3_gb2*H_p1_p1*H_p2_p2*T_a3_gc1/96 + 2*I*H_a3_gb2*H_p1_p2**2*T_a3_gc1/3 + I*H_a3_gb2*H_p1_p2*H_p2_p2*T_a3_gc1/6 -
```

```
7*I*H_a3_gc1*H_p1_p1**2*T_a3_gb2/32 - 5*I*H_a3_gc1*H_p1_p1*H_p1_p2*T_a3_gb2/32 + 13*I*H_a3_gc1*H_p1_p1*H_p2_p2*T_a3_gb2/48 -
7*I*H_a3_gc1*H_p1_p2**2*T_a3_gb2/24 - 5*I*H_a3_gc1*H_p1_p2*H_p2_p2*T_a3_gb2/24 + 29*I*H_a3_p1**2*H_p1_p1*T_gb2_gc1/96 +
I*H_a3_p1**2*H_p1_p2*T_gb2_gc1/24 + I*H_a3_p1**2*H_p2_p2*T_gb2_gc1/12 + 35*I*H_a3_p1*H_a3_p2*H_p1_p1*T_gb2_gc1/48 -
3*I*H_a3_p1*H_a3_p2*H_p1_p2*T_gb2_gc1/8 - I*H_a3_p1*H_a3_p2*H_p2_p2*T_gb2_gc1/4 + 5*I*H_a3_p1*H_gb2_p1*H_p1_p1*T_a3_gc1/48 -
3*I*H_a3_p1*H_gb2_p1*H_p1_p2*T_a3_gc1/8 - 7*I*H_a3_p1*H_gb2_p1*H_p2_p2*T_a3_gc1/48 + 5*I*H_a3_p1*H_gb2_p2*H_p1_p1*T_a3_gc1/48 -
9*I*H_a3_p1*H_gb2_p2*H_p1_p2*T_a3_gc1/16 - I*H_a3_p1*H_gc1_p1*H_p1_p1*T_a3_gb2/24 - I*H_a3_p1*H_gc1_p1*H_p1_p2*T_a3_gb2/12 -
5*I*H_a3_p1*H_gc1_p1*H_p2_p2*T_a3_gb2/12 + 11*I*H_a3_p1*H_gc1_p2*H_p1_p1*T_a3_gb2/32 + 3*I*H_a3_p1*H_gc1_p2*H_p1_p2*T_a3_gb2/8 +
I*H_a3_p1*H_gc1_p2*H_p2_p2*T_a3_gb2/12 + 65*I*H_a3_p2**2*H_p1_p1*T_gb2_gc1/96 + I*H_a3_p2**2*H_p1_p2*T_gb2_gc1/8 - I*H_a3_p2**2*H_p2_p2*T_gb2_gc1/12 +
9*I*H_a3_p2*H_gb2_p1*H_p1_p1*T_a3_gc1/32 - 7*I*H_a3_p2*H_gb2_p1*H_p1_p2*T_a3_gc1/24 - 13*I*H_a3_p2*H_gb2_p1*H_p2_p2*T_a3_gc1/48 +
15*I*H_a3_p2*H_gb2_p2*H_p1_p1*T_a3_gc1/32 - 5*I*H_a3_p2*H_gb2_p2*H_p1_p2*T_a3_gc1/24 - 9*I*H_a3_p2*H_gc1_p1*H_p1_p1*T_a3_gb2/32 +
11*I*H_a3_p2*H_gc1_p1*H_p1_p2*T_a3_gb2/24 - I*H_a3_p2*H_gc1_p1*H_p2_p2*T_a3_gb2/24 - 17*I*H_a3_p2*H_gc1_p2*H_p1_p1*T_a3_gb2/48 -
I*H_a3_p2*H_gc1_p2*H_p2_p2*T_a3_gb2/24 + 3*I*H_gb2_gc1*H_p1_p1**2*T_a3_a3/8 + 21*I*H_gb2_gc1*H_p1_p1*H_p1_p2*T_a3_a3/32 +
27*I*H_gb2_gc1*H_p1_p1*H_p2_p2*T_a3_a3/64 - 3*I*H_gb2_gc1*H_p1_p2**2*T_a3_a3/16 - 5*I*H_gb2_gc1*H_p1_p2*H_p2_p2*T_a3_a3/24 -
I*H_gb2_gc1*H_p2_p2**2*T_a3_a3/8 - I*H_gb2_p1*H_gc1_p1*H_p1_p1*T_a3_a3/24 - I*H_gb2_p1*H_gc1_p1*H_p1_p2*T_a3_a3/16 -
5*I*H_gb2_p1*H_gc1_p2*H_p1_p1*T_a3_a3/32 - 13*I*H_gb2_p1*H_gc1_p2*H_p1_p2*T_a3_a3/48 + I*H_gb2_p2*H_gc1_p1*H_p1_p1*T_a3_a3/4 +
25*I*H_gb2_p2*H_gc1_p1*H_p1_p2*T_a3_a3/48 + I*H_gb2_p2*H_gc1_p2*H_p1_p1*T_a3_a3/12 + 5*I*H_gb2_p2*H_gc1_p2*H_p1_p2*T_a3_a3/24 +
19*I*H_p1_p1**2*T_a3_a3*T_gb2_gc1/192 - I*H_p1_p1**2*T_a3_gb2*T_a3_gc1/3 - 7*I*H_p1_p1*H_p1_p2*T_a3_gb2*T_a3_gc1/12 -
5*I*H_p1_p2**2*T_a3_a3*T_gb2_gc1/48 + 5*I*H_p1_p2**2*T_a3_gb2*T_a3_gc1/12",
  "particular_CT_Gamma_cbar_ch_n_terms": 65,
  "particular_CT_Gamma_cstar_cc": "0",
  "particular_CT_Gamma_cstar_cc_n_terms": 0,
  "particular_CT_Gamma_hhh": "-H_a1_a1*H_a2_a2*H_p1_p1*H_p1_p2*T_a3_a3/48 - H_a1_a1*H_a2_a2*H_p1_p1*H_p2_p2*T_a3_a3/32 -
H_a1_a1*H_a2_a2*H_p1_p2**2*T_a3_a3/32 - H_a1_a1*H_a2_a2*H_p1_p2*H_p2_p2*T_a3_a3/48 + H_a1_a1*H_a2_a3*H_p1_p1**2*T_a2_a3/2 +
23*H_a1_a1*H_a2_a3*H_p1_p1*H_p1_p2*T_a2_a3/24 + 35*H_a1_a1*H_a2_a3*H_p1_p1*H_p2_p2*T_a2_a3/48 - 17*H_a1_a1*H_a2_a3*H_p1_p2**2*T_a2_a3/48 -
7*H_a1_a1*H_a2_a3*H_p1_p2*H_p2_p2*T_a2_a3/6 - 7*H_a1_a1*H_a2_a3*H_p2_p2**2*T_a2_a3/12 - H_a1_a1*H_a2_p1**2*H_p1_p2*T_a3_a3/48 -
H_a1_a1*H_a2_p1**2*H_p2_p2*T_a3_a3/48 + H_a1_a1*H_a2_p1*H_a2_p2*H_p1_p1*T_a3_a3/24 + H_a1_a1*H_a2_p1*H_a2_p2*H_p1_p2*T_a3_a3/24 -
5*H_a1_a1*H_a2_p1*H_a3_p1*H_p1_p1*T_a2_a3/24 - H_a1_a1*H_a2_p1*H_a3_p1*H_p1_p2*T_a2_a3/12 + H_a1_a1*H_a2_p1*H_a3_p1*H_p2_p2*T_a2_a3/8 -
H_a1_a1*H_a2_p1*H_a3_p2*H_p1_p1*T_a2_a3/3 + 7*H_a1_a1*H_a2_p1*H_a3_p2*H_p1_p2*T_a2_a3/16 + 31*H_a1_a1*H_a2_p1*H_a3_p2*H_p2_p2*T_a2_a3/48 +
H_a1_a1*H_a2_p2**2*H_p1_p1*T_a3_a3/24 + H_a1_a1*H_a2_p2**2*H_p1_p2*T_a3_a3/16 + H_a1_a1*H_a2_p2**2*H_p2_p2*T_a3_a3/48 -
5*H_a1_a1*H_a2_p2*H_a3_p1*H_p1_p1*T_a2_a3/24 - 13*H_a1_a1*H_a2_p2*H_a3_p1*H_p1_p2*T_a2_a3/48 - H_a1_a1*H_a2_p2*H_a3_p1*H_p2_p2*T_a2_a3/16 -
H_a1_a1*H_a2_p2*H_a3_p2*H_p1_p1*T_a2_a3/3 + 7*H_a1_a1*H_a2_p2*H_a3_p2*H_p1_p2*T_a2_a3/12 + 7*H_a1_a1*H_a2_p2*H_a3_p2*H_p2_p2*T_a2_a3/12 -
H_a1_a1*H_a3_a3*H_p1_p1**2*T_a2_a2/48 - H_a1_a1*H_a3_a3*H_p1_p1*H_p1_p2*T_a2_a2/24 - H_a1_a1*H_a3_a3*H_p1_p1*H_p2_p2*T_a2_a2/96 +
H_a1_a1*H_a3_a3*H_p1_p2**2*T_a2_a2/96 + H_a1_a1*H_a3_a3*H_p1_p2*H_p2_p2*T_a2_a2/48 + H_a1_a1*H_a3_p1*H_a3_p2*H_p2_p2*T_a2_a2/24 -
H_a1_a1*H_a3_p2**2*H_p1_p2*T_a2_a2/48 + H_a1_a1*H_a3_p2**2*H_p2_p2*T_a2_a2/48 - H_a1_a1*H_p1_p1**2*T_a2_a3**2/24 +
5*H_a1_a1*H_p1_p1*H_p1_p2*T_a2_a3**2/12 - H_a1_a1*H_p1_p1*H_p2_p2*T_a2_a3**2/4 + H_a1_a1*H_p1_p2**2*T_a2_a3**2/12 -
7*H_a1_a1*H_p1_p2*H_p2_p2*T_a2_a3**2/6 - 7*H_a1_a1*H_p2_p2**2*T_a2_a3**2/12 - H_a1_a2**2*H_p1_p1**2*T_a3_a3/4 -
31*H_a1_a2**2*H_p1_p1*H_p1_p2*T_a3_a3/48 - H_a1_a2**2*H_p1_p1*H_p2_p2*T_a3_a3/2 - 7*H_a1_a2**2*H_p1_p2**2*T_a3_a3/24 -
31*H_a1_a2**2*H_p1_p2*H_p2_p2*T_a3_a3/48 - H_a1_a2**2*H_p2_p2**2*T_a3_a3/4 - H_a1_a2*H_a1_a3*H_p1_p1**2*T_a2_a3/4 -
41*H_a1_a2*H_a1_a3*H_p1_p1*H_p1_p2*T_a2_a3/48 - 11*H_a1_a2*H_a1_a3*H_p1_p1*H_p2_p2*T_a2_a3/16 + H_a1_a2*H_a1_a3*H_p1_p2**2*T_a2_a3/3 +
H_a1_a2*H_a1_a3*H_p1_p2*H_p2_p2*T_a2_a3 + H_a1_a2*H_a1_a3*H_p2_p2**2*T_a2_a3/2 + H_a1_a2*H_a1_p1*H_a2_p1*H_p1_p1*T_a3_a3/6 +
7*H_a1_a2*H_a1_p1*H_a2_p1*H_p1_p2*T_a3_a3/24 + 3*H_a1_a2*H_a1_p1*H_a2_p1*H_p2_p2*T_a3_a3/16 + 5*H_a1_a2*H_a1_p1*H_a2_p2*H_p1_p1*T_a3_a3/24 +
3*H_a1_a2*H_a1_p1*H_a2_p2*H_p1_p2*T_a3_a3/8 + 5*H_a1_a2*H_a1_p1*H_a2_p2*H_p2_p2*T_a3_a3/24 + H_a1_a2*H_a1_p1*H_a3_p1*H_p1_p1*T_a2_a3/4 -
H_a1_a2*H_a1_p1*H_a3_p1*H_p1_p2*T_a2_a3/6 + 11*H_a1_a2*H_a1_p1*H_a3_p1*H_p2_p2*T_a2_a3/48 + 5*H_a1_a2*H_a1_p1*H_a3_p2*H_p1_p1*T_a2_a3/12 -
29*H_a1_a2*H_a1_p1*H_a3_p2*H_p1_p2*T_a2_a3/24 - 5*H_a1_a2*H_a1_p1*H_a3_p2*H_p2_p2*T_a2_a3/8 + H_a1_a2*H_a1_p2*H_a2_p1*H_p1_p1*T_a3_a3/24 +
H_a1_a2*H_a1_p2*H_a2_p1*H_p2_p2*T_a3_a3/24 + 3*H_a1_a2*H_a1_p2*H_a2_p2*H_p1_p1*T_a3_a3/16 + 7*H_a1_a2*H_a1_p2*H_a2_p2*H_p1_p2*T_a3_a3/24 +
H_a1_a2*H_a1_p2*H_a2_p2*H_p2_p2*T_a3_a3/6 + 3*H_a1_a2*H_a1_p2*H_a3_p1*H_p1_p1*T_a2_a3/8 + H_a1_a2*H_a1_p2*H_a3_p1*H_p1_p2*T_a2_a3/12 -
H_a1_a2*H_a1_p2*H_a3_p1*H_p2_p2*T_a2_a3/6 + 47*H_a1_a2*H_a1_p2*H_a3_p2*H_p1_p1*T_a2_a3/48 - 5*H_a1_a2*H_a1_p2*H_a3_p2*H_p1_p2*T_a2_a3/12 -
H_a1_a2*H_a1_p2*H_a3_p2*H_p2_p2*T_a2_a3/2 + H_a1_a2*H_a2_a3*H_p1_p1**2*T_a1_a3/2 + H_a1_a2*H_a2_a3*H_p1_p1*H_p1_p2*T_a1_a3 -
11*H_a1_a2*H_a2_a3*H_p1_p1*H_p2_p2*T_a1_a3/16 + H_a1_a2*H_a2_a3*H_p1_p2**2*T_a1_a3/3 - 41*H_a1_a2*H_a2_a3*H_p1_p2*H_p2_p2*T_a1_a3/48 -
H_a1_a2*H_a2_a3*H_p2_p2**2*T_a1_a3/4 - H_a1_a2*H_a2_p1*H_a3_p1*H_p1_p1*T_a1_a3/2 - 5*H_a1_a2*H_a2_p1*H_a3_p1*H_p1_p2*T_a1_a3/12 +
47*H_a1_a2*H_a2_p1*H_a3_p1*H_p2_p2*T_a1_a3/48 - H_a1_a2*H_a2_p1*H_a3_p2*H_p1_p1*T_a1_a3/6 + H_a1_a2*H_a2_p1*H_a3_p2*H_p1_p2*T_a1_a3/12 +
3*H_a1_a2*H_a2_p1*H_a3_p2*H_p2_p2*T_a1_a3/8 - 5*H_a1_a2*H_a2_p2*H_a3_p1*H_p1_p1*T_a1_a3/8 - 29*H_a1_a2*H_a2_p2*H_a3_p1*H_p1_p2*T_a1_a3/24 +
5*H_a1_a2*H_a2_p2*H_a3_p1*H_p2_p2*T_a1_a3/12 + 11*H_a1_a2*H_a2_p2*H_a3_p2*H_p1_p1*T_a1_a3/48 - H_a1_a2*H_a2_p2*H_a3_p2*H_p1_p2*T_a1_a3/6 +
H_a1_a2*H_a2_p2*H_a3_p2*H_p2_p2*T_a1_a3/4 + H_a1_a2*H_a3_a3*H_p1_p1**2*T_a1_a2/2 + 25*H_a1_a2*H_a3_a3*H_p1_p1*H_p1_p2*T_a1_a2/24 +
37*H_a1_a2*H_a3_a3*H_p1_p1*H_p2_p2*T_a1_a2/48 - 13*H_a1_a2*H_a3_a3*H_p1_p2**2*T_a1_a2/48 + 25*H_a1_a2*H_a3_a3*H_p1_p2*H_p2_p2*T_a1_a2/24 +
H_a1_a2*H_a3_a3*H_p2_p2**2*T_a1_a2/2 - 13*H_a1_a2*H_a3_p1**2*H_p1_p1*T_a1_a2/24 - 13*H_a1_a2*H_a3_p1**2*H_p1_p2*T_a1_a2/24 -
47*H_a1_a2*H_a3_p1**2*H_p2_p2*T_a1_a2/48 - H_a1_a2*H_a3_p1*H_a3_p2*H_p1_p1*T_a1_a2/3 + 13*H_a1_a2*H_a3_p1*H_a3_p2*H_p1_p2*T_a1_a2/8 -
H_a1_a2*H_a3_p1*H_a3_p2*H_p2_p2*T_a1_a2/3 - 47*H_a1_a2*H_a3_p2**2*H_p1_p1*T_a1_a2/48 - 13*H_a1_a2*H_a3_p2**2*H_p1_p2*T_a1_a2/24 -
13*H_a1_a2*H_a3_p2**2*H_p2_p2*T_a1_a2/24 + H_a1_a2*H_p1_p1**2*T_a1_a3*T_a2_a3/4 + 3*H_a1_a2*H_p1_p1*H_p1_p2*T_a1_a3*T_a2_a3/4 +
H_a1_a2*H_p1_p1*H_p2_p2*T_a1_a3*T_a2_a3/8 + H_a1_a2*H_p1_p2**2*T_a1_a3*T_a2_a3/2 + 3*H_a1_a2*H_p1_p2*H_p2_p2*T_a1_a3*T_a2_a3/4 +
H_a1_a2*H_p2_p2**2*T_a1_a3*T_a2_a3/4 - 17*H_a1_a3**2*H_p1_p1*H_p2_p2*T_a2_a2/48 - 17*H_a1_a3**2*H_p1_p2*H_p2_p2*T_a2_a2/48 -
H_a1_a3**2*H_p2_p2**2*T_a2_a2/4 + 5*H_a1_a3*H_a1_p1*H_a2_p1*H_p1_p1*T_a2_a3/12 + H_a1_a3*H_a1_p1*H_a2_p1*H_p1_p2*T_a2_a3/2 +
25*H_a1_a3*H_a1_p1*H_a2_p1*H_p2_p2*T_a2_a3/48 - 5*H_a1_a3*H_a1_p1*H_a2_p2*H_p1_p1*T_a2_a3/48 - 13*H_a1_a3*H_a1_p1*H_a2_p2*H_p1_p2*T_a2_a3/24 +
H_a1_a3*H_a1_p1*H_a2_p2*H_p2_p2*T_a2_a3/8 + 5*H_a1_a3*H_a1_p1*H_a3_p1*H_p2_p2*T_a2_a2/48 + H_a1_a3*H_a1_p1*H_a3_p2*H_p1_p1*T_a2_a2/48 -
H_a1_a3*H_a1_p1*H_a3_p2*H_p2_p2*T_a2_a2/24 + 37*H_a1_a3*H_a1_p2*H_a2_p1*H_p1_p1*T_a2_a3/48 - H_a1_a3*H_a1_p2*H_a2_p1*H_p1_p2*T_a2_a3/6 -
H_a1_a3*H_a1_p2*H_a2_p1*H_p2_p2*T_a2_a3/3 + 43*H_a1_a3*H_a1_p2*H_a2_p2*H_p1_p1*T_a2_a3/48 - 7*H_a1_a3*H_a1_p2*H_a2_p2*H_p1_p2*T_a2_a3/12 -
H_a1_a3*H_a1_p2*H_a2_p2*H_p2_p2*T_a2_a3/2 - H_a1_a3*H_a1_p2*H_a3_p1*H_p1_p1*T_a2_a2/48 - H_a1_a3*H_a1_p2*H_a3_p1*H_p1_p2*T_a2_a2/24 +
H_a1_a3*H_a1_p2*H_a3_p1*H_p2_p2*T_a2_a2/8 + H_a1_a3*H_a1_p2*H_a3_p2*H_p1_p1*T_a2_a2/16 + H_a1_a3*H_a1_p2*H_a3_p2*H_p1_p2*T_a2_a2/24 +
H_a1_a3*H_a1_p2*H_a3_p2*H_p2_p2*T_a2_a2/6 - 7*H_a1_a3*H_a2_a2*H_p1_p1**2*T_a1_a3/12 - 7*H_a1_a3*H_a2_a2*H_p1_p1*H_p1_p2*T_a1_a3/6 +
35*H_a1_a3*H_a2_a2*H_p1_p1*H_p2_p2*T_a1_a3/48 - 17*H_a1_a3*H_a2_a2*H_p1_p2**2*T_a1_a3/48 + 23*H_a1_a3*H_a2_a2*H_p1_p2*H_p2_p2*T_a1_a3/24 +
H_a1_a3*H_a2_a2*H_p2_p2**2*T_a1_a3/2 - H_a1_a3*H_a2_a3*H_p1_p1**2*T_a1_a2/4 - 7*H_a1_a3*H_a2_a3*H_p1_p1*H_p1_p2*T_a1_a2/48 -
H_a1_a3*H_a2_a3*H_p1_p1*H_p2_p2*T_a1_a2/3 + 25*H_a1_a3*H_a2_a3*H_p1_p2**2*T_a1_a2/24 - 7*H_a1_a3*H_a2_a3*H_p1_p2*H_p2_p2*T_a1_a2/48 -
H_a1_a3*H_a2_a3*H_p2_p2**2*T_a1_a2/4 + H_a1_a3*H_a2_p1**2*H_p1_p1*T_a1_a3/3 + H_a1_a3*H_a2_p1**2*H_p1_p2*T_a1_a3/3 -
19*H_a1_a3*H_a2_p1**2*H_p2_p2*T_a1_a3/16 + H_a1_a3*H_a2_p1*H_a2_p2*H_p1_p1*T_a1_a3/3 + 29*H_a1_a3*H_a2_p1*H_a2_p2*H_p1_p2*T_a1_a3/24 -
3*H_a1_a3*H_a2_p1*H_a2_p2*H_p2_p2*T_a1_a3/4 + 5*H_a1_a3*H_a2_p1*H_a3_p1*H_p1_p1*T_a1_a2/12 + H_a1_a3*H_a2_p1*H_a3_p1*H_p1_p2*T_a1_a2/3 +
7*H_a1_a3*H_a2_p1*H_a3_p1*H_p2_p2*T_a1_a2/16 - 17*H_a1_a3*H_a2_p1*H_a3_p2*H_p1_p1*T_a1_a2/48 - 11*H_a1_a3*H_a2_p1*H_a3_p2*H_p1_p2*T_a1_a2/8 -
H_a1_a3*H_a2_p1*H_a3_p2*H_p2_p2*T_a1_a2/6 - 47*H_a1_a3*H_a2_p2**2*H_p1_p1*T_a1_a3/48 - 13*H_a1_a3*H_a2_p2**2*H_p1_p2*T_a1_a3/24 -
13*H_a1_a3*H_a2_p2**2*H_p2_p2*T_a1_a3/24 + 25*H_a1_a3*H_a2_p2*H_a3_p1*H_p1_p1*T_a1_a2/48 - H_a1_a3*H_a2_p2*H_a3_p1*H_p2_p2*T_a1_a2/8 +
31*H_a1_a3*H_a2_p2*H_a3_p2*H_p1_p1*T_a1_a2/48 + 2*H_a1_a3*H_a2_p2*H_a3_p2*H_p1_p2*T_a1_a2/3 + H_a1_a3*H_a2_p2*H_a3_p2*H_p2_p2*T_a1_a2/4 -
3*H_a1_a3*H_p1_p1**2*T_a1_a2*T_a2_a3/8 - 3*H_a1_a3*H_p1_p1*H_p1_p2*T_a1_a2*T_a2_a3/4 - H_a1_a3*H_p1_p1*H_p2_p2*T_a1_a2*T_a2_a3/8 +
H_a1_a3*H_p1_p2*H_p2_p2*T_a1_a2*T_a2_a3/4 + H_a1_a3*H_p2_p2**2*T_a1_a2*T_a2_a3/4 + H_a1_p1**2*H_a2_a2*H_p1_p1*T_a3_a3/48 +
H_a1_p1**2*H_a2_a2*H_p1_p2*T_a3_a3/16 + H_a1_p1**2*H_a2_a2*H_p2_p2*T_a3_a3/24 - 13*H_a1_p1**2*H_a2_a3*H_p1_p1*T_a2_a3/24 -
13*H_a1_p1**2*H_a2_a3*H_p1_p2*T_a2_a3/24 - 47*H_a1_p1**2*H_a2_a3*H_p2_p2*T_a2_a3/48 - H_a1_p1**2*H_a2_p1**2*T_a3_a3/24 -
H_a1_p1**2*H_a2_p1*H_a2_p2*T_a3_a3/12 - H_a1_p1**2*H_a2_p1*H_a3_p1*T_a2_a3/6 - H_a1_p1**2*H_a2_p2**2*T_a3_a3/24 - H_a1_p1**2*H_a2_p2*H_a3_p1*T_a2_a3/6
```

```
+ H_a1_p1*H_a1_p2*H_a2_a2*H_p1_p2*T_a3_a3/24 + H_a1_p1*H_a1_p2*H_a2_a2*H_p2_p2*T_a3_a3/24 - 3*H_a1_p1*H_a1_p2*H_a2_a3*H_p1_p1*T_a2_a3/4 +
29*H_a1_p1*H_a1_p2*H_a2_a3*H_p1_p2*T_a2_a3/24 + H_a1_p1*H_a1_p2*H_a2_a3*H_p2_p2*T_a2_a3/3 - H_a1_p1*H_a1_p2*H_a2_p1**2*T_a3_a3/12 -
H_a1_p1*H_a1_p2*H_a2_p1*H_a2_p2*T_a3_a3/6 - H_a1_p1*H_a1_p2*H_a2_p1*H_a3_p1*T_a2_a3/3 - H_a1_p1*H_a1_p2*H_a2_p1*H_a3_p2*T_a2_a3/6 -
H_a1_p1*H_a1_p2*H_a2_p2**2*T_a3_a3/12 + H_a1_p1*H_a1_p2*H_a2_p2*H_a3_p2*T_a2_a3/6 - H_a1_p1*H_a1_p2*H_a3_a3*H_p2_p2*T_a2_a2/24 -
3*H_a1_p1*H_a1_p2*H_p1_p1*T_a2_a3**2/4 + H_a1_p1*H_a1_p2*H_p1_p2*T_a2_a3**2/3 + H_a1_p1*H_a1_p2*H_p2_p2*T_a2_a3**2/3 +
7*H_a1_p1*H_a2_a2*H_a3_p1*H_p1_p1*T_a1_a3/12 + 7*H_a1_p1*H_a2_a2*H_a3_p1*H_p1_p2*T_a1_a3/12 - H_a1_p1*H_a2_a2*H_a3_p1*H_p2_p2*T_a1_a3/3 -
H_a1_p1*H_a2_a2*H_a3_p2*H_p1_p1*T_a1_a3/16 - 13*H_a1_p1*H_a2_a2*H_a3_p2*H_p1_p2*T_a1_a3/48 - 5*H_a1_p1*H_a2_a2*H_a3_p2*H_p2_p2*T_a1_a3/24 -
H_a1_p1*H_a2_a3*H_a2_p1*H_p1_p1*T_a1_a3/2 - 7*H_a1_p1*H_a2_a3*H_a2_p1*H_p1_p2*T_a1_a3/12 + 43*H_a1_p1*H_a2_a3*H_a2_p1*H_p2_p2*T_a1_a3/48 +
H_a1_p1*H_a2_a3*H_a2_p2*H_p1_p1*T_a1_a3/8 - 13*H_a1_p1*H_a2_a3*H_a2_p2*H_p1_p2*T_a1_a3/24 - 5*H_a1_p1*H_a2_a3*H_a2_p2*H_p2_p2*T_a1_a3/48 +
H_a1_p1*H_a2_a3*H_a3_p1*H_p1_p1*T_a1_a2/4 + 2*H_a1_p1*H_a2_a3*H_a3_p1*H_p1_p2*T_a1_a2/3 + 31*H_a1_p1*H_a2_a3*H_a3_p1*H_p2_p2*T_a1_a2/48 -
H_a1_p1*H_a2_a3*H_a3_p2*H_p1_p1*T_a1_a2/8 + 25*H_a1_p1*H_a2_a3*H_a3_p2*H_p2_p2*T_a1_a2/48 + H_a1_p1*H_a2_p1**2*H_a3_p1*T_a1_a3/6 +
H_a1_p1*H_a2_p1**2*H_a3_p2*T_a1_a3/6 + H_a1_p1*H_a2_p1*H_a2_p2*H_a3_p1*T_a1_a3/6 - 5*H_a1_p1*H_a2_p1*H_a3_a3*H_p1_p1*T_a1_a2/24 -
H_a1_p1*H_a2_p1*H_a3_a3*H_p1_p2*T_a1_a2/3 - H_a1_p1*H_a2_p1*H_a3_p1**2*T_a1_a2/6 - H_a1_p1*H_a2_p2**2*H_a3_p2*T_a1_a3/6 -
3*H_a1_p1*H_a2_p2*H_a3_a3*H_p1_p2*T_a1_a2/16 + H_a1_p1*H_a2_p2*H_a3_p1*H_a3_p2*T_a1_a2/3 - H_a1_p2**2*H_a2_a2*H_p1_p1*T_a3_a3/48 -
H_a1_p2**2*H_a2_a2*H_p1_p2*T_a3_a3/48 - 19*H_a1_p2**2*H_a2_a3*H_p1_p1*T_a2_a3/16 + H_a1_p2**2*H_a2_a3*H_p1_p2*T_a2_a3/3 +
H_a1_p2**2*H_a2_a3*H_p2_p2*T_a2_a3/3 - H_a1_p2**2*H_a2_p1**2*T_a3_a3/24 - H_a1_p2**2*H_a2_p1*H_a2_p2*T_a3_a3/12 - H_a1_p2**2*H_a2_p1*H_a3_p1*T_a2_a3/6
- H_a1_p2**2*H_a2_p2**2*T_a3_a3/24 + H_a1_p2**2*H_a2_p2*H_a3_p1*T_a2_a3/6 + H_a1_p2**2*H_a2_p2*H_a3_p2*T_a2_a3/6 +
H_a1_p2**2*H_a3_a3*H_p1_p2*T_a2_a2/48 - H_a1_p2**2*H_a3_p2**2*T_a2_a2/24 - 3*H_a1_p2**2*H_p1_p1*T_a2_a3**2/4 + H_a1_p2**2*H_p1_p2*T_a2_a3**2/3 +
H_a1_p2**2*H_p2_p2*T_a2_a3**2/3 + 31*H_a1_p2*H_a2_a2*H_a3_p1*H_p1_p1*T_a1_a3/48 + 7*H_a1_p2*H_a2_a2*H_a3_p1*H_p1_p2*T_a1_a3/16 -
H_a1_p2*H_a2_a2*H_a3_p1*H_p2_p2*T_a1_a3/3 + H_a1_p2*H_a2_a2*H_a3_p2*H_p1_p1*T_a1_a3/8 - H_a1_p2*H_a2_a2*H_a3_p2*H_p1_p2*T_a1_a3/12 -
5*H_a1_p2*H_a2_a2*H_a3_p2*H_p2_p2*T_a1_a3/24 - H_a1_p2*H_a2_a3*H_a2_p1*H_p1_p1*T_a1_a3/3 - H_a1_p2*H_a2_a3*H_a2_p1*H_p1_p2*T_a1_a3/6 +
37*H_a1_p2*H_a2_a3*H_a2_p1*H_p2_p2*T_a1_a3/48 + 25*H_a1_p2*H_a2_a3*H_a2_p2*H_p1_p1*T_a1_a3/48 + H_a1_p2*H_a2_a3*H_a2_p2*H_p1_p2*T_a1_a3/2 +
5*H_a1_p2*H_a2_a3*H_a2_p2*H_p2_p2*T_a1_a3/12 - H_a1_p2*H_a2_a3*H_a3_p1*H_p1_p1*T_a1_a2/6 - 11*H_a1_p2*H_a2_a3*H_a3_p1*H_p1_p2*T_a1_a2/8 -
17*H_a1_p2*H_a2_a3*H_a3_p1*H_p2_p2*T_a1_a2/48 + 7*H_a1_p2*H_a2_a3*H_a3_p2*H_p1_p1*T_a1_a2/16 + H_a1_p2*H_a2_a3*H_a3_p2*H_p1_p2*T_a1_a2/3 +
5*H_a1_p2*H_a2_a3*H_a3_p2*H_p2_p2*T_a1_a2/12 - H_a1_p2*H_a2_p1**2*H_a3_p2*T_a1_a3/6 - H_a1_p2*H_a2_p1*H_a2_p2*H_a3_p1*T_a1_a3/6 -
H_a1_p2*H_a2_p1*H_a2_p2*H_a3_p2*T_a1_a3/3 + H_a1_p2*H_a2_p1*H_a3_a3*H_p1_p1*T_a1_a2/8 + 37*H_a1_p2*H_a2_p1*H_a3_a3*H_p1_p2*T_a1_a2/48 +
H_a1_p2*H_a2_p1*H_a3_a3*H_p2_p2*T_a1_a2/8 - H_a1_p2*H_a2_p1*H_a3_p1**2*T_a1_a2/6 - H_a1_p2*H_a2_p1*H_a3_p1*H_a3_p2*T_a1_a2/6 -
H_a1_p2*H_a2_p1*H_a3_p2**2*T_a1_a2/6 - H_a1_p2*H_a2_p2**2*H_a3_p2*T_a1_a3/6 - H_a1_p2*H_a2_p2*H_a3_a3*H_p1_p2*T_a1_a2/3 -
5*H_a1_p2*H_a2_p2*H_a3_a3*H_p2_p2*T_a1_a2/24 - H_a1_p2*H_a2_p2*H_a3_p2**2*T_a1_a2/6 + H_a2_a2*H_a3_a3*H_p1_p1*H_p1_p2*T_a1_a1/48 -
H_a2_a2*H_a3_a3*H_p1_p1*H_p2_p2*T_a1_a1/96 + H_a2_a2*H_a3_a3*H_p1_p2**2*T_a1_a1/96 - H_a2_a2*H_a3_a3*H_p1_p2*H_p2_p2*T_a1_a1/24 -
H_a2_a2*H_a3_a3*H_p2_p2**2*T_a1_a1/48 + H_a2_a2*H_a3_p1**2*H_p1_p1*T_a1_a1/48 - H_a2_a2*H_a3_p1**2*H_p1_p2*T_a1_a1/48 +
H_a2_a2*H_a3_p1*H_a3_p2*H_p1_p1*T_a1_a1/24 - 7*H_a2_a2*H_p1_p1**2*T_a1_a3**2/12 - 7*H_a2_a2*H_p1_p1*H_p1_p2*T_a1_a3**2/6 -
H_a2_a2*H_p1_p1*H_p2_p2*T_a1_a3**2/4 + H_a2_a2*H_p1_p2**2*T_a1_a3**2/12 + 5*H_a2_a2*H_p1_p2*H_p2_p2*T_a1_a3**2/12 - H_a2_a2*H_p2_p2**2*T_a1_a3**2/24 -
H_a2_a3**2*H_p1_p1**2*T_a1_a1/4 - 17*H_a2_a3**2*H_p1_p1*H_p1_p2*T_a1_a1/48 - 17*H_a2_a3**2*H_p1_p1*H_p2_p2*T_a1_a1/48 +
H_a2_a3*H_a2_p1*H_a3_p1*H_p1_p1*T_a1_a1/6 + H_a2_a3*H_a2_p1*H_a3_p1*H_p1_p2*T_a1_a1/24 + H_a2_a3*H_a2_p1*H_a3_p1*H_p2_p2*T_a1_a1/16 +
H_a2_a3*H_a2_p1*H_a3_p2*H_p1_p1*T_a1_a1/8 - H_a2_a3*H_a2_p1*H_a3_p2*H_p1_p2*T_a1_a1/24 - H_a2_a3*H_a2_p1*H_a3_p2*H_p2_p2*T_a1_a1/48 -
H_a2_a3*H_a2_p2*H_a3_p1*H_p1_p1*T_a1_a1/24 + H_a2_a3*H_a2_p2*H_a3_p1*H_p2_p2*T_a1_a1/48 + 5*H_a2_a3*H_a2_p2*H_a3_p2*H_p1_p1*T_a1_a1/48 +
H_a2_a3*H_p1_p1**2*T_a1_a2*T_a1_a3/4 + H_a2_a3*H_p1_p1*H_p1_p2*T_a1_a2*T_a1_a3/4 - H_a2_a3*H_p1_p1*H_p2_p2*T_a1_a2*T_a1_a3/8 -
3*H_a2_a3*H_p1_p2*H_p2_p2*T_a1_a2*T_a1_a3/4 - 3*H_a2_a3*H_p2_p2**2*T_a1_a2*T_a1_a3/8 + H_a2_p1**2*H_a3_a3*H_p1_p2*T_a1_a1/48 -
H_a2_p1**2*H_a3_p1**2*T_a1_a1/24 + H_a2_p1**2*H_p1_p1*T_a1_a3**2/3 + H_a2_p1**2*H_p1_p2*T_a1_a3**2/3 - 3*H_a2_p1**2*H_p2_p2*T_a1_a3**2/4 -
H_a2_p1*H_a2_p2*H_a3_a3*H_p1_p1*T_a1_a1/24 + H_a2_p1*H_a2_p2*H_p1_p1*T_a1_a3**2/3 + H_a2_p1*H_a2_p2*H_p1_p2*T_a1_a3**2/3 -
3*H_a2_p1*H_a2_p2*H_p2_p2*T_a1_a3**2/4 - H_a3_a3*H_p1_p1**2*T_a1_a2**2/24 - 7*H_a3_a3*H_p1_p1*H_p1_p2*T_a1_a2**2/12 -
3*H_a3_a3*H_p1_p1*H_p2_p2*T_a1_a2**2/4 - 11*H_a3_a3*H_p1_p2**2*T_a1_a2**2/12 - 7*H_a3_a3*H_p1_p2*H_p2_p2*T_a1_a2**2/12 -
H_a3_a3*H_p2_p2**2*T_a1_a2**2/24 + 3*H_a3_p1*H_a3_p2*H_p1_p1*T_a1_a2**2/4 + 11*H_a3_p1*H_a3_p2*H_p1_p2*T_a1_a2**2/6 +
3*H_a3_p1*H_a3_p2*H_p2_p2*T_a1_a2**2/4",
  "particular_CT_Gamma_hhh_n_terms": 326,
  "hhh_CT_bose_symmetric": true,
  "all_CT_evanescent": true,
  "verified_rows_zero": true,
  "particular_CT_Gamma_Kch_sha256": "2e9ba5428b9f587256c01a3ab1211991292f0b84288105d1278951d3e770cd55",
  "particular_CT_Gamma_cbar_ch_sha256": "7817eb9ece00108a173ed84b0050ef738ab08672b2acdeceab5fd83da4c50c07",
  "particular_CT_Gamma_cstar_cc_sha256": "5feceb66ffc86f38d952786c6d696c79c2dbc239dd4e91b46729d73a27fb57e9",
  "particular_CT_Gamma_hhh_sha256": "96ddfcfcb28b2e30d308e7fc37a99a4f8fc60a9b07345acd7ae8c1acdac4a0a5"
 },
 "verdict": "CANCELLABLE: a local, Bose-symmetric, purely evanescent kappa^3 counterterm X exists with sigma0-variation cancelling every evanescent
remainder on the rows c.c.K, c.c.cbar, h.h.c simultaneously (verified by substitution); not unique (free parameters)",
 "not_tested": [
  "ghost-equation compatibility of X_{cbar ch} with X_{Kch}",
  "X beyond three-point (four-point kappa^3 rows)",
  "finite evanescent remnants (pole level only)"
 ]
}
```


ledgers/Q07_milestones.json


```
{
 "landmark": "XVI-Q",
 "release": "Q07",
 "P6_outcome": "Partial",
 "stages": {
  "row_c_phi_phi": "closed at pole level (Q05)",
  "row_c_c_cbar": "4D closed (Q06); evanescent remainder cancellable by local evanescent CT (Q07)",
  "row_c_c_K": "4D closed (Q06); evanescent remainder cancellable by local evanescent CT (Q07)",
  "row_h_h_c": "4D closed (Q07, all channels +1, sigma_F tree-calibrated); evanescent remainder cancellable by local evanescent CT (Q07)",
  "graphs_missing": [],
  "graphs_derived_zero": [
   "T1",
   "T2",
   "T3"
  ]
 },
 "kappa3_CT_domain_reduction": "Partial: three-point kappa^3 CT = -Gamma_pole (4D part, sigma0-closed on all four rows) + X (local evanescent,
existence shown, non-unique); reduction to a basis of local invariant operators / field redefinitions not performed",
```

```
 "kappa3_defect": "zero on the 4D part of all four three-point rows; evanescent part sigma0-exact (pole level)",
 "kappa3_CT_solve": "solved at pole level (particular solution certified, Q07_evanescent_CT.json)",
 "sigma0_closure_kappa3": "CLOSED at pole level on the kappa^3 one-loop THREE-POINT rows (c.phi.phi, c.c.cbar, c.c.K, h.h.c) in the enlarged
(evanescent) CT domain",
 "not_implied": [
  "kappa^3 four-point / higher rows",
  "finite parts",
  "ghost-equation compatibility of X",
  "uniqueness of X",
  "bulk symmetry restoration",
  "any change to the 0.1 record"
 ],
 "errata": {
  "Q06-E2": "addressed: the F(h).Gamma_{cbar c h} channel is required (tree: sigma_F = i; mutation drop_E detected)"
 },
 "disclosed_revisions": {
  "Q07-R1": "disclosed scope revision of the frozen Q06 layer gate (code/p6q06/release_q06.py): files under code/p6q06/q07/ and tools/*q07* are left
to the Q07 gate, and the pinned Q05_guard_scan file count is a lower bound (as Q06-R1 did for the Q05 gate). Q05 gate untouched. release_q06.py is
pinned by q07_expected.json."
 },
 "O1_closure_unchanged": "CLOSED (scoped)",
 "bulk_symmetry_restored": false,
 "flags": {
  "execution_pass": true,
  "claim_status": "partial",
  "scope": "pole level, one loop, kappa^3, three-point rows"
 }
}
```


certificates/Q06_P6_status.json


```
{
 "landmark": "XVI-Q",
 "release": "Q06",
 "P6_outcome": "Partial",
 "stages": {
  "ghost_rows_4D_closed": true,
  "ghost_rows_evanescent_remainder": "explicit; removable by p22-type evanescent CTs; their kappa^3 s0-consistency OPEN",
  "G10_second_route": true,
  "V4_validated_gauge_identity": true,
  "graphs_evaluated_pole": [
   "G01",
   "G02",
   "G03",
   "G04",
   "G05",
   "G06",
   "G07",
   "G08",
   "G09",
   "G10",
   "G11",
   "G12",
   "G13",
   "G14",
   "G15"
  ],
  "graphs_missing": [],
  "graphs_derived_zero": [
   "T1",
   "T2",
   "T3"
  ],
  "row_c_phi_phi": "closed at pole level (Q05)",
  "row_c_c_cbar": "closed on 4D part; evanescent remainder OPEN",
  "row_c_c_K": "closed on 4D part; evanescent remainder OPEN",
  "row_h_h_c": "Missing (channel ledger recorded; not assembled)"
 },
 "h_h_c_ledger": [
  "Gamma_hhh (G01+G02+G08+G14+G15) [h -> s0 h(c)]",
  "delta S_2/delta h (EH + gauge fixing) . Gamma_{K h c} (G03+G07)",
  "s1 h = L_c h  . Gamma_hh pole (all kappa^2 hh self-energy graphs)",
  "delta S_EH3/delta h|_hh . Gamma_{K c} (D13)",
  "b eliminated: s cbar ~ F(h) => F(h) . Gamma_{cbar c h} (G04+G09)  [channel absent from the Q04 ledger]"
 ],
 "kappa3_CT_domain_reduction": "Missing",
 "kappa3_defect": "Missing",
 "kappa3_CT_solve": "Missing",
 "sigma0_closure_kappa3": "NOT established",
 "finite_parts": "Missing (pole level only)",
 "errata": {
  "Q06-E1": "Q05 ghost rows reported OPEN because of a Grassmann-sign error in the K-channel assembly (rows.ghost_row); not a property of the graphs
(low; corrected here, Q05 record kept)",
```

```
  "Q06-E2": "Q04 ledger for h.h.c omits the b-elimination channel F(h).Gamma_{cbar c h} (low; recorded)"
 },
 "disclosed_revisions": {
  "Q06-R1": "Q06-R1 (disclosed revision of the frozen Q05 layer): code/p6q05/release_q05.py now treats the pinned Q05_guard_scan n_files_scanned as a
LOWER bound (the recursive scan count grows when Q06 adds code files) and excludes Q06-owned files (code/p6q06/, tests/test_q06.py, tools/*q06*) from
its unpinned-file check; those are pinned by the Q06 gate. Q05_guard_scan.json regenerated (26 -> 33 files, still clean). release_q05.py is re-pinned
by q06_expected.json."
 },
 "O1_closure_unchanged": "CLOSED (scoped)",
 "bulk_symmetry_restored": false,
 "flags": {
  "execution_pass": true,
  "claim_status": "partial",
  "scope": "pole level, one loop, kappa^3"
 },
 "seconds": 881.3
}
```


certificates/Q06_ghost_rows.json


```
{
 "identity": "(S, Gamma_1^pole) restricted to c.c.X monomials, X = cbar / K (Q05 channels)",
 "diagnosis": "Q05 assembled the K channel (delta S1/delta h|_{X c_j} . Gamma_{K c_i}) with the orientation sign of the h channel.  In the K channel
c_i sits in Gamma_{K c_i}, to the right of c_j, so reordering to c1 c2 costs one extra Grassmann transposition: K_o enters with -sgn(o).  G10 (checked
by a second route), the c channels and the identical-c antisymmetrisation were correct.",
 "c.c.K": {
  "row": "ccK",
  "channel_coefficients": {
   "h1": "1",
   "h2": "-1",
   "K1": "-1",
   "K2": "1",
   "c1": "1",
   "c2": "1"
  },
  "channel_terms": {
   "h1": 101,
   "K1": 8,
   "h2": 101,
   "K2": 8,
   "c1": 54,
   "c2": 30
  },
  "unconstrained_fit_4D": {
   "channels": [
    "K1",
    "K2",
    "c1",
    "c2",
    "h1",
    "h2"
   ],
   "solution_up_to_scale": [
    "x5",
    "-x5",
    "-x5",
    "-x5",
    "-x5",
    "x5"
   ]
  },
  "residual_4D": "0",
  "residual_4D_zero": true,
  "evanescent_remainder": "-H_aK_p1*H_aK_p2*H_p1_p1*T_gc1_gc2/16 + H_aK_p1*H_aK_p2*H_p2_p2*T_gc1_gc2/16 + H_aK_p1*H_gc1_p1*H_p1_p1*T_aK_gc2/24 -
H_aK_p1*H_gc1_p1*H_p2_p2*T_aK_gc2/48 - H_aK_p1*H_gc1_p2*H_p1_p1*T_aK_gc2/48 - H_aK_p1*H_gc1_p2*H_p1_p2*T_aK_gc2/12 +
H_aK_p1*H_gc1_p2*H_p2_p2*T_aK_gc2/48 - H_aK_p1*H_gc2_p1*H_p1_p1*T_aK_gc1/48 + H_aK_p1*H_gc2_p1*H_p1_p2*T_aK_gc1/12 +
H_aK_p1*H_gc2_p1*H_p2_p2*T_aK_gc1/48 + H_aK_p1*H_gc2_p2*H_p1_p1*T_aK_gc1/48 - H_aK_p1*H_gc2_p2*H_p2_p2*T_aK_gc1/24 +
H_aK_p2*H_gc1_p1*H_p1_p1*T_aK_gc2/24 - H_aK_p2*H_gc1_p1*H_p2_p2*T_aK_gc2/48 - H_aK_p2*H_gc1_p2*H_p1_p1*T_aK_gc2/48 -
H_aK_p2*H_gc1_p2*H_p1_p2*T_aK_gc2/12 + H_aK_p2*H_gc1_p2*H_p2_p2*T_aK_gc2/48 - H_aK_p2*H_gc2_p1*H_p1_p1*T_aK_gc1/48 +
H_aK_p2*H_gc2_p1*H_p1_p2*T_aK_gc1/12 + H_aK_p2*H_gc2_p1*H_p2_p2*T_aK_gc1/48 + H_aK_p2*H_gc2_p2*H_p1_p1*T_aK_gc1/48 -
H_aK_p2*H_gc2_p2*H_p2_p2*T_aK_gc1/24",
  "evanescent_remainder_n_terms": 22,
  "evanescent_remainder_sha256": "1a99f4aa5af52a7dc8d4941804bddbb5e4e779ec0a73de4874a1cce58497887e",
  "evanescent_remainder_every_term_has_T": true,
  "Q05_assembly_residual_4D_n_terms": 10,
  "mutations_detected": {
   "drop_h1": true,
   "drop_K1": true,
   "drop_h2": true,
   "drop_K2": true,
   "drop_c1": true,
   "drop_c2": true,
   "flip_sigma_c": true,
```

```
   "Q05_K_sign": true
  }
 },
 "c.c.cbar": {
  "row": "ccb",
  "channel_coefficients": {
   "h1": "1",
   "h2": "-1",
   "K1": "-1",
   "K2": "1",
   "c1": "-1",
   "c2": "-1"
  },
  "channel_terms": {
   "h1": 81,
   "K1": 14,
   "h2": 81,
   "K2": 14,
   "c1": 68,
   "c2": 36
  },
  "unconstrained_fit_4D": {
   "channels": [
    "K1",
    "K2",
    "c1",
    "c2",
    "h1",
    "h2"
   ],
   "solution_up_to_scale": [
    "x5",
    "-x5",
    "x5",
    "x5",
    "-x5",
    "x5"
   ]
  },
  "residual_4D": "0",
  "residual_4D_zero": true,
  "evanescent_remainder": "-H_gb3_p1*H_p1_p1*H_p2_p2*T_gc1_gc2/16 + H_gb3_p1*H_p2_p2**2*T_gc1_gc2/16 - H_gb3_p2*H_p1_p1**2*T_gc1_gc2/16 +
H_gb3_p2*H_p1_p1*H_p2_p2*T_gc1_gc2/16 + H_gc1_p1*H_p1_p1**2*T_gb3_gc2/24 + H_gc1_p1*H_p1_p1*H_p1_p2*T_gb3_gc2/12 +
H_gc1_p1*H_p1_p1*H_p2_p2*T_gb3_gc2/48 - H_gc1_p1*H_p1_p2*H_p2_p2*T_gb3_gc2/24 - H_gc1_p1*H_p2_p2**2*T_gb3_gc2/48 - H_gc1_p2*H_p1_p1**2*T_gb3_gc2/48 -
H_gc1_p2*H_p1_p1*H_p1_p2*T_gb3_gc2/8 - H_gc1_p2*H_p1_p2**2*T_gb3_gc2/6 - H_gc1_p2*H_p1_p2*H_p2_p2*T_gb3_gc2/24 + H_gc1_p2*H_p2_p2**2*T_gb3_gc2/48 -
H_gc2_p1*H_p1_p1**2*T_gb3_gc1/48 + H_gc2_p1*H_p1_p1*H_p1_p2*T_gb3_gc1/24 + H_gc2_p1*H_p1_p2**2*T_gb3_gc1/6 + H_gc2_p1*H_p1_p2*H_p2_p2*T_gb3_gc1/8 +
H_gc2_p1*H_p2_p2**2*T_gb3_gc1/48 + H_gc2_p2*H_p1_p1**2*T_gb3_gc1/48 + H_gc2_p2*H_p1_p1*H_p1_p2*T_gb3_gc1/24 - H_gc2_p2*H_p1_p1*H_p2_p2*T_gb3_gc1/48 -
H_gc2_p2*H_p1_p2*H_p2_p2*T_gb3_gc1/12 - H_gc2_p2*H_p2_p2**2*T_gb3_gc1/24",
  "evanescent_remainder_n_terms": 24,
  "evanescent_remainder_sha256": "ff8920c71b47e3d7e1fd1c4055bb3a5e1a139b530817c5efb9dc45e13e5b55f6",
  "evanescent_remainder_every_term_has_T": true,
  "Q05_assembly_residual_4D_n_terms": 18,
  "mutations_detected": {
   "drop_h1": true,
   "drop_K1": true,
   "drop_h2": true,
   "drop_K2": true,
   "drop_c1": true,
   "drop_c2": true,
   "flip_sigma_c": true,
   "Q05_K_sign": true
  }
 },
 "status": "CLOSED on the 4D (non-evanescent) part at pole level for both rows; the remainder is purely evanescent (every term carries one T between
external polarisations).  Since the row is linear and the 4D part vanishes, local evanescent counterterms equal to minus the evanescent pole parts
(p22 prescription) remove it; whether those counterterms are s0-consistent on the full kappa^3 evanescent domain is OPEN."
}
```


certificates/Q06_G10_crosscheck.json


```
{
 "route2_equals_Q05_G10": true,
 "n_terms": 26,
 "sha256_route2": "2ac7149223a0f1df9951d9c1333608ff63cf5d171258d226a75c2881dadc11a2",
 "route": "graviton momentum l - p1; ghost lines l, p1+p2-l; V_c*cc slots swapped (x -1)"
}
```


certificates/Q06_V4_validation.json


```
{
 "vertex": "V4_grav = i x delta^4 S_EH (kappa^2 stripped), code/p6q06/v4.py",
 "gauge_n2_V3_vs_V2_zero": true,
```

```
 "gauge_n3_V4_vs_V3_zero": true,
 "V4_permutation_symmetric": true,
 "V4_terms_generic": 12732,
 "XVI-P_comparison": "not possible: no non-scaleless XVI-P graph contains V4 (the V4 tadpole is scaleless); validated instead by the exact EH gauge
identity delta0 S4 + delta1 S3 = 0"
}
```


certificates/Q06_fastpath.json


```
{
 "G06_fast_equals_expr": true,
 "G05_fast_equals_expr": true
}
```


certificates/Q06_P6_hhh.json


```
{
 "graphs": {
  "G01": {
   "pole_over_U_d4_residue": "-H_a1_a1*H_a2_a2*H_a3_a3*H_p1_p1**2/24 - H_a1_a1*H_a2_a2*H_a3_a3*H_p1_p1*H_p1_p2/12 -
134*H_a1_a1*H_a2_a2*H_a3_a3*H_p1_p1*H_p2_p2/45 + 1027*H_a1_a1*H_a2_a2*H_a3_a3*H_p1_p2**2/360 - H_a1_a1*H_a2_a2*H_a3_a3*H_p1_p2*H_p2_p2/12 -
H_a1_a1*H_a2_a2*H_a3_a3*H_p2_p2**2/24 + 23*H_a1_a1*H_a2_a2*H_a3_p1**2*H_p1_p1/240 + 7*H_a1_a1*H_a2_a2*H_a3_p1**2*H_p1_p2/48 +
2237*H_a1_a1*H_a2_a2*H_a3_p1**2*H_p2_p2/720 + 7*H_a1_a1*H_a2_a2*H_a3_p1*H_a3_p2*H_p1_p1/60 - 2219*H_a1_a1*H_a2_a2*H_a3_p1*H_a3_p2*H_p1_p2/360 +
7*H_a1_a1*H_a2_a2*H_a3_p1*H_a3_p2*H_p2_p2/60 + 2237*H_a1_a1*H_a2_a2*H_a3_p2**2*H_p1_p1/720 + 7*H_a1_a1*H_a2_a2*H_a3_p2**2*H_p1_p2/48 +
23*H_a1_a1*H_a2_a2*H_a3_p2**2*H_p2_p2/240 - 89*H_a1_a1*H_a2_a2*H_p1_p1**2*T_a3_a3/480 - 23*H_a1_a1*H_a2_a2*H_p1_p1*H_p1_p2*T_a3_a3/48 -
65*H_a1_a1*H_a2_a2*H_p1_p1*H_p2_p2*T_a3_a3/12 + 233*H_a1_a1*H_a2_a2*H_p1_p2**2*T_a3_a3/48 - 23*H_a1_a1*H_a2_a2*H_p1_p2*H_p2_p2*T_a3_a3/48 -
89*H_a1_a1*H_a2_a2*H_p2_p2**2*T_a3_a3/480 + 5*H_a1_a1*H_a2_a3**2*H_p1_p1**2/16 + 43*H_a1_a1*H_a2_a3**2*H_p1_p1*H_p1_p2/120 +
2141*H_a1_a1*H_a2_a3**2*H_p1_p1*H_p2_p2/720 - 787*H_a1_a1*H_a2_a3**2*H_p1_p2**2/360 + 103*H_a1_a1*H_a2_a3**2*H_p1_p2*H_p2_p2/120 +
103*H_a1_a1*H_a2_a3**2*H_p2_p2**2/240 - H_a1_a1*H_a2_a3*H_a2_p1*H_a3_p1*H_p1_p1/24 - 67*H_a1_a1*H_a2_a3*H_a2_p1*H_a3_p1*H_p1_p2/120 -
289*H_a1_a1*H_a2_a3*H_a2_p1*H_a3_p1*H_p2_p2/45 + H_a1_a1*H_a2_a3*H_a2_p1*H_a3_p2*H_p1_p1/60 + 385*H_a1_a1*H_a2_a3*H_a2_p1*H_a3_p2*H_p1_p2/72 -
31*H_a1_a1*H_a2_a3*H_a2_p1*H_a3_p2*H_p2_p2/60 - 7*H_a1_a1*H_a2_a3*H_a2_p2*H_a3_p1*H_p1_p1/40 + 1009*H_a1_a1*H_a2_a3*H_a2_p2*H_a3_p1*H_p1_p2/180 -
31*H_a1_a1*H_a2_a3*H_a2_p2*H_a3_p1*H_p2_p2/120 - 271*H_a1_a1*H_a2_a3*H_a2_p2*H_a3_p2*H_p1_p1/45 - 31*H_a1_a1*H_a2_a3*H_a2_p2*H_a3_p2*H_p1_p2/40 -
31*H_a1_a1*H_a2_a3*H_a2_p2*H_a3_p2*H_p2_p2/40 + 9*H_a1_a1*H_a2_a3*H_p1_p1**2*T_a2_a3/20 + 7*H_a1_a1*H_a2_a3*H_p1_p1*H_p1_p2*T_a2_a3/12 +
53*H_a1_a1*H_a2_a3*H_p1_p1*H_p2_p2*T_a2_a3/6 - 89*H_a1_a1*H_a2_a3*H_p1_p2**2*T_a2_a3/12 + 5*H_a1_a1*H_a2_a3*H_p1_p2*H_p2_p2*T_a2_a3/3 +
5*H_a1_a1*H_a2_a3*H_p2_p2**2*T_a2_a3/6 - 17*H_a1_a1*H_a2_p1**2*H_a3_a3*H_p1_p1/60 - 17*H_a1_a1*H_a2_p1**2*H_a3_a3*H_p1_p2/60 +
1111*H_a1_a1*H_a2_p1**2*H_a3_a3*H_p2_p2/360 + 5*H_a1_a1*H_a2_p1**2*H_a3_p1**2/24 + 5*H_a1_a1*H_a2_p1**2*H_a3_p1*H_a3_p2/8 -
131*H_a1_a1*H_a2_p1**2*H_a3_p2**2/45 - 29*H_a1_a1*H_a2_p1**2*H_p1_p1*T_a3_a3/120 - H_a1_a1*H_a2_p1**2*H_p1_p2*T_a3_a3/4 +
33*H_a1_a1*H_a2_p1**2*H_p2_p2*T_a3_a3/8 - 17*H_a1_a1*H_a2_p1*H_a2_p2*H_a3_a3*H_p1_p1/60 - 227*H_a1_a1*H_a2_p1*H_a2_p2*H_a3_a3*H_p1_p2/36 +
3*H_a1_a1*H_a2_p1*H_a2_p2*H_a3_a3*H_p2_p2/40 + 11*H_a1_a1*H_a2_p1*H_a2_p2*H_a3_p1**2/60 + 2561*H_a1_a1*H_a2_p1*H_a2_p2*H_a3_p1*H_a3_p2/360 +
29*H_a1_a1*H_a2_p1*H_a2_p2*H_a3_p2**2/60 - 233*H_a1_a1*H_a2_p1*H_a2_p2*H_p1_p2*T_a3_a3/24 - H_a1_a1*H_a2_p1*H_a2_p2*H_p2_p2*T_a3_a3/2 +
23*H_a1_a1*H_a2_p1*H_a3_p1*H_p1_p1*T_a2_a3/60 + H_a1_a1*H_a2_p1*H_a3_p1*H_p1_p2*T_a2_a3/12 - 37*H_a1_a1*H_a2_p1*H_a3_p1*H_p2_p2*T_a2_a3/4 +
H_a1_a1*H_a2_p1*H_a3_p2*H_p1_p1*T_a2_a3/12 + 25*H_a1_a1*H_a2_p1*H_a3_p2*H_p1_p2*T_a2_a3/3 - H_a1_a1*H_a2_p1*H_a3_p2*H_p2_p2*T_a2_a3 +
2201*H_a1_a1*H_a2_p2**2*H_a3_a3*H_p1_p1/720 + 11*H_a1_a1*H_a2_p2**2*H_a3_a3*H_p1_p2/240 + 23*H_a1_a1*H_a2_p2**2*H_a3_a3*H_p2_p2/240 -
143*H_a1_a1*H_a2_p2**2*H_a3_p1**2/45 - H_a1_a1*H_a2_p2**2*H_a3_p1*H_a3_p2/20 + 13*H_a1_a1*H_a2_p2**2*H_a3_p2**2/60 +
23*H_a1_a1*H_a2_p2**2*H_p1_p1*T_a3_a3/4 + 15*H_a1_a1*H_a2_p2**2*H_p1_p2*T_a3_a3/16 + 19*H_a1_a1*H_a2_p2**2*H_p2_p2*T_a3_a3/80 -
5*H_a1_a1*H_a2_p2*H_a3_p1*H_p1_p1*T_a2_a3/6 + 29*H_a1_a1*H_a2_p2*H_a3_p1*H_p1_p2*T_a2_a3/3 + H_a1_a1*H_a2_p2*H_a3_p1*H_p2_p2*T_a2_a3/3 -
121*H_a1_a1*H_a2_p2*H_a3_p2*H_p1_p1*T_a2_a3/12 - 2*H_a1_a1*H_a2_p2*H_a3_p2*H_p1_p2*T_a2_a3/3 - 2*H_a1_a1*H_a2_p2*H_a3_p2*H_p2_p2*T_a2_a3/3 +
H_a1_a1*H_a3_a3*H_p1_p1**2*T_a2_a2/40 + H_a1_a1*H_a3_a3*H_p1_p1*H_p1_p2*T_a2_a2/20 - 637*H_a1_a1*H_a3_a3*H_p1_p1*H_p2_p2*T_a2_a2/120 +
1217*H_a1_a1*H_a3_a3*H_p1_p2**2*T_a2_a2/240 - 21*H_a1_a1*H_a3_a3*H_p1_p2*H_p2_p2*T_a2_a2/80 - 89*H_a1_a1*H_a3_a3*H_p2_p2**2*T_a2_a2/480 -
H_a1_a1*H_a3_p1**2*H_p1_p1*T_a2_a2/40 - 161*H_a1_a1*H_a3_p1**2*H_p1_p2*T_a2_a2/240 + 389*H_a1_a1*H_a3_p1**2*H_p2_p2*T_a2_a2/80 +
107*H_a1_a1*H_a3_p1*H_a3_p2*H_p1_p1*T_a2_a2/120 - 289*H_a1_a1*H_a3_p1*H_a3_p2*H_p1_p2*T_a2_a2/30 + 39*H_a1_a1*H_a3_p1*H_a3_p2*H_p2_p2*T_a2_a2/40 +
101*H_a1_a1*H_a3_p2**2*H_p1_p1*T_a2_a2/20 - 37*H_a1_a1*H_a3_p2**2*H_p1_p2*T_a2_a2/80 + 19*H_a1_a1*H_a3_p2**2*H_p2_p2*T_a2_a2/80 -
H_a1_a1*H_p1_p1**2*T_a2_a2*T_a3_a3/12 + 5*H_a1_a1*H_p1_p1**2*T_a2_a3**2/24 - 29*H_a1_a1*H_p1_p1*H_p1_p2*T_a2_a2*T_a3_a3/96 +
5*H_a1_a1*H_p1_p1*H_p1_p2*T_a2_a3**2/16 - 185*H_a1_a1*H_p1_p1*H_p2_p2*T_a2_a2*T_a3_a3/24 + 95*H_a1_a1*H_p1_p1*H_p2_p2*T_a2_a3**2/16 +
85*H_a1_a1*H_p1_p2**2*T_a2_a2*T_a3_a3/12 - 125*H_a1_a1*H_p1_p2**2*T_a2_a3**2/24 - 31*H_a1_a1*H_p1_p2*H_p2_p2*T_a2_a2*T_a3_a3/48 +
5*H_a1_a1*H_p1_p2*H_p2_p2*T_a2_a3**2/6 - 31*H_a1_a1*H_p2_p2**2*T_a2_a2*T_a3_a3/96 + 5*H_a1_a1*H_p2_p2**2*T_a2_a3**2/12 +
5*H_a1_a2**2*H_a3_a3*H_p1_p1**2/16 + 107*H_a1_a2**2*H_a3_a3*H_p1_p1*H_p1_p2/120 + 2333*H_a1_a2**2*H_a3_a3*H_p1_p1*H_p2_p2/720 -
119*H_a1_a2**2*H_a3_a3*H_p1_p2**2/72 + 107*H_a1_a2**2*H_a3_a3*H_p1_p2*H_p2_p2/120 + 5*H_a1_a2**2*H_a3_a3*H_p2_p2**2/16 -
31*H_a1_a2**2*H_a3_p1**2*H_p1_p1/60 - 11*H_a1_a2**2*H_a3_p1**2*H_p1_p2/20 - 581*H_a1_a2**2*H_a3_p1**2*H_p2_p2/180 -
29*H_a1_a2**2*H_a3_p1*H_a3_p2*H_p1_p1/60 + 883*H_a1_a2**2*H_a3_p1*H_a3_p2*H_p1_p2/180 - 29*H_a1_a2**2*H_a3_p1*H_a3_p2*H_p2_p2/60 -
581*H_a1_a2**2*H_a3_p2**2*H_p1_p1/180 - 11*H_a1_a2**2*H_a3_p2**2*H_p1_p2/20 - 31*H_a1_a2**2*H_a3_p2**2*H_p2_p2/60 + 7*H_a1_a2**2*H_p1_p1**2*T_a3_a3/30
+ 3*H_a1_a2**2*H_p1_p1*H_p1_p2*T_a3_a3/4 + 49*H_a1_a2**2*H_p1_p1*H_p2_p2*T_a3_a3/12 - 65*H_a1_a2**2*H_p1_p2**2*T_a3_a3/24 +
3*H_a1_a2**2*H_p1_p2*H_p2_p2*T_a3_a3/4 + 7*H_a1_a2**2*H_p2_p2**2*T_a3_a3/30 - 31*H_a1_a2*H_a1_a3*H_a2_a3*H_p1_p1**2/60 -
31*H_a1_a2*H_a1_a3*H_a2_a3*H_p1_p1*H_p1_p2/30 - 1021*H_a1_a2*H_a1_a3*H_a2_a3*H_p1_p1*H_p2_p2/180 + 371*H_a1_a2*H_a1_a3*H_a2_a3*H_p1_p2**2/90 -
31*H_a1_a2*H_a1_a3*H_a2_a3*H_p1_p2*H_p2_p2/30 - 31*H_a1_a2*H_a1_a3*H_a2_a3*H_p2_p2**2/60 + 17*H_a1_a2*H_a1_a3*H_a2_p1*H_a3_p1*H_p1_p1/20 +
9*H_a1_a2*H_a1_a3*H_a2_p1*H_a3_p1*H_p1_p2/20 + 557*H_a1_a2*H_a1_a3*H_a2_p1*H_a3_p1*H_p2_p2/90 + 7*H_a1_a2*H_a1_a3*H_a2_p1*H_a3_p2*H_p1_p1/30 -
973*H_a1_a2*H_a1_a3*H_a2_p1*H_a3_p2*H_p1_p2/180 + H_a1_a2*H_a1_a3*H_a2_p1*H_a3_p2*H_p2_p2/3 + H_a1_a2*H_a1_a3*H_a2_p2*H_a3_p1*H_p1_p1/10 -
103*H_a1_a2*H_a1_a3*H_a2_p2*H_a3_p1*H_p1_p2/18 + H_a1_a2*H_a1_a3*H_a2_p2*H_a3_p1*H_p2_p2/60 + 1093*H_a1_a2*H_a1_a3*H_a2_p2*H_a3_p2*H_p1_p1/180 +
7*H_a1_a2*H_a1_a3*H_a2_p2*H_a3_p2*H_p1_p2/20 + 7*H_a1_a2*H_a1_a3*H_a2_p2*H_a3_p2*H_p2_p2/20 - 4*H_a1_a2*H_a1_a3*H_p1_p1**2*T_a2_a3/15 -
3*H_a1_a2*H_a1_a3*H_p1_p1*H_p1_p2*T_a2_a3/4 - 83*H_a1_a2*H_a1_a3*H_p1_p1*H_p2_p2*T_a2_a3/12 + 35*H_a1_a2*H_a1_a3*H_p1_p2**2*T_a2_a3/6 -
2*H_a1_a2*H_a1_a3*H_p1_p2*H_p2_p2*T_a2_a3/3 - H_a1_a2*H_a1_a3*H_p2_p2**2*T_a2_a3/3 + 7*H_a1_a2*H_a1_a1*H_a2_a3*H_a3_p1*H_p1_p1/20 +
7*H_a1_a2*H_a1_p1*H_a2_a3*H_a3_p1*H_p1_p2/20 + 1093*H_a1_a2*H_a1_p1*H_a2_a3*H_a3_p1*H_p2_p2/180 + H_a1_a2*H_a1_p1*H_a2_a3*H_a3_p2*H_p1_p1/60 -
103*H_a1_a2*H_a1_p1*H_a2_a3*H_a3_p2*H_p1_p2/18 + H_a1_a2*H_a1_p1*H_a2_a3*H_a3_p2*H_p2_p2/10 - H_a1_a2*H_a1_p1*H_a2_p1*H_a3_a3*H_p1_p1/24 +
19*H_a1_a2*H_a1_p1*H_a2_p1*H_a3_a3*H_p1_p2/40 - 1063*H_a1_a2*H_a1_p1*H_a2_p1*H_a3_a3*H_p2_p2/180 - 7*H_a1_a2*H_a1_p1*H_a2_p1*H_a3_p1**2/30 -
9*H_a1_a2*H_a1_p1*H_a2_p1*H_a3_p1*H_a3_p2/20 + 1069*H_a1_a2*H_a1_p1*H_a2_p1*H_a3_p2**2/180 + 13*H_a1_a2*H_a1_p1*H_a2_p1*H_p1_p1*T_a3_a3/40 +
H_a1_a2*H_a1_p1*H_a2_p1*H_p1_p2*T_a3_a3/3 - 187*H_a1_a2*H_a1_p1*H_a2_p1*H_p2_p2*T_a3_a3/24 + 2*H_a1_a2*H_a1_p1*H_a2_p2*H_a3_a3*H_p1_p1/15 +
463*H_a1_a2*H_a1_p1*H_a2_p2*H_a3_a3*H_p1_p2/72 + 2*H_a1_a2*H_a1_p1*H_a2_p2*H_a3_a3*H_p2_p2/15 - 19*H_a1_a2*H_a1_p1*H_a2_p2*H_a3_p1**2/60 -
1279*H_a1_a2*H_a1_p1*H_a2_p2*H_a3_p1*H_a3_p2/180 - 19*H_a1_a2*H_a1_p1*H_a2_p2*H_a3_p2**2/60 + 25*H_a1_a2*H_a1_p1*H_a2_p2*H_p1_p1*T_a3_a3/24 +
61*H_a1_a2*H_a1_p1*H_a2_p2*H_p1_p2*T_a3_a3/6 + 25*H_a1_a2*H_a1_p1*H_a2_p2*H_p2_p2*T_a3_a3/24 - H_a1_a2*H_a1_p1*H_a3_p1*H_p1_p1*T_a2_a3/15 +
H_a1_a2*H_a1_p1*H_a3_p1*H_p1_p2*T_a2_a3/3 + 17*H_a1_a2*H_a1_p1*H_a3_p1*H_p2_p2*T_a2_a3/2 - 5*H_a1_a2*H_a1_p1*H_a3_p2*H_p1_p1*T_a2_a3/6 -
25*H_a1_a2*H_a1_p1*H_a3_p2*H_p1_p2*T_a2_a3/3 + 7*H_a1_a2*H_a1_p1*H_a3_p2*H_p2_p2*T_a2_a3/12 + H_a1_a2*H_a1_p2*H_a2_a3*H_a3_p1*H_p1_p1/3 -
973*H_a1_a2*H_a1_p2*H_a2_a3*H_a3_p1*H_p1_p2/180 + 7*H_a1_a2*H_a1_p2*H_a2_a3*H_a3_p1*H_p2_p2/30 + 557*H_a1_a2*H_a1_p2*H_a2_a3*H_a3_p2*H_p1_p1/90 +
```

```
9*H_a1_a2*H_a1_p2*H_a2_a3*H_a3_p2*H_p1_p2/20 + 17*H_a1_a2*H_a1_p2*H_a2_a3*H_a3_p2*H_p2_p2/20 - 7*H_a1_a2*H_a1_p2*H_a2_p1*H_a3_a3*H_p1_p1/120 +
521*H_a1_a2*H_a1_p2*H_a2_p1*H_a3_a3*H_p1_p2/90 - 7*H_a1_a2*H_a1_p2*H_a2_p1*H_a3_a3*H_p2_p2/120 - 7*H_a1_a2*H_a1_p2*H_a2_p1*H_a3_p1**2/60 -
56*H_a1_a2*H_a1_p2*H_a2_p1*H_a3_p1*H_a3_p2/9 - 7*H_a1_a2*H_a1_p2*H_a2_p1*H_a3_p2**2/60 + 7*H_a1_a2*H_a1_p2*H_a2_p1*H_p1_p1*T_a3_a3/24 +
91*H_a1_a2*H_a1_p2*H_a2_p1*H_p1_p2*T_a3_a3/12 + 7*H_a1_a2*H_a1_p2*H_a2_p1*H_p2_p2*T_a3_a3/24 - 1063*H_a1_a2*H_a1_p2*H_a2_p2*H_a3_a3*H_p1_p1/180 +
19*H_a1_a2*H_a1_p2*H_a2_p2*H_a3_a3*H_p1_p2/40 - H_a1_a2*H_a1_p2*H_a2_p2*H_a3_a3*H_p2_p2/24 + 1069*H_a1_a2*H_a1_p2*H_a2_p2*H_a3_p1**2/180 -
9*H_a1_a2*H_a1_p2*H_a2_p2*H_a3_p1*H_a3_p2/20 - 7*H_a1_a2*H_a1_p2*H_a2_p2*H_a3_p2**2/30 - 187*H_a1_a2*H_a1_p2*H_a2_p2*H_p1_p1*T_a3_a3/24 +
H_a1_a2*H_a1_p2*H_a2_p2*H_p1_p2*T_a3_a3/3 + 13*H_a1_a2*H_a1_p2*H_a2_p2*H_p2_p2*T_a3_a3/40 + H_a1_a2*H_a1_p2*H_a3_p1*H_p1_p1*T_a2_a3/3 -
15*H_a1_a2*H_a1_p2*H_a3_p1*H_p1_p2*T_a2_a3/2 - H_a1_a2*H_a1_p2*H_a3_p1*H_p2_p2*T_a2_a3/3 + 15*H_a1_a2*H_a1_p2*H_a3_p2*H_p1_p1*T_a2_a3/2 +
H_a1_a2*H_a1_p2*H_a3_p2*H_p1_p2*T_a2_a3/6 + 7*H_a1_a2*H_a1_p2*H_a3_p2*H_p2_p2*T_a2_a3/12 - H_a1_a2*H_a2_a3*H_p1_p1**2*T_a1_a3/3 -
2*H_a1_a2*H_a2_a3*H_p1_p1*H_p1_p2*T_a1_a3/3 - 83*H_a1_a2*H_a2_a3*H_p1_p1*H_p2_p2*T_a1_a3/12 + 35*H_a1_a2*H_a2_a3*H_p1_p2**2*T_a1_a3/6 -
3*H_a1_a2*H_a2_a3*H_p1_p2*H_p2_p2*T_a1_a3/4 - 4*H_a1_a2*H_a2_a3*H_p2_p2**2*T_a1_a3/15 + 7*H_a1_a2*H_a2_p1*H_a3_p1*H_p1_p1*T_a1_a3/12 +
H_a1_a2*H_a2_p1*H_a3_p1*H_p1_p2*T_a1_a3/6 + 15*H_a1_a2*H_a2_p1*H_a3_p1*H_p2_p2*T_a1_a3/2 - H_a1_a2*H_a2_p1*H_a3_p2*H_p1_p1*T_a1_a3/3 -
15*H_a1_a2*H_a2_p1*H_a3_p2*H_p1_p2*T_a1_a3/2 + H_a1_a2*H_a2_p1*H_a3_p2*H_p2_p2*T_a1_a3/3 + 7*H_a1_a2*H_a2_p2*H_a3_p1*H_p1_p1*T_a1_a3/12 -
25*H_a1_a2*H_a2_p2*H_a3_p1*H_p1_p2*T_a1_a3/3 - 5*H_a1_a2*H_a2_p2*H_a3_p1*H_p2_p2*T_a1_a3/6 + 17*H_a1_a2*H_a2_p2*H_a3_p2*H_p1_p1*T_a1_a3/2 +
H_a1_a2*H_a2_p2*H_a3_p2*H_p1_p2*T_a1_a3/3 - H_a1_a2*H_a2_p2*H_a3_p2*H_p2_p2*T_a1_a3/15 + 9*H_a1_a2*H_a3_a3*H_p1_p1**2*T_a1_a2/20 +
73*H_a1_a2*H_a3_a3*H_p1_p1*H_p1_p2*T_a1_a2/60 + 183*H_a1_a2*H_a3_a3*H_p1_p1*H_p2_p2*T_a1_a2/20 - 407*H_a1_a2*H_a3_a3*H_p1_p2**2*T_a1_a2/60 +
73*H_a1_a2*H_a3_a3*H_p1_p2*H_p2_p2*T_a1_a2/60 + 9*H_a1_a2*H_a3_a3*H_p2_p2**2*T_a1_a2/20 - 47*H_a1_a2*H_a3_p1**2*H_p1_p1*T_a1_a2/60 +
7*H_a1_a2*H_a3_p1**2*H_p1_p2*T_a1_a2/20 - 128*H_a1_a2*H_a3_p1**2*H_p2_p2*T_a1_a2/15 - 19*H_a1_a2*H_a3_p1*H_a3_p2*H_p1_p1*T_a1_a2/10 +
223*H_a1_a2*H_a3_p1*H_a3_p2*H_p1_p2*T_a1_a2/15 - 19*H_a1_a2*H_a3_p1*H_a3_p2*H_p2_p2*T_a1_a2/10 - 128*H_a1_a2*H_a3_p2**2*H_p1_p1*T_a1_a2/15 +
7*H_a1_a2*H_a3_p2**2*H_p1_p2*T_a1_a2/20 - 47*H_a1_a2*H_a3_p2**2*H_p2_p2*T_a1_a2/60 + 7*H_a1_a2*H_p1_p1**2*T_a1_a2*T_a3_a3/24 -
H_a1_a2*H_p1_p1**2*T_a1_a3*T_a2_a3/12 + H_a1_a2*H_p1_p1*H_p1_p2*T_a1_a2*T_a3_a3 - H_a1_a2*H_p1_p1*H_p1_p2*T_a1_a3*T_a2_a3/4 +
65*H_a1_a2*H_p1_p1*H_p2_p2*T_a1_a2*T_a3_a3/6 - 49*H_a1_a2*H_p1_p1*H_p2_p2*T_a1_a3*T_a2_a3/6 - 53*H_a1_a2*H_p1_p2**2*T_a1_a2*T_a3_a3/6 +
23*H_a1_a2*H_p1_p2**2*T_a1_a3*T_a2_a3/3 + H_a1_a2*H_p1_p2*H_p2_p2*T_a1_a2*T_a3_a3 - H_a1_a2*H_p1_p2*H_p2_p2*T_a1_a3*T_a2_a3/4 +
7*H_a1_a2*H_p2_p2**2*T_a1_a2*T_a3_a3/24 - H_a1_a2*H_p2_p2**2*T_a1_a3*T_a2_a3/12 + 103*H_a1_a3**2*H_a2_a2*H_p1_p1**2/240 +
103*H_a1_a3**2*H_a2_a2*H_p1_p1*H_p1_p2/120 + 2141*H_a1_a3**2*H_a2_a2*H_p1_p1*H_p2_p2/720 - 787*H_a1_a3**2*H_a2_a2*H_p1_p2**2/360 +
43*H_a1_a3**2*H_a2_a2*H_p1_p2*H_p2_p2/120 + 5*H_a1_a3**2*H_a2_a2*H_p2_p2**2/16 - 31*H_a1_a3**2*H_a2_p1**2*H_p1_p1/60 -
31*H_a1_a3**2*H_a2_p1**2*H_p1_p2/60 - 587*H_a1_a3**2*H_a2_p1**2*H_p2_p2/180 - 31*H_a1_a3**2*H_a2_p1*H_a2_p2*H_p1_p1/60 +
883*H_a1_a3**2*H_a2_p1*H_a2_p2*H_p1_p2/180 - 11*H_a1_a3**2*H_a2_p1*H_a2_p2*H_p2_p2/20 - 115*H_a1_a3**2*H_a2_p2**2*H_p1_p1/36 -
29*H_a1_a3**2*H_a2_p2**2*H_p1_p2/60 - 31*H_a1_a3**2*H_a2_p2**2*H_p2_p2/60 + 41*H_a1_a3**2*H_p1_p1**2*T_a2_a2/120 +
41*H_a1_a3**2*H_p1_p1*H_p1_p2*T_a2_a2/60 + 19*H_a1_a3**2*H_p1_p1*H_p2_p2*T_a2_a2/5 - 131*H_a1_a3**2*H_p1_p2**2*T_a2_a2/40 +
11*H_a1_a3**2*H_p1_p2*H_p2_p2*T_a2_a2/60 + 7*H_a1_a3**2*H_p2_p2**2*T_a2_a2/30 - 31*H_a1_a3*H_a1_p1*H_a2_a2*H_a3_p1*H_p1_p1/40 -
31*H_a1_a3*H_a1_p1*H_a2_a2*H_a3_p1*H_p1_p2/40 - 271*H_a1_a3*H_a1_p1*H_a2_a2*H_a3_p1*H_p2_p2/45 - 31*H_a1_a3*H_a1_p1*H_a2_a2*H_a3_p2*H_p1_p1/120 +
1009*H_a1_a3*H_a1_p1*H_a2_a2*H_a3_p2*H_p1_p2/180 - 7*H_a1_a3*H_a1_p1*H_a2_a2*H_a3_p2*H_p2_p2/40 + 17*H_a1_a3*H_a1_p1*H_a2_a3*H_a2_p1*H_p1_p1/20 +
5*H_a1_a3*H_a1_p1*H_a2_a3*H_a2_p1*H_p1_p2/4 + 593*H_a1_a3*H_a1_p1*H_a2_a3*H_a2_p1*H_p2_p2/90 + 3*H_a1_a3*H_a1_p1*H_a2_a3*H_a2_p2*H_p1_p1/4 -
841*H_a1_a3*H_a1_p1*H_a2_a3*H_a2_p2*H_p1_p2/180 + 3*H_a1_a3*H_a1_p1*H_a2_a3*H_a2_p2*H_p2_p2/4 - 3*H_a1_a3*H_a1_p1*H_a2_p1**2*H_a3_p1/20 -
19*H_a1_a3*H_a1_p1*H_a2_p1**2*H_a3_p2/60 - 3*H_a1_a3*H_a1_p1*H_a2_p1*H_a2_p2*H_a3_p1/20 - 292*H_a1_a3*H_a1_p1*H_a2_p1*H_a2_p2*H_a3_p2/45 +
13*H_a1_a3*H_a1_p1*H_a2_p1*H_p1_p1*T_a2_a3/30 + 4*H_a1_a3*H_a1_p1*H_a2_p1*H_p1_p2*T_a2_a3/3 + 53*H_a1_a3*H_a1_p1*H_a2_p1*H_p2_p2*T_a2_a3/6 +
221*H_a1_a3*H_a1_p1*H_a2_p2**2*H_a3_p1/36 + H_a1_a3*H_a1_p1*H_a2_p2**2*H_a3_p2/12 - H_a1_a3*H_a1_p1*H_a2_p2*H_p1_p1*T_a2_a3/6 -
17*H_a1_a3*H_a1_p1*H_a2_p2*H_p1_p2*T_a2_a3/2 - 31*H_a1_a3*H_a1_p1*H_a3_p1*H_p1_p1*T_a2_a2/60 - 31*H_a1_a3*H_a1_p1*H_a3_p1*H_p1_p2*T_a2_a2/60 -
44*H_a1_a3*H_a1_p1*H_a3_p1*H_p2_p2*T_a2_a2/5 + 17*H_a1_a3*H_a1_p1*H_a3_p2*H_p1_p1*T_a2_a2/60 + 42*H_a1_a3*H_a1_p1*H_a3_p2*H_p1_p2*T_a2_a2/5 -
43*H_a1_a3*H_a1_p1*H_a3_p2*H_p2_p2*T_a2_a2/60 - 31*H_a1_a3*H_a1_p2*H_a2_a2*H_a3_p1*H_p1_p1/60 + 385*H_a1_a3*H_a1_p2*H_a2_a2*H_a3_p1*H_p1_p2/72 +
H_a1_a3*H_a1_p2*H_a2_a2*H_a3_p1*H_p2_p2/60 - 289*H_a1_a3*H_a1_p2*H_a2_a2*H_a3_p2*H_p1_p1/45 - 67*H_a1_a3*H_a1_p2*H_a2_a2*H_a3_p2*H_p1_p2/120 -
H_a1_a3*H_a1_p2*H_a2_a2*H_a3_p2*H_p2_p2/24 + 37*H_a1_a3*H_a1_p2*H_a2_a3*H_a2_p1*H_p1_p1/60 - 208*H_a1_a3*H_a1_p2*H_a2_a3*H_a2_p1*H_p1_p2/45 +
37*H_a1_a3*H_a1_p2*H_a2_a3*H_a2_p1*H_p2_p2/60 + 593*H_a1_a3*H_a1_p2*H_a2_a3*H_a2_p2*H_p1_p1/90 + 5*H_a1_a3*H_a1_p2*H_a2_a3*H_a2_p2*H_p1_p2/4 +
17*H_a1_a3*H_a1_p2*H_a2_a3*H_a2_p2*H_p2_p2/20 + H_a1_a3*H_a1_p2*H_a2_p1**2*H_a3_p1/6 + 277*H_a1_a3*H_a1_p2*H_a2_p1**2*H_a3_p2/45 -
1081*H_a1_a3*H_a1_p2*H_a2_p1*H_a2_p2*H_a3_p1/180 - H_a1_a3*H_a1_p2*H_a2_p1*H_a2_p2*H_a3_p2/60 + 5*H_a1_a3*H_a1_p2*H_a2_p1*H_p1_p1*T_a2_a3/12 -
35*H_a1_a3*H_a1_p2*H_a2_p1*H_p1_p2*T_a2_a3/6 + 11*H_a1_a3*H_a1_p2*H_a2_p1*H_p2_p2*T_a2_a3/12 - 7*H_a1_a3*H_a1_p2*H_a2_p2**2*H_a3_p1/60 -
7*H_a1_a3*H_a1_p2*H_a2_p2**2*H_a3_p2/30 + 95*H_a1_a3*H_a1_p2*H_a2_p2*H_p1_p1*T_a2_a3/12 + H_a1_a3*H_a1_p2*H_a2_p2*H_p1_p2*T_a2_a3 +
7*H_a1_a3*H_a1_p2*H_a2_p2*H_p2_p2*T_a2_a3/12 - 4*H_a1_a3*H_a1_p2*H_a3_p1*H_p1_p1*T_a2_a2/5 + 439*H_a1_a3*H_a1_p2*H_a3_p1*H_p1_p2*T_a2_a2/60 +
H_a1_a3*H_a1_p2*H_a3_p1*H_p2_p2*T_a2_a2/30 - 39*H_a1_a3*H_a1_p2*H_a3_p2*H_p1_p1*T_a2_a2/5 + 19*H_a1_a3*H_a1_p2*H_a3_p2*H_p1_p2*T_a2_a2/60 +
13*H_a1_a3*H_a1_p2*H_a3_p2*H_p2_p2*T_a2_a2/40 + 5*H_a1_a3*H_a2_a2*H_p1_p1**2*T_a1_a3/6 + 5*H_a1_a3*H_a2_a2*H_p1_p1*H_p1_p2*T_a1_a3/3 +
53*H_a1_a3*H_a2_a2*H_p1_p1*H_p2_p2*T_a1_a3/6 - 89*H_a1_a3*H_a2_a2*H_p1_p2**2*T_a1_a3/12 + 7*H_a1_a3*H_a2_a2*H_p1_p2*H_p2_p2*T_a1_a3/12 +
9*H_a1_a3*H_a2_a2*H_p2_p2**2*T_a1_a3/20 - 4*H_a1_a3*H_a2_a3*H_p1_p1**2*T_a1_a2/15 - 19*H_a1_a3*H_a2_a3*H_p1_p1*H_p1_p2*T_a1_a2/60 -
67*H_a1_a3*H_a2_a3*H_p1_p1*H_p2_p2*T_a1_a2/10 + 94*H_a1_a3*H_a2_a3*H_p1_p2**2*T_a1_a2/15 - 19*H_a1_a3*H_a2_a3*H_p1_p2*H_p2_p2*T_a1_a2/60 -
4*H_a1_a3*H_a2_a3*H_p2_p2**2*T_a1_a2/15 - 2*H_a1_a3*H_a2_p1**2*H_p1_p1*T_a1_a3/3 - 2*H_a1_a3*H_a2_p1**2*H_p1_p2*T_a1_a3/3 -
89*H_a1_a3*H_a2_p1**2*H_p2_p2*T_a1_a3/12 - 2*H_a1_a3*H_a2_p1*H_a2_p2*H_p1_p1*T_a1_a3/3 + 89*H_a1_a3*H_a2_p1*H_a2_p2*H_p1_p2*T_a1_a3/6 +
H_a1_a3*H_a2_p1*H_a2_p2*H_p2_p2*T_a1_a3/3 + 13*H_a1_a3*H_a2_p1*H_a3_p1*H_p1_p1*T_a1_a2/30 - 7*H_a1_a3*H_a2_p1*H_a3_p1*H_p1_p2*T_a1_a2/15 +
119*H_a1_a3*H_a2_p1*H_a3_p1*H_p2_p2*T_a1_a2/15 + H_a1_a3*H_a2_p1*H_a3_p2*H_p1_p1*T_a1_a2/60 - 107*H_a1_a3*H_a2_p1*H_a3_p2*H_p1_p2*T_a1_a2/15 +
23*H_a1_a3*H_a2_p1*H_a3_p2*H_p2_p2*T_a1_a2/30 - 29*H_a1_a3*H_a2_p2**2*H_p1_p1*T_a1_a3/3 - 23*H_a1_a3*H_a2_p2**2*H_p1_p2*T_a1_a3/12 -
47*H_a1_a3*H_a2_p2**2*H_p2_p2*T_a1_a3/60 + 3*H_a1_a3*H_a2_p2*H_a3_p1*H_p1_p1*T_a1_a2/5 - 259*H_a1_a3*H_a2_p2*H_a3_p1*H_p1_p2*T_a1_a2/30 -
2*H_a1_a3*H_a2_p2*H_a3_p1*H_p2_p2*T_a1_a2/5 + 81*H_a1_a3*H_a2_p2*H_a3_p2*H_p1_p1*T_a1_a2/10 - 7*H_a1_a3*H_a2_p2*H_a3_p2*H_p1_p2*T_a1_a2/15 -
H_a1_a3*H_a2_p2*H_a3_p2*H_p2_p2*T_a1_a2/15 - H_a1_a3*H_p1_p1**2*T_a1_a2*T_a2_a3/6 + 7*H_a1_a3*H_p1_p1**2*T_a1_a3*T_a2_a2/12 -
H_a1_a3*H_p1_p1*H_p1_p2*T_a1_a2*T_a2_a3/3 + 7*H_a1_a3*H_p1_p1*H_p1_p2*T_a1_a3*T_a2_a2/6 - 97*H_a1_a3*H_p1_p1*H_p2_p2*T_a1_a2*T_a2_a3/12 +
125*H_a1_a3*H_p1_p1*H_p2_p2*T_a1_a3*T_a2_a2/12 + 47*H_a1_a3*H_p1_p2**2*T_a1_a2*T_a2_a3/6 - 29*H_a1_a3*H_p1_p2**2*T_a1_a3*T_a2_a2/3 -
H_a1_a3*H_p1_p2*H_p2_p2*T_a1_a2*T_a2_a3/12 + H_a1_a3*H_p1_p2*H_p2_p2*T_a1_a3*T_a2_a2/6 - H_a1_a3*H_p2_p2**2*T_a1_a2*T_a2_a3/12 +
7*H_a1_a3*H_p2_p2**2*T_a1_a3*T_a2_a2/24 + 23*H_a1_p1**2*H_a2_a2*H_a3_a3*H_p1_p1/240 + 11*H_a1_p1**2*H_a2_a2*H_a3_a3*H_p1_p2/240 +
2201*H_a1_p1**2*H_a2_a2*H_a3_a3*H_p2_p2/720 + 13*H_a1_p1**2*H_a2_a2*H_a3_p1**2/60 - H_a1_p1**2*H_a2_a2*H_a3_p1*H_a3_p2/20 -
143*H_a1_p1**2*H_a2_a2*H_a3_p2**2/45 + 19*H_a1_p1**2*H_a2_a2*H_p1_p1*T_a3_a3/80 + 15*H_a1_p1**2*H_a2_a2*H_p1_p2*T_a3_a3/16 +
23*H_a1_p1**2*H_a2_a2*H_p2_p2*T_a3_a3/4 - 31*H_a1_p1**2*H_a2_a3**2*H_p1_p1/60 - 29*H_a1_p1**2*H_a2_a3**2*H_p1_p2/60 -
115*H_a1_p1**2*H_a2_a3**2*H_p2_p2/36 - 7*H_a1_p1**2*H_a2_a3*H_a2_p1*H_a3_p1/30 - 7*H_a1_p1**2*H_a2_a3*H_a2_p1*H_a3_p2/60 +
H_a1_p1**2*H_a2_a3*H_a2_p2*H_a3_p1/12 + 221*H_a1_p1**2*H_a2_a3*H_a2_p2*H_a3_p2/36 - 47*H_a1_p1**2*H_a2_a3*H_p1_p1*T_a2_a3/60 -
23*H_a1_p1**2*H_a2_a3*H_p1_p2*T_a2_a3/12 - 29*H_a1_p1**2*H_a2_a3*H_p2_p2*T_a2_a3/3 + 5*H_a1_p1**2*H_a2_p1**2*H_a3_a3/24 -
H_a1_p1**2*H_a2_p1**2*T_a3_a3/40 + 7*H_a1_p1**2*H_a2_p1*H_a2_p2*H_a3_a3/30 - 19*H_a1_p1**2*H_a2_p1*H_a2_p2*T_a3_a3/24 -
3*H_a1_p1**2*H_a2_p1*H_a3_p1*T_a2_a3/20 + H_a1_p1**2*H_a2_p1*H_a3_p2*T_a2_a3/2 - 227*H_a1_p1**2*H_a2_p2**2*H_a3_a3/72 -
37*H_a1_p1**2*H_a2_p2**2*T_a3_a3/6 + 5*H_a1_p1**2*H_a2_p2*H_a3_p1*T_a2_a3/3 + 65*H_a1_p1**2*H_a2_p2*H_a3_p2*T_a2_a3/6 -
H_a1_p1**2*H_a3_a3*H_p1_p1*T_a2_a2/40 + 149*H_a1_p1**2*H_a3_a3*H_p1_p2*T_a2_a2/240 + 661*H_a1_p1**2*H_a3_a3*H_p2_p2*T_a2_a2/120 +
H_a1_p1**2*H_a3_p1**2*T_a2_a2/5 - 161*H_a1_p1**2*H_a3_p1*H_a3_p2*T_a2_a2/120 - 27*H_a1_p1**2*H_a3_p2**2*T_a2_a2/5 +
H_a1_p1**2*H_p1_p1*T_a2_a2*T_a3_a3/12 - H_a1_p1**2*H_p1_p1*T_a2_a3**2/3 + 71*H_a1_p1**2*H_p1_p2*T_a2_a2*T_a3_a3/48 - 3*H_a1_p1**2*H_p1_p2*T_a2_a3**2/2
+ 49*H_a1_p1**2*H_p2_p2*T_a2_a2*T_a3_a3/6 - 157*H_a1_p1**2*H_p2_p2*T_a2_a3**2/24 + 3*H_a1_p1*H_a1_p2*H_a2_a2*H_a3_a3*H_p1_p1/40 -
227*H_a1_p1*H_a1_p2*H_a2_a2*H_a3_a3*H_p1_p2/36 - 17*H_a1_p1*H_a1_p2*H_a2_a2*H_a3_a3*H_p2_p2/60 + 29*H_a1_p1*H_a1_p2*H_a2_a2*H_a3_p1**2/60 +
2561*H_a1_p1*H_a1_p2*H_a2_a2*H_a3_p1*H_a3_p2/360 + 11*H_a1_p1*H_a1_p2*H_a2_a2*H_a3_p2**2/60 - H_a1_p1*H_a1_p2*H_a2_a2*H_p1_p1*T_a3_a3/2 -
233*H_a1_p1*H_a1_p2*H_a2_a2*H_p1_p2*T_a3_a3/24 - 11*H_a1_p1*H_a1_p2*H_a2_a3**2*H_p1_p1/20 + 883*H_a1_p1*H_a1_p2*H_a2_a3**2*H_p1_p2/180 -
31*H_a1_p1*H_a1_p2*H_a2_a3**2*H_p2_p2/60 - H_a1_p1*H_a1_p2*H_a2_a3*H_a2_p1*H_a3_p1/60 - 1081*H_a1_p1*H_a1_p2*H_a2_a3*H_a2_p1*H_a3_p2/180 -
292*H_a1_p1*H_a1_p2*H_a2_a3*H_a2_p2*H_a3_p1/45 - 3*H_a1_p1*H_a1_p2*H_a2_a3*H_a2_p2*H_a3_p2/20 + H_a1_p1*H_a1_p2*H_a2_a3*H_p1_p1*T_a2_a3/3 +
```

```
89*H_a1_p1*H_a1_p2*H_a2_a3*H_p1_p2*T_a2_a3/6 - 2*H_a1_p1*H_a1_p2*H_a2_a3*H_p2_p2*T_a2_a3/3 - 5*H_a1_p1*H_a1_p2*H_a2_p1**2*H_a3_a3/24 -
H_a1_p1*H_a1_p2*H_a2_p1**2*T_a3_a3/12 + 2279*H_a1_p1*H_a1_p2*H_a2_p1*H_a2_p2*H_a3_a3/360 + 103*H_a1_p1*H_a1_p2*H_a2_p1*H_a2_p2*T_a3_a3/12 -
7*H_a1_p1*H_a1_p2*H_a2_p1*H_a3_p1*T_a2_a3/6 - 26*H_a1_p1*H_a1_p2*H_a2_p1*H_a3_p2*T_a2_a3/3 + 7*H_a1_p1*H_a1_p2*H_a2_p2**2*H_a3_a3/30 -
19*H_a1_p1*H_a1_p2*H_a2_p2**2*T_a3_a3/24 - 9*H_a1_p1*H_a1_p2*H_a2_p2*H_a3_p1*T_a2_a3 - H_a1_p1*H_a1_p2*H_a2_p2*H_a3_p2*T_a2_a3/3 -
113*H_a1_p1*H_a1_p2*H_a3_a3*H_p1_p1*T_a2_a2/120 - 407*H_a1_p1*H_a1_p2*H_a3_a3*H_p1_p2*T_a2_a2/40 - 29*H_a1_p1*H_a1_p2*H_a3_a3*H_p2_p2*T_a2_a2/60 +
209*H_a1_p1*H_a1_p2*H_a3_p1**2*T_a2_a2/120 + 307*H_a1_p1*H_a1_p2*H_a3_p1*H_a3_p2*T_a2_a2/30 + 89*H_a1_p1*H_a1_p2*H_a3_p2**2*T_a2_a2/120 -
73*H_a1_p1*H_a1_p2*H_p1_p1*T_a2_a2*T_a3_a3/48 + 7*H_a1_p1*H_a1_p2*H_p1_p1*T_a2_a3**2/8 - 163*H_a1_p1*H_a1_p2*H_p1_p2*T_a2_a2*T_a3_a3/12 +
119*H_a1_p1*H_a1_p2*H_p1_p2*T_a2_a3**2/12 - 5*H_a1_p1*H_a1_p2*H_p2_p2*T_a2_a2*T_a3_a3/24 - H_a1_p1*H_a1_p2*H_p2_p2*T_a2_a3**2/6 -
2*H_a1_p1*H_a2_a2*H_a3_p1*H_p1_p1*T_a1_a3/3 - 2*H_a1_p1*H_a2_a2*H_a3_p1*H_p1_p2*T_a1_a3/3 - 121*H_a1_p1*H_a2_a2*H_a3_p1*H_p2_p2*T_a1_a3/12 +
H_a1_p1*H_a2_a2*H_a3_p2*H_p1_p1*T_a1_a3/3 + 29*H_a1_p1*H_a2_a2*H_a3_p2*H_p1_p2*T_a1_a3/3 - 5*H_a1_p1*H_a2_a2*H_a3_p2*H_p2_p2*T_a1_a3/6 +
7*H_a1_p1*H_a2_a3*H_a2_p1*H_p1_p1*T_a1_a3/12 + H_a1_p1*H_a2_a3*H_a2_p1*H_p1_p2*T_a1_a3 + 95*H_a1_p1*H_a2_a3*H_a2_p1*H_p2_p2*T_a1_a3/12 -
17*H_a1_p1*H_a2_a3*H_a2_p2*H_p1_p2*T_a1_a3/2 - H_a1_p1*H_a2_a3*H_a2_p2*H_p2_p2*T_a1_a3/6 - H_a1_p1*H_a2_a3*H_a3_p1*H_p1_p1*T_a1_a2/15 -
7*H_a1_p1*H_a2_a3*H_a3_p1*H_p1_p2*T_a1_a2/15 + 81*H_a1_p1*H_a2_a3*H_a3_p1*H_p2_p2*T_a1_a2/10 - 2*H_a1_p1*H_a2_a3*H_a3_p2*H_p1_p1*T_a1_a2/5 -
259*H_a1_p1*H_a2_a3*H_a3_p2*H_p1_p2*T_a1_a2/30 + 3*H_a1_p1*H_a2_a3*H_a3_p2*H_p2_p2*T_a1_a2/5 - H_a1_p1*H_a2_p1**2*H_a3_p1*T_a1_a3/3 -
H_a1_p1*H_a2_p1**2*H_a3_p2*T_a1_a3/3 - H_a1_p1*H_a2_p1*H_a2_p2*H_a3_p1*T_a1_a3/3 - 9*H_a1_p1*H_a2_p1*H_a2_p2*H_a3_p2*T_a1_a3 +
23*H_a1_p1*H_a2_p1*H_a3_a3*H_p1_p1*T_a1_a2/60 + 41*H_a1_p1*H_a2_p1*H_a3_a3*H_p1_p2*T_a1_a2/60 - 179*H_a1_p1*H_a2_p1*H_a3_a3*H_p2_p2*T_a1_a2/20 -
3*H_a1_p1*H_a2_p1*H_a3_p1**2*T_a1_a2/20 + 13*H_a1_p1*H_a2_p1*H_a3_p1*H_a3_p2*T_a1_a2/15 + 511*H_a1_p1*H_a2_p1*H_a3_p2**2*T_a1_a2/60 +
17*H_a1_p1*H_a2_p1*H_p1_p1*T_a1_a2*T_a3_a3/24 + H_a1_p1*H_a2_p1*H_p1_p1*T_a1_a3*T_a2_a3/12 + H_a1_p1*H_a2_p1*H_p1_p2*T_a1_a2*T_a3_a3/2 +
H_a1_p1*H_a2_p1*H_p1_p2*T_a1_a3*T_a2_a3 - 87*H_a1_p1*H_a2_p1*H_p2_p2*T_a1_a2*T_a3_a3/8 + 121*H_a1_p1*H_a2_p1*H_p2_p2*T_a1_a3*T_a2_a3/12 +
65*H_a1_p1*H_a2_p2**2*H_a3_p1*T_a1_a3/6 + 5*H_a1_p1*H_a2_p2**2*H_a3_p2*T_a1_a3/3 + 73*H_a1_p1*H_a2_p2*H_a3_a3*H_p1_p1*T_a1_a2/60 +
721*H_a1_p1*H_a2_p2*H_a3_a3*H_p1_p2*T_a1_a2/60 + 73*H_a1_p1*H_a2_p2*H_a3_a3*H_p2_p2*T_a1_a2/60 - 109*H_a1_p1*H_a2_p2*H_a3_p1**2*T_a1_a2/60 -
172*H_a1_p1*H_a2_p2*H_a3_p1*H_a3_p2*T_a1_a2/15 - 109*H_a1_p1*H_a2_p2*H_a3_p2**2*T_a1_a2/60 + 17*H_a1_p1*H_a2_p2*H_p1_p1*T_a1_a2*T_a3_a3/8 -
11*H_a1_p1*H_a2_p2*H_p1_p1*T_a1_a3*T_a2_a3/12 + 63*H_a1_p1*H_a2_p2*H_p1_p2*T_a1_a2*T_a3_a3/4 - 37*H_a1_p1*H_a2_p2*H_p1_p2*T_a1_a3*T_a2_a3/3 +
17*H_a1_p1*H_a2_p2*H_p2_p2*T_a1_a2*T_a3_a3/8 - 11*H_a1_p1*H_a2_p2*H_p2_p2*T_a1_a3*T_a2_a3/12 - H_a1_p1*H_a3_p1*H_p1_p1*T_a1_a2*T_a2_a3/3 -
H_a1_p1*H_a3_p1*H_p1_p1*T_a1_a3*T_a2_a2/3 - H_a1_p1*H_a3_p1*H_p1_p2*T_a1_a2*T_a2_a3/3 - H_a1_p1*H_a3_p1*H_p1_p2*T_a1_a3*T_a2_a2/3 +
32*H_a1_p1*H_a3_p1*H_p2_p2*T_a1_a2*T_a2_a3/3 - 307*H_a1_p1*H_a3_p1*H_p2_p2*T_a1_a3*T_a2_a2/24 - 4*H_a1_p1*H_a3_p2*H_p1_p1*T_a1_a2*T_a2_a3/3 +
5*H_a1_p1*H_a3_p2*H_p1_p1*T_a1_a3*T_a2_a2/6 - 34*H_a1_p1*H_a3_p2*H_p1_p2*T_a1_a2*T_a2_a3/3 + 149*H_a1_p1*H_a3_p2*H_p1_p2*T_a1_a3*T_a2_a2/12 +
H_a1_p1*H_a3_p2*H_p2_p2*T_a1_a2*T_a2_a3 - 17*H_a1_p1*H_a3_p2*H_p2_p2*T_a1_a3*T_a2_a2/12 + 1111*H_a1_p2**2*H_a2_a2*H_a3_a3*H_p1_p1/360 -
17*H_a1_p2**2*H_a2_a2*H_a3_a3*H_p1_p2/60 - 17*H_a1_p2**2*H_a2_a2*H_a3_a3*H_p2_p2/60 - 131*H_a1_p2**2*H_a2_a2*H_a3_p1**2/45 +
5*H_a1_p2**2*H_a2_a2*H_a3_p1*H_a3_p2/8 + 5*H_a1_p2**2*H_a2_a2*H_a3_p2**2/24 + 33*H_a1_p2**2*H_a2_a2*H_p1_p1*T_a3_a3/8 -
H_a1_p2**2*H_a2_a2*H_p1_p2*T_a3_a3/4 - 29*H_a1_p2**2*H_a2_a2*H_p2_p2*T_a3_a3/120 - 587*H_a1_p2**2*H_a2_a3**2*H_p1_p1/180 -
31*H_a1_p2**2*H_a2_a3**2*H_p1_p2/60 - 31*H_a1_p2**2*H_a2_a3**2*H_p2_p2/60 + 277*H_a1_p2**2*H_a2_a3*H_a2_p1*H_a3_p1/45 +
H_a1_p2**2*H_a2_a3*H_a2_p1*H_a3_p2/6 - 19*H_a1_p2**2*H_a2_a3*H_a2_p2*H_a3_p1/60 - 3*H_a1_p2**2*H_a2_a3*H_a2_p2*H_a3_p2/20 -
89*H_a1_p2**2*H_a2_a3*H_p1_p1*T_a2_a3/12 - 2*H_a1_p2**2*H_a2_a3*H_p1_p2*T_a2_a3/3 - 2*H_a1_p2**2*H_a2_a3*H_p2_p2*T_a2_a3/3 -
599*H_a1_p2**2*H_a2_p1**2*H_a3_a3/180 - 47*H_a1_p2**2*H_a2_p1**2*T_a3_a3/12 - 5*H_a1_p2**2*H_a2_p1*H_a2_p2*H_a3_a3/24 -
H_a1_p2**2*H_a2_p1*H_a2_p2*T_a3_a3/12 + 15*H_a1_p2**2*H_a2_p1*H_a3_p1*T_a2_a3/2 + 5*H_a1_p2**2*H_a2_p2**2*H_a3_a3/24 -
H_a1_p2**2*H_a2_p2**2*T_a3_a3/40 - H_a1_p2**2*H_a2_p2*H_a3_p1*T_a2_a3/3 - H_a1_p2**2*H_a2_p2*H_a3_p2*T_a2_a3/3 +
62*H_a1_p2**2*H_a3_a3*H_p1_p1*T_a2_a2/15 - 7*H_a1_p2**2*H_a3_a3*H_p1_p2*T_a2_a2/30 - 29*H_a1_p2**2*H_a3_a3*H_p2_p2*T_a2_a2/120 -
463*H_a1_p2**2*H_a3_p1**2*T_a2_a2/120 + H_a1_p2**2*H_a3_p1*H_a3_p2*T_a2_a2/30 - H_a1_p2**2*H_a3_p2**2*T_a2_a2/40 +
31*H_a1_p2**2*H_p1_p1*T_a2_a2*T_a3_a3/6 - 25*H_a1_p2**2*H_p1_p1*T_a2_a3**2/6 - 5*H_a1_p2**2*H_p1_p2*T_a2_a2*T_a3_a3/24 -
H_a1_p2**2*H_p1_p2*T_a2_a3**2/6 - 5*H_a1_p2**2*H_p2_p2*T_a2_a2*T_a3_a3/24 - H_a1_p2**2*H_p2_p2*T_a2_a3**2/6 - H_a1_p2*H_a2_a2*H_a3_p1*H_p1_p1*T_a1_a3
+ 25*H_a1_p2*H_a2_a2*H_a3_p1*H_p1_p2*T_a1_a3/3 + H_a1_p2*H_a2_a2*H_a3_p1*H_p2_p2*T_a1_a3/12 - 37*H_a1_p2*H_a2_a2*H_a3_p2*H_p1_p1*T_a1_a3/4 +
H_a1_p2*H_a2_a2*H_a3_p2*H_p1_p2*T_a1_a3/12 + 23*H_a1_p2*H_a2_a2*H_a3_p2*H_p2_p2*T_a1_a3/60 + 11*H_a1_p2*H_a2_a3*H_a2_p1*H_p1_p1*T_a1_a3/12 -
35*H_a1_p2*H_a2_a3*H_a2_p1*H_p1_p2*T_a1_a3/6 + 5*H_a1_p2*H_a2_a3*H_a2_p1*H_p2_p2*T_a1_a3/12 + 53*H_a1_p2*H_a2_a3*H_a2_p2*H_p1_p1*T_a1_a3/6 +
4*H_a1_p2*H_a2_a3*H_a2_p2*H_p1_p2*T_a1_a3/3 + 13*H_a1_p2*H_a2_a3*H_a2_p2*H_p2_p2*T_a1_a3/30 + 23*H_a1_p2*H_a2_a3*H_a3_p1*H_p1_p1*T_a1_a2/30 -
107*H_a1_p2*H_a2_a3*H_a3_p1*H_p1_p2*T_a1_a2/15 + H_a1_p2*H_a2_a3*H_a3_p1*H_p2_p2*T_a1_a2/60 + 119*H_a1_p2*H_a2_a3*H_a3_p2*H_p1_p1*T_a1_a2/15 -
7*H_a1_p2*H_a2_a3*H_a3_p2*H_p1_p2*T_a1_a2/15 + 13*H_a1_p2*H_a2_a3*H_a3_p2*H_p2_p2*T_a1_a2/30 + 15*H_a1_p2*H_a2_p1**2*H_a3_p2*T_a1_a3/2 -
26*H_a1_p2*H_a2_p1*H_a2_p2*H_a3_p1*T_a1_a3/3 - 7*H_a1_p2*H_a2_p1*H_a2_p2*H_a3_p2*T_a1_a3/6 + 3*H_a1_p2*H_a2_p1*H_a3_a3*H_p1_p1*T_a1_a2/10 +
177*H_a1_p2*H_a2_p1*H_a3_a3*H_p1_p2*T_a1_a2/20 + 3*H_a1_p2*H_a2_p1*H_a3_a3*H_p2_p2*T_a1_a2/10 - 13*H_a1_p2*H_a2_p1*H_a3_p1**2*T_a1_a2/20 -
44*H_a1_p2*H_a2_p1*H_a3_p1*H_a3_p2*T_a1_a2/5 - 13*H_a1_p2*H_a2_p1*H_a3_p2**2*T_a1_a2/20 + H_a1_p2*H_a2_p1*H_p1_p1*T_a1_a2*T_a3_a3/2 +
5*H_a1_p2*H_a2_p1*H_p1_p1*T_a1_a3*T_a2_a3/12 + 21*H_a1_p2*H_a2_p1*H_p1_p2*T_a1_a2*T_a3_a3/2 - 22*H_a1_p2*H_a2_p1*H_p1_p2*T_a1_a3*T_a2_a3/3 +
H_a1_p2*H_a2_p1*H_p2_p2*T_a1_a2*T_a3_a3/2 + 5*H_a1_p2*H_a2_p1*H_p2_p2*T_a1_a3*T_a2_a3/12 + H_a1_p2*H_a2_p2**2*H_a3_p1*T_a1_a3/2 -
3*H_a1_p2*H_a2_p2**2*H_a3_p2*T_a1_a3/20 - 179*H_a1_p2*H_a2_p2*H_a3_a3*H_p1_p1*T_a1_a2/20 + 41*H_a1_p2*H_a2_p2*H_a3_a3*H_p1_p2*T_a1_a2/60 +
23*H_a1_p2*H_a2_p2*H_a3_a3*H_p2_p2*T_a1_a2/60 + 511*H_a1_p2*H_a2_p2*H_a3_p1**2*T_a1_a2/60 + 13*H_a1_p2*H_a2_p2*H_a3_p1*H_a3_p2*T_a1_a2/15 -
3*H_a1_p2*H_a2_p2*H_a3_p2**2*T_a1_a2/20 - 87*H_a1_p2*H_a2_p2*H_p1_p1*T_a1_a2*T_a3_a3/8 + 121*H_a1_p2*H_a2_p2*H_p1_p1*T_a1_a3*T_a2_a3/12 +
H_a1_p2*H_a2_p2*H_p1_p2*T_a1_a2*T_a3_a3/2 + H_a1_p2*H_a2_p2*H_p1_p2*T_a1_a3*T_a2_a3 + 17*H_a1_p2*H_a2_p2*H_p2_p2*T_a1_a2*T_a3_a3/24 +
H_a1_p2*H_a2_p2*H_p2_p2*T_a1_a3*T_a2_a3/12 + H_a1_p2*H_a3_p1*H_p1_p1*T_a1_a2*T_a2_a3 - 7*H_a1_p2*H_a3_p1*H_p1_p1*T_a1_a3*T_a2_a2/6 -
9*H_a1_p2*H_a3_p1*H_p1_p2*T_a1_a2*T_a2_a3 + 125*H_a1_p2*H_a3_p1*H_p1_p2*T_a1_a3*T_a2_a2/12 - H_a1_p2*H_a3_p1*H_p2_p2*T_a1_a2*T_a2_a3/3 +
5*H_a1_p2*H_a3_p1*H_p2_p2*T_a1_a3*T_a2_a2/24 + 55*H_a1_p2*H_a3_p2*H_p1_p1*T_a1_a2*T_a2_a3/6 - 32*H_a1_p2*H_a3_p2*H_p1_p1*T_a1_a3*T_a2_a2/3 -
5*H_a1_p2*H_a3_p2*H_p1_p2*T_a1_a2*T_a2_a3/6 + 11*H_a1_p2*H_a3_p2*H_p1_p2*T_a1_a3*T_a2_a2/12 + H_a1_p2*H_a3_p2*H_p2_p2*T_a1_a2*T_a2_a3/12 +
17*H_a1_p2*H_a3_p2*H_p2_p2*T_a1_a3*T_a2_a2/24 - 89*H_a2_a2*H_a3_a3*H_p1_p1**2*T_a1_a1/480 - 21*H_a2_a2*H_a3_a3*H_p1_p1*H_p1_p2*T_a1_a1/80 -
637*H_a2_a2*H_a3_a3*H_p1_p1*H_p2_p2*T_a1_a1/120 + 1217*H_a2_a2*H_a3_a3*H_p1_p2**2*T_a1_a1/240 + H_a2_a2*H_a3_a3*H_p1_p2*H_p2_p2*T_a1_a1/20 +
H_a2_a2*H_a3_a3*H_p2_p2**2*T_a1_a1/40 + 19*H_a2_a2*H_a3_p1**2*H_p1_p1*T_a1_a1/80 - 37*H_a2_a2*H_a3_p1**2*H_p1_p2*T_a1_a1/80 +
101*H_a2_a2*H_a3_p1**2*H_p2_p2*T_a1_a1/20 + 39*H_a2_a2*H_a3_p1*H_a3_p2*H_p1_p1*T_a1_a1/40 - 289*H_a2_a2*H_a3_p1*H_a3_p2*H_p1_p2*T_a1_a1/30 +
107*H_a2_a2*H_a3_p1*H_a3_p2*H_p2_p2*T_a1_a1/120 + 389*H_a2_a2*H_a3_p2**2*H_p1_p1*T_a1_a1/80 - 161*H_a2_a2*H_a3_p2**2*H_p1_p2*T_a1_a1/240 -
H_a2_a2*H_a3_p2**2*H_p2_p2*T_a1_a1/40 - 31*H_a2_a2*H_p1_p1**2*T_a1_a1*T_a3_a3/96 + 5*H_a2_a2*H_p1_p1**2*T_a1_a3**2/12 -
31*H_a2_a2*H_p1_p1*H_p1_p2*T_a1_a1*T_a3_a3/48 + 5*H_a2_a2*H_p1_p1*H_p1_p2*T_a1_a3**2/6 - 185*H_a2_a2*H_p1_p1*H_p2_p2*T_a1_a1*T_a3_a3/24 +
95*H_a2_a2*H_p1_p1*H_p2_p2*T_a1_a3**2/16 + 85*H_a2_a2*H_p1_p2**2*T_a1_a1*T_a3_a3/12 - 125*H_a2_a2*H_p1_p2**2*T_a1_a3**2/24 -
29*H_a2_a2*H_p1_p2*H_p2_p2*T_a1_a1*T_a3_a3/96 + 5*H_a2_a2*H_p1_p2*H_p2_p2*T_a1_a3**2/16 - H_a2_a2*H_p2_p2**2*T_a1_a1*T_a3_a3/12 +
5*H_a2_a2*H_p2_p2**2*T_a1_a3**2/24 + 7*H_a2_a3**2*H_p1_p1**2*T_a1_a1/30 + 11*H_a2_a3**2*H_p1_p1*H_p1_p2*T_a1_a1/60 +
19*H_a2_a3**2*H_p1_p1*H_p2_p2*T_a1_a1/5 - 131*H_a2_a3**2*H_p1_p2**2*T_a1_a1/40 + 41*H_a2_a3**2*H_p1_p2*H_p2_p2*T_a1_a1/60 +
41*H_a2_a3**2*H_p2_p2**2*T_a1_a1/120 + 13*H_a2_a3*H_a2_p1*H_a3_p1*H_p1_p1*T_a1_a1/40 + 19*H_a2_a3*H_a2_p1*H_a3_p1*H_p1_p2*T_a1_a1/60 -
39*H_a2_a3*H_a2_p1*H_a3_p1*H_p2_p2*T_a1_a1/5 + H_a2_a3*H_a2_p1*H_a3_p2*H_p1_p1*T_a1_a1/30 + 439*H_a2_a3*H_a2_p1*H_a3_p2*H_p1_p2*T_a1_a1/60 -
4*H_a2_a3*H_a2_p1*H_a3_p2*H_p2_p2*T_a1_a1/5 - 43*H_a2_a3*H_a2_p2*H_a3_p1*H_p1_p1*T_a1_a1/60 + 42*H_a2_a3*H_a2_p2*H_a3_p1*H_p1_p2*T_a1_a1/5 +
17*H_a2_a3*H_a2_p2*H_a3_p1*H_p2_p2*T_a1_a1/60 - 44*H_a2_a3*H_a2_p2*H_a3_p2*H_p1_p1*T_a1_a1/5 - 31*H_a2_a3*H_a2_p2*H_a3_p2*H_p1_p2*T_a1_a1/60 -
31*H_a2_a3*H_a2_p2*H_a3_p2*H_p2_p2*T_a1_a1/60 + 7*H_a2_a3*H_p1_p1**2*T_a1_a1*T_a2_a3/24 - H_a2_a3*H_p1_p1**2*T_a1_a2*T_a1_a3/12 +
H_a2_a3*H_p1_p1*H_p1_p2*T_a1_a1*T_a2_a3/6 - H_a2_a3*H_p1_p1*H_p1_p2*T_a1_a2*T_a1_a3/12 + 125*H_a2_a3*H_p1_p1*H_p2_p2*T_a1_a1*T_a2_a3/12 -
97*H_a2_a3*H_p1_p1*H_p2_p2*T_a1_a2*T_a1_a3/12 - 29*H_a2_a3*H_p1_p2**2*T_a1_a1*T_a2_a3/3 + 47*H_a2_a3*H_p1_p2**2*T_a1_a2*T_a1_a3/6 +
7*H_a2_a3*H_p1_p2*H_p2_p2*T_a1_a1*T_a2_a3/6 - H_a2_a3*H_p1_p2*H_p2_p2*T_a1_a2*T_a1_a3/3 + 7*H_a2_a3*H_p2_p2**2*T_a1_a1*T_a2_a3/12 -
H_a2_a3*H_p2_p2**2*T_a1_a2*T_a1_a3/6 - 29*H_a2_p1**2*H_a3_a3*H_p1_p1*T_a1_a1/120 - 7*H_a2_p1**2*H_a3_a3*H_p1_p2*T_a1_a1/30 +
62*H_a2_p1**2*H_a3_a3*H_p2_p2*T_a1_a1/15 - H_a2_p1**2*H_a3_p1**2*T_a1_a1/40 + H_a2_p1**2*H_a3_p1*H_a3_p2*T_a1_a1/30 -
463*H_a2_p1**2*H_a3_p2**2*T_a1_a1/120 - 5*H_a2_p1**2*H_p1_p1*T_a1_a1*T_a3_a3/24 - H_a2_p1**2*H_p1_p1*T_a1_a3**2/6 -
5*H_a2_p1**2*H_p1_p2*T_a1_a1*T_a3_a3/24 - H_a2_p1**2*H_p1_p2*T_a1_a3**2/6 + 31*H_a2_p1**2*H_p2_p2*T_a1_a1*T_a3_a3/6 -
25*H_a2_p1**2*H_p2_p2*T_a1_a3**2/6 - 29*H_a2_p1*H_a2_p2*H_a3_a3*H_p1_p1*T_a1_a1/60 - 407*H_a2_p1*H_a2_p2*H_a3_a3*H_p1_p2*T_a1_a1/40 -
```

```
113*H_a2_p1*H_a2_p2*H_a3_a3*H_p2_p2*T_a1_a1/120 + 89*H_a2_p1*H_a2_p2*H_a3_p1**2*T_a1_a1/120 + 307*H_a2_p1*H_a2_p2*H_a3_p1*H_a3_p2*T_a1_a1/30 +
209*H_a2_p1*H_a2_p2*H_a3_p2**2*T_a1_a1/120 - 5*H_a2_p1*H_a2_p2*H_p1_p1*T_a1_a1*T_a3_a3/24 - H_a2_p1*H_a2_p2*H_p1_p1*T_a1_a3**2/6 -
163*H_a2_p1*H_a2_p2*H_p1_p2*T_a1_a1*T_a3_a3/12 + 119*H_a2_p1*H_a2_p2*H_p1_p2*T_a1_a3**2/12 - 73*H_a2_p1*H_a2_p2*H_p2_p2*T_a1_a1*T_a3_a3/48 +
7*H_a2_p1*H_a2_p2*H_p2_p2*T_a1_a3**2/8 + 17*H_a2_p1*H_a3_p1*H_p1_p1*T_a1_a1*T_a2_a3/24 + H_a2_p1*H_a3_p1*H_p1_p1*T_a1_a2*T_a1_a3/12 +
11*H_a2_p1*H_a3_p1*H_p1_p2*T_a1_a1*T_a2_a3/12 - 5*H_a2_p1*H_a3_p1*H_p1_p2*T_a1_a2*T_a1_a3/6 - 32*H_a2_p1*H_a3_p1*H_p2_p2*T_a1_a1*T_a2_a3/3 +
55*H_a2_p1*H_a3_p1*H_p2_p2*T_a1_a2*T_a1_a3/6 + 5*H_a2_p1*H_a3_p2*H_p1_p1*T_a1_a1*T_a2_a3/24 - H_a2_p1*H_a3_p2*H_p1_p1*T_a1_a2*T_a1_a3/3 +
125*H_a2_p1*H_a3_p2*H_p1_p2*T_a1_a1*T_a2_a3/12 - 9*H_a2_p1*H_a3_p2*H_p1_p2*T_a1_a2*T_a1_a3 - 7*H_a2_p1*H_a3_p2*H_p2_p2*T_a1_a1*T_a2_a3/6 +
H_a2_p1*H_a3_p2*H_p2_p2*T_a1_a2*T_a1_a3 + 661*H_a2_p2**2*H_a3_a3*H_p1_p1*T_a1_a1/120 + 149*H_a2_p2**2*H_a3_a3*H_p1_p2*T_a1_a1/240 -
H_a2_p2**2*H_a3_a3*H_p2_p2*T_a1_a1/40 - 27*H_a2_p2**2*H_a3_p1**2*T_a1_a1/5 - 161*H_a2_p2**2*H_a3_p1*H_a3_p2*T_a1_a1/120 +
H_a2_p2**2*H_a3_p2**2*T_a1_a1/5 + 49*H_a2_p2**2*H_p1_p1*T_a1_a1*T_a3_a3/6 - 157*H_a2_p2**2*H_p1_p1*T_a1_a3**2/24 +
71*H_a2_p2**2*H_p1_p2*T_a1_a1*T_a3_a3/48 - 3*H_a2_p2**2*H_p1_p2*T_a1_a3**2/2 + H_a2_p2**2*H_p2_p2*T_a1_a1*T_a3_a3/12 - H_a2_p2**2*H_p2_p2*T_a1_a3**2/3
- 17*H_a2_p2*H_a3_p1*H_p1_p1*T_a1_a1*T_a2_a3/12 + H_a2_p2*H_a3_p1*H_p1_p1*T_a1_a2*T_a1_a3 + 149*H_a2_p2*H_a3_p1*H_p1_p2*T_a1_a1*T_a2_a3/12 -
34*H_a2_p2*H_a3_p1*H_p1_p2*T_a1_a2*T_a1_a3/3 + 5*H_a2_p2*H_a3_p1*H_p2_p2*T_a1_a1*T_a2_a3/6 - 4*H_a2_p2*H_a3_p1*H_p2_p2*T_a1_a2*T_a1_a3/3 -
307*H_a2_p2*H_a3_p2*H_p1_p1*T_a1_a1*T_a2_a3/24 + 32*H_a2_p2*H_a3_p2*H_p1_p1*T_a1_a2*T_a1_a3/3 - H_a2_p2*H_a3_p2*H_p1_p2*T_a1_a1*T_a2_a3/3 -
H_a2_p2*H_a3_p2*H_p1_p2*T_a1_a2*T_a1_a3/3 - H_a2_p2*H_a3_p2*H_p2_p2*T_a1_a1*T_a2_a3/3 - H_a2_p2*H_a3_p2*H_p2_p2*T_a1_a2*T_a1_a3/3 -
H_a3_a3*H_p1_p1**2*T_a1_a1*T_a2_a2/12 + 5*H_a3_a3*H_p1_p1**2*T_a1_a2**2/24 - H_a3_a3*H_p1_p1*H_p1_p2*T_a1_a1*T_a2_a2/32 +
25*H_a3_a3*H_p1_p1*H_p1_p2*T_a1_a2**2/48 - 727*H_a3_a3*H_p1_p1*H_p2_p2*T_a1_a1*T_a2_a2/96 + 145*H_a3_a3*H_p1_p1*H_p2_p2*T_a1_a2**2/24 +
353*H_a3_a3*H_p1_p2**2*T_a1_a1*T_a2_a2/48 - 5*H_a3_a3*H_p1_p2**2*T_a1_a2**2 - H_a3_a3*H_p1_p2*H_p2_p2*T_a1_a1*T_a2_a2/32 +
25*H_a3_a3*H_p1_p2*H_p2_p2*T_a1_a2**2/48 - H_a3_a3*H_p2_p2**2*T_a1_a1*T_a2_a2/12 + 5*H_a3_a3*H_p2_p2**2*T_a1_a2**2/24 +
H_a3_p1**2*H_p1_p1*T_a1_a1*T_a2_a2/12 - H_a3_p1**2*H_p1_p1*T_a1_a2**2/3 - 21*H_a3_p1**2*H_p1_p2*T_a1_a1*T_a2_a2/16 + 5*H_a3_p1**2*H_p1_p2*T_a1_a2**2/6
+ 325*H_a3_p1**2*H_p2_p2*T_a1_a1*T_a2_a2/48 - 43*H_a3_p1**2*H_p2_p2*T_a1_a2**2/8 + 27*H_a3_p1*H_a3_p2*H_p1_p1*T_a1_a1*T_a2_a2/16 -
37*H_a3_p1*H_a3_p2*H_p1_p1*T_a1_a2**2/24 - 79*H_a3_p1*H_a3_p2*H_p1_p2*T_a1_a1*T_a2_a2/6 + 59*H_a3_p1*H_a3_p2*H_p1_p2*T_a1_a2**2/6 +
27*H_a3_p1*H_a3_p2*H_p2_p2*T_a1_a1*T_a2_a2/16 - 37*H_a3_p1*H_a3_p2*H_p2_p2*T_a1_a2**2/24 + 325*H_a3_p2**2*H_p1_p1*T_a1_a1*T_a2_a2/48 -
43*H_a3_p2**2*H_p1_p1*T_a1_a2**2/8 - 21*H_a3_p2**2*H_p1_p2*T_a1_a1*T_a2_a2/16 + 5*H_a3_p2**2*H_p1_p2*T_a1_a2**2/6 +
H_a3_p2**2*H_p2_p2*T_a1_a1*T_a2_a2/12 - H_a3_p2**2*H_p2_p2*T_a1_a2**2/3 - 3*H_p1_p1**2*T_a1_a1*T_a2_a2*T_a3_a3/16 + H_p1_p1**2*T_a1_a1*T_a2_a3**2/8 +
H_p1_p1**2*T_a1_a2**2*T_a3_a3/8 + H_p1_p1**2*T_a1_a3**2*T_a2_a2/4 - 3*H_p1_p1*H_p1_p2*T_a1_a1*T_a2_a2*T_a3_a3/8 +
H_p1_p1*H_p1_p2*T_a1_a1*T_a2_a3**2/16 + 7*H_p1_p1*H_p1_p2*T_a1_a2**2*T_a3_a3/16 + H_p1_p1*H_p1_p2*T_a1_a3**2*T_a2_a2/2 -
159*H_p1_p1*H_p2_p2*T_a1_a1*T_a2_a2*T_a3_a3/16 + 107*H_p1_p1*H_p2_p2*T_a1_a1*T_a2_a3**2/16 + 55*H_p1_p1*H_p2_p2*T_a1_a2**2*T_a3_a3/8 -
19*H_p1_p1*H_p2_p2*T_a1_a2*T_a1_a3*T_a2_a3/2 + 107*H_p1_p1*H_p2_p2*T_a1_a3**2*T_a2_a2/16 + 75*H_p1_p2**2*T_a1_a1*T_a2_a2*T_a3_a3/8 -
51*H_p1_p2**2*T_a1_a1*T_a2_a3**2/8 - 6*H_p1_p2**2*T_a1_a2**2*T_a3_a3 + 19*H_p1_p2**2*T_a1_a2*T_a1_a3*T_a2_a3/2 - 51*H_p1_p2**2*T_a1_a3**2*T_a2_a2/8 -
3*H_p1_p2*H_p2_p2*T_a1_a1*T_a2_a2*T_a3_a3/8 + H_p1_p2*H_p2_p2*T_a1_a1*T_a2_a3**2/2 + 7*H_p1_p2*H_p2_p2*T_a1_a2**2*T_a3_a3/16 +
H_p1_p2*H_p2_p2*T_a1_a3**2*T_a2_a2/16 - 3*H_p2_p2**2*T_a1_a1*T_a2_a2*T_a3_a3/16 + H_p2_p2**2*T_a1_a1*T_a2_a3**2/4 + H_p2_p2**2*T_a1_a2**2*T_a3_a3/8 +
H_p2_p2**2*T_a1_a3**2*T_a2_a2/8",
   "sha256": "0180cf543abac99cd38e2442c4cc5a749213f170f4306179da2e8b06f2641dbb",
   "n_terms": 797,
   "zero": false,
   "bose_12_symmetric": true,
   "bose_13_symmetric": true,
   "method": "fastpole ring path, t-truncated"
  },
  "GO2": {
   "pole_over_U_d4_residue": "-19*H_a1_a1*H_a2_a2*H_a3_a3*H_p1_p1**2/80 - 19*H_a1_a1*H_a2_a2*H_a3_a3*H_p1_p1*H_p1_p2/40 +
623*H_a1_a1*H_a2_a2*H_a3_a3*H_p1_p1*H_p2_p2/240 - 397*H_a1_a1*H_a2_a2*H_a3_a3*H_p1_p2**2/120 - 19*H_a1_a1*H_a2_a2*H_a3_a3*H_p1_p2*H_p2_p2/40 -
19*H_a1_a1*H_a2_a2*H_a3_a3*H_p2_p2**2/80 + 11*H_a1_a1*H_a2_a2*H_a3_p1**2*H_p1_p1/60 - 7*H_a1_a1*H_a2_a2*H_a3_p1**2*H_p1_p2/10 -
263*H_a1_a1*H_a2_a2*H_a3_p1**2*H_p2_p2/80 + 161*H_a1_a1*H_a2_a2*H_a3_p1*H_a3_p2*H_p1_p1/120 + 147*H_a1_a1*H_a2_a2*H_a3_p1*H_a3_p2*H_p1_p2/20 +
161*H_a1_a1*H_a2_a2*H_a3_p1*H_a3_p2*H_p2_p2/120 - 263*H_a1_a1*H_a2_a2*H_a3_p2**2*H_p1_p1/80 - 7*H_a1_a1*H_a2_a2*H_a3_p2**2*H_p1_p2/10 +
11*H_a1_a1*H_a2_a2*H_a3_p2**2*H_p2_p2/60 - 251*H_a1_a1*H_a2_a2*H_p1_p1**2*T_a3_a3/480 - 49*H_a1_a1*H_a2_a2*H_p1_p1*H_p1_p2*T_a3_a3/48 +
101*H_a1_a1*H_a2_a2*H_p1_p1*H_p2_p2*T_a3_a3/48 - 89*H_a1_a1*H_a2_a2*H_p1_p2**2*T_a3_a3/24 - 49*H_a1_a1*H_a2_a2*H_p1_p2*H_p2_p2*T_a3_a3/48 -
251*H_a1_a1*H_a2_a2*H_p2_p2**2*T_a3_a3/480 - 31*H_a1_a1*H_a2_a3**2*H_p1_p1**2/240 + 43*H_a1_a1*H_a2_a3**2*H_p1_p1*H_p1_p2/120 -
451*H_a1_a1*H_a2_a3**2*H_p1_p1*H_p2_p2/120 + 209*H_a1_a1*H_a2_a3**2*H_p1_p2**2/60 - 19*H_a1_a1*H_a2_a3**2*H_p1_p2*H_p2_p2/15 -
19*H_a1_a1*H_a2_a3**2*H_p2_p2**2/30 + 11*H_a1_a1*H_a2_a3*H_a2_p1*H_a3_p1*H_p1_p1/120 + 41*H_a1_a1*H_a2_a3*H_a2_p1*H_a3_p1*H_p1_p2/60 +
347*H_a1_a1*H_a2_a3*H_a2_p1*H_a3_p1*H_p2_p2/40 + 11*H_a1_a1*H_a2_a3*H_a2_p1*H_a3_p2*H_p1_p1/120 - 419*H_a1_a1*H_a2_a3*H_a2_p1*H_a3_p2*H_p1_p2/60 +
121*H_a1_a1*H_a2_a3*H_a2_p1*H_a3_p2*H_p2_p2/120 - 169*H_a1_a1*H_a2_a3*H_a2_p2*H_a3_p1*H_p1_p1/120 - 469*H_a1_a1*H_a2_a3*H_a2_p2*H_a3_p1*H_p1_p2/60 +
7*H_a1_a1*H_a2_a3*H_a2_p2*H_a3_p1*H_p2_p2/40 + 267*H_a1_a1*H_a2_a3*H_a2_p2*H_a3_p2*H_p1_p1/40 + 71*H_a1_a1*H_a2_a3*H_a2_p2*H_a3_p2*H_p1_p2/60 +
71*H_a1_a1*H_a2_a3*H_a2_p2*H_a3_p2*H_p2_p2/60 + H_a1_a1*H_a2_a3*H_p1_p1**2*T_a2_a3/2 + 2*H_a1_a1*H_a2_a3*H_p1_p1*H_p1_p2*T_a2_a3 -
13*H_a1_a1*H_a2_a3*H_p1_p1*H_p2_p2*T_a2_a3/2 + 47*H_a1_a1*H_a2_a3*H_p1_p2**2*T_a2_a3/6 - 4*H_a1_a1*H_a2_a3*H_p1_p2*H_p2_p2*T_a2_a3/3 -
2*H_a1_a1*H_a2_a3*H_p2_p2**2*T_a2_a3/3 - 17*H_a1_a1*H_a2_p1**2*H_a3_a3*H_p1_p1/120 - 17*H_a1_a1*H_a2_p1**2*H_a3_a3*H_p1_p2/120 -
1067*H_a1_a1*H_a2_p1**2*H_a3_a3*H_p2_p2/240 + 7*H_a1_a1*H_a2_p1**2*H_a3_p1**2/30 - 37*H_a1_a1*H_a2_p1**2*H_a3_p1*H_a3_p2/60 +
39*H_a1_a1*H_a2_p1**2*H_a3_p2**2/10 + 29*H_a1_a1*H_a2_p1**2*H_p1_p1*T_a3_a3/120 + H_a1_a1*H_a2_p1**2*H_p1_p2*T_a3_a3/6 -
25*H_a1_a1*H_a2_p1**2*H_p2_p2*T_a3_a3/6 - 17*H_a1_a1*H_a2_p1*H_a2_p2*H_a3_a3*H_p1_p1/120 + 34*H_a1_a1*H_a2_p1*H_a2_p2*H_a3_a3*H_p1_p2/5 -
39*H_a1_a1*H_a2_p1*H_a2_p2*H_a3_a3*H_p2_p2/40 + 73*H_a1_a1*H_a2_p1*H_a2_p2*H_a3_p1**2/60 - 217*H_a1_a1*H_a2_p1*H_a2_p2*H_a3_p1*H_a3_p2/30 -
H_a1_a1*H_a2_p1*H_a2_p2*H_a3_p2**2/30 + H_a1_a1*H_a2_p1*H_a2_p2*H_p1_p1*T_a3_a3/4 + 29*H_a1_a1*H_a2_p1*H_a2_p2*H_p1_p2*T_a3_a3/4 -
2*H_a1_a1*H_a2_p1*H_a2_p2*H_p2_p2*T_a3_a3/3 - 19*H_a1_a1*H_a2_p1*H_a3_p1*H_p1_p1*T_a2_a3/24 - H_a1_a1*H_a2_p1*H_a3_p1*H_p1_p2*T_a2_a3/6 +
97*H_a1_a1*H_a2_p1*H_a3_p1*H_p2_p2*T_a2_a3/12 - 7*H_a1_a1*H_a2_p1*H_a3_p2*H_p1_p1*T_a2_a3/24 - 15*H_a1_a1*H_a2_p1*H_a3_p2*H_p1_p2*T_a2_a3/2 +
3*H_a1_a1*H_a2_p1*H_a3_p2*H_p2_p2*T_a2_a3/4 - 577*H_a1_a1*H_a2_p2**2*H_a3_a3*H_p1_p1/240 + 16*H_a1_a1*H_a2_p2**2*H_a3_a3*H_p1_p2/15 +
11*H_a1_a1*H_a2_p2**2*H_a3_a3*H_p2_p2/60 + 97*H_a1_a1*H_a2_p2**2*H_a3_p1**2/30 - 41*H_a1_a1*H_a2_p2**2*H_a3_p1*H_a3_p2/30 -
7*H_a1_a1*H_a2_p2**2*H_a3_p2**2/10 - 13*H_a1_a1*H_a2_p2**2*H_p1_p1*T_a3_a3/6 + 31*H_a1_a1*H_a2_p2**2*H_p1_p2*T_a3_a3/24 +
9*H_a1_a1*H_a2_p2**2*H_p2_p2*T_a3_a3/20 - 15*H_a1_a1*H_a2_p2*H_a3_p1*H_p1_p1*T_a2_a3/8 - 17*H_a1_a1*H_a2_p2*H_a3_p1*H_p1_p2*T_a2_a3/2 -
H_a1_a1*H_a2_p2*H_a3_p1*H_p2_p2*T_a2_a3/4 + 73*H_a1_a1*H_a2_p2*H_a3_p2*H_p1_p1*T_a2_a3/12 + H_a1_a1*H_a2_p2*H_a3_p2*H_p1_p2*T_a2_a3/2 +
H_a1_a1*H_a2_p2*H_a3_p2*H_p2_p2*T_a2_a3/2 - 73*H_a1_a1*H_a3_a3*H_p1_p1**2*T_a2_a2/120 - 73*H_a1_a1*H_a3_a3*H_p1_p1*H_p1_p2*T_a2_a2/60 +
499*H_a1_a1*H_a3_a3*H_p1_p1*H_p2_p2*T_a2_a2/240 - 451*H_a1_a1*H_a3_a3*H_p1_p2**2*T_a2_a2/120 - 257*H_a1_a1*H_a3_a3*H_p1_p2*H_p2_p2*T_a2_a2/240 -
251*H_a1_a1*H_a3_a3*H_p2_p2**2*T_a2_a2/480 + 17*H_a1_a1*H_a3_p1**2*H_p1_p1*T_a2_a2/30 - 37*H_a1_a1*H_a3_p1**2*H_p1_p2*T_a2_a2/120 -
61*H_a1_a1*H_a3_p1**2*H_p2_p2*T_a2_a2/20 + 33*H_a1_a1*H_a3_p1*H_a3_p2*H_p1_p1*T_a2_a2/20 + 39*H_a1_a1*H_a3_p1*H_a3_p2*H_p1_p2*T_a2_a2/5 +
47*H_a1_a1*H_a3_p1*H_a3_p2*H_p2_p2*T_a2_a2/30 - 361*H_a1_a1*H_a3_p2**2*H_p1_p1*T_a2_a2/120 - 47*H_a1_a1*H_a3_p2**2*H_p1_p2*T_a2_a2/120 +
9*H_a1_a1*H_a3_p2**2*H_p2_p2*T_a2_a2/20 - 43*H_a1_a1*H_p1_p1**2*T_a2_a2*T_a3_a3/48 + 5*H_a1_a1*H_p1_p1**2*T_a2_a3**2/8 -
85*H_a1_a1*H_p1_p1*H_p1_p2*T_a2_a2*T_a3_a3/48 + 13*H_a1_a1*H_p1_p1*H_p1_p2*T_a2_a3**2/8 + 19*H_a1_a1*H_p1_p1*H_p2_p2*T_a2_a2*T_a3_a3/12 -
11*H_a1_a1*H_p1_p1*H_p2_p2*T_a2_a3**2/4 - 25*H_a1_a1*H_p1_p2**2*T_a2_a2*T_a3_a3/6 + 13*H_a1_a1*H_p1_p2**2*T_a2_a3**2/3 -
13*H_a1_a1*H_p1_p2*H_p2_p2*T_a2_a2*T_a3_a3/8 - H_a1_a1*H_p1_p2*H_p2_p2*T_a2_a3**2/12 - 13*H_a1_a1*H_p2_p2**2*T_a2_a2*T_a3_a3/16 -
H_a1_a1*H_p2_p2**2*T_a2_a3**2/24 - 31*H_a1_a2**2*H_a3_a3*H_p1_p1**2/240 - 7*H_a1_a2**2*H_a3_a3*H_p1_p1*H_p1_p2/8 -
35*H_a1_a2**2*H_a3_a3*H_p1_p1*H_p2_p2/8 + 9*H_a1_a2**2*H_a3_a3*H_p1_p2**2/4 - 7*H_a1_a2**2*H_a3_a3*H_p1_p2*H_p2_p2/8 -
31*H_a1_a2**2*H_a3_a3*H_p2_p2**2/240 + 5*H_a1_a2**2*H_a3_p1**2*H_p1_p1/24 + 2*H_a1_a2**2*H_a3_p1**2*H_p1_p2/3 + 33*H_a1_a2**2*H_a3_p1**2*H_p2_p2/8 +
5*H_a1_a2**2*H_a3_p1*H_a3_p2*H_p1_p1/12 - 13*H_a1_a2**2*H_a3_p1*H_a3_p2*H_p1_p2/2 + 5*H_a1_a2**2*H_a3_p1*H_a3_p2*H_p2_p2/12 +
33*H_a1_a2**2*H_a3_p2**2*H_p1_p1/8 + 2*H_a1_a2**2*H_a3_p2**2*H_p1_p2/3 + 5*H_a1_a2**2*H_a3_p2**2*H_p2_p2/24 + 37*H_a1_a2**2*H_p1_p1**2*T_a3_a3/120 +
H_a1_a2**2*H_p1_p1*H_p1_p2*T_a3_a3/4 - 29*H_a1_a2**2*H_p1_p1*H_p2_p2*T_a3_a3/8 + 19*H_a1_a2**2*H_p1_p2**2*T_a3_a3/6 +
```

```
H_a1_a2**2*H_p1_p2*H_p2_p2*T_a3_a3/4 + 37*H_a1_a2**2*H_p2_p2**2*T_a3_a3/120 + H_a1_a2*H_a1_a3*H_a2_a3*H_p1_p1**2/6 +
H_a1_a2*H_a1_a3*H_a2_a3*H_p1_p1*H_p1_p2/3 + 25*H_a1_a2*H_a1_a3*H_a2_a3*H_p1_p1*H_p2_p2/3 - 47*H_a1_a2*H_a1_a3*H_a2_a3*H_p1_p2**2/6 +
H_a1_a2*H_a1_a3*H_a2_a3*H_p1_p2*H_p2_p2/3 + H_a1_a2*H_a1_a3*H_a2_a3*H_p2_p2**2/6 - H_a1_a2*H_a1_a3*H_a2_p1*H_a3_p1*H_p1_p1/12 +
H_a1_a2*H_a1_a3*H_a2_p1*H_a3_p1*H_p1_p2 - 7*H_a1_a2*H_a1_a3*H_a2_p1*H_a3_p1*H_p2_p2 - 4*H_a1_a2*H_a1_a3*H_a2_p1*H_a3_p2*H_p1_p1/3 +
22*H_a1_a2*H_a1_a3*H_a2_p1*H_a3_p2*H_p1_p2/3 - 2*H_a1_a2*H_a1_a3*H_a2_p1*H_a3_p2*H_p2_p2/3 + H_a1_a2*H_a1_a3*H_a2_p2*H_a3_p1*H_p1_p1/4 +
17*H_a1_a2*H_a1_a3*H_a2_p2*H_a3_p1*H_p1_p2/2 + H_a1_a2*H_a1_a3*H_a2_p2*H_a3_p1*H_p2_p2/2 - 109*H_a1_a2*H_a1_a3*H_a2_p2*H_a3_p2*H_p1_p1/12 -
H_a1_a2*H_a1_a3*H_a2_p2*H_a3_p2*H_p1_p2/6 - H_a1_a2*H_a1_a3*H_a2_p2*H_a3_p2*H_p2_p2/6 - H_a1_a2*H_a1_a3*H_p1_p1**2*T_a2_a3/2 -
7*H_a1_a2*H_a1_a3*H_p1_p1*H_p1_p2*T_a2_a3/6 + 43*H_a1_a2*H_a1_a3*H_p1_p1*H_p2_p2*T_a2_a3/6 - 9*H_a1_a2*H_a1_a3*H_p1_p2**2*T_a2_a3 -
4*H_a1_a2*H_a1_a3*H_p1_p2*H_p2_p2*T_a2_a3/3 - 2*H_a1_a2*H_a1_a3*H_p2_p2**2*T_a2_a3/3 - H_a1_a2*H_a1_p1*H_a2_a3*H_a3_p1*H_p1_p1/6 -
H_a1_a2*H_a1_p1*H_a2_a3*H_a3_p1*H_p1_p2/6 - 109*H_a1_a2*H_a1_p1*H_a2_a3*H_a3_p1*H_p2_p2/12 + H_a1_a2*H_a1_p1*H_a2_a3*H_a3_p2*H_p1_p1/2 +
17*H_a1_a2*H_a1_p1*H_a2_a3*H_a3_p2*H_p1_p2/2 + H_a1_a2*H_a1_p1*H_a2_a3*H_a3_p2*H_p2_p2/4 + 11*H_a1_a2*H_a1_p1*H_a2_p1*H_a3_a3*H_p1_p1/120 -
H_a1_a2*H_a1_p1*H_a2_p1*H_a3_a3*H_p1_p2/2 + 97*H_a1_a2*H_a1_p1*H_a2_p1*H_a3_a3*H_p2_p2/12 - H_a1_a2*H_a1_p1*H_a2_p1*H_a3_p1**2/3 -
8*H_a1_a2*H_a1_p1*H_a2_p1*H_a3_p2**2 - 89*H_a1_a2*H_a1_p1*H_a2_p1*H_p1_p1*T_a3_a3/120 - 4*H_a1_a2*H_a1_p1*H_a2_p1*H_p1_p2*T_a3_a3/3 +
89*H_a1_a2*H_a1_p1*H_a2_p1*H_p2_p2*T_a3_a3/12 + 3*H_a1_a2*H_a1_p1*H_a2_p2*H_a3_a3*H_p1_p1/2 - 11*H_a1_a2*H_a1_p1*H_a2_p2*H_a3_a3*H_p1_p2/2 +
3*H_a1_a2*H_a1_p1*H_a2_p2*H_a3_a3*H_p2_p2/2 - 5*H_a1_a2*H_a1_p1*H_a2_p2*H_a3_p1**2/3 + 35*H_a1_a2*H_a1_p1*H_a2_p2*H_a3_p1*H_a3_p2/6 -
5*H_a1_a2*H_a1_p1*H_a2_p2*H_a3_p2**2/3 + 5*H_a1_a2*H_a1_p1*H_a2_p2*H_p1_p1*T_a3_a3/6 - 19*H_a1_a2*H_a1_p1*H_a2_p2*H_p1_p2*T_a3_a3/3 +
5*H_a1_a2*H_a1_p1*H_a2_p2*H_p2_p2*T_a3_a3/6 + 3*H_a1_a2*H_a1_p1*H_a3_p1*H_p1_p1*T_a2_a3/4 + 5*H_a1_a2*H_a1_p1*H_a3_p1*H_p1_p2*T_a2_a3/6 -
101*H_a1_a2*H_a1_p1*H_a3_p1*H_p2_p2*T_a2_a3/12 + 13*H_a1_a2*H_a1_p1*H_a3_p2*H_p1_p1*T_a2_a3/12 + 28*H_a1_a2*H_a1_p1*H_a3_p2*H_p1_p2*T_a2_a3/3 +
3*H_a1_a2*H_a1_p1*H_a3_p2*H_p2_p2*T_a2_a3/4 - 2*H_a1_a2*H_a1_p2*H_a2_a3*H_a3_p1*H_p1_p1/3 + 22*H_a1_a2*H_a1_p2*H_a2_a3*H_a3_p1*H_p1_p2/3 -
4*H_a1_a2*H_a1_p2*H_a2_a3*H_a3_p1*H_p2_p2/3 - 7*H_a1_a2*H_a1_p2*H_a2_a3*H_a3_p2*H_p1_p1 + H_a1_a2*H_a1_p2*H_a2_a3*H_a3_p2*H_p1_p2 -
H_a1_a2*H_a1_p2*H_a2_a3*H_a3_p2*H_p2_p2/12 - 23*H_a1_a2*H_a1_p2*H_a2_p1*H_a3_a3*H_p1_p2/3 - H_a1_a2*H_a1_p2*H_a2_p1*H_a3_p1**2/6 +
43*H_a1_a2*H_a1_p2*H_a2_p1*H_a3_p1*H_a3_p2/6 - H_a1_a2*H_a1_p2*H_a2_p1*H_a3_p2**2/6 - 5*H_a1_a2*H_a1_p2*H_a2_p1*H_p1_p1*T_a3_a3/12 -
25*H_a1_a2*H_a1_p2*H_a2_p1*H_p1_p2*T_a3_a3/3 - 5*H_a1_a2*H_a1_p2*H_a2_p1*H_p2_p2*T_a3_a3/12 + 97*H_a1_a2*H_a1_p2*H_a2_p2*H_a3_a3*H_p1_p1/12 -
H_a1_a2*H_a1_p2*H_a2_p2*H_a3_a3*H_p1_p2/2 + 11*H_a1_a2*H_a1_p2*H_a2_p2*H_a3_a3*H_p2_p2/120 - 8*H_a1_a2*H_a1_p2*H_a2_p2*H_a3_p1**2 -
H_a1_a2*H_a1_p2*H_a2_p2*H_a3_p2**2/3 + 89*H_a1_a2*H_a1_p2*H_a2_p2*H_p1_p1*T_a3_a3/12 - 4*H_a1_a2*H_a1_p2*H_a2_p2*H_p1_p2*T_a3_a3/3 -
89*H_a1_a2*H_a1_p2*H_a2_p2*H_p2_p2*T_a3_a3/120 - H_a1_a2*H_a1_p2*H_a3_p1*H_p1_p1*T_a2_a3/6 + 49*H_a1_a2*H_a1_p2*H_a3_p1*H_p1_p2*T_a2_a3/6 -
3*H_a1_a2*H_a1_p2*H_a3_p1*H_p2_p2*T_a2_a3/4 - 25*H_a1_a2*H_a1_p2*H_a3_p2*H_p1_p1*T_a2_a3/4 + 2*H_a1_a2*H_a1_p2*H_a3_p2*H_p1_p2*T_a2_a3 +
5*H_a1_a2*H_a1_p2*H_a3_p2*H_p2_p2*T_a2_a3/6 - 2*H_a1_a2*H_a2_a3*H_p1_p1**2*T_a1_a3/3 - 4*H_a1_a2*H_a2_a3*H_p1_p1*H_p1_p2*T_a1_a3/3 +
43*H_a1_a2*H_a2_a3*H_p1_p1*H_p2_p2*T_a1_a3/6 - 9*H_a1_a2*H_a2_a3*H_p1_p2**2*T_a1_a3 - 7*H_a1_a2*H_a2_a3*H_p1_p2*H_p2_p2*T_a1_a3/6 -
H_a1_a2*H_a2_a3*H_p2_p2**2*T_a1_a3/2 + 5*H_a1_a2*H_a2_p1*H_a3_p1*H_p1_p1*T_a1_a3/6 + 2*H_a1_a2*H_a2_p1*H_a3_p1*H_p1_p2*T_a1_a3 -
25*H_a1_a2*H_a2_p1*H_a3_p1*H_p2_p2*T_a1_a3/4 - 3*H_a1_a2*H_a2_p1*H_a3_p2*H_p1_p1*T_a1_a3/4 + 49*H_a1_a2*H_a2_p1*H_a3_p2*H_p1_p2*T_a1_a3/6 -
H_a1_a2*H_a2_p1*H_a3_p2*H_p2_p2*T_a1_a3/6 + 3*H_a1_a2*H_a2_p2*H_a3_p1*H_p1_p1*T_a1_a3/4 + 28*H_a1_a2*H_a2_p2*H_a3_p1*H_p1_p2*T_a1_a3/3 +
13*H_a1_a2*H_a2_p2*H_a3_p1*H_p2_p2*T_a1_a3/12 - 101*H_a1_a2*H_a2_p2*H_a3_p2*H_p1_p1*T_a1_a3/12 + 5*H_a1_a2*H_a2_p2*H_a3_p2*H_p1_p2*T_a1_a3/6 +
3*H_a1_a2*H_a2_p2*H_a3_p2*H_p2_p2*T_a1_a3/4 + H_a1_a2*H_a3_a3*H_p1_p1**2*T_a1_a2/2 - 15*H_a1_a2*H_a3_a3*H_p1_p1*H_p2_p2*T_a1_a2/2 +
35*H_a1_a2*H_a3_a3*H_p1_p2**2*T_a1_a2/6 + H_a1_a2*H_a3_a3*H_p2_p2**2*T_a1_a2/2 - 5*H_a1_a2*H_a3_p1**2*H_p1_p1*T_a1_a2/12 +
H_a1_a2*H_a3_p1**2*H_p1_p2*T_a1_a2/3 + 91*H_a1_a2*H_a3_p1**2*H_p2_p2*T_a1_a2/12 - 29*H_a1_a2*H_a3_p1*H_a3_p2*H_p1_p2*T_a1_a2/2 +
91*H_a1_a2*H_a3_p2**2*H_p1_p1*T_a1_a2/12 + H_a1_a2*H_a3_p2**2*H_p1_p2*T_a1_a2/3 - 5*H_a1_a2*H_a3_p2**2*H_p2_p2*T_a1_a2/12 +
11*H_a1_a2*H_p1_p1**2*T_a1_a2*T_a3_a3/8 - 4*H_a1_a2*H_p1_p1**2*T_a1_a3*T_a2_a3/3 + 9*H_a1_a2*H_p1_p1*H_p1_p2*T_a1_a2*T_a3_a3/4 -
17*H_a1_a2*H_p1_p1*H_p1_p2*T_a1_a3*T_a2_a3/6 - 6*H_a1_a2*H_p1_p1*H_p2_p2*T_a1_a2*T_a3_a3 + 6*H_a1_a2*H_p1_p1*H_p2_p2*T_a1_a3*T_a2_a3 +
23*H_a1_a2*H_p1_p2**2*T_a1_a2*T_a3_a3/3 - 61*H_a1_a2*H_p1_p2**2*T_a1_a3*T_a2_a3/6 + 9*H_a1_a2*H_p1_p2*H_p2_p2*T_a1_a2*T_a3_a3/4 -
17*H_a1_a2*H_p1_p2*H_p2_p2*T_a1_a3*T_a2_a3/6 + 11*H_a1_a2*H_p2_p2**2*T_a1_a2*T_a3_a3/8 - 4*H_a1_a2*H_p2_p2**2*T_a1_a3*T_a2_a3/3 -
19*H_a1_a3**2*H_a2_a2*H_p1_p1**2/30 - 19*H_a1_a3**2*H_a2_a2*H_p1_p1*H_p1_p2/15 - 451*H_a1_a3**2*H_a2_a2*H_p1_p1*H_p2_p2/120 +
209*H_a1_a3**2*H_a2_a2*H_p1_p2**2/60 + 43*H_a1_a3**2*H_a2_a2*H_p1_p2*H_p2_p2/120 - 31*H_a1_a3**2*H_a2_a2*H_p2_p2**2/240 +
47*H_a1_a3**2*H_a2_p1**2*H_p2_p2/12 - 47*H_a1_a3**2*H_a2_p1*H_a2_p2*H_p1_p2/6 + 11*H_a1_a3**2*H_a2_p2**2*H_p1_p1/3 - H_a1_a3**2*H_a2_p2**2*H_p1_p2/4 +
5*H_a1_a3**2*H_a2_p2**2*H_p2_p2/24 - 41*H_a1_a3**2*H_p1_p1**2*T_a2_a2/120 - 41*H_a1_a3**2*H_p1_p1*H_p1_p2*T_a2_a2/60 -
391*H_a1_a3**2*H_p1_p1*H_p2_p2*T_a2_a2/120 + 39*H_a1_a3**2*H_p1_p2**2*T_a2_a2/10 + 59*H_a1_a3**2*H_p1_p2*H_p2_p2*T_a2_a2/60 +
37*H_a1_a3**2*H_p2_p2**2*T_a2_a2/120 + 71*H_a1_a3*H_a1_p1*H_a2_a2*H_a3_p1*H_p1_p1/60 + 71*H_a1_a3*H_a1_p1*H_a2_a2*H_a3_p1*H_p1_p2/60 +
267*H_a1_a3*H_a1_p1*H_a2_a2*H_a3_p1*H_p2_p2/40 + 7*H_a1_a3*H_a1_p1*H_a2_a2*H_a3_p2*H_p1_p1/40 - 469*H_a1_a3*H_a1_p1*H_a2_a2*H_a3_p2*H_p1_p2/60 -
169*H_a1_a3*H_a1_p1*H_a2_a2*H_a3_p2*H_p2_p2/120 - H_a1_a3*H_a1_p1*H_a2_a3*H_a2_p1*H_p1_p1/12 - 7*H_a1_a3*H_a1_p1*H_a2_a3*H_a2_p1*H_p1_p2/6 -
97*H_a1_a3*H_a1_p1*H_a2_a3*H_a2_p1*H_p2_p2/12 - H_a1_a3*H_a1_p1*H_a2_a3*H_a2_p2*H_p1_p1/3 + 41*H_a1_a3*H_a1_p1*H_a2_a3*H_a2_p2*H_p1_p2/6 -
H_a1_a3*H_a1_p1*H_a2_a3*H_a2_p2*H_p2_p2/3 + 5*H_a1_a3*H_a1_p1*H_a2_p1**2*H_a3_p2/6 + 17*H_a1_a3*H_a1_p1*H_a2_p1*H_a2_p2*H_a3_p2/2 +
2*H_a1_a3*H_a1_p1*H_a2_p1*H_p1_p1*T_a2_a3/3 - H_a1_a3*H_a1_p1*H_a2_p1*H_p1_p2*T_a2_a3/2 - 91*H_a1_a3*H_a1_p1*H_a2_p1*H_p2_p2*T_a2_a3/12 -
13*H_a1_a3*H_a1_p1*H_a2_p2**2*H_a3_p1/2 + 4*H_a1_a3*H_a1_p1*H_a2_p2**2*H_a3_p2/3 + H_a1_a3*H_a1_p1*H_a2_p2*H_p1_p1*T_a2_a3/12 +
15*H_a1_a3*H_a1_p1*H_a2_p2*H_p1_p2*T_a2_a3/2 + H_a1_a3*H_a1_p1*H_a2_p2*H_p2_p2*T_a2_a3/12 + 31*H_a1_a3*H_a1_p1*H_a3_p1*H_p1_p1*T_a2_a2/60 +
31*H_a1_a3*H_a1_p1*H_a3_p1*H_p1_p2*T_a2_a2/60 + 751*H_a1_a3*H_a1_p1*H_a3_p1*H_p2_p2*T_a2_a2/120 + H_a1_a3*H_a1_p1*H_a3_p2*H_p1_p1*T_a2_a2/120 -
163*H_a1_a3*H_a1_p1*H_a3_p2*H_p1_p2*T_a2_a2/20 - 63*H_a1_a3*H_a1_p1*H_a3_p2*H_p2_p2*T_a2_a2/40 + 121*H_a1_a3*H_a1_p2*H_a2_a2*H_a3_p1*H_p1_p1/120 -
419*H_a1_a3*H_a1_p2*H_a2_a2*H_a3_p1*H_p1_p2/60 + 11*H_a1_a3*H_a1_p2*H_a2_a2*H_a3_p1*H_p2_p2/120 + 347*H_a1_a3*H_a1_p2*H_a2_a2*H_a3_p2*H_p1_p1/40 +
41*H_a1_a3*H_a1_p2*H_a2_a2*H_a3_p2*H_p1_p2/60 + 11*H_a1_a3*H_a1_p2*H_a2_a2*H_a3_p2*H_p2_p2/120 + 5*H_a1_a3*H_a1_p2*H_a2_a3*H_a2_p1*H_p1_p1/4 +
53*H_a1_a3*H_a1_p2*H_a2_a3*H_a2_p1*H_p1_p2/6 + 5*H_a1_a3*H_a1_p2*H_a2_a3*H_a2_p1*H_p2_p2/4 - 97*H_a1_a3*H_a1_p2*H_a2_a3*H_a2_p2*H_p1_p1/12 -
7*H_a1_a3*H_a1_p2*H_a2_a3*H_a2_p2*H_p1_p2/6 - H_a1_a3*H_a1_p2*H_a2_a3*H_a2_p2*H_p2_p2/12 - 5*H_a1_a3*H_a1_p2*H_a2_p1**2*H_a3_p1/6 -
25*H_a1_a3*H_a1_p2*H_a2_p1**2*H_a3_p2/3 + 41*H_a1_a3*H_a1_p2*H_a2_p1*H_a2_p2*H_a3_p1/6 - 2*H_a1_a3*H_a1_p2*H_a2_p1*H_a2_p2*H_a3_p2/3 +
5*H_a1_a3*H_a1_p2*H_a2_p1*H_p1_p1*T_a2_a3/3 + 28*H_a1_a3*H_a1_p2*H_a2_p1*H_p1_p2*T_a2_a3/3 + 19*H_a1_a3*H_a1_p2*H_a2_p1*H_p2_p2*T_a2_a3/12 -
H_a1_a3*H_a1_p2*H_a2_p2**2*H_a3_p1/6 - H_a1_a3*H_a1_p2*H_a2_p2**2*H_a3_p2/3 - 89*H_a1_a3*H_a1_p2*H_a2_p2*H_p1_p1*T_a2_a3/12 -
H_a1_a3*H_a1_p2*H_a2_p2*H_p1_p2*T_a2_a3/3 + 5*H_a1_a3*H_a1_p2*H_a2_p2*H_p2_p2*T_a2_a3/6 + 61*H_a1_a3*H_a1_p2*H_a3_p1*H_p1_p1*T_a2_a2/120 -
153*H_a1_a3*H_a1_p2*H_a3_p1*H_p1_p2*T_a2_a2/20 - 13*H_a1_a3*H_a1_p2*H_a3_p1*H_p2_p2*T_a2_a2/40 + 961*H_a1_a3*H_a1_p2*H_a3_p2*H_p1_p1*T_a2_a2/120 -
3*H_a1_a3*H_a1_p2*H_a3_p2*H_p1_p2*T_a2_a2/20 - 89*H_a1_a3*H_a1_p2*H_a3_p2*H_p2_p2*T_a2_a2/120 - 2*H_a1_a3*H_a2_a2*H_p1_p1**2*T_a1_a3/3 -
4*H_a1_a3*H_a2_a2*H_p1_p1*H_p1_p2*T_a1_a3/3 - 13*H_a1_a3*H_a2_a2*H_p1_p1*H_p2_p2*T_a1_a3/2 + 47*H_a1_a3*H_a2_a2*H_p1_p2**2*T_a1_a3/6 +
2*H_a1_a3*H_a2_a2*H_p1_p2*H_p2_p2*T_a1_a3 + H_a1_a3*H_a2_a2*H_p2_p2**2*T_a1_a3/2 - H_a1_a3*H_a2_a3*H_p1_p1**2*T_a1_a2/2 -
5*H_a1_a3*H_a2_a3*H_p1_p1*H_p1_p2*T_a1_a2/6 + 22*H_a1_a3*H_a2_a3*H_p1_p1*H_p2_p2*T_a1_a2/3 - 26*H_a1_a3*H_a2_a3*H_p1_p2**2*T_a1_a2/3 -
5*H_a1_a3*H_a2_a3*H_p1_p2*H_p2_p2*T_a1_a2/6 - H_a1_a3*H_a2_a3*H_p2_p2**2*T_a1_a2/2 - 5*H_a1_a3*H_a2_p1**2*H_p1_p1*T_a1_a3/6 -
5*H_a1_a3*H_a2_p1**2*H_p1_p2*T_a1_a3/6 + 43*H_a1_a3*H_a2_p1**2*H_p2_p2*T_a1_a3/6 - 5*H_a1_a3*H_a2_p1*H_a2_p2*H_p1_p1*T_a1_a3/6 -
101*H_a1_a3*H_a2_p1*H_a2_p2*H_p1_p2*T_a1_a3/6 - 5*H_a1_a3*H_a2_p1*H_a2_p2*H_p2_p2*T_a1_a3/6 + 2*H_a1_a3*H_a2_p1*H_a3_p1*H_p1_p1*T_a1_a2/3 +
11*H_a1_a3*H_a2_p1*H_a3_p1*H_p1_p2*T_a1_a2/6 - 77*H_a1_a3*H_a2_p1*H_a3_p1*H_p2_p2*T_a1_a2/12 - H_a1_a3*H_a2_p1*H_a3_p2*H_p1_p1*T_a1_a2 +
47*H_a1_a3*H_a2_p1*H_a3_p2*H_p1_p2*T_a1_a2/6 - H_a1_a3*H_a2_p1*H_a3_p2*H_p2_p2*T_a1_a2/3 + 41*H_a1_a3*H_a2_p2**2*H_p1_p1*T_a1_a3/6 -
7*H_a1_a3*H_a2_p2**2*H_p1_p2*T_a1_a3/6 - 5*H_a1_a3*H_a2_p2**2*H_p2_p2*T_a1_a3/12 + 7*H_a1_a3*H_a2_p2*H_a3_p1*H_p1_p1*T_a1_a2/12 +
55*H_a1_a3*H_a2_p2*H_a3_p1*H_p1_p2*T_a1_a2/6 + 11*H_a1_a3*H_a2_p2*H_a3_p1*H_p2_p2*T_a1_a2/12 - 17*H_a1_a3*H_a2_p2*H_a3_p2*H_p1_p1*T_a1_a2/2 +
2*H_a1_a3*H_a2_p2*H_a3_p2*H_p1_p2*T_a1_a2/3 + 3*H_a1_a3*H_a2_p2*H_a3_p2*H_p2_p2*T_a1_a2/4 - 7*H_a1_a3*H_p1_p1**2*T_a1_a2*T_a2_a3/6 -
H_a1_a3*H_p1_p1**2*T_a1_a3*T_a2_a2/12 - 7*H_a1_a3*H_p1_p1*H_p1_p2*T_a1_a2*T_a2_a3/3 - H_a1_a3*H_p1_p1*H_p1_p2*T_a1_a3*T_a2_a2/6 +
37*H_a1_a3*H_p1_p1*H_p2_p2*T_a1_a2*T_a2_a3/6 - 11*H_a1_a3*H_p1_p1*H_p2_p2*T_a1_a3*T_a2_a2/2 - 59*H_a1_a3*H_p1_p2**2*T_a1_a2*T_a2_a3/6 +
26*H_a1_a3*H_p1_p2**2*T_a1_a3*T_a2_a2/3 - 5*H_a1_a3*H_p1_p2*H_p2_p2*T_a1_a2*T_a2_a3/2 + 13*H_a1_a3*H_p1_p2*H_p2_p2*T_a1_a3*T_a2_a2/4 -
4*H_a1_a3*H_p2_p2**2*T_a1_a2*T_a2_a3/3 + 11*H_a1_a3*H_p2_p2**2*T_a1_a3*T_a2_a2/8 + 11*H_a1_p1**2*H_a2_a2*H_a3_a3*H_p1_p1/60 +
16*H_a1_p1**2*H_a2_a2*H_a3_a3*H_p1_p2/15 - 577*H_a1_p1**2*H_a2_a2*H_a3_a3*H_p2_p2/240 - 7*H_a1_p1**2*H_a2_a2*H_a3_p1**2/10 -
41*H_a1_p1**2*H_a2_a2*H_a3_p1*H_a3_p2/30 + 97*H_a1_p1**2*H_a2_a2*H_a3_p2**2/30 + 9*H_a1_p1**2*H_a2_a2*H_p1_p1*T_a3_a3/20 +
31*H_a1_p1**2*H_a2_a2*H_p1_p2*T_a3_a3/24 - 13*H_a1_p1**2*H_a2_a2*H_p2_p2*T_a3_a3/6 + 5*H_a1_p1**2*H_a2_a3**2*H_p1_p1/24 -
```

```
H_a1_p1**2*H_a2_a3**2*H_p1_p2/4 + 11*H_a1_p1**2*H_a2_a3**2*H_p2_p2/3 - H_a1_p1**2*H_a2_a3*H_a2_p1*H_a3_p1/3 - H_a1_p1**2*H_a2_a3*H_a2_p1*H_a3_p2/6 +
4*H_a1_p1**2*H_a2_a3*H_a2_p2*H_a3_p1/3 - 13*H_a1_p1**2*H_a2_a3*H_a2_p2*H_a3_p2/2 - 5*H_a1_p1**2*H_a2_a3*H_p1_p1*T_a2_a3/12 -
7*H_a1_p1**2*H_a2_a3*H_p1_p2*T_a2_a3/6 + 41*H_a1_p1**2*H_a2_a3*H_p2_p2*T_a2_a3/6 + 7*H_a1_p1**2*H_a2_p1**2*H_a3_a3/30 +
7*H_a1_p1**2*H_a2_p1**2*T_a3_a3/30 - 3*H_a1_p1**2*H_a2_p1*H_a2_p2*H_a3_a3/4 - 3*H_a1_p1**2*H_a2_p1*H_a2_p2*T_a3_a3/4 -
H_a1_p1**2*H_a2_p1*H_a3_p1*T_a2_a3/3 - H_a1_p1**2*H_a2_p1*H_a3_p2*T_a2_a3/6 + 9*H_a1_p1**2*H_a2_p2**2*H_a3_a3/4 + 9*H_a1_p1**2*H_a2_p2**2*T_a3_a3/4 +
4*H_a1_p1**2*H_a2_p2*H_a3_p1*T_a2_a3/3 - 13*H_a1_p1**2*H_a2_p2*H_a3_p2*T_a2_a3/2 + 17*H_a1_p1**2*H_a3_a3*H_p1_p1*T_a2_a2/30 +
173*H_a1_p1**2*H_a3_a3*H_p1_p2*T_a2_a2/120 - 87*H_a1_p1**2*H_a3_a3*H_p2_p2*T_a2_a2/40 - 7*H_a1_p1**2*H_a3_p1**2*T_a2_a2/10 -
41*H_a1_p1**2*H_a3_p1*H_a3_p2*T_a2_a2/30 + 97*H_a1_p1**2*H_a3_p2**2*T_a2_a2/30 + 5*H_a1_p1**2*H_p1_p1*T_a2_a2*T_a3_a3/6 -
5*H_a1_p1**2*H_p1_p1*T_a2_a3**2/8 + 5*H_a1_p1**2*H_p1_p2*T_a2_a2*T_a3_a3/3 - 11*H_a1_p1**2*H_p1_p2*T_a2_a3**2/12 -
31*H_a1_p1**2*H_p2_p2*T_a2_a2*T_a3_a3/16 + 19*H_a1_p1**2*H_p2_p2*T_a2_a3**2/6 - 39*H_a1_p1*H_a1_p2*H_a2_a2*H_a3_a3*H_p1_p1/40 +
34*H_a1_p1*H_a1_p2*H_a2_a2*H_a3_a3*H_p1_p2/5 - 17*H_a1_p1*H_a1_p2*H_a2_a2*H_a3_a3*H_p2_p2/120 - H_a1_p1*H_a1_p2*H_a2_a2*H_a3_p1**2/30 -
217*H_a1_p1*H_a1_p2*H_a2_a2*H_a3_p1*H_a3_p2/30 + 73*H_a1_p1*H_a1_p2*H_a2_a2*H_a3_p2**2/60 - 2*H_a1_p1*H_a1_p2*H_a2_a2*H_p1_p1*T_a3_a3/3 +
29*H_a1_p1*H_a1_p2*H_a2_a2*H_p1_p2*T_a3_a3/4 + H_a1_p1*H_a1_p2*H_a2_a2*H_p2_p2*T_a3_a3/4 - 47*H_a1_p1*H_a1_p2*H_a2_a3**2*H_p1_p2/6 -
2*H_a1_p1*H_a1_p2*H_a2_a3*H_a2_p1*H_a3_p1/3 + 41*H_a1_p1*H_a1_p2*H_a2_a3*H_a2_p1*H_a3_p2/6 + 17*H_a1_p1*H_a1_p2*H_a2_a3*H_a2_p2*H_a3_p1/2 -
5*H_a1_p1*H_a1_p2*H_a2_a3*H_p1_p1*T_a2_a3/6 - 101*H_a1_p1*H_a1_p2*H_a2_a3*H_p1_p2*T_a2_a3/6 - 5*H_a1_p1*H_a1_p2*H_a2_a3*H_p2_p2*T_a2_a3/6 +
13*H_a1_p1*H_a1_p2*H_a2_p1**2*H_a3_a3/12 + 13*H_a1_p1*H_a1_p2*H_a2_p1**2*T_a3_a3/12 - 15*H_a1_p1*H_a1_p2*H_a2_p1*H_a2_p2*H_a3_a3/2 -
15*H_a1_p1*H_a1_p2*H_a2_p1*H_a2_p2*T_a3_a3/2 - 2*H_a1_p1*H_a1_p2*H_a2_p1*H_a3_p1*T_a2_a3/3 + 41*H_a1_p1*H_a1_p2*H_a2_p1*H_a3_p2*T_a2_a3/6 -
3*H_a1_p1*H_a1_p2*H_a2_p2**2*H_a3_a3/4 - 3*H_a1_p1*H_a1_p2*H_a2_p2**2*T_a3_a3/4 + 17*H_a1_p1*H_a1_p2*H_a2_p2*H_a3_p1*T_a2_a3/2 -
31*H_a1_p1*H_a1_p2*H_a3_a3*H_p1_p1*T_a2_a2/60 + 443*H_a1_p1*H_a1_p2*H_a3_a3*H_p1_p2*T_a2_a2/60 + 7*H_a1_p1*H_a1_p2*H_a3_a3*H_p2_p2*T_a2_a2/30 -
H_a1_p1*H_a1_p2*H_a3_p1**2*T_a2_a2/30 - 217*H_a1_p1*H_a1_p2*H_a3_p1*H_a3_p2*T_a2_a2/30 + 73*H_a1_p1*H_a1_p2*H_a3_p2**2*T_a2_a2/60 -
5*H_a1_p1*H_a1_p2*H_p1_p1*T_a2_a2*T_a3_a3/24 - 5*H_a1_p1*H_a1_p2*H_p1_p1*T_a2_a3**2/6 + 47*H_a1_p1*H_a1_p2*H_p1_p2*T_a2_a2*T_a3_a3/6 -
9*H_a1_p1*H_a1_p2*H_p1_p2*T_a2_a3**2 + 5*H_a1_p1*H_a1_p2*H_p2_p2*T_a2_a2*T_a3_a3/8 - 5*H_a1_p1*H_a1_p2*H_p2_p2*T_a2_a3**2/6 +
H_a1_p1*H_a2_a2*H_a3_p1*H_p1_p1*T_a1_a3/2 + H_a1_p1*H_a2_a2*H_a3_p1*H_p1_p2*T_a1_a3/2 + 73*H_a1_p1*H_a2_a2*H_a3_p1*H_p2_p2*T_a1_a3/12 -
H_a1_p1*H_a2_a2*H_a3_p2*H_p1_p1*T_a1_a3/4 - 17*H_a1_p1*H_a2_a2*H_a3_p2*H_p1_p2*T_a1_a3/2 - 15*H_a1_p1*H_a2_a2*H_a3_p2*H_p2_p2*T_a1_a3/8 +
5*H_a1_p1*H_a2_a3*H_a2_p1*H_p1_p1*T_a1_a3/6 - H_a1_p1*H_a2_a3*H_a2_p1*H_p1_p2*T_a1_a3/3 - 89*H_a1_p1*H_a2_a3*H_a2_p1*H_p2_p2*T_a1_a3/12 +
H_a1_p1*H_a2_a3*H_a2_p2*H_p1_p1*T_a1_a3/12 + 15*H_a1_p1*H_a2_a3*H_a2_p2*H_p1_p2*T_a1_a3/2 + H_a1_p1*H_a2_a3*H_a2_p2*H_p2_p2*T_a1_a3/12 +
3*H_a1_p1*H_a2_a3*H_a3_p1*H_p1_p1*T_a1_a2/4 + 2*H_a1_p1*H_a2_a3*H_a3_p1*H_p1_p2*T_a1_a2/3 - 17*H_a1_p1*H_a2_a3*H_a3_p1*H_p2_p2*T_a1_a2/2 +
11*H_a1_p1*H_a2_a3*H_a3_p2*H_p1_p1*T_a1_a2/12 + 55*H_a1_p1*H_a2_a3*H_a3_p2*H_p1_p2*T_a1_a2/6 + 7*H_a1_p1*H_a2_a3*H_a3_p2*H_p2_p2*T_a1_a2/12 +
5*H_a1_p1*H_a2_p1**2*H_a3_p2*T_a1_a3/6 + 17*H_a1_p1*H_a2_p1*H_a2_p2*H_a3_p2*T_a1_a3/2 - 19*H_a1_p1*H_a2_p1*H_a3_a3*H_p1_p1*T_a1_a2/24 -
17*H_a1_p1*H_a2_p1*H_a3_a3*H_p1_p2*T_a1_a2/12 + 179*H_a1_p1*H_a2_p1*H_a3_a3*H_p2_p2*T_a1_a2/24 - H_a1_p1*H_a2_p1*H_a3_p1**2*T_a1_a2/3 -
8*H_a1_p1*H_a2_p1*H_a3_p2**2*T_a1_a2 - 13*H_a1_p1*H_a2_p1*H_p1_p1*T_a1_a2*T_a3_a3/8 + 19*H_a1_p1*H_a2_p1*H_p1_p1*T_a1_a3*T_a2_a3/12 -
9*H_a1_p1*H_a2_p1*H_p1_p2*T_a1_a2*T_a3_a3/4 + H_a1_p1*H_a2_p1*H_p1_p2*T_a1_a3*T_a2_a3/3 + 163*H_a1_p1*H_a2_p1*H_p2_p2*T_a1_a2*T_a3_a3/24 -
83*H_a1_p1*H_a2_p1*H_p2_p2*T_a1_a3*T_a2_a3/12 - 13*H_a1_p1*H_a2_p2**2*H_a3_p1*T_a1_a3/2 + 4*H_a1_p1*H_a2_p2**2*H_a3_p2*T_a1_a3/3 +
13*H_a1_p1*H_a2_p2*H_a3_a3*H_p1_p1*T_a1_a2/12 - 37*H_a1_p1*H_a2_p2*H_a3_a3*H_p1_p2*T_a1_a2/6 + 13*H_a1_p1*H_a2_p2*H_a3_a3*H_p2_p2*T_a1_a2/12 -
5*H_a1_p1*H_a2_p2*H_a3_p1**2*T_a1_a2/3 + 35*H_a1_p1*H_a2_p2*H_a3_p1*H_a3_p2*T_a1_a2/6 - 5*H_a1_p1*H_a2_p2*H_a3_p2**2*T_a1_a2/3 +
5*H_a1_p1*H_a2_p2*H_p1_p1*T_a1_a2*T_a3_a3/12 + H_a1_p1*H_a2_p2*H_p1_p1*T_a1_a3*T_a2_a3/2 - 7*H_a1_p1*H_a2_p2*H_p1_p2*T_a1_a2*T_a3_a3 +
49*H_a1_p1*H_a2_p2*H_p1_p2*T_a1_a3*T_a2_a3/6 + 5*H_a1_p1*H_a2_p2*H_p2_p2*T_a1_a2*T_a3_a3/12 + H_a1_p1*H_a2_p2*H_p2_p2*T_a1_a3*T_a2_a3/2 +
5*H_a1_p1*H_a3_p1*H_p1_p1*T_a1_a2*T_a2_a3/3 - H_a1_p1*H_a3_p1*H_p1_p1*T_a1_a3*T_a2_a2/6 + 5*H_a1_p1*H_a3_p1*H_p1_p2*T_a1_a2*T_a2_a3/3 -
H_a1_p1*H_a3_p1*H_p1_p2*T_a1_a3*T_a2_a2/6 - 47*H_a1_p1*H_a3_p1*H_p2_p2*T_a1_a2*T_a2_a3/6 + 17*H_a1_p1*H_a3_p1*H_p2_p2*T_a1_a3*T_a2_a2/3 +
3*H_a1_p1*H_a3_p2*H_p1_p1*T_a1_a2*T_a2_a3/2 - 5*H_a1_p1*H_a3_p2*H_p1_p1*T_a1_a3*T_a2_a2/12 + 10*H_a1_p1*H_a3_p2*H_p1_p2*T_a1_a2*T_a2_a3 -
53*H_a1_p1*H_a3_p2*H_p1_p2*T_a1_a3*T_a2_a2/6 + 13*H_a1_p1*H_a3_p2*H_p2_p2*T_a1_a2*T_a2_a3/12 - 49*H_a1_p1*H_a3_p2*H_p2_p2*T_a1_a3*T_a2_a2/24 -
1067*H_a1_p2**2*H_a2_a2*H_a3_a3*H_p1_p1/240 - 17*H_a1_p2**2*H_a2_a2*H_a3_a3*H_p1_p2/120 - 17*H_a1_p2**2*H_a2_a2*H_a3_a3*H_p2_p2/120 +
39*H_a1_p2**2*H_a2_a2*H_a3_p1**2/10 - 37*H_a1_p2**2*H_a2_a2*H_a3_p1*H_a3_p2/60 + 7*H_a1_p2**2*H_a2_a2*H_a3_p2**2/30 -
25*H_a1_p2**2*H_a2_a2*H_p1_p1*T_a3_a3/6 + H_a1_p2**2*H_a2_a2*H_p1_p2*T_a3_a3/6 + 29*H_a1_p2**2*H_a2_a2*H_p2_p2*T_a3_a3/120 +
47*H_a1_p2**2*H_a2_a3**2*H_p1_p1/12 - 25*H_a1_p2**2*H_a2_a3*H_a2_p1*H_a3_p1/3 - 5*H_a1_p2**2*H_a2_a3*H_a2_p1*H_a3_p2/6 +
5*H_a1_p2**2*H_a2_a3*H_a2_p2*H_a3_p1/6 + 43*H_a1_p2**2*H_a2_a3*H_p1_p1*T_a2_a3/6 - 5*H_a1_p2**2*H_a2_a3*H_p1_p2*T_a2_a3/6 -
5*H_a1_p2**2*H_a2_a3*H_p2_p2*T_a2_a3/6 + 19*H_a1_p2**2*H_a2_p1**2*H_a3_a3/4 + 19*H_a1_p2**2*H_a2_p1**2*T_a3_a3/4 +
13*H_a1_p2**2*H_a2_p1*H_a2_p2*H_a3_a3/12 + 13*H_a1_p2**2*H_a2_p1*H_a2_p2*T_a3_a3/12 - 25*H_a1_p2**2*H_a2_p1*H_a3_p1*T_a2_a3/3 -
5*H_a1_p2**2*H_a2_p1*H_a3_p2*T_a2_a3/6 + 7*H_a1_p2**2*H_a2_p2**2*H_a3_a3/30 + 7*H_a1_p2**2*H_a2_p2**2*T_a3_a3/30 +
5*H_a1_p2**2*H_a2_p2*H_a3_p1*T_a2_a3/6 - 491*H_a1_p2**2*H_a3_a3*H_p1_p1*T_a2_a2/120 + 19*H_a1_p2**2*H_a3_a3*H_p1_p2*T_a2_a2/60 +
29*H_a1_p2**2*H_a3_a3*H_p2_p2*T_a2_a2/120 + 39*H_a1_p2**2*H_a3_p1**2*T_a2_a2/10 - 37*H_a1_p2**2*H_a3_p1*H_a3_p2*T_a2_a2/60 +
7*H_a1_p2**2*H_a3_p2**2*T_a2_a2/30 - 61*H_a1_p2**2*H_p1_p1*T_a2_a2*T_a3_a3/16 + 13*H_a1_p2**2*H_p1_p1*T_a2_a3**2/4 +
5*H_a1_p2**2*H_p1_p2*T_a2_a2*T_a3_a3/8 - 5*H_a1_p2**2*H_p1_p2*T_a2_a3**2/6 + 5*H_a1_p2**2*H_p2_p2*T_a2_a2*T_a3_a3/8 -
5*H_a1_p2**2*H_p2_p2*T_a2_a3**2/6 + 3*H_a1_p2*H_a2_a2*H_a3_p1*H_p1_p1*T_a1_a3/4 - 15*H_a1_p2*H_a2_a2*H_a3_p1*H_p1_p2*T_a1_a3/2 -
7*H_a1_p2*H_a2_a2*H_a3_p1*H_p2_p2*T_a1_a3/24 + 97*H_a1_p2*H_a2_a2*H_a3_p2*H_p1_p1*T_a1_a3/12 - H_a1_p2*H_a2_a2*H_a3_p2*H_p1_p2*T_a1_a3/6 -
19*H_a1_p2*H_a2_a2*H_a3_p2*H_p2_p2*T_a1_a3/24 + 19*H_a1_p2*H_a2_a3*H_a2_p1*H_p1_p1*T_a1_a3/12 + 28*H_a1_p2*H_a2_a3*H_a2_p1*H_p1_p2*T_a1_a3/3 +
5*H_a1_p2*H_a2_a3*H_a2_p1*H_p2_p2*T_a1_a3/3 - 91*H_a1_p2*H_a2_a3*H_a2_p2*H_p1_p1*T_a1_a3/12 - H_a1_p2*H_a2_a3*H_a2_p2*H_p1_p2*T_a1_a3/2 +
2*H_a1_p2*H_a2_a3*H_a2_p2*H_p2_p2*T_a1_a3/3 - H_a1_p2*H_a2_a3*H_a3_p1*H_p1_p1*T_a1_a2/3 + 47*H_a1_p2*H_a2_a3*H_a3_p1*H_p1_p2*T_a1_a2/6 -
H_a1_p2*H_a2_a3*H_a3_p1*H_p2_p2*T_a1_a2 - 77*H_a1_p2*H_a2_a3*H_a3_p2*H_p1_p1*T_a1_a2/12 + 11*H_a1_p2*H_a2_a3*H_a3_p2*H_p1_p2*T_a1_a2/6 +
2*H_a1_p2*H_a2_a3*H_a3_p2*H_p2_p2*T_a1_a2/3 - 5*H_a1_p2*H_a2_p1**2*H_a3_p1*T_a1_a3/6 - 25*H_a1_p2*H_a2_p1**2*H_a3_p2*T_a1_a3/3 +
41*H_a1_p2*H_a2_p1*H_a2_p2*H_a3_p1*T_a1_a3/6 - 2*H_a1_p2*H_a2_p1*H_a2_p2*H_a3_p2*T_a1_a3/3 - H_a1_p2*H_a2_p1*H_a3_a3*H_p1_p1*T_a1_a2/2 -
25*H_a1_p2*H_a2_p1*H_a3_a3*H_p1_p2*T_a1_a2/3 - H_a1_p2*H_a2_p1*H_a3_a3*H_p2_p2*T_a1_a2/2 - H_a1_p2*H_a2_p1*H_a3_p1**2*T_a1_a2/6 +
43*H_a1_p2*H_a2_p1*H_a3_p1*H_a3_p2*T_a1_a2/6 - H_a1_p2*H_a2_p1*H_a3_p2**2*T_a1_a2/6 - 11*H_a1_p2*H_a2_p1*H_p1_p1*T_a1_a2*T_a3_a3/12 +
2*H_a1_p2*H_a2_p1*H_p1_p1*T_a1_a3*T_a2_a3 - 9*H_a1_p2*H_a2_p1*H_p1_p2*T_a1_a2*T_a3_a3 + 59*H_a1_p2*H_a2_p1*H_p1_p2*T_a1_a3*T_a2_a3/6 -
11*H_a1_p2*H_a2_p1*H_p2_p2*T_a1_a2*T_a3_a3/12 + 2*H_a1_p2*H_a2_p1*H_p2_p2*T_a1_a3*T_a2_a3 - H_a1_p2*H_a2_p2**2*H_a3_p1*T_a1_a3/6 -
H_a1_p2*H_a2_p2**2*H_a3_p2*T_a1_a3/3 + 179*H_a1_p2*H_a2_p2*H_a3_a3*H_p1_p1*T_a1_a2/24 - 17*H_a1_p2*H_a2_p2*H_a3_a3*H_p1_p2*T_a1_a2/12 -
19*H_a1_p2*H_a2_p2*H_a3_a3*H_p2_p2*T_a1_a2/24 - 8*H_a1_p2*H_a2_p2*H_a3_p1**2*T_a1_a2 - H_a1_p2*H_a2_p2*H_a3_p2**2*T_a1_a2/3 +
163*H_a1_p2*H_a2_p2*H_p1_p1*T_a1_a2*T_a3_a3/24 - 83*H_a1_p2*H_a2_p2*H_p1_p1*T_a1_a3*T_a2_a3/12 - 9*H_a1_p2*H_a2_p2*H_p1_p2*T_a1_a2*T_a3_a3/4 +
H_a1_p2*H_a2_p2*H_p1_p2*T_a1_a3*T_a2_a3/3 - 13*H_a1_p2*H_a2_p2*H_p2_p2*T_a1_a2*T_a3_a3/8 + 19*H_a1_p2*H_a2_p2*H_p2_p2*T_a1_a3*T_a2_a3/12 +
H_a1_p2*H_a3_p1*H_p1_p1*T_a1_a2*T_a2_a3/6 + H_a1_p2*H_a3_p1*H_p1_p1*T_a1_a3*T_a2_a2/4 + 26*H_a1_p2*H_a3_p1*H_p1_p2*T_a1_a2*T_a2_a3/3 -
49*H_a1_p2*H_a3_p1*H_p1_p2*T_a1_a3*T_a2_a2/6 - 5*H_a1_p2*H_a3_p1*H_p2_p2*T_a1_a2*T_a2_a3/12 - 17*H_a1_p2*H_a3_p1*H_p2_p2*T_a1_a3*T_a2_a2/24 -
17*H_a1_p2*H_a3_p2*H_p1_p1*T_a1_a2*T_a2_a3/3 + 89*H_a1_p2*H_a3_p2*H_p1_p1*T_a1_a3*T_a2_a2/12 + 17*H_a1_p2*H_a3_p2*H_p1_p2*T_a1_a2*T_a2_a3/6 -
H_a1_p2*H_a3_p2*H_p1_p2*T_a1_a3*T_a2_a2 + 19*H_a1_p2*H_a3_p2*H_p2_p2*T_a1_a2*T_a2_a3/12 - 13*H_a1_p2*H_a3_p2*H_p2_p2*T_a1_a3*T_a2_a2/8 -
251*H_a2_a2*H_a3_a3*H_p1_p1**2*T_a1_a1/480 - 257*H_a2_a2*H_a3_a3*H_p1_p1*H_p1_p2*T_a1_a1/240 + 499*H_a2_a2*H_a3_a3*H_p1_p1*H_p2_p2*T_a1_a1/240 -
451*H_a2_a2*H_a3_a3*H_p1_p2**2*T_a1_a1/120 - 73*H_a2_a2*H_a3_a3*H_p1_p2*H_p2_p2*T_a1_a1/60 - 73*H_a2_a2*H_a3_a3*H_p2_p2**2*T_a1_a1/120 +
9*H_a2_a2*H_a3_p1**2*H_p1_p1*T_a1_a1/20 - 47*H_a2_a2*H_a3_p1**2*H_p1_p2*T_a1_a1/120 - 361*H_a2_a2*H_a3_p1**2*H_p2_p2*T_a1_a1/120 +
47*H_a2_a2*H_a3_p1*H_a3_p2*H_p1_p1*T_a1_a1/30 + 39*H_a2_a2*H_a3_p1*H_a3_p2*H_p1_p2*T_a1_a1/5 + 33*H_a2_a2*H_a3_p1*H_a3_p2*H_p2_p2*T_a1_a1/20 -
61*H_a2_a2*H_a3_p2**2*H_p1_p1*T_a1_a1/20 - 37*H_a2_a2*H_a3_p2**2*H_p1_p2*T_a1_a1/120 + 17*H_a2_a2*H_a3_p2**2*H_p2_p2*T_a1_a1/30 -
13*H_a2_a2*H_p1_p1**2*T_a1_a1*T_a3_a3/16 - H_a2_a2*H_p1_p1**2*T_a1_a3**2/24 - 13*H_a2_a2*H_p1_p1*H_p1_p2*T_a1_a1*T_a3_a3/8 -
H_a2_a2*H_p1_p1*H_p1_p2*T_a1_a3**2/12 + 19*H_a2_a2*H_p1_p1*H_p2_p2*T_a1_a1*T_a3_a3/12 - 11*H_a2_a2*H_p1_p1*H_p2_p2*T_a1_a3**2/4 -
25*H_a2_a2*H_p1_p2**2*T_a1_a1*T_a3_a3/6 + 13*H_a2_a2*H_p1_p2**2*T_a1_a3**2/3 - 85*H_a2_a2*H_p1_p2*H_p2_p2*T_a1_a1*T_a3_a3/48 +
13*H_a2_a2*H_p1_p2*H_p2_p2*T_a1_a3**2/8 - 43*H_a2_a2*H_p2_p2**2*T_a1_a1*T_a3_a3/48 + 5*H_a2_a2*H_p2_p2**2*T_a1_a3**2/8 +
37*H_a2_a3**2*H_p1_p1**2*T_a1_a1/120 + 59*H_a2_a3**2*H_p1_p1*H_p1_p2*T_a1_a1/60 - 391*H_a2_a3**2*H_p1_p1*H_p2_p2*T_a1_a1/120 +
39*H_a2_a3**2*H_p1_p2**2*T_a1_a1/10 - 41*H_a2_a3**2*H_p1_p2*H_p2_p2*T_a1_a1/60 - 41*H_a2_a3**2*H_p2_p2**2*T_a1_a1/120 -
```

```
89*H_a2_a3*H_a2_p1*H_a3_p1*H_p1_p1*T_a1_a1/120 - 3*H_a2_a3*H_a2_p1*H_a3_p1*H_p1_p2*T_a1_a1/20 + 961*H_a2_a3*H_a2_p1*H_a3_p1*H_p2_p2*T_a1_a1/120 -
13*H_a2_a3*H_a2_p1*H_a3_p2*H_p1_p1*T_a1_a1/40 - 153*H_a2_a3*H_a2_p1*H_a3_p2*H_p1_p2*T_a1_a1/20 + 61*H_a2_a3*H_a2_p1*H_a3_p2*H_p2_p2*T_a1_a1/120 -
63*H_a2_a3*H_a2_p2*H_a3_p1*H_p1_p1*T_a1_a1/40 - 163*H_a2_a3*H_a2_p2*H_a3_p1*H_p1_p2*T_a1_a1/20 + H_a2_a3*H_a2_p2*H_a3_p1*H_p2_p2*T_a1_a1/120 +
751*H_a2_a3*H_a2_p2*H_a3_p2*H_p1_p1*T_a1_a1/120 + 31*H_a2_a3*H_a2_p2*H_a3_p2*H_p1_p2*T_a1_a1/60 + 31*H_a2_a3*H_a2_p2*H_a3_p2*H_p2_p2*T_a1_a1/60 +
11*H_a2_a3*H_p1_p1**2*T_a1_a1*T_a2_a3/8 - 4*H_a2_a3*H_p1_p1**2*T_a1_a2*T_a1_a3/3 + 13*H_a2_a3*H_p1_p1*H_p1_p2*T_a1_a1*T_a2_a3/4 -
5*H_a2_a3*H_p1_p1*H_p1_p2*T_a1_a2*T_a1_a3/2 - 11*H_a2_a3*H_p1_p1*H_p2_p2*T_a1_a1*T_a2_a3/2 + 37*H_a2_a3*H_p1_p1*H_p2_p2*T_a1_a2*T_a1_a3/6 +
26*H_a2_a3*H_p1_p2**2*T_a1_a1*T_a2_a3/3 - 59*H_a2_a3*H_p1_p2**2*T_a1_a2*T_a1_a3/6 - H_a2_a3*H_p1_p2*H_p2_p2*T_a1_a1*T_a2_a3/6 -
7*H_a2_a3*H_p1_p2*H_p2_p2*T_a1_a2*T_a1_a3/3 - H_a2_a3*H_p2_p2**2*T_a1_a1*T_a2_a3/12 - 7*H_a2_a3*H_p2_p2**2*T_a1_a2*T_a1_a3/6 +
29*H_a2_p1**2*H_a3_a3*H_p1_p1*T_a1_a1/120 + 19*H_a2_p1**2*H_a3_a3*H_p1_p2*T_a1_a1/60 - 491*H_a2_p1**2*H_a3_a3*H_p2_p2*T_a1_a1/120 +
7*H_a2_p1**2*H_a3_p1**2*T_a1_a1/30 - 37*H_a2_p1**2*H_a3_p1*H_a3_p2*T_a1_a1/60 + 39*H_a2_p1**2*H_a3_p2**2*T_a1_a1/10 +
5*H_a2_p1**2*H_p1_p1*T_a1_a1*T_a3_a3/8 - 5*H_a2_p1**2*H_p1_p1*T_a1_a3**2/6 + 5*H_a2_p1**2*H_p1_p2*T_a1_a1*T_a3_a3/8 -
5*H_a2_p1**2*H_p1_p2*T_a1_a3**2/6 - 61*H_a2_p1**2*H_p2_p2*T_a1_a1*T_a3_a3/16 + 13*H_a2_p1**2*H_p2_p2*T_a1_a3**2/4 +
7*H_a2_p1*H_a2_p2*H_a3_a3*H_p1_p1*T_a1_a1/30 + 443*H_a2_p1*H_a2_p2*H_a3_a3*H_p1_p2*T_a1_a1/60 - 31*H_a2_p1*H_a2_p2*H_a3_a3*H_p2_p2*T_a1_a1/120 +
73*H_a2_p1*H_a2_p2*H_a3_p1**2*T_a1_a1/60 - 217*H_a2_p1*H_a2_p2*H_a3_p1*H_a3_p2*T_a1_a1/30 - H_a2_p1*H_a2_p2*H_a3_p2**2*T_a1_a1/30 +
5*H_a2_p1*H_a2_p2*H_p1_p1*T_a1_a1*T_a3_a3/8 - 5*H_a2_p1*H_a2_p2*H_p1_p1*T_a1_a3**2/6 + 47*H_a2_p1*H_a2_p2*H_p1_p2*T_a1_a1*T_a3_a3/6 -
9*H_a2_p1*H_a2_p2*H_p1_p2*T_a1_a3**2 - 5*H_a2_p1*H_a2_p2*H_p2_p2*T_a1_a1*T_a3_a3/24 - 5*H_a2_p1*H_a2_p2*H_p2_p2*T_a1_a3**2/6 -
13*H_a2_p1*H_a3_p1*H_p1_p1*T_a1_a1*T_a2_a3/8 + 19*H_a2_p1*H_a3_p1*H_p1_p1*T_a1_a2*T_a1_a3/12 - H_a2_p1*H_a3_p1*H_p1_p2*T_a1_a1*T_a2_a3 +
17*H_a2_p1*H_a3_p1*H_p1_p2*T_a1_a2*T_a1_a3/6 + 89*H_a2_p1*H_a3_p1*H_p2_p2*T_a1_a1*T_a2_a3/12 - 17*H_a2_p1*H_a3_p1*H_p2_p2*T_a1_a2*T_a1_a3/3 -
17*H_a2_p1*H_a3_p2*H_p1_p1*T_a1_a1*T_a2_a3/24 - 5*H_a2_p1*H_a3_p2*H_p1_p1*T_a1_a2*T_a1_a3/12 - 49*H_a2_p1*H_a3_p2*H_p1_p2*T_a1_a1*T_a2_a3/6 +
26*H_a2_p1*H_a3_p2*H_p1_p2*T_a1_a2*T_a1_a3/3 + H_a2_p1*H_a3_p2*H_p2_p2*T_a1_a1*T_a2_a3/4 + H_a2_p1*H_a3_p2*H_p2_p2*T_a1_a2*T_a1_a3/6 -
87*H_a2_p2**2*H_a3_a3*H_p1_p1*T_a1_a1/40 + 173*H_a2_p2**2*H_a3_a3*H_p1_p2*T_a1_a1/120 + 17*H_a2_p2**2*H_a3_a3*H_p2_p2*T_a1_a1/30 +
97*H_a2_p2**2*H_a3_p1**2*T_a1_a1/30 - 41*H_a2_p2**2*H_a3_p1*H_a3_p2*T_a1_a1/30 - 7*H_a2_p2**2*H_a3_p2**2*T_a1_a1/10 -
31*H_a2_p2**2*H_p1_p1*T_a1_a1*T_a3_a3/16 + 19*H_a2_p2**2*H_p1_p1*T_a1_a3**2/6 + 5*H_a2_p2**2*H_p1_p2*T_a1_a1*T_a3_a3/3 -
11*H_a2_p2**2*H_p1_p2*T_a1_a3**2/12 + 5*H_a2_p2**2*H_p2_p2*T_a1_a1*T_a3_a3/6 - 5*H_a2_p2**2*H_p2_p2*T_a1_a3**2/8 -
49*H_a2_p2*H_a3_p1*H_p1_p1*T_a1_a1*T_a2_a3/24 + 13*H_a2_p2*H_a3_p1*H_p1_p1*T_a1_a2*T_a1_a3/12 - 53*H_a2_p2*H_a3_p1*H_p1_p2*T_a1_a1*T_a2_a3/6 +
10*H_a2_p2*H_a3_p1*H_p1_p2*T_a1_a2*T_a1_a3 - 5*H_a2_p2*H_a3_p1*H_p2_p2*T_a1_a1*T_a2_a3/12 + 3*H_a2_p2*H_a3_p1*H_p2_p2*T_a1_a2*T_a1_a3/2 +
17*H_a2_p2*H_a3_p2*H_p1_p1*T_a1_a1*T_a2_a3/3 - 47*H_a2_p2*H_a3_p2*H_p1_p1*T_a1_a2*T_a1_a3/6 - H_a2_p2*H_a3_p2*H_p1_p2*T_a1_a1*T_a2_a3/6 +
5*H_a2_p2*H_a3_p2*H_p1_p2*T_a1_a2*T_a1_a3/3 - H_a2_p2*H_a3_p2*H_p2_p2*T_a1_a1*T_a2_a3/6 + 5*H_a2_p2*H_a3_p2*H_p2_p2*T_a1_a2*T_a1_a3/3 -
43*H_a3_a3*H_p1_p1**2*T_a1_a1*T_a2_a2/48 + 5*H_a3_a3*H_p1_p1**2*T_a1_a2**2/8 - 29*H_a3_a3*H_p1_p1*H_p1_p2*T_a1_a1*T_a2_a2/16 +
7*H_a3_a3*H_p1_p1*H_p1_p2*T_a1_a2**2/8 + 25*H_a3_a3*H_p1_p1*H_p2_p2*T_a1_a1*T_a2_a2/16 - 25*H_a3_a3*H_p1_p1*H_p2_p2*T_a1_a2**2/8 -
101*H_a3_a3*H_p1_p2**2*T_a1_a1*T_a2_a2/24 + 43*H_a3_a3*H_p1_p2**2*T_a1_a2**2/12 - 29*H_a3_a3*H_p1_p2*H_p2_p2*T_a1_a1*T_a2_a2/16 +
7*H_a3_a3*H_p1_p2*H_p2_p2*T_a1_a2**2/8 - 43*H_a3_a3*H_p2_p2**2*T_a1_a1*T_a2_a2/48 + 5*H_a3_a3*H_p2_p2**2*T_a1_a2**2/8 +
5*H_a3_p1**2*H_p1_p1*T_a1_a1*T_a2_a2/6 - 5*H_a3_p1**2*H_p1_p1*T_a1_a2**2/8 - H_a3_p1**2*H_p1_p2*T_a1_a2**2/3 -
133*H_a3_p1**2*H_p2_p2*T_a1_a1*T_a2_a2/48 + 83*H_a3_p1**2*H_p2_p2*T_a1_a2**2/24 + 15*H_a3_p1*H_a3_p2*H_p1_p1*T_a1_a1*T_a2_a2/8 -
5*H_a3_p1*H_a3_p2*H_p1_p1*T_a1_a2**2/12 + 33*H_a3_p1*H_a3_p2*H_p1_p2*T_a1_a1*T_a2_a2/4 - 8*H_a3_p1*H_a3_p2*H_p1_p2*T_a1_a2**2 +
15*H_a3_p1*H_a3_p2*H_p2_p2*T_a1_a1*T_a2_a2/8 - 5*H_a3_p1*H_a3_p2*H_p2_p2*T_a1_a2**2/12 - 133*H_a3_p2**2*H_p1_p1*T_a1_a1*T_a2_a2/48 +
83*H_a3_p2**2*H_p1_p1*T_a1_a2**2/24 - H_a3_p2**2*H_p1_p2*T_a1_a2**2/3 + 5*H_a3_p2**2*H_p2_p2*T_a1_a1*T_a2_a2/6 - 5*H_a3_p2**2*H_p2_p2*T_a1_a2**2/8 -
19*H_p1_p1**2*T_a1_a1*T_a2_a2*T_a3_a3/16 + 17*H_p1_p1**2*T_a1_a1*T_a2_a3**2/16 + 17*H_p1_p1**2*T_a1_a2**2*T_a3_a3/16 -
2*H_p1_p1**2*T_a1_a2*T_a1_a3*T_a2_a3 + H_p1_p1**2*T_a1_a3**2*T_a2_a2/4 - 19*H_p1_p1*H_p1_p2*T_a1_a1*T_a2_a2*T_a3_a3/8 +
9*H_p1_p1*H_p1_p2*T_a1_a1*T_a2_a3**2/4 + 2*H_p1_p1*H_p1_p2*T_a1_a2**2*T_a3_a3 - 4*H_p1_p1*H_p1_p2*T_a1_a2*T_a1_a3*T_a2_a3 +
H_p1_p1*H_p1_p2*T_a1_a3**2*T_a2_a2/2 + 17*H_p1_p1*H_p2_p2*T_a1_a1*T_a2_a2*T_a3_a3/16 - 9*H_p1_p1*H_p2_p2*T_a1_a1*T_a2_a3**2/4 -
19*H_p1_p1*H_p2_p2*T_a1_a2**2*T_a3_a3/8 + 5*H_p1_p1*H_p2_p2*T_a1_a2*T_a1_a3*T_a2_a3 - 9*H_p1_p1*H_p2_p2*T_a1_a3**2*T_a2_a2/4 -
37*H_p1_p2**2*T_a1_a1*T_a2_a2*T_a3_a3/8 + 19*H_p1_p2**2*T_a1_a1*T_a2_a3**2/4 + 9*H_p1_p2**2*T_a1_a2**2*T_a3_a3/2 -
11*H_p1_p2**2*T_a1_a2*T_a1_a3*T_a2_a3 + 19*H_p1_p2**2*T_a1_a3**2*T_a2_a2/4 - 19*H_p1_p2*H_p2_p2*T_a1_a1*T_a2_a2*T_a3_a3/8 +
H_p1_p2*H_p2_p2*T_a1_a1*T_a2_a3**2/2 + 2*H_p1_p2*H_p2_p2*T_a1_a2**2*T_a3_a3 - 4*H_p1_p2*H_p2_p2*T_a1_a2*T_a1_a3*T_a2_a3 +
9*H_p1_p2*H_p2_p2*T_a1_a3**2*T_a2_a2/4 - 19*H_p2_p2**2*T_a1_a1*T_a2_a2*T_a3_a3/16 + H_p2_p2**2*T_a1_a1*T_a2_a3**2/4 +
17*H_p2_p2**2*T_a1_a2**2*T_a3_a3/16 - 2*H_p2_p2**2*T_a1_a2*T_a1_a3*T_a2_a3 + 17*H_p2_p2**2*T_a1_a3**2*T_a2_a2/16",
   "sha256": "b3e94480043d2ea5cbd641196c0bdd43c2d548d2b7686e8fa76c016141377639",
   "n_terms": 779,
   "zero": false,
   "bose_12_symmetric": true,
   "bose_13_symmetric": true,
   "channels_bose_related": true,
   "channel_n_terms": {
    "1": 504,
    "2": 504,
    "3": 787
   },
   "method": "fastpole ring path, V4 own"
  }
 },
 "note": "momentum p3 = -p1-p2 eliminated; Bose symmetry tested under (a1,p1)<->(a2,p2) and (a1,p1)<->(a3,p3) (full S3)"
}
```


ledgers/Q06_milestones.json


```
{
 "landmark": "XVI-Q",
 "release": "Q06",
 "P6_outcome": "Partial",
 "stages": {
  "ghost_rows_4D_closed": true,
  "ghost_rows_evanescent_remainder": "explicit; removable by p22-type evanescent CTs; their kappa^3 s0-consistency OPEN",
  "G10_second_route": true,
  "V4_validated_gauge_identity": true,
  "graphs_evaluated_pole": [
   "G01",
   "G02",
   "G03",
   "G04",
   "G05",
   "G06",
   "G07",
```

```
   "G08",
   "G09",
   "G10",
   "G11",
   "G12",
   "G13",
   "G14",
   "G15"
  ],
  "graphs_missing": [],
  "graphs_derived_zero": [
   "T1",
   "T2",
   "T3"
  ],
  "row_c_phi_phi": "closed at pole level (Q05)",
  "row_c_c_cbar": "closed on 4D part; evanescent remainder OPEN",
  "row_c_c_K": "closed on 4D part; evanescent remainder OPEN",
  "row_h_h_c": "Missing (channel ledger recorded; not assembled)"
 },
 "hard_stops": {
  "bulk_symmetry_restored": false,
  "insertion": "frozen",
  "XV_B_opened": false,
  "XIV_J_opened": false,
  "Route_B_discharged": false,
  "Missing_never_zero": true
 }
}
```


certificates/Q05_P6_status.json


```
{
 "landmark": "XVI-Q",
 "release": "Q05",
 "P6_outcome": "Partial",
 "stages": {
  "rules_V3_ghost_validated": true,
  "graphs_evaluated_pole": [
   "G03",
   "G04",
   "G05",
   "G06",
   "G07",
   "G08",
   "G09",
   "G10",
   "G11",
   "G12",
   "G13",
   "G14",
   "G15"
  ],
  "graphs_missing": [
   "G01",
   "G02"
  ],
  "graphs_derived_zero": [
   "T1",
   "T2",
   "T3"
  ],
  "row_c_phi_phi": "closed at pole level",
  "row_c_c_cbar": "OPEN (unresolved assembly)",
  "row_c_c_K": "OPEN (unresolved assembly)",
  "row_h_h_c": "Missing"
 },
 "counterterms": {
  "h_phi_phi + phi*_c_phi sector": "MS-bar CT = -(pole) of G05+G06+G12+G13 and G11 (local; Q(i) coefficients as certified); its sigma_0 / (S, CT)
consistency in row c.phi.phi is exactly the closed row above",
  "other sectors": "Missing (rows not closed / graphs missing)",
  "kappa3_domain_reduction": "NOT done (structural counts only, Q04)"
 },
 "defect": "c.phi.phi: zero at pole level; c.c.cbar, c.c.K: Open; h.h.c: Missing",
 "sigma0_closure_kappa3": "NOT established (only the c.phi.phi row)",
 "finite_parts": "Missing (pole level only)",
 "O1_closure_unchanged": "CLOSED (scoped)",
 "bulk_symmetry_restored": false,
 "errata": {
  "Q05-E1": "Q04 identity ledger row c.phi.phi omitted the (S1 h-phi-phi x Gamma_Kc) channel (medium; fixed in Q05 rows)",
  "Q05-E2": "code/q04/scalar_loop3.py docstring says G13/G14; the graphs are P6-G14/G15 (info; file is pinned by the Q04 layer gate and left
unchanged)",
  "Q05-E3": "own D13 orientation = -I x XVI-P (convention, disclosed in Q05_rules_validation)",
```

```
  "Q05-E4": "G10 loop momentum shifts had the wrong sign in the first Q05 evaluation (found while assembling the ghost rows; fixed before release;
certified G10 is the corrected one)"
 },
 "review_q04_folded": {
  "guard_scan": "hardened (Q05_guard_scan: recursive, 7 reviewer forms flagged) AND claim downgraded to best-effort policy scan (not a security
boundary)",
  "gate_unpinned_files": "Q05 gate rejects any code/**/*.py, tools/*.py or tests/*.py not pinned",
  "docstring": "Q05-E2"
 },
 "flags": {
  "execution_pass": true,
  "claim_status": "partial",
  "scope": "pole level, one loop, kappa^3"
 }
}
```


certificates/Q05_P6_rows.json


```
{
 "identity": "(S, Gamma_1^pole) restricted to three-field gh1 monomials (Q04 ledger rows)",
 "normalisation": "all amplitudes 'i x coefficient' / 'i Gamma'; antifield-channel signs calibrated on the classical (S0, S1) = 0 (tree, kappa^1;
O.2-verified master equation) and used unchanged",
 "c.phi.phi": {
  "row": "c.phi.phi",
  "calibration_sigma_phistar": "1",
  "tree_identity_closed_with_sigma": true,
  "channels": {
   "S0_hphiphi": "I*H_gc1_p1*H_p1_p1*H_p1_p2/8 - I*H_gc1_p1*H_p1_p1*H_p2_p2/48 + I*H_gc1_p1*H_p1_p2**2/4 - I*H_gc1_p1*H_p2_p2**2/24 -
I*H_gc1_p2*H_p1_p1**2/16 - I*H_gc1_p2*H_p1_p1*H_p1_p2/24 + I*H_gc1_p2*H_p1_p1*H_p2_p2/12 + I*H_gc1_p2*H_p1_p2**2/6 + I*H_gc1_p2*H_p1_p2*H_p2_p2/6",
   "S0_phistar": "I*H_gc1_p1*H_p1_p1*H_p2_p2/12 + I*H_gc1_p1*H_p2_p2**2/24 - I*H_gc1_p2*H_p1_p1**2/8 - I*H_gc1_p2*H_p1_p1*H_p1_p2/3 -
I*H_gc1_p2*H_p1_p1*H_p2_p2/12 - I*H_gc1_p2*H_p1_p2**2/6 - I*H_gc1_p2*H_p1_p2*H_p2_p2/6",
   "S1_Kc": "-I*H_gc1_p1*H_p1_p1*H_p1_p2/8 - I*H_gc1_p1*H_p1_p1*H_p2_p2/16 - I*H_gc1_p1*H_p1_p2**2/4 + 3*I*H_gc1_p2*H_p1_p1**2/16 +
3*I*H_gc1_p2*H_p1_p1*H_p1_p2/8"
  },
  "residual": "0",
  "closed_at_pole_level": true,
  "mutations_detected": {
   "flip_sigma": true,
   "drop_S1_Kc_channel": true,
   "drop_G05": false,
   "drop_G06": true,
   "drop_G12": true,
   "drop_G13": true
  },
  "G05_s0_projection_zero": true,
  "G05_note": "G05's pole is nonzero but its s0-projection vanishes identically, so the row cannot detect dropping G05 (recorded, not a failure; G05
enters other projections)",
  "evanescent_note": "every T structure dies under the s0 polarisation (T(p1, c) = 0): the row must vanish exactly; it does (no evanescent counterterm
needed or available in this row)",
  "ledger_erratum": "Q04 ledger listed only Gamma_phi phi = 0 for (S1, Gamma2); the S1 h-phi-phi vertex x Gamma_K c (D13) channel also enters and is
REQUIRED (dropping it leaves a nonzero residual)"
 },
 "ghost_rows": {
  "c.c.cbar": {
   "tree_sigma_c": "-1",
   "tree_closes_with_constant_sigma": true,
   "loop_residual_n_terms": 44,
   "loop_residual_zero": false,
   "loop_residual_4D_part_zero": false,
   "loop_residual_sha256": "cbffb51d23ad05c203b193387a22e983390d00ab5e117bd678f0e2e44a5a4f3d",
   "channel_terms": {
    "h": 72,
    "K": 28,
    "c1": 68,
    "c2": 36
   },
   "any_channel_coefficient_combination_closes": false,
   "status": "OPEN (unresolved): the assembled one-loop row does not close and no rescaling of its four channels closes it; NOT reported as an
obstruction. Diagnosis so far: the three-point ghost equation holds graph by graph (G04 = -2 G03[K->T], G09 = -2 G07[K->T]), so the cbar-c-h / K-c-h
graphs are mutually consistent; the defect lies in G10 (c* c c; one loop-shift bug already found and fixed in Q05), the K / c channels, or the row
assembly (Grassmann ordering of the two identical c legs). Next step: independent check of G10 and of the c-channel."
  },
  "c.c.K": {
   "tree_sigma_c": "1",
   "tree_closes_with_constant_sigma": true,
   "loop_residual_n_terms": 32,
   "loop_residual_zero": false,
   "loop_residual_4D_part_zero": false,
   "loop_residual_sha256": "094591a4ba9b96762929f1f09f00611e2544bf4ba3859a06d87da817d0b7a45a",
   "channel_terms": {
    "h": 54,
    "K": 16,
    "c1": 54,
```

```
    "c2": 30
   },
   "any_channel_coefficient_combination_closes": false,
   "status": "OPEN (unresolved): the assembled one-loop row does not close and no rescaling of its four channels closes it; NOT reported as an
obstruction. Diagnosis so far: the three-point ghost equation holds graph by graph (G04 = -2 G03[K->T], G09 = -2 G07[K->T]), so the cbar-c-h / K-c-h
graphs are mutually consistent; the defect lies in G10 (c* c c; one loop-shift bug already found and fixed in Q05), the K / c channels, or the row
assembly (Grassmann ordering of the two identical c legs). Next step: independent check of G10 and of the c-channel."
  }
 },
 "h.h.c": {
  "status": "Missing",
  "why": "needs G01 (V3^3; evaluation did not finish within the release budget) and G02 (V4 not constructed)"
 },
 "others": "h.phi.c, phi.b.c, phi.c.cbs, c.c.phis: zero by phi-parity; aux rows: no loop entry (Q04 ledger)"
}
```


certificates/Q05_rules_validation.json


```
{
 "validations": {
  "D01": {
   "mine_d4": "27*H_a1_a1*H_a2_a2*H_p1_p1**2/80 - 19*H_a1_a1*H_a2_p1**2*H_p1_p1/60 + 11*H_a1_a1*H_p1_p1**2*T_a2_a2/48 + 71*H_a1_a2**2*H_p1_p1**2/120 -
11*H_a1_a2*H_a1_p1*H_a2_p1*H_p1_p1/10 + 4*H_a1_a2*H_p1_p1**2*T_a1_a2/3 - 19*H_a1_p1**2*H_a2_a2*H_p1_p1/60 + 13*H_a1_p1**2*H_a2_p1**2/15 -
H_a1_p1**2*H_p1_p1*T_a2_a2/6 - 4*H_a1_p1*H_a2_p1*H_p1_p1*T_a1_a2/3 + 11*H_a2_a2*H_p1_p1**2*T_a1_a1/48 - H_a2_p1**2*H_p1_p1*T_a1_a1/6 +
H_p1_p1**2*T_a1_a1*T_a2_a2/8 + 3*H_p1_p1**2*T_a1_a2**2/4",
   "ratio": "I",
   "constant": true
  },
  "D03": {
   "mine_d4": "-7*H_a1_a1*H_a2_a2*H_p1_p1**2/48 + H_a1_a1*H_a2_p1**2*H_p1_p1/8 - 5*H_a1_a1*H_p1_p1**2*T_a2_a2/48 - H_a1_a2**2*H_p1_p1**2/12 +
H_a1_a2*H_a1_p1*H_a2_p1*H_p1_p1/12 - H_a1_a2*H_p1_p1**2*T_a1_a2/3 + H_a1_p1**2*H_a2_a2*H_p1_p1/8 - H_a1_p1**2*H_a2_p1**2/6 +
H_a1_p1**2*H_p1_p1*T_a2_a2/24 + H_a1_p1*H_a2_p1*H_p1_p1*T_a1_a2/3 - 5*H_a2_a2*H_p1_p1**2*T_a1_a1/48 + H_a2_p1**2*H_p1_p1*T_a1_a1/24 -
H_p1_p1**2*T_a1_a1*T_a2_a2/16 - H_p1_p1**2*T_a1_a2**2/4",
   "ratio": "I",
   "constant": true
  },
  "D06": {
   "mine_d4": "-3*I*H_gb_gc*H_p1_p1**2/8 + I*H_gb_p1*H_gc_p1*H_p1_p1/4 - I*H_p1_p1**2*T_gb_gc/2",
   "ratio": "I",
   "constant": true
  },
  "D13": {
   "mine_d4": "-3*I*H_aK_gc*H_aK_p1*H_p1_p1/8 + I*H_aK_p1**2*H_gc_p1/4 - I*H_aK_p1*H_p1_p1*T_aK_gc/2",
   "ratio": "-I",
   "constant": true
  }
 },
 "expected_ratios": {
  "D01": "I",
  "D03": "I",
  "D06": "I",
  "D13": "-I"
 },
 "all_constant": true,
 "ratios_as_expected": true,
 "convention_I": "mine/XVI-P = I is the Q04-established convention factor (own pole = i x XVI-P pole; D11 in Q04)",
 "D13_orientation_disclosure": "D13 (K'.c) matches XVI-P with constant ratio -I on all structures (T1, T2, T3, E2): an overall sign relative to the I
convention, i.e. the Grassmann ordering/orientation of the K'.c slot differs between the two codes. Own rows use own D13 consistently; the c.phi.phi
row (which uses own D13) closes exactly, so the own orientation is the one consistent with own vertices.",
 "ghost_equation_two_point": {
  "ratio_D06_over_D13_K_to_T": "-2",
  "expected_constant": true,
  "T": "T_mn = sym(k, cbar) - (1/2) eta k.cbar (cbar momentum k)"
 },
 "ghost_equation_three_point": {
  "ratio_G04_over_G03_K_to_T": "-2",
  "ratio_G09_over_G07_K_to_T": "-2",
  "ratio_total": "-2",
  "meaning": "ghost equation (Gamma depends on cbar only through K' = K + dPsi/dh): the cbar-c-h graphs equal -2 x the K-c-h graphs with K -> T(cbar),
graph by graph; same -2 as tree level and two-point"
 },
 "propagators": {
  "graviton": "i/k^2 (I - J/2), Q4S trace 4; derived: EH quadratic gauge invariant, EH + XVI-P gf (+D.D) = k^2 (I - J/2) (XVI-P K = -Box P)",
  "ghost": "-1/k^2 (from i cbar Box c)"
 }
}
```


certificates/Q05_P6_graphs.json


```
{
 "graphs": {
  "G04": {
```

```
   "pole_over_U_d4_residue": "-29*I*H_a3_a3*H_gb2_gc1*H_p1_p1*H_p1_p2/96 + I*H_a3_a3*H_gb2_gc1*H_p1_p1*H_p2_p2/4 -
35*I*H_a3_a3*H_gb2_gc1*H_p1_p2**2/48 + I*H_a3_a3*H_gb2_gc1*H_p1_p2*H_p2_p2/48 + 5*I*H_a3_a3*H_gb2_gc1*H_p2_p2**2/32 +
29*I*H_a3_a3*H_gb2_p1*H_gc1_p1*H_p1_p2/48 + 5*I*H_a3_a3*H_gb2_p1*H_gc1_p1*H_p2_p2/48 - 29*I*H_a3_a3*H_gb2_p1*H_gc1_p2*H_p1_p1/96 -
13*I*H_a3_a3*H_gb2_p1*H_gc1_p2*H_p1_p2/48 - 25*I*H_a3_a3*H_gb2_p1*H_gc1_p2*H_p2_p2/48 - 13*I*H_a3_a3*H_gb2_p2*H_gc1_p1*H_p1_p1/96 +
23*I*H_a3_a3*H_gb2_p2*H_gc1_p1*H_p1_p2/48 + I*H_a3_a3*H_gb2_p2*H_gc1_p1*H_p2_p2/16 - 17*I*H_a3_a3*H_gb2_p2*H_gc1_p2*H_p1_p1/48 -
7*I*H_a3_a3*H_gb2_p2*H_gc1_p2*H_p1_p2/24 - 11*I*H_a3_a3*H_gb2_p2*H_gc1_p2*H_p2_p2/24 - 3*I*H_a3_a3*H_p1_p1*H_p1_p2*T_gb2_gc1/16 +
13*I*H_a3_a3*H_p1_p1*H_p2_p2*T_gb2_gc1/48 - 17*I*H_a3_a3*H_p1_p2**2*T_gb2_gc1/24 - 3*I*H_a3_a3*H_p1_p2*H_p2_p2*T_gb2_gc1/16 +
I*H_a3_gb2*H_a3_gc1*H_p1_p1*H_p1_p2/6 + I*H_a3_gb2*H_a3_gc1*H_p1_p1*H_p2_p2/3 + I*H_a3_gb2*H_a3_gc1*H_p1_p2**2/24 +
I*H_a3_gb2*H_a3_gc1*H_p1_p2*H_p2_p2/8 - 5*I*H_a3_gb2*H_a3_gc1*H_p2_p2**2/24 - 5*I*H_a3_gb2*H_a3_p1*H_gc1_p1*H_p1_p2/24 +
17*I*H_a3_gb2*H_a3_p1*H_gc1_p1*H_p2_p2/48 - 13*I*H_a3_gb2*H_a3_p1*H_gc1_p2*H_p1_p1/48 - I*H_a3_gb2*H_a3_p1*H_gc1_p2*H_p1_p2/4 +
19*I*H_a3_gb2*H_a3_p1*H_gc1_p2*H_p2_p2/48 - I*H_a3_gb2*H_a3_p2*H_gc1_p1*H_p1_p1/24 + 5*I*H_a3_gb2*H_a3_p2*H_gc1_p1*H_p2_p2/16 -
11*I*H_a3_gb2*H_a3_p2*H_gc1_p2*H_p1_p1/16 - 5*I*H_a3_gb2*H_a3_p2*H_gc1_p2*H_p1_p2/8 + 3*I*H_a3_gb2*H_a3_p2*H_gc1_p2*H_p2_p2/16 +
I*H_a3_gb2*H_p1_p1*H_p1_p2*T_a3_gc1/6 + 3*I*H_a3_gb2*H_p1_p1*H_p2_p2*T_a3_gc1/8 - I*H_a3_gb2*H_p1_p2**2*T_a3_gc1/8 +
7*I*H_a3_gb2*H_p1_p2*H_p2_p2*T_a3_gc1/24 - 17*I*H_a3_gc1*H_a3_p1*H_gb2_p1*H_p1_p2/24 - 25*I*H_a3_gc1*H_a3_p1*H_gb2_p1*H_p2_p2/48 +
3*I*H_a3_gc1*H_a3_p1*H_gb2_p2*H_p1_p1/16 - 5*I*H_a3_gc1*H_a3_p1*H_gb2_p2*H_p1_p2/12 - 5*I*H_a3_gc1*H_a3_p1*H_gb2_p2*H_p2_p2/16 +
I*H_a3_gc1*H_a3_p2*H_gb2_p1*H_p1_p1/6 - I*H_a3_gc1*H_a3_p2*H_gb2_p1*H_p1_p2/3 - 7*I*H_a3_gc1*H_a3_p2*H_gb2_p1*H_p2_p2/48 +
7*I*H_a3_gc1*H_a3_p2*H_gb2_p2*H_p1_p1/16 + I*H_a3_gc1*H_a3_p2*H_gb2_p2*H_p1_p2/24 + 5*I*H_a3_gc1*H_a3_p2*H_gb2_p2*H_p2_p2/48 +
I*H_a3_gc1*H_p1_p1*H_p1_p2*T_a3_gb2/24 + I*H_a3_gc1*H_p1_p1*H_p2_p2*T_a3_gb2/12 + 5*I*H_a3_gc1*H_p1_p2**2*T_a3_gb2/24 +
I*H_a3_gc1*H_p1_p2*H_p2_p2*T_a3_gb2/4 - 5*I*H_a3_gc1*H_p2_p2**2*T_a3_gb2/24 + 3*I*H_a3_p1**2*H_gb2_gc1*H_p1_p2/8 -
13*I*H_a3_p1**2*H_gb2_gc1*H_p2_p2/48 + 3*I*H_a3_p1**2*H_gb2_p1*H_gc1_p2/8 + I*H_a3_p1**2*H_gb2_p2*H_gc1_p1/24 + I*H_a3_p1**2*H_gb2_p2*H_gc1_p2/3 +
7*I*H_a3_p1**2*H_p1_p2*T_gb2_gc1/24 - 7*I*H_a3_p1**2*H_p2_p2*T_gb2_gc1/24 - 13*I*H_a3_p1*H_a3_p2*H_gb2_gc1*H_p1_p1/48 +
5*I*H_a3_p1*H_a3_p2*H_gb2_gc1*H_p1_p2/8 - 13*I*H_a3_p1*H_a3_p2*H_gb2_gc1*H_p2_p2/24 - 5*I*H_a3_p1*H_a3_p2*H_gb2_p1*H_gc1_p1/24 +
11*I*H_a3_p1*H_a3_p2*H_gb2_p1*H_gc1_p2/12 - I*H_a3_p1*H_a3_p2*H_gb2_p2*H_gc1_p1/4 + 5*I*H_a3_p1*H_a3_p2*H_gb2_p2*H_gc1_p2/6 -
7*I*H_a3_p1*H_a3_p2*H_p1_p1*T_gb2_gc1/24 + 5*I*H_a3_p1*H_a3_p2*H_p1_p2*T_gb2_gc1/6 - 7*I*H_a3_p1*H_a3_p2*H_p2_p2*T_gb2_gc1/24 -
7*I*H_a3_p1*H_gb2_p1*H_p1_p2*T_a3_gc1/12 - I*H_a3_p1*H_gb2_p1*H_p2_p2*T_a3_gc1/2 + I*H_a3_p1*H_gb2_p2*H_p1_p1*T_a3_gc1/4 -
11*I*H_a3_p1*H_gb2_p2*H_p1_p2*T_a3_gc1/24 - 7*I*H_a3_p1*H_gb2_p2*H_p2_p2*T_a3_gc1/24 - I*H_a3_p1*H_gc1_p1*H_p1_p2*T_a3_gb2/6 +
2*I*H_a3_p1*H_gc1_p1*H_p2_p2*T_a3_gb2/3 - 5*I*H_a3_p1*H_gc1_p2*H_p1_p1*T_a3_gb2/12 - 13*I*H_a3_p1*H_gc1_p2*H_p1_p2*T_a3_gb2/24 +
7*I*H_a3_p1*H_gc1_p2*H_p2_p2*T_a3_gb2/24 - 5*I*H_a3_p2**2*H_gb2_gc1*H_p1_p1/48 + 3*I*H_a3_p2**2*H_gb2_gc1*H_p1_p2/8 -
5*I*H_a3_p2**2*H_gb2_gc1*H_p2_p2/48 - I*H_a3_p2**2*H_gb2_p1*H_gc1_p1/4 + 3*I*H_a3_p2**2*H_gb2_p1*H_gc1_p2/8 - 7*I*H_a3_p2**2*H_gb2_p2*H_gc1_p1/24 +
7*I*H_a3_p2**2*H_gb2_p2*H_gc1_p2/24 - 7*I*H_a3_p2**2*H_p1_p1*T_gb2_gc1/24 + 7*I*H_a3_p2**2*H_p1_p2*T_gb2_gc1/24 +
I*H_a3_p2*H_gb2_p1*H_p1_p1*T_a3_gc1/6 - 7*I*H_a3_p2*H_gb2_p1*H_p1_p2*T_a3_gc1/24 - I*H_a3_p2*H_gb2_p1*H_p2_p2*T_a3_gc1/3 +
5*I*H_a3_p2*H_gb2_p2*H_p1_p1*T_a3_gc1/8 + I*H_a3_p2*H_gb2_p2*H_p1_p2*T_a3_gc1/12 + I*H_a3_p2*H_gc1_p1*H_p1_p1*T_a3_gb2/24 -
3*I*H_a3_p2*H_gc1_p1*H_p1_p2*T_a3_gb2/8 + 3*I*H_a3_p2*H_gc1_p1*H_p2_p2*T_a3_gb2/8 - 5*I*H_a3_p2*H_gc1_p2*H_p1_p1*T_a3_gb2/12 -
2*I*H_a3_p2*H_gc1_p2*H_p1_p2*T_a3_gb2/3 + 5*I*H_a3_p2*H_gc1_p2*H_p2_p2*T_a3_gb2/24 - 7*I*H_gb2_gc1*H_p1_p1*H_p1_p2*T_a3_a3/32 +
17*I*H_gb2_gc1*H_p1_p1*H_p2_p2*T_a3_a3/48 - 5*I*H_gb2_gc1*H_p1_p2**2*T_a3_a3/8 + 5*I*H_gb2_gc1*H_p1_p2*H_p2_p2*T_a3_a3/48 +
5*I*H_gb2_gc1*H_p2_p2**2*T_a3_a3/32 + 25*I*H_gb2_p1*H_gc1_p1*H_p1_p2*T_a3_a3/48 - I*H_gb2_p1*H_gc1_p1*H_p2_p2*T_a3_a3/12 -
7*I*H_gb2_p1*H_gc1_p2*H_p1_p1*T_a3_a3/32 - 7*I*H_gb2_p1*H_gc1_p2*H_p1_p2*T_a3_a3/24 - 23*I*H_gb2_p1*H_gc1_p2*H_p2_p2*T_a3_a3/48 -
29*I*H_gb2_p2*H_gc1_p1*H_p1_p1*T_a3_a3/96 + 13*I*H_gb2_p2*H_gc1_p1*H_p1_p2*T_a3_a3/24 - I*H_gb2_p2*H_gc1_p1*H_p2_p2*T_a3_a3/16 -
17*I*H_gb2_p2*H_gc1_p2*H_p1_p1*T_a3_a3/24 - 5*I*H_gb2_p2*H_gc1_p2*H_p1_p2*T_a3_a3/8 - 5*I*H_gb2_p2*H_gc1_p2*H_p2_p2*T_a3_a3/8 -
I*H_p1_p1*H_p1_p2*T_a3_a3*T_gb2_gc1/8 + 3*I*H_p1_p1*H_p2_p2*T_a3_a3*T_gb2_gc1/8 + I*H_p1_p1*H_p2_p2*T_a3_gb2*T_a3_gc1/8 -
5*I*H_p1_p2**2*T_a3_a3*T_gb2_gc1/8 - I*H_p1_p2*H_p2_p2*T_a3_a3*T_gb2_gc1/8 + 3*I*H_p1_p2*H_p2_p2*T_a3_gb2*T_a3_gc1/8",
   "sha256": "97a2b71152cad13fceba92892757da7145b890adfdd00e7de5899abf31d75ca4",
   "n_terms": 124,
   "zero": false,
   "n_evanescent_terms": 63,
   "momentum_degree": 4,
   "seconds": 199.8
  },
  "G07": {
   "pole_over_U_d4_residue": "11*I*H_a3_a3*H_aK_aK*H_gc1_p1*H_p1_p1/192 + 5*I*H_a3_a3*H_aK_aK*H_gc1_p1*H_p1_p2/48 +
I*H_a3_a3*H_aK_aK*H_gc1_p1*H_p2_p2/16 - 5*I*H_a3_a3*H_aK_aK*H_gc1_p2*H_p1_p1/48 - I*H_a3_a3*H_aK_aK*H_gc1_p2*H_p1_p2/8 -
3*I*H_a3_a3*H_aK_aK*H_gc1_p2*H_p2_p2/64 - 25*I*H_a3_a3*H_aK_gc1*H_aK_p1*H_p1_p1/96 - 3*I*H_a3_a3*H_aK_gc1*H_aK_p1*H_p1_p2/8 -
I*H_a3_a3*H_aK_gc1*H_aK_p1*H_p2_p2/4 - 7*I*H_a3_a3*H_aK_gc1*H_aK_p2*H_p1_p1/48 + I*H_a3_a3*H_aK_gc1*H_aK_p2*H_p1_p2/16 +
I*H_a3_a3*H_aK_gc1*H_aK_p2*H_p2_p2/32 + 5*I*H_a3_a3*H_aK_p1**2*H_gc1_p1/96 + 5*I*H_a3_a3*H_aK_p1**2*H_gc1_p2/48 + I*H_a3_a3*H_aK_p1*H_aK_p2*H_gc1_p2/8
- 13*I*H_a3_a3*H_aK_p1*H_p1_p1*T_aK_gc1/48 - 7*I*H_a3_a3*H_aK_p1*H_p1_p2*T_aK_gc1/24 - I*H_a3_a3*H_aK_p1*H_p2_p2*T_aK_gc1/6 -
I*H_a3_a3*H_aK_p2**2*H_gc1_p1/6 - 5*I*H_a3_a3*H_aK_p2**2*H_gc1_p2/96 - 13*I*H_a3_a3*H_aK_p2*H_p1_p1*T_aK_gc1/48 -
I*H_a3_a3*H_aK_p2*H_p1_p2*T_aK_gc1/24 + I*H_a3_a3*H_gc1_p1*H_p1_p1*T_aK_aK/12 + I*H_a3_a3*H_gc1_p1*H_p1_p2*T_aK_aK/16 +
I*H_a3_a3*H_gc1_p1*H_p2_p2*T_aK_aK/96 + I*H_a3_a3*H_gc1_p2*H_p1_p1*T_aK_aK/32 + I*H_a3_a3*H_gc1_p2*H_p1_p2*T_aK_aK/48 +
I*H_a3_aK**2*H_gc1_p1*H_p1_p1/24 - I*H_a3_aK**2*H_gc1_p1*H_p1_p2/8 - I*H_a3_aK**2*H_gc1_p1*H_p2_p2/8 + I*H_a3_aK**2*H_gc1_p2*H_p1_p1/4 +
I*H_a3_aK**2*H_gc1_p2*H_p1_p2/6 + I*H_a3_aK*H_a3_gc1*H_aK_p1*H_p1_p2/6 + I*H_a3_aK*H_a3_gc1*H_aK_p1*H_p2_p2/6 - I*H_a3_aK*H_a3_gc1*H_aK_p2*H_p1_p1/6 -
I*H_a3_aK*H_a3_gc1*H_aK_p2*H_p1_p2/12 + I*H_a3_aK*H_a3_gc1*H_aK_p2*H_p2_p2/24 + 13*I*H_a3_aK*H_a3_p1*H_aK_gc1*H_p1_p1/48 +
5*I*H_a3_aK*H_a3_p1*H_aK_gc1*H_p1_p2/16 + 3*I*H_a3_aK*H_a3_p1*H_aK_gc1*H_p2_p2/16 - I*H_a3_aK*H_a3_p1*H_aK_p1*H_gc1_p1/24 -
3*I*H_a3_aK*H_a3_p1*H_aK_p1*H_gc1_p2/16 + 5*I*H_a3_aK*H_a3_p1*H_aK_p2*H_gc1_p1/48 - I*H_a3_aK*H_a3_p1*H_aK_p2*H_gc1_p2/6 +
3*I*H_a3_aK*H_a3_p1*H_p1_p1*T_aK_gc1/8 + I*H_a3_aK*H_a3_p1*H_p1_p2*T_aK_gc1/3 + I*H_a3_aK*H_a3_p1*H_p2_p2*T_aK_gc1/6 -
7*I*H_a3_aK*H_a3_p2*H_aK_gc1*H_p1_p1/48 - 5*I*H_a3_aK*H_a3_p2*H_aK_gc1*H_p1_p2/48 + I*H_a3_aK*H_a3_p2*H_aK_gc1*H_p2_p2/48 +
I*H_a3_aK*H_a3_p2*H_aK_p1*H_gc1_p1/24 - 5*I*H_a3_aK*H_a3_p2*H_aK_p1*H_gc1_p2/48 + 13*I*H_a3_aK*H_a3_p2*H_aK_p2*H_gc1_p1/48 +
I*H_a3_aK*H_a3_p2*H_aK_p2*H_gc1_p2/12 + I*H_a3_aK*H_a3_p2*H_p1_p1*T_aK_gc1/24 - I*H_a3_aK*H_aK_p1*H_p1_p1*T_a3_gc1/24 +
I*H_a3_aK*H_aK_p1*H_p1_p2*T_a3_gc1/8 + I*H_a3_aK*H_aK_p1*H_p2_p2*T_a3_gc1/8 - I*H_a3_aK*H_aK_p2*H_p1_p1*T_a3_gc1/8 -
I*H_a3_aK*H_aK_p2*H_p1_p2*T_a3_gc1/24 + I*H_a3_aK*H_gc1_p1*H_p1_p1*T_a3_aK/24 - I*H_a3_aK*H_gc1_p1*H_p1_p2*T_a3_aK/8 -
I*H_a3_aK*H_gc1_p1*H_p2_p2*T_a3_aK/8 + I*H_a3_aK*H_gc1_p2*H_p1_p1*T_a3_aK/4 + I*H_a3_aK*H_gc1_p2*H_p1_p2*T_a3_aK/6 -
5*I*H_a3_gc1*H_a3_p1*H_aK_aK*H_p1_p1/32 - 17*I*H_a3_gc1*H_a3_p1*H_aK_aK*H_p1_p2/96 - 11*I*H_a3_gc1*H_a3_p1*H_aK_aK*H_p2_p2/96 -
I*H_a3_gc1*H_a3_p1*H_aK_p1**2/48 + 5*I*H_a3_gc1*H_a3_p1*H_aK_p1*H_aK_p2/48 + 11*I*H_a3_gc1*H_a3_p1*H_aK_p2**2/48 -
5*I*H_a3_gc1*H_a3_p1*H_p1_p1*T_aK_aK/24 - 7*I*H_a3_gc1*H_a3_p1*H_p1_p2*T_aK_aK/48 - I*H_a3_gc1*H_a3_p1*H_p2_p2*T_aK_aK/24 +
13*I*H_a3_gc1*H_a3_p2*H_aK_aK*H_p1_p1/96 + 11*I*H_a3_gc1*H_a3_p2*H_aK_aK*H_p1_p2/96 + I*H_a3_gc1*H_a3_p2*H_aK_aK*H_p2_p2/96 -
5*I*H_a3_gc1*H_a3_p2*H_aK_p1**2/48 - 11*I*H_a3_gc1*H_a3_p2*H_aK_p1*H_aK_p2/48 - I*H_a3_gc1*H_a3_p2*H_aK_p2**2/48 -
I*H_a3_gc1*H_a3_p2*H_p1_p1*T_aK_aK/24 - I*H_a3_gc1*H_a3_p2*H_p1_p2*T_aK_aK/48 + I*H_a3_gc1*H_aK_p1*H_p1_p1*T_a3_aK/24 +
I*H_a3_gc1*H_aK_p1*H_p1_p2*T_a3_aK/24 + I*H_a3_gc1*H_aK_p1*H_p2_p2*T_a3_aK/24 - I*H_a3_gc1*H_aK_p2*H_p1_p1*T_a3_aK/24 -
I*H_a3_gc1*H_aK_p2*H_p1_p2*T_a3_aK/24 + I*H_a3_gc1*H_aK_p2*H_p2_p2*T_a3_aK/24 - I*H_a3_p1**2*H_aK_aK*H_gc1_p1/48 + 7*I*H_a3_p1**2*H_aK_aK*H_gc1_p2/96
+ I*H_a3_p1**2*H_aK_gc1*H_aK_p1/8 + 5*I*H_a3_p1**2*H_aK_gc1*H_aK_p2/48 + I*H_a3_p1**2*H_aK_p1*T_aK_gc1/12 + I*H_a3_p1**2*H_aK_p2*T_aK_gc1/6 -
I*H_a3_p1**2*H_gc1_p2*T_aK_aK/48 - 11*I*H_a3_p1*H_a3_p2*H_aK_aK*H_gc1_p1/96 + 5*I*H_a3_p1*H_a3_p2*H_aK_aK*H_gc1_p2/96 +
13*I*H_a3_p1*H_a3_p2*H_aK_gc1*H_aK_p1/48 + I*H_a3_p1*H_a3_p2*H_aK_gc1*H_aK_p2/16 + I*H_a3_p1*H_a3_p2*H_aK_p1*T_aK_gc1/12 +
I*H_a3_p1*H_a3_p2*H_aK_p2*T_aK_gc1/12 + I*H_a3_p1*H_a3_p2*H_gc1_p1*T_aK_aK/48 - I*H_a3_p1*H_a3_p2*H_gc1_p2*T_aK_aK/48 -
7*I*H_a3_p1*H_aK_aK*H_p1_p1*T_a3_gc1/48 - I*H_a3_p1*H_aK_aK*H_p1_p2*T_a3_gc1/6 - 5*I*H_a3_p1*H_aK_aK*H_p2_p2*T_a3_gc1/48 +
I*H_a3_p1*H_aK_gc1*H_p1_p1*T_a3_aK/4 + 5*I*H_a3_p1*H_aK_gc1*H_p1_p2*T_a3_aK/24 + I*H_a3_p1*H_aK_gc1*H_p2_p2*T_a3_aK/24 +
```

```
I*H_a3_p1*H_aK_p1*H_aK_p2*T_a3_gc1/24 - I*H_a3_p1*H_aK_p1*H_gc1_p1*T_a3_aK/12 + I*H_a3_p1*H_aK_p2**2*T_a3_gc1/8 -
I*H_a3_p1*H_aK_p2*H_gc1_p2*T_a3_aK/24 + 3*I*H_a3_p1*H_p1_p1*T_a3_aK*T_aK_gc1/8 - 3*I*H_a3_p1*H_p1_p1*T_a3_gc1*T_aK_aK/16 +
I*H_a3_p1*H_p1_p2*T_a3_aK*T_aK_gc1/4 - I*H_a3_p1*H_p1_p2*T_a3_gc1*T_aK_aK/8 + I*H_a3_p1*H_p2_p2*T_a3_aK*T_aK_gc1/24 -
I*H_a3_p1*H_p2_p2*T_a3_gc1*T_aK_aK/48 - 5*I*H_a3_p2**2*H_aK_aK*H_gc1_p1/96 + I*H_a3_p2**2*H_aK_aK*H_gc1_p2/48 + 7*I*H_a3_p2**2*H_aK_gc1*H_aK_p1/48 -
I*H_a3_p2**2*H_aK_gc1*H_aK_p2/24 + I*H_a3_p2**2*H_aK_p1*T_aK_gc1/12 + I*H_a3_p2**2*H_gc1_p1*T_aK_aK/48 + 5*I*H_a3_p2*H_aK_aK*H_p1_p1*T_a3_gc1/48 +
I*H_a3_p2*H_aK_aK*H_p1_p2*T_a3_gc1/12 - I*H_a3_p2*H_aK_gc1*H_p1_p1*T_a3_aK/12 - I*H_a3_p2*H_aK_gc1*H_p1_p2*T_a3_aK/24 -
I*H_a3_p2*H_aK_p1**2*T_a3_gc1/24 - I*H_a3_p2*H_aK_p1*H_aK_p2*T_a3_gc1/8 - I*H_a3_p2*H_aK_p1*H_gc1_p2*T_a3_aK/24 + I*H_a3_p2*H_aK_p2*H_gc1_p1*T_a3_aK/6
+ I*H_a3_p2*H_aK_p2*H_gc1_p2*T_a3_aK/12 + I*H_a3_p2*H_p1_p1*T_a3_aK*T_aK_gc1/8 - I*H_a3_p2*H_p1_p1*T_a3_gc1*T_aK_aK/16 +
I*H_a3_p2*H_p1_p2*T_a3_aK*T_aK_gc1/12 - I*H_a3_p2*H_p1_p2*T_a3_gc1*T_aK_aK/24 + 7*I*H_aK_aK*H_gc1_p1*H_p1_p1*T_a3_a3/192 +
I*H_aK_aK*H_gc1_p1*H_p1_p2*T_a3_a3/24 + I*H_aK_aK*H_gc1_p1*H_p2_p2*T_a3_a3/48 - I*H_aK_aK*H_gc1_p2*H_p1_p1*T_a3_a3/24 -
I*H_aK_aK*H_gc1_p2*H_p1_p2*T_a3_a3/12 - 3*I*H_aK_aK*H_gc1_p2*H_p2_p2*T_a3_a3/64 - 29*I*H_aK_gc1*H_aK_p1*H_p1_p1*T_a3_a3/96 -
I*H_aK_gc1*H_aK_p1*H_p1_p2*T_a3_a3/3 - I*H_aK_gc1*H_aK_p1*H_p2_p2*T_a3_a3/6 - 7*I*H_aK_gc1*H_aK_p2*H_p1_p1*T_a3_a3/16 -
3*I*H_aK_gc1*H_aK_p2*H_p1_p2*T_a3_a3/16 + I*H_aK_gc1*H_aK_p2*H_p2_p2*T_a3_a3/32 + 3*I*H_aK_p1**2*H_gc1_p1*T_a3_a3/32 +
I*H_aK_p1**2*H_gc1_p2*T_a3_a3/48 + I*H_aK_p1*H_aK_p2*H_gc1_p1*T_a3_a3/8 - I*H_aK_p1*H_aK_p2*H_gc1_p2*T_a3_a3/24 -
5*I*H_aK_p1*H_p1_p1*T_a3_a3*T_aK_gc1/16 - I*H_aK_p1*H_p1_p2*T_a3_a3*T_aK_gc1/4 - I*H_aK_p1*H_p2_p2*T_a3_a3*T_aK_gc1/12 -
I*H_aK_p2**2*H_gc1_p1*T_a3_a3/24 - 3*I*H_aK_p2**2*H_gc1_p2*T_a3_a3/32 - 9*I*H_aK_p2*H_p1_p1*T_a3_a3*T_aK_gc1/16 -
7*I*H_aK_p2*H_p1_p2*T_a3_a3*T_aK_gc1/24 + I*H_gc1_p1*H_p1_p1*T_a3_a3*T_aK_aK/16 - I*H_gc1_p1*H_p2_p2*T_a3_a3*T_aK_aK/32 +
3*I*H_gc1_p2*H_p1_p1*T_a3_a3*T_aK_aK/32 + I*H_gc1_p2*H_p1_p2*T_a3_a3*T_aK_aK/16",
   "sha256": "b96dbcc7daad1919facf8af06bb5d0f674626586255ce256faaf3a237fe6cc96",
   "n_terms": 165,
   "zero": false,
   "n_evanescent_terms": 100,
   "momentum_degree": 3,
   "seconds": 331.2
  },
  "G08": {
   "pole_over_U_d4_residue": "-3*H_a1_a1*H_a2_a2*H_a3_a3*H_p1_p1*H_p2_p2/16 + 3*H_a1_a1*H_a2_a2*H_a3_a3*H_p1_p2**2/16 -
H_a1_a1*H_a2_a2*H_a3_p1**2*H_p1_p2/48 + 13*H_a1_a1*H_a2_a2*H_a3_p1**2*H_p2_p2/48 - 5*H_a1_a1*H_a2_a2*H_a3_p1*H_a3_p2*H_p1_p1/24 -
19*H_a1_a1*H_a2_a2*H_a3_p1*H_a3_p2*H_p1_p2/24 - 5*H_a1_a1*H_a2_a2*H_a3_p1*H_a3_p2*H_p2_p2/24 + 13*H_a1_a1*H_a2_a2*H_a3_p2**2*H_p1_p1/48 -
H_a1_a1*H_a2_a2*H_a3_p2**2*H_p1_p2/48 - 7*H_a1_a1*H_a2_a2*H_p1_p1*H_p2_p2*T_a3_a3/48 + 7*H_a1_a1*H_a2_a2*H_p1_p2**2*T_a3_a3/48 +
H_a1_a1*H_a2_a3**2*H_p1_p1**2/24 - H_a1_a1*H_a2_a3**2*H_p1_p1*H_p1_p2/8 + H_a1_a1*H_a2_a3**2*H_p1_p1*H_p2_p2/8 - H_a1_a1*H_a2_a3**2*H_p1_p2**2/4 -
H_a1_a1*H_a2_a3*H_a2_p1*H_a3_p1*H_p1_p1/12 + H_a1_a1*H_a2_a3*H_a2_p1*H_a3_p1*H_p1_p2/6 - 11*H_a1_a1*H_a2_a3*H_a2_p1*H_a3_p1*H_p2_p2/24 -
H_a1_a1*H_a2_a3*H_a2_p1*H_a3_p2*H_p1_p1/6 + H_a1_a1*H_a2_a3*H_a2_p1*H_a3_p2*H_p1_p2/3 - 7*H_a1_a1*H_a2_a3*H_a2_p1*H_a3_p2*H_p2_p2/24 +
H_a1_a1*H_a2_a3*H_a2_p2*H_a3_p1*H_p1_p1/4 + 11*H_a1_a1*H_a2_a3*H_a2_p2*H_a3_p1*H_p1_p2/12 + 7*H_a1_a1*H_a2_a3*H_a2_p2*H_a3_p1*H_p2_p2/24 -
13*H_a1_a1*H_a2_a3*H_a2_p2*H_a3_p2*H_p1_p1/24 + H_a1_a1*H_a2_a3*H_p1_p1**2*T_a2_a3/24 - H_a1_a1*H_a2_a3*H_p1_p1*H_p1_p2*T_a2_a3/8 +
H_a1_a1*H_a2_a3*H_p1_p1*H_p2_p2*T_a2_a3/8 - H_a1_a1*H_a2_a3*H_p1_p2**2*T_a2_a3/4 + 5*H_a1_a1*H_a2_p1**2*H_a3_a3*H_p1_p1/24 +
5*H_a1_a1*H_a2_p1**2*H_a3_a3*H_p1_p2/24 + 23*H_a1_a1*H_a2_p1**2*H_a3_a3*H_p2_p2/48 - H_a1_a1*H_a2_p1**2*H_a3_p1**2/12 -
H_a1_a1*H_a2_p1**2*H_a3_p1*H_a3_p2/12 - H_a1_a1*H_a2_p1**2*H_a3_p2**2/3 + H_a1_a1*H_a2_p1**2*H_p1_p1*T_a3_a3/6 + H_a1_a1*H_a2_p1**2*H_p1_p2*T_a3_a3/6
+ 17*H_a1_a1*H_a2_p1**2*H_p2_p2*T_a3_a3/48 + 5*H_a1_a1*H_a2_p1*H_a2_p2*H_a3_a3*H_p1_p1/24 - H_a1_a1*H_a2_p1*H_a2_p2*H_a3_a3*H_p1_p2/3 +
5*H_a1_a1*H_a2_p1*H_a2_p2*H_a3_a3*H_p2_p2/24 - H_a1_a1*H_a2_p1*H_a2_p2*H_a3_p1**2/24 + 7*H_a1_a1*H_a2_p1*H_a2_p2*H_a3_p1*H_a3_p2/12 +
H_a1_a1*H_a2_p1*H_a2_p2*H_a3_p2**2/24 + H_a1_a1*H_a2_p1*H_a2_p2*H_p1_p1*T_a3_a3/6 - H_a1_a1*H_a2_p1*H_a2_p2*H_p1_p2*T_a3_a3/4 +
H_a1_a1*H_a2_p1*H_a2_p2*H_p2_p2*T_a3_a3/6 + H_a1_a1*H_a2_p1*H_a3_p1*H_p1_p2*T_a2_a3/4 - H_a1_a1*H_a2_p1*H_a3_p1*H_p2_p2*T_a2_a3/24 -
3*H_a1_a1*H_a2_p1*H_a3_p2*H_p1_p1*T_a2_a3/8 - 7*H_a1_a1*H_a2_p1*H_a3_p2*H_p2_p2*T_a2_a3/24 + 7*H_a1_a1*H_a2_p2**2*H_a3_a3*H_p1_p1/24 +
H_a1_a1*H_a2_p2**2*H_a3_a3*H_p1_p2/48 - 3*H_a1_a1*H_a2_p2**2*H_a3_p1**2/8 - H_a1_a1*H_a2_p2**2*H_a3_p1*H_a3_p2/24 +
5*H_a1_a1*H_a2_p2**2*H_p1_p1*T_a3_a3/24 - H_a1_a1*H_a2_p2**2*H_p1_p2*T_a3_a3/48 + H_a1_a1*H_a2_p2*H_a3_p1*H_p1_p1*T_a2_a3/4 +
7*H_a1_a1*H_a2_p2*H_a3_p1*H_p1_p2*T_a2_a3/12 + 7*H_a1_a1*H_a2_p2*H_a3_p1*H_p2_p2*T_a2_a3/24 - 5*H_a1_a1*H_a2_p2*H_a3_p2*H_p1_p1*T_a2_a3/12 -
7*H_a1_a1*H_a3_a3*H_p1_p1*H_p2_p2*T_a2_a2/48 + 7*H_a1_a1*H_a3_a3*H_p1_p2**2*T_a2_a2/48 - H_a1_a1*H_a3_p1**2*H_p1_p2*T_a2_a2/48 +
3*H_a1_a1*H_a3_p1**2*H_p2_p2*T_a2_a2/16 - H_a1_a1*H_a3_p1*H_a3_p2*H_p1_p1*T_a2_a2/8 - 13*H_a1_a1*H_a3_p1*H_a3_p2*H_p1_p2*T_a2_a2/24 -
H_a1_a1*H_a3_p1*H_a3_p2*H_p2_p2*T_a2_a2/6 + 11*H_a1_a1*H_a3_p2**2*H_p1_p1*T_a2_a2/48 + H_a1_a1*H_a3_p2**2*H_p1_p2*T_a2_a2/48 -
5*H_a1_a1*H_p1_p1*H_p2_p2*T_a2_a2*T_a3_a3/48 + 5*H_a1_a1*H_p1_p2**2*T_a2_a2*T_a3_a3/48 + H_a1_a2**2*H_a3_a3*H_p1_p1**2/24 +
7*H_a1_a2**2*H_a3_a3*H_p1_p1*H_p1_p2/24 + H_a1_a2**2*H_a3_a3*H_p1_p1*H_p2_p2/3 + H_a1_a2**2*H_a3_a3*H_p1_p2**2/6 +
7*H_a1_a2**2*H_a3_a3*H_p1_p2*H_p2_p2/24 + H_a1_a2**2*H_a3_a3*H_p2_p2**2/24 - H_a1_a2**2*H_a3_p1**2*H_p1_p1/12 - H_a1_a2**2*H_a3_p1**2*H_p1_p2/4 -
5*H_a1_a2**2*H_a3_p1**2*H_p2_p2/12 + H_a1_a2**2*H_a3_p1*H_a3_p2*H_p1_p2/3 - 5*H_a1_a2**2*H_a3_p2**2*H_p1_p1/12 - H_a1_a2**2*H_a3_p2**2*H_p1_p2/4 -
H_a1_a2**2*H_a3_p2**2*H_p2_p2/12 + H_a1_a2**2*H_p1_p1**2*T_a3_a3/24 + 7*H_a1_a2**2*H_p1_p1*H_p1_p2*T_a3_a3/24 + H_a1_a2**2*H_p1_p1*H_p2_p2*T_a3_a3/3 +
H_a1_a2**2*H_p1_p2**2*T_a3_a3/6 + 7*H_a1_a2**2*H_p1_p2*H_p2_p2*T_a3_a3/24 + H_a1_a2**2*H_p2_p2**2*T_a3_a3/24 + H_a1_a2*H_a1_a3*H_a2_a3*H_p1_p1**2/3 +
2*H_a1_a2*H_a1_a3*H_a2_a3*H_p1_p1*H_p1_p2/3 + H_a1_a2*H_a1_a3*H_a2_a3*H_p1_p1*H_p2_p2/3 + 2*H_a1_a2*H_a1_a3*H_a2_a3*H_p1_p2**2/3 +
2*H_a1_a2*H_a1_a3*H_a2_a3*H_p1_p2*H_p2_p2/3 + H_a1_a2*H_a1_a3*H_a2_a3*H_p2_p2**2/3 + H_a1_a2*H_a1_a3*H_a2_p1*H_a3_p1*H_p1_p2/6 +
H_a1_a2*H_a1_a3*H_a2_p1*H_a3_p1*H_p2_p2/2 - H_a1_a2*H_a1_a3*H_a2_p1*H_a3_p2*H_p1_p2/2 - H_a1_a2*H_a1_a3*H_a2_p1*H_a3_p2*H_p2_p2/6 +
H_a1_a2*H_a1_a3*H_a2_p2*H_a3_p1*H_p1_p1/4 - H_a1_a2*H_a1_a3*H_a2_p2*H_a3_p1*H_p1_p2/3 + 7*H_a1_a2*H_a1_a3*H_a2_p2*H_a3_p2*H_p1_p1/12 -
H_a1_a2*H_a1_a3*H_a2_p2*H_a3_p2*H_p1_p2/6 - H_a1_a2*H_a1_a3*H_a2_p2*H_a3_p2*H_p2_p2/6 + H_a1_a2*H_a1_a3*H_p1_p1**2*T_a2_a3/3 +
H_a1_a2*H_a1_a3*H_p1_p1*H_p1_p2*T_a2_a3 + 2*H_a1_a2*H_a1_a3*H_p1_p1*H_p2_p2*T_a2_a3/3 + 5*H_a1_a2*H_a1_a3*H_p1_p2**2*T_a2_a3/6 +
H_a1_a2*H_a1_a3*H_p1_p2*H_p2_p2*T_a2_a3 + H_a1_a2*H_a1_a3*H_p2_p2**2*T_a2_a3/2 - H_a1_a2*H_a1_p1*H_a2_a3*H_a3_p1*H_p1_p1/6 -
H_a1_a2*H_a1_p1*H_a2_a3*H_a3_p1*H_p1_p2/6 + 7*H_a1_a2*H_a1_p1*H_a2_a3*H_a3_p1*H_p2_p2/12 - H_a1_a2*H_a1_p1*H_a2_a3*H_a3_p2*H_p1_p2/3 +
H_a1_a2*H_a1_p1*H_a2_a3*H_a3_p2*H_p2_p2/4 - H_a1_a2*H_a1_p1*H_a2_p1*H_a3_a3*H_p1_p1/12 - H_a1_a2*H_a1_p1*H_a2_p1*H_a3_a3*H_p1_p2/3 -
17*H_a1_a2*H_a1_p1*H_a2_p1*H_a3_a3*H_p2_p2/24 + H_a1_a2*H_a1_p1*H_a2_p1*H_a3_p1**2/6 + H_a1_a2*H_a1_p1*H_a2_p1*H_a3_p1*H_a3_p2/3 +
2*H_a1_a2*H_a1_p1*H_a2_p1*H_a3_p2**2/3 - H_a1_a2*H_a1_p1*H_a2_p1*H_p1_p2*T_a3_a3/6 - 11*H_a1_a2*H_a1_p1*H_a2_p1*H_p2_p2*T_a3_a3/24 -
H_a1_a2*H_a1_p1*H_a2_p2*H_a3_a3*H_p1_p1/3 + H_a1_a2*H_a1_p1*H_a2_p2*H_a3_a3*H_p1_p2/12 - H_a1_a2*H_a1_p1*H_a2_p2*H_a3_a3*H_p2_p2/3 +
H_a1_a2*H_a1_p1*H_a2_p2*H_a3_p1**2/12 - H_a1_a2*H_a1_p1*H_a2_p2*H_a3_p1*H_a3_p2/2 + H_a1_a2*H_a1_p1*H_a2_p2*H_a3_p2**2/12 -
H_a1_a2*H_a1_p1*H_a2_p2*H_p1_p1*T_a3_a3/6 + H_a1_a2*H_a1_p1*H_a2_p2*H_p1_p2*T_a3_a3/12 - H_a1_a2*H_a1_p1*H_a2_p2*H_p2_p2*T_a3_a3/6 -
H_a1_a2*H_a1_p1*H_a3_p1*H_p1_p1*T_a2_a3/4 - H_a1_a2*H_a1_p1*H_a3_p1*H_p1_p2*T_a2_a3/3 + H_a1_a2*H_a1_p1*H_a3_p1*H_p2_p2*T_a2_a3/4 -
H_a1_a2*H_a1_p1*H_a3_p2*H_p1_p1*T_a2_a3/6 - H_a1_a2*H_a1_p1*H_a3_p2*H_p1_p2*T_a2_a3/6 + H_a1_a2*H_a1_p1*H_a3_p2*H_p2_p2*T_a2_a3/12 -
H_a1_a2*H_a1_p2*H_a2_a3*H_a3_p1*H_p1_p1/6 - H_a1_a2*H_a1_p2*H_a2_a3*H_a3_p1*H_p1_p2/2 + H_a1_a2*H_a1_p2*H_a2_a3*H_a3_p2*H_p1_p1/2 +
H_a1_a2*H_a1_p2*H_a2_a3*H_a3_p2*H_p1_p2/6 + H_a1_a2*H_a1_p2*H_a2_p1*H_a3_a3*H_p1_p1/12 + H_a1_a2*H_a1_p2*H_a2_p1*H_a3_a3*H_p1_p2/3 +
H_a1_a2*H_a1_p2*H_a2_p1*H_a3_a3*H_p2_p2/12 - H_a1_a2*H_a1_p2*H_a2_p1*H_a3_p1*H_a3_p2/3 + H_a1_a2*H_a1_p2*H_a2_p1*H_p1_p1*T_a3_a3/12 +
H_a1_a2*H_a1_p2*H_a2_p1*H_p1_p2*T_a3_a3/3 + H_a1_a2*H_a1_p2*H_a2_p1*H_p2_p2*T_a3_a3/12 - 17*H_a1_a2*H_a1_p2*H_a2_p2*H_a3_a3*H_p1_p1/24 -
H_a1_a2*H_a1_p2*H_a2_p2*H_a3_a3*H_p1_p2/3 - H_a1_a2*H_a1_p2*H_a2_p2*H_a3_a3*H_p2_p2/12 + 2*H_a1_a2*H_a1_p2*H_a2_p2*H_a3_p1**2/3 +
H_a1_a2*H_a1_p2*H_a2_p2*H_a3_p1*H_a3_p2/3 + H_a1_a2*H_a1_p2*H_a2_p2*H_a3_p2**2/6 - 11*H_a1_a2*H_a1_p2*H_a2_p2*H_p1_p1*T_a3_a3/24 -
H_a1_a2*H_a1_p2*H_a2_p2*H_p1_p2*T_a3_a3/6 - H_a1_a2*H_a1_p2*H_a3_p1*H_p1_p1*T_a2_a3/6 - H_a1_a2*H_a1_p2*H_a3_p1*H_p1_p2*T_a2_a3/3 +
H_a1_a2*H_a1_p2*H_a3_p1*H_p2_p2*T_a2_a3/6 - H_a1_a2*H_a1_p2*H_a3_p2*H_p1_p2*T_a2_a3/6 - H_a1_a2*H_a1_p2*H_a3_p2*H_p2_p2*T_a2_a3/6 +
H_a1_a2*H_a2_a3*H_p1_p1**2*T_a1_a3/2 + H_a1_a2*H_a2_a3*H_p1_p1*H_p1_p2*T_a1_a3 + 2*H_a1_a2*H_a2_a3*H_p1_p1*H_p2_p2*T_a1_a3/3 +
5*H_a1_a2*H_a2_a3*H_p1_p2**2*T_a1_a3/6 + H_a1_a2*H_a2_a3*H_p1_p2*H_p2_p2*T_a1_a3 + H_a1_a2*H_a2_a3*H_p2_p2**2*T_a1_a3/3 -
H_a1_a2*H_a2_p1*H_a3_p1*H_p1_p1*T_a1_a3/6 - H_a1_a2*H_a2_p1*H_a3_p1*H_p1_p2*T_a1_a3/6 + H_a1_a2*H_a2_p1*H_a3_p2*H_p1_p1*T_a1_a3/6 -
H_a1_a2*H_a2_p1*H_a3_p2*H_p1_p2*T_a1_a3/3 - H_a1_a2*H_a2_p1*H_a3_p2*H_p2_p2*T_a1_a3/6 + H_a1_a2*H_a2_p2*H_a3_p1*H_p1_p1*T_a1_a3/12 -
H_a1_a2*H_a2_p2*H_a3_p1*H_p1_p2*T_a1_a3/6 - H_a1_a2*H_a2_p2*H_a3_p1*H_p2_p2*T_a1_a3/6 + H_a1_a2*H_a2_p2*H_a3_p2*H_p1_p1*T_a1_a3/4 -
```

```
H_a1_a2*H_a2_p2*H_a3_p2*H_p1_p2*T_a1_a3/3 - H_a1_a2*H_a2_p2*H_a3_p2*H_p2_p2*T_a1_a3/4 + H_a1_a2*H_a3_a3*H_p1_p1**2*T_a1_a2/24 +
7*H_a1_a2*H_a3_a3*H_p1_p1*H_p1_p2*T_a1_a2/24 + H_a1_a2*H_a3_a3*H_p1_p1*H_p2_p2*T_a1_a2/3 + H_a1_a2*H_a3_a3*H_p1_p2**2*T_a1_a2/6 +
7*H_a1_a2*H_a3_a3*H_p1_p2*H_p2_p2*T_a1_a2/24 + H_a1_a2*H_a3_a3*H_p2_p2**2*T_a1_a2/24 - H_a1_a2*H_a3_p1**2*H_p1_p1*T_a1_a2/12 -
H_a1_a2*H_a3_p1**2*H_p1_p2*T_a1_a2/4 - 5*H_a1_a2*H_a3_p1**2*H_p2_p2*T_a1_a2/12 + H_a1_a2*H_a3_p1*H_a3_p2*H_p1_p2*T_a1_a2/3 -
5*H_a1_a2*H_a3_p2**2*H_p1_p1*T_a1_a2/12 - H_a1_a2*H_a3_p2**2*H_p1_p2*T_a1_a2/4 - H_a1_a2*H_a3_p2**2*H_p2_p2*T_a1_a2/12 +
H_a1_a2*H_p1_p1**2*T_a1_a2*T_a3_a3/24 + H_a1_a2*H_p1_p1**2*T_a1_a3*T_a2_a3/2 + 7*H_a1_a2*H_p1_p1*H_p1_p2*T_a1_a2*T_a3_a3/24 +
4*H_a1_a2*H_p1_p1*H_p1_p2*T_a1_a3*T_a2_a3/3 + H_a1_a2*H_p1_p1*H_p2_p2*T_a1_a2*T_a3_a3/24 + H_a1_a2*H_p1_p2**2*T_a1_a3*T_a2_a3/2 +
H_a1_a2*H_p1_p2**2*T_a1_a2*T_a3_a3/6 + H_a1_a2*H_p1_p2**2*T_a1_a3*T_a2_a3 + 7*H_a1_a2*H_p1_p2*H_p2_p2*T_a1_a2*T_a3_a3/24 +
4*H_a1_a2*H_p1_p2*H_p2_p2*T_a1_a3*T_a2_a3/3 + H_a1_a2*H_p2_p2**2*T_a1_a2*T_a3_a3/24 + H_a1_a2*H_p2_p2**2*T_a1_a3*T_a2_a3/2 +
H_a1_a3**2*H_a2_a2*H_p1_p1*H_p2_p2/8 - H_a1_a3**2*H_a2_a2*H_p1_p2**2/4 - H_a1_a3**2*H_a2_a2*H_p1_p2*H_p2_p2/8 + H_a1_a3**2*H_a2_a2*H_p2_p2**2/24 -
H_a1_a3**2*H_a2_p1**2*H_p1_p1/6 - H_a1_a3**2*H_a2_p1**2*H_p1_p2/6 - H_a1_a3**2*H_a2_p1**2*H_p2_p2/2 - H_a1_a3**2*H_a2_p1*H_a2_p2*H_p1_p1/6 +
H_a1_a3**2*H_a2_p1*H_a2_p2*H_p1_p2/2 - H_a1_a3**2*H_a2_p1*H_a2_p2*H_p2_p2/6 - H_a1_a3**2*H_a2_p2**2*H_p1_p1/4 + H_a1_a3**2*H_a2_p2**2*H_p1_p2/12 -
H_a1_a3**2*H_a2_p2**2*H_p2_p2/12 + H_a1_a3**2*H_p1_p1*H_p2_p2*T_a2_a2/8 - H_a1_a3**2*H_p1_p2**2*T_a2_a2/4 - H_a1_a3**2*H_p1_p2*H_p2_p2*T_a2_a2/8 +
H_a1_a3**2*H_p2_p2**2*T_a2_a2/24 - 13*H_a1_a3*H_a1_p1*H_a2_a2*H_a3_p1*H_p2_p2/24 + 7*H_a1_a3*H_a1_p1*H_a2_a2*H_a3_p2*H_p1_p1/24 +
11*H_a1_a3*H_a1_p1*H_a2_a2*H_a3_p2*H_p1_p2/12 + H_a1_a3*H_a1_p1*H_a2_a2*H_a3_p2*H_p2_p2/4 - H_a1_a3*H_a1_p1*H_a2_a3*H_a2_p1*H_p1_p2/6 +
H_a1_a3*H_a1_p1*H_a2_a3*H_a2_p1*H_p2_p2/3 - H_a1_a3*H_a1_p1*H_a2_a3*H_a2_p2*H_p1_p1/4 - H_a1_a3*H_a1_p1*H_a2_a3*H_a2_p2*H_p1_p2 -
H_a1_a3*H_a1_p1*H_a2_a3*H_a2_p2*H_p2_p2/4 - H_a1_a3*H_a1_p1*H_a2_p1**2*H_a3_p2/6 - 2*H_a1_a3*H_a1_p1*H_a2_p1*H_a2_p2*H_a3_p2/3 -
H_a1_a3*H_a1_p1*H_a2_p1*H_p1_p1*T_a2_a3/12 - H_a1_a3*H_a1_p1*H_a2_p1*H_p1_p2*T_a2_a3/3 - H_a1_a3*H_a1_p1*H_a2_p1*H_p2_p2*T_a2_a3/6 +
3*H_a1_a3*H_a1_p1*H_a2_p2**2*H_a3_p1/4 + H_a1_a3*H_a1_p1*H_a2_p2**2*H_a3_p2/12 - H_a1_a3*H_a1_p1*H_a2_p2*H_p1_p1*T_a2_a3/12 -
H_a1_a3*H_a1_p1*H_a2_p2*H_p1_p2*T_a2_a3/2 - H_a1_a3*H_a1_p1*H_a2_p2*H_p2_p2*T_a2_a3/4 - 3*H_a1_a3*H_a1_p1*H_a3_p1*H_p2_p2*T_a2_a2/8 +
H_a1_a3*H_a1_p1*H_a3_p2*H_p1_p1*T_a2_a2/8 + 7*H_a1_a3*H_a1_p1*H_a3_p2*H_p1_p2*T_a2_a2/12 + H_a1_a3*H_a1_p1*H_a3_p2*H_p2_p2*T_a2_a2/6 -
7*H_a1_a3*H_a1_p2*H_a2_a2*H_a3_p1*H_p1_p1/24 + H_a1_a3*H_a1_p2*H_a2_a2*H_a3_p1*H_p1_p2/3 - H_a1_a3*H_a1_p2*H_a2_a2*H_a3_p1*H_p2_p2/6 -
11*H_a1_a3*H_a1_p2*H_a2_a2*H_a3_p2*H_p1_p1/24 + H_a1_a3*H_a1_p2*H_a2_a2*H_a3_p2*H_p1_p2/6 - H_a1_a3*H_a1_p2*H_a2_a2*H_a3_p2*H_p2_p2/12 -
2*H_a1_a3*H_a1_p2*H_a2_a3*H_a2_p1*H_p1_p2/3 + H_a1_a3*H_a1_p2*H_a2_a3*H_a2_p2*H_p1_p1/3 - H_a1_a3*H_a1_p2*H_a2_a3*H_a2_p2*H_p1_p2/6 +
H_a1_a3*H_a1_p2*H_a2_p1**2*H_a3_p1/6 + H_a1_a3*H_a1_p2*H_a2_p1**2*H_a3_p2/2 - H_a1_a3*H_a1_p2*H_a2_p1*H_a2_p2*H_a3_p1/3 -
H_a1_a3*H_a1_p2*H_a2_p1*H_p1_p1*T_a2_a3/3 - 5*H_a1_a3*H_a1_p2*H_a2_p1*H_p1_p2*T_a2_a3/6 - H_a1_a3*H_a1_p2*H_a2_p1*H_p2_p2*T_a2_a3/3 +
H_a1_a3*H_a1_p2*H_a2_p2**2*H_a3_p1/6 + H_a1_a3*H_a1_p2*H_a2_p2**2*H_a3_p2/6 - H_a1_a3*H_a1_p2*H_a2_p2*H_p1_p2*T_a2_a3/6 -
H_a1_a3*H_a1_p2*H_a2_p2*H_p2_p2*T_a2_a3/6 - H_a1_a3*H_a1_p2*H_a3_p1*H_p1_p1*T_a2_a2/8 + H_a1_a3*H_a1_p2*H_a3_p1*H_p1_p2*T_a2_a2/3 -
H_a1_a3*H_a1_p2*H_a3_p1*H_p2_p2*T_a2_a2/12 - 7*H_a1_a3*H_a1_p2*H_a3_p2*H_p1_p1*T_a2_a2/24 + H_a1_a3*H_a1_p2*H_a3_p2*H_p1_p2*T_a2_a2/6 +
H_a1_a3*H_a2_a2*H_p1_p1*H_p2_p2*T_a1_a3/8 - H_a1_a3*H_a2_a2*H_p1_p2**2*T_a1_a3/4 - H_a1_a3*H_a2_a2*H_p1_p2*H_p2_p2*T_a1_a3/8 +
H_a1_a3*H_a2_a2*H_p2_p2**2*T_a1_a3/24 + H_a1_a3*H_a2_a3*H_p1_p1**2*T_a1_a2/3 + H_a1_a3*H_a2_a3*H_p1_p1*H_p1_p2*T_a1_a2/3 +
H_a1_a3*H_a2_a3*H_p1_p1*H_p2_p2*T_a1_a2/3 + H_a1_a3*H_a2_a3*H_p1_p2**2*T_a1_a2/6 + H_a1_a3*H_a2_a3*H_p1_p2*H_p2_p2*T_a1_a2/3 +
H_a1_a3*H_a2_a3*H_p2_p2**2*T_a1_a2/3 - H_a1_a3*H_a2_p1**2*H_p1_p1*T_a1_a3/6 - H_a1_a3*H_a2_p1**2*H_p1_p2*T_a1_a3/6 -
H_a1_a3*H_a2_p1**2*H_p2_p2*T_a1_a3/2 - H_a1_a3*H_a2_p1*H_a2_p2*H_p1_p1*T_a1_a3/6 + H_a1_a3*H_a2_p1*H_a2_p2*H_p1_p2*T_a1_a3/2 -
H_a1_a3*H_a2_p1*H_a2_p2*H_p2_p2*T_a1_a3/6 - H_a1_a3*H_a2_p1*H_a3_p1*H_p1_p1*T_a1_a2/12 + H_a1_a3*H_a2_p1*H_a3_p1*H_p1_p2*T_a1_a2/6 +
H_a1_a3*H_a2_p1*H_a3_p1*H_p2_p2*T_a1_a2/12 + H_a1_a3*H_a2_p1*H_a3_p2*H_p1_p1*T_a1_a2/4 - H_a1_a3*H_a2_p1*H_a3_p2*H_p2_p2*T_a1_a2/12 -
H_a1_a3*H_a2_p2**2*H_p1_p1*T_a1_a3/4 + H_a1_a3*H_a2_p2**2*H_p1_p2*T_a1_a3/12 - H_a1_a3*H_a2_p2**2*H_p2_p2*T_a1_a3/12 -
H_a1_a3*H_a2_p2*H_a3_p1*H_p1_p2*T_a1_a2/6 - H_a1_a3*H_a2_p2*H_a3_p1*H_p2_p2*T_a1_a2/12 + H_a1_a3*H_a2_p2*H_a3_p2*H_p1_p1*T_a1_a2/3 -
H_a1_a3*H_a2_p2*H_a3_p2*H_p1_p2*T_a1_a2/6 - H_a1_a3*H_a2_p2*H_a3_p2*H_p2_p2*T_a1_a2/4 + H_a1_a3*H_p1_p1**2*T_a1_a2*T_a2_a3/3 +
2*H_a1_a3*H_p1_p1*H_p1_p2*T_a1_a2*T_a2_a3/3 + 2*H_a1_a3*H_p1_p1*H_p2_p2*T_a1_a2*T_a2_a3/3 + H_a1_a3*H_p1_p1*H_p2_p2*T_a1_a3*T_a2_a2/8 +
H_a1_a3*H_p1_p2**2*T_a1_a2*T_a2_a3/3 - H_a1_a3*H_p1_p2**2*T_a1_a3*T_a2_a2/4 + 2*H_a1_a3*H_p1_p2*H_p2_p2*T_a1_a2*T_a2_a3/3 -
H_a1_a3*H_p1_p2*H_p2_p2*T_a1_a3*T_a2_a2/8 + H_a1_a3*H_p2_p2**2*T_a1_a2*T_a2_a3/2 + H_a1_a3*H_p2_p2**2*T_a1_a3*T_a2_a2/24 +
H_a1_p1**2*H_a2_a2*H_a3_a3*H_p1_p2/48 + 7*H_a1_p1**2*H_a2_a2*H_a3_a3*H_p2_p2/24 - H_a1_p1**2*H_a2_a2*H_a3_p1*H_a3_p2/24 -
3*H_a1_p1**2*H_a2_a2*H_a3_p2**2/8 - H_a1_p1**2*H_a2_a2*H_p1_p2*T_a3_a3/48 + 5*H_a1_p1**2*H_a2_a2*H_p2_p2*T_a3_a3/24 - H_a1_p1**2*H_a2_a3**2*H_p1_p1/12
+ H_a1_p1**2*H_a2_a3**2*H_p1_p2/12 - H_a1_p1**2*H_a2_a3**2*H_p2_p2/4 + H_a1_p1**2*H_a2_a3*H_a2_p1*H_a3_p1/6 + H_a1_p1**2*H_a2_a3*H_a2_p1*H_a3_p2/6 +
H_a1_p1**2*H_a2_a3*H_a2_p2*H_a3_p1/12 + 3*H_a1_p1**2*H_a2_a3*H_a2_p2*H_a3_p2/4 - H_a1_p1**2*H_a2_a3*H_p1_p1*T_a2_a3/12 +
H_a1_p1**2*H_a2_a3*H_p1_p2*T_a2_a3/12 - H_a1_p1**2*H_a2_a3*H_p2_p2*T_a2_a3/4 - H_a1_p1**2*H_a2_p1**2*H_a3_a3/12 - H_a1_p1**2*H_a2_p1**2*T_a3_a3/12 -
H_a1_p1**2*H_a2_p1*H_a2_p2*H_a3_a3/8 - H_a1_p1**2*H_a2_p1*H_a2_p2*T_a3_a3/24 + H_a1_p1**2*H_a2_p1*H_a3_p1*T_a2_a3/6 +
5*H_a1_p1**2*H_a2_p1*H_a3_p2*T_a2_a3/12 - 5*H_a1_p1**2*H_a2_p2**2*H_a3_a3/12 - H_a1_p1**2*H_a2_p2**2*T_a3_a3/4 - H_a1_p1**2*H_a2_p2*H_a3_p1*T_a2_a3/12
+ H_a1_p1**2*H_a2_p2*H_a3_p2*T_a2_a3/2 + H_a1_p1**2*H_a3_a3*H_p1_p2*T_a2_a2/48 + 5*H_a1_p1**2*H_a3_a3*H_p2_p2*T_a2_a2/24 -
H_a1_p1**2*H_a3_p1*H_a3_p2*T_a2_a2/24 - 7*H_a1_p1**2*H_a3_p2**2*T_a2_a2/24 - H_a1_p1**2*H_p1_p2*T_a2_a2*T_a3_a3/48 +
H_a1_p1**2*H_p2_p2*T_a2_a2*T_a3_a3/8 + 5*H_a1_p1*H_a1_p2*H_a2_a2*H_a3_a3*H_p1_p1/24 - H_a1_p1*H_a1_p2*H_a2_a2*H_a3_a3*H_p1_p2/3 +
5*H_a1_p1*H_a1_p2*H_a2_a2*H_a3_a3*H_p2_p2/24 + H_a1_p1*H_a1_p2*H_a2_a2*H_a3_p1**2/24 + 7*H_a1_p1*H_a1_p2*H_a2_a2*H_a3_p1*H_a3_p2/12 -
H_a1_p1*H_a1_p2*H_a2_a2*H_a3_p2**2/24 + H_a1_p1*H_a1_p2*H_a2_a2*H_p1_p1*T_a3_a3/6 - H_a1_p1*H_a1_p2*H_a2_a2*H_p1_p2*T_a3_a3/4 +
H_a1_p1*H_a1_p2*H_a2_a2*H_p2_p2*T_a3_a3/6 - H_a1_p1*H_a1_p2*H_a2_a3**2*H_p1_p1/6 + H_a1_p1*H_a1_p2*H_a2_a3**2*H_p1_p2/2 -
H_a1_p1*H_a1_p2*H_a2_a3**2*H_p2_p2/6 - H_a1_p1*H_a1_p2*H_a2_a3*H_a2_p1*H_a3_p2/3 - 2*H_a1_p1*H_a1_p2*H_a2_a3*H_a2_p2*H_a3_p1/3 -
H_a1_p1*H_a1_p2*H_a2_a3*H_p1_p1*T_a2_a3/6 + H_a1_p1*H_a1_p2*H_a2_a3*H_p1_p2*T_a2_a3/2 - H_a1_p1*H_a1_p2*H_a2_a3*H_p2_p2*T_a2_a3/6 -
H_a1_p1*H_a1_p2*H_a2_p1**2*H_a3_a3/12 - H_a1_p1*H_a1_p2*H_a2_p1**2*T_a3_a3/6 + H_a1_p1*H_a1_p2*H_a2_p1*H_a2_p2*H_a3_a3/2 +
H_a1_p1*H_a1_p2*H_a2_p1*H_a2_p2*T_a3_a3/3 + H_a1_p1*H_a1_p2*H_a2_p1*H_a3_p1*T_a2_a3/6 + H_a1_p1*H_a1_p2*H_a2_p1*H_a3_p2*T_a2_a3/6 -
H_a1_p1*H_a1_p2*H_a2_p2**2*H_a3_a3/8 - H_a1_p1*H_a1_p2*H_a2_p2**2*T_a3_a3/24 - H_a1_p1*H_a1_p2*H_a2_p2*H_a3_p1*T_a2_a3/2 +
H_a1_p1*H_a1_p2*H_a3_a3*H_p1_p1*T_a2_a2/8 - H_a1_p1*H_a1_p2*H_a3_a3*H_p1_p2*T_a2_a2/4 + H_a1_p1*H_a1_p2*H_a3_a3*H_p2_p2*T_a2_a2/6 +
H_a1_p1*H_a1_p2*H_a3_p1**2*T_a2_a2/24 + 5*H_a1_p1*H_a1_p2*H_a3_p1*H_a3_p2*T_a2_a2/12 - H_a1_p1*H_a1_p2*H_a3_p2**2*T_a2_a2/8 +
H_a1_p1*H_a1_p2*H_p1_p1*T_a2_a2*T_a3_a3/12 - H_a1_p1*H_a1_p2*H_p1_p2*T_a2_a2*T_a3_a3/6 + H_a1_p1*H_a1_p2*H_p2_p2*T_a2_a2*T_a3_a3/8 -
5*H_a1_p1*H_a2_a2*H_a3_p1*H_p2_p2*T_a1_a3/12 + 7*H_a1_p1*H_a2_a2*H_a3_p2*H_p1_p1*T_a1_a3/24 + 7*H_a1_p1*H_a2_a2*H_a3_p2*H_p1_p2*T_a1_a3/12 +
H_a1_p1*H_a2_a2*H_a3_p2*H_p2_p2*T_a1_a3/4 - H_a1_p1*H_a2_a3*H_a2_p1*H_p1_p1*T_a1_a3/6 - H_a1_p1*H_a2_a3*H_a2_p1*H_p1_p2*T_a1_a3/6 -
H_a1_p1*H_a2_a3*H_a2_p2*H_p1_p1*T_a1_a3/4 - H_a1_p1*H_a2_a3*H_a2_p2*H_p1_p2*T_a1_a3/2 - H_a1_p1*H_a2_a3*H_a2_p2*H_p2_p2*T_a1_a3/12 -
H_a1_p1*H_a2_a3*H_a3_p1*H_p1_p1*T_a1_a2/4 - H_a1_p1*H_a2_a3*H_a3_p1*H_p1_p2*T_a1_a2/6 + H_a1_p1*H_a2_a3*H_a3_p1*H_p2_p2*T_a1_a2/3 -
H_a1_p1*H_a2_a3*H_a3_p2*H_p1_p1*T_a1_a2/12 - H_a1_p1*H_a2_a3*H_a3_p2*H_p1_p2*T_a1_a2/6 - H_a1_p1*H_a2_p1**2*H_a3_p2*T_a1_a3/3 -
H_a1_p1*H_a2_p1*H_a2_p2*H_a3_p2*T_a1_a3/2 - H_a1_p1*H_a2_p1*H_a3_a3*H_p1_p2*T_a1_a2/4 - 7*H_a1_p1*H_a2_p1*H_a3_a3*H_p2_p2*T_a1_a2/24 +
H_a1_p1*H_a2_p1*H_a3_p1**2*T_a1_a2/6 + H_a1_p1*H_a2_p1*H_a3_p1*H_a3_p2*T_a1_a2/6 + H_a1_p1*H_a2_p1*H_a3_p2**2*T_a1_a2/3 +
H_a1_p1*H_a2_p1*H_p1_p1*T_a1_a2*T_a3_a3/12 - H_a1_p1*H_a2_p1*H_p1_p1*T_a1_a3*T_a2_a3/4 - H_a1_p1*H_a2_p1*H_p1_p2*T_a1_a2*T_a3_a3/12 -
H_a1_p1*H_a2_p1*H_p1_p2*T_a1_a3*T_a2_a3/3 - H_a1_p1*H_a2_p1*H_p2_p2*T_a1_a2*T_a3_a3/6 + H_a1_p1*H_a2_p1*H_p2_p2*T_a1_a3*T_a2_a2/6 +
H_a1_p1*H_a2_p2**2*H_a3_a3*H_p1_p2/24 - H_a1_p1*H_a2_p2**2*H_a3_p1*H_a3_p2/12 + H_a1_p1*H_a2_p2**2*H_a3_p2**2/12 - H_a1_p1*H_a2_p2*H_p1_p1*T_a3_a3/12 -
H_a1_p1*H_a2_p2*H_p1_p2*T_a1_a3*T_a2_a3/12 - H_a1_p1*H_a2_p2*H_p1_p2*T_a1_a3*T_a3_a3/6 - H_a1_p1*H_a2_p2*H_p2_p2*T_a1_a2*T_a3_a3/12 -
H_a1_p1*H_a2_p2*H_p2_p2*T_a1_a3*T_a2_a3/12 - H_a1_p1*H_a3_p1*H_p1_p1*T_a1_a2*T_a2_a3/3 - H_a1_p1*H_a3_p1*H_p1_p2*T_a1_a2*T_a2_a3/3 -
H_a1_p1*H_a3_p1*H_p2_p2*T_a1_a3*T_a2_a2/4 - H_a1_p1*H_a3_p2*H_p1_p1*T_a1_a2*T_a2_a3/4 + H_a1_p1*H_a3_p2*H_p1_p1*T_a1_a3*T_a2_a2/8 +
H_a1_p1*H_a3_p2*H_p1_p2*T_a1_a3*T_a2_a2/4 - H_a1_p1*H_a3_p2*H_p2_p2*T_a1_a2*T_a2_a3/6 + H_a1_p1*H_a3_p2*H_p2_p2*T_a1_a3*T_a2_a2/6 +
23*H_a1_p2**2*H_a2_a2*H_a3_a3*H_p1_p1/48 + 5*H_a1_p2**2*H_a2_a2*H_a3_a3*H_p1_p2/24 + 5*H_a1_p2**2*H_a2_a2*H_a3_a3*H_p2_p2/24 -
H_a1_p2**2*H_a2_a2*H_a3_p1**2/3 - H_a1_p2**2*H_a2_a2*H_a3_p1*H_a3_p2/12 - H_a1_p2**2*H_a2_a2*H_a3_p2**2/12 + 17*H_a1_p2**2*H_a2_a2*H_p1_p1*T_a3_a3/48
+ H_a1_p2**2*H_a2_a2*H_p1_p2*T_a3_a3/6 + H_a1_p2**2*H_a2_a2*H_p2_p2*T_a3_a3/6 - H_a1_p2**2*H_a2_a3**2*H_p1_p1/2 - H_a1_p2**2*H_a2_a3**2*H_p1_p2/6 -
H_a1_p2**2*H_a2_a3**2*H_p2_p2/6 + H_a1_p2**2*H_a2_a3*H_a2_p1*H_a3_p1/2 + H_a1_p2**2*H_a2_a3*H_a2_p1*H_a3_p2/6 - H_a1_p2**2*H_a2_a3*H_a2_p2*H_a3_p1/6 -
H_a1_p2**2*H_a2_a3*H_p1_p1*T_a2_a3/2 - H_a1_p2**2*H_a2_a3*H_p1_p2*T_a2_a3/6 - H_a1_p2**2*H_a2_a3*H_p2_p2*T_a2_a3/6 - H_a1_p2**2*H_a2_p1**2*H_a3_a3/3 -
```

```
H_a1_p2**2*H_a2_p1**2*T_a3_a3/3 - H_a1_p2**2*H_a2_p1*H_a2_p2*H_a3_a3/12 - H_a1_p2**2*H_a2_p1*H_a2_p2*T_a3_a3/6 + H_a1_p2**2*H_a2_p1*H_a3_p1*T_a2_a3/3
+ H_a1_p2**2*H_a2_p1*H_a3_p2*T_a2_a3/3 - H_a1_p2**2*H_a2_p2**2*H_a3_a3/12 - H_a1_p2**2*H_a2_p2**2*T_a3_a3/12 - H_a1_p2**2*H_a2_p2*H_a3_p1*T_a2_a3/3 +
17*H_a1_p2**2*H_a3_a3*H_p1_p1*T_a2_a2/48 + H_a1_p2**2*H_a3_a3*H_p1_p2*T_a2_a2/6 + H_a1_p2**2*H_a3_a3*H_p2_p2*T_a2_a2/6 -
H_a1_p2**2*H_a3_p1**2*T_a2_a2/4 - H_a1_p2**2*H_a3_p2**2*T_a2_a2/12 + 11*H_a1_p2**2*H_p1_p1*T_a2_a2*T_a3_a3/48 + H_a1_p2**2*H_p1_p2*T_a2_a2*T_a3_a3/8 +
H_a1_p2**2*H_p2_p2*T_a2_a2*T_a3_a3/8 - 7*H_a1_p2*H_a2_a2*H_a3_p1*H_p1_p1*T_a1_a3/24 - 3*H_a1_p2*H_a2_a2*H_a3_p1*H_p2_p2*T_a1_a3/8 -
H_a1_p2*H_a2_a2*H_a3_p2*H_p1_p1*T_a1_a3/24 + H_a1_p2*H_a2_a2*H_a3_p2*H_p1_p2*T_a1_a3/4 - H_a1_p2*H_a2_a3*H_a2_p1*H_p1_p1*T_a1_a3/3 -
5*H_a1_p2*H_a2_a3*H_a2_p1*H_p1_p2*T_a1_a3/6 - H_a1_p2*H_a2_a3*H_a2_p1*H_p2_p2*T_a1_a3/3 - H_a1_p2*H_a2_a3*H_a2_p2*H_p1_p1*T_a1_a3/6 -
H_a1_p2*H_a2_a3*H_a2_p2*H_p1_p2*T_a1_a3/3 - H_a1_p2*H_a2_a3*H_a2_p2*H_p2_p2*T_a1_a3/12 - H_a1_p2*H_a2_a3*H_a3_p1*H_p1_p1*T_a1_a2/12 +
H_a1_p2*H_a2_a3*H_a3_p1*H_p2_p2*T_a1_a2/4 + H_a1_p2*H_a2_a3*H_a3_p2*H_p1_p1*T_a1_a2/12 + H_a1_p2*H_a2_a3*H_a3_p2*H_p1_p2*T_a1_a2/6 -
H_a1_p2*H_a2_a3*H_a3_p2*H_p2_p2*T_a1_a2/12 + H_a1_p2*H_a2_p1**2*H_a3_p1*T_a1_a3/3 + H_a1_p2*H_a2_p1**2*H_a3_p2*T_a1_a3/3 +
H_a1_p2*H_a2_p1*H_a2_p2*H_a3_p1*T_a1_a3/6 + H_a1_p2*H_a2_p1*H_a2_p2*H_a3_p2*T_a1_a3/6 + 3*H_a1_p2*H_a2_p1*H_a3_a3*H_p1_p1*T_a1_a2/8 +
H_a1_p2*H_a2_p1*H_a3_a3*H_p1_p2*T_a1_a2/2 + 3*H_a1_p2*H_a2_p1*H_a3_a3*H_p2_p2*T_a1_a2/8 - H_a1_p2*H_a2_p1*H_a3_p1**2*T_a1_a2/4 -
H_a1_p2*H_a2_p1*H_a3_p1*H_a3_p2*T_a1_a2/2 - H_a1_p2*H_a2_p1*H_a3_p2**2*T_a1_a2/4 + 3*H_a1_p2*H_a2_p1*H_p1_p1*T_a1_a2*T_a3_a3/8 -
2*H_a1_p2*H_a2_p1*H_p1_p1*T_a1_a3*T_a2_a3/3 + H_a1_p2*H_a2_p1*H_p1_p2*T_a1_a2*T_a3_a3/2 - H_a1_p2*H_a2_p1*H_p1_p2*T_a1_a3*T_a2_a3 +
3*H_a1_p2*H_a2_p1*H_p2_p2*T_a1_a2*T_a3_a3/8 - 2*H_a1_p2*H_a2_p1*H_p2_p2*T_a1_a3*T_a2_a3/3 + 5*H_a1_p2*H_a2_p2**2*H_a3_p1*T_a1_a3/12 +
H_a1_p2*H_a2_p2**2*H_a3_p2*T_a1_a3/6 - 7*H_a1_p2*H_a2_p2*H_a3_a3*H_p1_p1*T_a1_a2/24 - H_a1_p2*H_a2_p2*H_a3_a3*H_p1_p2*T_a1_a2/4 +
H_a1_p2*H_a2_p2*H_a3_p1**2*T_a1_a2/3 + H_a1_p2*H_a2_p2*H_a3_p1*H_a3_p2*T_a1_a2/6 + H_a1_p2*H_a2_p2*H_a3_p2**2*T_a1_a2/6 -
H_a1_p2*H_a2_p2*H_p1_p1*T_a1_a2*T_a3_a3/24 - H_a1_p2*H_a2_p2*H_p1_p1*T_a1_a3*T_a2_a3/2 - H_a1_p2*H_a2_p2*H_p1_p2*T_a1_a2*T_a3_a3/12 -
H_a1_p2*H_a2_p2*H_p1_p2*T_a1_a3*T_a2_a3/3 + H_a1_p2*H_a2_p2*H_p2_p2*T_a1_a2*T_a3_a3/12 - H_a1_p2*H_a2_p2*H_p2_p2*T_a1_a3*T_a2_a3/4 -
H_a1_p2*H_a3_p1*H_p1_p1*T_a1_a2*T_a2_a3/12 - H_a1_p2*H_a3_p1*H_p1_p1*T_a1_a3*T_a2_a2/8 + H_a1_p2*H_a3_p1*H_p1_p2*T_a1_a2*T_a2_a3/6 +
5*H_a1_p2*H_a3_p1*H_p2_p2*T_a1_a2*T_a2_a3/12 - 7*H_a1_p2*H_a3_p1*H_p2_p2*T_a1_a3*T_a2_a2/24 - 5*H_a1_p2*H_a3_p2*H_p1_p1*T_a1_a2*T_a2_a3/12 +
H_a1_p2*H_a3_p2*H_p1_p1*T_a1_a3*T_a2_a2/8 - H_a1_p2*H_a3_p2*H_p1_p2*T_a1_a2*T_a2_a3/6 + H_a1_p2*H_a3_p2*H_p1_p2*T_a1_a3*T_a2_a2/4 -
H_a1_p2*H_a3_p2*H_p2_p2*T_a1_a2*T_a2_a3/4 + H_a1_p2*H_a3_p2*H_p2_p2*T_a1_a3*T_a2_a2/12 - 7*H_a2_a2*H_a3_a3*H_p1_p1*H_p2_p2*T_a1_a1/48 +
7*H_a2_a2*H_a3_a3*H_p1_p2**2*T_a1_a1/48 + H_a2_a2*H_a3_p1**2*H_p1_p2*T_a1_a1/48 + 11*H_a2_a2*H_a3_p1**2*H_p2_p2*T_a1_a1/48 -
H_a2_a2*H_a3_p1*H_a3_p2*H_p1_p1*T_a1_a1/6 - 13*H_a2_a2*H_a3_p1*H_a3_p2*H_p1_p2*T_a1_a1/24 - H_a2_a2*H_a3_p1*H_a3_p2*H_p2_p2*T_a1_a1/8 +
3*H_a2_a2*H_a3_p2**2*H_p1_p1*T_a1_a1/16 - H_a2_a2*H_a3_p2**2*H_p1_p2*T_a1_a1/48 - 5*H_a2_a2*H_p1_p1*H_p2_p2*T_a1_a1*T_a3_a3/48 +
5*H_a2_a2*H_p1_p2**2*T_a1_a1*T_a3_a3/48 + H_a2_a3**2*H_p1_p1**2*T_a1_a1/24 - H_a2_a3**2*H_p1_p1*H_p1_p2*T_a1_a1/8 +
H_a2_a3**2*H_p1_p1*H_p2_p2*T_a1_a1/8 - H_a2_a3**2*H_p1_p2**2*T_a1_a1/4 + H_a2_a3*H_a2_p1*H_a3_p1*H_p1_p2*T_a1_a1/6 -
7*H_a2_a3*H_a2_p1*H_a3_p1*H_p2_p2*T_a1_a1/24 - H_a2_a3*H_a2_p1*H_a3_p2*H_p1_p1*T_a1_a1/12 + H_a2_a3*H_a2_p1*H_a3_p2*H_p1_p2*T_a1_a1/3 -
H_a2_a3*H_a2_p1*H_a3_p2*H_p2_p2*T_a1_a1/8 + H_a2_a3*H_a2_p2*H_a3_p1*H_p1_p1*T_a1_a1/6 + 7*H_a2_a3*H_a2_p2*H_a3_p1*H_p1_p2*T_a1_a1/12 +
H_a2_a3*H_a2_p2*H_a3_p1*H_p2_p2*T_a1_a1/8 - 3*H_a2_a3*H_a2_p2*H_a3_p2*H_p1_p1*T_a1_a1/8 + H_a2_a3*H_p1_p1**2*T_a1_a1*T_a2_a3/24 +
H_a2_a3*H_p1_p1**2*T_a1_a2*T_a1_a3/2 - H_a2_a3*H_p1_p1*H_p1_p2*T_a1_a1*T_a2_a3/8 + 2*H_a2_a3*H_p1_p1*H_p1_p2*T_a1_a2*T_a1_a3/3 +
H_a2_a3*H_p1_p1*H_p2_p2*T_a1_a1*T_a2_a3/8 + 2*H_a2_a3*H_p1_p1*H_p2_p2*T_a1_a2*T_a1_a3/3 - H_a2_a3*H_p1_p2**2*T_a1_a1*T_a2_a3/4 +
H_a2_a3*H_p1_p2**2*T_a1_a2*T_a1_a3/3 + 2*H_a2_a3*H_p1_p2*H_p2_p2*T_a1_a2*T_a1_a3/3 + H_a2_a3*H_p2_p2**2*T_a1_a2*T_a1_a3/3 +
H_a2_p1**2*H_a3_a3*H_p1_p1*T_a1_a1/6 + H_a2_p1**2*H_a3_a3*H_p1_p2*T_a1_a1/6 + 17*H_a2_p1**2*H_a3_a3*H_p2_p2*T_a1_a1/48 -
H_a2_p1**2*H_a3_p1**2*T_a1_a1/12 - H_a2_p1**2*H_a3_p2**2*T_a1_a1/4 + H_a2_p1**2*H_p1_p1*T_a1_a1*T_a3_a3/8 + H_a2_p1**2*H_p1_p2*T_a1_a1*T_a3_a3/8 +
11*H_a2_p1**2*H_p2_p2*T_a1_a1*T_a3_a3/48 + H_a2_p1*H_a2_p2*H_a3_a3*H_p1_p1*T_a1_a1/6 - H_a2_p1*H_a2_p2*H_a3_a3*H_p1_p2*T_a1_a1/4 +
H_a2_p1*H_a2_p2*H_a3_a3*H_p2_p2*T_a1_a1/8 - H_a2_p1*H_a2_p2*H_a3_p1**2*T_a1_a1/8 + 5*H_a2_p1*H_a2_p2*H_a3_p1*H_a3_p2*T_a1_a1/12 +
H_a2_p1*H_a2_p2*H_a3_p2**2*T_a1_a1/24 + H_a2_p1*H_a2_p2*H_p1_p1*T_a1_a1*T_a3_a3/8 - H_a2_p1*H_a2_p2*H_p1_p2*T_a1_a1*T_a3_a3/6 +
H_a2_p1*H_a2_p2*H_p2_p2*T_a1_a1*T_a3_a3/12 + H_a2_p1*H_a3_p1*H_p1_p1*T_a1_a1*T_a2_a3/12 - H_a2_p1*H_a3_p1*H_p1_p1*T_a1_a2*T_a1_a3/4 +
H_a2_p1*H_a3_p1*H_p1_p2*T_a1_a1*T_a2_a3/4 - H_a2_p1*H_a3_p1*H_p1_p2*T_a1_a2*T_a1_a3/6 + H_a2_p1*H_a3_p1*H_p2_p2*T_a1_a1*T_a2_a3/8 -
5*H_a2_p1*H_a3_p1*H_p2_p2*T_a1_a2*T_a1_a3/12 - 7*H_a2_p1*H_a3_p2*H_p1_p1*T_a1_a1*T_a2_a3/24 + 5*H_a2_p1*H_a3_p2*H_p1_p1*T_a1_a2*T_a1_a3/12 +
H_a2_p1*H_a3_p2*H_p1_p2*T_a1_a2*T_a1_a3/6 - H_a2_p1*H_a3_p2*H_p2_p2*T_a1_a1*T_a2_a3/8 - H_a2_p1*H_a3_p2*H_p2_p2*T_a1_a2*T_a1_a3/12 +
5*H_a2_p2**2*H_a3_a3*H_p1_p1*T_a1_a1/24 + H_a2_p2**2*H_a3_a3*H_p1_p2*T_a1_a1/48 - 7*H_a2_p2**2*H_a3_p1**2*T_a1_a1/24 -
H_a2_p2**2*H_a3_p1*H_a3_p2*T_a1_a1/24 + H_a2_p2**2*H_p1_p1*T_a1_a1*T_a3_a3/8 - H_a2_p2**2*H_p1_p2*T_a1_a1*T_a3_a3/48 +
H_a2_p2*H_a3_p1*H_p1_p1*T_a1_a1*T_a2_a3/6 - H_a2_p2*H_a3_p1*H_p1_p1*T_a1_a2*T_a1_a3/6 + H_a2_p2*H_a3_p1*H_p1_p2*T_a1_a1*T_a2_a3/4 +
H_a2_p2*H_a3_p1*H_p2_p2*T_a1_a1*T_a2_a3/8 - H_a2_p2*H_a3_p1*H_p2_p2*T_a1_a2*T_a1_a3/4 - H_a2_p2*H_a3_p2*H_p1_p1*T_a1_a1*T_a2_a3/4 -
H_a2_p2*H_a3_p2*H_p1_p2*T_a1_a2*T_a1_a3/3 - H_a2_p2*H_a3_p2*H_p2_p2*T_a1_a2*T_a1_a3/3 - 5*H_a3_a3*H_p1_p1*H_p2_p2*T_a1_a1*T_a2_a2/48 +
5*H_a3_a3*H_p1_p2**2*T_a1_a1*T_a2_a2/48 + H_a3_p1**2*H_p1_p2*T_a1_a1*T_a2_a2/48 + 7*H_a3_p1**2*H_p2_p2*T_a1_a1*T_a2_a2/48 -
H_a3_p1*H_a3_p2*H_p1_p1*T_a1_a1*T_a2_a2/12 - 7*H_a3_p1*H_a3_p2*H_p1_p2*T_a1_a1*T_a2_a2/24 - H_a3_p1*H_a3_p2*H_p2_p2*T_a1_a1*T_a2_a2/12 +
7*H_a3_p2**2*H_p1_p1*T_a1_a1*T_a2_a2/48 + H_a3_p2**2*H_p1_p2*T_a1_a1*T_a2_a2/48 + H_p1_p1**2*T_a1_a2*T_a1_a3*T_a2_a3/2 +
H_p1_p1*H_p1_p2*T_a1_a2*T_a1_a3*T_a2_a3 - H_p1_p1*H_p2_p2*T_a1_a1*T_a2_a2*T_a3_a3/16 + H_p1_p1*H_p2_p2*T_a1_a2*T_a1_a3*T_a2_a3 +
H_p1_p2**2*T_a1_a1*T_a2_a2*T_a3_a3/16 + H_p1_p2**2*T_a1_a2*T_a1_a3*T_a2_a3/2 + H_p1_p2*H_p2_p2*T_a1_a2*T_a1_a3*T_a2_a3 +
H_p2_p2**2*T_a1_a2*T_a1_a3*T_a2_a3/2",
   "sha256": "3a037825e56686e4ec7705eba09c02a7c67b8e2fd5c73aa9774857d37c013339",
   "n_terms": 607,
   "zero": false,
   "n_evanescent_terms": 426,
   "momentum_degree": 4,
   "seconds": 278.1
  },
  "G09": {
   "pole_over_U_d4_residue": "25*I*H_a3_a3*H_gb2_gc1*H_p1_p1*H_p1_p2/96 + 7*I*H_a3_a3*H_gb2_gc1*H_p1_p1*H_p2_p2/48 +
3*I*H_a3_a3*H_gb2_gc1*H_p1_p2**2/8 + 3*I*H_a3_a3*H_gb2_gc1*H_p1_p2*H_p2_p2/16 - I*H_a3_a3*H_gb2_gc1*H_p2_p2**2/32 -
5*I*H_a3_a3*H_gb2_p1*H_gc1_p1*H_p1_p2/48 + 25*I*H_a3_a3*H_gb2_p1*H_gc1_p2*H_p1_p1/96 + I*H_a3_a3*H_gb2_p1*H_gc1_p2*H_p1_p2/6 +
I*H_a3_a3*H_gb2_p1*H_gc1_p2*H_p2_p2/8 - 5*I*H_a3_a3*H_gb2_p2*H_gc1_p1*H_p1_p1/96 - 7*I*H_a3_a3*H_gb2_p2*H_gc1_p1*H_p1_p2/24 -
I*H_a3_a3*H_gb2_p2*H_gc1_p1*H_p2_p2/12 + 7*I*H_a3_a3*H_gb2_p2*H_gc1_p2*H_p1_p1/48 - I*H_a3_a3*H_gb2_p2*H_gc1_p2*H_p1_p2/12 -
I*H_a3_a3*H_gb2_p2*H_gc1_p2*H_p2_p2/24 + 13*I*H_a3_a3*H_p1_p1*H_p1_p2*T_gb2_gc1/48 + 13*I*H_a3_a3*H_p1_p1*H_p2_p2*T_gb2_gc1/48 +
7*I*H_a3_a3*H_p1_p2**2*T_gb2_gc1/24 + 5*I*H_a3_a3*H_p1_p2*H_p2_p2*T_gb2_gc1/24 + I*H_a3_gb2*H_a3_gc1*H_p1_p1*H_p2_p2/6 -
I*H_a3_gb2*H_a3_gc1*H_p1_p2**2/6 - I*H_a3_gb2*H_a3_gc1*H_p1_p2*H_p2_p2/12 - I*H_a3_gb2*H_a3_gc1*H_p2_p2**2/24 + I*H_a3_gb2*H_a3_p1*H_gc1_p1*H_p1_p2/24
- 5*I*H_a3_gb2*H_a3_p1*H_gc1_p1*H_p2_p2/48 - 13*I*H_a3_gb2*H_a3_p1*H_gc1_p2*H_p1_p1/48 - I*H_a3_gb2*H_a3_p1*H_gc1_p2*H_p1_p2/8 -
I*H_a3_gb2*H_a3_p1*H_gc1_p2*H_p2_p2/48 - I*H_a3_gb2*H_a3_p2*H_gc1_p1*H_p1_p1/12 + 5*I*H_a3_gb2*H_a3_p2*H_gc1_p1*H_p1_p2/24 -
I*H_a3_gb2*H_a3_p2*H_gc1_p1*H_p2_p2/48 - 17*I*H_a3_gb2*H_a3_p2*H_gc1_p2*H_p1_p1/48 - I*H_a3_gb2*H_a3_p2*H_gc1_p2*H_p1_p2/8 -
5*I*H_a3_gb2*H_a3_p2*H_gc1_p2*H_p2_p2/48 + I*H_a3_gb2*H_p1_p1*H_p1_p2*T_a3_gc1/24 + I*H_a3_gb2*H_p1_p1*H_p2_p2*T_a3_gc1/8 -
I*H_a3_gb2*H_p1_p2**2*T_a3_gc1/8 - I*H_a3_gb2*H_p1_p2*H_p2_p2*T_a3_gc1/12 + I*H_a3_gc1*H_a3_p1*H_gb2_p1*H_p1_p2/24 -
5*I*H_a3_gc1*H_a3_p1*H_gb2_p1*H_p2_p2/48 - I*H_a3_gc1*H_a3_p1*H_gb2_p2*H_p1_p1/16 + I*H_a3_gc1*H_a3_p1*H_gb2_p2*H_p1_p2/8 -
5*I*H_a3_gc1*H_a3_p1*H_gb2_p2*H_p2_p2/48 + I*H_a3_gc1*H_a3_p2*H_gb2_p1*H_p1_p2/24 + I*H_a3_gc1*H_a3_p2*H_gb2_p1*H_p2_p2/16 +
I*H_a3_gc1*H_a3_p2*H_gb2_p2*H_p1_p1/48 + I*H_a3_gc1*H_a3_p2*H_gb2_p2*H_p1_p2/8 + I*H_a3_gc1*H_a3_p2*H_gb2_p2*H_p2_p2/16 -
I*H_a3_gc1*H_p1_p1*H_p1_p2*T_a3_gb2/24 + I*H_a3_gc1*H_p1_p1*H_p2_p2*T_a3_gb2/24 - I*H_a3_gc1*H_p1_p2**2*T_a3_gb2/24 -
I*H_a3_gc1*H_p2_p2**2*T_a3_gb2/24 - I*H_a3_p1**2*H_gb2_gc1*H_p1_p2/8 - 5*I*H_a3_p1**2*H_gb2_gc1*H_p2_p2/48 - I*H_a3_p1**2*H_gb2_p1*H_gc1_p2/8 +
I*H_a3_p1**2*H_gb2_p2*H_gc1_p1/24 - I*H_a3_p1**2*H_gb2_p2*H_gc1_p2/24 - I*H_a3_p1**2*H_p1_p2*T_gb2_gc1/12 - I*H_a3_p1**2*H_p2_p2*T_gb2_gc1/6 -
13*I*H_a3_p1*H_a3_p2*H_gb2_gc1*H_p1_p1/48 - 7*I*H_a3_p1*H_a3_p2*H_gb2_gc1*H_p1_p2/12 - I*H_a3_p1*H_a3_p2*H_gb2_gc1*H_p2_p2/4 +
I*H_a3_p1*H_a3_p2*H_gb2_p1*H_gc1_p1/24 - I*H_a3_p1*H_a3_p2*H_gb2_p1*H_gc1_p2/12 + I*H_a3_p1*H_a3_p2*H_gb2_p2*H_gc1_p1/12 -
3*I*H_a3_p1*H_a3_p2*H_p1_p1*T_gb2_gc1/8 - 5*I*H_a3_p1*H_a3_p2*H_p1_p2*T_gb2_gc1/12 - I*H_a3_p1*H_a3_p2*H_p2_p2*T_gb2_gc1/4 -
```

```
I*H_a3_p1*H_gb2_p1*H_p2_p2*T_a3_gc1/24 + I*H_a3_p1*H_gb2_p2*H_p1_p1*T_a3_gc1/24 + I*H_a3_p1*H_gb2_p2*H_p1_p2*T_a3_gc1/24 -
I*H_a3_p1*H_gb2_p2*H_p2_p2*T_a3_gc1/6 + I*H_a3_p1*H_gc1_p1*H_p1_p2*T_a3_gb2/12 - I*H_a3_p1*H_gc1_p2*H_p1_p1*T_a3_gb2/4 -
5*I*H_a3_p1*H_gc1_p2*H_p1_p2*T_a3_gb2/24 + 7*I*H_a3_p2**2*H_gb2_gc1*H_p1_p1/48 - I*H_a3_p2**2*H_gb2_gc1*H_p1_p2/24 + I*H_a3_p2**2*H_gb2_gc1*H_p2_p2/48
- I*H_a3_p2**2*H_gb2_p1*H_gc1_p1/24 - I*H_a3_p2**2*H_gb2_p1*H_gc1_p2/24 + I*H_a3_p2**2*H_gb2_p2*H_gc1_p1/24 + I*H_a3_p2**2*H_gb2_p2*H_gc1_p2/24 -
I*H_a3_p2**2*H_p1_p1*T_gb2_gc1/24 - I*H_a3_p2**2*H_p1_p2*T_gb2_gc1/12 + I*H_a3_p2*H_gb2_p1*H_p1_p1*T_a3_gc1/24 -
I*H_a3_p2*H_gb2_p1*H_p1_p2*T_a3_gc1/24 + 7*I*H_a3_p2*H_gb2_p2*H_p1_p1*T_a3_gc1/24 + I*H_a3_p2*H_gb2_p2*H_p1_p2*T_a3_gc1/4 -
I*H_a3_p2*H_gc1_p1*H_p1_p1*T_a3_gb2/24 + I*H_a3_p2*H_gc1_p1*H_p1_p2*T_a3_gb2/8 - I*H_a3_p2*H_gc1_p1*H_p2_p2*T_a3_gb2/24 -
I*H_a3_p2*H_gc1_p2*H_p1_p1*T_a3_gb2/6 - I*H_a3_p2*H_gc1_p2*H_p1_p2*T_a3_gb2/12 - I*H_a3_p2*H_gc1_p2*H_p2_p2*T_a3_gb2/12 +
29*I*H_gb2_gc1*H_p1_p1*H_p1_p2*T_a3_a3/96 + 7*I*H_gb2_gc1*H_p1_p1*H_p2_p2*T_a3_a3/16 + I*H_gb2_gc1*H_p1_p2**2*T_a3_a3/3 +
17*I*H_gb2_gc1*H_p1_p2*H_p2_p2*T_a3_a3/48 - I*H_gb2_gc1*H_p2_p2**2*T_a3_a3/32 - 3*I*H_gb2_p1*H_gc1_p1*H_p1_p2*T_a3_a3/16 -
I*H_gb2_p1*H_gc1_p1*H_p2_p2*T_a3_a3/8 + 29*I*H_gb2_p1*H_gc1_p2*H_p1_p1*T_a3_a3/96 + 7*I*H_gb2_p1*H_gc1_p2*H_p1_p2*T_a3_a3/24 +
5*I*H_gb2_p1*H_gc1_p2*H_p2_p2*T_a3_a3/24 - 13*I*H_gb2_p2*H_gc1_p1*H_p1_p1*T_a3_a3/96 - I*H_gb2_p2*H_gc1_p1*H_p1_p2*T_a3_a3/4 -
I*H_gb2_p2*H_gc1_p1*H_p2_p2*T_a3_a3/12 - I*H_gb2_p2*H_gc1_p2*H_p1_p1*T_a3_a3/16 - I*H_gb2_p2*H_gc1_p2*H_p1_p2**2*T_a3_a3/6 +
5*I*H_p1_p1*H_p1_p2*T_a3_a3*T_gb2_gc1/16 + 9*I*H_p1_p1*H_p2_p2*T_a3_a3*T_gb2_gc1/16 + I*H_p1_p2**2*T_a3_a3*T_gb2_gc1/4 +
3*I*H_p1_p2*H_p2_p2*T_a3_a3*T_gb2_gc1/8",
   "sha256": "3daf97ec67d970ac19f3de42199c201c2ccbf0309d85c653485f7d3021acd5d5",
   "n_terms": 113,
   "zero": false,
   "n_evanescent_terms": 55,
   "momentum_degree": 4,
   "seconds": 713.5
  },
  "G03": {
   "pole_over_U_d4_residue": "-5*I*H_a3_a3*H_aK_aK*H_gc1_p1*H_p1_p1/64 - I*H_a3_a3*H_aK_aK*H_gc1_p1*H_p1_p2/16 - I*H_a3_a3*H_aK_aK*H_gc1_p1*H_p2_p2/24
- 5*I*H_a3_a3*H_aK_aK*H_gc1_p2*H_p1_p1/24 - 11*I*H_a3_a3*H_aK_aK*H_gc1_p2*H_p1_p2/48 - 23*I*H_a3_a3*H_aK_aK*H_gc1_p2*H_p2_p2/192 +
29*I*H_a3_a3*H_aK_gc1*H_aK_p1*H_p1_p1/96 + 35*I*H_a3_a3*H_aK_gc1*H_aK_p1*H_p1_p2/48 + 11*I*H_a3_a3*H_aK_gc1*H_aK_p1*H_p2_p2/24 -
I*H_a3_a3*H_aK_gc1*H_aK_p2*H_p1_p1/4 - 23*I*H_a3_a3*H_aK_gc1*H_aK_p2*H_p1_p2/48 - 5*I*H_a3_a3*H_aK_gc1*H_aK_p2*H_p2_p2/32 -
29*I*H_a3_a3*H_aK_p1**2*H_gc1_p1/96 - 11*I*H_a3_a3*H_aK_p1**2*H_gc1_p2/48 - 5*I*H_a3_a3*H_aK_p1*H_aK_p2*H_gc1_p1/48 +
I*H_a3_a3*H_aK_p1*H_aK_p2*H_gc1_p2/16 + 3*I*H_a3_a3*H_aK_p1*H_p1_p1*T_aK_gc1/16 + 17*I*H_a3_a3*H_aK_p1*H_p1_p2*T_aK_gc1/24 +
23*I*H_a3_a3*H_aK_p1*H_p2_p2*T_aK_gc1/48 + I*H_a3_a3*H_aK_p2**2*H_gc1_p1/4 + 29*I*H_a3_a3*H_aK_p2**2*H_gc1_p2/96 -
13*I*H_a3_a3*H_aK_p2*H_p1_p1*T_aK_gc1/48 - 7*I*H_a3_a3*H_aK_p2*H_p1_p2*T_aK_gc1/24 + I*H_a3_a3*H_gc1_p1*H_p1_p1*T_aK_aK/48 -
5*I*H_a3_a3*H_gc1_p1*H_p1_p2*T_aK_aK/24 - 13*I*H_a3_a3*H_gc1_p1*H_p2_p2*T_aK_aK/96 + 3*I*H_a3_a3*H_gc1_p2*H_p1_p1*T_aK_aK/32 -
I*H_a3_a3*H_gc1_p2*H_p2_p2*T_aK_aK/24 + I*H_a3_aK**2*H_gc1_p1*H_p1_p1/48 - I*H_a3_aK**2*H_gc1_p1*H_p1_p2/8 - I*H_a3_aK**2*H_gc1_p1*H_p2_p2/16 +
7*I*H_a3_aK**2*H_gc1_p2*H_p1_p1/24 + I*H_a3_aK**2*H_gc1_p2*H_p1_p2/6 + I*H_a3_aK**2*H_gc1_p2*H_p2_p2/12 - I*H_a3_aK*H_a3_gc1*H_aK_p1*H_p1_p1/6 -
I*H_a3_aK*H_a3_gc1*H_aK_p1*H_p1_p2/24 - I*H_a3_aK*H_a3_gc1*H_aK_p2*H_p1_p1/3 - I*H_a3_aK*H_a3_gc1*H_aK_p2*H_p1_p2/8 +
5*I*H_a3_aK*H_a3_gc1*H_aK_p2*H_p2_p2/24 + 13*I*H_a3_aK*H_a3_p1*H_aK_gc1*H_p1_p1/48 + 13*I*H_a3_aK*H_a3_p1*H_aK_gc1*H_p1_p2/48 +
7*I*H_a3_aK*H_a3_p1*H_aK_gc1*H_p2_p2/48 + 5*I*H_a3_aK*H_a3_p1*H_aK_p1*H_gc1_p1/24 - I*H_a3_aK*H_a3_p1*H_aK_p1*H_gc1_p2/48 -
17*I*H_a3_aK*H_a3_p1*H_aK_p2*H_gc1_p1/48 - 13*I*H_a3_aK*H_a3_p1*H_aK_p2*H_gc1_p2/24 + 7*I*H_a3_aK*H_a3_p1*H_p1_p1*T_aK_gc1/24 -
I*H_a3_aK*H_a3_p1*H_p1_p2*T_aK_gc1/12 - I*H_a3_aK*H_a3_p1*H_p2_p2*T_aK_gc1/8 + 5*I*H_a3_aK*H_a3_p2*H_aK_gc1*H_p1_p1/48 +
7*I*H_a3_aK*H_a3_p2*H_aK_gc1*H_p1_p2/48 - I*H_a3_aK*H_a3_p2*H_aK_gc1*H_p2_p2/48 + I*H_a3_aK*H_a3_p2*H_aK_p1*H_gc1_p1/4 +
7*I*H_a3_aK*H_a3_p2*H_aK_p1*H_gc1_p2/48 - 3*I*H_a3_aK*H_a3_p2*H_aK_p2*H_gc1_p1/16 - I*H_a3_aK*H_a3_p2*H_aK_p2*H_gc1_p2/3 +
7*I*H_a3_aK*H_a3_p2*H_p1_p1*T_aK_gc1/24 + I*H_a3_aK*H_a3_p2*H_p1_p2*T_aK_gc1/4 - I*H_a3_aK*H_aK_p1*H_p1_p1*T_a3_gc1/6 +
I*H_a3_aK*H_aK_p1*H_p1_p2*T_a3_gc1/8 + I*H_a3_aK*H_aK_p1*H_p2_p2*T_a3_gc1/8 - 3*I*H_a3_aK*H_aK_p2*H_p1_p1*T_a3_gc1/8 -
5*I*H_a3_aK*H_aK_p2*H_p1_p2*T_a3_gc1/12 - I*H_a3_aK*H_gc1_p1*H_p1_p1*T_a3_aK/24 + I*H_a3_aK*H_gc1_p1*H_p1_p2*T_a3_aK/12 +
5*I*H_a3_aK*H_gc1_p2*H_p1_p1*T_a3_aK/24 + I*H_a3_aK*H_gc1_p2*H_p1_p2*T_a3_aK/6 + I*H_a3_aK*H_gc1_p2*H_p2_p2*T_a3_aK/12 -
13*I*H_a3_gc1*H_a3_p1*H_aK_aK*H_p1_p1/96 - 31*I*H_a3_gc1*H_a3_p1*H_aK_aK*H_p1_p2/96 - 23*I*H_a3_gc1*H_a3_p1*H_aK_aK*H_p2_p2/96 +
17*I*H_a3_gc1*H_a3_p1*H_aK_p1**2/48 + 25*I*H_a3_gc1*H_a3_p1*H_aK_p1*H_aK_p2/48 - I*H_a3_gc1*H_a3_p1*H_aK_p2**2/48 -
I*H_a3_gc1*H_a3_p1*H_p1_p1*T_aK_aK/6 + I*H_a3_gc1*H_a3_p1*H_p2_p2*T_aK_aK/48 + 7*I*H_a3_gc1*H_a3_p2*H_aK_aK*H_p1_p1/96 -
5*I*H_a3_gc1*H_a3_p2*H_aK_aK*H_p1_p2/96 - I*H_a3_gc1*H_a3_p2*H_aK_aK*H_p2_p2/96 + 3*I*H_a3_gc1*H_a3_p2*H_aK_p1**2/16 +
7*I*H_a3_gc1*H_a3_p2*H_aK_p1*H_aK_p2/48 - 7*I*H_a3_gc1*H_a3_p2*H_aK_p2**2/48 - 3*I*H_a3_gc1*H_a3_p2*H_p1_p1*T_aK_aK/16 -
I*H_a3_gc1*H_a3_p2*H_p1_p2*T_aK_aK/12 - I*H_a3_gc1*H_aK_p1*H_p1_p1*T_a3_aK/24 - 5*I*H_a3_gc1*H_aK_p1*H_p1_p2*T_a3_aK/24 -
I*H_a3_gc1*H_aK_p1*H_p2_p2*T_a3_aK/8 - I*H_a3_gc1*H_aK_p2*H_p1_p1*T_a3_aK/12 - I*H_a3_gc1*H_aK_p2*H_p1_p2*T_a3_aK/8 +
5*I*H_a3_gc1*H_aK_p2*H_p2_p2*T_a3_aK/24 + 5*I*H_a3_p1**2*H_aK_aK*H_gc1_p1/48 + 17*I*H_a3_p1**2*H_aK_aK*H_gc1_p2/96 - 3*I*H_a3_p1**2*H_aK_gc1*H_aK_p1/8
+ 13*I*H_a3_p1**2*H_aK_gc1*H_aK_p2/48 - 7*I*H_a3_p1**2*H_aK_p1*T_aK_gc1/24 + 7*I*H_a3_p1**2*H_aK_p2*T_aK_gc1/24 + I*H_a3_p1**2*H_gc1_p1*T_aK_aK/12 -
I*H_a3_p1**2*H_gc1_p2*T_aK_aK/24 + 19*I*H_a3_p1*H_a3_p2*H_aK_aK*H_gc1_p1/96 + 11*I*H_a3_p1*H_a3_p2*H_aK_aK*H_gc1_p2/32 -
43*I*H_a3_p1*H_a3_p2*H_aK_gc1*H_aK_p1/48 + 19*I*H_a3_p1*H_a3_p2*H_aK_gc1*H_aK_p2/48 - 3*I*H_a3_p1*H_a3_p2*H_aK_p1*T_aK_gc1/4 +
5*I*H_a3_p1*H_a3_p2*H_aK_p2*T_aK_gc1/12 + I*H_a3_p1*H_a3_p2*H_gc1_p1*T_aK_aK/4 - I*H_a3_p1*H_aK_aK*H_p1_p1*T_a3_gc1/8 -
7*I*H_a3_p1*H_aK_aK*H_p1_p2*T_a3_gc1/24 - 5*I*H_a3_p1*H_aK_aK*H_p2_p2*T_a3_gc1/24 + 5*I*H_a3_p1*H_aK_gc1*H_p1_p1*T_a3_aK/12 +
5*I*H_a3_p1*H_aK_gc1*H_p1_p2*T_a3_aK/12 + 7*I*H_a3_p1*H_aK_gc1*H_p2_p2*T_a3_aK/24 + 7*I*H_a3_p1*H_aK_p1**2*T_a3_gc1/24 +
I*H_a3_p1*H_aK_p1*H_aK_p2*T_a3_gc1/2 + I*H_a3_p1*H_aK_p1*H_gc1_p1*T_a3_aK/6 + I*H_a3_p1*H_aK_p1*H_gc1_p2*T_a3_aK/8 - I*H_a3_p1*H_aK_p2**2*T_a3_gc1/24
- 2*I*H_a3_p1*H_aK_p2*H_gc1_p1*T_a3_aK/3 - 7*I*H_a3_p1*H_aK_p2*H_gc1_p2*T_a3_aK/12 + 3*I*H_a3_p1*H_p1_p1*T_a3_aK*T_aK_gc1/8 -
3*I*H_a3_p1*H_p1_p1*T_a3_gc1*T_aK_aK/16 - I*H_a3_p1*H_p2_p2*T_a3_aK*T_aK_gc1/24 + I*H_a3_p1*H_p2_p2*T_a3_gc1*T_aK_aK/48 +
11*I*H_a3_p2**2*H_aK_aK*H_gc1_p1/96 + 7*I*H_a3_p2**2*H_aK_aK*H_gc1_p2/48 - 25*I*H_a3_p2**2*H_aK_gc1*H_aK_p1/48 + I*H_a3_p2**2*H_aK_gc1*H_aK_p2/8 -
13*I*H_a3_p2**2*H_aK_p1*T_aK_gc1/24 + I*H_a3_p2**2*H_gc1_p1*T_aK_aK/6 + I*H_a3_p2**2*H_gc1_p2*T_aK_aK/24 + I*H_a3_p2*H_aK_aK*H_p1_p1*T_a3_gc1/12 -
I*H_a3_p2*H_aK_aK*H_p1_p2*T_a3_gc1/24 + 5*I*H_a3_p2*H_aK_gc1*H_p1_p1*T_a3_aK/24 + I*H_a3_p2*H_aK_gc1*H_p1_p2*T_a3_aK/12 +
I*H_a3_p2*H_aK_p1**2*T_a3_gc1/12 + 5*I*H_a3_p2*H_aK_p1*H_aK_p2*T_a3_gc1/24 + 7*I*H_a3_p2*H_aK_p1*H_gc1_p1*T_a3_aK/24 +
5*I*H_a3_p2*H_aK_p1*H_gc1_p2*T_a3_aK/12 - 3*I*H_a3_p2*H_aK_p2*H_gc1_p1*T_a3_aK/8 - 7*I*H_a3_p2*H_aK_p2*H_gc1_p2*T_a3_aK/24 +
3*I*H_a3_p2*H_p1_p1*T_a3_aK*T_aK_gc1/8 - 3*I*H_a3_p2*H_p1_p1*T_a3_gc1*T_aK_aK/16 + I*H_a3_p2*H_p1_p2*T_a3_aK*T_aK_gc1/6 -
I*H_a3_p2*H_p1_p2*T_a3_gc1*T_aK_aK/12 - 19*I*H_aK_aK*H_gc1_p1*H_p1_p1*T_a3_a3/192 - 7*I*H_aK_aK*H_gc1_p1*H_p1_p2*T_a3_a3/48 -
I*H_aK_aK*H_gc1_p1*H_p2_p2*T_a3_a3/24 - 11*I*H_aK_aK*H_gc1_p2*H_p1_p1*T_a3_a3/48 - 5*I*H_aK_aK*H_gc1_p2*H_p1_p2*T_a3_a3/16 -
9*I*H_aK_aK*H_gc1_p2*H_p2_p2*T_a3_a3/64 + 7*I*H_aK_gc1*H_aK_p1*H_p1_p1*T_a3_a3/32 + 5*I*H_aK_gc1*H_aK_p1*H_p1_p2*T_a3_a3/8 +
I*H_aK_gc1*H_aK_p1*H_p2_p2*T_a3_a3/3 - 17*I*H_aK_gc1*H_aK_p2*H_p1_p1*T_a3_a3/48 - 7*I*H_aK_gc1*H_aK_p2*H_p1_p2*T_a3_a3/16 -
5*I*H_aK_gc1*H_aK_p2*H_p2_p2*T_a3_a3/32 - 25*I*H_aK_p1**2*H_gc1_p1*T_a3_a3/96 - I*H_aK_p1**2*H_gc1_p2*T_a3_a3/6 +
I*H_aK_p1*H_aK_p2*H_gc1_p1*T_a3_a3/12 + 7*I*H_aK_p1*H_aK_p2*H_gc1_p2*T_a3_a3/48 + I*H_aK_p1*H_p1_p1*T_a3_a3*T_aK_gc1/8 +
5*I*H_aK_p1*H_p1_p2*T_a3_a3*T_aK_gc1/8 + 3*I*H_aK_p1*H_p2_p2*T_a3_a3*T_aK_gc1/8 + I*H_aK_p1*H_p2_p2*T_a3_aK*T_a3_gc1/24 +
3*I*H_aK_p2**2*H_gc1_p1*T_a3_a3/8 + 11*I*H_aK_p2**2*H_gc1_p2*T_a3_a3/32 - 3*I*H_aK_p2*H_p1_p1*T_a3_a3*T_aK_gc1/8 -
I*H_aK_p2*H_p1_p1*T_a3_aK*T_a3_gc1/8 - I*H_aK_p2*H_p1_p2*T_a3_a3*T_aK_gc1/4 - 5*I*H_aK_p2*H_p1_p2*T_a3_aK*T_a3_gc1/12 -
I*H_gc1_p1*H_p1_p1*T_a3_aK**2/16 - 7*I*H_gc1_p1*H_p1_p2*T_a3_a3*T_aK_aK/24 + 5*I*H_gc1_p1*H_p1_p2*T_a3_aK**2/24 -
13*I*H_gc1_p1*H_p2_p2*T_a3_a3*T_aK_aK/96 + I*H_gc1_p1*H_p2_p2*T_a3_aK**2/16 + 7*I*H_gc1_p2*H_p1_p1*T_a3_a3*T_aK_aK/96 -
I*H_gc1_p2*H_p1_p1*T_a3_aK**2/12 - I*H_gc1_p2*H_p1_p2*T_a3_a3*T_aK_aK/12 - I*H_gc1_p2*H_p2_p2*T_a3_a3*T_aK_aK/16",
   "sha256": "6a6eb3b9679733313545f575791a13d2f6ef61fe2f162152806437563ee3f7cd",
   "n_terms": 178,
   "zero": false,
   "n_evanescent_terms": 111,
```

```
   "momentum_degree": 3,
   "seconds": 52.9
  },
  "G05": {
   "pole_over_U_d4_residue": "-H_a1_a1*H_p1_p1*H_p2_p2/2 + H_a1_a1*H_p1_p2**2/2 + H_a1_p1**2*H_p2_p2/2 - H_a1_p1*H_a1_p2*H_p1_p2 +
H_a1_p2**2*H_p1_p1/2 - H_p1_p1*H_p2_p2*T_a1_a1/2 + H_p1_p2**2*T_a1_a1/2",
   "sha256": "163ed2d356524f7d955f6507fca740878d4434dd9cb005cd4d311dc38f1fd404",
   "n_terms": 7,
   "zero": false,
   "n_evanescent_terms": 2,
   "momentum_degree": 4,
   "seconds": 7.5
  },
  "G06": {
   "pole_over_U_d4_residue": "-7*H_a1_a1*H_p1_p1*H_p1_p2/12 + 5*H_a1_a1*H_p1_p1*H_p2_p2/12 - H_a1_a1*H_p1_p2**2 + 5*H_a1_p1**2*H_p1_p2/12 -
5*H_a1_p1**2*H_p2_p2/12 + H_a1_p1*H_a1_p2*H_p1_p1/6 + 5*H_a1_p1*H_a1_p2*H_p1_p2/3 - 2*H_a1_p2**2*H_p1_p1/3 - 5*H_p1_p1*H_p1_p2*T_a1_a1/8 +
3*H_p1_p1*H_p2_p2*T_a1_a1/8 - H_p1_p2**2*T_a1_a1",
   "sha256": "e5906c1dd6fb30b525030ccddbda107bb36397a01156d953988a07436b20bcbe",
   "n_terms": 11,
   "zero": false,
   "n_evanescent_terms": 3,
   "momentum_degree": 4,
   "seconds": 6.4
  },
  "G10": {
   "pole_over_U_d4_residue": "I*H_gc1_gc2*H_p1_p1*H_p1_z/24 + I*H_gc1_gc2*H_p1_p1*H_p2_z/48 + I*H_gc1_gc2*H_p1_p2*H_p1_z/8 -
I*H_gc1_gc2*H_p1_p2*H_p2_z/8 - I*H_gc1_gc2*H_p1_z*H_p2_p2/48 - I*H_gc1_gc2*H_p2_p2*H_p2_z/24 + I*H_gc1_p1*H_gc2_p1*H_p1_z/24 -
I*H_gc1_p1*H_gc2_z*H_p1_p1/24 - 5*I*H_gc1_p1*H_gc2_z*H_p2_p2/48 - I*H_gc1_p1*H_p2_p2*T_gc2_z/8 - I*H_gc1_p2*H_gc2_p2*H_p2_z/24 -
I*H_gc1_p2*H_gc2_z*H_p1_p1/6 - I*H_gc1_p2*H_gc2_z*H_p1_p2/24 - I*H_gc1_p2*H_gc2_z*H_p2_p2/12 - I*H_gc1_p2*H_p1_p1*T_gc2_z/8 -
I*H_gc1_p2*H_p2_p2*T_gc2_z/16 + I*H_gc1_z*H_gc2_p1*H_p1_p1/12 + I*H_gc1_z*H_gc2_p1*H_p1_p2/24 + I*H_gc1_z*H_gc2_p1*H_p2_p2/6 +
5*I*H_gc1_z*H_gc2_p2*H_p1_p1/48 + I*H_gc1_z*H_gc2_p2*H_p2_p2/24 + I*H_gc2_p1*H_p1_p1*T_gc1_z/16 + I*H_gc2_p1*H_p2_p2*T_gc1_z/8 +
I*H_gc2_p2*H_p1_p1*T_gc1_z/8 + I*H_p1_p2*H_p1_z*T_gc1_gc2/8 - I*H_p1_p2*H_p2_z*T_gc1_gc2/8",
   "sha256": "2ac7149223a0f1df9951d9c1333608ff63cf5d171258d226a75c2881dadc11a2",
   "n_terms": 26,
   "zero": false,
   "n_evanescent_terms": 8,
   "momentum_degree": 3,
   "seconds": 6.1
  },
  "G11": {
   "pole_over_U_d4_residue": "I*H_gc1_p1*H_p2_p2/48 - I*H_gc1_p2*H_p1_p1/8 - I*H_gc1_p2*H_p1_p2/12",
   "sha256": "6b0ec75264e9747a4b1a5fe6cf0517153ecaf07e954ba6e6e9ca986ba42495f4",
   "n_terms": 3,
   "zero": false,
   "n_evanescent_terms": 0,
   "momentum_degree": 3,
   "seconds": 0.4
  },
  "G12": {
   "pole_over_U_d4_residue": "5*H_a1_a1*H_p1_p1*H_p1_p2/48 + 3*H_a1_a1*H_p1_p1*H_p2_p2/16 - H_a1_a1*H_p1_p2**2/8 - H_a1_a1*H_p1_p2*H_p2_p2/12 -
H_a1_a1*H_p2_p2**2/24 - H_a1_p1**2*H_p2_p2/24 - H_a1_p1*H_a1_p2*H_p1_p1/24 + H_a1_p1*H_a1_p2*H_p1_p2/4 + H_a1_p1*H_a1_p2*H_p2_p2/6 -
H_a1_p2**2*H_p1_p1/12 + H_a1_p2**2*H_p1_p2/6 + H_a1_p2**2*H_p2_p2/6 + 3*H_p1_p1*H_p1_p2*T_a1_a1/16 + 5*H_p1_p1*H_p2_p2*T_a1_a1/16 -
H_p1_p2**2*T_a1_a1/8",
   "sha256": "d9f4aeac1414749896e47620959a2a4ba8e1b27bd695d7bc5c3044f681f86027",
   "n_terms": 15,
   "zero": false,
   "n_evanescent_terms": 3,
   "momentum_degree": 4,
   "seconds": 1.7
  },
  "G13": {
   "pole_over_U_d4_residue": "H_a1_a1*H_p1_p1*H_p1_p2/24 + H_a1_a1*H_p1_p1*H_p2_p2/24 + H_a1_a1*H_p1_p2**2/12 + H_a1_a1*H_p1_p2*H_p2_p2/6 +
H_a1_a1*H_p2_p2**2/12 - H_a1_p1**2*H_p1_p2/6 - H_a1_p1**2*H_p2_p2/6 - H_a1_p1*H_a1_p2*H_p1_p2/3 - H_a1_p1*H_a1_p2*H_p2_p2/3 - H_a1_p2**2*H_p1_p2/3 -
H_a1_p2**2*H_p2_p2/3",
   "sha256": "d9a113f33da14437e5ed490f7a46341820bfc8e3da5de381b1296850c650d807",
   "n_terms": 11,
   "zero": false,
   "n_evanescent_terms": 0,
   "momentum_degree": 4,
   "seconds": 2.8
  }
 },
 "missing": {
  "G01": "V3^3 triangle: implemented (graphs.G01), evaluation did not finish within the release budget -> Missing (never zero)",
  "G02": "needs V4_grav (not constructed in Q05) -> Missing"
 },
 "carried_from_Q04": {
  "G14": "evaluated (Q04)",
  "G15": "evaluated (Q04)",
  "T1-T3": "derived 0 (scaleless)"
 },
 "convention": "pole residue at d -> 4 in units of U, hat/tilde kept (T_* evanescent); external momenta p1, p2, p3 = -p1-p2; graph-level conventions
in code/p6q05/graphs.py docstrings"
}
```

certificates/Q05_guard_scan.json


```
{
 "scope": "every .py under code/ (recursive), minus declared probe files",
 "probe_files": [
  "code/p6q05/guard_scan2.py",
  "code/q01/guard_probe.py",
  "code/q04/guard_scan.py"
 ],
 "n_files_scanned": 65,
 "rules": [
  "F1 primitive reference/alias/string",
  "F2 non-literal getattr",
  "F3 .replace on non-literal",
  "F4 builtin rebinding",
  "F5 .name comparison",
  "F6 dynamic exec/eval/import"
 ],
 "reviewer_evasions_flagged": {
  "aliased_lambdify": true,
  "getattr_concat_srepr": true,
  "fstring_replace": true,
  "dunder_str_replace": true,
  "format_replace": true,
  "rebound_str": true,
  "name_based_walk": true
 },
 "reviewed_exceptions": {
  "code/q01/o2_q01.py:34": "__import__('zipfile') for provenance zip hashing; no expression data",
  "code/q01/qguard.py:132": "getattr(klass, meth) inside the guard's own wrapper installation"
 },
 "all_reviewer_evasions_flagged": true,
 "scan_hits": [],
 "scan_clean": true,
 "claim": "best-effort policy scan; NOT a security boundary (dynamic Python can still evade it)",
 "runtime_closable": false
}
```


ledgers/Q05_milestones.json


```
{
 "landmark": "XVI-Q",
 "release": "Q05",
 "P6_outcome": "Partial",
 "stages": {
  "rules_V3_ghost_validated": true,
  "graphs_evaluated_pole": [
   "G03",
   "G04",
   "G05",
   "G06",
   "G07",
   "G08",
   "G09",
   "G10",
   "G11",
   "G12",
   "G13",
   "G14",
   "G15"
  ],
  "graphs_missing": [
   "G01",
   "G02"
  ],
  "graphs_derived_zero": [
   "T1",
   "T2",
   "T3"
  ],
  "row_c_phi_phi": "closed at pole level",
  "row_c_c_cbar": "OPEN (unresolved assembly)",
  "row_c_c_K": "OPEN (unresolved assembly)",
  "row_h_h_c": "Missing"
 },
 "hard_stops": {
  "bulk_symmetry_restored": false,
  "insertion": "frozen",
  "XV_B_opened": false,
  "XIV_J_opened": false,
  "Route_B_discharged": false,
  "Missing_never_zero": true
 }
```

```
}
```

certificates/Q04_release.json


```
{
 "landmark": "XVI-Q",
 "release": "Q04",
 "version_label": "XVI-Q Q04",
 "date": "2026-10-11",
 "kind": "P.6 entry (one-loop kappa^3 three-point): honest partial",
 "execution_pass": true,
 "publication": "accepted",
 "value_checks": {
  "expected_file_pinned": true,
  "every_certified_value_matches_pinned": true,
  "_value_mismatches": [],
  "code_rehashed": true,
  "_file_mismatches": [],
  "provenance_rehashed_all_pinned": true,
  "bound_q02_report_rehashed": true,
  "bound_q03_report_rehashed": true,
  "tools_source_set_pinned": true,
  "guard_leave_sympy_class_disclosed_and_scan_clean": true,
  "inventory_15_plus_3_tadpoles": true,
  "kappa_equals_E_and_1pt_2pt_counts": true,
  "vertex_hhhphiphi_two_routes_and_mutation": true,
  "engine_validated_vs_D11": true,
  "matter_ward_residue_zero_mutations_detect": true,
  "scalar_hhh_bose_symmetric": true,
  "graph_table_counts": true,
  "honest_partial_outcome": true,
  "no_missing_set_to_zero": true,
  "bulk_symmetry_restored_false": true,
  "Q01_release_layer_accepted": true,
  "Q02_release_layer_accepted": true,
  "Q03_release_layer_accepted": true,
  "O1_record_unchanged": true,
  "every_gate_read_in_source_set": true,
  "_reads_outside_source_set": []
 },
 "P6_outcome": "Partial",
 "P6_stages": {
  "inventory": "Constructed",
  "kappa3_vertices": "Constructed (V_hhh_phi_phi); V5_grav not needed (scaleless only)",
  "power_counting_kappa3_1pt_2pt_empty": true,
  "identity_ledger_rows": "Constructed (14 gh1 three-field rows; 4 need P.6 inputs)",
  "counterterm_domain_kappa3": "Partial (structural counts; BRST/sigma_0 reduction not done)",
  "engine_validated_vs_XVI_P_D11": true,
  "evaluation": "Partial: 2 graphs at pole level, 3 tadpoles derived 0, 13 Missing",
  "matter_subsector_ward_c_h_h_pole": "verified",
  "defect_assembly": "Missing (inputs Missing for every needs_inputs row)",
  "counterterm_solve_kappa3": "Missing",
  "sigma0_closure_kappa3": "Missing"
 },
 "P6_rows": {
  "h.h.c": "Missing inputs",
  "h.phi.c": "zero (phi-parity odd (every vertex even in {ph})",
  "h.b.c": "aux",
  "h.c.cbs": "aux",
  "phi.phi.c": "Missing inputs",
  "phi.b.c": "zero (phi-parity odd (every vertex even in {ph})",
  "phi.c.cbs": "zero (phi-parity odd (every vertex even in {ph})",
  "b.b.c": "aux",
  "b.c.cbs": "aux",
  "c.c.cb": "Missing inputs",
  "c.c.hs": "Missing inputs",
  "c.c.phis": "zero (phi-parity odd (every vertex even in {ph})",
  "c.c.bs": "aux",
  "c.cbs.cbs": "aux"
 },
 "P6_graphs": [
  {
   "id": "P6-G01",
   "external": "h,h,h",
   "status": "Missing (not evaluated in Q04)"
  },
  {
   "id": "P6-G02",
   "external": "h,h,h",
   "status": "Missing (not evaluated in Q04)"
  },
  {
   "id": "P6-G03",
```

```
   "external": "K,c,h",
   "status": "Missing (not evaluated in Q04)"
  },
  {
   "id": "P6-G04",
   "external": "c,cb,h",
   "status": "Missing (not evaluated in Q04)"
  },
  {
   "id": "P6-G05",
   "external": "h,phi,phi",
   "status": "Missing (not evaluated in Q04)"
  },
  {
   "id": "P6-G06",
   "external": "h,phi,phi",
   "status": "Missing (not evaluated in Q04)"
  },
  {
   "id": "P6-G07",
   "external": "K,c,h",
   "status": "Missing (not evaluated in Q04)"
  },
  {
   "id": "P6-G08",
   "external": "h,h,h",
   "status": "Missing (not evaluated in Q04)"
  },
  {
   "id": "P6-G09",
   "external": "c,cb,h",
   "status": "Missing (not evaluated in Q04)"
  },
  {
   "id": "P6-G10",
   "external": "c,c,cs",
   "status": "Missing (not evaluated in Q04)"
  },
  {
   "id": "P6-G11",
   "external": "c,phi,phis",
   "status": "Missing (not evaluated in Q04)"
  },
  {
   "id": "P6-G12",
   "external": "h,phi,phi",
   "status": "Missing (not evaluated in Q04)"
  },
  {
   "id": "P6-G13",
   "external": "h,phi,phi",
   "status": "Missing (not evaluated in Q04)"
  },
  {
   "id": "P6-G14",
   "external": "h,h,h",
   "status": "UV pole evaluated (exact residue, own rules, validated vs D11; Ward-checked); finite part Missing"
  },
  {
   "id": "P6-G15",
   "external": "h,h,h",
   "status": "UV pole evaluated (exact residue, own rules, validated vs D11; Ward-checked); finite part Missing"
  },
  {
   "id": "P6-T1",
   "external": "h,h,h",
   "status": "derived 0 (scaleless, no log member)"
  },
  {
   "id": "P6-T2",
   "external": "h,h,h",
   "status": "derived 0 (scaleless, no log member)"
  },
  {
   "id": "P6-T3",
   "external": "h,phi,phi",
   "status": "derived 0 (scaleless, no log member)"
  }
 ],
 "O1_closure": "CLOSED (scoped)",
 "O1_closure_confirmed": true,
 "O1_closure_scope": "one-loop, order kappa^2, bulk two-point retained sector of S_inv (h, phi, c, cbar, b + antifields; gh0 seeds), m=0, frozen gauge
fermion, under XVI-P's declared prescription (DRED: QdS loop momenta, Q4S vertex indices, MS-bar) INCLUDING the p22 evanescent counterterm contract
revision; NOT the functional ST4b, NOT bulk symmetry restoration, NOT XVI-O's own ledger (read-only, still NOT_CLOSED there)",
```

```
 "O1_not_implied": [
  "the functional / all-n-point ST4b identity (only the retained two-point projection is verified)",
  "bulk symmetry restoration (bulk_symmetry_restored stays false)",
  "any change to XVI-O's own O.1 ledger: XVI-O is read-only and its O.1 stays NOT_CLOSED there",
  "the three-point sector / P.6 (kappa^3) and higher orders or loops",
  "the insertion (frozen; XVI-N massless obstruction untouched)",
  "finite renormalised amplitudes with non-QdS (epsilon-scalar) external components (p22 Q4S-trace O(eps)x pole remnant stays disclosed)"
 ],
 "O1_caveats": [
  "same-box reviews: Q01/Q02 independent rebuilds ran on the author's machine (shared Python/sympy pin and TeX Live); not an independent machine",
  "D01/D03 pole values were never recomputed from independent Feynman rules in any review; cross-checked only via the exact cancellation of the c.h
defect in d",
  "full sigma_0 closure and the c.h defect were not re-derived in the Q02 review; they rely on the earlier p19-p22 independent checks on byte-
identical inputs",
  "D06 accepted via exact equality with XVI-P P15 Sigma (ghost equation), not via the reviewer's own Feynman rules"
 ],
 "XVI_O_ledger_O1": "NOT_CLOSED (read-only, unchanged)",
 "bulk_symmetry_restored": false,
 "symmetry_restored": false,
 "guard_leave_sympy_class": "disclosed; runtime-unclosable; enforced by static scan of package code (clean)",
 "flags": {
  "P6_inventory": "earned",
  "P6_identity_ledger": "earned",
  "P6_engine_validated": "earned",
  "P6_matter_subsector_ward_pole": "earned",
  "P6_evaluated": "partial",
  "P6_counterterms": "Missing",
  "P6_sigma0_closure": "Missing",
  "independent_rebuild_passed_q02": true,
  "independent_rebuild_passed_q03": true,
  "independent_rebuild_passed_q04": "pending",
  "author_rebuild_passed": "see reproducibility"
 },
 "hard_stops": {
  "bulk_symmetry_restored": false,
  "insertion": "frozen",
  "XV_B_opened": false,
  "XIV_J_opened": false,
  "Route_B_discharged": false,
  "Missing_never_zero": true
 },
 "XVI_Q_registered": "locally; not published"
}
```


certificates/Q04_P6_status.json


```
{
 "landmark": "XVI-Q",
 "release": "Q04",
 "P6_outcome": "Partial",
 "stages": {
  "inventory": "Constructed",
  "kappa3_vertices": "Constructed (V_hhh_phi_phi); V5_grav not needed (scaleless only)",
  "power_counting_kappa3_1pt_2pt_empty": true,
  "identity_ledger_rows": "Constructed (14 gh1 three-field rows; 4 need P.6 inputs)",
  "counterterm_domain_kappa3": "Partial (structural counts; BRST/sigma_0 reduction not done)",
  "engine_validated_vs_XVI_P_D11": true,
  "evaluation": "Partial: 2 graphs at pole level, 3 tadpoles derived 0, 13 Missing",
  "matter_subsector_ward_c_h_h_pole": "verified",
  "defect_assembly": "Missing (inputs Missing for every needs_inputs row)",
  "counterterm_solve_kappa3": "Missing",
  "sigma0_closure_kappa3": "Missing"
 },
 "rows_status": {
  "h.h.c": "Missing inputs",
  "h.phi.c": "zero (phi-parity odd (every vertex even in {ph})",
  "h.b.c": "aux",
  "h.c.cbs": "aux",
  "phi.phi.c": "Missing inputs",
  "phi.b.c": "zero (phi-parity odd (every vertex even in {ph})",
  "phi.c.cbs": "zero (phi-parity odd (every vertex even in {ph})",
  "b.b.c": "aux",
  "b.c.cbs": "aux",
  "c.c.cb": "Missing inputs",
  "c.c.hs": "Missing inputs",
  "c.c.phis": "zero (phi-parity odd (every vertex even in {ph})",
  "c.c.bs": "aux",
  "c.cbs.cbs": "aux"
 },
 "O1_closure_unchanged": "CLOSED (scoped) -- kappa^2 two-point; P.6 is outside that scope",
 "bulk_symmetry_restored": false,
 "seconds": 115.9
}
```

certificates/Q04_P6_inventory.json


```
{
 "vertices_used": {
  "V3_grav": [
   "h",
   "h",
   "h"
  ],
  "V4_grav": [
   "h",
   "h",
   "h",
   "h"
  ],
  "V_cb_c_h": [
   "cb",
   "c",
   "h"
  ],
  "V_K_c_h": [
   "K",
   "c",
   "h"
  ],
  "V_cs_c_c": [
   "cs",
   "c",
   "c"
  ],
  "V_phis_c_phi": [
   "phis",
   "c",
   "phi"
  ],
  "V_h_phi_phi": [
   "h",
   "phi",
   "phi"
  ],
  "V_hh_phi_phi": [
   "h",
   "h",
   "phi",
   "phi"
  ],
  "V5_grav": [
   "h",
   "h",
   "h",
   "h",
   "h"
  ],
  "V_hhh_phi_phi": [
   "h",
   "h",
   "h",
   "phi",
   "phi"
  ]
 },
 "kappa3_vertices_new": [
  "V5_grav",
  "V_hhh_phi_phi"
 ],
 "exact_in_kappa_vertices": {
  "V_cb_c_h": "ghost action = cbar . d(s h) with Psi linear in h (XVI-P conventions PSI) and s h = (1/2) L_c(eta + kappa h) affine in h
(GHOST_SCALING): no cbar c h^n vertex for n >= 2",
  "V_K_c_h": "K s h, s h affine in h: no K c h^n, n >= 2",
  "V_cs_c_c": "c* s c, s c = kappa a c.dc: exact at kappa^1",
  "V_phis_c_phi": "phi* s phi, s phi = kappa a c.dphi: exact at kappa^1",
  "gauge_fixing": "Psi linear in h -> b/h sector quadratic, no vertices"
 },
 "power_counting": {
  "rule": "one loop: I = V, sum_v (legs_v - 2) = (2I + E) - 2V = E  => kappa power = E",
  "verified_on_all_enumerated_graphs": true,
  "kappa3_one_point_one_loop": "empty (kappa^E with E = 1 is kappa^1)",
  "kappa3_two_point_one_loop": "empty (E = 2 is kappa^2)",
  "completeness": "each vertex of a one-loop E = 3 graph carries >= 1 external leg (3-leg vertices: exactly 1) -> at most 3 vertices; enumerated n =
1..3"
 },
 "n_by_E": {
  "1": 3,
  "2": 9,
  "3": 18,
```

```
 "4": 21,
 "5": 26,
 "6": 23,
 "7": 19,
 "8": 11,
 "9": 8
},
"E3_loops": [
 {
  "vertices": [
   "V3_grav",
   "V_K_c_h",
   "V_cb_c_h"
  ],
  "lines": [
   "c-cb",
   "h-h",
   "h-h"
  ],
  "external": [
   "K",
   "c",
   "h"
  ],
  "kappa": 3
 },
 {
  "vertices": [
   "V_K_c_h",
   "V_cb_c_h",
   "V_cb_c_h"
  ],
  "lines": [
   "c-cb",
   "c-cb",
   "h-h"
  ],
  "external": [
   "K",
   "c",
   "h"
  ],
  "kappa": 3
 },
 {
  "vertices": [
   "V_cb_c_h",
   "V_cb_c_h",
   "V_cs_c_c"
  ],
  "lines": [
   "c-cb",
   "c-cb",
   "h-h"
  ],
  "external": [
   "c",
   "c",
   "cs"
  ],
  "kappa": 3
 },
 {
  "vertices": [
   "V3_grav",
   "V_cb_c_h",
   "V_cb_c_h"
  ],
  "lines": [
   "c-cb",
   "h-h",
   "h-h"
  ],
  "external": [
   "c",
   "cb",
   "h"
  ],
  "kappa": 3
 },
 {
  "vertices": [
   "V_cb_c_h",
   "V_cb_c_h",
```

```
   "V_cb_c_h"
  ],
  "lines": [
   "c-cb",
   "c-cb",
   "h-h"
  ],
  "external": [
   "c",
   "cb",
   "h"
  ],
  "kappa": 3
 },
 {
  "vertices": [
   "V_cb_c_h",
   "V_h_phi_phi",
   "V_phis_c_phi"
  ],
  "lines": [
   "c-cb",
   "h-h",
   "phi-phi"
  ],
  "external": [
   "c",
   "phi",
   "phis"
  ],
  "kappa": 3
 },
 {
  "vertices": [
   "V3_grav",
   "V3_grav",
   "V3_grav"
  ],
  "lines": [
   "h-h",
   "h-h",
   "h-h"
  ],
  "external": [
   "h",
   "h",
   "h"
  ],
  "kappa": 3
 },
 {
  "vertices": [
   "V3_grav",
   "V4_grav"
  ],
  "lines": [
   "h-h",
   "h-h"
  ],
  "external": [
   "h",
   "h",
   "h"
  ],
  "kappa": 3
 },
 {
  "vertices": [
   "V_cb_c_h",
   "V_cb_c_h",
   "V_cb_c_h"
  ],
  "lines": [
   "c-cb",
   "c-cb",
   "c-cb"
  ],
  "external": [
   "h",
   "h",
   "h"
  ],
  "kappa": 3
 },
```

```
{
 "vertices": [
  "V_h_phi_phi",
  "V_h_phi_phi",
  "V_h_phi_phi"
 ],
 "lines": [
  "phi-phi",
  "phi-phi",
  "phi-phi"
 ],
 "external": [
  "h",
  "h",
  "h"
 ],
 "kappa": 3
},
{
 "vertices": [
  "V_h_phi_phi",
  "V_hh_phi_phi"
 ],
 "lines": [
  "phi-phi",
  "phi-phi"
 ],
 "external": [
  "h",
  "h",
  "h"
 ],
 "kappa": 3
},
{
 "vertices": [
  "V3_grav",
  "V_h_phi_phi",
  "V_h_phi_phi"
 ],
 "lines": [
  "h-h",
  "h-h",
  "phi-phi"
 ],
 "external": [
  "h",
  "phi",
  "phi"
 ],
 "kappa": 3
},
{
 "vertices": [
  "V3_grav",
  "V_hh_phi_phi"
 ],
 "lines": [
  "h-h",
  "h-h"
 ],
 "external": [
  "h",
  "phi",
  "phi"
 ],
 "kappa": 3
},
{
 "vertices": [
  "V_h_phi_phi",
  "V_h_phi_phi",
  "V_h_phi_phi"
 ],
 "lines": [
  "h-h",
  "phi-phi",
  "phi-phi"
 ],
 "external": [
  "h",
  "phi",
  "phi"
 ],
```

```
   "kappa": 3
  },
  {
   "vertices": [
    "V_h_phi_phi",
    "V_hh_phi_phi"
   ],
   "lines": [
    "h-h",
    "phi-phi"
   ],
   "external": [
    "h",
    "phi",
    "phi"
   ],
   "kappa": 3
  }
 ],
 "E3_tadpoles": [
  {
   "vertices": [
    "V5_grav"
   ],
   "lines": [
    "h-h"
   ],
   "external": [
    "h",
    "h",
    "h"
   ],
   "kappa": 3,
   "status": "derived 0 (scaleless): massless self-contraction, the loop momentum flows through ONE propagator and the vertex is a polynomial in l and
external momenta -> int d^dl l^{2k} / l^2 = 0 (XVI-P rule S0, no log member)"
  },
  {
   "vertices": [
    "V_hhh_phi_phi"
   ],
   "lines": [
    "phi-phi"
   ],
   "external": [
    "h",
    "h",
    "h"
   ],
   "kappa": 3,
   "status": "derived 0 (scaleless): massless self-contraction, the loop momentum flows through ONE propagator and the vertex is a polynomial in l and
external momenta -> int d^dl l^{2k} / l^2 = 0 (XVI-P rule S0, no log member)"
  },
  {
   "vertices": [
    "V_hhh_phi_phi"
   ],
   "lines": [
    "h-h"
   ],
   "external": [
    "h",
    "phi",
    "phi"
   ],
   "kappa": 3,
   "status": "derived 0 (scaleless): massless self-contraction, the loop momentum flows through ONE propagator and the vertex is a polynomial in l and
external momenta -> int d^dl l^{2k} / l^2 = 0 (XVI-P rule S0, no log member)"
  }
 ],
 "n_E3_loops": 15,
 "n_E3_tadpoles": 3,
 "by_external": {
  "K,c,h": 2,
  "c,c,cs": 1,
  "c,cb,h": 2,
  "c,phi,phis": 1,
  "h,h,h": 7,
  "h,phi,phi": 5
 },
 "equals_XVI_P_P23_15": true,
 "XVI_P_n": 15,
 "V_hhh_phi_phi": {
  "formula": "X3 = -h^3 + (1/2) t1 h^2 + (t1^2/8 - t2/4)(-h) + (t3/6 - t1 t2/8 + t1^3/48) eta^{-1}  (h with indices raised by eta).",
  "two_routes_agree": true,
```

```
  "mutation_tr_h_cubed_detected": true,
  "role": "enters only the scaleless tadpoles (E3 tadpoles 2, 3)"
 },
 "V5_grav": {
  "status": "NOT constructed",
  "why_not_needed": "enters only the self-contracted tadpole (hhh), which is scaleless for ANY polynomial vertex; no other one-loop E = 3 graph
contains it"
 }
}
```


certificates/Q04_P6_identity.json


```
{
 "identity": "(S, Gamma_1) = A_1 at kappa^3, one loop: (S_0, Gamma_1^[3]) + (S_1, Gamma_1^[2]) + (S_2, Gamma_1^[1]) + (S_3, Gamma_1^[0])",
 "terms": {
  "(S_0, Gamma_1^[3])": "three-point Gamma_1 with the linear BRST (s0 h = d c, s0 c = 0, s0 phi = 0, antifield equations of S_0) -- needs the P.6
graphs",
  "(S_1, Gamma_1^[2])": "two-point Gamma_1 (Q02, evaluated) with the quadratic BRST (c.dh, h dc, c.dc, c.dphi)",
  "(S_2, Gamma_1^[1])": "0: Gamma_1^[1] is one-point only and vanishes (D14a-c)",
  "(S_3, Gamma_1^[0])": "0: field independent"
 },
 "rows": [
  {
   "row": "h.h.c",
   "class": "needs_inputs",
   "inputs": {
    "S0_Gamma3": [
     "Gamma_hhh (5 graphs)",
     "Gamma_K c h (2)"
    ],
    "S1_Gamma2": [
     "Gamma_hh (Q02: D01, D02, D03, D11, D12)",
     "Gamma_K c (D13)"
    ],
    "matter_subsector": "scalar-loop part = matter Ward identity (evaluated, Q04)"
   }
  },
  {
   "row": "h.phi.c",
   "class": "zero",
   "why": "phi-parity odd (every vertex even in {phi, phi*})"
  },
  {
   "row": "h.b.c",
   "class": "aux",
   "why": "contains b / cbar* / b*: no interaction vertex (ghost-equation / nonminimal rows; not a loop entry)"
  },
  {
   "row": "h.c.cbs",
   "class": "aux",
   "why": "contains b / cbar* / b*: no interaction vertex (ghost-equation / nonminimal rows; not a loop entry)"
  },
  {
   "row": "phi.phi.c",
   "class": "needs_inputs",
   "inputs": {
    "S0_Gamma3": [
     "Gamma_h phi phi (4 graphs)",
     "Gamma_phi* c phi (1)"
    ],
    "S1_Gamma2": [
     "Gamma_phi phi = 0 (Q02)",
     "Gamma_hh not entering"
    ]
   }
  },
  {
   "row": "phi.b.c",
   "class": "zero",
   "why": "phi-parity odd (every vertex even in {phi, phi*})"
  },
  {
   "row": "phi.c.cbs",
   "class": "zero",
   "why": "phi-parity odd (every vertex even in {phi, phi*})"
  },
  {
   "row": "b.b.c",
   "class": "aux",
   "why": "contains b / cbar* / b*: no interaction vertex (ghost-equation / nonminimal rows; not a loop entry)"
  },
  {
   "row": "b.c.cbs",
```

```
   "class": "aux",
   "why": "contains b / cbar* / b*: no interaction vertex (ghost-equation / nonminimal rows; not a loop entry)"
  },
  {
   "row": "c.c.cb",
   "class": "needs_inputs",
   "inputs": {
    "S0_Gamma3": [
     "Gamma_cbar c h (2)",
     "Gamma_c c c* (1)"
    ],
    "S1_Gamma2": [
     "Gamma_cbar c (D06)"
    ]
   }
  },
  {
   "row": "c.c.hs",
   "class": "needs_inputs",
   "inputs": {
    "S0_Gamma3": [
     "Gamma_K c h (2)",
     "Gamma_c c c* (1)"
    ],
    "S1_Gamma2": [
     "Gamma_K c (D13)"
    ]
   }
  },
  {
   "row": "c.c.phis",
   "class": "zero",
   "why": "phi-parity odd (every vertex even in {phi, phi*})"
  },
  {
   "row": "c.c.bs",
   "class": "aux",
   "why": "contains b / cbar* / b*: no interaction vertex (ghost-equation / nonminimal rows; not a loop entry)"
  },
  {
   "row": "c.cbs.cbs",
   "class": "aux",
   "why": "contains b / cbar* / b*: no interaction vertex (ghost-equation / nonminimal rows; not a loop entry)"
  }
 ],
 "n_rows": 14,
 "counterterm_domain_kappa3": {
  "hhh": {
   "n_monomials_before_symmetry": 807,
   "n_with_T (evanescent)": 588
  },
  "h_phi_phi": {
   "n_monomials_before_symmetry": 21,
   "n_with_T (evanescent)": 6
  },
  "status": "structural count only (momentum conservation used through p1, p2; Bose symmetrisation and the BRST/sigma_0 reduction to the invariant
domain NOT done in Q04)"
 }
}
```


certificates/Q04_P6_eval.json


```
{
 "engine": "code/q04/uvpole.py (own Feynman rules, symbolic d via qguard, DRED hat/tilde split, IR-rearranged Taylor UV pole; units U = i/(16 pi^2
eps); 'i Gamma' convention)",
 "validation_D11": {
  "trace_coef": "1/2",
  "ratio_mine_over_XVI_P": "I",
  "constant_ratio": true,
  "n_structures": 14,
  "gt_reading": "tilde (evanescent Q(4-d)S metric); g4 = hat + tilde; XVI-P 'upper' structures weighted by their (m<->n, r<->s) orbit size"
 },
 "validation_mutation_trace_coef_1": {
  "trace_coef": "1",
  "ratio_mine_over_XVI_P": "(41*I*H_a1_a1*H_a2_a2*H_p1_p1**2 - 16*I*H_a1_a1*H_a2_p1**2*H_p1_p1 + 50*I*H_a1_a1*H_p1_p1**2*T_a2_a2 +
2*I*H_a1_a2**2*H_p1_p1**2 - 4*I*H_a1_a2*H_a1_p1*H_a2_p1*H_p1_p1 - 16*I*H_a1_p1**2*H_a2_a2*H_p1_p1 + 8*I*H_a1_p1**2*H_a2_p1**2 -
20*I*H_a1_p1**2*H_p1_p1*T_a2_a2 + 50*I*H_a2_a2*H_p1_p1**2*T_a1_a1 - 20*I*H_a2_p1**2*H_p1_p1*T_a1_a1 +
60*I*H_p1_p1**2*T_a1_a1*T_a2_a2)/(6*H_a1_a1*H_a2_a2*H_p1_p1**2 - 6*H_a1_a1*H_a2_p1**2*H_p1_p1 + 10*H_a1_a1*H_p1_p1**2*T_a2_a2 +
2*H_a1_a2**2*H_p1_p1**2 - 4*H_a1_a2*H_a1_p1*H_a2_p1*H_p1_p1 - 6*H_a1_p1**2*H_a2_a2*H_p1_p1 + 8*H_a1_p1**2*H_a2_p1**2 - 10*H_a1_p1**2*H_p1_p1*T_a2_a2 +
10*H_a2_a2*H_p1_p1**2*T_a1_a1 - 10*H_a2_p1**2*H_p1_p1*T_a1_a1 + 15*H_p1_p1**2*T_a1_a1*T_a2_a2)",
  "constant_ratio": false,
  "n_structures": 14,
  "gt_reading": "tilde (evanescent Q(4-d)S metric); g4 = hat + tilde; XVI-P 'upper' structures weighted by their (m<->n, r<->s) orbit size"
```

```
 },
 "scalar_loop_hhh": {
  "residue_d4_n_terms": 467,
  "residue_d4_sha256": "7c19fb73e61f58501255e48c07d908bbc46d5d9c7690d43241226e00474528c9",
  "residue_d4_n_evanescent_terms": 270,
  "residue_d4": "H_a1_a1*H_a2_a2*H_a3_a3*H_p1_p1**2/160 + H_a1_a1*H_a2_a2*H_a3_a3*H_p1_p1*H_p1_p2/80 + H_a1_a1*H_a2_a2*H_a3_a3*H_p1_p1*H_p2_p2/144 +
17*H_a1_a1*H_a2_a2*H_a3_a3*H_p1_p2**2/1440 + H_a1_a1*H_a2_a2*H_a3_a3*H_p1_p2*H_p2_p2/80 + H_a1_a1*H_a2_a2*H_a3_a3*H_p2_p2**2/160 -
H_a1_a1*H_a2_a2*H_a3_p1**2*H_p1_p1/160 - H_a1_a1*H_a2_a2*H_a3_p1**2*H_p1_p2/160 - H_a1_a1*H_a2_a2*H_a3_p1**2*H_p2_p2/144 -
H_a1_a1*H_a2_a2*H_a3_p1*H_a3_p2*H_p1_p2/90 - H_a1_a1*H_a2_a2*H_a3_p2**2*H_p1_p1/144 - H_a1_a1*H_a2_a2*H_a3_p2**2*H_p1_p2/160 -
H_a1_a1*H_a2_a2*H_a3_p2**2*H_p2_p2/160 + H_a1_a1*H_a2_a2*H_p1_p1**2*T_a3_a3/96 + H_a1_a1*H_a2_a2*H_p1_p1*H_p1_p2*T_a3_a3/48 +
H_a1_a1*H_a2_a2*H_p1_p2**2*T_a3_a3/48 + H_a1_a1*H_a2_a2*H_p1_p2*H_p2_p2*T_a3_a3/48 + H_a1_a1*H_a2_a2*H_p2_p2**2*T_a3_a3/96 -
H_a1_a1*H_a2_a3**2*H_p1_p1**2/80 - H_a1_a1*H_a2_a3**2*H_p1_p1*H_p1_p2/160 - H_a1_a1*H_a2_a3**2*H_p1_p1*H_p2_p2/144 + H_a1_a1*H_a2_a3**2*H_p1_p2**2/360
+ H_a1_a1*H_a2_a3**2*H_p1_p2*H_p2_p2/240 + H_a1_a1*H_a2_a3**2*H_p2_p2**2/480 + H_a1_a1*H_a2_a3*H_a2_p1*H_a3_p1*H_p1_p1/40 +
H_a1_a1*H_a2_a3*H_a2_p1*H_a3_p1*H_p2_p2/720 + H_a1_a1*H_a2_a3*H_a2_p1*H_a3_p2*H_p1_p1/80 - H_a1_a1*H_a2_a3*H_a2_p1*H_a3_p2*H_p1_p2/180 -
H_a1_a1*H_a2_a3*H_a2_p1*H_a3_p2*H_p2_p2/240 - H_a1_a1*H_a2_a3*H_a2_p2*H_a3_p1*H_p1_p2/720 + H_a1_a1*H_a2_a3*H_a2_p2*H_a3_p2*H_p1_p1/72 -
H_a1_a1*H_a2_a3*H_a2_p2*H_a3_p2*H_p1_p2/240 - H_a1_a1*H_a2_a3*H_a2_p2*H_a3_p2*H_p2_p2/240 - H_a1_a1*H_a2_a3*H_p1_p1**2*T_a2_a3/40 -
H_a1_a1*H_a2_p1**2*H_a3_a3*H_p1_p1/40 - H_a1_a1*H_a2_p1**2*H_a3_a3*H_p1_p2/40 - 19*H_a1_a1*H_a2_p1**2*H_a3_a3*H_p2_p2/1440 +
H_a1_a1*H_a2_p1**2*H_a3_p1**2/80 + H_a1_a1*H_a2_p1**2*H_a3_p1*H_a3_p2/40 + 11*H_a1_a1*H_a2_p1**2*H_a3_p2**2/720 -
H_a1_a1*H_a2_p1**2*H_p1_p1*T_a3_a3/24 - H_a1_a1*H_a2_p1**2*H_p1_p2*T_a3_a3/24 - H_a1_a1*H_a2_p1**2*H_p2_p2*T_a3_a3/48 -
H_a1_a1*H_a2_p1*H_a2_p2*H_a3_a3*H_p1_p1/40 - 17*H_a1_a1*H_a2_p1*H_a2_p2*H_a3_a3*H_p1_p2/720 - H_a1_a1*H_a2_p1*H_a2_p2*H_a3_a3*H_p2_p2/80 +
H_a1_a1*H_a2_p1*H_a2_p2*H_a3_p1**2/80 + 17*H_a1_a1*H_a2_p1*H_a2_p2*H_a3_p1*H_a3_p2/720 + H_a1_a1*H_a2_p1*H_a2_p2*H_a3_p2**2/60 -
H_a1_a1*H_a2_p1*H_a2_p2*H_p1_p1*T_a3_a3/24 - H_a1_a1*H_a2_p1*H_a2_p2*H_p1_p2*T_a3_a3/24 - H_a1_a1*H_a2_p1*H_a2_p2*H_p2_p2*T_a3_a3/48 +
H_a1_a1*H_a2_p1*H_a3_p1*H_p1_p1*T_a2_a3/40 - H_a1_a1*H_a2_p2**2*H_a3_a3*H_p1_p1/144 - H_a1_a1*H_a2_p2**2*H_a3_a3*H_p1_p2/160 -
H_a1_a1*H_a2_p2**2*H_a3_a3*H_p2_p2/160 + H_a1_a1*H_a2_p2**2*H_a3_p1**2/144 + H_a1_a1*H_a2_p2**2*H_a3_p2**2/120 - H_a1_a1*H_a2_p2**2*H_p1_p2*T_a3_a3/96
- H_a1_a1*H_a2_p2**2*H_p2_p2*T_a3_a3/96 + H_a1_a1*H_a3_a3*H_p1_p2**2*T_a2_a2/48 + H_a1_a1*H_a3_a3*H_p1_p2*H_p2_p2*T_a2_a2/48 +
H_a1_a1*H_a3_a3*H_p2_p2**2*T_a2_a2/96 - H_a1_a1*H_a3_p1**2*H_p1_p2*T_a2_a2/96 - H_a1_a1*H_a3_p1**2*H_p2_p2*T_a2_a2/96 +
H_a1_a1*H_a3_p1*H_a3_p2*H_p1_p1*T_a2_a2/48 - H_a1_a1*H_a3_p1*H_a3_p2*H_p1_p2*T_a2_a2/48 - H_a1_a1*H_a3_p2**2*H_p1_p2*T_a2_a2/96 -
H_a1_a1*H_a3_p2**2*H_p2_p2*T_a2_a2/96 - H_a1_a1*H_p1_p1**2*T_a2_a3**2/48 + H_a1_a1*H_p1_p1*H_p1_p2*T_a2_a2*T_a3_a3/192 -
H_a1_a1*H_p1_p1*H_p2_p2*T_a2_a2*T_a3_a3/96 + H_a1_a1*H_p1_p2**2*T_a2_a2*T_a3_a3/32 + H_a1_a1*H_p1_p2*H_p2_p2*T_a2_a2*T_a3_a3/32 +
H_a1_a1*H_p2_p2**2*T_a2_a2*T_a3_a3/64 - H_a1_a2**2*H_a3_a3*H_p1_p1**2/80 - 7*H_a1_a2**2*H_a3_a3*H_p1_p1*H_p1_p2/160 -
37*H_a1_a2**2*H_a3_a3*H_p1_p1*H_p2_p2/1440 - 5*H_a1_a2**2*H_a3_a3*H_p1_p2**2/144 - 7*H_a1_a2**2*H_a3_a3*H_p1_p2*H_p2_p2/160 -
H_a1_a2**2*H_a3_a3*H_p2_p2**2/80 + H_a1_a2**2*H_a3_p1**2*H_p1_p1/80 + H_a1_a2**2*H_a3_p1**2*H_p1_p2/60 + H_a1_a2**2*H_a3_p1**2*H_p2_p2/90 +
7*H_a1_a2**2*H_a3_p1*H_a3_p2*H_p1_p1/240 + 13*H_a1_a2**2*H_a3_p1*H_a3_p2*H_p1_p2/360 + 7*H_a1_a2**2*H_a3_p1*H_a3_p2*H_p2_p2/240 +
H_a1_a2**2*H_a3_p2**2*H_p1_p1/90 + H_a1_a2**2*H_a3_p2**2*H_p1_p2/60 + H_a1_a2**2*H_a3_p2**2*H_p2_p2/80 - H_a1_a2**2*H_p1_p1**2*T_a3_a3/48 -
7*H_a1_a2**2*H_p1_p1*H_p1_p2*T_a3_a3/96 - H_a1_a2**2*H_p1_p1*H_p2_p2*T_a3_a3/24 - H_a1_a2**2*H_p1_p2**2*T_a3_a3/16 -
7*H_a1_a2**2*H_p1_p2*H_p2_p2*T_a3_a3/96 - H_a1_a2**2*H_p2_p2**2*T_a3_a3/48 - H_a1_a2*H_a1_a3*H_a2_a3*H_p1_p1**2/120 -
H_a1_a2*H_a1_a3*H_a2_a3*H_p1_p1*H_p1_p2/60 - H_a1_a2*H_a1_a3*H_a2_a3*H_p1_p1*H_p2_p2/360 - H_a1_a2*H_a1_a3*H_a2_a3*H_p1_p2**2/45 -
H_a1_a2*H_a1_a3*H_a2_a3*H_p1_p2*H_p2_p2/60 - H_a1_a2*H_a1_a3*H_a2_a3*H_p2_p2**2/120 + H_a1_a2*H_a1_a3*H_a2_p1*H_a3_p1*H_p1_p1/120 +
H_a1_a2*H_a1_a3*H_a2_p1*H_a3_p1*H_p1_p2/60 + H_a1_a2*H_a1_a3*H_a2_p1*H_a3_p1*H_p2_p2/90 - H_a1_a2*H_a1_a3*H_a2_p1*H_a3_p2*H_p1_p1/120 +
H_a1_a2*H_a1_a3*H_a2_p1*H_a3_p2*H_p1_p2/180 + H_a1_a2*H_a1_a3*H_a2_p2*H_a3_p1*H_p1_p1/120 + H_a1_a2*H_a1_a3*H_a2_p2*H_a3_p1*H_p1_p2/72 +
H_a1_a2*H_a1_a3*H_a2_p2*H_a3_p1*H_p2_p2/120 - H_a1_a2*H_a1_a3*H_a2_p2*H_a3_p2*H_p1_p1/180 + H_a1_a2*H_a1_a3*H_a2_p2*H_a3_p2*H_p1_p2/120 +
H_a1_a2*H_a1_a3*H_a2_p2*H_a3_p2*H_p2_p2/120 - H_a1_a2*H_a1_a3*H_p1_p1**2*T_a2_a3/120 + H_a1_a2*H_a1_p1*H_a2_a3*H_a3_p1*H_p1_p1/120 +
H_a1_a2*H_a1_p1*H_a2_a3*H_a3_p1*H_p1_p2/120 - H_a1_a2*H_a1_p1*H_a2_a3*H_a3_p1*H_p2_p2/180 + H_a1_a2*H_a1_p1*H_a2_a3*H_a3_p2*H_p1_p1/120 +
H_a1_a2*H_a1_p1*H_a2_a3*H_a3_p2*H_p1_p2/72 + H_a1_a2*H_a1_p1*H_a2_a3*H_a3_p2*H_p2_p2/120 + H_a1_a2*H_a1_p1*H_a2_p1*H_a3_a3*H_p1_p1/40 +
H_a1_a2*H_a1_p1*H_a2_p1*H_a3_a3*H_p1_p2/20 + 19*H_a1_a2*H_a1_p1*H_a2_p1*H_a3_a3*H_p2_p2/720 - H_a1_a2*H_a1_p1*H_a2_p1*H_a3_p1**2/30 -
7*H_a1_a2*H_a1_p1*H_a2_p1*H_a3_p1*H_a3_p2/120 - 11*H_a1_a2*H_a1_p1*H_a2_p1*H_a3_p2**2/360 + H_a1_a2*H_a1_p1*H_a2_p1*H_p1_p1*T_a3_a3/24 +
H_a1_a2*H_a1_p1*H_a2_p1*H_p1_p2*T_a3_a3/12 + H_a1_a2*H_a1_p1*H_a2_p1*H_p2_p2*T_a3_a3/24 + H_a1_a2*H_a1_p1*H_a2_p2*H_a3_a3*H_p1_p1/40 +
7*H_a1_a2*H_a1_p1*H_a2_p2*H_a3_a3*H_p1_p2/144 + H_a1_a2*H_a1_p1*H_a2_p2*H_a3_a3*H_p2_p2/40 - H_a1_a2*H_a1_p1*H_a2_p2*H_a3_p1**2/30 -
19*H_a1_a2*H_a1_p1*H_a2_p2*H_a3_p1*H_a3_p2/360 - H_a1_a2*H_a1_p1*H_a2_p2*H_a3_p2**2/30 + H_a1_a2*H_a1_p1*H_a2_p2*H_p1_p1*T_a3_a3/24 +
H_a1_a2*H_a1_p1*H_a2_p2*H_p1_p2*T_a3_a3/12 + H_a1_a2*H_a1_p1*H_a2_p2*H_p2_p2*T_a3_a3/24 + H_a1_a2*H_a1_p1*H_a3_p1*H_p1_p1*T_a2_a3/120 +
H_a1_a2*H_a1_p2*H_a2_a3*H_a3_p1*H_p1_p2/180 - H_a1_a2*H_a1_p2*H_a2_a3*H_a3_p1*H_p2_p2/120 + H_a1_a2*H_a1_p2*H_a2_a3*H_a3_p2*H_p1_p1/90 +
H_a1_a2*H_a1_p2*H_a2_a3*H_a3_p2*H_p1_p2/60 + H_a1_a2*H_a1_p2*H_a2_a3*H_a3_p2*H_p2_p2/120 + H_a1_a2*H_a1_p2*H_a2_p1*H_a3_a3*H_p1_p1/80 +
7*H_a1_a2*H_a1_p2*H_a2_p1*H_a3_a3*H_p1_p2/360 + H_a1_a2*H_a1_p2*H_a2_p1*H_a3_a3*H_p2_p2/80 - H_a1_a2*H_a1_p2*H_a2_p1*H_a3_p1**2/60 -
H_a1_a2*H_a1_p2*H_a2_p1*H_a3_p1*H_a3_p2/36 - H_a1_a2*H_a1_p2*H_a2_p1*H_a3_p2**2/60 + H_a1_a2*H_a1_p2*H_a2_p1*H_p1_p1*T_a3_a3/48 +
H_a1_a2*H_a1_p2*H_a2_p1*H_p1_p2*T_a3_a3/24 + H_a1_a2*H_a1_p2*H_a2_p1*H_p2_p2*T_a3_a3/48 + 19*H_a1_a2*H_a1_p2*H_a2_p2*H_a3_a3*H_p1_p1/720 +
H_a1_a2*H_a1_p2*H_a2_p2*H_a3_a3*H_p1_p2/20 + H_a1_a2*H_a1_p2*H_a2_p2*H_a3_a3*H_p2_p2/40 - 11*H_a1_a2*H_a1_p2*H_a2_p2*H_a3_p1**2/360 -
7*H_a1_a2*H_a1_p2*H_a2_p2*H_a3_p1*H_a3_p2/120 - H_a1_a2*H_a1_p2*H_a2_p2*H_a3_p2**2/30 + H_a1_a2*H_a1_p2*H_a2_p2*H_p1_p1*T_a3_a3/24 +
H_a1_a2*H_a1_p2*H_a2_p2*H_p1_p2*T_a3_a3/12 + H_a1_a2*H_a1_p2*H_a2_p2*H_p2_p2*T_a3_a3/24 - H_a1_a2*H_a2_a3*H_p2_p2**2*T_a1_a3/120 +
H_a1_a2*H_a2_p2*H_a3_p2*H_p2_p2*T_a1_a3/120 - H_a1_a2*H_a3_a3*H_p1_p1**2*T_a1_a2/40 - H_a1_a2*H_a3_a3*H_p1_p1*H_p1_p2*T_a1_a2/10 -
H_a1_a2*H_a3_a3*H_p1_p1*H_p2_p2*T_a1_a2/20 - H_a1_a2*H_a3_a3*H_p1_p2**2*T_a1_a2/10 - H_a1_a2*H_a3_a3*H_p1_p2*H_p2_p2*T_a1_a2/10 -
H_a1_a2*H_a3_a3*H_p2_p2**2*T_a1_a2/40 + H_a1_a2*H_a3_p1**2*H_p1_p1*T_a1_a2/40 + H_a1_a2*H_a3_p1**2*H_p1_p2*T_a1_a2/20 +
H_a1_a2*H_a3_p1**2*H_p2_p2*T_a1_a2/40 + H_a1_a2*H_a3_p1*H_a3_p2*H_p1_p1*T_a1_a2/20 + H_a1_a2*H_a3_p1*H_a3_p2*H_p1_p2*T_a1_a2/10 +
H_a1_a2*H_a3_p1*H_a3_p2*H_p2_p2*T_a1_a2/20 + H_a1_a2*H_a3_p2**2*H_p1_p1*T_a1_a2/40 + H_a1_a2*H_a3_p2**2*H_p1_p2*T_a1_a2/20 +
H_a1_a2*H_a3_p2**2*H_p2_p2*T_a1_a2/40 - H_a1_a2*H_p1_p1**2*T_a1_a2*T_a3_a3/24 - H_a1_a2*H_p1_p1*H_p1_p2*T_a1_a2*T_a3_a3/6 -
H_a1_a2*H_p1_p1*H_p2_p2*T_a1_a2*T_a3_a3/12 - H_a1_a2*H_p1_p2**2*T_a1_a2*T_a3_a3/6 - H_a1_a2*H_p1_p2*H_p2_p2*T_a1_a2*T_a3_a3/6 -
H_a1_a2*H_p2_p2**2*T_a1_a2*T_a3_a3/24 + H_a1_a3**2*H_a2_a2*H_p1_p1**2/480 + H_a1_a3**2*H_a2_a2*H_p1_p1*H_p1_p2/240 -
H_a1_a3**2*H_a2_a2*H_p1_p1*H_p2_p2/144 + H_a1_a3**2*H_a2_a2*H_p1_p2**2/360 - H_a1_a3**2*H_a2_a2*H_p1_p2*H_p2_p2/160 - H_a1_a3**2*H_a2_a2*H_p2_p2**2/80
- H_a1_a3**2*H_a2_p1**2*H_p1_p1/120 - H_a1_a3**2*H_a2_p1**2*H_p1_p2/120 - H_a1_a3**2*H_a2_p1**2*H_p2_p2/180 - H_a1_a3**2*H_a2_p1*H_a2_p2*H_p1_p1/120 -
H_a1_a3**2*H_a2_p1*H_a2_p2*H_p1_p2/180 - H_a1_a3**2*H_a2_p1*H_a2_p2*H_p2_p2/240 + H_a1_a3**2*H_a2_p2**2*H_p1_p1/144 +
H_a1_a3**2*H_a2_p2**2*H_p1_p2/120 + H_a1_a3**2*H_a2_p2**2*H_p2_p2/80 - H_a1_a3**2*H_p1_p1*H_p2_p2*T_a2_a2/96 - H_a1_a3**2*H_p1_p2*H_p2_p2*T_a2_a2/96 -
H_a1_a3**2*H_p2_p2**2*T_a2_a2/48 - H_a1_a3*H_a1_p1*H_a2_a2*H_a3_p1*H_p1_p1/240 - H_a1_a3*H_a1_p1*H_a2_a2*H_a3_p1*H_p1_p2/240 +
H_a1_a3*H_a1_p1*H_a2_a2*H_a3_p1*H_p2_p2/72 - H_a1_a3*H_a1_p1*H_a2_a2*H_a3_p2*H_p1_p2/720 + H_a1_a3*H_a1_p1*H_a2_a3*H_a2_p1*H_p1_p1/120 +
H_a1_a3*H_a1_p1*H_a2_a3*H_a2_p1*H_p2_p2/360 - H_a1_a3*H_a1_p1*H_a2_a3*H_a2_p2*H_p1_p2/360 + H_a1_a3*H_a1_p1*H_a2_p1**2*H_a3_p1/120 +
H_a1_a3*H_a1_p1*H_a2_p1**2*H_a3_p2/120 + H_a1_a3*H_a1_p1*H_a2_p1*H_a2_p2*H_a3_p1/120 + H_a1_a3*H_a1_p1*H_a2_p1*H_a2_p2*H_a3_p2/180 +
H_a1_a3*H_a1_p1*H_a2_p1*H_p1_p1*T_a2_a3/120 - H_a1_a3*H_a1_p1*H_a2_p2**2*H_a3_p1/72 + H_a1_a3*H_a1_p1*H_a3_p1*H_p2_p2*T_a2_a2/48 -
H_a1_a3*H_a1_p2*H_a2_a2*H_a3_p1*H_p1_p1/240 - H_a1_a3*H_a1_p2*H_a2_a2*H_a3_p1*H_p1_p2/180 + H_a1_a3*H_a1_p2*H_a2_a2*H_a3_p1*H_p2_p2/80 +
H_a1_a3*H_a1_p2*H_a2_a2*H_a3_p2*H_p1_p1/720 + H_a1_a3*H_a1_p2*H_a2_a2*H_a3_p2*H_p2_p2/40 + H_a1_a3*H_a1_p2*H_a2_a3*H_a2_p1*H_p1_p1/60 +
H_a1_a3*H_a1_p2*H_a2_a3*H_a2_p1*H_p1_p2/45 + H_a1_a3*H_a1_p2*H_a2_a3*H_a2_p1*H_p2_p2/60 + H_a1_a3*H_a1_p2*H_a2_a3*H_a2_p2*H_p1_p1/360 +
H_a1_a3*H_a1_p2*H_a2_a3*H_a2_p2*H_p2_p2/120 - H_a1_a3*H_a1_p2*H_a2_p1**2*H_a3_p2/180 - H_a1_a3*H_a1_p2*H_a2_p1*H_a2_p2*H_a3_p1/360 -
H_a1_a3*H_a1_p2*H_a2_p1*H_a2_p2*H_a3_p2/120 - H_a1_a3*H_a1_p2*H_a2_p2**2*H_a3_p1/60 - H_a1_a3*H_a1_p2*H_a2_p2**2*H_a3_p2/30 +
H_a1_a3*H_a1_p2*H_a3_p1*H_p2_p2*T_a2_a2/48 + H_a1_a3*H_a1_p2*H_a3_p2*H_p2_p2*T_a2_a2/24 - H_a1_a3*H_a2_a2*H_p2_p2**2*T_a1_a3/40 -
H_a1_a3*H_a2_a3*H_p1_p1**2*T_a1_a2/120 - H_a1_a3*H_a2_a3*H_p1_p1*H_p1_p2*T_a1_a2/30 - H_a1_a3*H_a2_a3*H_p1_p1*H_p2_p2*T_a1_a2/60 -
H_a1_a3*H_a2_a3*H_p1_p2**2*T_a1_a2/30 - H_a1_a3*H_a2_a3*H_p1_p2*H_p2_p2*T_a1_a2/30 - H_a1_a3*H_a2_a3*H_p2_p2**2*T_a1_a2/120 +
H_a1_a3*H_a2_p1*H_a3_p1*H_p1_p1*T_a1_a2/120 + H_a1_a3*H_a2_p1*H_a3_p1*H_p1_p2*T_a1_a2/60 + H_a1_a3*H_a2_p1*H_a3_p1*H_p2_p2*T_a1_a2/120 +
```

```
H_a1_a3*H_a2_p1*H_a3_p2*H_p1_p1*T_a1_a2/120 + H_a1_a3*H_a2_p1*H_a3_p2*H_p1_p2*T_a1_a2/60 + H_a1_a3*H_a2_p1*H_a3_p2*H_p2_p2*T_a1_a2/120 +
H_a1_a3*H_a2_p2**2*H_p2_p2*T_a1_a3/40 + H_a1_a3*H_a2_p2*H_a3_p1*H_p1_p1*T_a1_a2/120 + H_a1_a3*H_a2_p2*H_a3_p1*H_p1_p2*T_a1_a2/60 +
H_a1_a3*H_a2_p2*H_a3_p1*H_p2_p2*T_a1_a2/120 + H_a1_a3*H_a2_p2*H_a3_p2*H_p1_p1*T_a1_a2/120 + H_a1_a3*H_a2_p2*H_a3_p2*H_p1_p2*T_a1_a2/60 +
H_a1_a3*H_a2_p2*H_a3_p2*H_p2_p2*T_a1_a2/120 - H_a1_a3*H_p2_p2**2*T_a1_a3*T_a2_a2/24 - H_a1_p1**2*H_a2_a2*H_a3_a3*H_p1_p1/160 -
H_a1_p1**2*H_a2_a2*H_a3_a3*H_p1_p2/160 - H_a1_p1**2*H_a2_a2*H_a3_a3*H_p2_p2/144 + H_a1_p1**2*H_a2_a2*H_a3_p1**2/120 +
H_a1_p1**2*H_a2_a2*H_a3_p2**2/144 - H_a1_p1**2*H_a2_a2*H_p1_p1*T_a3_a3/96 - H_a1_p1**2*H_a2_a2*H_p1_p2*T_a3_a3/96 + H_a1_p1**2*H_a2_a3**2*H_p1_p1/80 +
H_a1_p1**2*H_a2_a3**2*H_p1_p2/120 + H_a1_p1**2*H_a2_a3**2*H_p2_p2/144 - H_a1_p1**2*H_a2_a3*H_a2_p1*H_a3_p1/30 - H_a1_p1**2*H_a2_a3*H_a2_p1*H_a3_p2/60
- H_a1_p1**2*H_a2_a3*H_a2_p2*H_a3_p2/72 + H_a1_p1**2*H_a2_a3*H_p1_p1*T_a2_a3/40 + H_a1_p1**2*H_a2_p1**2*H_a3_a3/80 + H_a1_p1**2*H_a2_p1**2*T_a3_a3/48
+ H_a1_p1**2*H_a2_p1*H_a2_p2*H_a3_a3/80 + H_a1_p1**2*H_a2_p1*H_a2_p2*T_a3_a3/48 - H_a1_p1**2*H_a2_p1*H_a3_p1*T_a2_a3/30 +
H_a1_p1**2*H_a2_p2**2*H_a3_a3/144 + H_a1_p1**2*H_a3_a3*H_p1_p2*T_a2_a2/96 - H_a1_p1**2*H_a3_p1*H_a3_p2*T_a2_a2/48 + H_a1_p1**2*H_p1_p1*T_a2_a3**2/48 +
H_a1_p1**2*H_p1_p2*T_a2_a2*T_a3_a3/96 + H_a1_p1**2*H_p2_p2*T_a2_a2*T_a3_a3/96 - H_a1_p1*H_a1_p2*H_a2_a2*H_a3_a3*H_p1_p1/80 -
17*H_a1_p1*H_a1_p2*H_a2_a2*H_a3_a3*H_p1_p2/720 - H_a1_p1*H_a1_p2*H_a2_a2*H_a3_a3*H_p2_p2/40 + H_a1_p1*H_a1_p2*H_a2_a2*H_a3_p1**2/60 +
17*H_a1_p1*H_a1_p2*H_a2_a2*H_a3_p1*H_a3_p2/720 + H_a1_p1*H_a1_p2*H_a2_a2*H_a3_p2**2/80 - H_a1_p1*H_a1_p2*H_a2_a2*H_p1_p1*T_a3_a3/48 -
H_a1_p1*H_a1_p2*H_a2_a2*H_p1_p2*T_a3_a3/24 - H_a1_p1*H_a1_p2*H_a2_a2*H_p2_p2*T_a3_a3/24 - H_a1_p1*H_a1_p2*H_a2_a3**2*H_p1_p1/240 -
H_a1_p1*H_a1_p2*H_a2_a3**2*H_p1_p2/180 - H_a1_p1*H_a1_p2*H_a2_a3**2*H_p2_p2/120 - H_a1_p1*H_a1_p2*H_a2_a3*H_a2_p1*H_a3_p1/120 -
H_a1_p1*H_a1_p2*H_a2_a3*H_a2_p1*H_a3_p2/360 + H_a1_p1*H_a1_p2*H_a2_a3*H_a2_p2*H_a3_p1/180 + H_a1_p1*H_a1_p2*H_a2_a3*H_a2_p2*H_a3_p2/120 -
H_a1_p1*H_a1_p2*H_a2_p1*H_a2_p2*H_a3_a3/720 + H_a1_p1*H_a1_p2*H_a2_p2**2*H_a3_a3/80 + H_a1_p1*H_a1_p2*H_a2_p2**2*T_a3_a3/48 -
H_a1_p1*H_a1_p2*H_a3_a3*H_p1_p1*T_a2_a2/48 - H_a1_p1*H_a1_p2*H_a3_a3*H_p1_p2*T_a2_a2/24 - H_a1_p1*H_a1_p2*H_a3_a3*H_p2_p2*T_a2_a2/24 +
H_a1_p1*H_a1_p2*H_a3_p1**2*T_a2_a2/48 + H_a1_p1*H_a1_p2*H_a3_p1*H_a3_p2*T_a2_a2/24 + H_a1_p1*H_a1_p2*H_a3_p2**2*T_a2_a2/48 -
H_a1_p1*H_a1_p2*H_p1_p1*T_a2_a2*T_a3_a3/32 - H_a1_p1*H_a1_p2*H_p1_p2*T_a2_a2*T_a3_a3/16 - H_a1_p1*H_a1_p2*H_p2_p2*T_a2_a2*T_a3_a3/16 +
H_a1_p1*H_a2_a3*H_a3_p1*H_p1_p1*T_a1_a2/120 + H_a1_p1*H_a2_a3*H_a3_p1*H_p1_p2*T_a1_a2/60 + H_a1_p1*H_a2_a3*H_a3_p1*H_p2_p2*T_a1_a2/120 +
H_a1_p1*H_a2_a3*H_a3_p2*H_p1_p1*T_a1_a2/120 + H_a1_p1*H_a2_a3*H_a3_p2*H_p1_p2*T_a1_a2/60 + H_a1_p1*H_a2_a3*H_a3_p2*H_p2_p2*T_a1_a2/120 +
H_a1_p1*H_a2_p1*H_a3_a3*H_p1_p1*T_a1_a2/40 + H_a1_p1*H_a2_p1*H_a3_a3*H_p1_p2*T_a1_a2/20 + H_a1_p1*H_a2_p1*H_a3_a3*H_p2_p2*T_a1_a2/40 -
H_a1_p1*H_a2_p1*H_a3_p1**2*T_a1_a2/30 - H_a1_p1*H_a2_p1*H_a3_p1*H_a3_p2*T_a1_a2/15 - H_a1_p1*H_a2_p1*H_a3_p2**2*T_a1_a2/30 +
H_a1_p1*H_a2_p1*H_p1_p1*T_a1_a2*T_a3_a3/24 + H_a1_p1*H_a2_p1*H_p1_p2*T_a1_a2*T_a3_a3/12 + H_a1_p1*H_a2_p1*H_p2_p2*T_a1_a2*T_a3_a3/24 +
H_a1_p1*H_a2_p2*H_a3_a3*H_p1_p1*T_a1_a2/40 + H_a1_p1*H_a2_p2*H_a3_a3*H_p1_p2*T_a1_a2/20 + H_a1_p1*H_a2_p2*H_a3_a3*H_p2_p2*T_a1_a2/40 -
H_a1_p1*H_a2_p2*H_a3_p1**2*T_a1_a2/30 - H_a1_p1*H_a2_p2*H_a3_p1*H_a3_p2*T_a1_a2/15 - H_a1_p1*H_a2_p2*H_a3_p2**2*T_a1_a2/30 +
H_a1_p1*H_a2_p2*H_p1_p1*T_a1_a2*T_a3_a3/24 + H_a1_p1*H_a2_p2*H_p1_p2*T_a1_a2*T_a3_a3/12 + H_a1_p1*H_a2_p2*H_p2_p2*T_a1_a2*T_a3_a3/24 -
19*H_a1_p2**2*H_a2_a2*H_a3_a3*H_p1_p1/1440 - H_a1_p2**2*H_a2_a2*H_a3_a3*H_p1_p2/40 - H_a1_p2**2*H_a2_a2*H_a3_a3*H_p2_p2/40 +
11*H_a1_p2**2*H_a2_a2*H_a3_p1**2/720 + H_a1_p2**2*H_a2_a2*H_a3_p1*H_a3_p2/40 + H_a1_p2**2*H_a2_a2*H_a3_p2**2/80 -
H_a1_p2**2*H_a2_a2*H_p1_p1*T_a3_a3/48 - H_a1_p2**2*H_a2_a2*H_p1_p2*T_a3_a3/24 - H_a1_p2**2*H_a2_a2*H_p2_p2*T_a3_a3/24 -
H_a1_p2**2*H_a2_a3**2*H_p1_p1/180 - H_a1_p2**2*H_a2_a3**2*H_p1_p2/120 - H_a1_p2**2*H_a2_a3**2*H_p2_p2/120 - H_a1_p2**2*H_a2_a3*H_a2_p1*H_a3_p1/180 +
H_a1_p2**2*H_a2_a3*H_a2_p2*H_a3_p1/120 + H_a1_p2**2*H_a2_a3*H_a2_p2*H_a3_p2/120 + H_a1_p2**2*H_a2_p1**2*H_a3_a3/360 + H_a1_p2**2*H_a2_p2**2*H_a3_a3/80
+ H_a1_p2**2*H_a2_p2**2*T_a3_a3/48 - H_a1_p2**2*H_a3_a3*H_p1_p1*T_a2_a2/48 - H_a1_p2**2*H_a3_a3*H_p1_p2*T_a2_a2/24 -
H_a1_p2**2*H_a3_a3*H_p2_p2*T_a2_a2/24 + H_a1_p2**2*H_a3_p1**2*T_a2_a2/48 + H_a1_p2**2*H_a3_p1*H_a3_p2*T_a2_a2/24 + H_a1_p2**2*H_a3_p2**2*T_a2_a2/48 -
H_a1_p2**2*H_p1_p1*T_a2_a2*T_a3_a3/32 - H_a1_p2**2*H_p1_p2*T_a2_a2*T_a3_a3/16 - H_a1_p2**2*H_p2_p2*T_a2_a2*T_a3_a3/16 +
H_a1_p2*H_a2_a2*H_a3_p2*H_p2_p2*T_a1_a3/40 + H_a1_p2*H_a2_a3*H_a2_p2*H_p2_p2*T_a1_a3/120 + H_a1_p2*H_a2_a3*H_a3_p1*H_p1_p1*T_a1_a2/120 +
H_a1_p2*H_a2_a3*H_a3_p1*H_p1_p2*T_a1_a2/60 + H_a1_p2*H_a2_a3*H_a3_p1*H_p2_p2*T_a1_a2/120 + H_a1_p2*H_a2_a3*H_a3_p2*H_p1_p1*T_a1_a2/120 +
H_a1_p2*H_a2_a3*H_a3_p2*H_p1_p2*T_a1_a2/60 + H_a1_p2*H_a2_a3*H_a3_p2*H_p2_p2*T_a1_a2/120 + H_a1_p2*H_a2_p1*H_a3_a3*H_p1_p1*T_a1_a2/40 +
H_a1_p2*H_a2_p1*H_a3_a3*H_p1_p2*T_a1_a2/20 + H_a1_p2*H_a2_p1*H_a3_a3*H_p2_p2*T_a1_a2/40 - H_a1_p2*H_a2_p1*H_a3_p1**2*T_a1_a2/30 -
H_a1_p2*H_a2_p1*H_a3_p1*H_a3_p2*T_a1_a2/15 - H_a1_p2*H_a2_p1*H_a3_p2**2*T_a1_a2/30 + H_a1_p2*H_a2_p1*H_p1_p1*T_a1_a2*T_a3_a3/24 +
H_a1_p2*H_a2_p1*H_p1_p2*T_a1_a2*T_a3_a3/12 + H_a1_p2*H_a2_p1*H_p2_p2*T_a1_a2*T_a3_a3/24 - H_a1_p2*H_a2_p2**2*H_a3_p2*T_a1_a3/30 +
H_a1_p2*H_a2_p2*H_a3_a3*H_p1_p1*T_a1_a2/40 + H_a1_p2*H_a2_p2*H_a3_a3*H_p1_p2*T_a1_a2/20 + H_a1_p2*H_a2_p2*H_a3_a3*H_p2_p2*T_a1_a2/40 -
H_a1_p2*H_a2_p2*H_a3_p1**2*T_a1_a2/30 - H_a1_p2*H_a2_p2*H_a3_p1*H_a3_p2*T_a1_a2/15 - H_a1_p2*H_a2_p2*H_a3_p2**2*T_a1_a2/30 +
H_a1_p2*H_a2_p2*H_p1_p1*T_a1_a2*T_a3_a3/24 + H_a1_p2*H_a2_p2*H_p1_p2*T_a1_a2*T_a3_a3/12 + H_a1_p2*H_a2_p2*H_p2_p2*T_a1_a2*T_a3_a3/24 +
H_a1_p2*H_a3_p2*H_p2_p2*T_a1_a3*T_a2_a2/24 + H_a2_a2*H_a3_a3*H_p1_p1**2*T_a1_a1/96 + H_a2_a2*H_a3_a3*H_p1_p1*H_p1_p2*T_a1_a1/48 +
H_a2_a2*H_a3_a3*H_p1_p2**2*T_a1_a1/48 - H_a2_a2*H_a3_p1**2*H_p1_p1*T_a1_a1/96 - H_a2_a2*H_a3_p1**2*H_p1_p2*T_a1_a1/96 -
H_a2_a2*H_a3_p1*H_a3_p2*H_p1_p2*T_a1_a1/48 + H_a2_a2*H_a3_p1*H_a3_p2*H_p2_p2*T_a1_a1/48 - H_a2_a2*H_a3_p2**2*H_p1_p1*T_a1_a1/96 -
H_a2_a2*H_a3_p2**2*H_p1_p2*T_a1_a1/96 + H_a2_a2*H_p1_p1**2*T_a1_a1*T_a3_a3/64 + H_a2_a2*H_p1_p1*H_p1_p2*T_a1_a1*T_a3_a3/32 -
H_a2_a2*H_p1_p1*H_p2_p2*T_a1_a1*T_a3_a3/96 + H_a2_a2*H_p1_p2**2*T_a1_a1*T_a3_a3/32 + H_a2_a2*H_p1_p2*H_p2_p2*T_a1_a1*T_a3_a3/192 -
H_a2_a2*H_p2_p2**2*T_a1_a3**2/48 - H_a2_a3**2*H_p1_p1**2*T_a1_a1/48 - H_a2_a3**2*H_p1_p1*H_p1_p2*T_a1_a1/96 - H_a2_a3**2*H_p1_p1*H_p2_p2*T_a1_a1/96 +
H_a2_a3*H_a2_p1*H_a3_p1*H_p1_p1*T_a1_a1/24 + H_a2_a3*H_a2_p1*H_a3_p2*H_p1_p1*T_a1_a1/48 + H_a2_a3*H_a2_p2*H_a3_p2*H_p1_p1*T_a1_a1/48 -
H_a2_a3*H_p1_p1**2*T_a1_a1*T_a2_a3/24 - H_a2_p1**2*H_a3_a3*H_p1_p1*T_a1_a1/24 - H_a2_p1**2*H_a3_a3*H_p1_p2*T_a1_a1/24 -
H_a2_p1**2*H_a3_a3*H_p2_p2*T_a1_a1/48 + H_a2_p1**2*H_a3_p1**2*T_a1_a1/48 + H_a2_p1**2*H_a3_p1*H_a3_p2*T_a1_a1/24 + H_a2_p1**2*H_a3_p2**2*T_a1_a1/48 -
H_a2_p1**2*H_p1_p1*T_a1_a1*T_a3_a3/16 - H_a2_p1**2*H_p1_p2*T_a1_a1*T_a3_a3/16 - H_a2_p1**2*H_p2_p2*T_a1_a1*T_a3_a3/32 -
H_a2_p1*H_a2_p2*H_a3_a3*H_p1_p1*T_a1_a1/24 - H_a2_p1*H_a2_p2*H_a3_a3*H_p1_p2*T_a1_a1/24 - H_a2_p1*H_a2_p2*H_a3_a3*H_p2_p2*T_a1_a1/48 +
H_a2_p1*H_a2_p2*H_a3_p1**2*T_a1_a1/48 + H_a2_p1*H_a2_p2*H_a3_p1*H_a3_p2*T_a1_a1/24 - H_a2_p1*H_a2_p2*H_a3_p2**2*T_a1_a1/48 -
H_a2_p1*H_a2_p2*H_p1_p1*T_a1_a1*T_a3_a3/16 - H_a2_p1*H_a2_p2*H_p1_p2*T_a1_a1*T_a3_a3/16 - H_a2_p1*H_a2_p2*H_p2_p2*T_a1_a1*T_a3_a3/32 +
H_a2_p1*H_a3_p1*H_p1_p1*T_a1_a1*T_a2_a3/24 + H_a2_p2**2*H_a3_a3*H_p1_p2*T_a1_a1*T_a2_a2/96 + H_a3_p2**2*H_p1_p2*T_a1_a1*T_a2_a2/96 - H_a2_p2**2*H_a3_p1*H_a3_p2*T_a1_a1/48 +
H_a2_p2**2*H_p1_p1*T_a1_a1*T_a3_a3/96 + H_a2_p2**2*H_p1_p2*T_a1_a1*T_a3_a3/96 + H_a2_p2**2*H_p2_p2*T_a1_a3**2/48 - H_a3_a3*H_p1_p1**2*T_a1_a2**2/48 -
H_a3_a3*H_p1_p1*H_p1_p2*T_a1_a1*T_a2_a2/192 - H_a3_a3*H_p1_p1*H_p1_p2*H_p2_p2*T_a1_a2**2/12 - H_a3_a3*H_p1_p1*H_p2_p2*T_a1_a1*T_a2_a2/64 -
H_a3_a3*H_p1_p1*H_p2_p2*T_a1_a2**2/24 + H_a3_a3*H_p1_p2**2*T_a1_a1*T_a2_a2/48 - H_a3_a3*H_p1_p2**2*T_a1_a2**2/12 -
H_a3_a3*H_p1_p2*H_p2_p2*T_a1_a1*T_a2_a2/192 - H_a3_a3*H_p1_p2*H_p2_p2*T_a1_a2**2/12 - H_a3_a3*H_p2_p2**2*T_a1_a2**2/48 +
H_a3_p1**2*H_p1_p1*T_a1_a2**2/48 - H_a3_p1**2*H_p1_p2*T_a1_a1*T_a2_a2/96 + H_a3_p1**2*H_p1_p2*T_a1_a2**2/24 + H_a3_p1**2*H_p2_p2*T_a1_a2**2/48 +
H_a3_p1*H_a3_p2*H_p1_p1*T_a1_a1*T_a2_a2/32 + H_a3_p1*H_a3_p2*H_p1_p1*T_a1_a2**2/24 - H_a3_p1*H_a3_p2*H_p1_p2*T_a1_a1*T_a2_a2/48 +
H_a3_p1*H_a3_p2*H_p1_p2*T_a1_a2**2/12 + H_a3_p1*H_a3_p2*H_p2_p2*T_a1_a1*T_a2_a2/32 + H_a3_p1*H_a3_p2*H_p2_p2*T_a1_a2**2/24 +
H_a3_p2**2*H_p1_p1*T_a1_a2**2/48 - H_a3_p2**2*H_p1_p2*T_a1_a1*T_a2_a2/96 + H_a3_p2**2*H_p1_p2*T_a1_a2**2/24 + H_a3_p2**2*H_p2_p2*T_a1_a2**2/48 -
H_p1_p1**2*T_a1_a1*T_a2_a3**2/32 - H_p1_p1**2*T_a1_a2**2*T_a3_a3/32 - H_p1_p1*H_p1_p2*T_a1_a2**2*T_a3_a3/8 -
H_p1_p1*H_p2_p2*T_a1_a1*T_a2_a2*T_a3_a3/32 - H_p1_p1*H_p2_p2*T_a1_a2**2*T_a3_a3/16 + H_p1_p2**2*T_a1_a1*T_a2_a2*T_a3_a3/32 -
H_p1_p2**2*T_a1_a2**2*T_a3_a3/8 - H_p1_p2*H_p2_p2*T_a1_a2**2*T_a3_a3/8 - H_p2_p2**2*T_a1_a2**2*T_a3_a3/32 - H_p2_p2**2*T_a1_a3**2*T_a2_a2/32",
  "per_part_n_terms": {
   "triangle(0, 1, 2)": 368,
   "triangle(0, 2, 1)": 368,
   "bubble_X2(1, 2)_X10": 53,
   "bubble_X2(0, 2)_X11": 53,
   "bubble_X2(0, 1)_X12": 459
  },
  "bose_symmetric_a1_a2": true
 },
 "matter_ward_three_point": {
  "identity": "Gamma3(d0h, e1, e2) + Gamma2(d1h[xi,e1], e2) + Gamma2(d1h[xi,e2], e1) = 0 (diffeomorphism invariance of the scalar-loop W[g]); row
c.h.h, matter subsector",
  "declared": {
   "lie_sign": 1,
```

```
   "drop_bubbles": false,
   "residue_d4_zero": true,
   "residue_d4_4D_part_zero": true,
   "n_terms_residue_d4": 0,
   "coefficient_exact_in_d_zero": false,
   "n_terms_generic_d": 1720
  },
  "mutation_wrong_Lie_sign": {
   "lie_sign": -1,
   "drop_bubbles": false,
   "residue_d4_zero": false,
   "residue_d4_4D_part_zero": false,
   "n_terms_residue_d4": 36,
   "coefficient_exact_in_d_zero": false,
   "n_terms_generic_d": 1836
  },
  "mutation_drop_X2_bubbles": {
   "lie_sign": 1,
   "drop_bubbles": true,
   "residue_d4_zero": false,
   "residue_d4_4D_part_zero": false,
   "n_terms_residue_d4": 118,
   "coefficient_exact_in_d_zero": false,
   "n_terms_generic_d": 1500
  },
  "reading": "the MS-bar pole residue satisfies the identity exactly, incl. the evanescent (tilde) structures; the d-dependent coefficient differs by
O(d-4) x U, i.e. only finite (non-pole) terms, which Q04 does not evaluate"
 },
 "graph_table": [
  {
   "id": "P6-G01",
   "vertices": [
    "V3_grav",
    "V3_grav",
    "V3_grav"
   ],
   "lines": [
    "h-h",
    "h-h",
    "h-h"
   ],
   "external": "h,h,h",
   "status": "Missing (not evaluated in Q04)"
  },
  {
   "id": "P6-G02",
   "vertices": [
    "V3_grav",
    "V4_grav"
   ],
   "lines": [
    "h-h",
    "h-h"
   ],
   "external": "h,h,h",
   "status": "Missing (not evaluated in Q04)"
  },
  {
   "id": "P6-G03",
   "vertices": [
    "V3_grav",
    "V_K_c_h",
    "V_cb_c_h"
   ],
   "lines": [
    "h-h",
    "c-cb",
    "h-h"
   ],
   "external": "K,c,h",
   "status": "Missing (not evaluated in Q04)"
  },
  {
   "id": "P6-G04",
   "vertices": [
    "V3_grav",
    "V_cb_c_h",
    "V_cb_c_h"
   ],
   "lines": [
    "h-h",
    "c-cb",
    "h-h"
   ],
```

```
 "external": "c,cb,h",
 "status": "Missing (not evaluated in Q04)"
},
{
 "id": "P6-G05",
 "vertices": [
  "V3_grav",
  "V_h_phi_phi",
  "V_h_phi_phi"
 ],
 "lines": [
  "h-h",
  "phi-phi",
  "h-h"
 ],
 "external": "h,phi,phi",
 "status": "Missing (not evaluated in Q04)"
},
{
 "id": "P6-G06",
 "vertices": [
  "V3_grav",
  "V_hh_phi_phi"
 ],
 "lines": [
  "h-h",
  "h-h"
 ],
 "external": "h,phi,phi",
 "status": "Missing (not evaluated in Q04)"
},
{
 "id": "P6-G07",
 "vertices": [
  "V_K_c_h",
  "V_cb_c_h",
  "V_cb_c_h"
 ],
 "lines": [
  "h-h",
  "cb-c",
  "cb-c"
 ],
 "external": "K,c,h",
 "status": "Missing (not evaluated in Q04)"
},
{
 "id": "P6-G08",
 "vertices": [
  "V_cb_c_h",
  "V_cb_c_h",
  "V_cb_c_h"
 ],
 "lines": [
  "cb-c",
  "cb-c",
  "cb-c"
 ],
 "external": "h,h,h",
 "status": "Missing (not evaluated in Q04)"
},
{
 "id": "P6-G09",
 "vertices": [
  "V_cb_c_h",
  "V_cb_c_h",
  "V_cb_c_h"
 ],
 "lines": [
  "cb-c",
  "h-h",
  "cb-c"
 ],
 "external": "c,cb,h",
 "status": "Missing (not evaluated in Q04)"
},
{
 "id": "P6-G10",
 "vertices": [
  "V_cb_c_h",
  "V_cb_c_h",
  "V_cs_c_c"
 ],
 "lines": [
```

```
   "h-h",
   "cb-c",
   "c-cb"
  ],
  "external": "c,c,cs",
  "status": "Missing (not evaluated in Q04)"
 },
 {
  "id": "P6-G11",
  "vertices": [
   "V_cb_c_h",
   "V_h_phi_phi",
   "V_phis_c_phi"
  ],
  "lines": [
   "h-h",
   "phi-phi",
   "c-cb"
  ],
  "external": "c,phi,phis",
  "status": "Missing (not evaluated in Q04)"
 },
 {
  "id": "P6-G12",
  "vertices": [
   "V_h_phi_phi",
   "V_h_phi_phi",
   "V_h_phi_phi"
  ],
  "lines": [
   "phi-phi",
   "phi-phi",
   "h-h"
  ],
  "external": "h,phi,phi",
  "status": "Missing (not evaluated in Q04)"
 },
 {
  "id": "P6-G13",
  "vertices": [
   "V_h_phi_phi",
   "V_hh_phi_phi"
  ],
  "lines": [
   "phi-phi",
   "h-h"
  ],
  "external": "h,phi,phi",
  "status": "Missing (not evaluated in Q04)"
 },
 {
  "id": "P6-G14",
  "vertices": [
   "V_h_phi_phi",
   "V_h_phi_phi",
   "V_h_phi_phi"
  ],
  "lines": [
   "phi-phi",
   "phi-phi",
   "phi-phi"
  ],
  "external": "h,h,h",
  "status": "UV pole evaluated (exact residue, own rules, validated vs D11; Ward-checked); finite part Missing"
 },
 {
  "id": "P6-G15",
  "vertices": [
   "V_h_phi_phi",
   "V_hh_phi_phi"
  ],
  "lines": [
   "phi-phi",
   "phi-phi"
  ],
  "external": "h,h,h",
  "status": "UV pole evaluated (exact residue, own rules, validated vs D11; Ward-checked); finite part Missing"
 },
 {
  "id": "P6-T1",
  "vertices": [
   "V5_grav"
  ],
  "lines": [
```

```
    "h-h"
   ],
   "external": "h,h,h",
   "status": "derived 0 (scaleless, no log member)"
  },
  {
   "id": "P6-T2",
   "vertices": [
    "V_hhh_phi_phi"
   ],
   "lines": [
    "phi-phi"
   ],
   "external": "h,h,h",
   "status": "derived 0 (scaleless, no log member)"
  },
  {
   "id": "P6-T3",
   "vertices": [
    "V_hhh_phi_phi"
   ],
   "lines": [
    "h-h"
   ],
   "external": "h,phi,phi",
   "status": "derived 0 (scaleless, no log member)"
  }
 ],
 "n_evaluated_pole": 2,
 "n_missing": 13,
 "n_derived_zero": 3
}
```


certificates/Q03_O1_record.json


```
{
 "landmark": "XVI-Q",
 "release": "Q03",
 "O1_closure": "CLOSED (scoped)",
 "O1_closure_confirmed": true,
 "confirmation": {
  "report_sha256": "e4b54702bc768899f22a00a289de0c0aeb2bf9317b87a28c4a462f05ae4bf495",
  "reviewed_zip_sha256": "378651a3a279cde0a8fd67d4e39ac5833d5a94ff00f10f62a001be8ab2b244dc",
  "reviewed_tree_sha256": "4b551b9801cd14b8f2b81775942d535ab282a75c1557148a94a91a9dad88a810",
  "decision": "CONFIRM CLOSED (scoped)",
  "report_file": "reproducibility/independent_rebuild_report_q02.json",
  "report_sha256_now": "e4b54702bc768899f22a00a289de0c0aeb2bf9317b87a28c4a462f05ae4bf495",
  "binding_ok": true,
  "reviewer_conditions_for_the_label": [
   "record that ST4a with QdS externals needs the p14r 11 directions PLUS the p22 evanescent operators (outside p14r; sigma0 closure incl. them
verified in my p22 review) -- 'ST4a in the declared p14r domain' alone is true only on 4D externals",
   "record that the prescription is XVI-P's DRED one; XVI-O's o70-sync D11 kernel prescription is not the one used (both XVI-O kernel continuations
fail on evanescent externals, my p22 finding)"
  ],
  "conditions_discharged_by": [
   "counterterm_domain (enlarged)",
   "prescription"
  ]
 },
 "scope_verbatim_from_independent_review": "one-loop, order kappa^2, bulk two-point retained sector of S_inv (h, phi, c, cbar, b + antifields; gh0
seeds), m=0, frozen gauge fermion, under XVI-P's declared prescription (DRED: QdS loop momenta, Q4S vertex indices, MS-bar) INCLUDING the p22
evanescent counterterm contract revision; NOT the functional ST4b, NOT bulk symmetry restoration, NOT XVI-O's own ledger (read-only, still NOT_CLOSED
there)",
 "prescription": {
  "used": "XVI-P declared prescription: DRED (QdS loop momenta, Q4S vertex indices, opaque d via qguard), MS-bar, Feynman +i0, p22 evanescent
counterterms; m = 0; frozen gauge fermion",
  "not_used": "XVI-O o70-sync D11 kernel counterterm prescription (closure reason 8): superseded on evanescent legs, both kernel continuations fail
there (p22); reason 8 is NOT marked answered"
 },
 "counterterm_domain": {
  "p14r_4D_directions": [
   "c.K#0",
   "c.K#1",
   "c.K#2",
   "c.cb#0",
   "c.cb#1",
   "h.h#0",
   "h.h#1",
   "h.h#2",
   "h.h#3",
   "h.h#4",
   "phi.phi#0"
  ],
```

```
  "n_p14r_4D": 11,
  "p22_evanescent_operators": [
   "K'.c E2 (evanescent)",
   "cbar.c SE (evanescent)",
   "h.h evanescent: g4_mn gt_rs (p.p)^2",
   "h.h evanescent: g4_mr gt_ns (p.p)^2",
   "h.h evanescent: g4_rs gt_mn (p.p)^2",
   "h.h evanescent: gt_mn gt_rs (p.p)^2",
   "h.h evanescent: gt_mn p_r p_s (p.p)^1",
   "h.h evanescent: gt_mr gt_ns (p.p)^2",
   "h.h evanescent: gt_mr p_n p_s (p.p)^1",
   "h.h evanescent: gt_rs p_m p_n (p.p)^1"
  ],
  "n_p22_evanescent": 10,
  "n_enlarged_domain": 21,
  "p22_evanescent_coefficients": {
   "Kc_E2": "-1/16",
   "ccb_SE": "1/8",
   "hh_sum": {
    "g4_mn gt_rs (p.p)^2": "-7*I/120",
    "g4_mr gt_ns (p.p)^2": "-I/160",
    "g4_rs gt_mn (p.p)^2": "-7*I/120",
    "gt_mn gt_rs (p.p)^2": "I/160",
    "gt_mn p_r p_s (p.p)^1": "7*I/120",
    "gt_mr gt_ns (p.p)^2": "I/160",
    "gt_mr p_n p_s (p.p)^1": "I/160",
    "gt_rs p_m p_n (p.p)^1": "7*I/120"
   }
  },
  "statement": "The counterterm domain used for O.1 item 5 is the ENLARGED domain: the 11 p14r 4D directions (over Q(i)) PLUS the p22 evanescent
operators (disclosed p22 contract revision, outside p14r). With the declared QdS external momenta the local divergent part contains evanescent poles
that are cancelled only by the p22 evanescent operators; 'ST4a in the declared p14r domain' alone holds only for 4D externals. sigma_0 closure
including the evanescent operators: verified (p20/p22 independent reviews)."
 },
 "items": [
  {
   "item": "State retained quantum identity",
   "owner": "O.6",
   "XVI_O_status": "Constructed_as_statement",
   "evidence": "XVI-O ST4a/ST4b and (S,Gamma_1) = A_1 (retained_quantum_identity.tex) stated on the bulk two-point retained sector; 23 gh0/gh1 rows
enumerated mechanically",
   "earned": true
  },
  {
   "item": "List amplitudes constrained by identity",
   "owner": "O.4",
   "XVI_O_status": "manually_specified_partially_checked_O.4",
   "evidence": "every gh0/gh1 one/two-field row classified with an individual justification; independent 1PI enumeration from the loop-capable
vertices: E2 = 9, E1 = 3 (== XVI-P 9 / 3); antifield vertices c*cc, phi*c phi give no E <= 2 graph",
   "earned": true
  },
  {
   "item": "S_inv^{[2]} classical consistency",
   "owner": "O.2",
   "XVI_O_status": "Provisional_foundation_repaired_master_Partial",
   "evidence": "XVI-Q Q01: 2(S0,S2)+(S1,S1)=0 Constructed ((a) ell_4, (b) explicit antibracket expansion, (c) ghost/antifield); Q01 independent
rebuild PASSED",
   "earned": true
  },
  {
   "item": "Evaluate inventory graphs",
   "owner": "O.5",
   "XVI_O_status": "D11_kernel_Constructed_subsector_under_prescription_o70-sync_bulk_incomplete",
   "evidence": "all 9 E=2 + 3 E=1 graphs have exact values: D01, D02, D03, D11, D13, D14a-c (XVI-P, pinned); D04 (Q02, 0 by engine routes 1+2 and
independent route 4, with nonzero mutations, literature agreement); D05a (Q02, derived 0); D06 (Q02 == XVI-P Sigma); D12 reconciled (derived 0)",
   "earned": true
  },
  {
   "item": "Demonstrate identity remainder cancellation / CT solution",
   "owner": "O.6/O.7",
   "XVI_O_status": "Unresolved_ST4a_ST4b_split",
   "evidence": "ST4a: MS-bar CT in the ENLARGED domain = 11 p14r 4D directions over Q(i) (phi.phi = 0 earned) PLUS the p22 evanescent operators (K'.c
E2, cbar.c SE, 8 h.h evanescent structures), which alone cancel the evanescent poles with d-dimensional (QdS) externals; ST4b: c.h defect ZERO exactly
in d (P19, x = 0), c.b = ghost equation (Sigma == D06), c.cbar* unreachable, c.phi parity; full sigma_0 closure of the complete MS-bar CT incl. the
evanescent operators in every gh+1 sector (p20/p22); cohomology of the domain 0; no free-EOM equivalence used",
   "earned": true,
   "evidence_corrected_in": "Q03 (Q02 review finding ST4a_domain_wording, medium)"
  },
  {
   "item": "Enlarge target if identity requires more legs/orders",
   "owner": "O.1",
   "XVI_O_status": "rule_active_not_triggered",
   "evidence": "rule 1 evaluated below: not triggered",
   "earned": true
```

```
  }
 ],
 "all_earned": true,
 "closure_reasons": {
  "No retained quantum identity is verified (O.6 Unresolved).": {
   "status": "ANSWERED",
   "answer": "identity verified on the retained two-point sector (items 1, 5)."
  },
  "XVI-N supplies only classical (S_inv,S_inv)=O(kappa^2) through O(kappa) truncation; not a quantum Ward/BV identity.": {
   "status": "ANSWERED",
   "answer": "the quantum identity is the EA-route (S,Gamma_1) = A_1, not XVI-N's classical truncation."
  },
  "XVI-M selected-image matchings are Target I data, not sector amplitudes (M.8 Unresolved).": {
   "status": "ANSWERED",
   "answer": "XVI-M selected-image data are not used anywhere (graphs come from the action's vertices)."
  },
  "One-loop antifield source channels at kappa^2 remain Unresolved after withdrawing invalid gh=-1 assignments.": {
   "status": "ANSWERED",
   "answer": "one-loop antifield channels derived from vertices: K'.c = D13 (evaluated); phi*'.c odd (0); c*', cbar*', b* give no E <= 2 graph
(independent enumeration); no (Phi, Phi*) assignment used."
  },
  "O.4 is manually_specified_partially_checked_inventory \u2014 not a completeness claim.": {
   "status": "ANSWERED",
   "answer": "completeness by mechanical enumeration of rows AND graphs (two independent enumerations)."
  },
  "kappa^2 classical master Partial: (S0,S0)/(S0,S1) Constructed; 2(S0,S2)+(S1,S1) Partial (scalar OK; grav ell_4 Incomplete) --- not full
Constructed.": {
   "status": "ANSWERED",
   "answer": "kappa^2 master Constructed in Q01 (incl. grav ell_4)."
  },
  "ST4a local matching Unresolved; functional ST4b not claimed; kernel_two_point_Ward Constructed (kernel only).": {
   "status": "ANSWERED",
   "answer": "ST4a local matching = explicit CT in the declared domain; ST4b on the retained two-point projection verified (the FUNCTIONAL ST4b, all
n-point, is NOT claimed). Local matching uses the ENLARGED domain (p14r 4D + p22 evanescent)."
  },
  "D11 matter-loop kernel counterterm Constructed_subsector_under_prescription (o70-sync; four math prerequisites); NOT completed bulk quantum sector;
kernel Ward != bulk ST4b.": {
   "status": "NOT_ANSWERED_IN_XVI_O_TERMS (superseded prescription)",
   "answer": "XVI-O's o70-sync D11 matter-loop kernel counterterm prescription is NOT the one used and is not completed: both XVI-O kernel
continuations fail on evanescent external legs (p22 finding). O.1 is closed under XVI-P's declared DRED prescription (QdS loop momenta, Q4S vertex
indices, MS-bar, Feynman +i0, p22 evanescent counterterm contract revision). The bulk sector (hh, phi phi, cbar c, K'.c, one-point) is evaluated, not
only the D11 kernel; this does not complete XVI-O's kernel prescription."
  }
 },
 "enlarge_rules": {
  "If the stated identity constrains n-point or higher-loop pieces at the same coupling order, enlarge the target before Constructed.": {
   "triggered": false,
   "argument": "At one loop the kappa^2 component of (S,Gamma_1) is (S_0, Gamma_1[kappa^2]) + (S_1, Gamma_1[kappa^1]) (S_k: tree kappa^k terms incl.
the seeds D07-D09). One-loop graphs with E external legs are kappa^E (every vertex kappa^{legs-2}, p14r power counting; XVI-P p23): Gamma_1[kappa^1]
is one-point only and is 0 (D14a-c), Gamma_1[kappa^2] is two-point only (three-point starts at kappa^3; one-loop kappa^2 three-point sector empty).
Hence the identity at this order constrains no n >= 3 or higher-loop piece."
  },
  "If classical kappa^2 consistency fails, downgrade or repair O.2 before quantum contractions.": {
   "triggered": false,
   "argument": "classical kappa^2 consistency holds (Q01)."
  },
  "Never close the sector by silently identifying XVI-M selected graph with an action graph via ell_3.": {
   "respected": true,
   "argument": "no XVI-M graph is used or identified with an action graph; XVI_M pin unchanged; every graph is built from S_inv vertices."
  }
 },
 "downgrade_rules": {
  "If closure remains unknowable after O.6 statement, keep Provisional.": "not applicable: closure decided by the checks above",
  "If a required amplitude is omitted without individual justification, status fails.": "respected: no amplitude omitted; every row/graph individually
justified"
 },
 "not_implied": [
  "the functional / all-n-point ST4b identity (only the retained two-point projection is verified)",
  "bulk symmetry restoration (bulk_symmetry_restored stays false)",
  "any change to XVI-O's own O.1 ledger: XVI-O is read-only and its O.1 stays NOT_CLOSED there",
  "the three-point sector / P.6 (kappa^3) and higher orders or loops",
  "the insertion (frozen; XVI-N massless obstruction untouched)",
  "finite renormalised amplitudes with non-QdS (epsilon-scalar) external components (p22 Q4S-trace O(eps)x pole remnant stays disclosed)"
 ],
 "caveats": [
  "same-box reviews: Q01/Q02 independent rebuilds ran on the author's machine (shared Python/sympy pin and TeX Live); not an independent machine",
  "D01/D03 pole values were never recomputed from independent Feynman rules in any review; cross-checked only via the exact cancellation of the c.h
defect in d",
  "full sigma_0 closure and the c.h defect were not re-derived in the Q02 review; they rely on the earlier p19-p22 independent checks on byte-
identical inputs",
  "D06 accepted via exact equality with XVI-P P15 Sigma (ghost equation), not via the reviewer's own Feynman rules"
 ],
 "XVI_O_ledger": "read-only; O.1 there stays NOT_CLOSED (unchanged)"
}
```

certificates/Q02_bulk_identity.json


```
{
 "landmark": "XVI-Q",
 "release": "Q02",
 "XVI_O_ST4_definition_verbatim": {
  "ST4a": "Gamma_1,local = sum z_i C_i (gh=0)",
  "ST4b": "P_ret(S,C)=-P_ret A_1 (gh(C)=0; defect gh=1)",
  "withdrawn_OR": "local_OR_defect single target"
 },
 "bulk_counterterms_explicit": {
  "convention": "Ghat coordinates on the p14r kappa^2 densities (G = kappa^2 N_eps Ghat MB); MS-bar CT = -pole; c.cb in the route-G (c,cb) orientation
used by the p20 full closure",
  "coefficients": {
   "h.h#0": "49*I/480",
   "h.h#1": "41*I/160",
   "h.h#2": "-49*I/240",
   "h.h#3": "-41*I/80",
   "h.h#4": "43*I/120",
   "c.cb#0": "-3/8",
   "c.cb#1": "1/4",
   "c.K#0": "0",
   "c.K#1": "-3/8",
   "c.K#2": "1/4",
   "phi.phi#0": "0"
  },
  "domain_sectors_and_sizes": {
   "h.h": 5,
   "c.cb": 2,
   "c.K": 3,
   "phi.phi": 1
  },
  "every_label_in_domain": true,
  "n_directions": 11,
  "all_in_Q_i": true,
  "cbc_from_D06_equals_P15_orientation_poles": [
   true,
   true
  ],
  "cbc_orientation_ratio_routeG_over_P15": [
   "-1",
   "-1"
  ],
  "phiphi": "0 from Q02 D04 + D05a (in P19 the phi.phi coordinate 0 was a placeholder 'not an input'; now earned)",
  "evanescent_p22": {
   "Kc_E2": "-1/16",
   "ccb_SE": "1/8",
   "hh_sum": {
    "g4_mn gt_rs (p.p)^2": "-7*I/120",
    "g4_mr gt_ns (p.p)^2": "-I/160",
    "g4_rs gt_mn (p.p)^2": "-7*I/120",
    "gt_mn gt_rs (p.p)^2": "I/160",
    "gt_mn p_r p_s (p.p)^1": "7*I/120",
    "gt_mr gt_ns (p.p)^2": "I/160",
    "gt_mr p_n p_s (p.p)^1": "I/160",
    "gt_rs p_m p_n (p.p)^1": "7*I/120"
   },
   "role": "outside the p14r 4D domain (disclosed p22 contract revision); subtract the evanescent poles only; NOT needed for the identity (P19 ch
solve x = 0; evanescent poles decouple from the ch slots)"
  },
  "ok": true
 },
 "independent_graph_enumeration": {
  "E1": [
   [
    [
     "V3_grav"
    ],
    [
     "h-h"
    ],
    [
     "h"
    ]
   ],
   [
    [
     "V_cb_c_h"
    ],
    [
     "c-cb"
    ],
    [
     "h"
    ]
```

```
 ],
 [
  [
   "V_h_phi_phi"
  ],
  [
   "phi-phi"
  ],
  [
   "h"
  ]
 ]
],
"E2": [
 [
  [
   "V3_grav",
   "V3_grav"
  ],
  [
   "h-h",
   "h-h"
  ],
  [
   "h",
   "h"
  ]
 ],
 [
  [
   "V4_grav"
  ],
  [
   "h-h"
  ],
  [
   "h",
   "h"
  ]
 ],
 [
  [
   "V_K_c_h",
   "V_cb_c_h"
  ],
  [
   "c-cb",
   "h-h"
  ],
  [
   "K",
   "c"
  ]
 ],
 [
  [
   "V_cb_c_h",
   "V_cb_c_h"
  ],
  [
   "c-cb",
   "c-cb"
  ],
  [
   "h",
   "h"
  ]
 ],
 [
  [
   "V_cb_c_h",
   "V_cb_c_h"
  ],
  [
   "c-cb",
   "h-h"
  ],
  [
   "c",
   "cb"
  ]
 ],
 [
  [
```

```
    "V_h_phi_phi",
    "V_h_phi_phi"
   ],
   [
    "h-h",
    "phi-phi"
   ],
   [
    "phi",
    "phi"
   ]
  ],
  [
   [
    "V_h_phi_phi",
    "V_h_phi_phi"
   ],
   [
    "phi-phi",
    "phi-phi"
   ],
   [
    "h",
    "h"
   ]
  ],
  [
   [
    "V_hh_phi_phi"
   ],
   [
    "h-h"
   ],
   [
    "phi",
    "phi"
   ]
  ],
  [
   [
    "V_hh_phi_phi"
   ],
   [
    "phi-phi"
   ],
   [
    "h",
    "h"
   ]
  ]
 ],
 "n_E1": 3,
 "n_E2": 9
},
"vertex_checks": {
 "n_vertices": 15,
 "interaction_vertices": [
  "V3_grav",
  "V4_grav",
  "V_cb_c_h",
  "V_K_c_h",
  "V_cs_c_c",
  "V_phis_c_phi",
  "V_h_phi_phi",
  "V_hh_phi_phi"
 ],
 "phi_parity_even_all": true,
 "phi_parity_per_vertex": {
  "V2_grav": true,
  "V3_grav": true,
  "V4_grav": true,
  "M0_ghost": true,
  "V_cb_c_h": true,
  "V_K_c": true,
  "V_K_c_h": true,
  "V_cs_c_c": true,
  "V_phis_c_phi": true,
  "V_b_h": true,
  "V_b_b": true,
  "V_cbs_b": true,
  "M_phi": true,
  "V_h_phi_phi": true,
  "V_hh_phi_phi": true
 },
```

```
 "aux_b_cbs_bs_on_no_interaction_vertex": true,
 "quadratic_vertices_with_aux": [
  "V_b_h",
  "V_b_b",
  "V_cbs_b"
 ]
},
"rows": {
 "b.b": {
  "group": "gh0_two",
  "class": "zero",
  "reason": "auxiliary/nonminimal field on no interaction vertex (checked): no 1PI loop entry",
  "outcome": "zero"
 },
 "b.cbs": {
  "group": "gh0_two",
  "class": "zero",
  "reason": "auxiliary/nonminimal field on no interaction vertex (checked): no 1PI loop entry",
  "outcome": "zero"
 },
 "b.h": {
  "group": "gh0_two",
  "class": "zero",
  "reason": "auxiliary/nonminimal field on no interaction vertex (checked): no 1PI loop entry",
  "outcome": "zero"
 },
 "b.phi": {
  "group": "gh0_two",
  "class": "zero",
  "reason": "phi-parity: every vertex even in {phi, phi*} (checked) -> odd rows vanish identically",
  "outcome": "zero"
 },
 "bs.c": {
  "group": "gh0_two",
  "class": "zero",
  "reason": "auxiliary/nonminimal field on no interaction vertex (checked): no 1PI loop entry",
  "outcome": "zero"
 },
 "c.cb": {
  "group": "gh0_two",
  "class": "cbc",
  "outcome": "cancellable"
 },
 "c.hs": {
  "group": "gh0_two",
  "class": "Kc",
  "outcome": "cancellable"
 },
 "c.phis": {
  "group": "gh0_two",
  "class": "zero",
  "reason": "phi-parity: every vertex even in {phi, phi*} (checked) -> odd rows vanish identically",
  "outcome": "zero"
 },
 "cbs.cbs": {
  "group": "gh0_two",
  "class": "zero",
  "reason": "auxiliary/nonminimal field on no interaction vertex (checked): no 1PI loop entry",
  "outcome": "zero"
 },
 "cbs.h": {
  "group": "gh0_two",
  "class": "zero",
  "reason": "auxiliary/nonminimal field on no interaction vertex (checked): no 1PI loop entry",
  "outcome": "zero"
 },
 "cbs.phi": {
  "group": "gh0_two",
  "class": "zero",
  "reason": "phi-parity: every vertex even in {phi, phi*} (checked) -> odd rows vanish identically",
  "outcome": "zero"
 },
 "h.h": {
  "group": "gh0_two",
  "class": "hh",
  "outcome": "cancellable"
 },
 "h.phi": {
  "group": "gh0_two",
  "class": "zero",
  "reason": "phi-parity: every vertex even in {phi, phi*} (checked) -> odd rows vanish identically",
  "outcome": "zero"
 },
 "phi.phi": {
```

```
   "group": "gh0_two",
   "class": "phiphi",
   "outcome": "zero"
  },
  "b.c": {
   "group": "gh1_two",
   "class": "ST4b_ghost_equation",
   "reason": "c.b row of (S,Gamma) = b Sigma_cbc + (d b) G_Kc: the ghost equation",
   "outcome": "zero"
  },
  "c.cbs": {
   "group": "gh1_two",
   "class": "ST4b_unreachable",
   "reason": "c.cbar*: unreachable (XVI-P P22 a_closure: S0 quadratic part h.h, h.b, b.b, b.cbs)",
   "outcome": "zero"
  },
  "c.h": {
   "group": "gh1_two",
   "class": "ST4b_ch",
   "outcome": "zero"
  },
  "c.phi": {
   "group": "gh1_two",
   "class": "zero",
   "reason": "phi-parity: every vertex even in {phi, phi*} (checked) -> odd rows vanish identically",
   "outcome": "zero"
  },
  "h": {
   "group": "gh0_one",
   "class": "onepoint_h",
   "outcome": "zero"
  },
  "phi": {
   "group": "gh0_one",
   "class": "zero",
   "reason": "phi-parity: every vertex even in {phi, phi*} (checked) -> odd rows vanish identically",
   "outcome": "zero"
  },
  "b": {
   "group": "gh0_one",
   "class": "zero",
   "reason": "auxiliary/nonminimal field on no interaction vertex (checked): no 1PI loop entry",
   "outcome": "zero"
  },
  "cbs": {
   "group": "gh0_one",
   "class": "zero",
   "reason": "auxiliary/nonminimal field on no interaction vertex (checked): no 1PI loop entry",
   "outcome": "zero"
  },
  "c": {
   "group": "gh1_one",
   "class": "zero",
   "reason": "gh1 one-point defect c^mu(p=0): no 1PI one-loop graph has a single external ghost (the only ghost vertex V_cb_c_h would need a second
internal c to close the cbar line) and no Lorentz-covariant constant vector exists",
   "outcome": "zero"
  }
 },
 "n_rows": 23,
 "unclassified_rows": [],
 "ST4a": {
  "hh": {
   "graphs": [
    "D01",
    "D02",
    "D03",
    "D11",
    "D12"
   ],
   "values": {
    "D01_D03": "XVI-P P16_batchC (evaluated)",
    "D02": "0 derived (P15_batchA)",
    "D12": "0 (DERIVED: every member int (l^2)^j/l^2 is scaleless with omega = 2 + 2j > 0 -- a pure power divergence; no 1/eps on the UV or the IR
side; MS-bar counterterm 0) ; XVI-P P15_batchA: vanishing_under_declared_prescription",
    "D11": "XVI-P P18_D11 (matching M1-M3; evanescent EV_M2)"
   },
   "counterterm": "p14r 4D h.h MS-bar directions + p22 evanescent h.h operators (sum coefficients pinned in P22)",
   "outcome": "cancellable",
   "evidence_flags": {
    "batchC_evaluated": true,
    "batchA_evaluated": true
   }
  },
  "cbc": {
```

```
   "graphs": [
    "D06"
   ],
   "D06_equals_XVI_P_Sigma": true,
   "poles": {
    "S1_delta^rho_nu_p4": "-3/8",
    "S2_p^rho_p_nu_p2": "1/4",
    "SE_Gt^rho_nu_p4": "-1/8"
   },
   "counterterm": {
    "4D (delta p^4, p^rho p_nu p^2)": {
     "S1_delta^rho_nu_p4": "3/8",
     "S2_p^rho_p_nu_p2": "-1/4"
    },
    "evanescent SE (p22 ghost-equation partner)": "1/8"
   },
   "SE_pole_plus_p22_CT_zero": true,
   "outcome": "cancellable"
  },
  "phiphi": {
   "graphs": [
    "D04",
    "D05a"
   ],
   "D04": "0",
   "D04_routes_agree": true,
   "D05a": "0 (DERIVED: every member int (l^2)^j/l^2 is scaleless with omega = 2 + 2j > 0 -- a pure power divergence; no 1/eps on the UV or the IR
side; MS-bar counterterm 0)",
   "total_pole": "0",
   "counterterm": "z_phiphi = 0 (none needed; the declared phi phi domain {phi Box^2 phi} is untouched)",
   "outcome": "zero"
  },
  "Kc": {
   "graphs": [
    "D13"
   ],
   "4D_poles_in_p14r_cK_basis": true,
   "E2_pole": "1/16",
   "p22_Kc_E2_CT": "-1/16",
   "E2_pole_plus_CT_zero": true,
   "outcome": "cancellable"
  },
  "onepoint_h": {
   "graphs": [
    "D14a",
    "D14b",
    "D14c"
   ],
   "value": "0 derived (P15_batchA)",
   "outcome": "zero"
  }
 },
 "ST4b": {
  "ST4b_ch": {
   "source": "XVI-P P19_defect: projected defect ZERO exactly in d; counterterm solve outcome ZERO",
   "projected_identity_verified": "earned",
   "outcome": "zero"
  },
  "ST4b_ghost_equation": {
   "source": "XVI-P P15_D13 ghost_equation (64 components, exact in d) + P22 a_closure c.b (closed; exact in the gt sector); Sigma there == D06 here
exactly",
   "ghost_equation_agree": true,
   "c_b_closed": true,
   "D06_is_that_Sigma": true,
   "outcome": "zero"
  },
  "ST4b_unreachable": {
   "source": "P22 a_closure c.cbs",
   "outcome": "zero"
  }
 },
 "sigma0_closure": {
  "XVI_P_full_closure_all_gh1_sectors": "P19_defect full_closure_mutation_detector + renormalised_defect_zero",
  "renormalised_defect_zero": true,
  "mutation_detector_cases_detected": {
   "drop_graph_D11": false,
   "drop_graph_D13 (c.K data = 0, Sigma held fixed)": true,
   "flip_sign_Kc (c.K data -> -c.K, Sigma held fixed)": true,
   "drop_graph_D03": true,
   "drop_graph_D01": true,
   "flip_phase_D03": true,
   "symmetry_factor_D03_x2": true,
   "Sigma_orientation_unconverted (p19 error)": true
  },
```

```
  "phiphi_CT": "0 (D04 + D05a = 0); sigma_0 rank on the phi.phi direction is 0 (XVI-P P19 D_cap_ker_sigma0), so the direction is sigma_0-closed for
any coefficient; NO free-EOM equivalence is used (XVI-O 0.6 domain item 4)",
  "phiphi_sigma0_rank": 0,
  "phi_odd_rows": "identically zero (parity)",
  "closed": true,
  "full_closure_with_complete_MSbar_CT": "p20 (corrects p19): with the P15 Sigma coordinates converted from the (cb,c) to the route-G (c,cb)
orientation (measured ratio -1), the DERIVED factor +1 gives full sigma_0 closure of the complete MS-bar counterterm (h.h + c.K + c.cb) in every ghost
+1 sector; the p19 table above (factors 1, -1, i, -i on UNCONVERTED coordinates) is the bookkeeping error, kept for the erratum",
  "relative_factor_derived_equals_P15": true,
  "sigma0_ranks": {
   "h.h": {
    "dim": 5,
    "sigma0_rank": 3,
    "P_ch_sigma0_rank": 3
   },
   "c.cb": {
    "dim": 2,
    "sigma0_rank": 2,
    "P_ch_sigma0_rank": 0
   },
   "c.K": {
    "dim": 3,
    "sigma0_rank": 3,
    "P_ch_sigma0_rank": 1
   },
   "phi.phi": {
    "dim": 1,
    "sigma0_rank": 0,
    "P_ch_sigma0_rank": 0
   }
  },
  "cohomology_dim_D": 0
 },
 "bulk_identity_outcome": "cancellable",
 "residual": null,
 "E2_enumeration_XVI_P_p23": {
  "enumerated": 9,
  "frozen": 9,
  "missing_from_enumeration": [],
  "not_in_frozen_inventory": [],
  "equal": true
 },
 "bulk_counterterms": {
  "hh": "p14r MS-bar h.h + p22 evanescent (pinned)",
  "Kc": "p14r c.K' + p22 E2 (-1/16)",
  "cbc": {
   "4D": {
    "S1_delta^rho_nu_p4": "3/8",
    "S2_p^rho_p_nu_p2": "-1/4"
   },
   "SE": "1/8"
  },
  "phiphi": "0",
  "onepoint": "0"
 }
}
```


certificates/Q01_O2.json


```
{
 "landmark": "XVI-Q",
 "release": "Q01",
 "equation": "2(S0,S2) + (S1,S1) = 0 on S_inv (classical BV master at kappa^2)",
 "items": {
  "a_ell4": {
   "status": "Constructed",
   "argument": "XVI-P p23 ell_4 export (3360 terms, re-executed byte-identical) equals an INDEPENDENT Gamma-Gamma reconstruction term by term; the
single normalisation (-2) is fixed by XVI-O's conformal probe and equals XVI-O's convention ell_4 = -2[eps^4](sqrt(-q)R); the export also reproduces
XVI-O's probe exactly",
   "detail": {
    "route": "independent Gamma-Gamma series (this file), not XVI-P quartic.py",
    "normalisation_from_XVI_O_probe": "-2",
    "n_terms_mine": 3360,
    "n_terms_export": 3360,
    "full_polynomial_equal": true,
    "n_terms_difference": 0,
    "export_on_XVI_O_probe_equal": true,
    "seconds": 2.1,
    "mutation_one_term_x2_detected": true
   }
  },
  "b_antibracket_expansion": {
   "status": "Constructed",
```

```
   "argument": "(S,S)|af0,grav at kappa^2 = 2 int W_2, W_2 = sum E(ell_4) s0 h + E(ell_3) s1 h (s2 = 0 derived) -- the explicit component form of
2(S0,S2)+(S1,S1); W_2 has 15428 terms and E_{c^mu}(W_2) = 0 exactly (independent XVI-Q jet engine; agrees with XVI-P grav_noether re-executed)",
   "detail": {
    "unmutated": {
     "kappa0": {
      "W_terms": 132,
      "E_c_W_terms": {
       "0": 0,
       "1": 0,
       "2": 0,
       "3": 0
      },
      "antibracket_af0_vanishes": true
     },
     "kappa1": {
      "W_terms": 2148,
      "E_c_W_terms": {
       "0": 0,
       "1": 0,
       "2": 0,
       "3": 0
      },
      "antibracket_af0_vanishes": true
     },
     "kappa2": {
      "W_terms": 15428,
      "E_c_W_terms": {
       "0": 0,
       "1": 0,
       "2": 0,
       "3": 0
      },
      "antibracket_af0_vanishes": true
     },
     "seconds": 4.6
    },
    "mutation_scale_ell4_term_detected": true,
    "mutation_drop_s1_detected": true
   }
  },
  "c_ghost_antifield": {
   "status": "Constructed",
   "argument": "re-executed exact XVI-P P13 master: af1/af2 (invariant and Psi-transformed variables) and the af0 ghost/gauge-fixing cell vanish at
kappa^0..kappa^2 with s2 DERIVED zero in all sectors (not invented; answers XVI-O's 'Absent under inherited s'); s nilpotent exactly; 7 mutations each
produce residuals. Evidence = re-execution of XVI-P's engine (no second independent engine for this sector in Q01).",
   "kappa2_table": {
    "af0_grav_Noether": true,
    "af0_scalar_Noether": true,
    "af0_ghost_gf_transformed": true,
    "af1_invariant": true,
    "af1_transformed": true,
    "af2_invariant": true,
    "af2_transformed": true
   },
   "s2": {
    "h": "derived_zero",
    "c": "derived_zero",
    "phi": "derived_zero",
    "cb": "derived_zero",
    "b": "derived_zero"
   }
  }
 },
 "item_status": {
  "a_ell4": "Constructed",
  "b_antibracket_expansion": "Constructed",
  "c_ghost_antifield": "Constructed"
 },
 "O2_kappa2_master_status": "Constructed",
 "scope": "classical, S_inv, through O(kappa^2), m = 0; NOT quantum, NOT the prepared S_D, NOT the insertion",
 "exports_sha256": {
  "2": "8ef4395c51434fe073109065e31773460c3708937a16ea29a036abf6c72c5030",
  "3": "183798063fa89839e9a80ae4a81e6c2f9c491a7ad386540d1a4e33382b0686de",
  "4": "d71d9a839b5e6388abe32ccb6bb334b87cca8e104f98a99ec198b923dde099fc"
 },
 "exports_identical_to_p23": true,
 "classical_certs_content_identical_to_p23": {
  "P13_quartic_verified": true,
  "P13_brst_derivation": true,
  "P13_master_kappa2": true,
  "P13_boundary_identity": true,
  "P13_gauge_fixing_map": true
 },
 "master_mutations": {
```

```
  "flip_sigma_c": {
   "claim_status": "Partial",
   "n_residuals": 6
  },
  "drop_cstar": {
   "claim_status": "Partial",
   "n_residuals": 6
  },
  "alter_gf_shift": {
   "claim_status": "Partial",
   "n_residuals": 2
  },
  "flip_odd_derivative_sign": {
   "claim_status": "Partial",
   "n_residuals": 6
  },
  "corrupt_a2": {
   "claim_status": "Partial",
   "n_residuals": 1
  },
  "corrupt_ell4": {
   "claim_status": "Partial",
   "n_residuals": 1
  },
  "drop_s1_transport": {
   "claim_status": "Partial",
   "n_residuals": 2
  }
 },
 "W_terms": {
  "kappa0": 132,
  "kappa1": 2148,
  "kappa2": 15428
 }
}
```


intake/errata_p23.json


```
{
 "source": "XVI-P p23 independent review (provenance/XVI-P/independent_rebuild_report_p23.json, verdict PASS (reproduction) with findings)",
 "errata": [
  {
   "id": "E1",
   "severity": "medium",
   "item": "P23_O1_scope O.5 line 'all E=2 kappa^2 graphs evaluated'",
   "correction": "OVERCLAIM vs XVI-O bulk O.5: D04 (phi phi bubble) and D05a (phi phi grav tadpole) never evaluated (legitimately outside the ch
projection by phi -> -phi parity), D06 only as the ghost self-energy inside P15_D13; D12 Missing. Bulk O.5 is open; carried to Q02."
  },
  {
   "id": "E2",
   "severity": "low",
   "item": "P23 reason for O.1 not closable ('grav ell_4 Incomplete')",
   "correction": "stale/incomplete: XVI-P ell_4 (3360 terms) reproduces XVI-O's conformal probe exactly (blocker [0]); remaining XVI-O blockers were
[1] explicit O(kappa^2) antibracket expansion and the ghost/antifield sector -- addressed in Q01 O.2 (b),(c)."
  },
  {
   "id": "E3",
   "severity": "low",
   "item": "d guard bypasses",
   "correction": "sympy.polys.rem namespace; polytools.rem captured before dred import; sp.reduced; sp.groebner; Matrix.subs d->Dummy then Dummy->4;
Poly(...).rep.eval -- fixed in the XVI-Q guard (entry-point rule at the sympy.polys construction level)."
  },
  {
   "id": "E4",
   "severity": "low",
   "item": "p23 gate",
   "correction": "misses per-graph h.h counterterm edits and P6_entry_n_graphs edits -- the XVI-Q gate value-checks every certified number it imports.
Also: p23 accepted p23_conventions.py UNPINNED (its sha was not in P23_PINNED_SHA256; README/REBUILD likewise) -- the XVI-Q gate pins every
conventions/expected-values file it relies on."
  },
  {
   "id": "E5",
   "severity": "low",
   "item": "P.6 entry kappa^3 inventory",
   "correction": "omits scaleless 5-leg-vertex tadpoles (zero in DRED but must be listed, never Missing->zero); recorded for the P.6 continuation (out
of Q01 scope)."
  },
  {
   "id": "E6",
   "severity": "low",
   "item": "P23_O1_scope O.5 evidence 'D12 Missing never zero per P14'",
   "correction": "stale p14r status: D12 was evaluated in XVI-P p15 (vanishing_under_declared_prescription, derived; two routes, IR-mass and cutoff
analysis) and independently confirmed in XVI-Q Q02; XVI-O never labelled D12 'Missing' (provisional_listed, unevaluated)."
```

```
  },
  {
   "id": "E7",
   "severity": "low",
   "item": "XVI-O diagram_inventory D04 sym_factor '1/2' (Unverified)",
   "correction": "D04 has two internal lines of different species and no automorphism fixing the external legs: symmetry factor 1 (used in Q02;
irrelevant to the value since D04 = 0 exactly)."
  }
 ],
 "revisions": {
  "Q02": "E2 severity info->low; E4 extended with the unpinned p23_conventions.py note (Q01 independent review)."
 }
}
```

registration/local_registration.json

```
{
 "landmark": "XVI-Q",
 "series": "XVI",
 "registered": "locally (this file only); NOT published, NOT pushed; no GitHub edit",
 "how_found": "No local series registry exists in /workspace: enter-companion/ is an unversioned scraped snapshot of the published catalogue (its
catalogue lists XVI-A..XVI-I; not a git checkout) and XVI-J..XVI-P are not listed anywhere locally. Registration state is package-local: XVI-P records
'XVI_P_registered': false in its RELEASE_STATUS; XVI-O has no registration flag at all. XVI-Q is therefore registered as a self-contained local record
plus hash-pinned predecessor imports; enter-companion untouched.",
 "decision": "user (2026-10-10): register a new landmark XVI-Q that imports XVI-P's verified ell_4 to discharge O.2, states the bulk identity, and
attempts O.1 closure",
 "predecessors_read_only": {
  "XVI-O": "o70-sync (imported files pinned in intake/imports.json)",
  "XVI-N": "XVI-N.tex + r10 status (pinned)",
  "XVI-P": "p23, source zip 3f4a88db462923553272e8a41407c0c06aacbc431736efdfeda10c27aae8a177 (frozen)"
 },
 "release_plan": {
  "Q01": "intake + O.2 classical kappa^2 master (a)(b)(c) + guard + value-checking gate",
  "Q02": "bulk O.5 graphs D04/D05a/D06, bulk identity ST4a/ST4b, O.1 attempt"
 },
 "hard_stops": {
  "O1_closure": "NOT_CLOSED unless every condition earned",
  "bulk_symmetry_restored": false,
  "insertion": "frozen (XVI-N thm:massless-obstruction)",
  "XV_B_opened": false,
  "XIV_J_opened": false,
  "Route_B_discharged": false,
  "XVI_M_pin": "unchanged",
  "Missing_never_zero": true
 },
 "corrections": {
  "Q02": "two Q01 inaccuracies fixed (independent review Q01): the catalogue lists XVI-A..I (not 'A,B,D,H,I only'); XVI-O has no registration flag
(Q01 wrongly cited 'XVI-O RELEASE_STATUS')."
 }
}
```

20 Appendix: source listings

code/p6q05/ghost_eq3.py

```
#!/usr/bin/env python3
"""XVI-Q Q05 diagnostic: three-point ghost equation (G04+G09) vs (G03+G07)[K -> T(cbar)], two-point D06 vs D13[K->T]."""
import json, sys  # noqa: E401
from pathlib import Path
import sympy as sp
HERE = Path(__file__).resolve().parent
sys.path.insert(0, str(HERE))
import rows as R, uvpole as U, tensor as TT, graphs as GR  # noqa: E401,E402
k = U.V({"p2": 1})   # untracked by the Taylor parameter t (G03/G07 extract t^3)
gb = U.vec("gb2")
Tm = TT.sym(k, gb)
Tm.eta = -sp.Rational(1, 2) * U.dot(k, gb)
orig = GR.ext
GR.ext = lambda a: Tm if a == "aK" else orig(a)
g3, g7 = R.strip_t(GR.G03()), R.strip_t(GR.G07())
GR.ext = orig
g4, g9 = R.strip_t(R.load("G04")), R.strip_t(R.load("G09"))
out = {"ratio_total": str(sp.cancel((g4 + g9) / (g3 + g7))) if (g3 + g7) != 0 else "den0",
       "ratio_G04_G03T": str(sp.cancel(g4 / g3)) if g3 != 0 else "den0",
       "ratio_G09_G07T": str(sp.cancel(g9 / g7)) if g7 != 0 else "den0"}
(HERE.parents[1] / "build/logs/q05_ghost_eq3.json").write_text(json.dumps(out, indent=1) + "\n")
print(out)
```

code/p6q05/graphs.py

```
#!/usr/bin/env python3
"""XVI-Q Q05 P.6 graph UV poles with own rules (gravity.py, validated vs XVI-P D03/D06/D13 (+D01 for V3)).
External legs: leg 1 momentum p1, leg 2 p2, leg 3 p3 = -p1-p2 (all incoming). Gravitons pol a_i (rank-1 sym), ghost c
pol 'gc_i', cbar pol 'gb_i', K pol 'aK' (rank-1 sym), c* pol 'z'. Pole in units of U (as Q04), MS-bar residue at d->4
with hat/tilde kept (T_* = evanescent).  Grassmann orientation: open ghost lines run from the external c to the
external cbar/K/c*; ghost loops carry -1.  usage: graphs.py GID -> build/logs/q05_GID.json"""
from __future__ import annotations
import json, sys, time  # noqa: E401
from pathlib import Path
import sympy as sp

HERE = Path(__file__).resolve().parent
Q = HERE.parents[1]
sys.path.insert(0, str(HERE))
import gravity as G  # noqa: E402
import uvpole as U  # noqa: E402
import tensor as TT  # noqa: E402
import dred  # noqa: E402

GH = 1 / (-sp.I)                    # ghost line numerator factor relative to i/k^2 (validated on D03/D06)
p1, p2, l = U.vec("p1"), U.vec("p2"), U.vec("l")
p3 = -(p1 + p2)


def ext(a):
    return TT.sym(U.vec(a), U.vec(a))


def r4(e, g):
    return sp.expand(dred.substitute_d(e, 4, f"Q05 {g}: MS-bar residue"))


def pole(num, shifts, deg, sym, g):
    return r4(U.loop_pole(num, [U.V()] + shifts[1:] if not shifts else shifts, deg, sym), g)


def G08():
    """ghost triangle, external h1 h2 h3, both orientations, ghost loop -1."""
    tot = 0
    for legs in ((("a1", p1), ("a2", p2), ("a3", p3)), (("a1", p1), ("a3", p3), ("a2", p2))):
        (A, kA), (B, kB), (C, kC) = legs
        vA = G.V_cb_c_h(G.probe_vec("xAb"), G.probe_vec("xAc"), l, ext(A), kA)
        vB = G.V_cb_c_h(G.probe_vec("xBb"), G.probe_vec("xBc"), l + kA, ext(B), kB)
        vC = G.V_cb_c_h(G.probe_vec("xCb"), G.probe_vec("xCc"), l + kA + kB, ext(C), kC)
        n = G.ghost_line(sp.expand(vA * vB), "xAb", "xBc", 1 / GH)
        n = G.ghost_line(sp.expand(n * vC), "xBb", "xCc", 1 / GH)
        n = G.ghost_line(n, "xCb", "xAc", 1 / GH)
        tot += U.loop_pole(-n, [U.V(), kA, kA + kB], 4, 1)
    return r4(tot, "G08")


def G12():
    """h(p1) at A (two phi), phi(p2) at B, phi(p3) at C; graviton C->B carries l."""
    vA = G.V_hphiphi(ext("a1"), l + p2, p3 - l)
    vB = G.V_hphiphi(G.probe_mat("hB"), p2, -l - p2)
    vC = G.V_hphiphi(G.probe_mat("hC"), p3, l - p3)
    n = G.graviton_line(sp.expand(vB * vC), "hB", "hC", 1)
    return r4(U.loop_pole(sp.expand(n * vA), [U.V(), p2, -p3], 4, 1), "G12")


def G13():
    """V_h_phi_phi (internal h, ext phi, internal phi) + V_hh_phi_phi (ext h, internal h, ext phi, internal phi);
    both placements of the two external phi."""
    tot = 0
    for pa, pb in ((p2, p3), (p3, p2)):
        vA = G.V_hphiphi(G.probe_mat("hA"), pa, -l - pa)
        vB = G.V_hhphiphi(ext("a1"), G.probe_mat("hB"), pb, l + pa)
        n = G.graviton_line(sp.expand(vA * vB), "hA", "hB", 1)
        tot += U.loop_pole(n, [U.V(), pa], 4, 1)
    return r4(tot, "G13")


def G11():
    """ext c(p1) -> V_cb_c_h (internal h, internal cbar) ; V_h_phi_phi (internal h, ext phi(p2), internal phi) ;
    V_phis_c_phi (ext phi*(p3), internal c, internal phi).  Graviton V1->V2 carries l (incoming at V2)."""
    gc = U.vec("gc1")
    v1 = G.V_cb_c_h(G.probe_vec("xb"), gc, p1, G.probe_mat("h1"), -l)           # cbar out: p1 - l
    v2 = G.V_hphiphi(G.probe_mat("h2"), p2, -l - p2)                              # phi out to V3: l + p2
    v3 = G.V_phis_c_phi(G.probe_vec("xc"), l + p2)                                 # c in: p1 - l ; phi in: l + p2
    n = G.graviton_line(sp.expand(v1 * v2), "h1", "h2", 1)
    n = G.ghost_line(sp.expand(n * v3), "xb", "xc", 1 / GH)
    return r4(U.loop_pole(n, [U.V(), p2, -p1], 3, 1), "G11")
```

```
def G11_swapped():
    """G11 with the external phi at p3 and phi* at p2 (same graph, legs exchanged)."""
    gc = U.vec("gc1")
    v1 = G.V_cb_c_h(G.probe_vec("xb"), gc, p1, G.probe_mat("h1"), -l)
    v2 = G.V_hphiphi(G.probe_mat("h2"), p3, -l - p3)
    v3 = G.V_phis_c_phi(G.probe_vec("xc"), l + p3)
    n = G.contract_vertices(v1, v2, [("h", "h1", "h2")])
    n = G.ghost_line(sp.expand(n * v3), "xb", "xc", 1 / GH)
    return r4(U.loop_pole(n, [U.V(), p3, -p1], 3, 1), "G11 swapped")


def G09():
    """ext c(p1) at A, ext cbar(p2) at C, ext h(p3) at B (on the ghost line); internal graviton A--C."""
    gc, gb = U.vec("gc1"), U.vec("gb2")
    vA = G.V_cb_c_h(G.probe_vec("xAb"), gc, p1, G.probe_mat("hA"), l)              # cbar out: p1 + l
    vB = G.V_cb_c_h(G.probe_vec("xBb"), G.probe_vec("xBc"), p1 + l, ext("a3"), p3)  # out: p1 + l + p3
    vC = G.V_cb_c_h(gb, G.probe_vec("xCc"), p1 + l + p3, G.probe_mat("hC"), -l)
    n = G.ghost_line(sp.expand(vA * vB), "xAb", "xBc", 1 / GH)
    n = G.ghost_line(sp.expand(n * vC), "xBb", "xCc", 1 / GH)
    n = G.graviton_line(n, "hA", "hC", 1)
    return r4(U.loop_pole(n, [U.V(), p1, p1 + p3], 4, 1), "G09")


def G07():
    """ext c(p1) at A, ext h(p3) at B (ghost line), ext K(p2) at C (V_K_c_h); internal graviton A--C."""
    gc = U.vec("gc1")
    vA = G.V_cb_c_h(G.probe_vec("xAb"), gc, p1, G.probe_mat("hA"), l)
    vB = G.V_cb_c_h(G.probe_vec("xBb"), G.probe_vec("xBc"), p1 + l, ext("a3"), p3)
    vC = G.V_K_c_h(ext("aK"), G.probe_vec("xCc"), p1 + l + p3, G.probe_mat("hC"), -l)
    n = G.ghost_line(sp.expand(vA * vB), "xAb", "xBc", 1 / GH)
    n = G.ghost_line(sp.expand(n * vC), "xBb", "xCc", 1 / GH)
    n = G.graviton_line(n, "hA", "hC", 1)
    return r4(U.loop_pole(n, [U.V(), p1, p1 + p3], 3, 1), "G07")


def G10():
    """ext c(p1) at A, ext c(p2) at B (each V_cb_c_h), internal graviton A--B, both internal ghosts end at
    V_cs_c_c (ext c*(p3)).  Ordering c*(z) c_A c_B with c_A the line from A."""
    vA = G.V_cb_c_h(G.probe_vec("xAb"), U.vec("gc1"), p1, G.probe_mat("hA"), l)     # out: p1 + l
    vB = G.V_cb_c_h(G.probe_vec("xBb"), U.vec("gc2"), p2, G.probe_mat("hB"), -l)    # out: p2 - l
    vC = G.V_cs_c_c(U.vec("z"), G.probe_vec("xCa"), p1 + l, G.probe_vec("xCb"), p2 - l)
    n = G.graviton_line(sp.expand(vA * vB), "hA", "hB", 1)
    n = G.ghost_line(sp.expand(n * vC), "xAb", "xCa", 1 / GH)
    n = G.ghost_line(n, "xBb", "xCb", 1 / GH)
    return r4(U.loop_pole(n, [U.V(), p1, -p2], 3, 1), "G10")


def _v3(a_ext, k_ext, pa, ka, pb, kb):
    return G.V3_grav((ext(a_ext), k_ext), (G.probe_mat(pa), ka), (G.probe_mat(pb), kb))


def G05():
    """V3 (ext h p1) -- V_h_phi_phi (ext phi p2) -- V_h_phi_phi (ext phi p3); phi line C->B carries l."""
    A = _v3("a1", p1, "hAB", l + p2, "hAC", p3 - l)
    B = G.V_hphiphi(G.probe_mat("hB"), p2, l)
    C = G.V_hphiphi(G.probe_mat("hC"), p3, -l)
    n = G.contract_vertices(A, B, [("h", "hAB", "hB")])
    n = G.contract_vertices(n, C, [("h", "hAC", "hC")])
    return r4(U.loop_pole(n, [U.V(), p2, -p3], 4, 1), "G05")


def G06():
    """V3 (ext h p1) -- V_hh_phi_phi (ext phi p2, phi p3); two graviton lines, symmetry 1/2."""
    A = _v3("a1", p1, "hA1", -l, "hA2", l - p1)
    B = G.V_hhphiphi(G.probe_mat("hB1"), G.probe_mat("hB2"), p2, p3)
    n = G.contract_vertices(A, B, [("h", "hA1", "hB1"), ("h", "hA2", "hB2")])
    return r4(U.loop_pole(n, [U.V(), -p1], 4, sp.Rational(1, 2)), "G06")


def _g34(kslot):
    """V3 (ext h p3) ; B = V_cb_c_h (ext c p1) ; C = V_cb_c_h (ext cbar p2) or V_K_c_h (ext K p2)."""
    A = _v3("a3", p3, "hAB", -l, "hAC", l - p3)
    B = G.V_cb_c_h(G.probe_vec("xb"), U.vec("gc1"), p1, G.probe_mat("hB"), l)
    if kslot:
        C = G.V_K_c_h(ext("aK"), G.probe_vec("xc"), p1 + l, G.probe_mat("hC"), p3 - l)
    else:
        C = G.V_cb_c_h(U.vec("gb2"), G.probe_vec("xc"), p1 + l, G.probe_mat("hC"), p3 - l)
    n = G.contract_vertices(B, C, [("c", "xb", "xc", GH)])
    n = G.contract_vertices(A, n, [("h", "hAB", "hB"), ("h", "hAC", "hC")])
    return n


def G04():
```

```
    return r4(U.loop_pole(_g34(False), [U.V(), p1, -p3], 4, 1), "G04")


def G03():
    return r4(U.loop_pole(_g34(True), [U.V(), p1, -p3], 3, 1), "G03")


def G01():
    """V3^3 triangle, one orientation, symmetry 1."""
    A = _v3("a1", p1, "hAB", -(l + p1), "hAC", l)
    B = _v3("a2", p2, "hBA", l + p1, "hBC", -(l + p1 + p2))
    C = _v3("a3", p3, "hCB", l + p1 + p2, "hCA", -l)
    n = G.contract_vertices(A, B, [("h", "hAB", "hBA")])
    n = G.contract_vertices(n, C, [("h", "hBC", "hCB"), ("h", "hAC", "hCA")])
    return r4(U.loop_pole(n, [U.V(), p1, p1 + p2], 4, 1), "G01")


if __name__ == "__main__":
    g = sys.argv[1]
    t0 = time.time()
    res = globals()[g]()
    out = {"graph": "P6-" + g, "pole_over_U_d4_residue": str(res), "n_terms": len(sp.Add.make_args(res)),
           "zero": res == 0, "seconds": round(time.time() - t0, 1)}
    (Q / "build" / "logs" / f"q05_{g}.json").write_text(json.dumps(out, indent=1) + "\n")
    print(g, out["n_terms"], out["zero"], out["seconds"])
```

code/p6q05/gravity.py

```
#!/usr/bin/env python3
"""XVI-Q Q05 own gravity/ghost Feynman rules from the XVI-P declared action (conventions.py, read-only):
  S_EH = -(2/kappa^2) int L_GG,  L_GG = sqrt(-g) g^{mn}(G^a_{mb} G^b_{na} - G^a_{mn} G^b_{ab}),  g = eta + kappa h (+---)
  S_gf (b eliminated, XVI-P P13 'b(2D-b): eliminating b gives +D_mu D^mu'):  int D_mu[h] D^mu[h],
        D^mu[h] = d_nu h^{mu nu} - (1/2) d^mu tr h
  S_gh  = int i cbar_mu D^mu[ (d c + d c) + kappa L_c h ]   (XVI-P: kinetic 'i Box', vertex '2 i a D[L_c h]', a = 1/2)
  S_K   = int K^{mn} (1/2) [ (d_m c_n + d_n c_m) + kappa (L_c h)_{mn} ]   (K s h, s h = (1/2) L_c(eta + kappa h))
  scalar: (1/2) sqrt(-g) g^{mn} d phi d phi.
Plane waves: field_i = pol_i exp(-i k_i x) (all momenta incoming), d -> -i k.  A vertex = i x (multilinear
coefficient of the action).  kappa powers are tracked by the caller (vertex functions are at kappa = 1).
Internal lines are closed by PROBES: the two ends of a line get probe polarisations; the product of vertices is
contracted with the propagator numerator (graviton: i/(Z k^2) [ (eta eta + eta eta)/2 - (1/2) eta eta ], Q4S indices;
ghost: <c cbar> = i K^{-1}; scalar: i/k^2).  Normalisations Z etc. are DERIVED in derive_propagators()."""
from __future__ import annotations
import itertools, re, sys  # noqa: E401
from pathlib import Path
import sympy as sp

sys.path.insert(0, str(Path(__file__).resolve().parent))
sys.path.insert(0, str(Path(__file__).resolve().parents[1] / "q04"))
import uvpole as U  # noqa: E402
import tensor as TT  # noqa: E402
from tensor import Mat, ETA, ev  # noqa: E402

I = sp.I
_lab = itertools.count()


def L():
    return f"i{next(_lab)}"


# ---------------------------------------------------------------------- Christoffel (first kind), one plane wave
def gamma_terms(E, k, c, m, b):
    """Gamma_{c,mb} = 1/2 (d_m h_cb + d_b h_cm - d_c h_mb), d -> -i k: list of (coef, factors)."""
    h = -I / 2
    return [(h, [("v", k, m), ("m", E, c, b)]), (h, [("v", k, b), ("m", E, c, m)]), (-h, [("v", k, c), ("m", E, m, b)])]


def _mul(t1, t2):
    return [(a * b, fa + fb) for a, fa in t1 for b, fb in t2]


def prefactor_terms(order_wave, m, n, a, c, b, d):
    """sqrt(-g) g^{mn} g^{ac} g^{bd}: order 0 (order_wave None) or first order in wave (E, k)."""
    if order_wave is None:
        return [(1, [("m", ETA(), m, n), ("m", ETA(), a, c), ("m", ETA(), b, d)])]
    E = order_wave
    trE = E.tr()
    return [(trE / 2, [("m", ETA(), m, n), ("m", ETA(), a, c), ("m", ETA(), b, d)]),
            (-1, [("m", E, m, n), ("m", ETA(), a, c), ("m", ETA(), b, d)]),
            (-1, [("m", ETA(), m, n), ("m", E, a, c), ("m", ETA(), b, d)]),
            (-1, [("m", ETA(), m, n), ("m", ETA(), a, c), ("m", E, b, d)])]
```

```
def LGG_piece(pref_wave, w1, w2):
    """L_GG with the prefactor at order 0/1 (pref_wave) and Gamma's from waves w1, w2 (each (E, k))."""
    m, n, a, b, c, d = (L() for _ in range(6))
    P = prefactor_terms(pref_wave, m, n, a, c, b, d)
    t1 = _mul(_mul(P, gamma_terms(w1[0], w1[1], c, m, b)), gamma_terms(w2[0], w2[1], d, n, a))
    m2, n2, a2, b2, c2, d2 = (L() for _ in range(6))
    P2 = prefactor_terms(pref_wave, m2, n2, a2, c2, b2, d2)
    t2 = _mul(_mul(P2, gamma_terms(w1[0], w1[1], c2, m2, n2)), gamma_terms(w2[0], w2[1], d2, a2, b2))
    return ev(*t1) - ev(*t2)


def LGG2(w1, w2):
    """multilinear coefficient of t1 t2 in L_GG (both orders of the two Gammas)."""
    return sp.expand(LGG_piece(None, w1, w2) + LGG_piece(None, w2, w1))


def LGG3(w1, w2, w3):
    ws = [w1, w2, w3]
    tot = 0
    for p in itertools.permutations(range(3)):
        tot += LGG_piece(ws[p[0]][0], ws[p[1]], ws[p[2]])
    return sp.expand(tot)


def V3_grav(w1, w2, w3):
    """i x delta^3 S_EH (kappa^1 stripped): S_EH cubic = -2 kappa L_GG^(3)."""
    return sp.expand(I * (-2) * LGG3(w1, w2, w3))


def Dvec(E, k):
    """D^mu[h] for one wave as a vector: -i (E.k - (1/2) k tr E)."""
    return (-I) * (E.left(k) + (-sp.Rational(1, 2) * E.tr()) * k)


def S2_EH_gf(E1, E2, k):
    """delta^2 (S_EH + S_gf) with wave 1 (E1, k), wave 2 (E2, -k), kappa^0."""
    eh = -2 * LGG2((E1, k), (E2, -k))
    gf = 2 * U.dot(Dvec(E1, k), Dvec(E2, -k))
    return sp.expand(eh), sp.expand(gf)


# ---------------------------------------------------------------------- scalar vertices on Mat polarisations
def X1(E, x, y):
    return sp.expand(sp.Rational(1, 2) * E.tr() * U.dot(x, y) - U.dot(E.left(x), y))


def X2cross(E1, E2, x, y):
    i1, i2 = L(), L()
    pc = lambda A, B: ev((1, [("v", x, i1), ("m", A, i1, i2), ("m", B, i2, L_ := L()), ("v", y, L_)]))  # noqa: E731
    return sp.expand(pc(E1, E2) + pc(E2, E1) - sp.Rational(1, 2) * (E1.tr() * U.dot(E2.left(x), y) + E2.tr() * U.dot(E1.left(x), y))
                     + (sp.Rational(1, 4) * E1.tr() * E2.tr() - sp.Rational(1, 2) * ev((1, [("m", E1, i1, i2), ("m", E2, i2, i1)]))) * U.dot(x, y))


def V_hphiphi(E, k1, k2):
    return -I * X1(E, k1, k2)


def V_hhphiphi(E1, E2, k1, k2):
    return -I * X2cross(E1, E2, k1, k2)


# ---------------------------------------------------------------------- ghost / antifield vertices
def Lc_h(E, k, cvec, q):
    """(L_c h)_{mn} as a Mat for wave (E, k) and ghost polarisation cvec with momentum q:
    c^r d_r h_mn + h_rn d_m c^r + h_mr d_n c^r,  d -> -i (k on h, q on c)."""
    out = Mat(0, [(c0 * (-I) * U.dot(cvec, k), u, w) for c0, u, w in E.items])
    if E.eta != 0:
        out.eta = E.eta * (-I) * U.dot(cvec, k)
    Ec = E.left(cvec)                                   # (c.E)_n  (E symmetric)
    out.items += [(-I, q, Ec), (-I, Ec, q)]
    return out


def V_cb_c_h(cbv, cv, q, E, k):
    """i x coefficient of cbar(cbv) c(cv, q) h(E, k) in S_gh (kappa^1): i * i cbar.D[L_c h] with P = q + k."""
    M = Lc_h(E, k, cv, q)
    P = q + k
    D = (-I) * (M.left(P) + (-sp.Rational(1, 2) * M.tr()) * P)
    return sp.expand(I * I * U.dot(cbv, D))


def V_K_c_h(KE, cv, q, E, k):
    """i x coefficient of K(KE) c(cv, q) h(E, k) in S_K: i (1/2) K^{mn} (L_c h)_{mn} (K a symmetric 'polarisation' Mat)."""
    M = Lc_h(E, k, cv, q)
```

```
    i1, i2 = L(), L()
    return sp.expand(I * sp.Rational(1, 2) * ev((1, [("m", KE, i1, i2), ("m", M, i2, i1)])))


# ---------------------------------------------------------------- probes and line contraction
def probe_mat(name):
    """symmetric graviton probe sym(nameU, nameW)."""
    return TT.sym(U.V({name + "U": 1}), U.V({name + "W": 1}))


def probe_vec(name):
    return U.V({name: 1})


_SYM = re.compile(r"^(H|T|s)_(.+)$")


def _parse_dot(sym, probes):
    """symbol -> (kind, (a, b)) with a, b basis names; s_x == l.x."""
    m = _SYM.match(sym.name)
    if not m:
        return None
    kind, rest = m.groups()
    if kind == "s":
        a, b = "l", rest
    else:
        a, b = rest.split("_")
    if a in probes or b in probes:
        return kind, a, b
    return None


def contract(expr, pairs):
    """contract probe vectors pairwise with the 4S metric: pairs = [(p, q)] meaning sum_mu p^mu q_mu over a basis.
    Graviton lines use probes (XU, XW) and (YU, YW) with the propagator numerator applied by the caller as pairs.
    Implementation: every monomial is linear in each probe; probe dots are rewritten as index factors and evaluated."""
    probes = {p for pr in pairs for p in pr}
    expr = sp.expand(expr)
    syms = [s for s in expr.free_symbols if (_parse_dot(s, probes) or (None,))[0] == "T"]
    expr = sp.expand(expr.xreplace({s: 0 for s in syms}))   # 4S dots appear as H + T: keep the H copy as the full dot
    lab = {}
    for p, q in pairs:
        lab[p] = lab[q] = L()
    out = 0
    for term in sp.Add.make_args(expr):
        coef = term
        facs = []
        seen = {}
        for s, e in term.as_powers_dict().items():
            pd = _parse_dot(s, probes) if isinstance(s, sp.Symbol) else None
            if pd is None:
                continue
            _, a, b = pd
            for _ in range(int(e)):
                coef = coef / s
                ina, inb = a in probes, b in probes
                if ina and inb:
                    facs.append(("m", ETA(), lab[a], lab[b]))
                elif ina:
                    facs.append(("v", U.V({b: 1}), lab[a]))
                else:
                    facs.append(("v", U.V({a: 1}), lab[b]))
        out += TT.evaluate((coef, facs))
    return sp.expand(out)


def graviton_line(expr, A, B, Z, c=sp.Rational(1, 2)):
    """close a graviton line between probe A (at one vertex) and probe B (other vertex), momentum-independent
    numerator only: [ (eta_ac eta_bd + eta_ad eta_bc)/2 - c eta_ab eta_cd ] / Z ; the i/k^2 is put in by the caller."""
    t1 = contract(expr, [(A + "U", B + "U"), (A + "W", B + "W")])
    t2 = contract(expr, [(A + "U", B + "W"), (A + "W", B + "U")])
    t3 = contract(expr, [(A + "U", A + "W"), (B + "U", B + "W")])
    return sp.expand((sp.Rational(1, 2) * (t1 + t2) - c * t3) / Z)


def ghost_line(expr, A, B, Zg=1):
    return sp.expand(contract(expr, [(A, B)]) / Zg)


# ---------------------------------------------------------------- antifield vertices for s c, s phi (XVI-P P13 s_images)
def V_cs_c_c(zv, c1, q1, c2, q2):
    """i x coefficient of c*(zv) c(c1, q1) c(c2, q2) in int c*_mu (kappa/2) c^r d_r c^mu  (ordering c* c1 c2;
    the c1 <-> c2 exchange carries the Grassmann sign -1)."""
    t = sp.Rational(1, 2) * ((-I) * U.dot(c1, q2) * U.dot(zv, c2) - (-I) * U.dot(c2, q1) * U.dot(zv, c1))
```

```
    return sp.expand(I * t)


def V_phis_c_phi(cv, qphi):
    """i x coefficient of phi* c(cv) phi(qphi) in int phi* (kappa/2) c^r d_r phi."""
    return sp.expand(I * sp.Rational(1, 2) * (-I) * U.dot(cv, qphi))


# ---------------------------------------------------------------- fast path: contract two vertices pattern-wise
def _split_probe(expr, probes):
    """expr -> {probe monomial (H-copies only): coefficient}  (T-copies of probe dots dropped, as in contract())."""
    expr = sp.expand(expr)
    drop = [s for s in expr.free_symbols if (_parse_dot(s, probes) or (None,))[0] == "T"]
    expr = sp.expand(expr.xreplace({s: 0 for s in drop}))
    out = {}
    for term in sp.Add.make_args(expr):
        pat, coef = sp.Integer(1), sp.Integer(1)
        for f, e in term.as_powers_dict().items():
            if isinstance(f, sp.Symbol) and _parse_dot(f, probes):
                pat *= f ** e
            else:
                coef *= f ** e
        out[pat] = out.get(pat, 0) + coef
    return out


def contract_vertices(A, B, lines):
    """sum over internal lines between vertex expressions A and B.
    lines: list of ('h', probeA, probeB) graviton (numerator (I - J/2)) or ('c', probeA, probeB, factor) ghost/vector.
    Equivalent to graviton_line/ghost_line applied to sp.expand(A*B), but pattern-wise (memoised)."""
    probes = set()
    for ln in lines:
        if ln[0] == "h":
            probes |= {ln[1] + "U", ln[1] + "W", ln[2] + "U", ln[2] + "W"}
        else:
            probes |= {ln[1], ln[2]}
    SA, SB = _split_probe(A, probes), _split_probe(B, probes)
    cache = {}
    tot = 0
    for pa, ca in SA.items():
        for pb, cb in SB.items():
            key = pa * pb
            if key not in cache:
                v = key
                for ln in lines:
                    v = graviton_line(v, ln[1], ln[2], 1) if ln[0] == "h" else ghost_line(v, ln[1], ln[2], 1 / ln[3])
                cache[key] = v
            if cache[key] != 0:
                tot += cache[key] * ca * cb
    return sp.expand(tot)
```

code/p6q05/guard_scan2.py

```
#!/usr/bin/env python3
"""XVI-Q Q05 hardened static policy scan (Q04 independent review: 7 evasion forms; scan limited to code/q0*/*.py).
Scans EVERY .py under code/ (recursive).  Flags, outside the declared probe files:
  F1 any reference (call, alias import, attribute, bare name, or string constant) to an escape primitive
     (srepr, parse_expr, lambdify, dup_eval*, dmp_eval*, as_coefficients_dict, evalf, N) -- closes aliased imports
     and getattr(x, 'srepr')
  F2 getattr / __getattribute__ with a NON-literal attribute name (string concatenation)
  F3 .replace(...) whose receiver is not a string literal, a Path, or allow-listed -- closes str/f-string/format/__str__
     rename-and-reparse
  F4 rebinding of builtins str / repr / format / getattr
  F5 name-based symbol identification: comparisons on `.name` (tree walk recognising d by name)
CLAIM (honest, downgraded): best-effort POLICY scan of the package's own code.  It is NOT a security boundary: Python
is dynamic, so determined code can still evade it (e.g. exec of a computed string, importlib with computed names);
F6 flags exec/eval/compile/importlib/__import__ to raise the cost, but the guarantee is review + scan, not proof."""
from __future__ import annotations
import ast, json, sys  # noqa: E401
from pathlib import Path

Q = Path(__file__).resolve().parents[2]
PRIM = {"srepr", "parse_expr", "lambdify", "dup_eval", "dmp_eval", "dmp_eval_in", "dmp_eval_tail", "dup_eval_in",
        "as_coefficients_dict", "evalf", "N"}
BUILTINS = {"str", "repr", "format", "getattr"}
DYN = {"exec", "eval", "compile", "__import__", "import_module"}
PROBE_FILES = {"code/q04/guard_scan.py", "code/q01/guard_probe.py", "code/p6q05/guard_scan2.py"}
# .replace receivers that are provably plain text manipulation of non-expression data (file text / labels)
REPLACE_OK = {}
# reviewed exceptions (line-pinned; the gate pins file sha256, so a moved/edited line re-triggers review):
REVIEWED = {("code/q01/o2_q01.py", 34): "__import__('zipfile') for provenance zip hashing; no expression data",
            ("code/q01/qguard.py", 132): "getattr(klass, meth) inside the guard's own wrapper installation"}
# 'N' as a bare name is a common local (numerators); sympy.N is caught via import alias / attribute (.N) instead.
```

```
def files():
    return sorted(p for p in (Q / "code").rglob("*.py") if "__pycache__" not in p.parts)


def _hits(rel, tree):
    h = []
    for n in ast.walk(tree):
        if isinstance(n, (ast.Name,)) and n.id in (PRIM - {"N"}) | DYN and (rel, n.lineno) not in REVIEWED:
            h.append(f"{rel}:{n.lineno}: F1 name {n.id}")
        if isinstance(n, ast.Attribute) and n.attr in PRIM | {"import_module"}:
            h.append(f"{rel}:{n.lineno}: F1 attr .{n.attr}")
        if isinstance(n, ast.alias) and (n.name.split(".")[-1] in PRIM | DYN):
            h.append(f"{rel}:{getattr(n, 'lineno', 0)}: F1 import {n.name}")
        if isinstance(n, ast.Constant) and isinstance(n.value, str) and n.value in PRIM:
            h.append(f"{rel}:{n.lineno}: F1 string '{n.value}'")
        if isinstance(n, ast.Call):
            f = n.func
            nm = f.attr if isinstance(f, ast.Attribute) else f.id if isinstance(f, ast.Name) else None
            if nm in ("getattr", "__getattribute__") and len(n.args) >= 2 and not isinstance(n.args[1], ast.Constant) \
                    and (rel, n.lineno) not in REVIEWED:
                h.append(f"{rel}:{n.lineno}: F2 getattr non-literal")
            if nm == "replace" and isinstance(f, ast.Attribute):
                recv = f.value
                lit = isinstance(recv, ast.Constant)
                if not lit and (rel, n.lineno) not in REPLACE_OK:
                    h.append(f"{rel}:{n.lineno}: F3 .replace on non-literal")
        if isinstance(n, (ast.Assign, ast.AnnAssign, ast.AugAssign)):
            tg = n.targets if isinstance(n, ast.Assign) else [n.target]
            for t in tg:
                if isinstance(t, ast.Name) and t.id in BUILTINS:
                    h.append(f"{rel}:{n.lineno}: F4 rebinding {t.id}")
        if isinstance(n, (ast.FunctionDef, ast.arg)) and getattr(n, "name", getattr(n, "arg", None)) in BUILTINS:
            h.append(f"{rel}:{n.lineno}: F4 rebinding {getattr(n, 'name', getattr(n, 'arg', None))}")
        if isinstance(n, ast.Compare):
            for side in [n.left] + n.comparators:
                if isinstance(side, ast.Attribute) and side.attr == "name":
                    h.append(f"{rel}:{n.lineno}: F5 compare on .name")
    return h


def scan():
    out = []
    for p in files():
        rel = p.relative_to(Q).as_posix()
        if rel in PROBE_FILES:
            continue
        out += _hits(rel, ast.parse(p.read_text()))
    return out


EVASIONS = {   # the reviewer's 7 forms, each must be flagged
    "aliased_lambdify": "from sympy import lambdify as f\ng = f(x, e)\n",
    "getattr_concat_srepr": "g = getattr(sp, 'sr' + 'epr')(e)\n",
    "fstring_replace": "v = sp.sympify(f'{e}'.replace('d', '4'))\n",
    "dunder_str_replace": "v = sp.sympify(e.__str__().replace('d', '4'))\n",
    "format_replace": "v = sp.sympify(format(e).replace('d', '4'))\n",
    "rebound_str": "str = lambda x: x\nv = str(e)\n",
    "name_based_walk": "def w(x):\n    if x.name == 'd':\n        return 4\n",
}


def selftest():
    return {k: bool(_hits("<probe>", ast.parse(src))) for k, src in EVASIONS.items()}


def report():
    h = scan()
    st = selftest()
    return {"scope": "every .py under code/ (recursive), minus declared probe files", "probe_files": sorted(PROBE_FILES),
            "n_files_scanned": len(files()) - len([p for p in files() if p.relative_to(Q).as_posix() in PROBE_FILES]),
            "rules": ["F1 primitive reference/alias/string", "F2 non-literal getattr", "F3 .replace on non-literal",
                      "F4 builtin rebinding", "F5 .name comparison", "F6 dynamic exec/eval/import"],
            "reviewer_evasions_flagged": st, "reviewed_exceptions": {f"{a}:{b}": v for (a, b), v in REVIEWED.items()},
"all_reviewer_evasions_flagged": all(st.values()),
            "scan_hits": h, "scan_clean": not h,
            "claim": "best-effort policy scan; NOT a security boundary (dynamic Python can still evade it)",
            "runtime_closable": False}


if __name__ == "__main__":
    r = report()
    print(json.dumps({k: r[k] for k in ("n_files_scanned", "all_reviewer_evasions_flagged", "scan_clean")}), *r["scan_hits"], sep="\n")
```

code/p6q05/q05_certs.py

```
#!/usr/bin/env python3
"""XVI-Q Q05 certificates (P.6 continued; honest partial):
  Q05_rules_validation.json  own gravity/ghost rules vs XVI-P D01/D03/D06/D13 (constant ratios), two-point ghost equation
  Q05_P6_graphs.json         UV poles of P6-G03..G13 (own rules); G01/G02 Missing
  Q05_P6_rows.json           kappa^3 identity rows at pole level: c.phi.phi (closed, + mutations); c.c.cbar / c.c.K
                             (tree calibrated; loop residual nonzero, UNRESOLVED); h.h.c Missing
  Q05_P6_status.json         outcome per stage, errata, disclosures
  Q05_guard_scan.json        hardened static policy scan (Q04 review)"""
from __future__ import annotations
import hashlib, json, sys, time  # noqa: E401
from concurrent.futures import ProcessPoolExecutor
from pathlib import Path

HERE = Path(__file__).resolve().parent
Q = HERE.parents[1]
CERT = Q / "certificates"
LOG = Q / "build" / "logs"
sys.path.insert(0, str(HERE))
GRAPHS = ["G03", "G04", "G05", "G06", "G07", "G08", "G09", "G10", "G11", "G12", "G13"]
DEG3 = {"G03", "G07", "G10", "G11"}


def task_val(name):
    import validate as V
    return name, {"D01": V.D01, "D03": V.D03, "D06": V.D06, "D13": V.D13}[name]()


def task_graph(g):
    import sympy as sp
    import graphs as GR
    t0 = time.time()
    r = {"G03": GR.G03, "G04": GR.G04, "G05": GR.G05, "G06": GR.G06, "G07": GR.G07, "G08": GR.G08, "G09": GR.G09,
         "G10": GR.G10, "G11": GR.G11, "G12": GR.G12, "G13": GR.G13}[g]()
    s = str(r)
    LOG.mkdir(parents=True, exist_ok=True)
    (LOG / f"q05_{g}.json").write_text(json.dumps({"graph": "P6-" + g, "pole_over_U_d4_residue": s}) + "\n")
    return g, {"pole_over_U_d4_residue": s, "sha256": hashlib.sha256(s.encode()).hexdigest(),
               "n_terms": len(sp.Add.make_args(r)), "zero": r == 0,
               "n_evanescent_terms": sum(1 for t in sp.Add.make_args(r) if any(x.name.startswith("T_") for x in t.free_symbols)),
               "momentum_degree": 3 if g in DEG3 else 4, "seconds": round(time.time() - t0, 1)}


def ghost_eq2():
    import sympy as sp
    import rows as R
    import uvpole as U
    import gravity as G
    import tensor as TT
    q, gb = U.vec("p1"), U.vec("gb")
    d06 = R.D06_own("gb", "X", q)
    d06 = R.strip_t(G.contract(sp.expand(d06 * U.dot(G.probe_vec("Y"), U.vec("gc"))), [("X", "Y")]))
    d13 = R.D13_own("gc", q)
    Tm = TT.sym(-q, gb)
    Tm.eta = -sp.Rational(1, 2) * U.dot(-q, gb)
    pair = sp.expand(TT.ev((1, [("m", G.probe_mat("E"), "i", "j"), ("m", Tm, "j", "i")])))
    d13T = R.strip_t(R.contract_id(d13 * sp.I, pair, "K", "E", "h"))
    return str(sp.cancel(d06 / d13T))


def task_ghost_eq3():
    import sympy as sp
    import rows as R
    import uvpole as U
    import tensor as TT
    import graphs as GR
    k, gb = U.V({"p2": 1}), U.vec("gb2")
    Tm = TT.sym(k, gb)
    Tm.eta = -sp.Rational(1, 2) * U.dot(k, gb)
    orig = GR.ext
    GR.ext = lambda a: Tm if a == "aK" else orig(a)
    try:
        g3, g7 = R.strip_t(GR.G03()), R.strip_t(GR.G07())
    finally:
        GR.ext = orig
    g4, g9 = R.strip_t(GR.G04()), R.strip_t(GR.G09())
    return {"ratio_G04_over_G03_K_to_T": str(sp.cancel(g4 / g3)), "ratio_G09_over_G07_K_to_T": str(sp.cancel(g9 / g7)),
            "ratio_total": str(sp.cancel((g4 + g9) / (g3 + g7))),
            "meaning": "ghost equation (Gamma depends on cbar only through K' = K + dPsi/dh): the cbar-c-h graphs equal "
                       "-2 x the K-c-h graphs with K -> T(cbar), graph by graph; same -2 as tree level and two-point"}


def rows_cert():
    import itertools
```

```
    import sympy as sp
    import rows as R
    sol, a, b = R.calibrate_cphiphi()
    sig = sol[0]
    main = R.row_cphiphi(sig)
    muts = {"flip_sigma": R.row_cphiphi(-sig)["zero"] is False}
    # drop channels / graphs
    t = R.row_cphiphi(sig)
    muts["drop_S1_Kc_channel"] = t["without_S1_Kc_zero"] is False
    for g in ("G05", "G06", "G12", "G13"):
        keep = [x for x in ("G05", "G06", "G12", "G13") if x != g]
        hpp = sum(R.load(x) for x in keep)
        th = R.strip_t(-sp.I * R.polarise(hpp, "a1", "p1", "gc1"))
        rr = sp.expand(th + R.strip_t(sp.sympify(t["S0_phistar"])) + R.strip_t(sp.sympify(t["S1_Kc"])))
        muts[f"drop_{g}"] = rr != 0
    g05 = R.strip_t(-sp.I * R.polarise(R.load("G05"), "a1", "p1", "gc1"))
    cpp = {"row": "c.phi.phi", "calibration_sigma_phistar": str(sig),
           "tree_identity_closed_with_sigma": sp.expand(a + sig * b) == 0,
           "channels": {k: main[k] for k in ("S0_hphiphi", "S0_phistar", "S1_Kc")}, "residual": main["residual"],
           "closed_at_pole_level": main["zero"], "mutations_detected": muts,
           "G05_s0_projection_zero": g05 == 0,
           "G05_note": "G05's pole is nonzero but its s0-projection vanishes identically, so the row cannot detect "
                       "dropping G05 (recorded, not a failure; G05 enters other projections)",
           "evanescent_note": "every T structure dies under the s0 polarisation (T(p1, c) = 0): the row must vanish "
                              "exactly; it does (no evanescent counterterm needed or available in this row)",
           "ledger_erratum": "Q04 ledger listed only Gamma_phi phi = 0 for (S1, Gamma2); the S1 h-phi-phi vertex x "
                             "Gamma_K c (D13) channel also enters and is REQUIRED (dropping it leaves a nonzero residual)"}
    ghost = {}
    for tg, sc in (("ccb", -1), ("ccK", 1)):
        tree = R.ghost_row(tg, loop=False)
        sig_tree = sp.cancel(-tree["h"] / tree["c"])
        ch, res = R.ghost_row(tg, loop=True, sig_c=sc)
        T = [x for x in res.free_symbols if x.name.startswith("T_")]
        a_ = sp.symbols("x0:4")
        e = sp.expand(ch["h"] * a_[0] + ch["K"] * a_[1] + ch["c1"] * a_[2] + ch["c2"] * a_[3])
        syms = [x for x in e.free_symbols if x not in a_]
        fit = sp.solve(sp.Poly(e, *syms).coeffs(), a_, dict=True)
        nontriv = [f for f in fit if any(v != 0 for v in f.values())]
        ghost["c.c.cbar" if tg == "ccb" else "c.c.K"] = {
            "tree_sigma_c": str(sig_tree), "tree_closes_with_constant_sigma": not sp.sympify(sig_tree).free_symbols,
            "loop_residual_n_terms": len(sp.Add.make_args(res)), "loop_residual_zero": res == 0,
            "loop_residual_4D_part_zero": sp.expand(res.xreplace({x: 0 for x in T})) == 0,
            "loop_residual_sha256": hashlib.sha256(str(res).encode()).hexdigest(),
            "channel_terms": {k: len(sp.Add.make_args(v)) for k, v in ch.items() if k != "c"},
            "any_channel_coefficient_combination_closes": bool(nontriv),
            "status": "OPEN (unresolved): the assembled one-loop row does not close and no rescaling of its four "
                      "channels closes it; NOT reported as an obstruction. Diagnosis so far: the three-point ghost "
                      "equation holds graph by graph (G04 = -2 G03[K->T], G09 = -2 G07[K->T]), so the cbar-c-h / K-c-h "
                      "graphs are mutually consistent; the defect lies in G10 (c* c c; one loop-shift bug already found "
                      "and fixed in Q05), the K / c channels, or the row assembly (Grassmann ordering of the two "
                      "identical c legs). Next step: independent check of G10 and of the c-channel."}
    return {"identity": "(S, Gamma_1^pole) restricted to three-field gh1 monomials (Q04 ledger rows)",
            "normalisation": "all amplitudes 'i x coefficient' / 'i Gamma'; antifield-channel signs calibrated on the "
                             "classical (S0, S1) = 0 (tree, kappa^1; O.2-verified master equation) and used unchanged",
            "c.phi.phi": cpp, "ghost_rows": ghost,
            "h.h.c": {"status": "Missing", "why": "needs G01 (V3^3; evaluation did not finish within the release "
                                                  "budget) and G02 (V4 not constructed)"},
            "others": "h.phi.c, phi.b.c, phi.c.cbs, c.c.phis: zero by phi-parity; aux rows: no loop entry (Q04 ledger)"}


def main():
    t0 = time.time()
    with ProcessPoolExecutor(max_workers=6) as ex:
        fg = [ex.submit(task_graph, g) for g in sorted(GRAPHS, key=lambda g: g not in ("G07", "G08", "G04", "G09"))]
        fv = [ex.submit(task_val, n) for n in ("D01", "D03", "D06", "D13")]
        fe = ex.submit(task_ghost_eq3)
        graphs = dict(f.result() for f in fg)
        vals = dict(f.result() for f in fv)
        geq3 = fe.result()
    val = {"validations": vals,
           "expected_ratios": {"D01": "I", "D03": "I", "D06": "I", "D13": "-I"},
           "all_constant": all(v["constant"] for v in vals.values()),
           "ratios_as_expected": all(vals[k]["ratio"] == r for k, r in {"D01": "I", "D03": "I", "D06": "I", "D13": "-I"}.items()),
           "convention_I": "mine/XVI-P = I is the Q04-established convention factor (own pole = i x XVI-P pole; D11 in Q04)",
           "D13_orientation_disclosure": "D13 (K'.c) matches XVI-P with constant ratio -I on all structures (T1, T2, T3, "
                                         "E2): an overall sign relative to the I convention, i.e. the Grassmann "
                                         "ordering/orientation of the K'.c slot differs between the two codes. Own "
                                         "rows use own D13 consistently; the c.phi.phi row (which uses own D13) closes "
                                         "exactly, so the own orientation is the one consistent with own vertices.",
           "ghost_equation_two_point": {"ratio_D06_over_D13_K_to_T": ghost_eq2(), "expected_constant": True,
                                        "T": "T_mn = sym(k, cbar) - (1/2) eta k.cbar (cbar momentum k)"},
           "ghost_equation_three_point": geq3,
           "propagators": {"graviton": "i/k^2 (I - J/2), Q4S trace 4; derived: EH quadratic gauge invariant, EH + XVI-P "
                                       "gf (+D.D) = k^2 (I - J/2) (XVI-P K = -Box P)", "ghost": "-1/k^2 (from i cbar Box c)"}}
```

```
    (CERT / "Q05_rules_validation.json").write_text(json.dumps(val, indent=1) + "\n")
    gr = {"graphs": graphs, "missing": {"G01": "V3^3 triangle: implemented (graphs.G01), evaluation did not finish within "
                                               "the release budget -> Missing (never zero)",
                                        "G02": "needs V4_grav (not constructed in Q05) -> Missing"},
          "carried_from_Q04": {"G14": "evaluated (Q04)", "G15": "evaluated (Q04)", "T1-T3": "derived 0 (scaleless)"},
          "convention": "pole residue at d -> 4 in units of U, hat/tilde kept (T_* evanescent); external momenta p1, p2, "
                        "p3 = -p1-p2; graph-level conventions in code/p6q05/graphs.py docstrings"}
    (CERT / "Q05_P6_graphs.json").write_text(json.dumps(gr, indent=1) + "\n")
    rw = rows_cert()
    (CERT / "Q05_P6_rows.json").write_text(json.dumps(rw, indent=1) + "\n")
    ok_rules = val["all_constant"] and val["ratios_as_expected"]
    cpp_ok = rw["c.phi.phi"]["closed_at_pole_level"] and rw["c.phi.phi"]["G05_s0_projection_zero"] and \
        all(v for k, v in rw["c.phi.phi"]["mutations_detected"].items() if k != "drop_G05")
    st = {"landmark": "XVI-Q", "release": "Q05", "P6_outcome": "Partial",
          "stages": {"rules_V3_ghost_validated": ok_rules,
                     "graphs_evaluated_pole": sorted(graphs) + ["G14", "G15"], "graphs_missing": ["G01", "G02"],
                     "graphs_derived_zero": ["T1", "T2", "T3"],
                     "row_c_phi_phi": "closed at pole level" if cpp_ok else "NOT closed",
                     "row_c_c_cbar": "OPEN (unresolved assembly)", "row_c_c_K": "OPEN (unresolved assembly)",
                     "row_h_h_c": "Missing"},
          "counterterms": {"h_phi_phi + phi*_c_phi sector": "MS-bar CT = -(pole) of G05+G06+G12+G13 and G11 (local; "
                           "Q(i) coefficients as certified); its sigma_0 / (S, CT) consistency in row c.phi.phi is "
                           "exactly the closed row above" if cpp_ok else "Missing",
                           "other sectors": "Missing (rows not closed / graphs missing)",
                           "kappa3_domain_reduction": "NOT done (structural counts only, Q04)"},
          "defect": "c.phi.phi: zero at pole level; c.c.cbar, c.c.K: Open; h.h.c: Missing",
          "sigma0_closure_kappa3": "NOT established (only the c.phi.phi row)",
          "finite_parts": "Missing (pole level only)",
          "O1_closure_unchanged": "CLOSED (scoped)", "bulk_symmetry_restored": False,
          "errata": {"Q05-E1": "Q04 identity ledger row c.phi.phi omitted the (S1 h-phi-phi x Gamma_Kc) channel (medium; "
                               "fixed in Q05 rows)",
                     "Q05-E2": "code/q04/scalar_loop3.py docstring says G13/G14; the graphs are P6-G14/G15 (info; file is "
                               "pinned by the Q04 layer gate and left unchanged)",
                     "Q05-E3": "own D13 orientation = -I x XVI-P (convention, disclosed in Q05_rules_validation)",
                     "Q05-E4": "G10 loop momentum shifts had the wrong sign in the first Q05 evaluation (found while "
                               "assembling the ghost rows; fixed before release; certified G10 is the corrected one)"},
          "review_q04_folded": {"guard_scan": "hardened (Q05_guard_scan: recursive, 7 reviewer forms flagged) AND claim "
                                              "downgraded to best-effort policy scan (not a security boundary)",
                                "gate_unpinned_files": "Q05 gate rejects any code/**/*.py, tools/*.py or tests/*.py not "
                                                       "pinned", "docstring": "Q05-E2"},
          "flags": {"execution_pass": ok_rules, "claim_status": "partial", "scope": "pole level, one loop, kappa^3"}}
    (CERT / "Q05_P6_status.json").write_text(json.dumps(st, indent=1) + "\n")
    import guard_scan2
    (CERT / "Q05_guard_scan.json").write_text(json.dumps(guard_scan2.report(), indent=1) + "\n")
    print(json.dumps(st["stages"]), round(time.time() - t0, 1))


if __name__ == "__main__":
    main()
```

code/p6q05/release_q05.py

```
#!/usr/bin/env python3
"""XVI-Q Q05 release gate (fails closed; on reject no RELEASE_STATUS.json is left).
(1) code/p6q05/q05_expected.json pinned; EVERY Q01..Q05 certificate equals its pinned copy; EVERY code/**/*.py, tools/*.py,
    tests/*.py file is pinned and re-hashed -- an unpinned file is a reject (Q04 review); provenance via Q04 layer gate;
    bound Q04 independent report re-hashed;
(2) hardened scan run live: clean and all 7 reviewer evasion forms flagged; claim recorded as best-effort;
(3) semantic: own-rule validations constant with ratios D01/D03/D06 = I, D13 = -I (disclosed); two-point ghost equation
    constant; row c.phi.phi residual exactly 0 with every mutation detected and the S1.Gamma_Kc channel required;
    ghost rows reported OPEN (never closed/obstruction); G01/G02 Missing; P6_outcome Partial; O.1 record unchanged;
(4) Q04 layer gate accepted (certificates/Q04_release.json)."""
from __future__ import annotations
import hashlib, json, sys  # noqa: E401
from pathlib import Path

Q = Path(__file__).resolve().parents[2]
sys.path.insert(0, str(Q / "tools"))
import source_set_q05 as SS  # noqa: E402
CERT = Q / "certificates"
EXPECTED_SHA256 = "2b9291e05ed07c04bbf1d7250c05b123c32213ee4eef002ec5f92c6d4673e0b9"
Q04_REPORT_SHA = "bc2b16d9"
STATUS = (Q / "RELEASE_STATUS.json", CERT / "RELEASE_STATUS.json")
HARD_STOPS = {"bulk_symmetry_restored": False, "insertion": "frozen", "XV_B_opened": False, "XIV_J_opened": False,
              "Route_B_discharged": False, "Missing_never_zero": True}
READS: set[str] = set()


def rd(rel):
    READS.add(rel)
    return (Q / rel).read_bytes()
```

```
def jr(rel):
    return json.loads(rd(rel))


def sha_b(b):
    return hashlib.sha256(b).hexdigest()


def strip(o):
    if isinstance(o, dict):
        return {k: strip(v) for k, v in o.items() if k != "seconds"}
    if isinstance(o, list):
        return [strip(v) for v in o]
    return o


def code_files():
    out = {p.relative_to(Q).as_posix() for p in (Q / "code").rglob("*.py") if "__pycache__" not in p.parts}
    out |= {p.relative_to(Q).as_posix() for p in (Q / "tools").glob("*.py")}
    out |= {p.relative_to(Q).as_posix() for p in (Q / "tests").glob("*.py")}
    # Q06-owned files are pinned by the Q06 gate (code/p6q06/release_q06.py), not by this layer
    q06 = {f for f in out if f.startswith("code/p6q06/") or f.split("/")[-1] in ("test_q06.py",) or
           (f.startswith("tools/") and "q06" in f)}
    return out - {"code/p6q05/release_q05.py"} - q06


def checks():
    c = {}
    eb = rd("code/p6q05/q05_expected.json") if (Q / "code/p6q05/q05_expected.json").is_file() else b""
    c["expected_file_pinned"] = bool(eb) and sha_b(eb) == EXPECTED_SHA256
    exp = json.loads(eb) if eb else {}
    certs = exp.get("certificates", {})
    def _layer(n, o):
        # the live scan count grows when later layers (Q06+) add code files; the pinned count is a lower bound
        if n == "Q05_guard_scan" and isinstance(o, dict):
            o = dict(o)
            ok_n = isinstance(o.get("n_files_scanned"), int) and o["n_files_scanned"] >= certs[n]["n_files_scanned"]
            o["n_files_scanned"] = certs[n]["n_files_scanned"] if ok_n else -1
        return o
    bad = [n for n, ref in certs.items() if not (CERT / f"{n}.json").is_file()
           or _layer(n, strip(jr(f"certificates/{n}.json"))) != ref]
    c["every_certified_value_matches_pinned"] = len(certs) >= 24 and not bad
    fs = exp.get("file_sha256", {})
    c["pinned_files_rehashed"] = bool(fs) and all((Q / f).is_file() and sha_b(rd(f)) == h for f, h in fs.items())
    c["no_unpinned_code_tools_tests_file"] = bool(fs) and code_files() <= set(fs)
    rep = rd("reproducibility/independent_rebuild_report_q04.json")
    c["bound_q04_report_rehashed"] = sha_b(rep).startswith(Q04_REPORT_SHA) and \
        sha_b(rep) == fs.get("reproducibility/independent_rebuild_report_q04.json")
    sys.path.insert(0, str(Q / "code" / "p6q05"))
    import guard_scan2
    READS.add("code/p6q05/guard_scan2.py")
    live = guard_scan2.report()
    gs = jr("certificates/Q05_guard_scan.json")
    c["scan_live_clean_and_evasions_flagged"] = live["scan_clean"] and live["all_reviewer_evasions_flagged"] and \
        gs["scan_clean"] is True and "NOT a security boundary" in gs["claim"]
    v = jr("certificates/Q05_rules_validation.json")
    c["rules_validated"] = v["all_constant"] is True and v["ratios_as_expected"] is True and \
        {k: x["ratio"] for k, x in v["validations"].items()} == {"D01": "I", "D03": "I", "D06": "I", "D13": "-I"}
    c["D13_orientation_disclosed"] = "-I" in v["D13_orientation_disclosure"]
    c["ghost_equation_two_point_constant"] = v["ghost_equation_two_point"]["ratio_D06_over_D13_K_to_T"] == "-2"
    g3 = v["ghost_equation_three_point"]
    c["ghost_equation_three_point"] = g3["ratio_G04_over_G03_K_to_T"] == g3["ratio_G09_over_G07_K_to_T"] == "-2"
    g = jr("certificates/Q05_P6_graphs.json")
    c["graphs_evaluated_nonzero"] = sorted(g["graphs"]) == ["G03", "G04", "G05", "G06", "G07", "G08", "G09", "G10",
                                                            "G11", "G12", "G13"] and \
        all(x["zero"] is False and x["sha256"] == sha_b(x["pole_over_U_d4_residue"].encode()) for x in g["graphs"].values())
    c["G01_G02_missing"] = set(g["missing"]) == {"G01", "G02"}
    r = jr("certificates/Q05_P6_rows.json")
    cp = r["c.phi.phi"]
    c["row_c_phi_phi_closed_with_mutations"] = cp["residual"] == "0" and cp["closed_at_pole_level"] is True and \
        cp["tree_identity_closed_with_sigma"] is True and len(cp["mutations_detected"]) >= 6 and \
        all(v for k, v in cp["mutations_detected"].items() if k != "drop_G05") and \
        cp["mutations_detected"].get("drop_S1_Kc_channel") is True and cp["G05_s0_projection_zero"] is True
    c["ghost_rows_open_not_overclaimed"] = all(x["loop_residual_zero"] is False and x["status"].startswith("OPEN")
                                               and x["tree_closes_with_constant_sigma"] is True
                                               for x in r["ghost_rows"].values()) and r["h.h.c"]["status"] == "Missing"
    st = jr("certificates/Q05_P6_status.json")
    c["honest_partial"] = st["P6_outcome"] == "Partial" and st["sigma0_closure_kappa3"].startswith("NOT") and \
        st["stages"]["graphs_missing"] == ["G01", "G02"] and st["finite_parts"].startswith("Missing")
    c["errata_recorded"] = set(st["errata"]) == {"Q05-E1", "Q05-E2", "Q05-E3", "Q05-E4"}
    o1 = jr("certificates/Q03_O1_record.json")
    c["O1_record_unchanged"] = o1["O1_closure"] == "CLOSED (scoped)" and o1["O1_closure_confirmed"] is True and \
        st["O1_closure_unchanged"] == "CLOSED (scoped)" and st["bulk_symmetry_restored"] is False
    q4 = jr("certificates/Q04_release.json") if (CERT / "Q04_release.json").is_file() else {}
```

```
    c["Q04_layer_accepted"] = q4.get("publication") == "accepted"
    c["gate_reads_in_source_set"] = READS <= set(SS.source_files(Q)) | {f"certificates/{n}.json" for n in certs} | \
        {"certificates/Q05_guard_scan.json", "certificates/Q05_rules_validation.json", "certificates/Q05_P6_graphs.json",
         "certificates/Q05_P6_rows.json", "certificates/Q05_P6_status.json", "certificates/Q03_O1_record.json",
         "certificates/Q04_release.json"}
    return c, st


def main():
    for p in STATUS:
        p.unlink(missing_ok=True)
    try:
        c, st = checks()
    except Exception as e:  # noqa: BLE001
        c, st = {"exception": False, "error": repr(e)}, {}
    ok = all(v is True for k, v in c.items() if k != "error")
    rec = {"landmark": "XVI-Q", "release": "Q05", "decision": "ACCEPT" if ok else "REJECT", "checks": c}
    (CERT / "Q05_release.json").write_text(json.dumps(rec, indent=1) + "\n")
    if not ok:
        print(json.dumps(rec, indent=1))
        return 1
    status = {"landmark": "XVI-Q", "release": "Q05", "status": "released (honest partial: P.6 continued)",
              "P6_outcome": st["P6_outcome"], "stages": st["stages"], "O1_closure": "CLOSED (scoped)",
              "O1_closure_confirmed": True, "hard_stops": HARD_STOPS,
              "independent_rebuild_passed_q04": True, "independent_rebuild_q05": "pending",
              "execution_pass": True, "claim_status": "partial", "gate": "code/p6q05/release_q05.py"}
    for p in STATUS:
        p.write_text(json.dumps(status, indent=1) + "\n")
    (Q / "ledgers").mkdir(exist_ok=True)
    (Q / "ledgers" / "Q05_milestones.json").write_text(json.dumps({"landmark": "XVI-Q", "release": "Q05",
        "P6_outcome": st["P6_outcome"], "stages": st["stages"], "hard_stops": HARD_STOPS}, indent=1) + "\n")
    print("ACCEPT")
    return 0


if __name__ == "__main__":
    raise SystemExit(main())
```

code/p6q05/rows.py

```
#!/usr/bin/env python3
"""XVI-Q Q05 kappa^3 identity rows at UV-pole level: (S, Gamma_1^pole) restricted to a three-field monomial.
All amplitudes are 'i x coefficient' (vertices) / 'i Gamma' (loop poles, units U) in the same normalisation, so a
row is linear in them.  The relative antibracket sign sigma of each antifield channel is CALIBRATED on the classical
identity (S_0, S_1) = 0 (tree level, kappa^1) -- the O.2-verified master equation -- and then used unchanged at one loop.
Row c.phi.phi (c p1, phi p2, phi p3):
   (S0, G3): Gamma_h phi phi [h -> s0 h(c)]  +  sigma * sum_phi p_phi^2 Gamma_{phi* c phi}
   (S1, G2): delta S_1/delta h |phi phi  contracted with Gamma_{K c} (D13 own pole)      (Gamma_phi phi = 0, Q02)
Pole-level requirement: every evanescent structure dies under the s0 polarisation (T(p1, c) = 0), so the row must
vanish EXACTLY (no evanescent counterterm can absorb a remainder)."""
from __future__ import annotations
import json, sys  # noqa: E401
from pathlib import Path
import sympy as sp

HERE = Path(__file__).resolve().parent
Q = HERE.parents[1]
sys.path.insert(0, str(HERE))
import gravity as G  # noqa: E402
import uvpole as U  # noqa: E402
import tensor as TT  # noqa: E402
import graphs as GR  # noqa: E402
import dred  # noqa: E402

I = sp.I
p1, p2, l = GR.p1, GR.p2, GR.l
p3 = GR.p3
LOG = Q / "build" / "logs"


def load(g):
    return sp.sympify(json.loads((LOG / f"q05_{g}.json").read_text())["pole_over_U_d4_residue"])


def polarise(Qa, a, u, w):
    """quadratic form Q(a) in symbols H_a_*, T_a_* -> bilinear B(u, w) (= value at E = sym(u, w))."""
    rep = {}
    for s in Qa.free_symbols:
        n = s.name
        if not (n.startswith("H_") or n.startswith("T_")):
            continue
        kind, x, y = n.split("_")
        if a not in (x, y):
```

```
            continue
        F = U.H if kind == "H" else U.T
        if x == a and y == a:
            rep[s] = sum(F(v1, v2) for v1 in ("tu", "tw") for v2 in ("tu", "tw"))
        else:
            o = y if x == a else x
            rep[s] = F("tu", o) + F("tw", o)
    e = sp.expand(Qa.xreplace(rep))
    tu = [s for s in e.free_symbols if "tu" in s.name.split("_")]
    tw = [s for s in e.free_symbols if "tw" in s.name.split("_")]
    mixed = 0
    for term in sp.Add.make_args(e):
        nu = sum(sp.degree(term, s) for s in tu)
        nw = sum(sp.degree(term, s) for s in tw)
        if nu == 1 and nw == 1:
            mixed += term
    B = sp.expand(mixed / 2)
    # substitute u, w
    rep2 = {}
    for s in B.free_symbols:
        kind, x, y = (s.name.split("_") + [None])[:3] if s.name.count("_") == 2 else (None, None, None)
        if kind is None or ("tu" not in (x, y) and "tw" not in (x, y)):
            continue
        sub = {"tu": u, "tw": w}
        X = sub.get(x, x)
        Y = sub.get(y, y)
        rep2[s] = _dot_sym(kind, X, Y)
    return sp.expand(B.xreplace(rep2))


def _dot_sym(kind, X, Y):
    mom = lambda z: z.startswith("p")  # noqa: E731
    if kind == "T" and (mom(X) or mom(Y)):
        return 0
    if X == Y and mom(X) and X == "p1":
        return U.H("p1", "p1")
    return (U.H if kind == "H" else U.T)(X, Y)


def kc_tree(KE, gc, q):
    """tree K.c amplitude: i (1/2) K:(d c + d c),  d -> -i q."""
    M = TT.sym(q, gc, -I)
    i1, i2 = G.L(), G.L()
    return sp.expand(I * TT.ev((1, [("m", KE, i1, i2), ("m", M, i2, i1)])))


def k_channel(Kamp, phis):
    """delta S1/delta h|phi phi (vertex V_h_phi_phi) contracted with (K-amplitude / i) through the identity pairing."""
    V = G.V_hphiphi(G.probe_mat("E"), *phis)
    e = sp.expand(Kamp * V / I)
    t1 = G.contract(e, [("KU", "EU"), ("KW", "EW")])
    t2 = G.contract(e, [("KU", "EW"), ("KW", "EU")])
    return sp.expand((t1 + t2) / 2)


def phistar_tree(gc):
    return sp.expand(U.dot(p3, p3) * G.V_phis_c_phi(gc, p2) + U.dot(p2, p2) * G.V_phis_c_phi(gc, p3))


def calibrate_cphiphi():
    gc = U.vec("gc1")
    KE = G.probe_mat("K")
    a = k_channel(kc_tree(KE, gc, p1), (p2, p3))
    b = phistar_tree(gc)
    r = sp.cancel(-a / b)
    return ([r] if not r.free_symbols else []), sp.expand(a), sp.expand(b)


def D13_own_K():
    """own D13 K.c pole as a function of a probe K (KU, KW), c pol gc1 momentum p1 (K carries -p1)."""
    gc = U.vec("gc1")
    A = G.V_cb_c_h(G.probe_vec("gA"), gc, p1, G.probe_mat("hA"), l)
    B = G.V_K_c_h(G.probe_mat("K"), G.probe_vec("gB"), p1 + l, G.probe_mat("hB"), -l)
    n = G.contract_vertices(A, B, [("h", "hA", "hB"), ("c", "gA", "gB", GR.GH)])
    return sp.expand(dred.substitute_d(U.loop_pole(n, [U.V(), p1], 3, 1), 4, "Q05 row: D13 own pole"))


def row_cphiphi(sig):
    hpp = sum(load(g) for g in ("G05", "G06", "G12", "G13"))
    t_h = -I * polarise(hpp, "a1", "p1", "gc1")                    # E = s0 h = sym(p1, gc, -i)
    g11a = load("G11")                                              # phi p2, phi* p3
    g11b = GR.G11_swapped()                                         # phi p3, phi* p2
    t_s = sig * (U.dot(p3, p3) * g11a + U.dot(p2, p2) * g11b)
    t_k = k_channel(D13_own_K(), (p2, p3))
```

```
    t = sp.Symbol("t")
    t_h, t_s, t_k = (sp.expand(x.subs(t, 1)) for x in (t_h, t_s, t_k))   # t is a grading parameter only
    R = sp.expand(t_h + t_s + t_k)
    return {"S0_hphiphi": str(t_h), "S0_phistar": str(sp.expand(t_s)), "S1_Kc": str(t_k), "residual": str(R),
            "zero": R == 0, "without_S1_Kc_zero": sp.expand(t_h + t_s) == 0}


if __name__ == "__main__":
    sol, a, b = calibrate_cphiphi()
    print("calibration sigma:", sol, "| tree K-channel:", a, "| tree phi*:", b)
    if sol:
        r = row_cphiphi(sol[0])
        print({k: (v if k in ("zero", "without_S1_Kc_zero") else v[:200]) for k, v in r.items()})


# ============================================================================ ghost rows (two identical Grassmann c's)
def _V(name):
    return U.V({name: 1})


def remap(expr, mp):
    """momentum relabelling of a pole expression: mp maps 'p1','p2' to U.V vectors (no t); p3 implicit;
    polarisation renames via mp entries for non-momentum names (str -> str)."""
    rep = {}
    for s in expr.free_symbols:
        parts = s.name.split("_")
        if len(parts) != 3 or parts[0] not in ("H", "T"):
            continue
        kind, x, y = parts
        def vec(z):  # noqa: E306
            m = mp.get(z, z)
            return m if isinstance(m, U.V) else _V(m)
        X, Y = vec(x), vec(y)
        if kind == "T":
            mom = any(k.startswith("p") for k in list(X) + list(Y))
            rep[s] = 0 if mom else U.T(next(iter(X)), next(iter(Y)))
        else:
            rep[s] = sp.expand(sum(cx * cy * U.H(*sorted((kx, ky))) if True else 0
                                   for kx, cx in X.items() for ky, cy in Y.items()))
    return sp.expand(expr.xreplace(rep))


P1, P2 = _V("p1"), _V("p2")
P3 = -(P1 + P2)


def strip_t(e):
    return sp.expand(sp.sympify(e).subs(sp.Symbol("t"), 1))


def contract_id(A, B, pa, pb, kind):
    """identity pairing of probe pa (in A) with probe pb (in B): kind 'h' (sym pair) or 'c' (vector)."""
    e = sp.expand(A * B / I)
    if kind == "c":
        return G.contract(e, [(pa, pb)])
    t1 = G.contract(e, [(pa + "U", pb + "U"), (pa + "W", pb + "W")])
    t2 = G.contract(e, [(pa + "U", pb + "W"), (pa + "W", pb + "U")])
    return sp.expand((t1 + t2) / 2)


def kin_cbc(gb, X, q):
    """tree cbar(gb) c(X, q) amplitude: i * coefficient of i cbar.Box c (Box -> -q^2)."""
    return sp.expand(I * I * (-U.dot(q, q)) * U.dot(gb, X))


def kin_Kc(KE, X, q):
    return kc_tree(KE, X, q)


def D13_own(gcname, q):
    gc = U.vec(gcname)
    A = G.V_cb_c_h(G.probe_vec("gA"), gc, q, G.probe_mat("hA"), 1)
    B = G.V_K_c_h(G.probe_mat("K"), G.probe_vec("gB"), q + l, G.probe_mat("hB"), -1)
    n = G.contract_vertices(A, B, [("h", "hA", "hB"), ("c", "gA", "gB", GR.GH)])
    return strip_t(dred.substitute_d(U.loop_pole(n, [U.V(), q], 3, 1), 4, "Q05 row: D13 own pole"))


def D06_own(gbname, X, q):
    """own D06 cbar(gb) c(probe X, momentum q) pole."""
    A = G.V_cb_c_h(G.probe_vec("gA"), G.probe_vec(X), q, G.probe_mat("hA"), 1)
    B = G.V_cb_c_h(U.vec(gbname), G.probe_vec("gB"), q + l, G.probe_mat("hB"), -1)
    n = G.contract_vertices(A, B, [("h", "hA", "hB"), ("c", "gA", "gB", GR.GH)])
    return strip_t(dred.substitute_d(U.loop_pole(n, [U.V(), q], 4, 1), 4, "Q05 row: D06 own pole"))
```

```
def ghost_row(target, loop=True, sig_c=None, drop=()):
    """target 'ccb' (c p1, c p2, cbar p3) or 'ccK' (c p1, c p2, K p3).  Channels:
       h:  Gamma_{X c h}[h -> s0 h(c_j)]               (tree: S1 vertex; loop: G04+G09 / G03+G07)
       K:  delta S1/delta h |_{X c} . Gamma_{K c}       (loop only; D13 own)
       c:  delta Gamma/delta c . delta Gamma/delta c*   (tree: kinetic x V_cs; loop: kinetic x G10 + Gamma_{X c} x V_cs)
    antisymmetrised in (c1, p1) <-> (c2, p2) with the Grassmann sign -1."""
    t = sp.Symbol("t")
    gc1, gc2 = U.vec("gc1"), U.vec("gc2")
    if target == "ccb":
        Xside = lambda cv, q, E, k: G.V_cb_c_h(U.vec("gb3"), cv, q, E, k)      # noqa: E731
        kin = lambda Xp, q: kin_cbc(U.vec("gb3"), Xp, q)                       # noqa: E731
        loopX = ("G04", "G09")      # computed with c p1, cbar p2, h p3
    else:
        Xside = lambda cv, q, E, k: G.V_K_c_h(GR.ext("aK"), cv, q, E, k)        # noqa: E731
        kin = lambda Xp, q: kin_Kc(GR.ext("aK"), Xp, q)                         # noqa: E731
        loopX = ("G03", "G07")
    p1v, p2v = U.vec("p1"), U.vec("p2")
    ch = {"h": 0, "K": 0, "c": 0}
    for (ci, pi, cj, pj, sgn) in (("gc1", p1v, "gc2", p2v, 1), ("gc2", p2v, "gc1", p1v, -1)):
        # h channel: X(c_i, p_i) with h -> s0 h(c_j, p_j)
        Es = TT.sym(pj, U.vec(cj), -I)
        if not loop:
            ch["h"] += sgn * Xside(U.vec(ci), pi, Es, pj)
        else:
            # loop Gamma_{X c h} stored with c(gc1,p1), X at p2, h(a3) at p3: relabel (p1,p2,p3) -> (p_i, p3, p_j)
            Gx = sum(load(g) for g in loopX)
            Pi, Pj = (P1, P2) if ci == "gc1" else (P2, P1)
            Gx = remap(Gx, {"p1": Pi, "p2": P3, "gc1": ci, "gb2": "gb3"})
            ch["h"] += sgn * (-I) * polarise(Gx, "a3", "p" + ("1" if pj is p1v else "2"), cj)
            # K channel: delta S1/delta h |_{X c_j} (h momentum p_i) . Gamma_{K c}(c_i, p_i)
            Kamp = D13_own(ci, pi)
            ch["K"] += sgn * contract_id(Kamp, Xside(U.vec(cj), pj, G.probe_mat("E"), pi), "K", "E", "h")
    # c channel (already antisymmetric through V_cs_c_c)
    Vcs = G.V_cs_c_c(G.probe_vec("Z"), gc1, p1v, gc2, p2v)
    q3 = -(p1v + p2v)   # momentum of the c leg contracted into X: c(X, q) with q = -p3 = p1 + p2
    ch["c"] += contract_id(kin(G.probe_vec("X"), q3), Vcs, "X", "Z", "c")
    if loop:
        ch["c"] = 0                                                                  # the tree piece is not a loop term
        g10 = load("G10")                                                           # c* pol z at p3
        kz = kin(G.probe_vec("X"), q3)
        # Gamma_{c* c c}(z) . kinetic(X): rename the c* polarisation z -> probe ZZ and pair it with X
        g10p = g10.xreplace({x: sp.Symbol("_".join("ZZ" if q == "z" else q for q in x.name.split("_")))
                             for x in g10.free_symbols if "z" in x.name.split("_")})
        ch["c1"] = sp.expand(G.contract(sp.expand(kz * g10p / I), [("X", "ZZ")]))
        if target == "ccb":
            gx = D06_own("gb3", "X", q3)
        else:
            gx = strip_t(D13_K_ext("X", q3))
        ch["c2"] = contract_id(gx, Vcs, "X", "Z", "c")
    ch = {k: strip_t(v) for k, v in ch.items()}
    if sig_c is None:
        return ch
    R = sp.expand(sum(v * (sig_c if k in ("c", "c1", "c2") else 1) for k, v in ch.items() if k not in drop))
    return ch, R


def D13_K_ext(Xc, q):
    """own D13 with the EXTERNAL K = ext('aK') and the c leg a probe Xc (momentum q)."""
    A = G.V_cb_c_h(G.probe_vec("gA"), G.probe_vec(Xc), q, G.probe_mat("hA"), 1)
    B = G.V_K_c_h(GR.ext("aK"), G.probe_vec("gB"), q + 1, G.probe_mat("hB"), -1)
    n = G.contract_vertices(A, B, [("h", "hA", "hB"), ("c", "gA", "gB", GR.GH)])
    return dred.substitute_d(U.loop_pole(n, [U.V(), q], 3, 1), 4, "Q05 row: D13 own pole (ext K)")
```

code/p6q05/tensor.py

```
#!/usr/bin/env python3
"""XVI-Q Q05 index-contraction evaluator on top of q04 uvpole vectors.

Factors: ('v', V, i)      vector with index label i
         ('m', Mat, i, j) rank-2 tensor   Mat = eta*eta_4S + sum c u (x) w   (NOT symmetrised; order u -> i, w -> j)
A term is (coef, [factors]); every label appears once (free) or twice (contracted, metric = eta_4S).
Contracted labels resolve into chains (vector ... vector) and cycles (traces); tr eta_4S = 4 (Q4S vertex indices).
Free labels are not supported (vertices are always fully contracted with polarisations / probe vectors)."""
from __future__ import annotations
import sys
from pathlib import Path
import sympy as sp

sys.path.insert(0, str(Path(__file__).resolve().parents[1] / "q04"))
import uvpole as U  # noqa: E402

TR_ETA = 4
```

```
class Mat:
    def __init__(self, eta=0, items=()):
        self.eta = eta
        self.items = list(items)

    def T(self):
        return Mat(self.eta, [(c, w, u) for c, u, w in self.items])

    def left(self, v):                    # v . M  (contract v with first index)
        out = U.V()
        if self.eta != 0:
            out = out + self.eta * v
        for c, u, w in self.items:
            out = out + (c * U.dot(v, u)) * w
        return out

    def tr(self):
        return sp.expand(TR_ETA * self.eta + sum(c * U.dot(u, w) for c, u, w in self.items))


def sym(u, w, c=1):
    """c * (u w + w u)/2"""
    h = sp.Rational(1, 2) * c
    return Mat(0, [(h, u, w), (h, w, u)])


def eps_mat(e):
    """q04 Eps (sum c (u w + w u)/2) -> Mat."""
    m = Mat(getattr(e, "eta", 0))
    for c, u, w in e:
        m.items += [(sp.Rational(1, 2) * c, u, w), (sp.Rational(1, 2) * c, w, u)]
    return m


def ETA():
    return Mat(1)


def _chain_value(start_vec, mats):
    v = start_vec
    for M in mats:
        v = M.left(v)
    return v


def evaluate(term):
    coef, facs = term
    # adjacency: label -> list of (factor index, slot)
    occ = {}
    for fi, f in enumerate(facs):
        for s, lab in enumerate(f[2:]):
            occ.setdefault(lab, []).append((fi, s))
    for lab, o in occ.items():
        if len(o) != 2:
            raise ValueError(f"label {lab} appears {len(o)} times")
    used = set()
    total = sp.Integer(1) * coef

    def other_end(fi, s):
        lab = facs[fi][2 + s]
        a, b = occ[lab]
        return b if a == (fi, s) else a

    def oriented(fi, enter_slot):
        """matrix factor fi entered through slot enter_slot -> (Mat oriented, exit slot)."""
        M = facs[fi][1]
        return (M, 1) if enter_slot == 0 else (M.T(), 0)

    # chains from vectors
    for fi, f in enumerate(facs):
        if f[0] != "v" or fi in used:
            continue
        used.add(fi)
        mats = []
        cur = other_end(fi, 0)
        while facs[cur[0]][0] == "m":
            used.add(cur[0])
            M, ex = oriented(cur[0], cur[1])
            mats.append(M)
            cur = other_end(cur[0], ex)
        used.add(cur[0])
        total = total * U.dot(_chain_value(f[1], mats), facs[cur[0]][1])
```

```
    # cycles of matrices
    for fi, f in enumerate(facs):
        if fi in used:
            continue
        mats = []
        cur = (fi, 0)
        while True:
            used.add(cur[0])
            M, ex = oriented(cur[0], cur[1])
            mats.append(M)
            nxt = other_end(cur[0], ex)
            if nxt[0] == fi:
                break
            cur = nxt
        total = total * _trace(mats)
    return sp.expand(total)


def _trace(mats):
    if len(mats) == 1:
        return mats[0].tr()
    M0, rest = mats[0], mats[1:]
    out = M0.eta * _trace([Mat(rest[0].eta, rest[0].items)] + rest[1:]) if M0.eta != 0 else 0
    if M0.eta != 0 and len(rest) == 1:
        out = M0.eta * rest[0].tr()
    for c, u, w in M0.items:
        out += c * U.dot(_chain_value(w, rest), u)
    return sp.expand(out)


def ev(*terms):
    return sp.expand(sum(evaluate(t) for t in terms))
```

code/p6q05/validate.py

```
#!/usr/bin/env python3
"""XVI-Q Q05: validate own gravity/ghost rules against XVI-P pinned two-point poles.
  D03 (ghost loop, hh)  D06 (cbar c: ghost-graviton bubble)  D13 (K'.c)  D01 (graviton loop, V3 x V3, symmetry 1/2)
usage: validate.py NAME  -> build/logs/q05_val_NAME.json  (ratio mine / XVI-P per structure; constant ratio required)"""
from __future__ import annotations
import json, sys, time  # noqa: E401
from pathlib import Path
import sympy as sp

HERE = Path(__file__).resolve().parent
Q = HERE.parents[1]
sys.path.insert(0, str(HERE))
import gravity as G  # noqa: E402
import uvpole as U  # noqa: E402
import tensor as TT  # noqa: E402
import dred  # noqa: E402

PV = Q / "provenance" / "XVI-P" / "certificates"
GHOST_NUM = 1 / (-sp.I)          # <c cbar> = -1/k^2 = (i/k^2) * i  (loop_pole supplies i/k^2)


def res4(e, why):
    return sp.expand(dred.substitute_d(e, 4, why))


def hh_reference(result):
    """XVI-P hh structures contracted with a1 a1 a2 a2, orbit-weighted (the mapping validated on D11 in Q04)."""
    H, T = U.H, U.T
    idx = {"m": 1, "n": 1, "r": 2, "s": 2}
    tot = 0
    for k, v in result.items():
        parts = k.split(" ")
        e = int(parts[-1][-1]) if parts[-1].startswith("(p.p)") else 0
        parts = [f for f in parts if not f.startswith("(p.p)")]
        val = H("p1", "p1") ** e
        orbit = set()
        for s1 in (0, 1):
            for s2 in (0, 1):
                mp = {"m": "nm"[s1], "n": "mn"[s1], "r": "sr"[s2], "s": "rs"[s2]}
                ren = []
                for f in parts:
                    nm, ix = f.split("_")
                    nix = "".join(mp[c] for c in ix)
                    ren.append(nm + "_" + ("".join(sorted(nix)) if nm in ("g4", "gt") else nix))
                orbit.add(tuple(sorted(ren)))
        for f in parts:
            nm, ix = f.split("_")
            a, b = (f"a{idx[ix[0]]}", f"a{idx[ix[1]]}") if len(ix) == 2 else (f"a{idx[ix]}", None)
```

```
            val *= {"g4": lambda: H(a, b) + T(a, b), "gt": lambda: T(a, b), "p": lambda: H(a, "p1")}[nm]()
        tot += len(orbit) * sp.sympify(v["UV_pole"]) * val
    return sp.expand(tot)


def ratio(mine, ref):
    r = sp.cancel(mine / ref) if ref != 0 else None
    return {"ratio": str(r), "constant": r is not None and not r.free_symbols}


def ext_mat(a):
    return TT.sym(U.vec(a), U.vec(a))


def D03():
    p, l = U.vec("p1"), U.vec("l")
    E1, E2 = ext_mat("a1"), ext_mat("a2")
    A = G.V_cb_c_h(G.probe_vec("gAb"), G.probe_vec("gAc"), l, E1, p)          # c in (l), cbar out (l+p)
    B = G.V_cb_c_h(G.probe_vec("gBb"), G.probe_vec("gBc"), l + p, E2, -p)     # c in (l+p), cbar out (l)
    num = G.ghost_line(G.ghost_line(sp.expand(A * B), "gAb", "gBc", 1 / GHOST_NUM), "gBb", "gAc", 1 / GHOST_NUM)
    mine = res4(U.loop_pole(-num, [U.V(), p], 4, 1), "Q05 D03 validation")   # -1 ghost loop
    ref = json.loads((PV / "P16_batchC.json").read_text())["graphs"]["D03"]["result"]
    return {"mine_d4": str(mine), **ratio(mine, hh_reference(ref))}


def D01():
    p, l = U.vec("p1"), U.vec("l")
    E1, E2 = ext_mat("a1"), ext_mat("a2")
    A = G.V3_grav((E1, p), (G.probe_mat("hA1"), -l), (G.probe_mat("hA2"), l - p + 0 * l))
    # lines: hA1 -- hB1 carries l (from A to B: incoming at B = l, at A = -l); hA2 -- hB2 carries (l - p)... at A incoming p + (-l) + x = 0 -> x = l -
p
    B = G.V3_grav((E2, -p), (G.probe_mat("hB1"), l), (G.probe_mat("hB2"), p - l))
    num = G.contract_vertices(A, B, [("h", "hA1", "hB1"), ("h", "hA2", "hB2")])
    mine = res4(U.loop_pole(num, [U.V(), -p], 4, sp.Rational(1, 2)), "Q05 D01 validation")
    ref = json.loads((PV / "P16_batchC.json").read_text())["graphs"]["D01"]["result"]
    return {"mine_d4": str(mine), **ratio(mine, hh_reference(ref))}


def D06():
    p, l = U.vec("p1"), U.vec("l")
    gc, gb = U.vec("gc"), U.vec("gb")
    A = G.V_cb_c_h(G.probe_vec("gA"), gc, p, G.probe_mat("hA"), l)            # ext c (p), h in (l), cbar out (p+l)
    B = G.V_cb_c_h(gb, G.probe_vec("gB"), p + l, G.probe_mat("hB"), -l)       # c in (p+l), h in (-l), ext cbar
    num = G.graviton_line(sp.expand(A * B), "hA", "hB", 1)
    num = G.ghost_line(num, "gA", "gB", 1 / GHOST_NUM)
    mine = res4(U.loop_pole(num, [U.V(), p], 4, 1), "Q05 D06 validation")
    D = json.loads((Q / "certificates" / "Q02_bulk_graphs.json").read_text())["D06"]["laurent"]
    H, T = U.H, U.T
    pp = H("p1", "p1")
    ref = sp.sympify(D["S1_delta^rho_nu_p4"]["UV_pole"]) * (H("gb", "gc") + T("gb", "gc")) * pp ** 2 + \
        sp.sympify(D["S2_p^rho_p_nu_p2"]["UV_pole"]) * H("gb", "p1") * H("gc", "p1") * pp + \
        sp.sympify(D["SE_Gt^rho_nu_p4"]["UV_pole"]) * T("gb", "gc") * pp ** 2
    return {"mine_d4": str(mine), **ratio(mine, sp.expand(ref))}


def D13():
    p, l = U.vec("p1"), U.vec("l")
    KE, gc = ext_mat("aK"), U.vec("gc")
    A = G.V_cb_c_h(G.probe_vec("gA"), gc, p, G.probe_mat("hA"), l)            # ext c (p), cbar out (p+l)
    B = G.V_K_c_h(KE, G.probe_vec("gB"), p + l, G.probe_mat("hB"), -l)        # ext K, c in (p+l)
    num = G.graviton_line(sp.expand(A * B), "hA", "hB", 1)
    num = G.ghost_line(num, "gA", "gB", 1 / GHOST_NUM)
    mine = res4(U.loop_pole(num, [U.V(), p], 3, 1), "Q05 D13 validation")
    r = json.loads((PV / "P15_D13.json").read_text())["result"]
    H, T = U.H, U.T
    pp, aKp, gcp = H("p1", "p1"), H("aK", "p1"), H("gc", "p1")
    ref = sp.sympify(r["T2"]["UV_pole"]) * 2 * (H("aK", "gc") + T("aK", "gc")) * aKp * pp + \
        sp.sympify(r["T3"]["UV_pole"]) * aKp ** 2 * gcp + sp.sympify(r["E2"]["UV_pole"]) * 2 * T("aK", "gc") * aKp * pp + \
        sp.sympify(r["T1"]["UV_pole"]) * (H("aK", "aK") + T("aK", "aK")) * gcp * pp
    return {"mine_d4": str(mine), **ratio(mine, sp.expand(ref))}


if __name__ == "__main__":
    name = sys.argv[1]
    t0 = time.time()
    out = {"name": name, **globals()[name](), "seconds": round(time.time() - t0, 1)}
    (Q / "build" / "logs" / f"q05_val_{name}.json").write_text(json.dumps(out, indent=1) + "\n")
    print(name, out["ratio"], out["constant"], out["seconds"])
```

code/p6q06/fastpole.py

```
#!/usr/bin/env python3
"""XVI-Q Q06 fast path for the large hhh graphs (G01 V3^3 triangle, G02 V3-V4 bubble): the SAME algorithm as
```

```
p6q05/gravity.contract_vertices + q04/uvpole.loop_pole, but the heavy polynomial products are done in a sparse
polynomial ring over QQ(i) (sympy PolyRing) instead of Expr expansion, and every partial product is truncated in the
Taylor parameter t at the pole order omega (exact: t-degrees only add).  d never enters the ring: it appears only in
uvpole.integrate_pole, which is applied to the final t^omega coefficient as an Expr (guarded, as before).
Equivalence with the Expr path is checked on G05/G06/G12 in q06_certs (bit-identical poles)."""
from __future__ import annotations
import itertools, sys  # noqa: E401
from pathlib import Path
import sympy as sp
from sympy.polys.rings import ring
from sympy.polys.domains import QQ_I

HERE = Path(__file__).resolve().parent
sys.path.insert(0, str(HERE.parent / "p6q05"))
import gravity as G  # noqa: E402
import uvpole as U  # noqa: E402

t, Lsym = U.t, U.L
Linv = sp.Symbol("Linv")


class Fast:
    def __init__(self, names):
        names = sorted(set(names))
        gens = [U.H(x, y) for x, y in itertools.combinations_with_replacement(names, 2)]
        gens += [U.T(x, y) for x, y in itertools.combinations_with_replacement(names, 2)]
        gens += [U.S(x) for x in names] + [t, Lsym, Linv]
        self.gens = list(dict.fromkeys(gens))
        self.R, *_ = ring(self.gens, QQ_I)
        self.idx = {g: i for i, g in enumerate(self.gens)}
        self.it = self.idx[t]

    def to(self, e):
        e = sp.expand(sp.sympify(e).xreplace({Lsym ** -k: Linv ** k for k in range(1, 12)}))
        return self.R.from_expr(e) if e != 0 else self.R.zero

    def back(self, p):
        e = p.as_expr()
        return sp.expand(e.xreplace({Linv: 1 / Lsym}))

    def trunc(self, p, w):
        return self.R({m: c for m, c in p.terms() if m[self.it] <= w})

    def split(self, p, probes):
        """{pattern exponent tuple over probe gens: coefficient ring element}; T-copies of probe dots dropped."""
        pg = [i for i, g in enumerate(self.gens) if g.name.startswith(("H_", "s_")) and
              any(q in probes for q in g.name.split("_")[1:])]
        tg = {i for i, g in enumerate(self.gens) if g.name.startswith("T_") and
              any(q in probes for q in g.name.split("_")[1:])}
        out = {}
        for m, c in p.terms():
            if any(m[i] for i in tg):
                continue
            key = tuple((i, m[i]) for i in pg if m[i])
            mm = list(m)
            for i, _ in key:
                mm[i] = 0
            out.setdefault(key, []).append((tuple(mm), c))
        return {k: self.R(dict(v)) for k, v in out.items()}

    def pattern_expr(self, key):
        e = sp.Integer(1)
        for i, k in key:
            e *= self.gens[i] ** k
        return e

    def contract(self, A, B, lines, w):
        probes = set()
        for ln in lines:
            probes |= ({ln[1] + "U", ln[1] + "W", ln[2] + "U", ln[2] + "W"} if ln[0] == "h" else {ln[1], ln[2]})
        SA, SB = self.split(A, probes), self.split(B, probes)
        cache = {}
        tot = self.R.zero
        for ka, ca in SA.items():
            for kb, cb in SB.items():
                key = (ka, kb)
                if key not in cache:
                    v = self.pattern_expr(ka) * self.pattern_expr(kb)
                    for ln in lines:
                        v = G.graviton_line(v, ln[1], ln[2], 1) if ln[0] == "h" else G.ghost_line(v, ln[1], ln[2], 1 / ln[3])
                    cache[key] = self.to(v)
                if cache[key]:
                    tot += self.trunc(self.trunc(cache[key] * ca, w) * cb, w)
        return tot
```

```
    def loop_pole(self, numer, dens, omega, prefactor):
        den = self.to(U.taylor_denominators(dens, omega))
        full = self.trunc(numer, omega) * den
        top = self.R({m: c for m, c in full.terms() if m[self.it] == omega})
        e = self.back(top).subs(t, 1) if top else sp.Integer(0)
        return sp.expand(prefactor * sp.I ** len(dens) * U.integrate_pole(e))
```

code/p6q06/g10_route2.py

```
#!/usr/bin/env python3
"""XVI-Q Q06 independent second route for P6-G10 (c* c c): loop momentum relabelled so that the internal graviton
carries l - p1 (ghost lines l and p1 + p2 - l), and the V_{c* c c} vertex is entered with its two c slots in the
opposite order (compensated by the Grassmann sign -1).  Must equal the Q05 G10 pole exactly."""
from __future__ import annotations
import sys
from pathlib import Path
import sympy as sp

HERE = Path(__file__).resolve().parent
sys.path.insert(0, str(HERE.parent / "p6q05"))
import graphs as GR  # noqa: E402
import gravity as G  # noqa: E402
import uvpole as U  # noqa: E402

p1, p2, l = GR.p1, GR.p2, GR.l


def G10_route2():
    vA = G.V_cb_c_h(G.probe_vec("xAb"), U.vec("gc1"), p1, G.probe_mat("hA"), l - p1)
    vB = G.V_cb_c_h(G.probe_vec("xBb"), U.vec("gc2"), p2, G.probe_mat("hB"), p1 - l)
    vC = G.V_cs_c_c(U.vec("z"), G.probe_vec("xCb"), p1 + p2 - l, G.probe_vec("xCa"), l)
    n = G.contract_vertices(vA, vB, [("h", "hA", "hB")])
    n = G.contract_vertices(n, vC, [("c", "xAb", "xCa", GR.GH), ("c", "xBb", "xCb", GR.GH)])
    return GR.r4(U.loop_pole(-n, [U.V(), -p1, -(p1 + p2)], 3, 1), "G10 route2")


if __name__ == "__main__":
    r = G10_route2()
    print(sp.expand(r - GR.G10()) == 0)
```

code/p6q06/hhh_fast.py

```
#!/usr/bin/env python3
"""XVI-Q Q06 P.6 hhh graphs with the fast ring path (fastpole.py):
  G01  V3^3 triangle (one orientation, symmetry 1; momenta as p6q05/graphs.G01)
  G02  V3 (one external h) -- two internal gravitons -- V4 (two external h), symmetry 1/2, summed over the 3 choices
usage: hhh_fast.py G01 | G02_1 | G02_2 | G02_3 | G02 (sum) -> build/logs/q06_<id>.json"""
from __future__ import annotations
import json, sys, time  # noqa: E401
from pathlib import Path
import sympy as sp

HERE = Path(__file__).resolve().parent
Q = HERE.parents[1]
sys.path.insert(0, str(HERE.parent / "p6q05"))
sys.path.insert(0, str(HERE))
import gravity as G  # noqa: E402
import graphs as GR  # noqa: E402
import uvpole as U  # noqa: E402
import fastpole as FP  # noqa: E402
import v4  # noqa: E402

p1, p2, l, p3 = GR.p1, GR.p2, GR.l, GR.p3
EXT = {"1": ("a1", p1), "2": ("a2", p2), "3": ("a3", p3)}
W = 4
LOG = Q / "build" / "logs"


def _names(probes):
    return ["a1", "a2", "a3", "p1", "p2", "l"] + [f"{x}{y}" for x in probes for y in "UW"]


def G01():
    F = FP.Fast(_names(["hAB", "hAC", "hBA", "hBC", "hCB", "hCA"]))
    A = F.trunc(F.to(GR._v3("a1", p1, "hAB", -(l + p1), "hAC", l)), W)
    B = F.trunc(F.to(GR._v3("a2", p2, "hBA", l + p1, "hBC", -(l + p1 + p2))), W)
    C = F.trunc(F.to(GR._v3("a3", p3, "hCB", l + p1 + p2, "hCA", -l)), W)
    n = F.contract(A, B, [("h", "hAB", "hBA")], W)
    n = F.contract(n, C, [("h", "hBC", "hCB"), ("h", "hAC", "hCA")], W)
    return GR.r4(F.loop_pole(n, [U.V(), p1, p1 + p2], W, 1), "G01")
```

```
def G02(ch):
    a, k = EXT[ch]
    o = [EXT[c] for c in "123" if c != ch]
    F = FP.Fast(_names(["hA1", "hA2", "hB1", "hB2"]))
    A = F.trunc(F.to(GR._v3(a, k, "hA1", -l, "hA2", l - k)), W)
    B = F.trunc(F.to(v4.V4_grav((GR.ext(o[0][0]), o[0][1]), (GR.ext(o[1][0]), o[1][1]),
                                (G.probe_mat("hB1"), l), (G.probe_mat("hB2"), k - l))), W)
    n = F.contract(A, B, [("h", "hA1", "hB1"), ("h", "hA2", "hB2")], W)
    return GR.r4(F.loop_pole(n, [U.V(), -k], W, sp.Rational(1, 2)), f"G02 ch{ch}")


def run(gid):
    t0 = time.time()
    if gid == "G02":
        r = sp.expand(sum(sp.sympify(json.loads((LOG / f"q06_G02_{c}.json").read_text())["pole_over_U_d4_residue"])
                          for c in "123"))
    elif gid == "G01":
        r = G01()
    else:
        r = G02(gid[-1])
    out = {"graph": "P6-" + gid, "pole_over_U_d4_residue": str(r), "n_terms": len(sp.Add.make_args(r)),
           "zero": r == 0, "seconds": round(time.time() - t0, 1)}
    (LOG / f"q06_{gid}.json").write_text(json.dumps(out, indent=1) + "\n")
    return out


if __name__ == "__main__":
    o = run(sys.argv[1])
    print(sys.argv[1], o["n_terms"], o["zero"], o["seconds"])
```

code/p6q06/q06_certs.py

```
#!/usr/bin/env python3
"""XVI-Q Q06 certificates (P.6 continued; honest partial):
  Q06_ghost_rows.json        c.c.cbar / c.c.K rows: diagnosis + fix (K-channel Grassmann sign), 4D closure, unique fit,
                             mutations, explicit evanescent remainder
  Q06_G10_crosscheck.json    second route for G10 (re-routed loop momentum, swapped Wick order) == Q05 G10
  Q06_V4_validation.json     V4_grav: EH gauge identity n=2 (V3/V2 control) and n=3 (V4/V3) exactly zero; permutation symmetry
  Q06_fastpath.json          ring fast path == Expr path (G05, G06 bit-identical)
  Q06_P6_hhh.json            G01 (V3^3 triangle) and G02 (V3-V4 bubble, 3 channels) UV poles; Bose checks
  Q06_P6_status.json         outcome per stage, errata, open items"""
from __future__ import annotations
import hashlib, json, sys, time  # noqa: E401
from concurrent.futures import ProcessPoolExecutor
from pathlib import Path

HERE = Path(__file__).resolve().parent
Q = HERE.parents[1]
CERT = Q / "certificates"
LOG = Q / "build" / "logs"
sys.path.insert(0, str(HERE))
sys.path.insert(0, str(HERE.parent / "p6q05"))


def sha(s):
    return hashlib.sha256(s.encode()).hexdigest()


def task(name):
    import sympy as sp
    t0 = time.time()
    if name in ("G01", "G02_1", "G02_2", "G02_3"):
        import hhh_fast
        o = hhh_fast.run(name)
        o.pop("seconds", None)
        return name, o
    if name == "V4":
        import v4
        r = {"gauge_n2_V3_vs_V2_zero": v4.gauge_check(2) == 0, "gauge_n3_V4_vs_V3_zero": v4.gauge_check(3) == 0}
        r["V4_permutation_symmetric"], r["V4_terms_generic"] = v4.symmetry_check()
        return name, r
    if name == "G10":
        import g10_route2
        import rows as R
        r2 = g10_route2.G10_route2()
        old = R.load("G10")
        return name, {"route2_equals_Q05_G10": sp.expand(r2 - old) == 0, "n_terms": len(sp.Add.make_args(r2)),
                      "sha256_route2": sha(str(r2)), "route": "graviton momentum l - p1; ghost lines l, p1+p2-l; V_c*cc slots swapped (x -1)"}
    if name == "fast":
        import fastpole as FP
        import graphs as GR
        import gravity as G
```

```
        import uvpole as U
        import rows as R
        p1, p2, l, p3 = GR.p1, GR.p2, GR.l, GR.p3
        out = {}
        F = FP.Fast(["a1", "a2", "a3", "p1", "p2", "l"] + [f"{x}{y}" for x in ("hA1", "hA2", "hB1", "hB2") for y in "UW"])
        A = F.to(GR._v3("a1", p1, "hA1", -l, "hA2", l - p1))
        B = F.to(G.V_hhphiphi(G.probe_mat("hB1"), G.probe_mat("hB2"), p2, p3))
        r = GR.r4(F.loop_pole(F.contract(A, B, [("h", "hA1", "hB1"), ("h", "hA2", "hB2")], 4), [U.V(), -p1], 4,
                              sp.Rational(1, 2)), "G06 fast")
        out["G06_fast_equals_expr"] = sp.expand(r - R.load("G06")) == 0
        F = FP.Fast(["a1", "a2", "a3", "p1", "p2", "l"] + [f"{x}{y}" for x in ("hAB", "hAC", "hB", "hC") for y in "UW"])
        A = F.to(GR._v3("a1", p1, "hAB", l + p2, "hAC", p3 - l))
        B = F.to(G.V_hphiphi(G.probe_mat("hB"), p2, l))
        C = F.to(G.V_hphiphi(G.probe_mat("hC"), p3, -l))
        n = F.contract(F.contract(A, B, [("h", "hAB", "hB")], 4), C, [("h", "hAC", "hC")], 4)
        r = GR.r4(F.loop_pole(n, [U.V(), p2, -p3], 4, 1), "G05 fast")
        out["G05_fast_equals_expr"] = sp.expand(r - R.load("G05")) == 0
        return name, out
    if name in ("ccK", "ccb"):
        import rows6
        return name, rows6.row(name)
    raise KeyError(name)


def swap(e, mp):
    import sympy as sp
    rep = {}
    for s in e.free_symbols:
        k, x, y = s.name.split("_")
        rep[s] = sp.Symbol("_".join([k] + sorted([mp.get(x, x), mp.get(y, y)])))
    return sp.expand(e.xreplace(rep))


def main():
    import sympy as sp
    CERT.mkdir(exist_ok=True)
    t0 = time.time()
    names = ["G01", "G02_1", "G02_2", "G02_3", "V4", "G10", "fast", "ccK", "ccb"]
    with ProcessPoolExecutor(5) as ex:
        res = dict(ex.map(task, names))
    rows = {"identity": "(S, Gamma_1^pole) restricted to c.c.X monomials, X = cbar / K (Q05 channels)",
            "diagnosis": ("Q05 assembled the K channel (delta S1/delta h|_{X c_j} . Gamma_{K c_i}) with the orientation sign "
                          "of the h channel.  In the K channel c_i sits in Gamma_{K c_i}, to the right of c_j, so reordering to "
                          "c1 c2 costs one extra Grassmann transposition: K_o enters with -sgn(o).  G10 (checked by a second "
                          "route), the c channels and the identical-c antisymmetrisation were correct."),
            "c.c.K": res["ccK"], "c.c.cbar": res["ccb"],
            "status": ("CLOSED on the 4D (non-evanescent) part at pole level for both rows; the remainder is purely evanescent "
                       "(every term carries one T between external polarisations).  Since the row is linear and the 4D part vanishes, local "
                       "evanescent counterterms equal to minus the evanescent pole parts (p22 prescription) remove it; whether those "
                       "counterterms are s0-consistent on the full kappa^3 evanescent domain is OPEN.")}
    (CERT / "Q06_ghost_rows.json").write_text(json.dumps(rows, indent=1) + "\n")
    (CERT / "Q06_G10_crosscheck.json").write_text(json.dumps(res["G10"], indent=1) + "\n")
    (CERT / "Q06_V4_validation.json").write_text(json.dumps(
        {"vertex": "V4_grav = i x delta^4 S_EH (kappa^2 stripped), code/p6q06/v4.py", **res["V4"],
         "XVI-P_comparison": "not possible: no non-scaleless XVI-P graph contains V4 (the V4 tadpole is scaleless); "
                             "validated instead by the exact EH gauge identity delta0 S4 + delta1 S3 = 0"}, indent=1) + "\n")
    (CERT / "Q06_fastpath.json").write_text(json.dumps(res["fast"], indent=1) + "\n")
    g02 = sp.expand(sum(sp.sympify(res[f"G02_{c}"]["pole_over_U_d4_residue"]) for c in "123"))
    g01 = sp.sympify(res["G01"]["pole_over_U_d4_residue"])
    sw = {"a1": "a2", "a2": "a1", "p1": "p2", "p2": "p1"}
    import rows as R

    def b13(e):
        return sp.expand(R.remap(e, {"p1": R.P3, "p2": R.P2, "a1": "a3", "a3": "a1"}) - e) == 0
    hhh = {"graphs": {
        "G01": {"pole_over_U_d4_residue": str(g01), "sha256": sha(str(g01)), "n_terms": len(sp.Add.make_args(g01)),
                "zero": g01 == 0, "bose_12_symmetric": swap(g01, sw) == g01, "bose_13_symmetric": b13(g01), "method": "fastpole ring path,
t-truncated"},
        "G02": {"pole_over_U_d4_residue": str(g02), "sha256": sha(str(g02)), "n_terms": len(sp.Add.make_args(g02)),
                "zero": g02 == 0, "bose_12_symmetric": swap(g02, sw) == g02, "bose_13_symmetric": b13(g02),
                "channels_bose_related": swap(sp.sympify(res["G02_1"]["pole_over_U_d4_residue"]), sw)
                == sp.sympify(res["G02_2"]["pole_over_U_d4_residue"]),
                "channel_n_terms": {c: res[f"G02_{c}"]["n_terms"] for c in "123"}, "method": "fastpole ring path, V4 own"}},
        "note": "momentum p3 = -p1-p2 eliminated; Bose symmetry tested under (a1,p1)<->(a2,p2) and (a1,p1)<->(a3,p3) (full S3)"}
    (CERT / "Q06_P6_hhh.json").write_text(json.dumps(hhh, indent=1) + "\n")
    ok_rows = all(res[t]["residual_4D_zero"] and all(res[t]["mutations_detected"].values()) for t in ("ccK", "ccb"))
    st = {"landmark": "XVI-Q", "release": "Q06", "P6_outcome": "Partial",
          "stages": {"ghost_rows_4D_closed": ok_rows, "ghost_rows_evanescent_remainder": "explicit; removable by p22-type evanescent CTs; their
kappa^3 s0-consistency OPEN",
                     "G10_second_route": res["G10"]["route2_equals_Q05_G10"],
                     "V4_validated_gauge_identity": res["V4"]["gauge_n3_V4_vs_V3_zero"],
                     "graphs_evaluated_pole": ["G01", "G02", "G03", "G04", "G05", "G06", "G07", "G08", "G09", "G10",
                                               "G11", "G12", "G13", "G14", "G15"],
                     "graphs_missing": [], "graphs_derived_zero": ["T1", "T2", "T3"],
```

```
                    "row_c_phi_phi": "closed at pole level (Q05)",
                    "row_c_c_cbar": "closed on 4D part; evanescent remainder OPEN",
                    "row_c_c_K": "closed on 4D part; evanescent remainder OPEN",
                    "row_h_h_c": "Missing (channel ledger recorded; not assembled)"},
          "h_h_c_ledger": ["Gamma_hhh (G01+G02+G08+G14+G15) [h -> s0 h(c)]",
                           "delta S_2/delta h (EH + gauge fixing) . Gamma_{K h c} (G03+G07)",
                           "s1 h = L_c h  . Gamma_hh pole (all kappa^2 hh self-energy graphs)",
                           "delta S_EH3/delta h|_hh . Gamma_{K c} (D13)",
                           "b eliminated: s cbar ~ F(h) => F(h) . Gamma_{cbar c h} (G04+G09)  [channel absent from the Q04 ledger]"],
          "kappa3_CT_domain_reduction": "Missing", "kappa3_defect": "Missing", "kappa3_CT_solve": "Missing",
          "sigma0_closure_kappa3": "NOT established", "finite_parts": "Missing (pole level only)",
          "errata": {"Q06-E1": "Q05 ghost rows reported OPEN because of a Grassmann-sign error in the K-channel assembly "
                               "(rows.ghost_row); not a property of the graphs (low; corrected here, Q05 record kept)",
                     "Q06-E2": "Q04 ledger for h.h.c omits the b-elimination channel F(h).Gamma_{cbar c h} (low; recorded)"},
          "disclosed_revisions": {"Q06-R1": "Q06-R1 (disclosed revision of the frozen Q05 layer): code/p6q05/release_q05.py now treats the pinned
Q05_guard_scan n_files_scanned as a LOWER bound (the recursive scan count grows when Q06 adds code files) and excludes Q06-owned files (code/p6q06/,
tests/test_q06.py, tools/*q06*) from its unpinned-file check; those are pinned by the Q06 gate. Q05_guard_scan.json regenerated (26 -> 33 files, still
clean). release_q05.py is re-pinned by q06_expected.json."},
          "O1_closure_unchanged": "CLOSED (scoped)", "bulk_symmetry_restored": False,
          "flags": {"execution_pass": ok_rows and res["V4"]["gauge_n3_V4_vs_V3_zero"], "claim_status": "partial",
                    "scope": "pole level, one loop, kappa^3"}, "seconds": round(time.time() - t0, 1)}
    (CERT / "Q06_P6_status.json").write_text(json.dumps(st, indent=1) + "\n")
    print("Q06 certs", st["flags"], st["seconds"])


if __name__ == "__main__":
    main()
```

code/p6q06/q07/evct.py

```
#!/usr/bin/env python3
"""XVI-Q Q07 goal 1: sigma0-consistency of the evanescent counterterms that remove the ghost-row evanescent remainder.
CT_ev := - (evanescent part of every one-loop pole entering the rows: Gamma_{Kch}, Gamma_{cbar ch}, Gamma_{c* cc},
Gamma_{Kc}, Gamma_{cbar c}), i.e. the p22 prescription (local: each pole is a polynomial).  The rows are linear in the
loop inputs, so sigma0(Gamma_pole + CT_ev) on a row = the row evaluated on the 4D-PROJECTED inputs (every T set to 0 in
every loop input; tree vertices untouched, they are d-dimensional).  Test: that row vanishes IDENTICALLY (including any
T structure the tree vertices could generate).  If it does, the evanescent CTs are sigma0-consistent on these rows."""
from __future__ import annotations
import sys
from pathlib import Path
import sympy as sp

HERE = Path(__file__).resolve().parent
sys.path.insert(0, str(HERE.parents[1] / "p6q05"))
sys.path.insert(0, str(HERE.parent))
import rows as R  # noqa: E402
import rows6  # noqa: E402


def four_d(e):
    e = sp.sympify(e)
    return sp.expand(e.xreplace({x: 0 for x in e.free_symbols if x.name.startswith("T_")}))


def projected_parts(target):
    orig = (R.load, R.D13_own, R.D06_own, R.D13_K_ext)
    try:
        R.load = lambda g: four_d(orig[0](g))                       # noqa: E731
        R.D13_own = lambda *a: four_d(orig[1](*a))                  # noqa: E731
        R.D06_own = lambda *a: four_d(orig[2](*a))                  # noqa: E731
        R.D13_K_ext = lambda *a: four_d(orig[3](*a))                # noqa: E731
        return rows6.parts(target)
    finally:
        R.load, R.D13_own, R.D06_own, R.D13_K_ext = orig


def test(target):
    P = projected_parts(target)
    C = rows6.coeffs(target)
    Rr = sp.expand(sum(C[k] * P[k] for k in P))
    n_t = sum(1 for t in sp.Add.make_args(Rr) if any(x.name.startswith("T_") for x in t.free_symbols)) if Rr != 0 else 0
    # control: dropping the CT on one input (keep its evanescent pole) must re-open the row
    full = rows6.parts(target)
    ctrl = sp.expand(Rr + C["c1"] * (full["c1"] - P["c1"]))
    return {"row": target, "residual_with_evanescent_CTs": str(Rr), "zero": Rr == 0, "n_T_terms": n_t,
            "control_without_G10_evanescent_CT_nonzero": ctrl != 0}


if __name__ == "__main__":
    for t in ("ccK", "ccb"):
        r = test(t)
        print(t, r["zero"], r["n_T_terms"], r["control_without_G10_evanescent_CT_nonzero"], r["residual_with_evanescent_CTs"][:300])
```

code/p6q06/q07/evfit.py

```
#!/usr/bin/env python3
"""XVI-Q Q07 goal 1 (main test): do LOCAL evanescent kappa^3 counterterms exist that make the ghost rows vanish
identically?  Inputs: kappa^2 two-point poles with the p22 evanescent CTs applied (4D-projected Gamma_{Kc},
Gamma_{cbar c}); kappa^3 poles Gamma_{X c h} (X = K / cbar) and Gamma_{c* c c} plus a GENERIC local evanescent
counterterm each: every monomial of the right polarisation multidegree and momentum degree containing at least one T
(tilde dot) between polarisations, with a free coefficient.  The row is linear, so existence = solvability of a linear
system over Q(i).  Solvable -> the evanescent remainder is sigma0-exact (cancellable); not solvable -> explicit
evanescent obstruction on these rows (residual of the least-squares-free reduction recorded)."""
from __future__ import annotations
import itertools, sys  # noqa: E401
from pathlib import Path
import sympy as sp

HERE = Path(__file__).resolve().parent
sys.path.insert(0, str(HERE.parents[1] / "p6q05"))
sys.path.insert(0, str(HERE.parent))
sys.path.insert(0, str(HERE))
import rows as R  # noqa: E402
import rows6  # noqa: E402
import uvpole as U  # noqa: E402
from evct import four_d  # noqa: E402

MOM = ("p1", "p2")


def monomials(pols, nmom):
    """all products of H/T dot symbols with exact degree pols[name] in each polarisation and total momentum degree
    nmom, at least one T (T only between polarisations)."""
    names = sorted(pols)
    slots = []
    for n in names:
        slots += [n] * pols[n]
    out = set()

    def rec(rem, mom_left, acc, hasT):
        if not rem:
            if mom_left % 2 == 0 and hasT:
                k = mom_left // 2
                for pp in itertools.combinations_with_replacement([U.H("p1", "p1"), U.H("p1", "p2"), U.H("p2", "p2")], k):
                    out.add(acc * sp.Mul(*pp))
            return
        x = rem[0]
        rest = list(rem[1:])
        # pair x with a momentum
        if mom_left > 0:
            for m in MOM:
                rec(rest, mom_left - 1, acc * U.H(x, m), hasT)
        # pair x with another remaining slot
        seen = set()
        for j, y in enumerate(rest):
            if y in seen:
                continue
            seen.add(y)
            r2 = rest[:j] + rest[j + 1:]
            rec(r2, mom_left, acc * U.H(x, y), hasT)
            rec(r2, mom_left, acc * U.T(x, y), True)
    rec(slots, nmom, sp.Integer(1), False)
    return sorted(out, key=str)


def ansatz(pols, nmom, tag):
    ms = monomials(pols, nmom)
    xs = sp.symbols(f"{tag}0:{len(ms)}")
    return sp.expand(sp.I * sum(x * m for x, m in zip(xs, ms))), list(xs)


def run(target):
    if target == "ccK":
        A, xa = ansatz({"aK": 2, "gc1": 1, "a3": 2}, 3, "xa")
        loopX = ("G03", "G07")
    else:
        A, xa = ansatz({"gb2": 1, "gc1": 1, "a3": 2}, 4, "xa")
        loopX = ("G04", "G09")
    B, xb = ansatz({"z": 1, "gc1": 1, "gc2": 1}, 3, "xb")
    orig = (R.load, R.D13_own, R.D06_own, R.D13_K_ext)
    try:
        def load(g):
            e = orig[0](g)
            if g == loopX[0]:
                return sp.expand(e + A)
            if g == "G10":
                return sp.expand(e + B)
            return e
```

```
        R.load = load
        R.D13_own = lambda *a: four_d(orig[1](*a))                # noqa: E731
        R.D06_own = lambda *a: four_d(orig[2](*a))                # noqa: E731
        R.D13_K_ext = lambda *a: four_d(orig[3](*a))              # noqa: E731
        P = rows6.parts(target)
    finally:
        R.load, R.D13_own, R.D06_own, R.D13_K_ext = orig
    C = rows6.coeffs(target)
    tot = sp.expand(sum(C[k] * P[k] for k in P))
    xs = xa + xb
    gens = sorted([s for s in tot.free_symbols if s not in set(xs)], key=str)
    eqs = sp.Poly(tot, *gens).coeffs()
    sol = sp.linsolve(eqs, xs)
    ok = sol != sp.EmptySet
    res = {"row": target, "n_ct_params": len(xs), "n_equations": len(eqs), "solvable": ok}
    if ok:
        s0 = list(sol)[0]
        free = set().union(*[sp.sympify(v).free_symbols for v in s0]) & set(xs)
        res["n_free_params"] = len(free)
        part = {x: (sp.Integer(0) if x in free else sp.sympify(v)) for x, v in zip(xs, s0)}
        ctA = sp.expand(A.xreplace({x: v.xreplace({f: 0 for f in free}) for x, v in part.items() if x in xa}))
        ctB = sp.expand(B.xreplace({x: v.xreplace({f: 0 for f in free}) for x, v in part.items() if x in xb}))
        res["particular_CT_Gamma_Xch"] = str(ctA)
        res["particular_CT_Gamma_cstar_cc"] = str(ctB)
        res["particular_CT_Xch_n_terms"] = len(sp.Add.make_args(ctA)) if ctA != 0 else 0
        res["particular_CT_cstar_cc_n_terms"] = len(sp.Add.make_args(ctB)) if ctB != 0 else 0
    else:
        # rank deficiency report: solve with the remainder only on the evanescent part
        res["note"] = "no local evanescent kappa^3 CT in the ansatz removes the remainder"
    return res


if __name__ == "__main__":
    for t in ("ccK", "ccb"):
        r = run(t)
        print({k: (v if not isinstance(v, str) or len(v) < 200 else v[:200]) for k, v in r.items()}, flush=True)
```

code/p6q06/q07/evfit2.py

```
#!/usr/bin/env python3
"""XVI-Q Q07 goals 1+3: joint test whether LOCAL evanescent kappa^3 counterterms X exist with sigma0 X = - (evanescent
remainder) on ALL kappa^3 three-point rows at once (c.c.K, c.c.cbar, h.h.c; c.phi.phi has no remainder).
CT = -Gamma_pole + X.  Unknowns: generic evanescent (>= one T) local monomials for Gamma_{Kch}, Gamma_{cbar ch},
Gamma_{c* cc}, Gamma_hhh with the right multidegree; the SAME Gamma_{Kch} / Gamma_{cbar ch} counterterm enters the
ghost rows and the h.h.c row.  kappa^2 poles enter unmodified (their CT = -pole, p22, is sigma0-closed at kappa^2).
Linear system over Q(i): solvable -> the kappa^3 three-point defect is cancellable (pole level); otherwise the
reduced residual is an explicit evanescent obstruction on these rows."""
from __future__ import annotations
import json, sys, time  # noqa: E401
from pathlib import Path
import sympy as sp

HERE = Path(__file__).resolve().parent
Q = HERE.parents[2]
sys.path.insert(0, str(HERE.parents[1] / "p6q05"))
sys.path.insert(0, str(HERE.parent))
sys.path.insert(0, str(HERE))
import rows as R  # noqa: E402
import rows6  # noqa: E402
import gravity as G  # noqa: E402
import hhc  # noqa: E402
from evfit import ansatz  # noqa: E402
LOG = Q / "build" / "logs"


def ghost_row_total(target, XA, XB):
    loopX = ("G03", "G07") if target == "ccK" else ("G04", "G09")
    orig = R.load
    try:
        def load(g):
            e = orig(g)
            if g == loopX[0]:
                return sp.expand(e + XA)
            if g == "G10":
                return sp.expand(e + XB)
            return e
        R.load = load
        P = rows6.parts(target)
    finally:
        R.load = orig
    C = rows6.coeffs(target)
    return sp.expand(sum(C[k] * P[k] for k in P))
```

```
def hhc_ansatz_total(XH, XK, XC, sig):
    """row h.h.c contributions of the ansatz CTs (channels A, D, E) + the stored loop residual."""
    base = sp.sympify(json.loads((LOG / "q07_hhc_row.json").read_text())["residual"])
    A = R.strip_t(-sp.I * R.polarise(sp.expand(XH), "a1", "p1", "gc1"))
    D = 0
    E = 0
    for i in ("2", "3"):
        ai, ki, Fi = hhc.EXT[i]
        Gk = R.polarise(sp.expand(hhc._relabel(XK, i)), "aK", "KU", "KW")
        eh, gf = G.S2_EH_gf(hhc.ext(ai), G.probe_mat("E"), ki)
        D += R.strip_t(R.contract_id(Gk, sp.I * (eh + gf), "K", "E", "h"))
        E += R.strip_t(hhc.subst_vec(sp.expand(hhc._relabel(XC, i)), "gb2", G.Dvec(hhc.ext(ai), Fi)))
    return sp.expand(base + A + D + sig * E)


def main():
    t0 = time.time()
    XK, xk = ansatz({"aK": 2, "gc1": 1, "a3": 2}, 3, "xk")
    XC, xc = ansatz({"gb2": 1, "gc1": 1, "a3": 2}, 4, "xc")
    XS, xs = ansatz({"z": 1, "gc1": 1, "gc2": 1}, 3, "xs")
    XH, xh = ansatz({"a1": 2, "a2": 2, "a3": 2}, 4, "xh")
    sig = sp.sympify(json.loads((LOG / "q07_hhc_tree.json").read_text())["sigma_F"])
    rows = {"ccK": ghost_row_total("ccK", XK, XS), "ccb": ghost_row_total("ccb", XC, XS),
            "hhc": hhc_ansatz_total(XH, XK, XC, sig)}
    xall = xk + xc + xs + xh
    X = set(xall)
    eqs = []
    for r in rows.values():
        gens = sorted([s for s in r.free_symbols if s not in X], key=str)
        eqs += sp.Poly(r, *gens).coeffs()
    # Bose symmetry of the local hhh counterterm (it must come from a local functional): (12) and (13) swaps
    for mp in ({"p1": R.P2, "p2": R.P1, "a1": "a2", "a2": "a1"}, {"p1": R.P3, "p2": R.P2, "a1": "a3", "a3": "a1"}):
        d = sp.expand(XH - R.remap(XH, mp))
        if d != 0:
            gens = sorted([s_ for s_ in d.free_symbols if s_ not in X], key=str)
            eqs += sp.Poly(d, *gens).coeffs()
    sol = sp.linsolve(eqs, xall)
    ok = sol != sp.EmptySet
    out = {"unknowns": {"Gamma_Kch": len(xk), "Gamma_cbar_ch": len(xc), "Gamma_cstar_cc": len(xs), "Gamma_hhh": len(xh)},
           "n_equations": len(eqs), "solvable": ok, "seconds": round(time.time() - t0, 1)}
    if ok:
        s0 = list(sol)[0]
        free = set().union(*[sp.sympify(v).free_symbols for v in s0]) & X
        out["n_free"] = len(free)
        val = {x: sp.sympify(v).xreplace({f: 0 for f in free}) for x, v in zip(xall, s0)}
        for name, (Xe, xx) in {"Gamma_Kch": (XK, xk), "Gamma_cbar_ch": (XC, xc), "Gamma_cstar_cc": (XS, xs),
                               "Gamma_hhh": (XH, xh)}.items():
            e = sp.expand(Xe.xreplace({x: val[x] for x in xx}))
            out[f"particular_CT_{name}"] = str(e)
            out[f"particular_CT_{name}_n_terms"] = len(sp.Add.make_args(e)) if e != 0 else 0
        # verification: substitute the particular solution and recompute every row
        e = sp.expand(XH.xreplace({x: val[x] for x in xh}))
        out["hhh_CT_bose_symmetric"] = all(sp.expand(e - R.remap(e, mp)) == 0 for mp in (
            {"p1": R.P2, "p2": R.P1, "a1": "a2", "a2": "a1"}, {"p1": R.P3, "p2": R.P2, "a1": "a3", "a3": "a1"}))
        out["all_CT_evanescent"] = all(all(any(y.name.startswith("T_") for y in t.free_symbols)
                                           for t in sp.Add.make_args(sp.sympify(out[f"particular_CT_{n}"])))
                                       for n in ("Gamma_Kch", "Gamma_cbar_ch", "Gamma_hhh") if out[f"particular_CT_{n}"] != "0")
        out["verified_rows_zero"] = all(sp.expand(r.xreplace(val)) == 0 for r in rows.values())
    (LOG / "q07_evfit_joint.json").write_text(json.dumps(out, indent=1) + "\n")
    print({k: v for k, v in out.items() if not k.startswith("particular_CT_") or k.endswith("n_terms")})


if __name__ == "__main__":
    main()
```

code/p6q06/q07/hhc.py

```
#!/usr/bin/env python3
"""XVI-Q Q07 goal 2: row h.h.c (c(gc1) p1, h(a2) p2, h(a3) p3) of (S, Gamma_1) at kappa^3, UV-pole level.
Channels (linearised antibracket, 'i x coefficient' normalisation as in the Q05/Q06 rows):
  A  Gamma_hhh [h(a1) -> s0 h(c)]                          (G01+G02+G08 + scalar G14+G15, own)
  B  Gamma_hh(h_i, s1 h(h_j, c)),  s1 h = (1/2) L_c h      (own: graviton + ghost + scalar loops)
  C  delta S_EH3/delta h|_{h2 h3} . Gamma_{K c}            (D13 own)
  D  delta S_2/delta h|_{h_i} (EH + gf) . Gamma_{K h_j c} (G03+G07)
  E  s cbar = sigma_F F(h_i), F = D[h]:  F . Gamma_{cbar c h_j}   (G04+G09)  -- channel absent from the Q04 ledger
Tree calibration: (S,S)/2 at kappa^1 on h.h.c = A_t + B_t + sigma_F E_t = 0 fixes sigma_F; C, D enter with the
c.phi.phi K-channel convention (+1).  Polarisation matrices inside loops use t-FREE momenta."""
from __future__ import annotations
import json, sys, time  # noqa: E401
from pathlib import Path
import sympy as sp
```

```
HERE = Path(__file__).resolve().parent
Q = HERE.parents[2]
sys.path.insert(0, str(HERE.parents[1] / "p6q05"))
sys.path.insert(0, str(HERE.parent))
import gravity as G  # noqa: E402
import graphs as GR  # noqa: E402
import rows as R  # noqa: E402
import uvpole as U  # noqa: E402
import tensor as TT  # noqa: E402

I = sp.I
p1, p2, l, p3 = GR.p1, GR.p2, GR.l, GR.p3
F1, F2 = U.V({"p1": 1}), U.V({"p2": 1})          # t-free copies (for polarisation-side momenta)
F3 = -(F1 + F2)
LOG = Q / "build" / "logs"
GC = U.vec("gc1")
EXT = {"2": ("a2", p2, F2), "3": ("a3", p3, F3)}


def ext(a):
    return GR.ext(a)


def strip(e):
    return R.strip_t(e)


def subst_vec(e, name, V):
    """linear substitution of the vector polarisation ‘name‘ by V = {basis: coef} in H_/T_ symbols."""
    rep = {}
    for s in e.free_symbols:
        parts = s.name.split("_")
        if len(parts) != 3 or parts[0] not in ("H", "T") or name not in parts[1:]:
            continue
        o = parts[2] if parts[1] == name else parts[1]
        if parts[0] == "H":
            rep[s] = sum(c * U.H(k, o) for k, c in V.items())
        else:
            rep[s] = 0 if o in ("p1", "p2") else sum(c * U.T(k, o) for k, c in V.items() if k not in ("p1", "p2"))
    return sp.expand(e.xreplace(rep))


# ---------------------------------------------------------------- tree calibration
def tree():
    Es = TT.sym(p1, GC, -I)
    A = G.V3_grav((Es, p1), (ext("a2"), p2), (ext("a3"), p3))
    B = 0
    E = 0
    for i, j in (("2", "3"), ("3", "2")):
        ai, ki, _ = EXT[i]
        aj, kj, _ = EXT[j]
        M = G.Lc_h(ext(aj), kj, GC, p1)
        M.items = [(sp.Rational(1, 2) * c, u, w) for c, u, w in M.items]
        M.eta = sp.Rational(1, 2) * M.eta
        eh, gf = G.S2_EH_gf(ext(ai), M, ki)
        B += I * (eh + gf)
        E += G.V_cb_c_h(G.Dvec(ext(ai), ki), GC, p1, ext(aj), kj)
    A, B, E = strip(A), strip(B), strip(E)
    sig = sp.Symbol("sigma_F")
    tot = sp.expand(A + B + sig * E)
    gens = sorted([s for s in tot.free_symbols if s != sig], key=str)
    sol = sp.solve(sp.Poly(tot, *gens).coeffs(), sig, dict=True)
    return {"sigma_F": str(sol[0][sig]) if sol else None, "closes": bool(sol),
            "without_E_zero": sp.expand(A + B) == 0, "A_terms": len(sp.Add.make_args(A))}


# ---------------------------------------------------------------- own loops
def gamma_hh(E1, k, E2):
    """own Gamma_hh pole: legs (E1, k) and (E2, -k); graviton bubble (1/2) + ghost loop (-1) + scalar bubble (1/2)."""
    A = G.V3_grav((E1, k), (G.probe_mat("hA1"), -l), (G.probe_mat("hA2"), l - k))
    B = G.V3_grav((E2, -k), (G.probe_mat("hB1"), l), (G.probe_mat("hB2"), k - l))
    n = G.contract_vertices(A, B, [("h", "hA1", "hB1"), ("h", "hA2", "hB2")])
    grav = U.loop_pole(n, [U.V(), -k], 4, sp.Rational(1, 2))
    gA = G.V_cb_c_h(G.probe_vec("gAb"), G.probe_vec("gAc"), l, E1, k)
    gB = G.V_cb_c_h(G.probe_vec("gBb"), G.probe_vec("gBc"), l + k, E2, -k)
    num = G.ghost_line(G.ghost_line(sp.expand(gA * gB), "gAb", "gBc", 1 / GR.GH), "gBb", "gAc", 1 / GR.GH)
    gh = U.loop_pole(-num, [U.V(), k], 4, 1)
    sA = G.V_hphiphi(E1, l, -l - k)
    sB = G.V_hphiphi(E2, l + k, -l)
    sc = U.loop_pole(sp.expand(sA * sB), [U.V(), k], 4, sp.Rational(1, 2))
    return GR.r4(grav, "Q07 hh grav"), GR.r4(gh, "Q07 hh ghost"), GR.r4(sc, "Q07 hh scalar")


def scalar_hhh():
```

```
    """own scalar loops for Gamma_hhh: triangle (G14) + V_hh_phi_phi bubbles (G15, 3 channels, symmetry 1/2)."""
    vA = G.V_hphiphi(ext("a1"), l, -l - p1)
    vB = G.V_hphiphi(ext("a2"), l + p1, -l - p1 - p2)
    vC = G.V_hphiphi(ext("a3"), l + p1 + p2, -l)
    tri = U.loop_pole(sp.expand(vA * vB * vC), [U.V(), p1, p1 + p2], 4, 1)
    bub = 0
    legs = {"1": ("a1", p1), "2": ("a2", p2), "3": ("a3", p3)}
    for c in "123":
        a, k = legs[c]
        o = [legs[x] for x in "123" if x != c]
        v1 = G.V_hphiphi(ext(a), l, -l - k)
        v2 = G.V_hhphiphi(ext(o[0][0]), ext(o[1][0]), l + k, -l)
        bub += U.loop_pole(sp.expand(v1 * v2), [U.V(), k], 4, sp.Rational(1, 2))
    return GR.r4(tri, "Q07 G14"), GR.r4(bub, "Q07 G15")


def load_any(g):
    for f in (LOG / f"q06_{g}.json", LOG / f"q05_{g}.json", LOG / f"q07_{g}.json"):
        if f.is_file():
            return sp.sympify(json.loads(f.read_text())["pole_over_U_d4_residue"])
    if g == "G02" and all((LOG / f"q06_G02_{c}.json").is_file() for c in "123"):   # fresh builds: sum of channels
        return sp.expand(sum(load_any(f"G02_{c}") for c in "123"))
    raise FileNotFoundError(g)


def chan_A():
    hhh = sum(load_any(g) for g in ("G01", "G02", "G08", "G14", "G15"))
    return strip(-I * R.polarise(sp.expand(hhh), "a1", "p1", "gc1"))


def chan_B(i):
    j = "3" if i == "2" else "2"
    ai, ki, _ = EXT[i]
    aj, _, Fj = EXT[j]
    M = G.Lc_h(ext(aj), Fj, GC, F1)
    M.items = [(sp.Rational(1, 2) * c, u, w) for c, u, w in M.items]
    M.eta = sp.Rational(1, 2) * M.eta
    g, h, s = gamma_hh(ext(ai), ki, M)
    return {"grav": strip(g), "ghost": strip(h), "scalar": strip(s)}


def chan_C():
    V = G.V3_grav((G.probe_mat("E"), p1), (ext("a2"), p2), (ext("a3"), p3))
    return strip(R.contract_id(R.D13_own("gc1", p1), V, "K", "E", "h"))


def _relabel(e, i):
    """stored X c h poles: c p1, X p2, h(a3) p3.  i = '2': as stored; i = '3': X at p3, h(a2) at p2."""
    return e if i == "2" else R.remap(e, {"p1": R.P1, "p2": R.P3, "a3": "a2"})


def chan_D(i):
    ai, ki, _ = EXT[i]
    Gk = _relabel(load_any("G03") + load_any("G07"), i)
    Gk = R.polarise(sp.expand(Gk), "aK", "KU", "KW")
    eh, gf = G.S2_EH_gf(ext(ai), G.probe_mat("E"), ki)
    return strip(R.contract_id(Gk, I * (eh + gf), "K", "E", "h"))


def chan_E(i):
    ai, _, Fi = EXT[i]
    Gb = _relabel(load_any("G04") + load_any("G09"), i)
    return strip(subst_vec(sp.expand(Gb), "gb2", G.Dvec(ext(ai), Fi)))


TASKS = {"tree": tree, "A": chan_A, "B2": lambda: chan_B("2"), "B3": lambda: chan_B("3"), "C": chan_C,
         "D2": lambda: chan_D("2"), "D3": lambda: chan_D("3"), "E2": lambda: chan_E("2"), "E3": lambda: chan_E("3"),
         "S": scalar_hhh}


if __name__ == "__main__":
    k = sys.argv[1]
    t0 = time.time()
    r = TASKS[k]()
    if k == "S":
        for g, v in zip(("G14", "G15"), r):
            (LOG / f"q07_{g}.json").write_text(json.dumps({"graph": "P6-" + g, "pole_over_U_d4_residue": str(v),
                                                           "n_terms": len(sp.Add.make_args(v))}) + "\n")
        r = {"G14": len(sp.Add.make_args(r[0])), "G15": len(sp.Add.make_args(r[1]))}
    out = {k2: str(v) for k2, v in r.items()} if isinstance(r, dict) else {"value": str(r)}
    (LOG / f"q07_hhc_{k}.json").write_text(json.dumps(out, indent=1) + "\n")
    print(k, {a: (b[:80] if isinstance(b, str) else b) for a, b in out.items()}, round(time.time() - t0, 1))
```

code/p6q06/q07/hhc_row.py

```
#!/usr/bin/env python3
"""XVI-Q Q07: assemble row h.h.c from the hhc.py channel files; residual, 4D part, matter-sector check, unconstrained fit,
mutations."""
from __future__ import annotations
import json, sys  # noqa: E401
from pathlib import Path
import sympy as sp

HERE = Path(__file__).resolve().parent
Q = HERE.parents[2]
sys.path.insert(0, str(HERE.parents[1] / "p6q05"))
sys.path.insert(0, str(HERE))
import rows as R  # noqa: E402
import hhc  # noqa: E402
from evct import four_d  # noqa: E402
LOG = Q / "build" / "logs"


def rd(k):
    return json.loads((LOG / f"q07_hhc_{k}.json").read_text())


def parts():
    sig = sp.sympify(rd("tree")["sigma_F"])
    P = {"A": sp.sympify(rd("A")["value"]) if (LOG / "q07_hhc_A.json").is_file() else hhc.chan_A()}
    for i in ("2", "3"):
        b = rd("B" + i)
        for s in ("grav", "ghost", "scalar"):
            P[f"B{i}_{s}"] = sp.sympify(b[s])
        P["D" + i] = sp.sympify(rd("D" + i)["value"])
        P["E" + i] = sig * sp.sympify(rd("E" + i)["value"])
    P["C"] = sp.sympify(rd("C")["value"])
    return P, sig


def row():
    P, sig = parts()
    Rr = sp.expand(sum(P.values()))
    R4 = four_d(Rr)
    # matter (scalar-loop) sector alone: A_scalar + B_scalar
    As = R.strip_t(-sp.I * R.polarise(sp.expand(hhc.load_any("G14") + hhc.load_any("G15")), "a1", "p1", "gc1"))
    matter = sp.expand(As + P["B2_scalar"] + P["B3_scalar"])
    groups = {"A": ["A"], "B": [k for k in P if k.startswith("B")], "C": ["C"], "D": ["D2", "D3"], "E": ["E2", "E3"]}
    muts = {f"drop_{g}": four_d(sp.expand(Rr - sum(P[k] for k in ks))) != 0 for g, ks in groups.items()}
    muts["flip_sigma_F"] = four_d(sp.expand(Rr - 2 * (P["E2"] + P["E3"]))) != 0
    cs = sp.symbols("y0:5")
    G4 = {g: four_d(sp.expand(sum(P[k] for k in ks))) for g, ks in groups.items()}
    tot = sp.expand(sum(c * G4[g] for c, g in zip(cs, groups)))
    gens = sorted([s for s in tot.free_symbols if s not in cs], key=str)
    fit = sp.linsolve(sp.Poly(tot, *gens).coeffs(), cs)
    return {"sigma_F": str(sig), "channel_terms": {k: len(sp.Add.make_args(v)) for k, v in P.items()},
            "residual": str(Rr), "residual_zero": Rr == 0, "residual_4D": str(R4), "residual_4D_zero": R4 == 0,
            "residual_n_terms": len(sp.Add.make_args(Rr)) if Rr != 0 else 0,
            "residual_4D_n_terms": len(sp.Add.make_args(R4)) if R4 != 0 else 0,
            "matter_sector_closes": matter == 0, "matter_sector_4D_closes": four_d(matter) == 0,
            "unconstrained_fit_4D": {"groups": list(groups), "solution": [str(x) for x in list(fit)[0]] if fit else None},
            "mutations_detected": muts}


if __name__ == "__main__":
    r = row()
    (LOG / "q07_hhc_row.json").write_text(json.dumps(r, indent=1) + "\n")
    print({k: (v if not isinstance(v, str) or len(v) < 150 else v[:150]) for k, v in r.items()})
```

code/p6q06/q07/q07_certs.py

```
#!/usr/bin/env python3
"""XVI-Q Q07 certificates: h.h.c row (tree calibration, own Gamma_hh incl. scalar loop, own G14/G15, channel E =
erratum Q06-E2), evanescent CTs (naive test + joint local-CT existence on all kappa^3 three-point rows), status."""
from __future__ import annotations
import hashlib, json, subprocess, sys, time  # noqa: E401
from concurrent.futures import ThreadPoolExecutor
from pathlib import Path
import sympy as sp

HERE = Path(__file__).resolve().parent
Q = HERE.parents[2]
sys.path.insert(0, str(HERE.parents[1] / "p6q05"))
sys.path.insert(0, str(HERE.parent))
sys.path.insert(0, str(HERE))
CERT = Q / "certificates"
```

```
LOG = Q / "build" / "logs"
PY = sys.executable


def sha(s):
    return hashlib.sha256(s.encode()).hexdigest()


def run(args):
    r = subprocess.run([PY, *args], cwd=Q, capture_output=True, text=True)
    if r.returncode:
        raise RuntimeError(f"{args}: {r.stderr[-2000:]}")
    return r.stdout


def main():
    t0 = time.time()
    LOG.mkdir(parents=True, exist_ok=True)
    for f in LOG.glob("q07_*.json"):
        f.unlink()
    with ThreadPoolExecutor(10) as ex:
        list(ex.map(run, [["code/p6q06/q07/hhc.py", k] for k in ("tree", "S", "B2", "B3", "C", "D2", "D3", "E2", "E3")]))
    run(["code/p6q06/q07/hhc.py", "A"])
    run(["code/p6q06/q07/hhc_row.py"])
    run(["code/p6q06/q07/evfit2.py"])
    import evct
    naive = {t: evct.test(t) for t in ("ccK", "ccb")}
    tree = json.loads((LOG / "q07_hhc_tree.json").read_text())
    row = json.loads((LOG / "q07_hhc_row.json").read_text())
    res = sp.sympify(row["residual"])
    g = {k: sp.sympify(json.loads((LOG / f"q07_{k}.json").read_text())["pole_over_U_d4_residue"]) for k in ("G14", "G15")}
    hhc = {"row": "h.h.c: c(gc1) p1, h(a2) p2, h(a3) p3; (S, Gamma_1^pole) at kappa^3",
           "channels": {"A": "Gamma_hhh (G01+G02+G08+G14+G15) at h(p1) -> s0 h(c)",
                        "B": "own Gamma_hh (graviton + ghost + scalar loops) with one leg s1 h = (1/2) L_c h",
                        "C": "delta S_EH3/delta h|_{h2 h3} . Gamma_{Kc} (D13 own)",
                        "D": "delta S_2/delta h|_{h_i} (EH + gauge fixing) . Gamma_{K h_j c} (G03+G07)",
                        "E": "s cbar = sigma_F F(h_i): F . Gamma_{cbar c h_j} (G04+G09) -- erratum Q06-E2 channel"},
           "tree_calibration": {"sigma_F": tree["sigma_F"], "tree_row_closes": tree["closes"] == "True",
                                "tree_row_without_E_nonzero": tree["without_E_zero"] == "False"},
           "scalar_loops_own": {k: {"n_terms": len(sp.Add.make_args(v)), "sha256": sha(str(v))} for k, v in g.items()},
           "matter_sector_closes_exactly": row["matter_sector_closes"],
           "channel_terms": row["channel_terms"], "residual_4D": row["residual_4D"],
           "residual_4D_zero": row["residual_4D_zero"],
           "unconstrained_fit_4D": row["unconstrained_fit_4D"], "mutations_detected": row["mutations_detected"],
           "evanescent_remainder": row["residual"], "evanescent_remainder_sha256": sha(row["residual"]),
           "evanescent_remainder_n_terms": row["residual_n_terms"],
           "evanescent_remainder_every_term_has_T": all(any(s.name.startswith("T_") for s in t.free_symbols)
                                                        for t in sp.Add.make_args(res)) and res != 0}
    (CERT / "Q07_hhc_row.json").write_text(json.dumps(hhc, indent=1) + "\n")
    joint = json.loads((LOG / "q07_evfit_joint.json").read_text())
    for k in list(joint):
        if k.startswith("particular_CT_") and not k.endswith("n_terms"):
            joint[k + "_sha256"] = sha(joint[k])
    ev = {"question": "sigma0-consistency of evanescent kappa^3 counterterms removing the evanescent remainders",
          "naive_p22_type": {"definition": "CT_ev = -(T-part of every loop input), i.e. rows on T-projected inputs",
                             **{t: {"zero": v["zero"], "n_T_terms_left": v["n_T_terms"]} for t, v in naive.items()},
                             "verdict": "NOT sigma0-consistent: tree vertices regenerate T structures from the 4D loop parts"},
          "joint_local_CT": {"definition": "CT = -Gamma_pole + X, X = generic local evanescent (>= one T) kappa^3 "
                                           "monomials for Gamma_{Kch}, Gamma_{cbar ch}, Gamma_{c*cc}, Gamma_hhh (Bose "
                                           "symmetric); the same X_{Kch}, X_{cbar ch} enter c.c.K/c.c.cbar and h.h.c; "
                                           "kappa^2 poles unmodified", **{k: v for k, v in joint.items() if k != "seconds"}},
          "verdict": ("CANCELLABLE: a local, Bose-symmetric, purely evanescent kappa^3 counterterm X exists with "
                      "sigma0-variation cancelling every evanescent remainder on the rows c.c.K, c.c.cbar, h.h.c "
                      "simultaneously (verified by substitution); not unique (free parameters)")
          if joint.get("solvable") and joint.get("verified_rows_zero") and joint.get("hhh_CT_bose_symmetric") and
          joint.get("all_CT_evanescent") else "NOT cancellable in the ansatz (explicit obstruction)",
          "not_tested": ["ghost-equation compatibility of X_{cbar ch} with X_{Kch}",
                         "X beyond three-point (four-point kappa^3 rows)", "finite evanescent remnants (pole level only)"]}
    (CERT / "Q07_evanescent_CT.json").write_text(json.dumps(ev, indent=1) + "\n")
    hhc_ok = hhc["residual_4D_zero"] and all(row["mutations_detected"].values()) and hhc["tree_calibration"]["tree_row_closes"]
    canc = ev["verdict"].startswith("CANCELLABLE")
    st = {"landmark": "XVI-Q", "release": "Q07", "P6_outcome": "Partial",
          "stages": {"row_c_phi_phi": "closed at pole level (Q05)",
                     "row_c_c_cbar": "4D closed (Q06); evanescent remainder cancellable by local evanescent CT (Q07)",
                     "row_c_c_K": "4D closed (Q06); evanescent remainder cancellable by local evanescent CT (Q07)",
                     "row_h_h_c": ("4D closed (Q07, all channels +1, sigma_F tree-calibrated); evanescent remainder "
                                   "cancellable by local evanescent CT (Q07)") if hhc_ok and canc else "OPEN",
                     "graphs_missing": [], "graphs_derived_zero": ["T1", "T2", "T3"]},
          "kappa3_CT_domain_reduction": ("Partial: three-point kappa^3 CT = -Gamma_pole (4D part, sigma0-closed on all "
                                         "four rows) + X (local evanescent, existence shown, non-unique); reduction to "
                                         "a basis of local invariant operators / field redefinitions not performed"),
          "kappa3_defect": "zero on the 4D part of all four three-point rows; evanescent part sigma0-exact (pole level)",
          "kappa3_CT_solve": "solved at pole level (particular solution certified, Q07_evanescent_CT.json)",
          "sigma0_closure_kappa3": ("CLOSED at pole level on the kappa^3 one-loop THREE-POINT rows (c.phi.phi, c.c.cbar, "
```

```
                                "c.c.K, h.h.c) in the enlarged (evanescent) CT domain" if hhc_ok and canc else
                                "NOT established"),
          "not_implied": ["kappa^3 four-point / higher rows", "finite parts", "ghost-equation compatibility of X",
                          "uniqueness of X", "bulk symmetry restoration", "any change to the 0.1 record"],
          "errata": {"Q06-E2": "addressed: the F(h).Gamma_{cbar c h} channel is required (tree: sigma_F = i; mutation "
                               "drop_E detected)"},
          "disclosed_revisions": {"Q07-R1": "disclosed scope revision of the frozen Q06 layer gate "
                                            "(code/p6q06/release_q06.py): files under code/p6q06/q07/ and tools/*q07* "
                                            "are left to the Q07 gate, and the pinned Q05_guard_scan file count is a "
                                            "lower bound (as Q06-R1 did for the Q05 gate). Q05 gate untouched. "
                                            "release_q06.py is pinned by q07_expected.json."},
          "O1_closure_unchanged": "CLOSED (scoped)", "bulk_symmetry_restored": False,
          "flags": {"execution_pass": bool(hhc_ok and canc), "claim_status": "partial",
                    "scope": "pole level, one loop, kappa^3, three-point rows"}, "seconds": round(time.time() - t0, 1)}
    (CERT / "Q07_P6_status.json").write_text(json.dumps(st, indent=1) + "\n")
    print("Q07 certs", st["flags"], st["sigma0_closure_kappa3"][:40], st["seconds"])


if __name__ == "__main__":
    main()
```

code/p6q06/q07/q08/geq.py

```
#!/usr/bin/env python3
"""XVI-Q Q08: ghost-equation map on symbol polynomials.  Gamma_{cbar c h} = rho * Gamma_{K c h}[K -> T(cbar)],
T(cbar) = sym(k, cbar) - (1/2)(k.cbar) eta, k = p2 (K / cbar momentum).  Linear in K: X(K) -> bilinear B(u, w)
(rows.polarise), then X(sym(k, gb)) by u = k, w = gb, and X(eta) = sum_mu B(e_mu, e_mu) by index contraction:
H_u_x H_w_y -> H_x_y, T_u_x T_w_y -> T_x_y, mixed -> 0, H_u_w -> 4, T_u_w -> 0 (d -> 4 residue level; hat/tilde
orthogonal).  Validated against the tensor-machinery map of Q05 ghost_eq3 on the loop poles."""
from __future__ import annotations
import sys
from pathlib import Path
import sympy as sp

HERE = Path(__file__).resolve().parent
sys.path.insert(0, str(HERE.parents[2] / "p6q05"))
import rows as R  # noqa: E402
import uvpole as U  # noqa: E402


def _B(X):
    return sp.expand(R.polarise(sp.expand(X), "aK", "u", "w"))


def _sub_vec(e, name, V):
    rep = {}
    for s in e.free_symbols:
        p = s.name.split("_")
        if len(p) != 3 or name not in p[1:]:
            continue
        o = p[2] if p[1] == name else p[1]
        if o == name:
            continue
        rep[s] = sum(c * (U.H(k, o) if p[0] == "H" else (0 if o.startswith("p") or k.startswith("p") else U.T(k, o)))
                     for k, c in V.items())
    return e.xreplace(rep)


def _trace(B):
    out = 0
    for t in sp.Add.make_args(B):
        c, fu, fw, uw = t, None, None, None
        for f in sp.Mul.make_args(t):
            b, n = f.as_base_exp()
            if not isinstance(b, sp.Symbol):
                continue
            p = b.name.split("_")
            if len(p) != 3:
                continue
            if set(p[1:]) == {"u", "w"}:
                uw = p[0]
                c = c / f
            elif "u" in p[1:]:
                fu = (p[0], p[2] if p[1] == "u" else p[1])
                c = c / f
            elif "w" in p[1:]:
                fw = (p[0], p[2] if p[1] == "w" else p[1])
                c = c / f
        if uw is not None:
            out += c * (4 if uw == "H" else 0)
        elif fu and fw:
            if fu[0] != fw[0]:
                continue
```

```
            if fu[0] == "T" and (fu[1].startswith("p") or fw[1].startswith("p")):
                continue
            out += c * (U.H(fu[1], fw[1]) if fu[0] == "H" else U.T(fu[1], fw[1]))
        else:
            raise ValueError(t)
    return sp.expand(out)


def Tmap(X):
    B = _B(X)
    s = _sub_vec(_sub_vec(B, "u", {"p2": sp.Rational(1, 2)}), "w", {"gb2": 1})
    s2 = _sub_vec(_sub_vec(B, "u", {"gb2": sp.Rational(1, 2)}), "w", {"p2": 1})
    return sp.expand(s + s2 - sp.Rational(1, 2) * U.H("gb2", "p2") * _trace(B))


if __name__ == "__main__":
    K = R.strip_t(R.load("G03") + R.load("G07"))
    C = R.strip_t(R.load("G04") + R.load("G09"))
    TK = Tmap(K)
    print("ratio", sp.cancel(C / TK) if TK != 0 else "0", "ratio G04/G03", sp.cancel(R.strip_t(R.load("G04")) / Tmap(R.strip_t(R.load("G03")))))
```

code/p6q06/q07/q08/q08_certs.py

```
#!/usr/bin/env python3
"""XVI-Q Q08 certificates: ghost-equation compatibility of X, structure of the free directions, status."""
from __future__ import annotations
import hashlib, json, pickle, subprocess, sys, time  # noqa: E401
from pathlib import Path
import sympy as sp
from sympy.polys.matrices import DomainMatrix

HERE = Path(__file__).resolve().parent
Q = HERE.parents[3]
sys.path.insert(0, str(HERE.parents[2] / "p6q05"))
sys.path.insert(0, str(HERE))
CERT = Q / "certificates"
LOG = Q / "build" / "logs"


def run(a):
    r = subprocess.run([sys.executable, *a], cwd=Q, capture_output=True, text=True)
    if r.returncode:
        raise RuntimeError(r.stderr[-2000:])


def main():
    t0 = time.time()
    import rows as R
    from geq import Tmap
    K = R.strip_t(R.load("G03") + R.load("G07"))
    C = R.strip_t(R.load("G04") + R.load("G09"))
    loops = {"rho_total": str(sp.cancel(C / Tmap(K))),
             "rho_G04_G03": str(sp.cancel(R.strip_t(R.load("G04")) / Tmap(R.strip_t(R.load("G03"))))),
             "rho_G09_G07": str(sp.cancel(R.strip_t(R.load("G09")) / Tmap(R.strip_t(R.load("G07")))))}
    run(["code/p6q06/q07/q08/triv.py"])
    run(["code/p6q06/q07/q08/sectors.py"])
    tr = json.loads((LOG / "q08_triv.json").read_text())
    se = json.loads((LOG / "q08_sectors.json").read_text())
    d = pickle.loads((LOG / "q08_triv.pkl").read_bytes())
    AA, bb = d["A"].col_join(d["Ag"]), d["b"].col_join(d["bg"])
    sol = list(sp.linsolve((AA, bb)))
    part = None
    if sol:
        x = sp.Matrix(sol[0])
        x = x.xreplace({s: 0 for s in x.free_symbols})
        part = {"verified_A_x_eq_b": all(sp.expand(v) == 0 for v in (AA * x - bb)),
                "n_nonzero": sum(1 for v in x if v != 0), "sha256": hashlib.sha256(str(list(x)).encode()).hexdigest()}
    geq = {"ghost_equation": "Gamma_{cbar c h} = rho Gamma_{K c h}[K -> T(cbar) = sym(k,cbar) - (1/2)(k.cbar) eta]",
           "loop_poles": loops, "loops_satisfy_with_rho_minus_2": all(v == "-2" for v in loops.values()),
           "constraint_imposed": "X_{cbar ch} = -2 X_{Kch}[K -> T(cbar)]",
           "solvable_with_ghost_equation": tr["solvable_with_ghost_equation"], "particular_solution": part,
           "dim_free_without_ghost_eq": tr["dim_N"], "dim_free_with_ghost_eq": tr["dim_N_with_ghost_eq"],
           "trace_convention": "eta-trace at d4-residue level: hat trace 4, tilde trace 0 (geq.py); validated by "
                               "the loop poles obeying the map with one constant"}
    (CERT / "Q08_ghost_equation.json").write_text(json.dumps(geq, indent=1) + "\n")
    fr = {"homogeneous_space": {"dim_N": tr["dim_N"], "dim_N_with_ghost_eq": tr["dim_N_with_ghost_eq"],
                                "rank_rows_bose": tr["rank_rows_bose"], "n_unknowns": tr["n_unknowns"]},
          "single_sector_blind_directions": se,
          "trivial_families": tr["families"],
          "dim_N_cap_all_families_upper_bound": tr["dim_N_cap_all_families_upper_bound"],
          "dim_N_cap_all_families_best_lambda": tr["dim_N_cap_all_families_best_lambda"],
          "dim_trivial_best_lambda_within_ghost_eq_subspace": se["dim_trivial_best_lambda_cap_N_and_ghost_eq"],
          "reading": ("Of the 152 free directions (104 with the ghost equation), at least 97 (55 within the ghost-equation "
```

```
                    "subspace) lie in the span of fitted BRST-trivial families (field redefinitions h -> h + F(h,h), "
                    "c -> c + G(c,h), gauge-fermion change cbar.Phi(h,h)); at most 116 can. The families are only "
                    "partly closed in N with one uniform lambda (F 65/121, G 11/66, P 12/109): the relative "
                    "normalisations are FITTED, not derived, and the families are not complete at three points. "
                    "Remaining >= 36 directions are not shown trivial: 26 hhh-only sigma0-closed evanescent cubic "
                    "structures (candidate invariant evanescent operators) and blind directions of the three-point rows "
                    "(69 Kch-only, 12 c*cc-only) constrained only by the ghost equation / four-point rows.")}
    (CERT / "Q08_free_directions.json").write_text(json.dumps(fr, indent=1) + "\n")
    ok1 = geq["loops_satisfy_with_rho_minus_2"] and geq["solvable_with_ghost_equation"] and bool(part and part["verified_A_x_eq_b"])
    st = {"landmark": "XVI-Q", "release": "Q08", "P6_outcome": "Partial",
          "goals": {"1_ghost_equation_compatibility": "ESTABLISHED: X can be chosen ghost-equation compatible "
                    "(rho = -2 from the loops); free directions 152 -> 104" if ok1 else "NOT established",
                    "2_invariant_operator_basis": "Partial: no explicit invariant-operator (R^2/Riemann^2-type "
                    "evanescent) basis constructed; three-point cohomology-type count only (26 hhh-only closed "
                    "evanescent cubic structures); X not expressed in an operator basis",
                    "3_free_parameters": "Partial: >= 97 (fitted lambda) of 152 in BRST-trivial family span, <= 116; "
                    ">= 36 uncharacterised; physical vs trivial NOT settled",
                    "4_p22_consistency": "OPEN: not reached (cubic completion of the p22 kappa^2 evanescent "
                    "operators not constructed)"},
          "sigma0_closure_kappa3": "unchanged from Q07: CLOSED at pole level on the one-loop three-point rows "
                                   "(enlarged evanescent CT domain); now also with a ghost-equation-compatible X",
          "not_implied": ["kappa^3 four-point / higher rows", "finite parts", "uniqueness of X",
                          "physical nature of the remaining free directions", "consistency with p22 operators",
                          "bulk symmetry restoration", "any change to the O.1 record"],
          "disclosed_revisions": {"Q08-R1": "disclosed scope revision of the frozen Q07 layer gate "
                                            "(code/p6q06/q07/release_q07.py): files under code/p6q06/q07/q08/ and "
                                            "tools/*q08* are left to the Q08 gate. release_q07.py is pinned by "
                                            "q08_expected.json. Q05 and Q06 gates untouched."},
          "O1_closure_unchanged": "CLOSED (scoped)", "bulk_symmetry_restored": False,
          "flags": {"execution_pass": bool(ok1), "claim_status": "partial",
                    "scope": "pole level, one loop, kappa^3, three-point rows"}, "seconds": round(time.time() - t0, 1)}
    (CERT / "Q08_P6_status.json").write_text(json.dumps(st, indent=1) + "\n")
    print("Q08 certs", st["flags"], st["seconds"])


if __name__ == "__main__":
    main()
```

code/p6q06/q07/q08/q09/diag.py

```
#!/usr/bin/env python3
"""XVI-Q Q09 diagnostic: evaluate the rows on one BRST-trivial family vector, row by row."""
import json, pickle, sys  # noqa: E401
from pathlib import Path
import sympy as sp
HERE = Path(__file__).resolve().parent
Q = HERE.parents[4]
sys.path[:0] = [str(HERE.parents[3] / "p6q05"), str(HERE.parents[2]), str(HERE.parents[1]), str(HERE.parent), str(HERE)]
import evfit2 as E2  # noqa: E402
from evfit import monomials  # noqa: E402
LOG = Q / "build" / "logs"
I = sp.I
MS = {"K": monomials({"aK": 2, "gc1": 1, "a3": 2}, 3), "C": monomials({"gb2": 1, "gc1": 1, "a3": 2}, 4),
      "S": monomials({"z": 1, "gc1": 1, "gc2": 1}, 3), "H": monomials({"a1": 2, "a2": 2, "a3": 2}, 4)}


def split(v):
    out, o = {}, 0
    for k in "KCSH":
        n = len(MS[k])
        out[k] = sp.expand(I * sum(c * m for c, m in zip(v[o:o + n], MS[k])))
        o += n
    return out


def rows(v):
    X = split(v)
    sig = sp.sympify(json.loads((LOG / "q07_hhc_tree.json").read_text())["sigma_F"])
    base = sp.sympify(json.loads((LOG / "q07_hhc_row.json").read_text())["residual"])
    r = {"ccK": E2.ghost_row_total("ccK", X["K"], X["S"]) - E2.ghost_row_total("ccK", 0, 0),
         "ccb": E2.ghost_row_total("ccb", X["C"], X["S"]) - E2.ghost_row_total("ccb", 0, 0),
         "hhc": E2.hhc_ansatz_total(X["H"], X["K"], X["C"], sig) - base}
    return {k: sp.expand(v_) for k, v_ in r.items()}, X


if __name__ == "__main__":
    fam, lam, k = sys.argv[1], sp.sympify(sys.argv[2]), int(sys.argv[3])
    d = pickle.loads((LOG / "q08_triv.pkl").read_bytes())
    p, q = d["fams"][fam][k]
    v = [sp.sympify(a) + lam * sp.sympify(b) for a, b in zip(p, q)]
    r, X = rows(v)
    print({kk: len(sp.Add.make_args(x)) if x != 0 else 0 for kk, x in r.items()})
    print({kk: str(x)[:300] for kk, x in X.items() if x != 0})
```

```
    print({kk: str(x)[:300] for kk, x in r.items() if x != 0})
```

code/p6q06/q07/q08/q09/fam9.py

```
#!/usr/bin/env python3
"""XVI-Q Q09 (a): BRST-trivial families with DERIVED relative normalisation.
sigma(Y) = (S0, Y).  For Y = K.F(h,h): X_hhh = D-op(F) [dS0/dh . dY/dK], X_Khc = A-op(F) [dY/dh . s0 h].  The row
operators applied to X are again A-op and D-op; they commute, so sigma^2 = 0 on the rows forces
row(X) = A(D F) + lambda D(A F) = 0 with lambda = -1 (the antibracket's relative sign between the field and the
antifield derivative of a fermionic Y).  Same for Y = cbar.Phi(h,h) with the E-op (s cbar = sigma_F F(h)):
lambda_P = -1.  Y = c*.G(c,h): both pieces are dY/dc* . dS0/dc (same derivative type), lambda_G = +1 in the Q08
construction's normalisation.  Here: membership of every generator in N = ker(rows, Bose) at the derived lambda,
and the trace content of the failures."""
import json, pickle, sys  # noqa: E401
from pathlib import Path
import sympy as sp
from sympy.polys.matrices import DomainMatrix
HERE = Path(__file__).resolve().parent
Q = HERE.parents[4]
sys.path[:0] = [str(HERE.parents[3] / "p6q05"), str(HERE.parents[2]), str(HERE.parents[1]), str(HERE)]
from evfit import monomials  # noqa: E402
LOG = Q / "build" / "logs"
MS = {"K": monomials({"aK": 2, "gc1": 1, "a3": 2}, 3), "C": monomials({"gb2": 1, "gc1": 1, "a3": 2}, 4),
      "S": monomials({"z": 1, "gc1": 1, "gc2": 1}, 3), "H": monomials({"a1": 2, "a2": 2, "a3": 2}, 4)}
NAMES = [m for k in "KCSH" for m in MS[k]]
TRACE = {"H_aK_aK", "T_aK_aK", "H_a1_a1", "T_a1_a1", "H_a2_a2", "T_a2_a2", "H_a3_a3", "T_a3_a3"}
DERIVED = {"F": -1, "P": -1, "G": 1}


def has_trace(v):
    return any(c != 0 and {s.name for s in NAMES[i].free_symbols} & TRACE for i, c in enumerate(v))


def main():
    d = pickle.loads((LOG / "q08_triv.pkl").read_bytes())
    A = d["A"]
    res = {}
    vecs = []
    for name, pairs in d["fams"].items():
        lam = DERIVED[name]
        n_in, fails, fails_trace = 0, 0, 0
        for p, q in pairs:
            v = sp.Matrix(p) + lam * sp.Matrix(q)
            if all(sp.expand(x) == 0 for x in A * v):
                n_in += 1
                vecs.append(v)
            else:
                fails += 1
                fails_trace += has_trace(list(p) + []) or has_trace(list(q))
        res[name] = {"lambda_derived": str(lam), "n_generators": len(pairs), "n_in_N": n_in, "n_fail": fails,
                     "n_fail_with_trace_monomials": fails_trace}
    if vecs:
        res["dim_trivial_span_derived"] = DomainMatrix.from_Matrix(sp.Matrix.hstack(*vecs)).rank()
        Ag = d["Ag"]
        M = sp.Matrix.hstack(*vecs)
        res["dim_trivial_span_derived_inside_ghost_eq"] = res["dim_trivial_span_derived"] - \
            DomainMatrix.from_Matrix(Ag * M).rank()
    (LOG / "q09_fam.json").write_text(json.dumps(res, indent=1) + "\n")
    print(res)


if __name__ == "__main__":
    main()
```

code/p6q06/q07/q08/q09/guard_scan3.py

```
#!/usr/bin/env python3
"""XVI-Q Q09 scan extension (Q07 review, low): on top of p6q05/guard_scan2 (unchanged), flag
 (i) operator.methodcaller in any form (import, alias, attribute), and
 (ii) coefficient / exponent walks aimed at the opaque d: X.coeff(d..), X.as_coeff_exponent(d), X.as_independent(d),
     X.collect(d), X.as_poly(d), degree(X, d), and dict walks X.as_powers_dict()[d] / .get(d) /
     X.as_coefficients_dict()[d] / .get(d), where the argument is a d-like name (d, D, dim, dsym, d_sym, DIM, D_SYM)
     or an attribute ending in such a name.
Static and best-effort (the runtime guard stays primary); scope: every .py under code/, tools/, tests/ minus the
declared probe files of guard_scan2 and this file's probes."""
from __future__ import annotations
import ast, json, sys  # noqa: E401
from pathlib import Path

HERE = Path(__file__).resolve().parent
Q = HERE.parents[4]
sys.path.insert(0, str(HERE.parents[3] / "p6q05"))
```

```
import guard_scan2 as G2  # noqa: E402
DNAMES = {"d", "D", "dim", "dsym", "d_sym", "DIM", "D_SYM"}
WALK = {"coeff", "as_coeff_exponent", "as_independent", "collect", "as_poly", "degree"}
DICTS = set(json.loads((HERE / "guard_scan3_rules.json").read_text())["dict_walk_methods"])
SELF = "code/p6q06/q07/q08/q09/guard_scan3.py"


def _dlike(n):
    return (isinstance(n, ast.Name) and n.id in DNAMES) or (isinstance(n, ast.Attribute) and n.attr in DNAMES)


def _dictcall(n):
    return isinstance(n, ast.Call) and isinstance(n.func, ast.Attribute) and n.func.attr in DICTS


def hits(tree):
    out = []
    for n in ast.walk(tree):
        if isinstance(n, ast.Name) and n.id == "methodcaller":
            out.append(("methodcaller", n.lineno))
        if isinstance(n, ast.Attribute) and n.attr == "methodcaller":
            out.append(("methodcaller", n.lineno))
        if isinstance(n, (ast.Import, ast.ImportFrom)) and {"methodcaller"} & {a.name for a in n.names}:
            out.append(("methodcaller_import", n.lineno))
        if isinstance(n, ast.Constant) and n.value == "methodcaller":
            out.append(("methodcaller_string", n.lineno))
        if isinstance(n, ast.Call):
            f = n.func
            name = f.attr if isinstance(f, ast.Attribute) else (f.id if isinstance(f, ast.Name) else None)
            if name in WALK and any(_dlike(a) for a in n.args):
                out.append((f"walk_{name}_d", n.lineno))
            if name == "get" and isinstance(f, ast.Attribute) and _dictcall(f.value) and any(_dlike(a) for a in n.args):
                out.append(("dict_walk_get_d", n.lineno))
        if isinstance(n, ast.Subscript) and _dictcall(n.value) and _dlike(n.slice):
            out.append(("dict_walk_subscript_d", n.lineno))
    return out


def files():
    fs = []
    for top in ("code", "tools", "tests"):
        fs += [p for p in (Q / top).rglob("*.py") if "__pycache__" not in p.parts]
    return sorted(p.relative_to(Q).as_posix() for p in fs if p.relative_to(Q).as_posix() not in (SELF,)
                  and p.relative_to(Q).as_posix() not in G2.PROBE_FILES)


PROBES = {
    "methodcaller_attr": "import operator\noperator.methodcaller('subs', d, 4)(e)",
    "methodcaller_alias": "from operator import methodcaller as mc\nmc('subs', d, 4)(e)",
    "methodcaller_getattr": "import operator\ngetattr(operator, 'methodcaller')('subs', d, 4)(e)",
    "coeff_d": "e.coeff(d, 1)",
    "coeff_exponent_d": "e.as_coeff_exponent(d)",
    "independent_d": "e.as_independent(d)",
    "degree_d": "sp.degree(e, d)",
    "powers_dict_subscript": "e.as_powers_dict()[d]",
    "powers_dict_get": "e.as_powers_dict().get(d)",
    "coefficients_dict_get": "e.as_coefficients_dict().get(qg.dsym)",
}


def report():
    hit = {}
    for rel in files():
        h = hits(ast.parse((Q / rel).read_text()))
        if h:
            hit[rel] = h
    probes = {k: bool(hits(ast.parse(v))) for k, v in PROBES.items()}
    base = G2.report()
    return {"scope": "every .py under code/, tools/, tests/ (recursive) minus declared probe files",
            "n_files_scanned": len(files()), "extension_hits": hit, "extension_clean": not hit,
            "probes_flagged": probes, "all_probes_flagged": all(probes.values()),
            "base_scan2_clean": base["scan_clean"], "base_scan2_evasions_flagged": base["all_reviewer_evasions_flagged"]}


if __name__ == "__main__":
    print(json.dumps(report(), indent=1))
```

code/p6q06/q07/q08/q09/q09_certs.py

```
#!/usr/bin/env python3
"""XVI-Q Q09 certificates: Q07-review fixes (sigma0 relabel, Q04 independent report bound by hash, scan extension)
and goal (a) derived normalisations of the BRST-trivial families; (b)-(d) status."""
from __future__ import annotations
```

```
import hashlib, json, subprocess, sys, time  # noqa: E401
from pathlib import Path

HERE = Path(__file__).resolve().parent
Q = HERE.parents[4]
CERT = Q / "certificates"
LOG = Q / "build" / "logs"
sys.path.insert(0, str(HERE))
Q04_REPORT = "reproducibility/independent_rebuild_report_q04.json"
Q04_REPORT_SHA256 = "bc2b16d99f01cf172a5c67f6ee1abb5fdb7b9b5c30e6ad3e510c241ffa442a45"


def main():
    t0 = time.time()
    r = subprocess.run([sys.executable, "code/p6q06/q07/q08/q09/fam9.py"], cwd=Q, capture_output=True, text=True)
    if r.returncode:
        raise RuntimeError(r.stderr[-2000:])
    fam = json.loads((LOG / "q09_fam.json").read_text())
    import guard_scan3
    sc = guard_scan3.report()
    (CERT / "Q09_guard_scan3.json").write_text(json.dumps(sc, indent=1) + "\n")
    q4 = hashlib.sha256((Q / Q04_REPORT).read_bytes()).hexdigest()
    (CERT / "Q09_q04_report_binding.json").write_text(json.dumps(
        {"file": Q04_REPORT, "sha256": q4, "pinned": Q04_REPORT_SHA256, "bound": q4 == Q04_REPORT_SHA256}, indent=1) + "\n")
    tr = json.loads((CERT / "Q08_free_directions.json").read_text())
    fa = {"derivation": ("sigma(Y) = (S0, Y). Y = K.F: X_hhh = D-op(F), X_Khc = A-op(F); the row operators are the same "
                         "A-op and D-op and commute, so sigma^2 = 0 fixes lambda_F = -1 (relative sign of the field / "
                         "antifield derivatives of a fermionic Y); likewise lambda_P = -1 with the E-op; lambda_G = +1 "
                         "(both pieces are dY/dc* . dS0/dc)."),
          "families": {k: v for k, v in fam.items() if k in ("F", "G", "P")},
          "dim_trivial_span_derived": fam.get("dim_trivial_span_derived"),
          "dim_trivial_span_derived_inside_ghost_eq": fam.get("dim_trivial_span_derived_inside_ghost_eq"),
          "Q08_fitted_counts_superseded": {"fitted_lower_bound_in_N": tr["dim_N_cap_all_families_best_lambda"],
                                           "status": "fitted, not derivation-backed; superseded by the derived counts"},
          "diagnosis": ("F at the derived lambda: 139/204 generators in N; every one of the 65 failures carries a "
                        "trace monomial (K or h trace), pointing to the eta-trace / d-convention inside the probe "
                        "contraction rather than to the derivation. The Q08 G and P constructions are NOT closed at the "
                        "derived lambda (0/33, 0/134): their pieces were built by hand and are inconsistent with the row "
                        "operators. Both are OPEN.")}
    (CERT / "Q09_trivial_families.json").write_text(json.dumps(fa, indent=1) + "\n")
    n_g = tr["homogeneous_space"]["dim_N_with_ghost_eq"]
    st = {"landmark": "XVI-Q", "release": "Q09", "P6_outcome": "Partial",
          "sigma0_status_kappa3": "evanescent kappa^3 three-point defect sigma0-exact at pole level; X non-unique "
                                  "(ghost-eq compatible)",
          "relabel_note": "supersedes the Q07/Q08 wording 'sigma0 closure CLOSED at pole level' (Q07 review, medium); "
                          "frozen Q07/Q08 certificates left unchanged",
          "goals": {"a_derived_normalisations": "Partial: lambda_F = lambda_P = -1, lambda_G = +1 derived; F verified on "
                    "139/204 generators (failures all trace-type, OPEN); G, P constructions not closed (OPEN)",
                    "b_classification": f"Partial: derived trivial span {fam.get('dim_trivial_span_derived')} "
                    f"({fam.get('dim_trivial_span_derived_inside_ghost_eq')} inside the ghost-equation subspace of dim "
                    f"{n_g}); the remaining directions are NOT classified (trivial vs physical)",
                    "c_invariant_operator_basis": "OPEN: not constructed; X not expressed in an operator basis",
                    "d_p22_consistency": "OPEN: not reached"},
          "review_fixes": {"sigma0_relabel": True, "Q04_report_bound": q4 == Q04_REPORT_SHA256,
                           "scan_extension": sc["extension_clean"] and sc["all_probes_flagged"] and sc["base_scan2_clean"]},
          "not_implied": ["uniqueness or physicality of X", "kappa^3 four-point / higher rows", "finite parts",
                          "p22 consistency", "bulk symmetry restoration", "any change to the O.1 record"],
          "disclosed_revisions": {"Q09-R1": "disclosed scope revision of the frozen Q08 layer gate "
                                            "(code/p6q06/q07/q08/release_q08.py): files under code/p6q06/q07/q08/q09/ "
                                            "and tools/*q09* are left to the Q09 gate; release_q08.py is pinned by "
                                            "q09_expected.json. Q05-Q07 gates untouched. guard_scan2 (Q05) untouched; "
                                            "the extension is a new Q09 module."},
          "O1_closure_unchanged": "CLOSED (scoped)", "bulk_symmetry_restored": False,
          "flags": {"execution_pass": True, "claim_status": "partial",
                    "scope": "pole level, one loop, kappa^3, three-point rows"}, "seconds": round(time.time() - t0, 1)}
    (CERT / "Q09_P6_status.json").write_text(json.dumps(st, indent=1) + "\n")
    print("Q09 certs", st["review_fixes"], st["seconds"])


if __name__ == "__main__":
    main()
```

code/p6q06/q07/q08/q09/q10/blind10.py

```
#!/usr/bin/env python3
"""XVI-Q Q10: anatomy of the Q08/Q09 BRST-family vectors (reads the frozen q08_triv.pkl; no new loop data).
For every generator (P = first piece, Q = second piece) of F, G, P:
  * blind  : the generator's vector at the derived lambda is supported only on coordinates that the rows+Bose
             system A does not see at all (zero columns of A), i.e. membership in N is vacuous;
  * kpure  : support only in the K sector (X_Khc);
  * in_N   : A(P + lam Q) = 0 at the derived lambda;
  * lam_sol: the set of lambda making A(P + lam Q) = 0 (None = no lambda; 'any' = both pieces in N).
Also: dims of the blind subspace of N, of the derived trivial span modulo blind directions, and the >=36 bound."""
```

```
import json, pickle, sys  # noqa: E401
from pathlib import Path
import sympy as sp
from sympy.polys.matrices import DomainMatrix
HERE = Path(__file__).resolve().parent
Q = HERE.parents[5]
sys.path[:0] = [str(HERE.parents[4] / "p6q05"), str(HERE.parents[3]), str(HERE.parents[2]), str(HERE.parents[1]), str(HERE)]
from evfit import monomials  # noqa: E402
LOG = Q / "build" / "logs"
MS = {"K": monomials({"aK": 2, "gc1": 1, "a3": 2}, 3), "C": monomials({"gb2": 1, "gc1": 1, "a3": 2}, 4),
      "S": monomials({"z": 1, "gc1": 1, "gc2": 1}, 3), "H": monomials({"a1": 2, "a2": 2, "a3": 2}, 4)}
SECT = [k for k in "KCSH" for _ in MS[k]]
DERIVED = {"F": -1, "P": -1, "G": 1}
L = sp.Symbol("lam")


def rk(M):
    return DomainMatrix.from_Matrix(M).rank() if M.shape[1] else 0


def main():
    d = pickle.loads((LOG / "q08_triv.pkl").read_bytes())
    A, Ag = d["A"], d["Ag"]
    n = A.shape[1]
    zero_cols = {j for j in range(n) if all(A[i, j] == 0 for i in range(A.shape[0]))}
    res = {"n_unknowns": n, "n_blind_coordinates": len(zero_cols),
           "blind_coordinates_by_sector": {k: sum(1 for j in zero_cols if SECT[j] == k) for k in "KCSH"}}
    passing, passing_nonblind = [], []
    for name, pairs in d["fams"].items():
        lam = DERIVED[name]
        st = {"n": len(pairs), "in_N": 0, "in_N_blind": 0, "in_N_nonblind": 0, "fail": 0, "fail_kpure": 0,
              "zero_vector": 0, "lam_any": 0, "lam_unique": {}, "lam_none": 0}
        for p, q in pairs:
            P, Qv = sp.Matrix(p), sp.Matrix(q)
            v = P + lam * Qv
            supp = {j for j in range(n) if v[j] != 0}
            if not supp:
                st["zero_vector"] += 1
                continue
            blind = supp <= zero_cols
            kpure = all(SECT[j] == "K" for j in supp)
            r = (A * v).applyfunc(sp.expand)
            if all(x == 0 for x in r):
                st["in_N"] += 1
                passing.append(v)
                if blind:
                    st["in_N_blind"] += 1
                else:
                    st["in_N_nonblind"] += 1
                    passing_nonblind.append(v)
            else:
                st["fail"] += 1
                st["fail_kpure"] += kpure
            ap, aq = (A * P).applyfunc(sp.expand), (A * Qv).applyfunc(sp.expand)
            if all(x == 0 for x in ap) and all(x == 0 for x in aq):
                st["lam_any"] += 1
                continue
            eqs = [sp.expand(a + L * b) for a, b in zip(ap, aq) if sp.expand(a + L * b) != 0]
            sol = sp.solve(eqs, L, dict=True)
            if sol:
                key = str(sol[0][L])
                st["lam_unique"][key] = st["lam_unique"].get(key, 0) + 1
            else:
                st["lam_none"] += 1
        res[name] = st
    Ad = A
    res["dim_N"] = n - rk(Ad)
    res["dim_N_with_ghost_eq"] = n - rk(A.col_join(Ag))
    # blind subspace of N: spanned by unit vectors on zero columns
    res["dim_blind_subspace"] = len(zero_cols)
    Bmat = sp.Matrix.hstack(*[sp.Matrix([1 if i == j else 0 for i in range(n)]) for j in sorted(zero_cols)]) \
        if zero_cols else sp.zeros(n, 0)
    res["dim_blind_inside_ghost_eq"] = Bmat.shape[1] - rk(Ag * Bmat) if Bmat.shape[1] else 0
    if passing:
        M = sp.Matrix.hstack(*passing)
        res["dim_trivial_span_derived"] = rk(M)
        res["dim_trivial_span_derived_mod_blind"] = rk(M.row_join(Bmat)) - Bmat.shape[1]
        res["dim_trivial_span_derived_inside_ghost_eq"] = rk(M) - rk(Ag * M)
        MB = M.row_join(Bmat)
        res["dim_trivial_plus_blind"] = rk(MB)
    (LOG / "q10_blind.json").write_text(json.dumps(res, indent=1, default=str) + "\n")
    print(json.dumps(res, indent=1, default=str))
```

```
if __name__ == "__main__":
    main()
```

code/p6q06/q07/q08/q09/q10/q10_certs.py

```
#!/usr/bin/env python3
"""XVI-Q Q10 certificates: formal erratum on the Q08 trivial counts (and the Q09 '139' count), anatomy of the
family vectors (blind directions, per-generator lambda solutions), restated >=36 bound, new-manuscript intake."""
from __future__ import annotations
import hashlib, json, subprocess, sys, time  # noqa: E401
from pathlib import Path

HERE = Path(__file__).resolve().parent
Q = HERE.parents[5]
CERT = Q / "certificates"
LOG = Q / "build" / "logs"
INTAKE = "intake/new_manuscripts_q10.json"


def main():
    t0 = time.time()
    r = subprocess.run([sys.executable, "code/p6q06/q07/q08/q09/q10/blind10.py"], cwd=Q, capture_output=True, text=True)
    if r.returncode:
        raise RuntimeError(r.stderr[-2000:])
    b = json.loads((LOG / "q10_blind.json").read_text())
    q8 = json.loads((CERT / "Q08_free_directions.json").read_text())
    q9 = json.loads((CERT / "Q09_trivial_families.json").read_text())
    F, Gf, P = b["F"], b["G"], b["P"]
    anat = {"source": "frozen Q08 linear system and family vectors (build/logs/q08_triv.pkl, regenerated by q08_certs)",
            "blind_coordinates": {"n": b["n_blind_coordinates"], "by_sector": b["blind_coordinates_by_sector"],
                                  "dim_inside_ghost_eq": b["dim_blind_inside_ghost_eq"],
                                  "meaning": "X coordinates on which no three-point row and no Bose condition depends "
                                             "(zero columns of A): free at this order, unconstrained by the rows"},
            "families": {k: b[k] for k in ("F", "G", "P")},
            "dim_N": b["dim_N"], "dim_N_with_ghost_eq": b["dim_N_with_ghost_eq"],
            "dim_span_passing": b["dim_trivial_span_derived"],
            "dim_span_passing_mod_blind": b["dim_trivial_span_derived_mod_blind"],
            "dim_span_passing_inside_ghost_eq": b["dim_trivial_span_derived_inside_ghost_eq"],
            "dim_span_passing_plus_blind": b["dim_trivial_plus_blind"],
            "reading": ("F: of 204 generators 15 vanish after the h-leg symmetrisation; for ALL 124 nonzero generators "
                        "that pass, each piece is separately in N (lambda arbitrary), so the lambda test is vacuous on "
                        "them; 62 of them are supported only on blind coordinates. 64 of the 65 failures admit NO "
                        "lambda at all (1 needs lambda=-2). G: 28/33 admit no lambda, 5 need lambda=i/2. P: 132/134 "
                        "admit no lambda, 2 need lambda=-2. The Q08 hand-built pieces are therefore not sigma-images "
                        "of the stated Y for most generators: the defect is in the construction, not in the "
                        "normalisation, and no single normalisation (derived or fitted) can repair it.")}
    (CERT / "Q10_family_anatomy.json").write_text(json.dumps(anat, indent=1) + "\n")
    lb = q8["homogeneous_space"]["dim_N"] - q8["dim_N_cap_all_families_upper_bound"]
    err = {"id": "Q10-E1", "severity": "medium",
           "withdrawn": {"Q08": ["'>=97 directions of N inside the trivial span at the best fitted lambda'",
                                 "'>=55 trivial directions'", "'>=36 unexplained directions' as a count of NON-trivial "
                                 "directions"],
                         "Q09": ["'F verified on 139/204 generators' (15 of the 139 are zero vectors and the other 124 "
                                 "pass for every lambda)", "'derived trivial span 59 (27 inside the ghost-equation "
                                 "subspace)' as a count of BRST-trivial directions"]},
           "reason": anat["reading"],
           "restated_bound": {"statement": "dim N - dim(N cap span{all Q08 family pieces}) = 152 - 116 = 36",
                              "Q08_value_recorded": lb,
                              "caveat": ("lambda-independent and exact as linear algebra, but the spanning pieces are "
                                         "NOT verified sigma-images (anatomy above), so 36 is a lower bound on the "
                                         "directions of N outside that span only; it is neither a lower bound on the "
                                         "non-trivial (physical) directions nor an upper bound on the trivial ones")},
           "what_remains_established": ["dim N = 152, dim N with ghost equation = 104 (Q08, linear algebra)",
                                        f"blind coordinates {b['n_blind_coordinates']} (K {b['blind_coordinates_by_sector']['K']}, "
                                        f"S {b['blind_coordinates_by_sector']['S']}); {b['dim_blind_inside_ghost_eq']} inside the "
                                        "ghost-equation subspace"],
           "frozen_layers": "Q08/Q09 certificates unchanged; this erratum supersedes the listed statements",
           "Q09_superseded_values": {"F_n_in_N": q9["families"]["F"]["n_in_N"], "dim_trivial_span_derived": q9["dim_trivial_span_derived"]}}
    (CERT / "Q10_erratum_Q08_counts.json").write_text(json.dumps(err, indent=1) + "\n")
    nm = json.loads((Q / INTAKE).read_text())
    (CERT / "Q10_new_manuscripts.json").write_text(json.dumps(
        {"intake_file": INTAKE, "sha256": hashlib.sha256((Q / INTAKE).read_bytes()).hexdigest(),
         "head_commit": nm["head_commit"], "ids": [i["id"] for i in nm["items"]],
         "openai": nm["openai"]["reading"][:160], "premises_imported": nm["premises_imported"],
         "relevance_to_Q10": "none (no BRST / DRED / evanescent / counterterm content); cited as context only",
         "XV_B_opened": False}, indent=1) + "\n")
    st = {"landmark": "XVI-Q", "release": "Q10", "P6_outcome": "Partial",
          "sigma0_status_kappa3": "evanescent kappa^3 three-point defect sigma0-exact at pole level; X non-unique "
                                  "(ghost-eq compatible)",
          "goals": {"erratum_Q08_counts": "DONE: Q10-E1 withdraws the Q08 >=97/>=55/>=36 trivial counts and the Q09 "
                                          "139 / 59 counts",
                    "K_sector_blind_directions": f"DONE: {b['n_blind_coordinates']} blind coordinates (K "
                    f"{b['blind_coordinates_by_sector']['K']}, S {b['blind_coordinates_by_sector']['S']}); 62 of the 124 "
```

```
                    "passing F generators live only there",
                    "bound_36_restated": "DONE: exact, but only a bound on N outside the (unverified) family span",
                    "F_trace_failures": "OPEN: 64/65 failures admit no lambda; the defect is in the Q08 construction; "
                                        "the eta-trace hypothesis is neither confirmed nor excluded",
                    "G_P_families": "OPEN: Q08 constructions shown not to be sigma-images (no lambda for 28/33, 132/134); "
                                    "not rebuilt and not proven absent",
                    "classification_remaining": "OPEN",
                    "invariant_evanescent_basis": "OPEN: not constructed",
                    "p22_kappa2_consistency": "OPEN: not reached"},
          "new_manuscripts": "En05-A/B/C, En06-A/B, En07-A and the OpenAI separation edges read; none imported",
          "not_implied": ["uniqueness or physicality of X", "any count of BRST-trivial directions",
                          "kappa^3 four-point / higher rows", "p22 consistency", "bulk symmetry restoration",
                          "any change to the 0.1 record"],
          "disclosed_revisions": {"Q10-R1": "disclosed scope revision of the frozen Q09 layer gate "
                                            "(code/p6q06/q07/q08/q09/release_q09.py): files under "
                                            "code/p6q06/q07/q08/q09/q10/ and tools/*q10* are left to the Q10 gate; "
                                            "and the Q09 gate treats n_files_scanned of Q09_guard_scan3 as a lower bound (new Q10 files are scanned,
not exempted); release_q09.py is pinned by q10_expected.json. Q05-Q08 gates untouched."},
          "errata": ["Q10-E1"], "O1_closure_unchanged": "CLOSED (scoped)", "bulk_symmetry_restored": False,
          "flags": {"execution_pass": True, "claim_status": "partial",
                    "scope": "pole level, one loop, kappa^3, three-point rows"}, "seconds": round(time.time() - t0, 1)}
    (CERT / "Q10_P6_status.json").write_text(json.dumps(st, indent=1) + "\n")
    print("Q10 certs", st["seconds"])


if __name__ == "__main__":
    main()
```

code/p6q06/q07/q08/q09/q10/q11/class11.py

```
#!/usr/bin/env python3
"""XVI-Q Q11 goal 2 (conditional): decomposition of N_g (rows + Bose + ghost equation) into
 (i) the span of the Ktilde-based sigma-images that pass (derived signs), (ii) blind coordinates, (iii) the rest."""
import json, pickle  # noqa: E401
from pathlib import Path
import sympy as sp
from sympy.polys.matrices import DomainMatrix
HERE = Path(__file__).resolve().parent
LOG = HERE.parents[6] / "build" / "logs"


def rk(M):
    return DomainMatrix.from_Matrix(M).rank() if M.shape[1] else 0


def main():
    d = pickle.loads((LOG / "q08_triv.pkl").read_bytes())
    A, Ag = d["A"], d["Ag"]
    n = A.shape[1]
    keep = pickle.loads((LOG / "q11_sigma_vecs.pkl").read_bytes())
    V = keep[("F", "derived_minus1")] + keep[("G", "derived_minus1")]
    M = sp.Matrix.hstack(*[sp.Matrix(v) for v in V])
    zc = [j for j in range(n) if all(A[i, j] == 0 for i in range(A.shape[0]))]
    Bm = sp.Matrix.hstack(*[sp.Matrix([1 if i == j else 0 for i in range(n)]) for j in zc])
    Bg = Bm.shape[1] - rk(Ag * Bm)                      # blind directions inside the ghost-eq subspace
    full = A.col_join(Ag)
    dimNg = n - rk(full)
    r_triv = rk(M)
    r_tb = rk(M.row_join(Bm))
    # blind directions satisfying the ghost equation: kernel of Ag restricted to blind coords
    ker = (Ag * Bm).nullspace()
    Bk = sp.Matrix.hstack(*[Bm * k for k in ker]) if ker else sp.zeros(n, 0)
    r_tbg = rk(M.row_join(Bk))
    res = {"dim_N_g": dimNg, "dim_sigma_span_passing": r_triv, "dim_blind_in_N_g": Bg,
           "dim_sigma_plus_blindg": r_tbg, "dim_sigma_cap_blindg": r_triv + Bg - r_tbg,
           "dim_remaining_N_g_mod_(sigma+blind)": dimNg - r_tbg,
           "n_passing_vectors": {"F": len(keep[("F", "derived_minus1")]), "G": len(keep[("G", "derived_minus1")])},
           "dim_sigma_plus_all_blind": r_tb}
    (LOG / "q11_class.json").write_text(json.dumps(res, indent=1) + "\n")
    print(res)


if __name__ == "__main__":
    main()
```

code/p6q06/q07/q08/q09/q10/q11/diag11.py

```
#!/usr/bin/env python3
"""XVI-Q Q11 diagnostic: for every Q08 family generator, which block of the linear system (row ccK, row ccb,
row hhc, Bose symmetry of X_hhh) the two pieces violate."""
import json, pickle, sys  # noqa: E401
from pathlib import Path
```

```
import sympy as sp
HERE = Path(__file__).resolve().parent
Q = HERE.parents[6]
sys.path[:0] = [str(HERE.parents[5] / "p6q05"), str(HERE.parents[4]), str(HERE.parents[3]), str(HERE.parents[2])]
import rows as R  # noqa: E402
import evfit2 as E2  # noqa: E402
from evfit import ansatz  # noqa: E402
LOG = Q / "build" / "logs"


def blocks():
    XK, xk = ansatz({"aK": 2, "gc1": 1, "a3": 2}, 3, "xk")
    XC, xc = ansatz({"gb2": 1, "gc1": 1, "a3": 2}, 4, "xc")
    XS, xs = ansatz({"z": 1, "gc1": 1, "gc2": 1}, 3, "xs")
    XH, xh = ansatz({"a1": 2, "a2": 2, "a3": 2}, 4, "xh")
    sig = sp.sympify(json.loads((LOG / "q07_hhc_tree.json").read_text())["sigma_F"])
    xall = xk + xc + xs + xh
    X = set(xall)
    out = {}
    rows = {"ccK": E2.ghost_row_total("ccK", XK, XS) - E2.ghost_row_total("ccK", sp.Integer(0), sp.Integer(0)),
            "ccb": E2.ghost_row_total("ccb", XC, XS) - E2.ghost_row_total("ccb", sp.Integer(0), sp.Integer(0)),
            "hhc": E2.hhc_ansatz_total(XH, XK, XC, sig) - E2.hhc_ansatz_total(sp.Integer(0), sp.Integer(0), sp.Integer(0), sig)}
    for k, r in rows.items():
        r = sp.expand(r)
        gens = sorted([s for s in r.free_symbols if s not in X], key=str)
        out[k] = sp.linear_eq_to_matrix(sp.Poly(r, *gens).coeffs(), xall)[0]
    beq = []
    for mp in ({"p1": R.P2, "p2": R.P1, "a1": "a2", "a2": "a1"}, {"p1": R.P3, "p2": R.P2, "a1": "a3", "a3": "a1"}):
        d = sp.expand(XH - R.remap(XH, mp))
        gens = sorted([s for s in d.free_symbols if s not in X], key=str)
        beq += sp.Poly(d, *gens).coeffs()
    out["bose"] = sp.linear_eq_to_matrix(beq, xall)[0]
    return out


def main():
    B = blocks()
    pickle.dump(B, open(LOG / "q11_blocks.pkl", "wb"))
    d = pickle.loads((LOG / "q08_triv.pkl").read_bytes())
    res = {}
    for name, pairs in d["fams"].items():
        cnt = {}
        for p, q in pairs:
            P, Qv = sp.Matrix(p), sp.Matrix(q)
            if not any(P) and not any(Qv):
                continue
            key = []
            for b, M in B.items():
                ap = any(sp.expand(x) != 0 for x in M * P)
                aq = any(sp.expand(x) != 0 for x in M * Qv)
                key.append(f"{b}:{'P' if ap else ''}{'Q' if aq else ''}")
            k = " ".join(key)
            cnt[k] = cnt.get(k, 0) + 1
        res[name] = cnt
    (LOG / "q11_diag.json").write_text(json.dumps(res, indent=1) + "\n")
    print(json.dumps(res, indent=1))


if __name__ == "__main__":
    main()
```

code/p6q06/q07/q08/q09/q10/q11/q11_certs.py

```
#!/usr/bin/env python3
"""XVI-Q Q11 certificates: sigma-images in the gauge-fixed BV variables (Ktilde = K + cbar.F'), diagnosis of the
Q08 F defect, conditional classification of the ghost-equation free space N_g, status."""
from __future__ import annotations
import json, subprocess, sys, time  # noqa: E401
from pathlib import Path

HERE = Path(__file__).resolve().parent
Q = HERE.parents[6]
CERT = Q / "certificates"
LOG = Q / "build" / "logs"
STEPS = ["diag11.py", "sigma11.py", "scan11.py", "trace11.py", "class11.py"]


def main():
    t0 = time.time()
    for s in STEPS:
        r = subprocess.run([sys.executable, str(HERE / s)], cwd=Q, capture_output=True, text=True)
        if r.returncode:
            raise RuntimeError(s + r.stderr[-2000:])
```

```
    diag = json.loads((LOG / "q11_diag.json").read_text())
    sig = json.loads((LOG / "q11_sigma.json").read_text())
    scan = json.loads((LOG / "q11_scan.json").read_text())
    tr = json.loads((LOG / "q11_trace.json").read_text())
    cl = json.loads((LOG / "q11_class.json").read_text())
    sig.pop("seconds", None)
    si = {"construction": ("Y built from Ktilde = K + cbar.F'(h) (canonical shift by Psi = cbar.F, B integrated out); "
                           "the cbar partner of every K-functional is rho*T(.), rho = -2 (Q08 ghost equation). "
                           "sigma = (S0, .) applied with the row channel operators (D, A, E), antibracket sign -1 "
                           "between antifield-type and field-type pieces; G family includes the c*cc piece "
                           "(h -> s0 h, antisymmetrised in the two c's)."),
          "Q08_F_defect_diagnosis": {"blocks_violated_by_Q08_pieces": diag,
                                     "reading": ("the Q08 F pieces violate only the h.h.c row; 12 generators fail through "
                                                 "the antifield-type piece alone, i.e. Q08 F omitted the cbar partner "
                                                 "(Ktilde) and the E channel")},
          "results_at_derived_signs": sig,
          "lam_mu_scan_diagnostic": scan,
          "trace_correlation": tr,
          "reading": ("With the Ktilde construction 71 of 194 nonzero F images and 8 of 33 G images lie in N (all of them "
                      "also satisfy the ghost equation). The remaining failures are all in the h.h.c row (G also in the "
                      "ghost rows). No single normalisation works for every generator (lam/mu scan), and failures occur "
                      "with and without trace monomials, so the eta-trace hypothesis is NOT the (sole) cause. The "
                      "row operators as inserted on evanescent X do not compose to zero with this sigma: sigma^2 = 0 "
                      "is not realised on the evanescent sector of the h.h.c row. Diagnosis: OPEN (candidate: the "
                      "4D-momentum rule T(p, .) = 0 for hat components, under which s0 h_hat = 0, versus the "
                      "d-dimensional gauge structure of the kinetic operator).")}
    (CERT / "Q11_sigma_images.json").write_text(json.dumps(si, indent=1) + "\n")
    cla = {"N_g": cl["dim_N_g"], "sigma_span_passing": cl["dim_sigma_span_passing"],
           "blind_in_N_g": cl["dim_blind_in_N_g"], "sigma_cap_blind": cl["dim_sigma_cap_blindg"],
           "sigma_plus_blind": cl["dim_sigma_plus_blindg"], "unclassified": cl["dim_remaining_N_g_mod_(sigma+blind)"],
           "status": ("CONDITIONAL: the 46 directions are sigma-images of local Ktilde-functionals that lie in N_g, hence "
                      "trivial IF this sigma is the correct linearised Slavnov-Taylor operator on the evanescent sector; "
                      "that is exactly what the failures above put in doubt. 24 blind directions (9 overlapping the "
                      "sigma span). 43 directions are unclassified: NOT shown to be physical."),
           "physical_evanescent_directions": "not established (no direction certified physical)"}
    (CERT / "Q11_classification.json").write_text(json.dumps(cla, indent=1) + "\n")
    st = {"landmark": "XVI-Q", "release": "Q11", "P6_outcome": "Partial",
          "sigma0_status_kappa3": "evanescent kappa^3 three-point defect sigma0-exact at pole level; X non-unique "
                                  "(ghost-eq compatible)",
          "goals": {"1_sigma_images": "Partial: genuine Ktilde construction built (F, G; P absorbed as the cbar partner of "
                                      "F); 71/194 F and 8/33 G in N; Q08 F defect diagnosed (missing cbar partner / E "
                                      "channel); remaining failures OPEN (not trace-driven)",
                    "2_classification": "Partial (conditional): N_g 104 = sigma span 46 + blind 24 - overlap 9 + 43 "
                                        "unclassified",
                    "3_invariant_basis": "OPEN: not constructed",
                    "3_p22_consistency": "OPEN: not reached"},
          "not_implied": ["uniqueness or physicality of X", "an unconditional count of BRST-trivial directions",
                          "any physical evanescent direction", "p22 consistency", "bulk symmetry restoration",
                          "any change to the O.1 record"],
          "disclosed_revisions": {"Q11-R1": "disclosed scope revision of the frozen Q10 layer gate "
                                            "(code/p6q06/q07/q08/q09/q10/release_q10.py): files under "
                                            "code/p6q06/q07/q08/q09/q10/q11/ and tools/*q11* are left to the Q11 gate; "
                                            "and the Q10 gate treats n_files_scanned of Q09_guard_scan3 as a lower bound (new Q11 files are scanned,
not exempted); release_q10.py is pinned by q11_expected.json. Q05-Q09 gates untouched."},
          "premises_note": "OpenAI catalogue and En05-En07 sheets are not premises (Q10 intake)",
          "O1_closure_unchanged": "CLOSED (scoped)", "bulk_symmetry_restored": False, "insertion": "frozen",
          "flags": {"execution_pass": True, "claim_status": "partial",
                    "scope": "pole level, one loop, kappa^3, three-point rows"}, "seconds": round(time.time() - t0, 1)}
    (CERT / "Q11_P6_status.json").write_text(json.dumps(st, indent=1) + "\n")
    print("Q11 certs", st["seconds"])


if __name__ == "__main__":
    main()
```

code/p6q06/q07/q08/q09/q10/q11/release_q11.py

```
#!/usr/bin/env python3
"""XVI-Q Q11 release gate (fails closed; on reject no RELEASE_STATUS.json is left).
(1) code/p6q06/q07/q08/q09/q10/q11/q11_expected.json pinned; EVERY Q01..Q11 certificate equals its pinned copy (Q05 guard-scan
    file count: lower bound); EVERY code/**/*.py, tools/*.py, tests/*.py (incl. the Q05 and Q06 gates) pinned and
    re-hashed; an unpinned file is a reject;
(2) live hardened static scan clean;
(3) semantic: h.h.c tree calibrated, 4D residual 0, every mutation detected, unique fit, matter sector exact,
    remainder explicit and evanescent; evanescent-CT naive test recorded as NOT consistent; joint local CT solvable,
    verified, Bose, evanescent, CT shas; status consistent; O.1 unchanged; Q06 layer accepted; Q07-R1 disclosed."""
from __future__ import annotations
import hashlib, json, sys  # noqa: E401
from pathlib import Path

Q = Path(__file__).resolve().parents[7]
sys.path.insert(0, str(Q / "tools"))
import source_set_q06_q07_q08_q09_q10_q11 as SS  # noqa: E402
```

```
CERT = Q / "certificates"
EXPECTED_SHA256 = "fcddf961c3b758695b60b25d0a5728c10df5d36b7e12829e2943876d31db9d68"
STATUS = (Q / "RELEASE_STATUS.json", CERT / "RELEASE_STATUS.json")
HARD_STOPS = {"bulk_symmetry_restored": False, "insertion": "frozen", "XV_B_opened": False, "XIV_J_opened": False,
              "Route_B_discharged": False, "Missing_never_zero": True}
READS: set[str] = set()
Q07_CERTS = ["Q07_hhc_row", "Q07_evanescent_CT", "Q07_P6_status", "Q07_release"]
Q08_CERTS = ["Q08_ghost_equation", "Q08_free_directions", "Q08_P6_status", "Q08_release"]
Q09_CERTS = ["Q09_guard_scan3", "Q09_q04_report_binding", "Q09_trivial_families", "Q09_P6_status"]
Q10_CERTS = ["Q10_family_anatomy", "Q10_erratum_Q08_counts", "Q10_new_manuscripts", "Q10_P6_status"]
Q11_CERTS = ["Q11_sigma_images", "Q11_classification", "Q11_P6_status"]
Q04_REPORT_SHA256 = "bc2b16d99f01cf172a5c67f6ee1abb5fdb7b9b5c30e6ad3e510c241ffa442a45"
SELF = "code/p6q06/q07/q08/q09/q10/q11/release_q11.py"


def rd(rel):
    READS.add(rel)
    return (Q / rel).read_bytes()


def jr(rel):
    return json.loads(rd(rel))


def sha_b(b):
    return hashlib.sha256(b).hexdigest()


def strip(o):
    if isinstance(o, dict):
        return {k: strip(v) for k, v in o.items() if k != "seconds"}
    if isinstance(o, list):
        return [strip(v) for v in o]
    return o


def code_files():
    out = {p.relative_to(Q).as_posix() for p in (Q / "code").rglob("*.py") if "__pycache__" not in p.parts}
    out |= {p.relative_to(Q).as_posix() for p in (Q / "tools").glob("*.py")}
    out |= {p.relative_to(Q).as_posix() for p in (Q / "tests").glob("*.py")}
    return out - {SELF}


def checks():
    c = {}
    ep = "code/p6q06/q07/q08/q09/q10/q11/q11_expected.json"
    eb = rd(ep) if (Q / ep).is_file() else b""
    c["expected_file_pinned"] = bool(eb) and sha_b(eb) == EXPECTED_SHA256
    exp = json.loads(eb) if eb else {}
    certs = exp.get("certificates", {})

    def _layer(n, o):
        if n in ("Q05_guard_scan", "Q09_guard_scan3") and isinstance(o, dict):  # file counts are lower bounds (Q11-R1)
            o = dict(o)
            ok_n = isinstance(o.get("n_files_scanned"), int) and o["n_files_scanned"] >= certs[n]["n_files_scanned"]
            o["n_files_scanned"] = certs[n]["n_files_scanned"] if ok_n else -1
        return o
    bad = [n for n, ref in certs.items() if not (CERT / f"{n}.json").is_file()
           or _layer(n, strip(jr(f"certificates/{n}.json"))) != ref]
    c["every_certified_value_matches_pinned"] = len(certs) >= 44 and set(Q07_CERTS + Q08_CERTS + Q09_CERTS + Q10_CERTS + Q11_CERTS + ["Q09_release",
"Q10_release"]) <= set(certs) and not bad
    fs = exp.get("file_sha256", {})
    c["pinned_files_rehashed"] = bool(fs) and all((Q / f).is_file() and sha_b(rd(f)) == h for f, h in fs.items())
    c["no_unpinned_code_tools_tests_file"] = bool(fs) and code_files() <= set(fs) and \
        {"code/p6q05/release_q05.py", "code/p6q06/release_q06.py", "code/p6q06/q07/release_q07.py", "code/p6q06/q07/q08/release_q08.py",
"code/p6q06/q07/q08/q09/release_q09.py", "code/p6q06/q07/q08/q09/q10/release_q10.py"} <= set(fs)
    sys.path.insert(0, str(Q / "code" / "p6q05"))
    import guard_scan2
    READS.add("code/p6q05/guard_scan2.py")
    live = guard_scan2.report()
    c["scan_live_clean_incl_q07"] = live["scan_clean"] is True and live["all_reviewer_evasions_flagged"] is True
    sys.path.insert(0, str(Q / "code/p6q06/q07/q08/q09"))  # scan3 module (pinned)
    import guard_scan3
    READS.add("code/p6q06/q07/q08/q09/guard_scan3.py")
    READS.add("code/p6q06/q07/q08/q09/guard_scan3_rules.json")
    s3 = guard_scan3.report()
    c["scan3_live_clean_probes_flagged"] = s3["extension_clean"] is True and s3["all_probes_flagged"] is True and \
        s3["base_scan2_clean"] is True and len(s3["probes_flagged"]) >= 10
    c["Q04_report_bound_by_hash"] = sha_b(rd("reproducibility/independent_rebuild_report_q04.json")) == Q04_REPORT_SHA256
    a = jr("certificates/Q10_family_anatomy.json")
    fm = a["families"]
    c["anatomy_values"] = a["blind_coordinates"]["n"] == 42 and a["blind_coordinates"]["by_sector"] == \
        {"K": 30, "C": 0, "S": 12, "H": 0} and a["blind_coordinates"]["dim_inside_ghost_eq"] == 24 and \
        fm["F"]["n"] == 204 and fm["F"]["zero_vector"] == 15 and fm["F"]["in_N"] == 124 and \
```

```
        fm["F"]["lam_any"] == 124 and fm["F"]["in_N_blind"] == 62 and fm["F"]["lam_none"] == 64 and \
        fm["G"]["lam_none"] == 28 and fm["G"]["lam_unique"] == {"I/2": 5} and fm["P"]["lam_none"] == 132 and \
        fm["P"]["lam_unique"] == {"-2": 2} and a["dim_N"] == 152 and a["dim_N_with_ghost_eq"] == 104
    e = jr("certificates/Q10_erratum_Q08_counts.json")
    c["erratum_withdraws_counts"] = e["id"] == "Q10-E1" and len(e["withdrawn"]["Q08"]) == 3 and \
        len(e["withdrawn"]["Q09"]) == 2 and e["restated_bound"]["Q08_value_recorded"] == 36 and \
        "NOT verified" in e["restated_bound"]["caveat"]
    nm = jr("certificates/Q10_new_manuscripts.json")
    c["new_manuscripts_context_only"] = nm["premises_imported"] == [] and nm["XV_B_opened"] is False and \
        nm["sha256"] == sha_b(rd("intake/new_manuscripts_q10.json"))
    si = jr("certificates/Q11_sigma_images.json")
    r = si["results_at_derived_signs"]
    c["sigma_image_values"] = r["F"]["n"] == 204 and r["F"]["derived_minus1"]["in_N"] == 71 and \
        r["F"]["derived_minus1"]["in_N_and_ghost_eq"] == 71 and r["F"]["derived_minus1"]["fail_blocks"] == {"hhc": 123} \
        and r["G"]["n"] == 33 and r["G"]["derived_minus1"]["in_N"] == 8 and r["F"]["leak"] == 0 and r["G"]["leak"] == 0 \
        and "OPEN" in si["reading"] and "NOT the (sole) cause" in si["reading"]
    cl = jr("certificates/Q11_classification.json")
    c["classification_values"] = cl["N_g"] == 104 and cl["sigma_span_passing"] == 46 and cl["blind_in_N_g"] == 24 and \
        cl["sigma_cap_blind"] == 9 and cl["unclassified"] == 43 and cl["status"].startswith("CONDITIONAL") and \
        cl["physical_evanescent_directions"].startswith("not established")
    st = jr("certificates/Q11_P6_status.json")
    gl = st["goals"]
    c["status_scoped_honest"] = st["P6_outcome"] == "Partial" and st["flags"]["claim_status"] == "partial" and \
        st["sigma0_status_kappa3"] == ("evanescent kappa^3 three-point defect sigma0-exact at pole level; X non-unique "
                                       "(ghost-eq compatible)") and gl["1_sigma_images"].startswith("Partial") and \
        gl["2_classification"].startswith("Partial") and gl["3_invariant_basis"].startswith("OPEN") and \
        gl["3_p22_consistency"].startswith("OPEN") and st["insertion"] == "frozen" and \
        "any physical evanescent direction" in st["not_implied"]
    c["revision_disclosed"] = "Q11-R1" in st["disclosed_revisions"]
    q10s = jr("certificates/Q10_P6_status.json")
    c["Q10_erratum_kept"] = "Q10-E1" in q10s["errata"]
    q10r = jr("certificates/Q10_release.json") if (CERT / "Q10_release.json").is_file() else {}
    c["Q10_layer_accepted"] = q10r.get("decision") == "ACCEPT"
    q8 = jr("certificates/Q08_P6_status.json")
    c["Q08_result_unchanged"] = q8["goals"]["1_ghost_equation_compatibility"].startswith("ESTABLISHED")
    o1 = jr("certificates/Q03_O1_record.json")
    c["O1_record_unchanged"] = o1["O1_closure"] == "CLOSED (scoped)" and o1["O1_closure_confirmed"] is True and \
        st["O1_closure_unchanged"] == "CLOSED (scoped)" and st["bulk_symmetry_restored"] is False
    c["gate_reads_in_source_set"] = READS <= set(SS.source_files(Q)) | {f"certificates/{n}.json" for n in certs} | \
        {"certificates/Q03_O1_record.json", "certificates/Q10_release.json", "intake/new_manuscripts_q10.json"}
    return c, st


def main():
    for p in STATUS:
        p.unlink(missing_ok=True)
    try:
        c, st = checks()
    except Exception as e:  # noqa: BLE001
        c, st = {"exception": False, "error": repr(e)}, {}
    ok = all(v is True for k, v in c.items() if k != "error")
    rec = {"landmark": "XVI-Q", "release": "Q11", "decision": "ACCEPT" if ok else "REJECT", "checks": c}
    (CERT / "Q11_release.json").write_text(json.dumps(rec, indent=1) + "\n")
    if not ok:
        print(json.dumps(rec, indent=1))
        return 1
    status = {"landmark": "XVI-Q", "release": "Q11", "status": "released (honest partial: sigma-images in Ktilde variables, conditional
classification)",
              "P6_outcome": st["P6_outcome"], "sigma0_status_kappa3": st["sigma0_status_kappa3"],
              "O1_closure": "CLOSED (scoped)", "O1_closure_confirmed": True, "hard_stops": HARD_STOPS,
              "independent_rebuild_q05": "not requested", "independent_rebuild_q06": "not requested",
              "independent_rebuild_q07": "PASSED (reviewer; report sha 03f5393f)", "independent_rebuild_q08": "PASSED on execution (reviewer; report
sha cc67001e)", "independent_rebuild_q09": "not requested", "independent_rebuild_q10": "not requested", "independent_rebuild_q11": "not requested",
"goals": st["goals"], "execution_pass": True, "claim_status": "partial", "gate": SELF}
    for p in STATUS:
        p.write_text(json.dumps(status, indent=1) + "\n")
    (Q / "ledgers").mkdir(exist_ok=True)
    (Q / "ledgers" / "Q11_milestones.json").write_text(json.dumps({"landmark": "XVI-Q", "release": "Q11", **strip(st)},
                                                                  indent=1) + "\n")
    print("ACCEPT")
    return 0


if __name__ == "__main__":
    raise SystemExit(main())
```

code/p6q06/q07/q08/q09/q10/q11/scan11.py

```
#!/usr/bin/env python3
"""XVI-Q Q11 diagnostic (not used for any claim of triviality): per F generator, the set of (lam, mu) with
rows(D F + mu E phi + lam (A F + mu A phi)) = 0, to locate the normalisation that the derivation must reproduce."""
import json, pickle, sys  # noqa: E401
from pathlib import Path
```

```
import sympy as sp
HERE = Path(__file__).resolve().parent
sys.path.insert(0, str(HERE))
import sigma11 as S  # noqa: E402
L, M = sp.symbols("lam mu")


def main():
    S.SIG = sp.sympify(json.loads((S.LOG / "q07_hhc_tree.json").read_text())["sigma_F"])
    A = pickle.loads((S.LOG / "q08_triv.pkl").read_bytes())["A"]
    sols = {}
    for m in S.monomials({"aK": 2, "a2": 2, "a3": 2}, 2):
        f = sp.expand(m + S.R.remap(m, S.SW23))
        if f == 0:
            continue
        phi = S.cbar_partner(f)
        dF = sp.Matrix(S.vec("H", S.D_op(f))[0])
        aF = sp.Matrix(S.vec("K", S.A_on_K(f))[0])
        eP = sp.Matrix(S.vec("H", S.E_op(phi))[0]) if phi != 0 else sp.zeros(S.NDIM, 1)
        aP = sp.Matrix(S.vec("C", S.A_on_C(phi))[0]) if phi != 0 else sp.zeros(S.NDIM, 1)
        r = (A * dF + M * (A * eP) + L * (A * aF) + L * M * (A * aP)).applyfunc(sp.expand)
        eqs = [e for e in r if e != 0]
        if not eqs:
            k = "any"
        else:
            s = sp.solve(eqs, [L, M], dict=True)
            k = str(sorted([(str(x.get(L, "free")), str(x.get(M, "free"))) for x in s])) if s else "none"
        sols[k] = sols.get(k, 0) + 1
    (S.LOG / "q11_scan.json").write_text(json.dumps(sols, indent=1) + "\n")
    print(json.dumps(sols, indent=1))


if __name__ == "__main__":
    main()
```

code/p6q06/q07/q08/q09/q10/q11/sigma11.py

```
#!/usr/bin/env python3
"""XVI-Q Q11 goal 1: BRST-trivial directions as genuine sigma-images in the gauge-fixed BV variables.
Gauge fixing by the canonical shift with Psi = cbar.F(h) (B integrated out) means the antifield of h enters only
through Ktilde = K + cbar.F'(h), and the ghost equation (Q08, rho = -2) is exactly this statement on the amplitudes:
Gamma_{cbar..} = rho * Gamma_{K..}[K -> T(cbar)].  A local gh -1 functional must therefore be built from Ktilde:
  Y_F = Ktilde.F(h,h) :  (Y_K, Y_cbar) = (K.F, rho*T(K.F))   -- Q08 kept only K.F (no cbar part)
  Y_G = c*.G(c,h)     :  sigma components X_Khc (c* -> D^T Ktilde), X_cbar ch = rho*T(X_Khc), X_c*cc (h -> s0 h)
                         -- Q08 had no X_c*cc piece and a hand-made cbar piece
sigma = (S0, .) is then applied with the SAME channel operators that build the rows (D-op: K -> dS2/dh; A-op:
h -> s0 h; E-op: cbar -> sigma_F F(h)), with the antibracket's relative sign -1 between antifield-type (D, E) and
field-type (A) pieces (Q09 derivation).  No per-generator fitting: one test per generator of
   rows(X) = 0, Bose(X) = 0, ghost-equation(X) = 0.
The only convention not fixed a priori is the sign eps_S of the c*cc piece relative to the c-ordering of the
rows (two identical Grassmann c's); both signs are reported."""
import json, pickle, sys, time  # noqa: E401
from pathlib import Path
import sympy as sp
HERE = Path(__file__).resolve().parent
Q = HERE.parents[6]
sys.path[:0] = [str(HERE.parents[5] / "p6q05"), str(HERE.parents[4]), str(HERE.parents[3]), str(HERE.parents[2])]
import rows as R  # noqa: E402
import gravity as G  # noqa: E402
import uvpole as U  # noqa: E402
import hhc  # noqa: E402
from evfit import monomials  # noqa: E402
from geq import Tmap  # noqa: E402
import triv  # noqa: E402
LOG = Q / "build" / "logs"
I = sp.I
RHO = -2
MS = {"K": monomials({"aK": 2, "gc1": 1, "a3": 2}, 3), "C": monomials({"gb2": 1, "gc1": 1, "a3": 2}, 4),
      "S": monomials({"z": 1, "gc1": 1, "gc2": 1}, 3), "H": monomials({"a1": 2, "a2": 2, "a3": 2}, 4)}
NK, NC, NS, NH = (len(MS[k]) for k in "KCSH")
OFF = {"K": 0, "C": NK, "S": NK + NC, "H": NK + NC + NS}
NDIM = OFF["H"] + NH
SW23 = {"p1": R.P1, "p2": R.P3, "a2": "a3", "a3": "a2"}
REL = {"1": {}, "2": {"p1": R.P2, "p2": R.P1, "a2": "a1"}, "3": {"p1": R.P3, "p2": R.P1, "a2": "a1", "a3": "a2"}}
MOM = {"1": hhc.p1, "2": hhc.p2, "3": hhc.p3}
SIG = None


def vec(sector, e):
    v, out = triv.coords(sp.expand(e), MS[sector], sector)
    w = [0] * NDIM
    for i, c in enumerate(v):
        w[OFF[sector] + i] = c
```

```
    return w, out != 0


def add(*vs):
    return [sum(x) for x in zip(*vs)]


def scale(c, v):
    return [c * x for x in v]


def D_op(f):
    """X_hhh from Y = K.f (K at p1 with aK; h a2 at p2, a3 at p3; f symmetric in the h legs)."""
    xh = 0
    for i in "123":
        fi = R.remap(f, REL[i]) if REL[i] else f
        fi = R.polarise(fi, "aK", "KU", "KW")
        eh, gf = G.S2_EH_gf(hhc.ext(f"a{i}"), G.probe_mat("E"), MOM[i])
        xh += R.strip_t(R.contract_id(fi, I * (eh + gf), "K", "E", "h"))
    return xh


def A_on_K(f):
    """X_Khc from Y = K.f : h -> s0 h on the h legs, layout K(aK) p2, c(gc1) p1, h(a3) p3."""
    fk = R.remap(f, {"p1": R.P2, "p2": R.P1})
    return R.strip_t(-I * R.polarise(fk, "a2", "p1", "gc1"))


def E_op(phi):
    """X_hhh from Y = cbar.phi (cbar gv at p1; h a2 p2, a3 p3): cbar -> sigma_F F(h)."""
    xh = 0
    for i in "123":
        fi = R.remap(phi, REL[i]) if REL[i] else phi
        D = G.Dvec(hhc.ext(f"a{i}"), U.V({"p1": 1}) if i == "1" else (hhc.F2 if i == "2" else hhc.F3))
        xh += SIG * hhc.subst_vec(fi, "gv", D)
    return R.strip_t(xh)


def A_on_C(phi):
    fc = R.remap(phi, {"p1": R.P2, "p2": R.P1})
    fc = hhc.subst_vec(fc, "gv", {"gb2": 1})
    return R.strip_t(-I * R.polarise(sp.expand(fc), "a2", "p1", "gc1"))


def cbar_partner(f):
    """phi with Y_cbar = cbar.phi = rho*T(K.f): T acts at the K momentum (moved to p2 for geq.Tmap), back to
    the cbar(gv)@p1 layout."""
    f2 = R.remap(f, {"p1": R.P2, "p2": R.P1, "a2": "a1"})            # K@p2, h a1@p1, a3@p3
    t = sp.expand(RHO * Tmap(f2))                                       # cbar(gb2)@p2
    back = R.remap(t, {"p1": R.P2, "p2": R.P1, "a1": "a2", "gb2": "gv"})  # cbar(gv)@p1, a2@p2, a3@p3
    return sp.expand(back)


def family_F():
    out = []
    for m in monomials({"aK": 2, "a2": 2, "a3": 2}, 2):
        f = sp.expand(m + R.remap(m, SW23))
        if f == 0:
            continue
        phi = cbar_partner(f)
        vh1, l1 = vec("H", D_op(f))
        vk, l2 = vec("K", A_on_K(f))
        vh2, l3 = vec("H", E_op(phi)) if phi != 0 else ([0] * NDIM, False)
        vc, l4 = vec("C", A_on_C(phi)) if phi != 0 else ([0] * NDIM, False)
        out.append({"anti": add(vh1, vh2), "field": add(vk, vc), "Q08_anti": vh1, "Q08_field": vk,
                    "leak": l1 or l2 or l3 or l4})
    return out


def family_G():
    out = []
    for m in monomials({"gv": 1, "gc1": 1, "a3": 2}, 2):            # c*(gv)@p2, c(gc1)@p1, h(a3)@p3
        xk = sp.expand(-I * U.H("aK", "p2") * hhc.subst_vec(m, "gv", {"aK": 1}))   # c* -> D^T Ktilde (K part)
        xc = sp.expand(RHO * Tmap(xk))                                              # its cbar partner
        s = R.remap(m, {"p1": R.P1, "p2": R.P3, "gv": "z"})                       # c*(z)@p3, c@p1, h(a3)@p2
        s = R.strip_t(-I * R.polarise(sp.expand(s), "a3", "p2", "gc2"))            # h -> s0 h = D c(gc2)@p2
        s_sw = R.remap(s, {"p1": R.P2, "p2": R.P1, "gc1": "gc2", "gc2": "gc1"})
        xs_anti = sp.expand(s - s_sw)
        vk, l1 = vec("K", xk)
        vc, l2 = vec("C", xc)
        vs, l3 = vec("S", xs_anti)
        out.append({"anti": add(vk, vc), "field": vs, "leak": l1 or l2 or l3})
    return out
```

```
def main():
    global SIG
    t0 = time.time()
    SIG = sp.sympify(json.loads((LOG / "q07_hhc_tree.json").read_text())["sigma_F"])
    d = pickle.loads((LOG / "q08_triv.pkl").read_bytes())
    A, Ag = d["A"], d["Ag"]
    B = pickle.loads((LOG / "q11_blocks.pkl").read_bytes()) if (LOG / "q11_blocks.pkl").is_file() else None

    def zero(M, v):
        return all(sp.expand(x) == 0 for x in M * sp.Matrix(v))

    res = {}
    keep = {}
    for name, fam in (("F", family_F()), ("G", family_G())):
        st = {"n": len(fam), "leak": sum(g["leak"] for g in fam), "zero_vector": 0}
        for lab, lam in (("derived_minus1", -1), ("plus1", 1)):
            ok, okg, vecs, fails = 0, 0, [], {}
            for g in fam:
                v = add(g["anti"], scale(lam, g["field"]))
                if not any(sp.expand(x) != 0 for x in v):
                    continue
                if zero(A, v):
                    ok += 1
                    vecs.append(v)
                    okg += zero(Ag, v)
                elif B is not None:
                    k = " ".join(b for b, M in B.items() if not zero(M, v))
                    fails[k] = fails.get(k, 0) + 1
            st[lab] = {"in_N": ok, "in_N_and_ghost_eq": okg, "fail_blocks": fails}
            keep[(name, lab)] = vecs
        st["zero_vector"] = sum(1 for g in fam if not any(sp.expand(x) != 0 for x in add(g["anti"], g["field"]))
                                and not any(sp.expand(x) != 0 for x in g["anti"]))
        res[name] = st
    pickle.dump(keep, open(LOG / "q11_sigma_vecs.pkl", "wb"))
    res["seconds"] = round(time.time() - t0, 1)
    (LOG / "q11_sigma.json").write_text(json.dumps(res, indent=1, default=str) + "\n")
    print(json.dumps(res, indent=1, default=str))


if __name__ == "__main__":
    main()
```

code/p6q06/q07/q08/q09/q10/q11/test_q11.py

```
#!/usr/bin/env python3
"""XVI-Q Q11 tests: gate fail-closed on value / honesty edits, unpinned files, and Q09-gate edits."""
import json, shutil, subprocess, sys  # noqa: E401
from pathlib import Path
import pytest

Q = Path(__file__).resolve().parents[7]
GATE = "code/p6q06/q07/q08/q09/q10/q11/release_q11.py"


def _copy(tmp):
    dst = tmp / "XVI-Q"
    top = {"build", "dist", "archival", "manuscript"}
    shutil.copytree(Q, dst, ignore=lambda d, names: [n for n in names if n == "__pycache__" or (Path(d) == Q and n in top)])
    return dst


def _edit(root, rel, fn):
    p = root / rel
    o = json.loads(p.read_text())
    fn(o)
    p.write_text(json.dumps(o, indent=1) + "\n")


def _gate(root):
    return subprocess.run([sys.executable, GATE], cwd=root, capture_output=True, text=True)


SI, CL, S = ("certificates/Q11_sigma_images.json", "certificates/Q11_classification.json",
             "certificates/Q11_P6_status.json")
MUT = {
    "F_inflated": lambda r: _edit(r, SI, lambda o: o["results_at_derived_signs"]["F"]["derived_minus1"].update(in_N=194)),
    "reading_softened": lambda r: _edit(r, SI, lambda o: o.update(reading="all sigma-images close")),
    "unclassified_hidden": lambda r: _edit(r, CL, lambda o: o.update(unclassified=0)),
    "unconditional": lambda r: _edit(r, CL, lambda o: o.update(status="ESTABLISHED")),
    "physical_claimed": lambda r: _edit(r, CL, lambda o: o.update(physical_evanescent_directions="43 physical")),
    "basis_claimed": lambda r: _edit(r, S, lambda o: o["goals"].update(**{"3_invariant_basis": "DONE"})),
    "p22_claimed": lambda r: _edit(r, S, lambda o: o["goals"].update(**{"3_p22_consistency": "consistent"})),
```

```
    "old_label": lambda r: _edit(r, S, lambda o: o.update(sigma0_status_kappa3="CLOSED at pole level")),
    "insertion_unfrozen": lambda r: _edit(r, S, lambda o: o.update(insertion="open")),
    "revision_hidden": lambda r: _edit(r, S, lambda o: o.update(disclosed_revisions={})),
    "q04_report_edit": lambda r: (r / "reproducibility/independent_rebuild_report_q04.json").write_text("{}\n"),
    "unpinned_file": lambda r: (r / "code/p6q06/q07/q08/q09/q10/q11/sneak.py").write_text("x = 1\n"),
    "q10_gate_edit": lambda r: (r / "code/p6q06/q07/q08/q09/q10/release_q10.py").write_text(
        (r / "code/p6q06/q07/q08/q09/q10/release_q10.py").read_text() + "\n"),
}


@pytest.mark.parametrize("name", sorted(MUT))
def test_gate_rejects(tmp_path, name):
    root = _copy(tmp_path)
    MUT[name](root)
    r = _gate(root)
    assert r.returncode != 0
    assert not (root / "RELEASE_STATUS.json").exists()


def test_gate_accepts(tmp_path):
    r = _gate(_copy(tmp_path))
    assert r.returncode == 0, r.stdout
```

code/p6q06/q07/q08/q09/q10/q11/trace11.py

```
#!/usr/bin/env python3
"""XVI-Q Q11 diagnostic: trace content of F generators vs. membership of the derived sigma-image in N."""
import json, pickle, sys  # noqa: E401
from pathlib import Path
import sympy as sp
HERE = Path(__file__).resolve().parent
sys.path.insert(0, str(HERE))
import sigma11 as S  # noqa: E402
TR = {"H_aK_aK", "H_a2_a2", "H_a3_a3", "T_aK_aK", "T_a2_a2", "T_a3_a3"}


def main():
    S.SIG = sp.sympify(json.loads((S.LOG / "q07_hhc_tree.json").read_text())["sigma_F"])
    A = pickle.loads((S.LOG / "q08_triv.pkl").read_bytes())["A"]
    tab = {}
    for m in S.monomials({"aK": 2, "a2": 2, "a3": 2}, 2):
        f = sp.expand(m + S.R.remap(m, S.SW23))
        if f == 0:
            continue
        names = {s.name for s in m.free_symbols}
        tr = "trace" if names & TR else "no_trace"
        ev = "evanescent" if any(n.startswith("T_") for n in names) else "4d"
        phi = S.cbar_partner(f)
        v = S.add(S.vec("H", S.D_op(f))[0], S.vec("H", S.E_op(phi))[0] if phi != 0 else [0] * S.NDIM,
                  S.scale(-1, S.add(S.vec("K", S.A_on_K(f))[0], S.vec("C", S.A_on_C(phi))[0] if phi != 0 else [0] * S.NDIM)))
        v0 = S.add(S.vec("H", S.D_op(f))[0], S.scale(-1, S.vec("K", S.A_on_K(f))[0]))
        ok = all(sp.expand(x) == 0 for x in A * sp.Matrix(v))
        ok0 = all(sp.expand(x) == 0 for x in A * sp.Matrix(v0))
        k = f"{tr}/{ev}: Ktilde={'in' if ok else 'out'} K-only={'in' if ok0 else 'out'}"
        tab[k] = tab.get(k, 0) + 1
    (S.LOG / "q11_trace.json").write_text(json.dumps(tab, indent=1, sort_keys=True) + "\n")
    print(json.dumps(tab, indent=1, sort_keys=True))


if __name__ == "__main__":
    main()
```

code/p6q06/q07/q08/q09/q10/release_q10.py

```
#!/usr/bin/env python3
"""XVI-Q Q10 release gate (fails closed; on reject no RELEASE_STATUS.json is left).
(1) code/p6q06/q07/q08/q09/q10/q10_expected.json pinned; EVERY Q01..Q10 certificate equals its pinned copy (Q05 guard-scan
    file count: lower bound); EVERY code/**/*.py, tools/*.py, tests/*.py (incl. the Q05 and Q06 gates) pinned and
    re-hashed; an unpinned file is a reject;
(2) live hardened static scan clean;
(3) semantic: h.h.c tree calibrated, 4D residual 0, every mutation detected, unique fit, matter sector exact,
    remainder explicit and evanescent; evanescent-CT naive test recorded as NOT consistent; joint local CT solvable,
    verified, Bose, evanescent, CT shas; status consistent; O.1 unchanged; Q06 layer accepted; Q07-R1 disclosed."""
from __future__ import annotations
import hashlib, json, sys  # noqa: E401
from pathlib import Path

Q = Path(__file__).resolve().parents[6]
sys.path.insert(0, str(Q / "tools"))
import source_set_q06_q07_q08_q09_q10 as SS  # noqa: E402
CERT = Q / "certificates"
EXPECTED_SHA256 = "45e812586e419899e4dd800bb6a7cbdeed8c7ef1462d22f5a467e882fa2ae080"
```

```
STATUS = (Q / "RELEASE_STATUS.json", CERT / "RELEASE_STATUS.json")
HARD_STOPS = {"bulk_symmetry_restored": False, "insertion": "frozen", "XV_B_opened": False, "XIV_J_opened": False,
              "Route_B_discharged": False, "Missing_never_zero": True}
READS: set[str] = set()
Q07_CERTS = ["Q07_hhc_row", "Q07_evanescent_CT", "Q07_P6_status", "Q07_release"]
Q08_CERTS = ["Q08_ghost_equation", "Q08_free_directions", "Q08_P6_status", "Q08_release"]
Q09_CERTS = ["Q09_guard_scan3", "Q09_q04_report_binding", "Q09_trivial_families", "Q09_P6_status"]
Q10_CERTS = ["Q10_family_anatomy", "Q10_erratum_Q08_counts", "Q10_new_manuscripts", "Q10_P6_status"]
Q04_REPORT_SHA256 = "bc2b16d99f01cf172a5c67f6ee1abb5fdb7b9b5c30e6ad3e510c241ffa442a45"
SELF = "code/p6q06/q07/q08/q09/q10/release_q10.py"


def rd(rel):
    READS.add(rel)
    return (Q / rel).read_bytes()


def jr(rel):
    return json.loads(rd(rel))


def sha_b(b):
    return hashlib.sha256(b).hexdigest()


def strip(o):
    if isinstance(o, dict):
        return {k: strip(v) for k, v in o.items() if k != "seconds"}
    if isinstance(o, list):
        return [strip(v) for v in o]
    return o


def code_files():
    out = {p.relative_to(Q).as_posix() for p in (Q / "code").rglob("*.py") if "__pycache__" not in p.parts}
    out |= {p.relative_to(Q).as_posix() for p in (Q / "tools").glob("*.py")}
    out |= {p.relative_to(Q).as_posix() for p in (Q / "tests").glob("*.py")}
    # Q11-R1 (disclosed scope revision): Q11 files (code/p6q06/q07/q08/q09/q10/q11/, tools/*q11*) are pinned by the Q11 gate
    out = {f for f in out if not (f.startswith("code/p6q06/q07/q08/q09/q10/q11/") or (f.startswith("tools/") and "q11" in f))}
    return out - {SELF}


def checks():
    c = {}
    ep = "code/p6q06/q07/q08/q09/q10/q10_expected.json"
    eb = rd(ep) if (Q / ep).is_file() else b""
    c["expected_file_pinned"] = bool(eb) and sha_b(eb) == EXPECTED_SHA256
    exp = json.loads(eb) if eb else {}
    certs = exp.get("certificates", {})

    def _layer(n, o):
        if n in ("Q05_guard_scan", "Q09_guard_scan3") and isinstance(o, dict):  # file counts are lower bounds (Q11-R1)
            o = dict(o)
            ok_n = isinstance(o.get("n_files_scanned"), int) and o["n_files_scanned"] >= certs[n]["n_files_scanned"]
            o["n_files_scanned"] = certs[n]["n_files_scanned"] if ok_n else -1
        return o
    bad = [n for n, ref in certs.items() if not (CERT / f"{n}.json").is_file()
           or _layer(n, strip(jr(f"certificates/{n}.json"))) != ref]
    c["every_certified_value_matches_pinned"] = len(certs) >= 44 and set(Q07_CERTS + Q08_CERTS + Q09_CERTS + Q10_CERTS + ["Q09_release"]) <=
set(certs) and not bad
    fs = exp.get("file_sha256", {})
    c["pinned_files_rehashed"] = bool(fs) and all((Q / f).is_file() and sha_b(rd(f)) == h for f, h in fs.items())
    c["no_unpinned_code_tools_tests_file"] = bool(fs) and code_files() <= set(fs) and \
        {"code/p6q05/release_q05.py", "code/p6q06/release_q06.py", "code/p6q06/q07/release_q07.py", "code/p6q06/q07/q08/release_q08.py",
"code/p6q06/q07/q08/q09/release_q09.py"} <= set(fs)
    sys.path.insert(0, str(Q / "code" / "p6q05"))
    import guard_scan2
    READS.add("code/p6q05/guard_scan2.py")
    live = guard_scan2.report()
    c["scan_live_clean_incl_q07"] = live["scan_clean"] is True and live["all_reviewer_evasions_flagged"] is True
    sys.path.insert(0, str(Q / "code/p6q06/q07/q08/q09"))  # scan3 module (pinned)
    import guard_scan3
    READS.add("code/p6q06/q07/q08/q09/guard_scan3.py")
    READS.add("code/p6q06/q07/q08/q09/guard_scan3_rules.json")
    s3 = guard_scan3.report()
    c["scan3_live_clean_probes_flagged"] = s3["extension_clean"] is True and s3["all_probes_flagged"] is True and \
        s3["base_scan2_clean"] is True and len(s3["probes_flagged"]) >= 10
    c["Q04_report_bound_by_hash"] = sha_b(rd("reproducibility/independent_rebuild_report_q04.json")) == Q04_REPORT_SHA256
    a = jr("certificates/Q10_family_anatomy.json")
    fm = a["families"]
    c["anatomy_values"] = a["blind_coordinates"]["n"] == 42 and a["blind_coordinates"]["by_sector"] == \
        {"K": 30, "C": 0, "S": 12, "H": 0} and a["blind_coordinates"]["dim_inside_ghost_eq"] == 24 and \
        fm["F"]["n"] == 204 and fm["F"]["zero_vector"] == 15 and fm["F"]["in_N"] == 124 and \
        fm["F"]["lam_any"] == 124 and fm["F"]["in_N_blind"] == 62 and fm["F"]["lam_none"] == 64 and \
```

```
        fm["G"]["lam_none"] == 28 and fm["G"]["lam_unique"] == {"I/2": 5} and fm["P"]["lam_none"] == 132 and \
        fm["P"]["lam_unique"] == {"-2": 2} and a["dim_N"] == 152 and a["dim_N_with_ghost_eq"] == 104
    e = jr("certificates/Q10_erratum_Q08_counts.json")
    c["erratum_withdraws_counts"] = e["id"] == "Q10-E1" and len(e["withdrawn"]["Q08"]) == 3 and \
        len(e["withdrawn"]["Q09"]) == 2 and e["restated_bound"]["Q08_value_recorded"] == 36 and \
        "NOT verified" in e["restated_bound"]["caveat"]
    nm = jr("certificates/Q10_new_manuscripts.json")
    c["new_manuscripts_context_only"] = nm["premises_imported"] == [] and nm["XV_B_opened"] is False and \
        nm["sha256"] == sha_b(rd("intake/new_manuscripts_q10.json"))
    st = jr("certificates/Q10_P6_status.json")
    gl = st["goals"]
    c["status_scoped_honest"] = st["P6_outcome"] == "Partial" and st["flags"]["claim_status"] == "partial" and \
        st["sigma0_status_kappa3"] == ("evanescent kappa^3 three-point defect sigma0-exact at pole level; X non-unique "
                                       "(ghost-eq compatible)") and all(gl[k].startswith("OPEN") for k in (
            "F_trace_failures", "G_P_families", "classification_remaining", "invariant_evanescent_basis",
            "p22_kappa2_consistency")) and "Q10-E1" in st["errata"] and \
        "any count of BRST-trivial directions" in st["not_implied"]
    c["revision_disclosed"] = "Q10-R1" in st["disclosed_revisions"]
    q9r = jr("certificates/Q09_release.json") if (CERT / "Q09_release.json").is_file() else {}
    c["Q09_layer_accepted"] = q9r.get("decision") == "ACCEPT"
    q8 = jr("certificates/Q08_P6_status.json")
    c["Q08_result_unchanged"] = q8["goals"]["1_ghost_equation_compatibility"].startswith("ESTABLISHED")
    o1 = jr("certificates/Q03_O1_record.json")
    c["O1_record_unchanged"] = o1["O1_closure"] == "CLOSED (scoped)" and o1["O1_closure_confirmed"] is True and \
        st["O1_closure_unchanged"] == "CLOSED (scoped)" and st["bulk_symmetry_restored"] is False
    c["gate_reads_in_source_set"] = READS <= set(SS.source_files(Q)) | {f"certificates/{n}.json" for n in certs} | \
        {"certificates/Q03_O1_record.json", "certificates/Q09_release.json", "intake/new_manuscripts_q10.json"}
    return c, st


def main():
    for p in STATUS:
        p.unlink(missing_ok=True)
    try:
        c, st = checks()
    except Exception as e:  # noqa: BLE001
        c, st = {"exception": False, "error": repr(e)}, {}
    ok = all(v is True for k, v in c.items() if k != "error")
    rec = {"landmark": "XVI-Q", "release": "Q10", "decision": "ACCEPT" if ok else "REJECT", "checks": c}
    (CERT / "Q10_release.json").write_text(json.dumps(rec, indent=1) + "\n")
    if not ok:
        print(json.dumps(rec, indent=1))
        return 1
    status = {"landmark": "XVI-Q", "release": "Q10", "status": "released (honest partial: erratum Q10-E1 on trivial counts, family anatomy)",
              "P6_outcome": st["P6_outcome"], "sigma0_status_kappa3": st["sigma0_status_kappa3"],
              "O1_closure": "CLOSED (scoped)", "O1_closure_confirmed": True, "hard_stops": HARD_STOPS,
              "independent_rebuild_q05": "not requested", "independent_rebuild_q06": "not requested",
              "independent_rebuild_q07": "PASSED (reviewer; report sha 03f5393f)", "independent_rebuild_q08": "PASSED on execution (reviewer; report
sha cc67001e)", "independent_rebuild_q09": "not requested", "independent_rebuild_q10": "pending", "goals": st["goals"], "execution_pass": True,
"claim_status": "partial", "gate": SELF}
    for p in STATUS:
        p.write_text(json.dumps(status, indent=1) + "\n")
    (Q / "ledgers").mkdir(exist_ok=True)
    (Q / "ledgers" / "Q10_milestones.json").write_text(json.dumps({"landmark": "XVI-Q", "release": "Q10", **strip(st)},
                                                                  indent=1) + "\n")
    print("ACCEPT")
    return 0


if __name__ == "__main__":
    raise SystemExit(main())
```

code/p6q06/q07/q08/q09/q10/test_q10.py

```
#!/usr/bin/env python3
"""XVI-Q Q10 tests: gate fail-closed on value / honesty edits, unpinned files, and Q09-gate edits."""
import json, shutil, subprocess, sys  # noqa: E401
from pathlib import Path
import pytest

Q = Path(__file__).resolve().parents[6]
GATE = "code/p6q06/q07/q08/q09/q10/release_q10.py"


def _copy(tmp):
    dst = tmp / "XVI-Q"
    top = {"build", "dist", "archival", "manuscript"}
    shutil.copytree(Q, dst, ignore=lambda d, names: [n for n in names if n == "__pycache__" or (Path(d) == Q and n in top)])
    return dst


def _edit(root, rel, fn):
    p = root / rel
```

```
    o = json.loads(p.read_text())
    fn(o)
    p.write_text(json.dumps(o, indent=1) + "\n")


def _gate(root):
    return subprocess.run([sys.executable, GATE], cwd=root, capture_output=True, text=True)


A_, E_, S = ("certificates/Q10_family_anatomy.json", "certificates/Q10_erratum_Q08_counts.json",
             "certificates/Q10_P6_status.json")
MUT = {
    "blind_count": lambda r: _edit(r, A_, lambda o: o["blind_coordinates"].update(n=0)),
    "F_lambda_nonvacuous": lambda r: _edit(r, A_, lambda o: o["families"]["F"].update(lam_any=0)),
    "G_fixed": lambda r: _edit(r, A_, lambda o: o["families"]["G"].update(lam_none=0)),
    "erratum_dropped": lambda r: _edit(r, E_, lambda o: o["withdrawn"].update(Q08=[])),
    "caveat_dropped": lambda r: _edit(r, E_, lambda o: o["restated_bound"].update(caveat="")),
    "basis_claimed": lambda r: _edit(r, S, lambda o: o["goals"].update(invariant_evanescent_basis="DONE")),
    "GP_claimed": lambda r: _edit(r, S, lambda o: o["goals"].update(G_P_families="DONE")),
    "old_label": lambda r: _edit(r, S, lambda o: o.update(sigma0_status_kappa3="CLOSED at pole level")),
    "revision_hidden": lambda r: _edit(r, S, lambda o: o.update(disclosed_revisions={})),
    "intake_edit": lambda r: _edit(r, "intake/new_manuscripts_q10.json", lambda o: o.update(premises_imported=["En05-B"])),
    "q04_report_edit": lambda r: (r / "reproducibility/independent_rebuild_report_q04.json").write_text("{}\n"),
    "unpinned_file": lambda r: (r / "code/p6q06/q07/q08/q09/q10/sneak.py").write_text("x = 1\n"),
    "q09_gate_edit": lambda r: (r / "code/p6q06/q07/q08/q09/release_q09.py").write_text(
        (r / "code/p6q06/q07/q08/q09/release_q09.py").read_text() + "\n"),
}


@pytest.mark.parametrize("name", sorted(MUT))
def test_gate_rejects(tmp_path, name):
    root = _copy(tmp_path)
    MUT[name](root)
    r = _gate(root)
    assert r.returncode != 0
    assert not (root / "RELEASE_STATUS.json").exists()


def test_gate_accepts(tmp_path):
    r = _gate(_copy(tmp_path))
    assert r.returncode == 0, r.stdout
```

code/p6q06/q07/q08/q09/release_q09.py

```
#!/usr/bin/env python3
"""XVI-Q Q09 release gate (fails closed; on reject no RELEASE_STATUS.json is left).
(1) code/p6q06/q07/q08/q09/q09_expected.json pinned; EVERY Q01..Q09 certificate equals its pinned copy (Q05 guard-scan
    file count: lower bound); EVERY code/**/*.py, tools/*.py, tests/*.py (incl. the Q05 and Q06 gates) pinned and
    re-hashed; an unpinned file is a reject;
(2) live hardened static scan clean;
(3) semantic: h.h.c tree calibrated, 4D residual 0, every mutation detected, unique fit, matter sector exact,
    remainder explicit and evanescent; evanescent-CT naive test recorded as NOT consistent; joint local CT solvable,
    verified, Bose, evanescent, CT shas; status consistent; O.1 unchanged; Q06 layer accepted; Q07-R1 disclosed."""
from __future__ import annotations
import hashlib, json, sys  # noqa: E401
from pathlib import Path

Q = Path(__file__).resolve().parents[5]
sys.path.insert(0, str(Q / "tools"))
import source_set_q06_q07_q08_q09 as SS  # noqa: E402
CERT = Q / "certificates"
EXPECTED_SHA256 = "b998ce8dc9a332093c9bd0f04bab8a32bc2cbe31efb7d23a303240733bef110e"
STATUS = (Q / "RELEASE_STATUS.json", CERT / "RELEASE_STATUS.json")
HARD_STOPS = {"bulk_symmetry_restored": False, "insertion": "frozen", "XV_B_opened": False, "XIV_J_opened": False,
              "Route_B_discharged": False, "Missing_never_zero": True}
READS: set[str] = set()
Q07_CERTS = ["Q07_hhc_row", "Q07_evanescent_CT", "Q07_P6_status", "Q07_release"]
Q08_CERTS = ["Q08_ghost_equation", "Q08_free_directions", "Q08_P6_status", "Q08_release"]
Q09_CERTS = ["Q09_guard_scan3", "Q09_q04_report_binding", "Q09_trivial_families", "Q09_P6_status"]
Q04_REPORT_SHA256 = "bc2b16d99f01cf172a5c67f6ee1abb5fdb7b9b5c30e6ad3e510c241ffa442a45"
SELF = "code/p6q06/q07/q08/q09/release_q09.py"


def rd(rel):
    READS.add(rel)
    return (Q / rel).read_bytes()


def jr(rel):
    return json.loads(rd(rel))


def sha_b(b):
```

```
    return hashlib.sha256(b).hexdigest()


def strip(o):
    if isinstance(o, dict):
        return {k: strip(v) for k, v in o.items() if k != "seconds"}
    if isinstance(o, list):
        return [strip(v) for v in o]
    return o


def code_files():
    out = {p.relative_to(Q).as_posix() for p in (Q / "code").rglob("*.py") if "__pycache__" not in p.parts}
    out |= {p.relative_to(Q).as_posix() for p in (Q / "tools").glob("*.py")}
    out |= {p.relative_to(Q).as_posix() for p in (Q / "tests").glob("*.py")}
    # Q10-R1 (disclosed scope revision): Q10 files (code/p6q06/q07/q08/q09/q10/, tools/*q10*) are pinned by the Q10 gate
    out = {f for f in out if not (f.startswith("code/p6q06/q07/q08/q09/q10/") or (f.startswith("tools/") and "q10" in f))}
    return out - {SELF}


def checks():
    c = {}
    ep = "code/p6q06/q07/q08/q09/q09_expected.json"
    eb = rd(ep) if (Q / ep).is_file() else b""
    c["expected_file_pinned"] = bool(eb) and sha_b(eb) == EXPECTED_SHA256
    exp = json.loads(eb) if eb else {}
    certs = exp.get("certificates", {})

    def _layer(n, o):
        if n in ("Q05_guard_scan", "Q09_guard_scan3") and isinstance(o, dict):  # Q10-R1: file counts are lower bounds
            o = dict(o)
            ok_n = isinstance(o.get("n_files_scanned"), int) and o["n_files_scanned"] >= certs[n]["n_files_scanned"]
            o["n_files_scanned"] = certs[n]["n_files_scanned"] if ok_n else -1
        return o
    bad = [n for n, ref in certs.items() if not (CERT / f"{n}.json").is_file()
           or _layer(n, strip(jr(f"certificates/{n}.json"))) != ref]
    c["every_certified_value_matches_pinned"] = len(certs) >= 44 and set(Q07_CERTS + Q08_CERTS + Q09_CERTS) <= set(certs) and not bad
    fs = exp.get("file_sha256", {})
    c["pinned_files_rehashed"] = bool(fs) and all((Q / f).is_file() and sha_b(rd(f)) == h for f, h in fs.items())
    c["no_unpinned_code_tools_tests_file"] = bool(fs) and code_files() <= set(fs) and \
        {"code/p6q05/release_q05.py", "code/p6q06/release_q06.py", "code/p6q06/q07/release_q07.py", "code/p6q06/q07/q08/release_q08.py"} <= set(fs)
    sys.path.insert(0, str(Q / "code" / "p6q05"))
    import guard_scan2
    READS.add("code/p6q05/guard_scan2.py")
    live = guard_scan2.report()
    c["scan_live_clean_incl_q07"] = live["scan_clean"] is True and live["all_reviewer_evasions_flagged"] is True
    sys.path.insert(0, str(Q / "code/p6q06/q07/q08/q09"))
    import guard_scan3
    READS.add("code/p6q06/q07/q08/q09/guard_scan3.py")
    READS.add("code/p6q06/q07/q08/q09/guard_scan3_rules.json")
    s3 = guard_scan3.report()
    c["scan3_live_clean_probes_flagged"] = s3["extension_clean"] is True and s3["all_probes_flagged"] is True and \
        s3["base_scan2_clean"] is True and len(s3["probes_flagged"]) >= 10
    c["Q04_report_bound_by_hash"] = sha_b(rd("reproducibility/independent_rebuild_report_q04.json")) == Q04_REPORT_SHA256
    t = jr("certificates/Q09_trivial_families.json")
    fm = t["families"]
    c["derived_lambdas_honest"] = fm["F"]["lambda_derived"] == "-1" and fm["P"]["lambda_derived"] == "-1" and \
        fm["G"]["lambda_derived"] == "1" and fm["F"]["n_in_N"] == 139 and fm["F"]["n_fail"] == 65 and \
        fm["F"]["n_fail_with_trace_monomials"] == 65 and fm["G"]["n_in_N"] == 0 and fm["P"]["n_in_N"] == 0 and \
        t["dim_trivial_span_derived"] == 59 and "OPEN" in t["diagnosis"]
    st = jr("certificates/Q09_P6_status.json")
    gl = st["goals"]
    c["status_scoped_honest"] = st["P6_outcome"] == "Partial" and st["flags"]["claim_status"] == "partial" and \
        st["sigma0_status_kappa3"] == ("evanescent kappa^3 three-point defect sigma0-exact at pole level; X non-unique "
                                       "(ghost-eq compatible)") and gl["c_invariant_operator_basis"].startswith("OPEN") \
        and gl["d_p22_consistency"].startswith("OPEN") and gl["a_derived_normalisations"].startswith("Partial") and \
        gl["b_classification"].startswith("Partial") and all(v is True for v in st["review_fixes"].values())
    c["revision_disclosed"] = "Q09-R1" in st["disclosed_revisions"]
    q8 = jr("certificates/Q08_P6_status.json")
    c["Q08_result_unchanged"] = q8["goals"]["1_ghost_equation_compatibility"].startswith("ESTABLISHED")
    o1 = jr("certificates/Q03_O1_record.json")
    c["O1_record_unchanged"] = o1["O1_closure"] == "CLOSED (scoped)" and o1["O1_closure_confirmed"] is True and \
        st["O1_closure_unchanged"] == "CLOSED (scoped)" and st["bulk_symmetry_restored"] is False
    q8r = jr("certificates/Q08_release.json") if (CERT / "Q08_release.json").is_file() else {}
    c["Q08_layer_accepted"] = q8r.get("decision") == "ACCEPT"
    c["gate_reads_in_source_set"] = READS <= set(SS.source_files(Q)) | {f"certificates/{n}.json" for n in certs} | \
        {"certificates/Q03_O1_record.json", "certificates/Q08_release.json"}
    return c, st


def main():
    for p in STATUS:
        p.unlink(missing_ok=True)
    try:
```

```
        c, st = checks()
    except Exception as e:  # noqa: BLE001
        c, st = {"exception": False, "error": repr(e)}, {}
    ok = all(v is True for k, v in c.items() if k != "error")
    rec = {"landmark": "XVI-Q", "release": "Q09", "decision": "ACCEPT" if ok else "REJECT", "checks": c}
    (CERT / "Q09_release.json").write_text(json.dumps(rec, indent=1) + "\n")
    if not ok:
        print(json.dumps(rec, indent=1))
        return 1
    status = {"landmark": "XVI-Q", "release": "Q09", "status": "released (honest partial: P.6 evanescent CT structure, review fixes)",
              "P6_outcome": st["P6_outcome"], "sigma0_status_kappa3": st["sigma0_status_kappa3"],
              "O1_closure": "CLOSED (scoped)", "O1_closure_confirmed": True, "hard_stops": HARD_STOPS,
              "independent_rebuild_q05": "not requested", "independent_rebuild_q06": "not requested",
              "independent_rebuild_q07": "PASSED (reviewer; report sha 03f5393f)", "independent_rebuild_q08": "in progress (reviewer)",
"independent_rebuild_q09": "pending", "goals": st["goals"], "execution_pass": True, "claim_status": "partial", "gate": SELF}
    for p in STATUS:
        p.write_text(json.dumps(status, indent=1) + "\n")
    (Q / "ledgers").mkdir(exist_ok=True)
    (Q / "ledgers" / "Q09_milestones.json").write_text(json.dumps({"landmark": "XVI-Q", "release": "Q09", **strip(st)},
                                                                  indent=1) + "\n")
    print("ACCEPT")
    return 0


if __name__ == "__main__":
    raise SystemExit(main())
```

code/p6q06/q07/q08/q09/test_q09.py

```
#!/usr/bin/env python3
"""XVI-Q Q09 tests: gate fail-closed on value / honesty edits and unpinned files; subst_vec / ansatz sanity."""
import json, shutil, subprocess, sys  # noqa: E401
from pathlib import Path
import pytest

Q = Path(__file__).resolve().parents[5]
GATE = "code/p6q06/q07/q08/q09/release_q09.py"


def _copy(tmp):
    dst = tmp / "XVI-Q"
    top = {"build", "dist", "archival", "manuscript"}
    shutil.copytree(Q, dst, ignore=lambda d, names: [n for n in names if n == "__pycache__" or (Path(d) == Q and n in top)])
    return dst


def _edit(root, rel, fn):
    p = root / rel
    o = json.loads(p.read_text())
    fn(o)
    p.write_text(json.dumps(o, indent=1) + "\n")


def _gate(root):
    return subprocess.run([sys.executable, GATE], cwd=root, capture_output=True, text=True)


T_, S, B_ = "certificates/Q09_trivial_families.json", "certificates/Q09_P6_status.json", "certificates/Q09_q04_report_binding.json"
MUT = {
    "lambda_F": lambda r: _edit(r, T_, lambda o: o["families"]["F"].update(lambda_derived="1")),
    "F_all_in_N": lambda r: _edit(r, T_, lambda o: o["families"]["F"].update(n_in_N=204, n_fail=0)),
    "span_inflated": lambda r: _edit(r, T_, lambda o: o.update(dim_trivial_span_derived=104)),
    "old_label": lambda r: _edit(r, S, lambda o: o.update(sigma0_status_kappa3="CLOSED at pole level")),
    "basis_claimed": lambda r: _edit(r, S, lambda o: o["goals"].update(c_invariant_operator_basis="DONE")),
    "p22_claimed": lambda r: _edit(r, S, lambda o: o["goals"].update(d_p22_consistency="consistent")),
    "revision_hidden": lambda r: _edit(r, S, lambda o: o.update(disclosed_revisions={})),
    "q04_report_edit": lambda r: (r / "reproducibility/independent_rebuild_report_q04.json").write_text("{}\n"),
    "scan3_rules_edit": lambda r: (r / "code/p6q06/q07/q08/q09/guard_scan3_rules.json").write_text('{"dict_walk_methods": []}\n'),
    "methodcaller_snuck_in": lambda r: (r / "code/p6q06/q07/q08/q09/sneak.py").write_text("from operator import methodcaller\n"),
    "q08_gate_edit": lambda r: (r / "code/p6q06/q07/q08/release_q08.py").write_text((r / "code/p6q06/q07/q08/release_q08.py").read_text() + "\n"),
}


@pytest.mark.parametrize("name", sorted(MUT))
def test_gate_rejects(tmp_path, name):
    root = _copy(tmp_path)
    MUT[name](root)
    r = _gate(root)
    assert r.returncode != 0
    assert not (root / "RELEASE_STATUS.json").exists()


def test_gate_accepts(tmp_path):
```

```
    r = _gate(_copy(tmp_path))
    assert r.returncode == 0, r.stdout


def test_scan3_probes():
    sys.path.insert(0, str(Q / "code" / "p6q06" / "q07" / "q08" / "q09"))
    import ast
    import guard_scan3 as g
    assert all(g.hits(ast.parse(v)) for v in g.PROBES.values())
    assert not g.hits(ast.parse("e.coeff(t, 3)\nx.as_powers_dict()[s]"))
```

code/p6q06/q07/q08/release_q08.py

```
#!/usr/bin/env python3
"""XVI-Q Q08 release gate (fails closed; on reject no RELEASE_STATUS.json is left).
(1) code/p6q06/q07/q08/q08_expected.json pinned; EVERY Q01..Q08 certificate equals its pinned copy (Q05 guard-scan
    file count: lower bound); EVERY code/**/*.py, tools/*.py, tests/*.py (incl. the Q05 and Q06 gates) pinned and
    re-hashed; an unpinned file is a reject;
(2) live hardened static scan clean;
(3) semantic: h.h.c tree calibrated, 4D residual 0, every mutation detected, unique fit, matter sector exact,
    remainder explicit and evanescent; evanescent-CT naive test recorded as NOT consistent; joint local CT solvable,
    verified, Bose, evanescent, CT shas; status consistent; O.1 unchanged; Q06 layer accepted; Q07-R1 disclosed."""
from __future__ import annotations
import hashlib, json, sys  # noqa: E401
from pathlib import Path

Q = Path(__file__).resolve().parents[4]
sys.path.insert(0, str(Q / "tools"))
import source_set_q06_q07_q08 as SS  # noqa: E402
CERT = Q / "certificates"
EXPECTED_SHA256 = "7e24cb02449aefaabad1fe2e849a722217d93e12290fd8ee0a243403d74996d5"
STATUS = (Q / "RELEASE_STATUS.json", CERT / "RELEASE_STATUS.json")
HARD_STOPS = {"bulk_symmetry_restored": False, "insertion": "frozen", "XV_B_opened": False, "XIV_J_opened": False,
              "Route_B_discharged": False, "Missing_never_zero": True}
READS: set[str] = set()
Q07_CERTS = ["Q07_hhc_row", "Q07_evanescent_CT", "Q07_P6_status", "Q07_release"]
Q08_CERTS = ["Q08_ghost_equation", "Q08_free_directions", "Q08_P6_status"]
SELF = "code/p6q06/q07/q08/release_q08.py"


def rd(rel):
    READS.add(rel)
    return (Q / rel).read_bytes()


def jr(rel):
    return json.loads(rd(rel))


def sha_b(b):
    return hashlib.sha256(b).hexdigest()


def strip(o):
    if isinstance(o, dict):
        return {k: strip(v) for k, v in o.items() if k != "seconds"}
    if isinstance(o, list):
        return [strip(v) for v in o]
    return o


def code_files():
    out = {p.relative_to(Q).as_posix() for p in (Q / "code").rglob("*.py") if "__pycache__" not in p.parts}
    out |= {p.relative_to(Q).as_posix() for p in (Q / "tools").glob("*.py")}
    out |= {p.relative_to(Q).as_posix() for p in (Q / "tests").glob("*.py")}
    # Q09-R1 (disclosed scope revision): Q09 files (code/p6q06/q07/q08/q09/, tools/*q09*) are pinned by the Q09 gate
    out = {f for f in out if not (f.startswith("code/p6q06/q07/q08/q09/") or (f.startswith("tools/") and "q09" in f))}
    return out - {SELF}


def checks():
    c = {}
    ep = "code/p6q06/q07/q08/q08_expected.json"
    eb = rd(ep) if (Q / ep).is_file() else b""
    c["expected_file_pinned"] = bool(eb) and sha_b(eb) == EXPECTED_SHA256
    exp = json.loads(eb) if eb else {}
    certs = exp.get("certificates", {})

    def _layer(n, o):
        if n == "Q05_guard_scan" and isinstance(o, dict):
            o = dict(o)
            ok_n = isinstance(o.get("n_files_scanned"), int) and o["n_files_scanned"] >= certs[n]["n_files_scanned"]
            o["n_files_scanned"] = certs[n]["n_files_scanned"] if ok_n else -1
```

```
        return o
    bad = [n for n, ref in certs.items() if not (CERT / f"{n}.json").is_file()
           or _layer(n, strip(jr(f"certificates/{n}.json"))) != ref]
    c["every_certified_value_matches_pinned"] = len(certs) >= 39 and set(Q07_CERTS + Q08_CERTS) <= set(certs) and not bad
    fs = exp.get("file_sha256", {})
    c["pinned_files_rehashed"] = bool(fs) and all((Q / f).is_file() and sha_b(rd(f)) == h for f, h in fs.items())
    c["no_unpinned_code_tools_tests_file"] = bool(fs) and code_files() <= set(fs) and \
        {"code/p6q05/release_q05.py", "code/p6q06/release_q06.py", "code/p6q06/q07/release_q07.py"} <= set(fs)
    sys.path.insert(0, str(Q / "code" / "p6q05"))
    import guard_scan2
    READS.add("code/p6q05/guard_scan2.py")
    live = guard_scan2.report()
    c["scan_live_clean_incl_q07"] = live["scan_clean"] is True and live["all_reviewer_evasions_flagged"] is True
    g = jr("certificates/Q08_ghost_equation.json")
    pt = g["particular_solution"] or {}
    c["ghost_equation_compatible"] = g["loops_satisfy_with_rho_minus_2"] is True and \
        g["loop_poles"] == {"rho_total": "-2", "rho_G04_G03": "-2", "rho_G09_G07": "-2"} and \
        g["solvable_with_ghost_equation"] is True and pt.get("verified_A_x_eq_b") is True and \
        g["dim_free_without_ghost_eq"] == 152 and g["dim_free_with_ghost_eq"] == 104
    f = jr("certificates/Q08_free_directions.json")
    c["free_directions_not_overclaimed"] = f["homogeneous_space"]["dim_N"] == 152 and \
        f["dim_N_cap_all_families_best_lambda"] <= f["dim_N_cap_all_families_upper_bound"] < 152 and \
        "FITTED" in f["reading"] and "not shown trivial" in f["reading"]
    st = jr("certificates/Q08_P6_status.json")
    gl = st["goals"]
    c["status_scoped_honest"] = st["P6_outcome"] == "Partial" and st["flags"]["claim_status"] == "partial" and \
        gl["2_invariant_operator_basis"].startswith("Partial") and gl["3_free_parameters"].startswith("Partial") and \
        gl["4_p22_consistency"].startswith("OPEN") and gl["1_ghost_equation_compatibility"].startswith("ESTABLISHED") \
        and "uniqueness of X" in st["not_implied"]
    c["revision_disclosed"] = "Q08-R1" in st["disclosed_revisions"]
    q7 = jr("certificates/Q07_P6_status.json")
    c["Q07_result_unchanged"] = q7["sigma0_closure_kappa3"].startswith("CLOSED at pole level")
    o1 = jr("certificates/Q03_O1_record.json")
    c["O1_record_unchanged"] = o1["O1_closure"] == "CLOSED (scoped)" and o1["O1_closure_confirmed"] is True and \
        st["O1_closure_unchanged"] == "CLOSED (scoped)" and st["bulk_symmetry_restored"] is False
    q7r = jr("certificates/Q07_release.json") if (CERT / "Q07_release.json").is_file() else {}
    c["Q07_layer_accepted"] = q7r.get("decision") == "ACCEPT"
    c["gate_reads_in_source_set"] = READS <= set(SS.source_files(Q)) | {f"certificates/{n}.json" for n in certs} | \
        {"certificates/Q03_O1_record.json", "certificates/Q07_release.json"}
    return c, st


def main():
    for p in STATUS:
        p.unlink(missing_ok=True)
    try:
        c, st = checks()
    except Exception as e:  # noqa: BLE001
        c, st = {"exception": False, "error": repr(e)}, {}
    ok = all(v is True for k, v in c.items() if k != "error")
    rec = {"landmark": "XVI-Q", "release": "Q08", "decision": "ACCEPT" if ok else "REJECT", "checks": c}
    (CERT / "Q08_release.json").write_text(json.dumps(rec, indent=1) + "\n")
    if not ok:
        print(json.dumps(rec, indent=1))
        return 1
    status = {"landmark": "XVI-Q", "release": "Q08", "status": "released (honest partial: P.6 evanescent CT structure)",
              "P6_outcome": st["P6_outcome"], "sigma0_closure_kappa3": st["sigma0_closure_kappa3"],
              "O1_closure": "CLOSED (scoped)", "O1_closure_confirmed": True, "hard_stops": HARD_STOPS,
              "independent_rebuild_q05": "not requested", "independent_rebuild_q06": "not requested",
              "independent_rebuild_q07": "in progress (reviewer)", "independent_rebuild_q08": "pending", "goals": st["goals"], "execution_pass": True,
"claim_status": "partial", "gate": SELF}
    for p in STATUS:
        p.write_text(json.dumps(status, indent=1) + "\n")
    (Q / "ledgers").mkdir(exist_ok=True)
    (Q / "ledgers" / "Q08_milestones.json").write_text(json.dumps({"landmark": "XVI-Q", "release": "Q08", **strip(st)},
                                                                  indent=1) + "\n")
    print("ACCEPT")
    return 0


if __name__ == "__main__":
    raise SystemExit(main())
```

code/p6q06/q07/q08/sectors.py

```
#!/usr/bin/env python3
"""XVI-Q Q08: sector decomposition of the homogeneous space N (from triv.py's pickled matrices): kernel dims of the
rows restricted to single sectors, and the trivial span (best fitted lambda) intersected with the ghost-equation
subspace."""
import json, pickle, sys  # noqa: E401
from pathlib import Path
import sympy as sp
from sympy.polys.matrices import DomainMatrix
```

```
Q = Path(__file__).resolve().parents[4]
LOG = Q / "build" / "logs"
d = pickle.loads((LOG / "q08_triv.pkl").read_bytes())
A, Ag, fams = d["A"], d["Ag"], d["fams"]
tr = json.loads((LOG / "q08_triv.json").read_text())
nK, nC, nS = 134, 81, 18
cuts = {"Kch": (0, nK), "cbar_ch": (nK, nK + nC), "cstar_cc": (nK + nC, nK + nC + nS), "hhh": (nK + nC + nS, A.shape[1])}
out = {}
for k, (a, b) in cuts.items():
    sub = A[:, a:b]
    out[f"dim_N_{k}_only"] = (b - a) - DomainMatrix.from_Matrix(sub).rank()
lams = {n: sp.sympify(v["best_uniform_lambda"]) for n, v in tr["families"].items()}
B = sp.Matrix.hstack(*[sp.Matrix(p) + lams[n] * sp.Matrix(q) for n, pr in fams.items() for p, q in pr])
AA = A.col_join(Ag)
rB = DomainMatrix.from_Matrix(B).rank()
out["dim_trivial_best_lambda_cap_N_and_ghost_eq"] = rB - DomainMatrix.from_Matrix(AA * B).rank()
(LOG / "q08_sectors.json").write_text(json.dumps(out, indent=1) + "\n")
print(out)
```

code/p6q06/q07/q08/test_q08.py

```
#!/usr/bin/env python3
"""XVI-Q Q08 tests: gate fail-closed on value / honesty edits and unpinned files; subst_vec / ansatz sanity."""
import json, shutil, subprocess, sys  # noqa: E401
from pathlib import Path
import pytest

Q = Path(__file__).resolve().parents[4]
GATE = "code/p6q06/q07/q08/release_q08.py"


def _copy(tmp):
    dst = tmp / "XVI-Q"
    top = {"build", "dist", "archival", "manuscript"}
    shutil.copytree(Q, dst, ignore=lambda d, names: [n for n in names if n == "__pycache__" or (Path(d) == Q and n in top)])
    return dst


def _edit(root, rel, fn):
    p = root / rel
    o = json.loads(p.read_text())
    fn(o)
    p.write_text(json.dumps(o, indent=1) + "\n")


def _gate(root):
    return subprocess.run([sys.executable, GATE], cwd=root, capture_output=True, text=True)


G_, F_, S = "certificates/Q08_ghost_equation.json", "certificates/Q08_free_directions.json", "certificates/Q08_P6_status.json"
MUT = {
    "rho": lambda r: _edit(r, G_, lambda o: o["loop_poles"].update(rho_total="2")),
    "ghost_unsolvable_hidden": lambda r: _edit(r, G_, lambda o: o.update(dim_free_with_ghost_eq=152)),
    "particular_unverified": lambda r: _edit(r, G_, lambda o: o["particular_solution"].update(verified_A_x_eq_b=False)),
    "all_trivial_claim": lambda r: _edit(r, F_, lambda o: o.update(dim_N_cap_all_families_upper_bound=152)),
    "fitted_hidden": lambda r: _edit(r, F_, lambda o: o.update(reading="all trivial")),
    "basis_claimed": lambda r: _edit(r, S, lambda o: o["goals"].update({"2_invariant_operator_basis": "DONE"})),
    "p22_claimed": lambda r: _edit(r, S, lambda o: o["goals"].update({"4_p22_consistency": "consistent"})),
    "revision_hidden": lambda r: _edit(r, S, lambda o: o.update(disclosed_revisions={})),
    "q07_value": lambda r: _edit(r, "certificates/Q07_hhc_row.json", lambda o: o.update(residual_4D="x")),
    "unpinned_code_file": lambda r: (r / "code/p6q06/q07/q08/extra.py").write_text("x = 1\n"),
    "q07_gate_edit": lambda r: (r / "code/p6q06/q07/release_q07.py").write_text((r / "code/p6q06/q07/release_q07.py").read_text() + "\n"),
}


@pytest.mark.parametrize("name", sorted(MUT))
def test_gate_rejects(tmp_path, name):
    root = _copy(tmp_path)
    MUT[name](root)
    r = _gate(root)
    assert r.returncode != 0
    assert not (root / "RELEASE_STATUS.json").exists()


def test_gate_accepts(tmp_path):
    r = _gate(_copy(tmp_path))
    assert r.returncode == 0, r.stdout


def test_tmap_trace():
    sys.path.insert(0, str(Q / "code" / "p6q06" / "q07" / "q08"))
    import geq
    import uvpole as U
```

```
    assert geq._trace(U.H("u", "w")) == 4 and geq._trace(U.T("u", "w")) == 0
    assert geq._trace(U.H("u", "x") * U.H("w", "y")) == U.H("x", "y")
```

code/p6q06/q07/q08/triv.py

```
#!/usr/bin/env python3
"""XVI-Q Q08 goals 1-3: structure of the local evanescent kappa^3 counterterm X (three-point rows).
 A x = -b : rows (c.c.K, c.c.cbar, h.h.c) linear in the X coordinates (Q07 ansatz incl. Bose symmetry of X_hhh).
 (1) ghost equation: add X_{cbar ch} = rho X_{Kch}[K -> T(cbar)], rho = -2 (fixed by the loop poles, geq.py).
 (3) homogeneous space N = ker A; BRST-trivial directions sigma_lin(Y) for gh -1 local Y at three points:
     F: Y = K.F(h,h)        -> X_hhh = sum_i dS2/dh_i . F_i,      X_Khc ~ K.F(s0h c, h)
     G: Y = c*.G(c,h)       -> X_cbar ch ~ cbar.D[s0h G],          X_Khc ~ K.s0h(G)
     P: Y = cbar.Phi(h,h)   -> X_hhh ~ sigma_F F(h_i).Phi_i,       X_cbar ch ~ cbar.Phi(s0h c, h)
     the relative normalisation lambda of the two pieces of each family is not derived but FITTED (one constant per
     family, required to be the same for every monomial: a nontrivial check); the triviality claim is conditional
     on that.  Output: dims of N, of N with the ghost equation, of the trivial span inside N, remainder."""
from __future__ import annotations
import json, sys, time  # noqa: E401
from pathlib import Path
import sympy as sp
from sympy.polys.matrices import DomainMatrix

HERE = Path(__file__).resolve().parent
Q = HERE.parents[3]
sys.path.insert(0, str(HERE.parents[2] / "p6q05"))
sys.path.insert(0, str(HERE.parents[1]))
sys.path.insert(0, str(HERE.parent))
sys.path.insert(0, str(HERE))
import rows as R  # noqa: E402
import gravity as G  # noqa: E402
import uvpole as U  # noqa: E402
import hhc  # noqa: E402
import evfit2 as E2  # noqa: E402
from evfit import ansatz, monomials  # noqa: E402
from geq import Tmap  # noqa: E402
LOG = Q / "build" / "logs"
I = sp.I
RHO = -2


def coords(expr, ms, tag):
    idx = {m: i for i, m in enumerate(ms)}
    v = [0] * len(ms)
    out = 0
    for t in sp.Add.make_args(sp.expand(expr)):
        if t == 0:
            continue
        c, m = t.as_independent(*[s for s in t.free_symbols if s.name[:2] in ("H_", "T_")], as_Add=False)
        if m in idx:
            v[idx[m]] += c / I
        else:
            out += t
    return v, out


def build():
    XK, xk = ansatz({"aK": 2, "gc1": 1, "a3": 2}, 3, "xk")
    XC, xc = ansatz({"gb2": 1, "gc1": 1, "a3": 2}, 4, "xc")
    XS, xs = ansatz({"z": 1, "gc1": 1, "gc2": 1}, 3, "xs")
    XH, xh = ansatz({"a1": 2, "a2": 2, "a3": 2}, 4, "xh")
    sig = sp.sympify(json.loads((LOG / "q07_hhc_tree.json").read_text())["sigma_F"])
    rows = [E2.ghost_row_total("ccK", XK, XS), E2.ghost_row_total("ccb", XC, XS), E2.hhc_ansatz_total(XH, XK, XC, sig)]
    xall = xk + xc + xs + xh
    X = set(xall)
    eqs = []
    for r in rows:
        gens = sorted([s for s in r.free_symbols if s not in X], key=str)
        eqs += sp.Poly(r, *gens).coeffs()
    beq = []
    for mp in ({"p1": R.P2, "p2": R.P1, "a1": "a2", "a2": "a1"}, {"p1": R.P3, "p2": R.P2, "a1": "a3", "a3": "a1"}):
        d = sp.expand(XH - R.remap(XH, mp))
        gens = sorted([s for s in d.free_symbols if s not in X], key=str)
        beq += sp.Poly(d, *gens).coeffs()
    geq = sp.expand(XC - RHO * Tmap(XK))
    gens = sorted([s for s in geq.free_symbols if s not in X], key=str)
    geqs = sp.Poly(geq, *gens).coeffs()
    A, b = sp.linear_eq_to_matrix(eqs + beq, xall)
    Ag, bg = sp.linear_eq_to_matrix(geqs, xall)
    ms = {"K": monomials({"aK": 2, "gc1": 1, "a3": 2}, 3), "C": monomials({"gb2": 1, "gc1": 1, "a3": 2}, 4),
          "S": monomials({"z": 1, "gc1": 1, "gc2": 1}, 3), "H": monomials({"a1": 2, "a2": 2, "a3": 2}, 4)}
    return A, b, Ag, bg, ms, sig
```

```
def fam_vectors(ms, sig):
    """for each family and monomial: (piece1 coords, piece2 coords) as full-length vectors."""
    nK, nC, nS, nH = (len(ms[k]) for k in "KCSH")
    off = {"K": 0, "C": nK, "S": nK + nC, "H": nK + nC + nS}
    N = off["H"] + nH

    def full(sector, v):
        w = [0] * N
        for i, c in enumerate(v):
            w[off[sector] + i] = c
        return w

    fams = {}
    leak = {}
    # F family: F(K at p1; a2 at p2, a3 at p3), symmetrised in the two h legs
    Fm = monomials({"aK": 2, "a2": 2, "a3": 2}, 2)
    sw23 = {"p1": R.P1, "p2": R.P3, "a2": "a3", "a3": "a2"}
    rel = {"1": {}, "2": {"p1": R.P2, "p2": R.P1, "a2": "a1"}, "3": {"p1": R.P3, "p2": R.P1, "a2": "a1", "a3": "a2"}}
    mom = {"1": hhc.p1, "2": hhc.p2, "3": hhc.p3}
    out = []
    lk = 0
    for m in Fm:
        f = sp.expand(m + R.remap(m, sw23))
        if f == 0:
            continue
        xh_ = 0
        for i in "123":
            fi = R.remap(f, rel[i]) if rel[i] else f
            fi = R.polarise(fi, "aK", "KU", "KW")
            eh, gf = G.S2_EH_gf(hhc.ext(f"a{i}"), G.probe_mat("E"), mom[i])
            xh_ += R.strip_t(R.contract_id(fi, I * (eh + gf), "K", "E", "h"))
        fk = R.remap(f, {"p1": R.P2, "p2": R.P1})                 # K at p2, a2-leg at p1, a3 at p3
        xk_ = R.strip_t(-I * R.polarise(fk, "a2", "p1", "gc1"))
        vh, o1 = coords(xh_, ms["H"], "H")
        vk, o2 = coords(xk_, ms["K"], "K")
        lk += (o1 != 0) + (o2 != 0)
        out.append((full("H", vh), full("K", vk)))
    fams["F"] = out
    leak["F"] = lk
    # G family: G(gv; gc1 at p1, a3 at p3), vector index gv, momentum degree 2
    out = []
    lk = 0
    for m in monomials({"gv": 1, "gc1": 1, "a3": 2}, 2):
        xc_ = -sp.Rational(1, 2) * U.H("p2", "p2") * hhc.subst_vec(m, "gv", {"gb2": 1})
        xk_ = -I * U.H("aK", "p2") * hhc.subst_vec(m, "gv", {"aK": 1})
        vc, o1 = coords(sp.expand(xc_), ms["C"], "C")
        vk, o2 = coords(sp.expand(xk_), ms["K"], "K")
        lk += (o1 != 0) + (o2 != 0)
        out.append((full("C", vc), full("K", vk)))
    fams["G"] = out
    leak["G"] = lk
    # P family: Phi(gv; a2 at p2, a3 at p3), symmetrised, momentum degree 3
    out = []
    lk = 0
    relv = {"1": {}, "2": {"p1": R.P2, "p2": R.P1, "a2": "a1"}, "3": {"p1": R.P3, "p2": R.P1, "a2": "a1", "a3": "a2"}}
    for m in monomials({"gv": 1, "a2": 2, "a3": 2}, 3):
        f = sp.expand(m + R.remap(m, sw23))
        if f == 0:
            continue
        xh_ = 0
        for i in "123":
            fi = R.remap(f, relv[i]) if relv[i] else f
            D = G.Dvec(hhc.ext(f"a{i}"), U.V({"p1": 1}) if i == "1" else (hhc.F2 if i == "2" else hhc.F3))
            xh_ += sig * hhc.subst_vec(fi, "gv", D)
        fc = R.remap(f, {"p1": R.P2, "p2": R.P1})                 # cbar at p2, a2-leg at p1, a3 at p3
        fc = hhc.subst_vec(fc, "gv", {"gb2": 1})
        xc_ = R.strip_t(-I * R.polarise(sp.expand(fc), "a2", "p1", "gc1"))
        vh, o1 = coords(sp.expand(R.strip_t(xh_)), ms["H"], "H")
        vc, o2 = coords(xc_, ms["C"], "C")
        lk += (o1 != 0) + (o2 != 0)
        out.append((full("H", vh), full("C", vc)))
    fams["P"] = out
    leak["P"] = lk
    return fams, leak


def main():
    t0 = time.time()
    A, b, Ag, bg, ms, sig = build()
    dA = DomainMatrix.from_Matrix(A)
    full = DomainMatrix.from_Matrix(A.col_join(Ag))
    n = A.shape[1]
    rA = dA.rank()
    rAg = full.rank()
```

```
    res = {"n_unknowns": n, "rank_rows_bose": rA, "dim_N": n - rA, "rank_with_ghost_eq": rAg,
           "dim_N_with_ghost_eq": n - rAg}
    # (1) solvability with the ghost equation
    aug = DomainMatrix.from_Matrix(A.col_join(Ag).row_join(b.col_join(bg)))
    res["solvable_with_ghost_equation"] = aug.rank() == rAg
    # (3) trivial families
    fams, leak = fam_vectors(ms, sig)
    res["family_leak_terms_outside_ansatz"] = leak
    import pickle
    (LOG / "q08_triv.pkl").write_bytes(pickle.dumps({"A": A, "Ag": Ag, "b": b, "bg": bg, "fams": fams}))
    cands = [0, 1, -1, 2, -2, sp.Rational(1, 2), -sp.Rational(1, 2), I, -I, 2 * I, -2 * I, I / 2, -I / 2, 4, -4]
    lam = {}
    best = {}
    for name, pairs in fams.items():
        P = sp.Matrix.hstack(*[sp.Matrix(p) for p, q in pairs])
        Qm = sp.Matrix.hstack(*[sp.Matrix(q) for p, q in pairs])
        S = P.row_join(Qm)
        rS = DomainMatrix.from_Matrix(S).rank()
        rAS = DomainMatrix.from_Matrix(A * S).rank()
        sc = {}
        for l in cands:
            M = P + l * Qm
            sc[str(l)] = DomainMatrix.from_Matrix(M).rank() - DomainMatrix.from_Matrix(A * M).rank()
        lb = max(sc, key=lambda k: sc[k])
        lam[name] = {"n_generators": len(pairs), "dim_span_both_pieces": rS,
                     "dim_N_cap_span_both_pieces_upper_bound": rS - rAS,
                     "best_uniform_lambda": lb, "dim_N_cap_family_at_best_lambda": sc[lb], "scan": sc}
        best[name] = P + sp.sympify(lb) * Qm
    res["families"] = lam
    Ball = sp.Matrix.hstack(*best.values())
    Sall = sp.Matrix.hstack(*[sp.Matrix(v) for pr in fams.values() for pq in pr for v in pq])
    res["dim_N_cap_all_families_upper_bound"] = DomainMatrix.from_Matrix(Sall).rank() - DomainMatrix.from_Matrix(A * Sall).rank()
    res["dim_N_cap_all_families_best_lambda"] = DomainMatrix.from_Matrix(Ball).rank() - DomainMatrix.from_Matrix(A * Ball).rank()
    res["dim_N_mod_trivial_lower"] = res["dim_N"] - res["dim_N_cap_all_families_upper_bound"]
    res["seconds"] = round(time.time() - t0, 1)
    (LOG / "q08_triv.json").write_text(json.dumps(res, indent=1) + "\n")
    print(res)


if __name__ == "__main__":
    main()
```

code/p6q06/q07/release_q07.py

```
#!/usr/bin/env python3
"""XVI-Q Q07 release gate (fails closed; on reject no RELEASE_STATUS.json is left).
(1) code/p6q06/q07/q07_expected.json pinned; EVERY Q01..Q07 certificate equals its pinned copy (Q05 guard-scan
    file count: lower bound); EVERY code/**/*.py, tools/*.py, tests/*.py (incl. the Q05 and Q06 gates) pinned and
    re-hashed; an unpinned file is a reject;
(2) live hardened static scan clean;
(3) semantic: h.h.c tree calibrated, 4D residual 0, every mutation detected, unique fit, matter sector exact,
    remainder explicit and evanescent; evanescent-CT naive test recorded as NOT consistent; joint local CT solvable,
    verified, Bose, evanescent, CT shas; status consistent; O.1 unchanged; Q06 layer accepted; Q07-R1 disclosed."""
from __future__ import annotations
import hashlib, json, sys  # noqa: E401
from pathlib import Path

Q = Path(__file__).resolve().parents[3]
sys.path.insert(0, str(Q / "tools"))
import source_set_q06_q07 as SS  # noqa: E402
CERT = Q / "certificates"
EXPECTED_SHA256 = "8d2c4d5c63816e89e1b883a2cd63df6d1c3457ebe914790c2594a68ff980e746"
STATUS = (Q / "RELEASE_STATUS.json", CERT / "RELEASE_STATUS.json")
HARD_STOPS = {"bulk_symmetry_restored": False, "insertion": "frozen", "XV_B_opened": False, "XIV_J_opened": False,
              "Route_B_discharged": False, "Missing_never_zero": True}
READS: set[str] = set()
Q07_CERTS = ["Q07_hhc_row", "Q07_evanescent_CT", "Q07_P6_status"]
SELF = "code/p6q06/q07/release_q07.py"


def rd(rel):
    READS.add(rel)
    return (Q / rel).read_bytes()


def jr(rel):
    return json.loads(rd(rel))


def sha_b(b):
    return hashlib.sha256(b).hexdigest()
```

```
def strip(o):
    if isinstance(o, dict):
        return {k: strip(v) for k, v in o.items() if k != "seconds"}
    if isinstance(o, list):
        return [strip(v) for v in o]
    return o


def code_files():
    out = {p.relative_to(Q).as_posix() for p in (Q / "code").rglob("*.py") if "__pycache__" not in p.parts}
    out |= {p.relative_to(Q).as_posix() for p in (Q / "tools").glob("*.py")}
    out |= {p.relative_to(Q).as_posix() for p in (Q / "tests").glob("*.py")}
    # Q08-R1 (disclosed scope revision): Q08 files (code/p6q06/q07/q08/, tools/*q08*) are pinned by the Q08 gate
    out = {f for f in out if not (f.startswith("code/p6q06/q07/q08/") or (f.startswith("tools/") and "q08" in f))}
    return out - {SELF}


def checks():
    c = {}
    ep = "code/p6q06/q07/q07_expected.json"
    eb = rd(ep) if (Q / ep).is_file() else b""
    c["expected_file_pinned"] = bool(eb) and sha_b(eb) == EXPECTED_SHA256
    exp = json.loads(eb) if eb else {}
    certs = exp.get("certificates", {})

    def _layer(n, o):
        if n == "Q05_guard_scan" and isinstance(o, dict):
            o = dict(o)
            ok_n = isinstance(o.get("n_files_scanned"), int) and o["n_files_scanned"] >= certs[n]["n_files_scanned"]
            o["n_files_scanned"] = certs[n]["n_files_scanned"] if ok_n else -1
        return o
    bad = [n for n, ref in certs.items() if not (CERT / f"{n}.json").is_file()
           or _layer(n, strip(jr(f"certificates/{n}.json"))) != ref]
    c["every_certified_value_matches_pinned"] = len(certs) >= 35 and set(Q07_CERTS) <= set(certs) and not bad
    fs = exp.get("file_sha256", {})
    c["pinned_files_rehashed"] = bool(fs) and all((Q / f).is_file() and sha_b(rd(f)) == h for f, h in fs.items())
    c["no_unpinned_code_tools_tests_file"] = bool(fs) and code_files() <= set(fs) and \
        {"code/p6q05/release_q05.py", "code/p6q06/release_q06.py"} <= set(fs)
    sys.path.insert(0, str(Q / "code" / "p6q05"))
    import guard_scan2
    READS.add("code/p6q05/guard_scan2.py")
    live = guard_scan2.report()
    c["scan_live_clean_incl_q07"] = live["scan_clean"] is True and live["all_reviewer_evasions_flagged"] is True
    h = jr("certificates/Q07_hhc_row.json")
    c["hhc_tree_calibrated"] = h["tree_calibration"] == {"sigma_F": "I", "tree_row_closes": True,
                                                         "tree_row_without_E_nonzero": True}
    sol = h["unconstrained_fit_4D"]["solution"]
    c["hhc_4D_closed_with_mutations"] = h["residual_4D_zero"] is True and h["residual_4D"] == "0" and \
        len(h["mutations_detected"]) == 6 and all(v is True for v in h["mutations_detected"].values()) and \
        sol is not None and len(set(sol)) == 1 and sol[0] != "0" and h["matter_sector_closes_exactly"] is True
    c["hhc_remainder_explicit_evanescent"] = h["evanescent_remainder_every_term_has_T"] is True and \
        h["evanescent_remainder_n_terms"] > 0 and h["evanescent_remainder_sha256"] == sha_b(h["evanescent_remainder"].encode())
    e = jr("certificates/Q07_evanescent_CT.json")
    nv = e["naive_p22_type"]
    c["naive_CT_recorded_not_consistent"] = nv["ccK"]["zero"] is False and nv["ccb"]["zero"] is False and \
        nv["verdict"].startswith("NOT")
    j = e["joint_local_CT"]
    c["joint_CT_solvable_verified"] = j["solvable"] is True and j["verified_rows_zero"] is True and \
        j["hhh_CT_bose_symmetric"] is True and j["all_CT_evanescent"] is True and j["n_free"] > 0 and \
        all(j[f"particular_CT_{n}_sha256"] == sha_b(j[f"particular_CT_{n}"].encode())
            for n in ("Gamma_Kch", "Gamma_cbar_ch", "Gamma_cstar_cc", "Gamma_hhh")) and \
        e["verdict"].startswith("CANCELLABLE") and "not unique" in e["verdict"]
    st = jr("certificates/Q07_P6_status.json")
    c["status_scoped_honest"] = st["P6_outcome"] == "Partial" and st["flags"]["claim_status"] == "partial" and \
        st["sigma0_closure_kappa3"].startswith("CLOSED at pole level on the kappa^3 one-loop THREE-POINT rows") and \
        "kappa^3 four-point / higher rows" in st["not_implied"] and "finite parts" in st["not_implied"] and \
        st["kappa3_CT_domain_reduction"].startswith("Partial")
    c["revision_disclosed"] = "Q07-R1" in st["disclosed_revisions"] and "Q06-E2" in st["errata"]
    o1 = jr("certificates/Q03_O1_record.json")
    c["O1_record_unchanged"] = o1["O1_closure"] == "CLOSED (scoped)" and o1["O1_closure_confirmed"] is True and \
        st["O1_closure_unchanged"] == "CLOSED (scoped)" and st["bulk_symmetry_restored"] is False
    q6 = jr("certificates/Q06_release.json") if (CERT / "Q06_release.json").is_file() else {}
    c["Q06_layer_accepted"] = q6.get("decision") == "ACCEPT"
    c["gate_reads_in_source_set"] = READS <= set(SS.source_files(Q)) | {f"certificates/{n}.json" for n in certs} | \
        {"certificates/Q03_O1_record.json", "certificates/Q06_release.json"}
    return c, st


def main():
    for p in STATUS:
        p.unlink(missing_ok=True)
    try:
        c, st = checks()
    except Exception as e:  # noqa: BLE001
```

```
        c, st = {"exception": False, "error": repr(e)}, {}
    ok = all(v is True for k, v in c.items() if k != "error")
    rec = {"landmark": "XVI-Q", "release": "Q07", "decision": "ACCEPT" if ok else "REJECT", "checks": c}
    (CERT / "Q07_release.json").write_text(json.dumps(rec, indent=1) + "\n")
    if not ok:
        print(json.dumps(rec, indent=1))
        return 1
    status = {"landmark": "XVI-Q", "release": "Q07", "status": "released (honest partial: P.6 three-point rows)",
              "P6_outcome": st["P6_outcome"], "stages": st["stages"], "sigma0_closure_kappa3": st["sigma0_closure_kappa3"],
              "O1_closure": "CLOSED (scoped)", "O1_closure_confirmed": True, "hard_stops": HARD_STOPS,
              "independent_rebuild_q05": "not requested", "independent_rebuild_q06": "not requested",
              "independent_rebuild_q07": "pending", "execution_pass": True, "claim_status": "partial", "gate": SELF}
    for p in STATUS:
        p.write_text(json.dumps(status, indent=1) + "\n")
    (Q / "ledgers").mkdir(exist_ok=True)
    (Q / "ledgers" / "Q07_milestones.json").write_text(json.dumps({"landmark": "XVI-Q", "release": "Q07", **strip(st)},
                                                                  indent=1) + "\n")
    print("ACCEPT")
    return 0


if __name__ == "__main__":
    raise SystemExit(main())
```

code/p6q06/q07/test_q07.py

```
#!/usr/bin/env python3
"""XVI-Q Q07 tests: gate fail-closed on value / honesty edits and unpinned files; subst_vec / ansatz sanity."""
import json, shutil, subprocess, sys  # noqa: E401
from pathlib import Path
import pytest

Q = Path(__file__).resolve().parents[3]
GATE = "code/p6q06/q07/release_q07.py"


def _copy(tmp):
    dst = tmp / "XVI-Q"
    top = {"build", "dist", "archival", "manuscript"}
    shutil.copytree(Q, dst, ignore=lambda d, names: [n for n in names if n == "__pycache__" or (Path(d) == Q and n in top)])
    return dst


def _edit(root, rel, fn):
    p = root / rel
    o = json.loads(p.read_text())
    fn(o)
    p.write_text(json.dumps(o, indent=1) + "\n")


def _gate(root):
    return subprocess.run([sys.executable, GATE], cwd=root, capture_output=True, text=True)


H, E, S = "certificates/Q07_hhc_row.json", "certificates/Q07_evanescent_CT.json", "certificates/Q07_P6_status.json"
MUT = {
    "hhc_4D_residual": lambda r: _edit(r, H, lambda o: o.update(residual_4D="x")),
    "sigma_F": lambda r: _edit(r, H, lambda o: o["tree_calibration"].update(sigma_F="-I")),
    "hhc_mutation_undetected": lambda r: _edit(r, H, lambda o: o["mutations_detected"].update(drop_E=False)),
    "remainder_value": lambda r: _edit(r, H, lambda o: o.update(evanescent_remainder="0")),
    "naive_claimed": lambda r: _edit(r, E, lambda o: o["naive_p22_type"]["ccK"].update(zero=True)),
    "CT_value": lambda r: _edit(r, E, lambda o: o["joint_local_CT"].update(particular_CT_Gamma_hhh="0")),
    "CT_unique_claim": lambda r: _edit(r, E, lambda o: o.update(verdict="CANCELLABLE uniquely")),
    "closure_overclaim": lambda r: _edit(r, S, lambda o: o.update(sigma0_closure_kappa3="CLOSED")),
    "not_implied_dropped": lambda r: _edit(r, S, lambda o: o.update(not_implied=[])),
    "revision_hidden": lambda r: _edit(r, S, lambda o: o.update(disclosed_revisions={})),
    "unpinned_code_file": lambda r: (r / "code/p6q06/q07/extra.py").write_text("x = 1\n"),
    "q06_gate_edit": lambda r: (r / "code/p6q06/release_q06.py").write_text((r / "code/p6q06/release_q06.py").read_text() + "\n"),
}


@pytest.mark.parametrize("name", sorted(MUT))
def test_gate_rejects(tmp_path, name):
    root = _copy(tmp_path)
    MUT[name](root)
    r = _gate(root)
    assert r.returncode != 0
    assert not (root / "RELEASE_STATUS.json").exists()


def test_gate_accepts(tmp_path):
    r = _gate(_copy(tmp_path))
    assert r.returncode == 0, r.stdout
```

```
def test_subst_vec_and_ansatz():
    sys.path.insert(0, str(Q / "code" / "p6q06" / "q07"))
    import hhc
    import evfit
    import uvpole as U
    e = U.H("gb2", "p1") + U.T("gb2", "a2")
    r = hhc.subst_vec(e, "gb2", {"a3": 2, "p1": 1})
    assert r == 2 * U.H("a3", "p1") + U.H("p1", "p1") + 2 * U.T("a3", "a2")
    ms = evfit.monomials({"z": 1, "gc1": 1}, 2)
    assert ms and all(any(s.name.startswith("T_") for s in m.free_symbols) for m in ms)
```

code/p6q06/release_q06.py

```
#!/usr/bin/env python3
"""XVI-Q Q06 release gate (fails closed; on reject no RELEASE_STATUS.json is left).
(1) code/p6q06/q06_expected.json pinned; EVERY Q01..Q06 certificate equals its pinned copy; EVERY code/**/*.py,
    tools/*.py, tests/*.py file (incl. the Q05 gate) is pinned and re-hashed; an unpinned file is a reject;
(2) hardened static scan (p6q05/guard_scan2) run live over code/ (incl. p6q06): clean, reviewer evasions flagged;
(3) semantic: ghost rows closed on the 4D part with all mutations detected and a unique unconstrained fit, evanescent
    remainder explicit (every term has a T) and NOT claimed closed; G10 second route equal; V4 gauge identity exact;
    fast path equals Expr path; G01/G02 nonzero with Bose checks; h.h.c Missing; kappa^3 sigma0 closure NOT
    established; P6 Partial; 0.1 record unchanged; Q05 layer gate accepted."""
from __future__ import annotations
import hashlib, json, sys  # noqa: E401
from pathlib import Path

Q = Path(__file__).resolve().parents[2]
sys.path.insert(0, str(Q / "tools"))
import source_set_q06 as SS  # noqa: E402
CERT = Q / "certificates"
EXPECTED_SHA256 = "1e8394983c4364b7192baf360ced8a93c1083229d07fc5764ba325f78f4da291"
STATUS = (Q / "RELEASE_STATUS.json", CERT / "RELEASE_STATUS.json")
HARD_STOPS = {"bulk_symmetry_restored": False, "insertion": "frozen", "XV_B_opened": False, "XIV_J_opened": False,
              "Route_B_discharged": False, "Missing_never_zero": True}
READS: set[str] = set()
Q06_CERTS = ["Q06_ghost_rows", "Q06_G10_crosscheck", "Q06_V4_validation", "Q06_fastpath", "Q06_P6_hhh", "Q06_P6_status"]


def rd(rel):
    READS.add(rel)
    return (Q / rel).read_bytes()


def jr(rel):
    return json.loads(rd(rel))


def sha_b(b):
    return hashlib.sha256(b).hexdigest()


def strip(o):
    if isinstance(o, dict):
        return {k: strip(v) for k, v in o.items() if k != "seconds"}
    if isinstance(o, list):
        return [strip(v) for v in o]
    return o


def code_files():
    out = {p.relative_to(Q).as_posix() for p in (Q / "code").rglob("*.py") if "__pycache__" not in p.parts}
    out |= {p.relative_to(Q).as_posix() for p in (Q / "tools").glob("*.py")}
    out |= {p.relative_to(Q).as_posix() for p in (Q / "tests").glob("*.py")}
    # Q07-R1 (disclosed scope revision): Q07 files live under code/p6q06/q07/ and tools/*q06_q07*; they are pinned by
    # the Q07 gate (code/p6q06/q07/release_q07.py), not by this layer
    out = {f for f in out if not (f.startswith("code/p6q06/q07/") or (f.startswith("tools/") and "q07" in f))}
    return out - {"code/p6q06/release_q06.py"}


def checks():
    c = {}
    eb = rd("code/p6q06/q06_expected.json") if (Q / "code/p6q06/q06_expected.json").is_file() else b""
    c["expected_file_pinned"] = bool(eb) and sha_b(eb) == EXPECTED_SHA256
    exp = json.loads(eb) if eb else {}
    certs = exp.get("certificates", {})
    def _layer(n, o):
        # Q07-R1: the recursive scan count grows with later layers; the pinned count is a lower bound
        if n == "Q05_guard_scan" and isinstance(o, dict):
            o = dict(o)
            ok_n = isinstance(o.get("n_files_scanned"), int) and o["n_files_scanned"] >= certs[n]["n_files_scanned"]
            o["n_files_scanned"] = certs[n]["n_files_scanned"] if ok_n else -1
```

```
        return o
    bad = [n for n, ref in certs.items() if not (CERT / f"{n}.json").is_file()
           or _layer(n, strip(jr(f"certificates/{n}.json"))) != ref]
    c["every_certified_value_matches_pinned"] = len(certs) >= 30 and set(Q06_CERTS) <= set(certs) and not bad
    fs = exp.get("file_sha256", {})
    c["pinned_files_rehashed"] = bool(fs) and all((Q / f).is_file() and sha_b(rd(f)) == h for f, h in fs.items())
    c["no_unpinned_code_tools_tests_file"] = bool(fs) and code_files() <= set(fs)
    sys.path.insert(0, str(Q / "code" / "p6q05"))
    import guard_scan2
    READS.add("code/p6q05/guard_scan2.py")
    live = guard_scan2.report()
    c["scan_live_clean_incl_q06"] = live["scan_clean"] is True and live["all_reviewer_evasions_flagged"] is True and \
        "recursive" in live["scope"]
    r = jr("certificates/Q06_ghost_rows.json")
    ok = True
    for t in ("c.c.K", "c.c.cbar"):
        x = r[t]
        sol = x["unconstrained_fit_4D"]["solution_up_to_scale"]
        ok &= x["residual_4D_zero"] is True and x["residual_4D"] == "0" and len(x["mutations_detected"]) == 8 and \
            all(v is True for v in x["mutations_detected"].values()) and sol is not None and "0" not in sol and \
            x["evanescent_remainder_every_term_has_T"] is True and x["evanescent_remainder_n_terms"] > 0 and \
            x["evanescent_remainder_sha256"] == sha_b(x["evanescent_remainder"].encode()) and \
            x["channel_coefficients"]["K1"] == "-1" and x["channel_coefficients"]["K2"] == "1"
    c["ghost_rows_4D_closed_with_mutations"] = ok
    c["ghost_rows_evanescent_not_overclaimed"] = "OPEN" in r["status"] and "Grassmann" in r["diagnosis"]
    g10 = jr("certificates/Q06_G10_crosscheck.json")
    c["G10_second_route_equal"] = g10["route2_equals_Q05_G10"] is True
    v = jr("certificates/Q06_V4_validation.json")
    c["V4_gauge_identity_exact"] = v["gauge_n2_V3_vs_V2_zero"] is True and v["gauge_n3_V4_vs_V3_zero"] is True and \
        v["V4_permutation_symmetric"] is True
    f = jr("certificates/Q06_fastpath.json")
    c["fastpath_equals_expr_path"] = f == {"G06_fast_equals_expr": True, "G05_fast_equals_expr": True}
    h = jr("certificates/Q06_P6_hhh.json")["graphs"]
    c["G01_G02_evaluated"] = set(h) == {"G01", "G02"} and all(
        x["zero"] is False and x["sha256"] == sha_b(x["pole_over_U_d4_residue"].encode()) and x["bose_12_symmetric"] is True and
x["bose_13_symmetric"] is True
        for x in h.values()) and h["G02"]["channels_bose_related"] is True
    st = jr("certificates/Q06_P6_status.json")
    c["honest_partial"] = st["P6_outcome"] == "Partial" and st["sigma0_closure_kappa3"].startswith("NOT") and \
        st["stages"]["graphs_missing"] == [] and st["stages"]["row_h_h_c"].startswith("Missing") and \
        st["kappa3_defect"] == "Missing" and st["kappa3_CT_solve"] == "Missing" and st["finite_parts"].startswith("Missing")
    c["errata_recorded"] = set(st["errata"]) == {"Q06-E1", "Q06-E2"} and "Q06-R1" in st.get("disclosed_revisions", {})
    o1 = jr("certificates/Q03_O1_record.json")
    c["O1_record_unchanged"] = o1["O1_closure"] == "CLOSED (scoped)" and o1["O1_closure_confirmed"] is True and \
        st["O1_closure_unchanged"] == "CLOSED (scoped)" and st["bulk_symmetry_restored"] is False
    q5 = jr("certificates/Q05_release.json") if (CERT / "Q05_release.json").is_file() else {}
    c["Q05_layer_accepted"] = q5.get("decision") == "ACCEPT"
    c["gate_reads_in_source_set"] = READS <= set(SS.source_files(Q)) | {f"certificates/{n}.json" for n in certs} | \
        {f"certificates/{n}.json" for n in Q06_CERTS} | {"certificates/Q03_O1_record.json", "certificates/Q05_release.json"}
    return c, st


def main():
    for p in STATUS:
        p.unlink(missing_ok=True)
    try:
        c, st = checks()
    except Exception as e:  # noqa: BLE001
        c, st = {"exception": False, "error": repr(e)}, {}
    ok = all(v is True for k, v in c.items() if k != "error")
    rec = {"landmark": "XVI-Q", "release": "Q06", "decision": "ACCEPT" if ok else "REJECT", "checks": c}
    (CERT / "Q06_release.json").write_text(json.dumps(rec, indent=1) + "\n")
    if not ok:
        print(json.dumps(rec, indent=1))
        return 1
    status = {"landmark": "XVI-Q", "release": "Q06", "status": "released (honest partial: P.6 continued)",
              "P6_outcome": st["P6_outcome"], "stages": st["stages"], "O1_closure": "CLOSED (scoped)",
              "O1_closure_confirmed": True, "hard_stops": HARD_STOPS,
              "independent_rebuild_passed_q04": True, "independent_rebuild_q05": "not requested (user: Q06 only)",
              "independent_rebuild_q06": "pending",
              "execution_pass": True, "claim_status": "partial", "gate": "code/p6q06/release_q06.py"}
    for p in STATUS:
        p.write_text(json.dumps(status, indent=1) + "\n")
    (Q / "ledgers").mkdir(exist_ok=True)
    (Q / "ledgers" / "Q06_milestones.json").write_text(json.dumps({"landmark": "XVI-Q", "release": "Q06",
        "P6_outcome": st["P6_outcome"], "stages": st["stages"], "hard_stops": HARD_STOPS}, indent=1) + "\n")
    print("ACCEPT")
    return 0


if __name__ == "__main__":
    raise SystemExit(main())
```

code/p6q06/rows6.py

```
#!/usr/bin/env python3
"""XVI-Q Q06 ghost rows c.c.cbar / c.c.K at UV-pole level -- diagnosis and fix of the Q05 OPEN residual.
Channels per orientation o in {1: (c_i, c_j) = (c1, c2), 2: (c2, c1)} as in p6q05/rows.ghost_row:
  h_o : Gamma_{X c_i h}[h -> s0 h(c_j)]         (G03+G07 / G04+G09)
  K_o : delta S1/delta h|_{X c_j} . Gamma_{K c_i} (D13 own)
  c1  : Gamma_{c* c c} (G10) x kinetic X.c ;  c2 : Gamma_{X c} (D13 / D06 own) x V_{c* c c}
FIX (Q06): in the K channel the external c_i (inside Gamma_{K c_i}) stands to the RIGHT of c_j (inside the S1 vertex),
the opposite order to the h channel (c_i in Gamma, c_j in s0 h); bringing the monomial to the row ordering c1 c2 costs
one extra Grassmann transposition: K_o enters with sign -sgn(o) where Q05 used +sgn(o).  The tree row has no K channel,
so the tree calibration (sigma_c) is unchanged.
Result: with the fix the 4D (non-evanescent) part of both rows vanishes; the channel coefficients are the UNIQUE
solution of an unconstrained 6-coefficient fit (each channel and orientation free), so every channel is needed.
What remains is a purely evanescent remainder (every term carries one T(pol, pol)), recorded explicitly."""
from __future__ import annotations
import hashlib, json, sys  # noqa: E401
from pathlib import Path
import sympy as sp

HERE = Path(__file__).resolve().parent
Q = HERE.parents[1]
sys.path.insert(0, str(HERE.parent / "p6q05"))
import rows as R  # noqa: E402
import gravity as G  # noqa: E402
import uvpole as U  # noqa: E402
import graphs as GR  # noqa: E402

I = sp.I
SIGMA_C = {"ccb": -1, "ccK": 1}          # tree calibration (Q05, unchanged)


def parts(target):
    gc1, gc2 = U.vec("gc1"), U.vec("gc2")
    p1v, p2v = U.vec("p1"), U.vec("p2")
    if target == "ccb":
        def Xside(cv, q, E, k):
            return G.V_cb_c_h(U.vec("gb3"), cv, q, E, k)

        def kin(Xp, q):
            return R.kin_cbc(U.vec("gb3"), Xp, q)
        loopX = ("G04", "G09")
    else:
        def Xside(cv, q, E, k):
            return G.V_K_c_h(GR.ext("aK"), cv, q, E, k)

        def kin(Xp, q):
            return R.kin_Kc(GR.ext("aK"), Xp, q)
        loopX = ("G03", "G07")
    out = {}
    for (ci, pi, cj, pj, o) in (("gc1", p1v, "gc2", p2v, 1), ("gc2", p2v, "gc1", p1v, 2)):
        Gx = sum(R.load(g) for g in loopX)
        Pi = R.P1 if ci == "gc1" else R.P2
        Gx = R.remap(Gx, {"p1": Pi, "p2": R.P3, "gc1": ci, "gb2": "gb3"})
        out[f"h{o}"] = (-I) * R.polarise(Gx, "a3", "p1" if o == 2 else "p2", cj)
        out[f"K{o}"] = R.contract_id(R.D13_own(ci, pi), Xside(U.vec(cj), pj, G.probe_mat("E"), pi), "K", "E", "h")
    Vcs = G.V_cs_c_c(G.probe_vec("Z"), gc1, p1v, gc2, p2v)
    q3 = -(p1v + p2v)
    g10 = R.load("G10")
    kz = kin(G.probe_vec("X"), q3)
    g10p = g10.xreplace({x: sp.Symbol("_".join("ZZ" if q == "z" else q for q in x.name.split("_")))
                         for x in g10.free_symbols if "z" in x.name.split("_")})
    out["c1"] = sp.expand(G.contract(sp.expand(kz * g10p / I), [("X", "ZZ")]))
    gx = R.D06_own("gb3", "X", q3) if target == "ccb" else R.strip_t(R.D13_K_ext("X", q3))
    out["c2"] = R.contract_id(gx, Vcs, "X", "Z", "c")
    return {k: R.strip_t(v) for k, v in out.items()}


def coeffs(target, q05=False):
    s = SIGMA_C[target]
    kq = 1 if q05 else -1
    return {"h1": 1, "h2": -1, "K1": kq, "K2": -kq, "c1": s, "c2": s}


def four_d(e):
    return sp.expand(e.xreplace({x: 0 for x in e.free_symbols if x.name.startswith("T_")}))


def fit(P):
    cs = sp.symbols("x0:6")
    ks = sorted(P)
    tot = sp.expand(sum(c * four_d(P[k]) for c, k in zip(cs, ks)))
    gens = [x for x in tot.free_symbols if x not in cs]
    sol = sp.linsolve(sp.Poly(tot, *gens).coeffs(), cs)
    return ks, [str(v) for v in list(sol)[0]] if sol else None
```

```
def row(target):
    P = parts(target)
    C, Cq = coeffs(target), coeffs(target, q05=True)
    Rf = sp.expand(sum(C[k] * P[k] for k in P))
    Rq = sp.expand(sum(Cq[k] * P[k] for k in P))
    muts = {}
    for k in P:
        muts[f"drop_{k}"] = four_d(sp.expand(Rf - C[k] * P[k])) != 0
    muts["flip_sigma_c"] = four_d(sp.expand(Rf - 2 * C["c1"] * (P["c1"] + P["c2"]))) != 0
    muts["Q05_K_sign"] = four_d(Rq) != 0
    ev = Rf
    ks, sol = fit(P)
    return {"row": target, "channel_coefficients": {k: str(v) for k, v in C.items()},
            "channel_terms": {k: len(sp.Add.make_args(v)) for k, v in P.items()},
            "unconstrained_fit_4D": {"channels": ks, "solution_up_to_scale": sol},
            "residual_4D": str(four_d(Rf)), "residual_4D_zero": four_d(Rf) == 0,
            "evanescent_remainder": str(ev), "evanescent_remainder_n_terms": len(sp.Add.make_args(ev)),
            "evanescent_remainder_sha256": hashlib.sha256(str(ev).encode()).hexdigest(),
            "evanescent_remainder_every_term_has_T": all(any(x.name.startswith("T_") for x in t.free_symbols)
                                                         for t in sp.Add.make_args(ev)),
            "Q05_assembly_residual_4D_n_terms": len(sp.Add.make_args(four_d(Rq))),
            "mutations_detected": muts}


if __name__ == "__main__":
    out = {t: row(t) for t in ("ccK", "ccb")}
    (Q / "build" / "logs" / "q06_ghost_rows.json").write_text(json.dumps(out, indent=1) + "\n")
    for t, r in out.items():
        print(t, r["residual_4D_zero"], r["evanescent_remainder_n_terms"], r["unconstrained_fit_4D"], r["mutations_detected"])
```

code/p6q06/v4.py

```
#!/usr/bin/env python3
"""XVI-Q Q06: quartic Einstein-Hilbert vertex V4_grav from the XVI-P action (same conventions as p6q05/gravity.py):
S_EH = -(2/kappa^2) int L_GG, L_GG = sqrt(-g) g^{mn} g^{ac} g^{bd} (G_{c,mb} G_{d,na} - G_{c,mn} G_{d,ab}), g = eta + kappa h.
The prefactor P = sqrt(-g) g^{mn} g^{ac} g^{bd} is expanded to second order (polarised coefficient of t1 t2):
  sqrt(-g):  -1/2 tr(E1 E2) + 1/4 trE1 trE2 ;  g^{-1}: -E (1st), E1 E2 + E2 E1 (2nd).
Validation (no non-scaleless XVI-P graph contains V4): the Einstein-Hilbert gauge identity order by order,
  delta0 S_{n+1} + delta1 S_n = 0,  delta0 h = d xi + d xi,  delta1 h = L_xi h,
checked exactly for n = 2 (V3 vs V2, machinery control) and n = 3 (V4 vs V3), plus total permutation symmetry of V4."""
from __future__ import annotations
import itertools, sys  # noqa: E401
from pathlib import Path
import sympy as sp

HERE = Path(__file__).resolve().parent
sys.path.insert(0, str(HERE.parent / "p6q05"))
import gravity as G  # noqa: E402
import uvpole as U  # noqa: E402
import tensor as TT  # noqa: E402
from tensor import ETA, ev  # noqa: E402

I = sp.I
L = G.L


def _slot(M, x, y):
    return [("m", M, x, y)]


def _chain(M1, M2, x, y):
    z = L()
    return [("m", M1, x, z), ("m", M2, z, y)]


def prefactor_terms2(E1, E2, m, n, a, c, b, d):
    """coefficient of t1 t2 in sqrt(-g) g^{mn} g^{ac} g^{bd} at h = t1 E1 + t2 E2."""
    slots = [(m, n), (a, c), (b, d)]
    i1, i2 = L(), L()
    s2 = sp.Rational(1, 4) * E1.tr() * E2.tr() - sp.Rational(1, 2) * ev((1, [("m", E1, i1, i2), ("m", E2, i2, i1)]))
    out = []

    def build(fill):
        f = []
        for k, (x, y) in enumerate(slots):
            f += fill.get(k, None) or _slot(ETA(), x, y)
        return f
    out.append((s2, build({})))
    for k, (x, y) in enumerate(slots):
        out.append((-sp.Rational(1, 2) * E1.tr(), build({k: _slot(E2, x, y)})))
        out.append((-sp.Rational(1, 2) * E2.tr(), build({k: _slot(E1, x, y)})))
```

```
        out.append((1, build({k: _chain(E1, E2, x, y)})))
        out.append((1, build({k: _chain(E2, E1, x, y)})))
    for k1, k2 in itertools.combinations(range(3), 2):
        (x1, y1), (x2, y2) = slots[k1], slots[k2]
        out.append((1, build({k1: _slot(E1, x1, y1), k2: _slot(E2, x2, y2)})))
        out.append((1, build({k1: _slot(E2, x1, y1), k2: _slot(E1, x2, y2)})))
    return out


def LGG_piece2(pref, w1, w2):
    m, n, a, b, c, d = (L() for _ in range(6))
    P = prefactor_terms2(pref[0], pref[1], m, n, a, c, b, d)
    t1 = G._mul(G._mul(P, G.gamma_terms(w1[0], w1[1], c, m, b)), G.gamma_terms(w2[0], w2[1], d, n, a))
    m2, n2, a2, b2, c2, d2 = (L() for _ in range(6))
    P2 = prefactor_terms2(pref[0], pref[1], m2, n2, a2, c2, b2, d2)
    t2 = G._mul(G._mul(P2, G.gamma_terms(w1[0], w1[1], c2, m2, n2)), G.gamma_terms(w2[0], w2[1], d2, a2, b2))
    return ev(*t1) - ev(*t2)


def LGG4(w1, w2, w3, w4):
    ws = [w1, w2, w3, w4]
    tot = 0
    for i, j in itertools.permutations(range(4), 2):
        rest = [ws[k][0] for k in range(4) if k not in (i, j)]
        tot += LGG_piece2(rest, ws[i], ws[j])
    return sp.expand(tot)


def V4_grav(w1, w2, w3, w4):
    """i x delta^4 S_EH (kappa^2 stripped): S_EH quartic = -2 kappa^2 L_GG^(4)."""
    return sp.expand(I * (-2) * LGG4(w1, w2, w3, w4))


def V2_EH(w1, w2):
    return sp.expand(I * (-2) * G.LGG2(w1, w2))


def gauge_check(n):
    """delta0 V_{n+1} + delta1 V_n with xi (momentum q) and n graviton waves a_i (momenta k_i, sum k + q = 0)."""
    xi, q = U.vec("xi"), U.vec("q")
    ks = [U.vec("k1"), U.vec("k2")] + ([U.vec("k3")] if n == 3 else [])
    ks = ks[:n - 1] + [-(sum(ks[:n - 1], U.V()) + q)]
    Es = [TT.sym(U.vec(f"a{i+1}"), U.vec(f"a{i+1}")) for i in range(n)]
    W = [(Es[i], ks[i]) for i in range(n)]
    lin = (TT.sym(q, xi, -2 * I), q)
    Vn1 = G.V3_grav if n == 2 else V4_grav
    Vn = V2_EH if n == 2 else G.V3_grav
    tot = Vn1(lin, *W)
    for i in range(n):
        Wi = list(W)
        Wi[i] = (G.Lc_h(Es[i], ks[i], xi, q), ks[i] + q)
        tot += Vn(*Wi)
    return sp.expand(tot)


def symmetry_check():
    ks = [U.vec("k1"), U.vec("k2"), U.vec("k3")]
    ks.append(-(ks[0] + ks[1] + ks[2]))
    W = [(TT.sym(U.vec(f"a{i+1}"), U.vec(f"a{i+1}")), ks[i]) for i in range(4)]
    base = V4_grav(*W)
    sw = V4_grav(W[1], W[0], W[3], W[2])
    return sp.expand(base - sw) == 0, len(sp.Add.make_args(base))


if __name__ == "__main__":
    import json, time  # noqa: E401
    t0 = time.time()
    r = {"gauge_n2_V3_vs_V2_zero": gauge_check(2) == 0}
    print(r, time.time() - t0, flush=True)
    g3 = gauge_check(3)
    r["gauge_n3_V4_vs_V3_zero"] = g3 == 0
    r["gauge_n3_residual_terms"] = len(sp.Add.make_args(g3))
    print(r, time.time() - t0, flush=True)
    r["V4_permutation_symmetric"], r["V4_terms_generic"] = symmetry_check()
    r["seconds"] = round(time.time() - t0, 1)
    print(r, flush=True)
    (HERE.parents[1] / "build" / "logs" / "q06_V4_validation.json").write_text(json.dumps(r, indent=1) + "\n")
```

code/q01/ell4_independent.py

```
"""XVI-Q Q01 0.2(a): INDEPENDENT reconstruction of the full 10-component ell_4 and exact comparison with the XVI-P p23
export (3360 terms).  Route (independent of XVI-P's quartic.py): the Gamma-Gamma Lagrangian
    L_GG = sqrt(-g) g^{mn} (Gamma^a_{mb} Gamma^b_{na} - Gamma^a_{mn} Gamma^b_{ab}),   g = eta + eps H,
```

```
with H_{mk}, dH_{mk,r} independent jet symbols, g^{-1} by the Neumann series and sqrt(-g) by exp(tr log / 2), both
truncated exactly at eps^4.  ell_4 := s * [eps^4] L_GG with the overall sign/normalisation s fixed by matching XVI-O's
conformal probe ONLY (one number), then the full polynomial is compared term by term."""
from __future__ import annotations
import json
import sys
import time
from pathlib import Path

import sympy as sp

QROOT = Path(__file__).resolve().parents[2]
ETA = [1, -1, -1, -1]
EPS = sp.Symbol("eps")


def Hs(m, k):
    a, b = sorted((m, k))
    return sp.Symbol(f"H{a}{b}")


def dHs(m, k, r):
    a, b = sorted((m, k))
    return sp.Symbol(f"dH{a}{b}_{r}")


def build():
    """exact eps-grading: [eps^4] L_GG = sum over (H-degree split) with two Gammas (each one dH); computed in the
    sparse polynomial ring QQ[H, dH] (exact)."""
    from sympy.polys.rings import ring
    from sympy.polys.domains import QQ
    names = [f"H{a}{b}" for a in range(4) for b in range(a, 4)] + \
            [f"dH{a}{b}_{r}" for a in range(4) for b in range(a, 4) for r in range(4)]
    R, *gens = ring(",".join(names), QQ)
    gd = dict(zip(names, gens))
    H = [[gd[f"H{min(m, k)}{max(m, k)}"] for k in range(4)] for m in range(4)]
    dH = [[[gd[f"dH{min(m, k)}{max(m, k)}_{r}"] for r in range(4)] for k in range(4)] for m in range(4)]
    A = [[ETA[m] * H[m][k] for k in range(4)] for m in range(4)]          # eta^{-1} H

    def mm(X, Y):
        return [[sum((X[i][k] * Y[k][j] for k in range(4)), R.zero) for j in range(4)] for i in range(4)]
    I4 = [[R.one if i == j else R.zero for j in range(4)] for i in range(4)]
    A2 = mm(A, A)
    # graded pieces: g^{-1} = (1 - eps A + eps^2 A^2) eta ;  entries (k: matrix) with [i][j] -> X[i][j]*ETA[j]
    ginv = {0: [[I4[i][j] * ETA[j] for j in range(4)] for i in range(4)],
            1: [[-A[i][j] * ETA[j] for j in range(4)] for i in range(4)],
            2: [[A2[i][j] * ETA[j] for j in range(4)] for i in range(4)]}
    t1 = sum((A[i][i] for i in range(4)), R.zero)
    t2 = sum((A2[i][i] for i in range(4)), R.zero)
    # sqrt(det(1+eps A)) = 1 + eps t1/2 + eps^2 (t1^2/8 - t2/4) + ...
    sq = {0: R.one, 1: t1 / 2, 2: t1 ** 2 / 8 - t2 / 4}
    # Gamma_k^a_{mn} (H-degree k): 1/2 ginv_k^{ab} (dH_{bn,m} + dH_{bm,n} - dH_{mn,b})
    G = {k: [[[sum((ginv[k][a][b] * (dH[b][n][m] + dH[b][m][n] - dH[m][n][b]) for b in range(4)), R.zero) / 2
               for n in range(4)] for m in range(4)] for a in range(4)] for k in range(3)}
    L = R.zero
    for i in range(3):
        for j in range(3 - i):
            for k in range(3 - i - j):
                l_ = 2 - i - j - k
                acc = R.zero
                for m in range(4):
                    for n in range(4):
                        gmn = ginv[j][m][n]
                        if not gmn:
                            continue
                        inner = R.zero
                        for a in range(4):
                            for b in range(4):
                                inner += G[k][a][m][b] * G[l_][b][n][a] - G[k][a][m][n] * G[l_][b][a][b]
                        acc += gmn * inner
                L += sq[i] * acc
    return sp.expand(L.as_expr())


def probe(e):
    phi, f = sp.Symbol("phi"), sp.symbols("f0:4")
    sub = {}
    for s in e.free_symbols:
        n = s.name
        if n.startswith("dH"):
            m, k, r = int(n[2]), int(n[3]), int(n.split("_")[1])
            sub[s] = ETA[m] * f[r] if m == k else 0
        elif n.startswith("H"):
            m, k = int(n[1]), int(n[2])
```

```
            sub[s] = ETA[m] * phi if m == k else 0
    return sp.expand(e.xreplace(sub)), phi, f


def main(export_path):
    t0 = time.time()
    exp_ = sp.sympify(Path(export_path).read_text())            # XVI-P p23 export (srepr)
    c4 = build()
    xo = None
    pr_c4, phi, f = probe(c4)
    xo = sp.expand(-3 * phi ** 2 * f[0] ** 2 + 3 * phi ** 2 * (f[1] ** 2 + f[2] ** 2 + f[3] ** 2))   # XVI-O probe
    s = sp.cancel(xo / pr_c4) if pr_c4 != 0 else None
    mine = sp.expand(s * c4)
    diff = sp.expand(mine - exp_)
    pr_exp, _, _ = probe(exp_)
    out = {"route": "independent Gamma-Gamma series (this file), not XVI-P quartic.py",
           "normalisation_from_XVI_O_probe": str(s), "n_terms_mine": len(sp.Add.make_args(mine)),
           "n_terms_export": len(sp.Add.make_args(exp_)), "full_polynomial_equal": diff == 0,
           "n_terms_difference": 0 if diff == 0 else len(sp.Add.make_args(diff)),
           "export_on_XVI_O_probe_equal": sp.expand(pr_exp - xo) == 0, "seconds": round(time.time() - t0, 1)}
    # mutation: one coefficient of the export changed must be detected
    t1 = sp.Add.make_args(exp_)[0]
    out["mutation_one_term_x2_detected"] = sp.expand(mine - (exp_ + t1)) != 0
    print(json.dumps(out, indent=1))
    return out


if __name__ == "__main__":
    main(sys.argv[1])
```

code/q01/guard_probe.py

```
import sys, json; from pathlib import Path; sys.path.insert(0, str(Path(__file__).resolve().parent))
from sympy.polys.polytools import rem as captured_rem
import qguard, sympy as sp, sympy.polys as spp
d=qguard.d; y=sp.Symbol('y'); Dm=sp.Dummy('D'); X=sp.Symbol('X')
E={"lin":d*X+1/(d-1),"pow":1/(d**2-1),"alone":d,"mixed":(d**2-2*d-2)/(32*(d**2-1))}
num=lambda e: sp.numer(sp.together(e))
P={"polys_ns_rem":lambda e: spp.rem(num(e),d-4,d),"captured_rem":lambda e: captured_rem(num(e),d-4,d),
   "reduced":lambda e: sp.reduced(num(e),[d-4],d),"groebner":lambda e: sp.groebner([num(e)-X, d-4],d,X),
   "Matrix_Dummy":lambda e: sp.Matrix([e]).subs(d,Dm).subs(Dm,4),"rep_eval":lambda e: sp.Poly(num(e),d).rep.eval(4),
   "rem":lambda e: sp.rem(num(e),d-4,d),"resultant":lambda e: sp.resultant(num(e),d-4,d),"compose":lambda e: sp.compose(num(e),4,d),
   "Poly_shift":lambda e: sp.Poly(num(e),d).shift(-4),"Matrix_y":lambda e: sp.Matrix([e]).subs(d,y).subs(y,4),
   "subs":lambda e: e.subs(d,4),"xreplace":lambda e: e.xreplace({d:4}),"evalf":lambda e: e.evalf(subs={d:4}),
   "lambdify":lambda e: sp.lambdify(d,e)(4),"limit":lambda e: sp.limit(e,d,4),"apart_residue":lambda e: sp.apart(num(e)/(d-4),d),"apart_list":lambda
e: sp.apart_list(num(e)/(d-4),d)}
miss=[]
for k,f in P.items():
    for n,e in E.items():
        try: f(e); miss.append((k,n,"NOT_RAISED"))
        except qguard.PrematureD4Substitution: pass
        except Exception as x: miss.append((k,n,type(x).__name__))
e = E["mixed"]
alg = {"cancel_factor": str(sp.factor(sp.cancel(e + 1 / (d + 1)))), "domain_only_poly": str(sp.Poly(X**2*e + X, X).as_expr().equals(X**2*e + X)),
       "authorised_value": str(qguard.dred.substitute_d(e, 4, "Q01 probe")), "d_free_rem": str(sp.rem(X**3 + 1, X - 1, X))}
print(json.dumps({"forms": sorted(P), "n_forms": len(P), "n_exprs": len(E), "missed": miss, "algebra": alg,
                  "entry_points": qguard.ENTRY_POINTS}))
```

code/q01/o2_q01.py

```
#!/usr/bin/env python3
"""XVI-Q Q01: writes certificates Q01_intake.json, Q01_O2.json, Q01_guard.json from the build products
(build/xvip_run = re-executed XVI-P p23 classical stages; build/mut_* = master mutations)."""
from __future__ import annotations
import hashlib
import json
import subprocess
import sys
from pathlib import Path

Q = Path(__file__).resolve().parents[2]
sys.path.insert(0, str(Q / "code" / "q01"))
import q01_conventions as QC  # noqa: E402
import ell4_independent as L4  # noqa: E402
import o2b_antibracket as AB  # noqa: E402

RUN = Q / "build" / "xvip_run"
CERT = Q / "certificates"


def sha(p):
    return hashlib.sha256(Path(p).read_bytes()).hexdigest()
```

```
def content_sha(p):
    sys.path.insert(0, str(RUN / "code" / "classical"))
    import certio
    return certio.file_content_sha(p, ())


def intake():
    imp = json.loads((Q / "intake" / "imports.json").read_text())
    ok = {k: (Q / k).is_file() and sha(Q / k) == v["sha256"] for k, v in imp["files"].items()}
    z = __import__("zipfile").ZipFile(Q / "provenance/XVI-P/XVI-P-p23-source-only.zip")
    extracted_ok = all(sha(Q / "imports" / ("XVI-P-p23" + n[len("XVI-P"):])) == hashlib.sha256(z.read(n)).hexdigest()
                       for n in z.namelist() if not n.endswith("/"))
    return {"pins_ok": all(ok.values()), "n_pins": len(ok), "pins": ok,
            "XVI_P_p23_zip_ok": imp["XVI_P_p23_zip_sha256"] == QC.XVI_P_P23_ZIP_SHA256,
            "XVI_P_p23_extracted_identical": extracted_ok, "n_extracted": len([n for n in z.namelist() if not n.endswith("/")]),
            "O1_ledger_sha_ok": imp["files"]["provenance/XVI-O/ledgers/O_quantum_target.json"]["sha256"] == QC.O1_LEDGER_SHA256,
            "errata": json.loads((Q / "intake" / "errata_p23.json").read_text())["errata"],
            "registration": json.loads((Q / "registration" / "local_registration.json").read_text())["registered"]}


def o2():
    ex = {k: sha(RUN / "exports" / f"ell{k}_generating_polynomial.srepr") for k in (2, 3, 4)}
    a = L4.main(RUN / "exports" / "ell4_generating_polynomial.srepr")
    b = {"unmutated": AB.main(RUN / "exports")}
    for m in ("scale_ell4_term", "drop_s1"):
        import importlib
        importlib.reload(AB)
        r = AB.main(RUN / "exports", mutate=m)
        b[f"mutation_{m}_detected"] = not r["kappa2"]["antibracket_af0_vanishes"]
    importlib.reload(AB)
    master = json.loads((RUN / "certificates" / "P13_master_kappa2.json").read_text())
    brst = json.loads((RUN / "certificates" / "P13_brst_derivation.json").read_text())
    same = {c: content_sha(RUN / "certificates" / f"{c}.json") == content_sha(Q / "provenance" / "XVI-P" / "certificates" / f"{c}.json")
            for c in QC.EXPECTED["classical_certs_content_identical_to_p23"]}
    muts = {}
    for m in QC.EXPECTED["master_mutation_residuals"]:
        c = json.loads((Q / "build" / f"mut_{m}" / "certificates" / "P13_master_kappa2.json").read_text())
        muts[m] = {"claim_status": c["claim_status"], "n_residuals": len(c["required_master_residuals"])}
    tbl = master["table_by_kappa_and_antifield_number"]["kappa2"]
    items = {
        "a_ell4": {"status": "Constructed" if (a["full_polynomial_equal"] and a["export_on_XVI_O_probe_equal"]
                                               and ex[4] == QC.EXPORT_SHA256[4]) else "Partial",
                   "argument": "XVI-P p23 ell_4 export (3360 terms, re-executed byte-identical) equals an INDEPENDENT "
                               "Gamma-Gamma reconstruction term by term; the single normalisation (-2) is fixed by "
                               "XVI-O's conformal probe and equals XVI-O's convention ell_4 = -2[eps^4](sqrt(-q)R); the "
                               "export also reproduces XVI-O's probe exactly", "detail": a},
        "b_antibracket_expansion": {
            "status": "Constructed" if all(b["unmutated"][k]["antibracket_af0_vanishes"] for k in ("kappa0", "kappa1", "kappa2"))
            and b["mutation_scale_ell4_term_detected"] and b["mutation_drop_s1_detected"] else "Partial",
            "argument": "(S,S)|af0,grav at kappa^2 = 2 int W_2, W_2 = sum E(ell_4) s0 h + E(ell_3) s1 h (s2 = 0 derived) -- the "
                        "explicit component form of 2(SO,S2)+(S1,S1); W_2 has 15428 terms and E_{c^mu}(W_2) = 0 exactly "
                        "(independent XVI-Q jet engine; agrees with XVI-P grav_noether re-executed)", "detail": b},
        "c_ghost_antifield": {
            "status": "Constructed" if (all(tbl.values()) and brst["claim_status"] == "s2_derived_zero_all_sectors"
                                        and brst["nilpotency_exact_all_orders"] and all(same.values())
                                        and all(muts[m]["n_residuals"] == n for m, n in QC.EXPECTED["master_mutation_residuals"].items()))
            else "Partial",
            "argument": "re-executed exact XVI-P P13 master: af1/af2 (invariant and Psi-transformed variables) and the af0 "
                        "ghost/gauge-fixing cell vanish at kappa^0..kappa^2 with s2 DERIVED zero in all sectors (not "
                        "invented; answers XVI-O's 'Absent under inherited s'); s nilpotent exactly; 7 mutations each "
                        "produce residuals. Evidence = re-execution of XVI-P's engine (no second independent engine for "
                        "this sector in Q01).", "kappa2_table": tbl, "s2": brst["s2_per_sector"]}}
    st = {k: v["status"] for k, v in items.items()}
    o2_status = "Constructed" if all(v == "Constructed" for v in st.values()) else "Partial"
    return {"equation": "2(SO,S2) + (S1,S1) = 0 on S_inv (classical BV master at kappa^2)",
            "items": items, "item_status": st, "O2_kappa2_master_status": o2_status,
            "scope": "classical, S_inv, through O(kappa^2), m = 0; NOT quantum, NOT the prepared S_D, NOT the insertion",
            "exports_sha256": ex, "exports_identical_to_p23": all(ex[k] == QC.EXPORT_SHA256[k] for k in ex),
            "classical_certs_content_identical_to_p23": same, "master_mutations": muts,
            "W_terms": {k: b["unmutated"][k]["W_terms"] for k in ("kappa0", "kappa1", "kappa2")}}


def guard():
    p = subprocess.run([sys.executable, str(Q / "code" / "q01" / "guard_probe.py")], capture_output=True, text=True,
                       timeout=600)
    return json.loads(p.stdout.strip().splitlines()[-1])


def main():
    CERT.mkdir(exist_ok=True)
    out = {"Q01_intake": intake(), "Q01_O2": o2(), "Q01_guard": guard()}
    for k, v in out.items():
```

```
        (CERT / f"{k}.json").write_text(json.dumps({"landmark": "XVI-Q", "release": "Q01", **v}, indent=1, default=str) + "\n")
    print(json.dumps({"intake": out["Q01_intake"]["pins_ok"], "O2": out["Q01_O2"]["item_status"],
                      "guard_missed": out["Q01_guard"]["missed"]}))


if __name__ == "__main__":
    main()
```

code/q01/o2b_antibracket.py

```
"""XVI-Q Q01 O.2(b): explicit O(kappa^n) antibracket expansion, af-0 gravitational sector, n = 0, 1, 2, on an
INDEPENDENT jet engine (this file; exact sparse polynomial ring over QQ), from the ell_2..ell_4 exports.

Derivation (HT antibracket, XVI-O convention sheet): S = S_EH + S_af + ..., S_af = int h*^{mn} (s h)_{mn} + ...,
S_EH = sum_n kappa^{n-2} int ell_n,  s = s0 + kappa s1 (+ kappa^2 s2, s2 = 0 DERIVED: P13_brst_derivation).
(S,S)|_{Phi*=0, grav} = 2 int sum_{m<=n} (delta S_EH / delta h_{mn}) (s h)_{mn}.  The kappa^n coefficient
(n = 0,1,2) is 2 int W_n with W_n = sum_{m<=n'} [E_{mn'}(ell_{n+2}) (s0 h)_{mn'} + E_{mn'}(ell_{n+1}) (s1 h)_{mn'}];
for n = 2 this IS the af-0 grav part of 2(S0,S2) + (S1,S1) written in components.  W_n is linear in the ghost, so
int W_n = 0 for all compactly supported c  <=>  E_{c^mu}(W_n) = 0 for mu = 0..3 (exact, no exactness theorem needed:
int c^mu N_mu = 0 for all c <=> N_mu = 0).  Conventions: s0 h_mn = a(eta_nn d_m c^n + eta_mm d_n c^m),
s1 h_mn = a(c^r d_r h_mn + h_rn d_m c^r + h_mr d_n c^r), a = 1/2 (XVI-O sheet: xi = a c, g = eta + kappa h)."""
from __future__ import annotations
import itertools
import json
import sys
import time
from fractions import Fraction
from pathlib import Path

import sympy as sp
from sympy.polys.domains import QQ
from sympy.polys.rings import ring

ETA = [1, -1, -1, -1]
A = QQ(1, 2)
IDX = [(a, b) for a in range(4) for b in range(a, 4)]
HJ, CJ = 3, 2                                   # jet orders needed (see docstring of main)


def multis(k):
    return [tuple(sorted(t)) for t in itertools.combinations_with_replacement(range(4), k)]


ALL_H = [(ab, al) for ab in IDX for k in range(HJ + 1) for al in multis(k)]
ALL_C = [(mu, al) for mu in range(4) for k in range(CJ + 1) for al in multis(k)]
NAMES = [f"h{a}{b}_" + "".join(map(str, al)) for (a, b), al in ALL_H] + [f"c{m}_" + "".join(map(str, al)) for m, al in ALL_C]
R, *GENS = ring(",".join(NAMES), QQ)
G = dict(zip(NAMES, GENS))
POS = {g: i for i, g in enumerate(GENS)}


def h(a, b, al=()):
    a, b = sorted((a, b))
    return G[f"h{a}{b}_" + "".join(map(str, tuple(sorted(al))))]


def c(m, al=()):
    return G[f"c{m}_" + "".join(map(str, tuple(sorted(al))))]


SHIFT = {}
for i, nm in enumerate(NAMES):
    base, al = nm.split("_")
    al = tuple(int(x) for x in al)
    for r in range(4):
        nn = base + "_" + "".join(map(str, tuple(sorted(al + (r,)))))
        SHIFT[(i, r)] = G.get(nn)


def D(p, r):
    """total derivative d/dx^r on the jet ring (exact)."""
    out = R.zero
    used = set()
    for mon in p.monoms():
        used.update(i for i, e in enumerate(mon) if e)
    for i in used:
        t = SHIFT[(i, r)]
        if t is None:
            raise RuntimeError(f"jet order exceeded at {NAMES[i]}")
        out += p.diff(GENS[i]) * t
    return out
```

```
def euler(p, var):
    """E_var(p) = sum_al (-D)^al dp/d var_al  for var = ('h',a,b) or ('c',m)."""
    out = R.zero
    kind = var[0]
    for k in range(HJ + 1):
        for al in multis(k):
            g = h(var[1], var[2], al) if kind == "h" else (c(var[1], al) if k <= CJ else None)
            if g is None:
                continue
            q = p.diff(g)
            if not q:
                continue
            for r in al:
                q = -D(q, r)
            out += q
    return out


def load_export(path):
    e = sp.sympify(Path(path).read_text())
    out = R.zero
    for term in sp.Add.make_args(sp.expand(e)):
        coeff, rest = term.as_coeff_Mul()
        m = R(QQ(Fraction(str(coeff))))
        for f in sp.Mul.make_args(rest):
            base, ex = f.as_base_exp()
            n = base.name
            if n.startswith("dH"):
                g = h(int(n[2]), int(n[3]), (int(n.split("_")[1]),))
            else:
                g = h(int(n[1]), int(n[2]))
            m *= g ** int(ex)
        out += m
    return out


def s0(m, n):
    return A * (ETA[n] * c(n, (m,)) + ETA[m] * c(m, (n,)))


def s1(m, n):
    return A * (sum(c(r) * h(m, n, (r,)) for r in range(4)) + sum(h(r, n) * c(r, (m,)) for r in range(4))
                + sum(h(m, r) * c(r, (n,)) for r in range(4)))


def main(exports_dir, mutate=None):
    t0 = time.time()
    ell = {k: load_export(Path(exports_dir) / f"ell{k}_generating_polynomial.srepr") for k in (2, 3, 4)}
    if mutate == "scale_ell4_term":
        t = ell[4].terms()[0]
        ell[4] += R({t[0]: t[1]})
    if mutate == "drop_s1":
        global s1
        s1 = lambda m, n: R.zero  # noqa: E731
    E = {k: {ab: euler(ell[k], ("h",) + ab) for ab in IDX} for k in (2, 3, 4)}
    out = {}
    for n in (0, 1, 2):
        W = sum((E[n + 2][ab] * s0(*ab) for ab in IDX), R.zero)
        if n >= 1:
            W += sum((E[n + 1][ab] * s1(*ab) for ab in IDX), R.zero)
        N = {mu: euler(W, ("c", mu)) for mu in range(4)}
        out[f"kappa{n}"] = {"W_terms": len(W.terms()), "E_c_W_terms": {mu: len(N[mu].terms()) for mu in range(4)},
                            "antibracket_af0_vanishes": all(not N[mu] for mu in range(4))}
    out["seconds"] = round(time.time() - t0, 1)
    return out


if __name__ == "__main__":
    res = {"route": "independent jet engine (XVI-Q); W_n = E(ell_{n+2}) s0 h + E(ell_{n+1}) s1 h; E_c(W_n) = 0",
           "unmutated": main(sys.argv[1])}
    for m in ("scale_ell4_term", "drop_s1"):
        r = main(sys.argv[1], mutate=m)
        res[f"mutation_{m}"] = {"kappa2_vanishes": r["kappa2"]["antibracket_af0_vanishes"],
                                "detected": not r["kappa2"]["antibracket_af0_vanishes"]}
    print(json.dumps(res, indent=1))
```

code/q01/q01_conventions.py

```
"""XVI-Q Q01 conventions: pins and EVERY certified number the gate value-checks."""
LANDMARK, RELEASE, VERSION_LABEL, DATE = "XVI-Q", "Q01", "XVI-Q Q01", "2026-10-10"
XVI_P_P23_ZIP_SHA256 = "3f4a88db462923553272e8a41407c0c06aacbc431736efdfeda10c27aae8a177"
P23_INDEPENDENT_REPORT_SHA256 = "2e3e5bf325c9a0e7"            # prefix, full sha in intake/imports.json
O1_LEDGER_SHA256 = "3cd3d1ba491381ba0a73d1677bfed10d3743e0575f61c71c325e626e4a78a645"
```

```
EXPORT_SHA256 = {   # FILE sha256 of the exported .srepr (byte-identical to the published p23 exports; P13 records polynomial content shas)
    2: "8ef4395c51434fe073109065e31773460c3708937a16ea29a036abf6c72c5030",
    3: "183798063fa89839e9a80ae4a81e6c2f9c491a7ad386540d1a4e33382b0686de",
    4: "d71d9a839b5e6388abe32ccb6bb334b87cca8e104f98a99ec198b923dde099fc"}
EXPECTED = {
    "ell4_n_terms": 3360, "ell4_normalisation_from_XVI_O_probe": "-2", "ell4_full_polynomial_equal": True,
    "ell4_probe_equal": True,
    "W_terms": {"kappa0": 132, "kappa1": 2148, "kappa2": 15428},
    "E_c_W_all_zero": {"kappa0": True, "kappa1": True, "kappa2": True},
    "master_mutation_residuals": {"flip_sigma_c": 6, "drop_cstar": 6, "alter_gf_shift": 2,
                                  "flip_odd_derivative_sign": 6, "corrupt_a2": 1, "corrupt_ell4": 1,
                                  "drop_s1_transport": 2},
    "classical_certs_content_identical_to_p23": ["P13_quartic_verified", "P13_brst_derivation", "P13_master_kappa2",
                                                 "P13_boundary_identity", "P13_gauge_fixing_map"],
    "guard_forms": 18, "guard_exprs": 4,   # Q02 revision: guard_forms 16 -> 18 (+ apart_residue, apart_list)
    "O2_items": {"a_ell4": "Constructed", "b_antibracket_expansion": "Constructed", "c_ghost_antifield": "Constructed"},
}
HARD_STOPS = {"O1_closure": "NOT_CLOSED", "bulk_symmetry_restored": False, "insertion": "frozen",
              "XV_B_opened": False, "XIV_J_opened": False, "Route_B_discharged": False, "Missing_never_zero": True}
```

code/q01/qguard.py

```
"""XVI-Q d guard (Q01; Q02 revision: generator-only Poly refusal, apart removed from the allow-list): XVI-P p23 dred (imported read-only, imports/XVI-
P-p23) + ONE entry-point rule.

p23 review bypasses (sympy.polys.rem namespace, polytools.rem captured before import, reduced, groebner,
Matrix.subs d->Dummy then Dummy->4, Poly(...).rep.eval) all reach either the polynomial constructor Poly.new (class
attribute, looked up at call time -> namespace aliases and pre-import captures cannot avoid it) or an internal rename
of d.  Rule: such an operation touching d is admitted only if the ENTRY function -- the first sympy frame entered from
non-sympy code -- is an allow-listed rational-normalisation entry (cancel/factor/together/apart/simplify/expand/...
and arithmetic dunders), or inside dred.authorised_d.  Everything else raises PrematureD4Substitution (deny by default).
"""
from __future__ import annotations
import sys
from pathlib import Path

QROOT = Path(__file__).resolve().parents[2]
sys.path.insert(0, str(QROOT / "imports" / "XVI-P-p23" / "code" / "p5_entry"))
import sympy as sp  # noqa: E402
import sympy.polys.polytools as _pt  # noqa: E402
import dred  # noqa: E402   (XVI-P p23, pristine import)

d = dred.d
PrematureD4Substitution = dred.PrematureD4Substitution
ALLOW_ENTRY = frozenset({
    "cancel", "factor", "factor_list", "factor_terms", "together", "simplify", "expand",
    "expand_mul", "ratsimp", "radsimp", "gcd", "lcm", "gcd_terms", "numer", "denom", "fraction", "as_numer_denom",
    "nsimplify_none", "count_ops", "equals", "is_zero", "could_extract_minus_sign",
    "__add__", "__radd__", "__sub__", "__rsub__", "__mul__", "__rmul__", "__truediv__", "__rtruediv__", "__pow__",
    "__rpow__", "__neg__", "__eq__", "__ne__", "__hash__", "__new__", "_eval_power", "flatten",
})
_INTERNAL_MARK = (f"{Path(sp.__file__).parent}", "mpmath", str(Path(dred.__file__).resolve()), str(Path(__file__).resolve()))
ENTRY_POINTS = ["Poly.new (polynomial construction with d as generator or in the domain)",
                "dred._check_rule (any internal rename/substitution of d)"]


def _is_internal(fr):
    f = fr.f_code.co_filename
    return any(m in f for m in _INTERNAL_MARK)


WRAPPERS = frozenset({"threaded_func", "wrapper", "_sympifyit_wrapper", "__sympifyit_wrapper", "binary_op_wrapper",
                      "_func", "wrapped", "inner", "decorator", "cacheit", "wrapper_cache"})
POLY_CLASSES = frozenset({"Poly", "PurePoly", "DMP", "DMP_Python", "DUP_Flint", "PolyElement", "PolyRing"})


def _entry_frame():
    """first sympy frame entered from non-sympy code, skipping decorator-wrapper frames."""
    fr = sys._getframe(2)
    chain = []
    while fr is not None and _is_internal(fr):
        chain.append(fr)
        fr = fr.f_back
    for f in reversed(chain):                       # outermost internal frame first
        if f.f_code.co_name not in WRAPPERS:
            return f
    return chain[-1] if chain else None


def entry_function():
    f = _entry_frame()
    return getattr(f.f_code, "co_qualname", f.f_code.co_name) if f is not None else None
```

```
def _entry_allowed():
    q = entry_function()
    if q is None:
        return False
    if q.split(".")[0] in POLY_CLASSES:             # Poly(...), Poly.method(...) are never an allowed entry
        return False
    return q.split(".")[-1] in ALLOW_ENTRY


def _authorised():
    return bool(getattr(dred._AUTH, "active", False))


def _deny(what):
    raise PrematureD4Substitution(f"XVI-Q guard: {what} (entry {entry_function()!r} not allow-listed); use "
                                  "dred.substitute_d / dred.authorised_d")


def _touches_d(gens, dom):
    """Q02 revision: refuse only when d is a polynomial GENERATOR (every reviewer bypass -- rem/reduced/groebner/
    compose/resultant/shift/rep.eval/apart -- evaluates or divides in the d variable).  d inside the coefficient
    domain (e.g. Poly(expr, x) over QQ(d), as in XVI-P integrals.fp_two_point) cannot set d = 4 and is admitted."""
    return any(dred._has_d(g) for g in gens)


_orig_new = _pt.Poly.new.__func__


def _guarded_new(cls, rep, *gens):
    if not _authorised():
        dom = getattr(rep, "dom", None)
        if _touches_d(gens, dom) and not _entry_allowed():
            _deny("Poly construction with d in the polynomial domain")
    return _orig_new(cls, rep, *gens)


_pt.Poly.new = classmethod(_guarded_new)
_p23_rule = dred._check_rule


def _check_rule(pairs, what, depth=2):
    if _authorised():
        return
    for old, new in pairs:
        if dred._has_d(old) and not _entry_allowed():
            _deny(f"{what} of {old} -> {new}")
    _p23_rule(pairs, what, depth + 1)


dred._check_rule = _check_rule

# Q03: PolyRing / FracField with d as a GENERATOR (sympy.polys.rings.ring / fields.field / sring, then
# PolyElement.evaluate(d, 4)) bypassed Poly.new (Q02 independent review).  Constructing such rings stays admitted (QQ[d]
# is the legitimate coefficient domain of Poly(expr, x)); every way of assigning a VALUE to a generator of a ring/field
# that has d among its symbols (evaluate / subs / __call__ / compose, on PolyElement and FracElement) is refused unless
# the entry is allow-listed or inside dred.authorised_d.
import sympy.polys.rings as _rings  # noqa: E402
import sympy.polys.fields as _fields  # noqa: E402


def _ring_has_d(el):
    r = getattr(el, "ring", None) or getattr(el, "field", None)
    return r is not None and any(dred._has_d(sp.sympify(x)) for x in getattr(r, "symbols", ()))


EVAL_ENTRY = frozenset({"evaluate", "subs", "__call__", "compose"})   # internal sympy use (e.g. heugcd evaluation
# points inside an allow-listed cancel/apart) is admitted: it never returns a d = 4 value to the caller.


def _wrap_eval(klass, meth):
    orig = getattr(klass, meth)

    def _m(self, *a, **k):
        direct = not _is_internal(sys._getframe(1))        # called from non-sympy code (the reviewer bypass)
        if not _authorised() and _ring_has_d(self) and (direct or (entry_function() or "").split(".")[-1] in EVAL_ENTRY):
            _deny(f"{klass.__name__}.{meth} on a ring/field with d as a generator")
        return orig(self, *a, **k)
    _m.__name__ = meth
    setattr(klass, meth, _m)


for _k in (_rings.PolyElement, _fields.FracElement):
    for _meth in ("evaluate", "subs", "__call__", "compose"):
```

```
        if hasattr(_k, _meth):
            _wrap_eval(_k, _meth)
ENTRY_POINTS.append("PolyElement/FracElement evaluate/subs/__call__/compose on a ring/field with d as generator")
INSTALLED = True
```

code/q01/release_q01.py

```
#!/usr/bin/env python3
"""XVI-Q Q01 release gate.  Value-checks EVERY certified number (q01_conventions.EXPECTED) in Q01_intake / Q01_O2 /
Q01_guard, the import pins and the hard stops.  Fails closed; on reject no RELEASE_STATUS.json is left behind."""
from __future__ import annotations
import json
import sys
from pathlib import Path

Q = Path(__file__).resolve().parents[2]
sys.path.insert(0, str(Q / "code" / "q01"))
import q01_conventions as QC  # noqa: E402

CERT = Q / "certificates"
STATUS = ()   # Q02: Q01 is a layer record (certificates/Q01_release.json); RELEASE_STATUS.json is written by the current release gate


def jl(n):
    f = CERT / f"{n}.json"
    return json.loads(f.read_text()) if f.is_file() else {}


def checks():
    it, o2, g = jl("Q01_intake"), jl("Q01_O2"), jl("Q01_guard")
    E = QC.EXPECTED
    a = o2.get("items", {}).get("a_ell4", {}).get("detail", {})
    b = o2.get("items", {}).get("b_antibracket_expansion", {}).get("detail", {}).get("unmutated", {})
    c = {}
    c["intake_pins"] = it.get("pins_ok") is True and it.get("XVI_P_p23_zip_ok") is True
    c["intake_extracted_identical"] = it.get("XVI_P_p23_extracted_identical") is True
    c["intake_O1_ledger"] = it.get("O1_ledger_sha_ok") is True
    c["errata_recorded"] = [e.get("id") for e in it.get("errata", [])][:5] == ["E1", "E2", "E3", "E4", "E5"]   # Q02 appends E6, E7
    c["ell4_n_terms"] = a.get("n_terms_mine") == E["ell4_n_terms"] == a.get("n_terms_export")
    c["ell4_normalisation"] = a.get("normalisation_from_XVI_O_probe") == E["ell4_normalisation_from_XVI_O_probe"]
    c["ell4_full_equal"] = a.get("full_polynomial_equal") is E["ell4_full_polynomial_equal"] and a.get("n_terms_difference") == 0
    c["ell4_probe_equal"] = a.get("export_on_XVI_O_probe_equal") is E["ell4_probe_equal"]
    c["ell4_mutation"] = a.get("mutation_one_term_x2_detected") is True
    c["exports_sha"] = {int(k): v for k, v in o2.get("exports_sha256", {}).items()} == QC.EXPORT_SHA256
    c["W_terms"] = all(b.get(k, {}).get("W_terms") == v for k, v in E["W_terms"].items()) \
        and o2.get("W_terms") == E["W_terms"]
    c["E_c_W_zero"] = all(b.get(k, {}).get("antibracket_af0_vanishes") is v and
                          all(x == 0 for x in b.get(k, {}).get("E_c_W_terms", {"x": 1}).values())
                          for k, v in E["E_c_W_all_zero"].items())
    bd = o2.get("items", {}).get("b_antibracket_expansion", {}).get("detail", {})
    c["antibracket_mutations"] = bd.get("mutation_scale_ell4_term_detected") is True and bd.get("mutation_drop_s1_detected") is True
    c["master_mutations"] = {m: v.get("n_residuals") for m, v in o2.get("master_mutations", {}).items()} \
        == E["master_mutation_residuals"] and all(v.get("claim_status") != "verified" for v in o2.get("master_mutations", {}).values())
    c["classical_certs_identical"] = sorted(k for k, v in o2.get("classical_certs_content_identical_to_p23", {}).items() if v) \
        == sorted(E["classical_certs_content_identical_to_p23"])
    c["kappa2_table_all_true"] = all(o2.get("items", {}).get("c_ghost_antifield", {}).get("kappa2_table", {"x": False}).values())
    c["s2_derived_zero"] = set(o2.get("items", {}).get("c_ghost_antifield", {}).get("s2", {}).values()) == {"derived_zero"}
    c["O2_items"] = o2.get("item_status") == E["O2_items"] and o2.get("O2_kappa2_master_status") == "Constructed"
    c["guard"] = g.get("n_forms") == E["guard_forms"] and g.get("n_exprs") == E["guard_exprs"] and g.get("missed") == []
    c["guard_algebra"] = g.get("algebra", {}).get("authorised_value") == "1/80" and g.get("algebra", {}).get("d_free_rem") == "2"
    return c, it, o2, g


def main():
    try:
        c, it, o2, g = checks()
    except Exception as exc:  # noqa: BLE001
        c, it, o2, g = {"error": f"{type(exc).__name__}: {exc}"}, {}, {}, {}
    ok = all(v is True for v in c.values())
    for s in STATUS:
        s.unlink(missing_ok=True)
    rec = {"landmark": QC.LANDMARK, "release": QC.RELEASE, "version_label": QC.VERSION_LABEL, "date": QC.DATE,
           "execution_pass": ok, "publication": "accepted" if ok else "REJECTED", "value_checks": c,
           "claim_status": ("Q01: O.2 classical kappa^2 BV master 2(S0,S2)+(S1,S1)=0 on S_inv Constructed in XVI-Q: (a) full "
                            "ell_4 independently reconstructed (3360/3360) and matched to XVI-O's probe/convention; (b) explicit "
                            "af-0 antibracket expansion E_c(W_2)=0 exactly on an independent engine; (c) ghost/antifield cells "
                            "via re-executed exact P13 master with s2 derived zero. XVI-O's own O.2 record is unchanged "
                            "(read-only). O.1 NOT_CLOSED (bulk O.5, retained identity O.6/O.7 open -> Q02); bulk symmetry "
                            "not restored.") if ok else "Q01: not accepted",
           "scope": o2.get("scope"),
           "flags": {"intake_pinned": "earned" if ok else "not_earned",
                     "O2_ell4_constructed": "earned" if ok else "not_earned",
                     "O2_antibracket_expansion_constructed": "earned" if ok else "not_earned",
```

```
                    "O2_ghost_antifield_constructed": "earned" if ok else "not_earned",
                    "O2_kappa2_master_constructed": "earned" if ok else "not_earned",
                    "d_guard_q01_verified": "earned" if ok else "not_earned",
                    "bulk_O5_evaluated": "not_earned", "bulk_identity_verified": "not_earned",
                    "author_rebuild_passed": "see reproducibility", "independent_rebuild_passed": "pending"},
          "O1_closure": "NOT_CLOSED", "bulk_symmetry_restored": False, "symmetry_restored": False,
          "hard_stops": QC.HARD_STOPS, "XVI_Q_registered": "locally (registration/local_registration.json); not published",
          "O1_checklist_after_Q01": {"O.2": "Constructed (XVI-Q Q01)", "O.4": "inventory (XVI-P; errata E1)",
                                     "O.5": "bulk OPEN (D04, D05a, D06 bulk, D12 Missing)", "O.6": "bulk statement open (ST4a/ST4b)",
                                     "O.7": "bulk CT solution open", "enlarge": "not triggered at kappa^2 (XVI-P p23)"}}
    if ok:
        txt = json.dumps(rec, indent=1) + "\n"
        for s in STATUS:
            s.write_text(txt)
        (CERT / "Q01_release.json").write_text(txt)
        (CERT / "Q01_release_rejected.json").unlink(missing_ok=True)
        (Q / "ledgers").mkdir(exist_ok=True)
        (Q / "ledgers" / "Q01_milestones.json").write_text(json.dumps(
            {"landmark": "XVI-Q", "release": "Q01", "milestones": rec["O1_checklist_after_Q01"], "flags": rec["flags"],
             "hard_stops": QC.HARD_STOPS}, indent=1) + "\n")
    else:
        (CERT / "Q01_release.json").unlink(missing_ok=True)
        (CERT / "Q01_release_rejected.json").write_text(json.dumps(rec, indent=1) + "\n")
    print(json.dumps({"publication": rec["publication"], "failed": [k for k, v in c.items() if v is not True]}))
    return 0 if ok else 1


if __name__ == "__main__":
    raise SystemExit(main())
```

code/q02/bulk_graphs.py

```
#!/usr/bin/env python3
"""XVI-Q Q02 bulk O.5 graphs D04, D05a, D06, D12 on the frozen XVI-P p23 engine (re-executed copy build/xvip_run),
under the XVI-P declared prescription (DRED, opaque d -- qguard loaded FIRST, MS-bar, Feynman +i0, m = 0).

D04 phi phi mixed bubble (two V_h_phi_phi; graviton + scalar line), D06 cbar c mixed bubble (two V_cb_c_h; graviton +
ghost line): integrands built with XVI-P feynman.py vertices/propagators (ordered extraction, independent components,
Hessian-inverse propagators), same prefactor bookkeeping as amplitude_rules.build_integrands.
  route 1: integrals.fp_two_point (Feynman parameters, symmetric integration in QdS, NO scaleless rule);
  route 2 (D04): exact invariant fit N = sum c (l.l)^a (l.p)^b (p.p)^c verified identically, then IBP J_two_point;
  route 2 (D06): comparison with XVI-P P15_D13 ghost_self_energy (pinned import).
Symmetry factors derived by Wick counting (automorphisms of the graph with labelled external legs): D04 = D06 = 1.
D05a, D12: one massless line, vertex without loop-momentum dependence on the contracted legs (a2 carries no
derivative of h; for D12 the phi legs carry l but the integrand is a polynomial over l^2): pure power divergences,
omega in {2, 4}, no logarithmic member => DR value 0 DERIVED (no pole on the UV or the IR side), never by convention.
Gamma_1 convention (XVI-P): i Gamma_1 = graph sum; coefficients reported per kappa^2, in units of MB = Ihat(1,1)."""
from __future__ import annotations
import json
import sys
import time
from fractions import Fraction as Fr
from pathlib import Path

Q = Path(__file__).resolve().parents[2]
sys.path.insert(0, str(Q / "code" / "q01"))
import qguard  # noqa: E402,F401   (d guard installed before any XVI-P module)
RUN = Q / "build" / "xvip_run"
for sub in ("code/p5_calc", "code/p5_entry", "code/classical"):
    sys.path.insert(0, str(RUN / sub))
import sympy as sp  # noqa: E402
import dred  # noqa: E402
import feynman as F  # noqa: E402
import amplitude_rules as AR  # noqa: E402
import integrals as IL  # noqa: E402

assert dred.__file__ == qguard.dred.__file__, "one dred module (the guarded one)"
I, d, eps = sp.I, dred.d, dred.eps
ETA = (1, -1, -1, -1)
LS, PS = sp.symbols("l0:4"), sp.symbols("p0:4")
PP, MB, Lg = IL.PP, IL.MB, IL.L
NAMES = {**{r: LS[r] for r in range(4)}, **{4 + r: PS[r] for r in range(4)}}
P_NUMS = [(3, 1, 2, -1), (5, 2, -1, 1), (7, -3, 1, 2)]
SEEDS = (11, 23)


def sym(poly):
    return sp.expand(poly.to_sympy(NAMES)) if hasattr(poly, "to_sympy") else sp.expand(poly)


def dot(a, b):
    return sum(ETA[r] * a[r] * b[r] for r in range(4))
```

```
def gt(r, s):
    return sp.Symbol(f"Gt_{min(r, s)}{max(r, s)}")


def expand_coeff(c):
    """A(d) * MB -> pole / MS-bar finite (local + coefficient of L); d = 4 - 2 eps, MB = 1/eps + 2 - L + O(eps)."""
    ms = IL.master_series(1)
    a = sp.expand(dred.substitute_d(sp.sympify(c), 4 - 2 * eps, "Q02 bulk: Laurent expansion after all contractions"))
    ser = sp.series(a * (ms[-1] / eps + ms[0] + ms[1] * eps), eps, 0, 1).removeO()
    pole, fin = sp.simplify(ser.coeff(eps, -1)), sp.expand(sp.simplify(ser.coeff(eps, 0)))
    return {"A_exact_d": str(sp.factor(c)), "UV_pole": str(pole), "MSbar_finite": str(fin),
            "coefficient_of_L": str(sp.expand(fin).coeff(Lg)), "local": str(sp.expand(fin).subs(Lg, 0))}


def exact_fit(rows, rhs, n):
    dg = sp.Rational(173, 37)
    chosen, M = [], []
    for i, row in enumerate(rows):
        cand = M + [[dred.substitute_d(sp.sympify(x), dg, "Q02 fit: numerical rank test (row selection only)") for x in row]]
        if sp.Matrix(cand).rank() > len(M):
            M, chosen = cand, chosen + [i]
        if len(chosen) == n:
            break
    if len(chosen) < n:
        return None, False
    sol = [sp.factor(sp.cancel(x)) for x in sp.Matrix([rows[i] for i in chosen]).LUsolve(sp.Matrix([rhs[i] for i in chosen]))]
    ok = all(sp.cancel(sum(sp.sympify(a) * x for a, x in zip(row, sol)) - b) == 0 for row, b in zip(rows, rhs))
    return sol, ok


def fit(G, S):
    """G, S[name]: dict key -> expression in PS, PP, Gt; exact fit G = sum c_name S_name over realizations."""
    names = list(S)
    rows, rhs = [], []
    for pn in P_NUMS:
        for s in SEEDS:
            gm = IL.realization(pn, s)
            for k in sorted(set(G) | set().union(*[set(S[n]) for n in names])):
                rows.append([IL.realize(S[n].get(k, 0), pn, gm) for n in names])
                rhs.append(IL.realize(G.get(k, 0), pn, gm))
    sol, ok = exact_fit(rows, rhs, len(names))
    sol = sol or [None] * len(names)
    return {n: c for n, c in zip(names, sol) if c is not None and c != 0}, ok, len(rhs)


def prefactor_norm(S_f, loop_sign, phases, n_explicit_i):
    """Gamma_1 coefficient per kappa^2: -i * S_f * loop_sign * (i)^2 [two kappa^1 vertices] * i^phase * i^n."""
    return sp.expand(-I * sp.nsimplify(S_f) * loop_sign * I ** 2 * I ** phases * I ** n_explicit_i)


# ---------------------------------------------------------------------------------------------- D04
def D04_integrand(S_f=1, mutate=None):
    V = F.vertices()["V_h_phi_phi"]
    Pi = AR.pinv_sparse()
    if mutate == "drop_trace_part_of_graviton_propagator":
        dd = F.deDonder_tensor_numerator()

        def e(i):
            m, n = F.HC[i]
            return ETA[m] if m == n else 0
        Pi = {}
        for i in range(10):
            for j in range(10):
                v = sp.nsimplify(dd[i, j] + sp.Rational(1, 2) * e(i) * e(j))
                if v != 0:
                    Pi[(i, j)] = Fr(int(v.p), int(v.q))
    VA = {k: sym(v) for k, v in F.vt_route(V, [(-1, 0), (0, 1), (1, -1)]).items()}   # h_c: -l, phi_ext: p, phi: l-p
    VB = {k: sym(v) for k, v in F.vt_route(V, [(1, 0), (0, -1), (-1, 1)]).items()}   # h_d: l, phi_ext: -p, phi: p-l
    N = 0
    for (c, _, _), pa in VA.items():
        for (dd_, _, _), pb in VB.items():
            pv = Pi.get((c, dd_))
            if pv:
                N += pa * pb * sp.Rational(pv.numerator, pv.denominator)
    N = sp.expand(N)
    # propagators: graviton i P^{-1}/l^2, scalar i/(l-p)^2  -> two explicit i
    norm = prefactor_norm(S_f, 1, V.phase * 2, 2)
    return N, norm


def D04(S_f=1, mutate=None):
    N, norm = D04_integrand(S_f, mutate)
    R1 = sp.expand(norm * IL.fp_two_point(N, LS, PS) / MB)
```

```
    S = {"p4": {(): PP ** 2}}          # the only scalar structure: p.Gt.p = 0 identically (external p in the 4D part)
    c1, ok1, n1 = fit({(): R1}, S)
    out = {"route1_fp": {k: str(v) for k, v in c1.items()}, "route1_fit_exact": ok1, "route1_rows": n1}
    if mutate:
        return out, c1
    # route 2: invariant fit of N then IBP J
    l2, lp = sp.Symbol("l.l"), sp.Symbol("l.p")
    mons = [(a, b, 2 - a - b) for a in range(3) for b in range(3 - a)]
    cs = sp.symbols(f"c0:{len(mons)}")
    ansatz = sum(c * dot(LS, LS) ** a * dot(LS, PS) ** b * dot(PS, PS) ** e for c, (a, b, e) in zip(cs, mons))
    eqs = sp.Poly(sp.expand(ansatz - N), *LS, *PS).coeffs()
    sol = sp.solve(eqs, cs, dict=True)
    inv_ok = bool(sol) and sp.expand(ansatz.xreplace(sol[0]) - N) == 0
    R2 = 0
    for c, (a, b, e) in zip(cs, mons):
        cv = sol[0].get(c, 0) if sol else 0
        if cv != 0:
            R2 += cv * PP ** e * IL.J_two_point(a, b, f"Q02_D04[{a},{b}]")
    R2 = sp.expand(norm * R2 / MB)
    A1 = c1.get("p4", sp.Integer(0)) if ok1 else None
    A2 = sp.factor(sp.cancel(R2 / PP ** 2))
    # route 3 (hand, independent of the engine): T.P.T' with T_ab = p_a q_b + p_b q_a - eta_ab p.q, de Donder P (Q4S
    # trace 4): T.P.T' = 2(p.p')(q.q') + 2(p.q')(q.p') - 2(p.q)(p'.q'); p' = -p, q = l - p, q' = -q  =>  2 p^2 (l-p)^2
    hand = sp.expand(2 * (-PP) * (-sp.Symbol("q.q")) + 2 * (-sp.Symbol("p.q")) ** 2 - 2 * sp.Symbol("p.q") * sp.Symbol("p.q"))
    out.update({"route2_invariant_fit_identical": inv_ok, "route2_IBP_A": str(A2),
                "route1_A": str(A1), "N_factorised": str(sp.factor(N)),
                "N_proportional_to_p2_times_scalar_denominator": sp.expand(N - sp.Rational(1, 2) * dot(PS, PS) * dot([LS[r] - PS[r] for r in
range(4)], [LS[r] - PS[r] for r in range(4)])) == 0,
                "route3_hand_contraction": str(hand) + "  (= 2 p^2 (l-p)^2 with q = l - p; engine N = 1/2 p^2 (l-p)^2: same shape, engine vertex
normalisation)",
                "reading": "N cancels the scalar propagator: D04 = (p^2/2) * int 1/l^2 (graviton line) = scaleless, omega = 2 power divergence, no log
member: D04 = 0 exactly in d by BOTH routes (route 1 uses no scaleless rule)",
                "routes_agree": A1 is not None and sp.cancel(A1 - A2) == 0,
                "N_invariant_form": {f"(l.l)^{a}(l.p)^{b}(p.p)^{e}": str(sol[0].get(c, 0)) for c, (a, b, e) in zip(cs, mons)} if sol else None,
                "structure": "Gamma_1^{phi phi}(p) = kappa^2 A(d) (p^2)^2 MB  (no evanescent structure: Gt.p = 0)",
                "value": "0" if (A1 == 0 and A2 == 0) else "nonzero",
                "laurent": expand_coeff(A1) if A1 not in (None, 0) else {"UV_pole": "0", "MSbar_finite": "0", "local": "0", "coefficient_of_L": "0"},
                "status": "Evaluated_vanishing_under_declared_prescription" if (A1 == 0 and A2 == 0) else "Evaluated"})
    return out, c1


# ---------------------------------------------------------------------------------------------------- D06
def D06(S_f=1):
    Vg = F.vertices()["V_cb_c_h"]
    Pi = AR.pinv_sparse()
    Gh = AR.ghost_num()
    VA = {k: sym(v) for k, v in F.vt_route(Vg, [(0, -1), (1, 0), (-1, 1)]).items()}   # cbar_rho: -p, c^mu: l, h_a: p-l
    VB = {k: sym(v) for k, v in F.vt_route(Vg, [(-1, 0), (0, 1), (1, -1)]).items()}   # cbar_sig: -l, c^nu: p, h_b: l-p
    N = {}
    for (rho, mu, a), pa in VA.items():
        for (sig, nu, b), pb in VB.items():
            g = Gh.get((mu, sig))
            pv = Pi.get((a, b))
            if g and pv:
                N[(rho, nu)] = N.get((rho, nu), 0) + pa * pb * sp.Rational(g.numerator, g.denominator) * sp.Rational(pv.numerator, pv.denominator)
    # ghost numerator carries its i; graviton propagator: one explicit i
    norm = prefactor_norm(S_f, 1, Vg.phase * 2, 1)
    G = {k: sp.expand(norm * IL.fp_two_point(sp.expand(v), LS, PS) / MB) for k, v in N.items() if sp.expand(v) != 0}
    pu = lambda r: ETA[r] * PS[r]  # noqa: E731
    S = {"S1_delta^rho_nu_p4": {(r, s): (1 if r == s else 0) * PP ** 2 for r in range(4) for s in range(4)},
         "S2_p^rho_p_nu_p2": {(r, s): ETA[r] * PS[r] * PS[s] * PP for r in range(4) for s in range(4)},
         "SE_Gt^rho_nu_p4": {(r, s): ETA[r] * gt(r, s) * PP ** 2 for r in range(4) for s in range(4)}}
    c, ok, n = fit(G, S)
    return {"fit": {k: str(v) for k, v in c.items()}, "fit_exact": ok, "rows": n,
            "laurent": {k: expand_coeff(v) for k, v in c.items()}}, c


def tadpoles():
    """D05a, D12: explicit degree / scaleless analysis on the engine integrands."""
    out = {}
    V2s = F.vertices()["V_hh_phi_phi"]
    Pi = AR.pinv_sparse()
    # D12: phi legs contracted (l, -l), external h_a(p), h_b(-p)
    VA = {k: sym(v) for k, v in F.vt_route(V2s, [(0, 1), (0, -1), (1, 0), (-1, 0)]).items()}
    deg12 = max((sp.Poly(v, *LS).total_degree() for v in VA.values() if v != 0), default=0)
    # D05a: h legs contracted (l, -l), external phi(p), phi(-p)
    VB = {k: sym(v) for k, v in F.vt_route(V2s, [(1, 0), (-1, 0), (0, 1), (0, -1)]).items()}
    N5 = 0
    for (c, dd_, _, _), v in VB.items():
        pv = Pi.get((c, dd_))
        if pv:
            N5 += v * sp.Rational(pv.numerator, pv.denominator)
    N5 = sp.expand(N5)
    deg5 = sp.Poly(N5, *LS).total_degree() if N5 != 0 else 0
```

```
    for name, deg, note in (("D12", deg12, "phi phi self-contraction on V_hh_phi_phi; numerator carries l (phi legs)"),
                            ("D05a", deg5, "h h self-contraction on V_hh_phi_phi; a2 has no derivative of h")):
        omegas = sorted({4 - 2 + k for k in range(0, deg + 1, 2)})       # int d^4 l (l)^k / l^2
        out[name] = {"integrand": note, "loop_numerator_degree_in_l": deg, "omegas": omegas,
                     "logarithmic_member": 0 in omegas, "UV_pole": "0", "IR_pole": "0",
                     "value": "0 (DERIVED: every member int (l^2)^j/l^2 is scaleless with omega = 2 + 2j > 0 -- a pure "
                              "power divergence; no 1/eps on the UV or the IR side; MS-bar counterterm 0)",
                     "status": "Evaluated_vanishing_under_declared_prescription"}
    out["D05a"]["numerator_l_independent"] = all(sp.diff(N5, x) == 0 for x in LS)
    return out


def main():
    t0 = time.time()
    res = {"prescription": "XVI-P p14r/p22: DRED (QdS loop momenta, Q4S vertex indices), d opaque (qguard), MS-bar, "
                           "Feynman +i0, m = 0 (XVI-O bulk_mass_policy; XVI-N massless obstruction: insertion untouched)",
           "Gamma1_convention": "i Gamma_1 = graph sum; coefficients per kappa^2 in units of MB = Ihat(1,1) = 1/eps + 2 - L + O(eps)",
           "symmetry_factors": {"D04": "1 (two distinct-species internal lines; no automorphism fixing external legs; "
                                       "XVI-O's '1/2 Unverified' corrected)",
                                "D06": "1 (as XVI-O)", "D05a": "1/2", "D12": "1/2"}}
    res["D04"], c4 = D04()
    m = {}
    _, c_half = D04(S_f=sp.Rational(1, 2), mutate="S_f_half")
    m["D04_S_f_half_still_zero (zero is S_f-independent; not a detector)"] = c_half.get("p4", 0) == 0
    _, c_tr = D04(mutate="drop_trace_part_of_graviton_propagator")
    m["D04_drop_trace_part_changes_A"] = sp.cancel(c_tr.get("p4", 0) - c4.get("p4", 0)) != 0
    m["D04_drop_trace_part_value"] = str(c_tr.get("p4", 0))
    res["D06"], c6 = D06()
    _, c6h = D06(S_f=sp.Rational(1, 2))
    m["D06_S_f_half_changes_coefficients"] = any(sp.cancel(c6h.get(k, 0) - c6.get(k, 0)) != 0 for k in c6)
    p15 = json.loads((Q / "provenance" / "XVI-P" / "certificates" / "P15_D13.json").read_text())["ghost_self_energy_pole"]
    ref = {"S1_delta^rho_nu_p4": p15["coefficients_exact_d"]["S1"], "S2_p^rho_p_nu_p2": p15["coefficients_exact_d"]["S2"],
           "SE_Gt^rho_nu_p4": p15["coefficients_exact_d"]["SE"]}
    res["D06"]["XVI_P_P15_ghost_self_energy_coefficients"] = p15["coefficients_exact_d"]
    res["D06"]["equals_XVI_P_P15_exactly"] = all(sp.cancel(sp.sympify(ref[k], locals={"d": d}) - c6.get(k, 0)) == 0 for k in ref) and set(c6) ==
set(ref)
    res["D06"]["reading"] = ("full one-loop cbar c two-point under the declared prescription; independent construction (this file) "
                             "equals XVI-P P15_D13 ghost self-energy (route 2 there; ghost equation verified there) exactly in d")
    res["tadpoles"] = tadpoles()
    res["mutations"] = m
    res["seconds"] = round(time.time() - t0, 1)
    return res


if __name__ == "__main__":
    r = main()
    out = Path(sys.argv[1]) if len(sys.argv) > 1 else Q / "build" / "q02_bulk_graphs.json"
    out.write_text(json.dumps(r, indent=1, default=str) + "\n")
    print(json.dumps({k: r[k] for k in ("D04", "D06", "mutations")}, indent=1, default=str)[:4000])
```

code/q02/bulk_identity.py

```
#!/usr/bin/env python3
"""XVI-Q Q02: bulk retained identity (XVI-O ST4a/ST4b, read-only definitions) on the FULL one-loop kappa^2 bulk two-point
sector of S_inv (m = 0), assembled from every evaluated graph; bulk counterterms; full sigma_O closure; O.1 walk.

XVI-O (symmetry_defect.json ST4_split, verbatim):  ST4a: Gamma_1,local = sum z_i C_i (gh = 0);
                                                   ST4b: P_ret (S, C) = - P_ret A_1 (gh(C) = 0; defect gh = 1).
Rows are enumerated MECHANICALLY: every two-field monomial (and every one-field monomial) of ghost number 0 (ST4a,
Gamma_1 entries) and 1 (ST4b, defect entries) built from the declared field/antifield content; each row gets evidence:
  graph values (Q02 bulk graphs; XVI-P pinned certificates), phi-parity (every interaction vertex even in {phi, phi*}:
  checked on the XVI-P vertex inventory), source/auxiliary inventory (b, cbar*, b* on no interaction vertex: checked).
Outcome per row: zero / cancellable / obstruction (explicit residual)."""
from __future__ import annotations
import itertools
import json
import sys
from pathlib import Path

import sympy as sp

Q = Path(__file__).resolve().parents[2]
PV = Q / "provenance" / "XVI-P" / "certificates"
XO = Q / "provenance" / "XVI-O"


def jl(p):
    return json.loads(Path(p).read_text())


FIELDS = {  # name: (ghost number, phi-parity, kind)
    "h": (0, 0, "field"), "phi": (0, 1, "field"), "b": (0, 0, "aux"), "c": (1, 0, "ghost"), "cb": (-1, 0, "antighost"),
    "hs": (-1, 0, "antifield (K' in Psi-transformed variables)"), "phis": (-1, 1, "antifield"), "cs": (-2, 0, "antifield"),
```

```
    "cbs": (0, 0, "antifield (nonminimal)"), "bs": (-1, 0, "antifield (nonminimal)")}
LEG_ALIAS = {"K": "hs", "h": "h", "phi": "phi", "phis": "phis", "c": "c", "cb": "cb", "cs": "cs", "cbs": "cbs", "b": "b"}


def vertex_checks():
    inv = jl(PV / "P14_amplitude_rules.json")["vertex_inventory"]
    inter = [v for v in inv if len(v["legs"]) >= 3]
    parity_even = {v["name"]: sum(FIELDS[LEG_ALIAS[l]][1] for l in v["legs"]) % 2 == 0 for v in inv}
    aux_free = {v["name"]: not ({"b", "cbs", "bs"} & {LEG_ALIAS[l] for l in v["legs"]}) for v in inter}
    return {"n_vertices": len(inv), "interaction_vertices": [v["name"] for v in inter],
            "phi_parity_even_all": all(parity_even.values()), "phi_parity_per_vertex": parity_even,
            "aux_b_cbs_bs_on_no_interaction_vertex": all(aux_free.values()),
            "quadratic_vertices_with_aux": [v["name"] for v in inv if len(v["legs"]) == 2 and {"b", "cbs"} & set(v["legs"])]}


def enumerate_graphs():
    """Independent one-loop 1PI enumeration from the p14r loop-capable vertex list (P14_counterterm_domain): propagating
    pairs h-h, phi-phi, c-cb (b, cbar*, b* are on no loop-capable vertex).  Tadpoles (one vertex, one self-contraction)
    and bubbles (two vertices, two lines between them).  Returns the E = 1 and E = 2 graphs by (vertices, lines, external)."""
    V = jl(PV / "P14_counterterm_domain.json")["loop_capable_vertices"]
    pair = {("h", "h"), ("phi", "phi"), ("c", "cb"), ("cb", "c")}
    out = set()
    for vn, legs in V.items():
        for i, j in itertools.combinations(range(len(legs)), 2):
            if (legs[i], legs[j]) in pair:
                ext = tuple(sorted(l for k, l in enumerate(legs) if k not in (i, j)))
                out.add(((vn,), tuple(sorted([legs[i] + "-" + legs[j]] if legs[i] <= legs[j] else [legs[j] + "-" + legs[i]])), ext))
    names = sorted(V)
    for a in range(len(names)):
        for b in range(a, len(names)):
            L1, L2 = V[names[a]], V[names[b]]
            for i1, j1 in itertools.permutations(range(len(L1)), 2):
                for i2, j2 in itertools.permutations(range(len(L2)), 2):
                    if (L1[i1], L2[i2]) in pair and (L1[j1], L2[j2]) in pair:
                        ext = tuple(sorted([l for k, l in enumerate(L1) if k not in (i1, j1)] + [l for k, l in enumerate(L2) if k not in (i2, j2)]))
                        lines = tuple(sorted("-".join(sorted((L1[x], L2[y]))) for x, y in ((i1, i2), (j1, j2))))
                        out.add((tuple(sorted((names[a], names[b]))), lines, ext))
    E1 = sorted({g for g in out if len(g[2]) == 1})
    E2 = sorted({g for g in out if len(g[2]) == 2})
    return {"E1": [list(map(list, g)) for g in E1], "E2": [list(map(list, g)) for g in E2], "n_E1": len(E1), "n_E2": len(E2)}


def rows():
    names = list(FIELDS)
    two = sorted({tuple(sorted(p)) for p in itertools.combinations_with_replacement(names, 2)})
    out = {"gh0_two": [], "gh1_two": [], "gh0_one": [], "gh1_one": []}
    for p in two:
        g = sum(FIELDS[x][0] for x in p)
        if g in (0, 1):
            out[f"gh{g}_two"].append(p)
    for x in names:
        g = FIELDS[x][0]
        if g in (0, 1):
            out[f"gh{g}_one"].append((x,))
    return out


def classify(row, vc):
    s = set(row)
    if sum(FIELDS[x][1] for x in row) % 2:
        return "zero", "phi-parity: every vertex even in {phi, phi*} (checked) -> odd rows vanish identically"
    if s & {"b", "cbs", "bs"}:
        if row == ("b", "c"):
            return "ST4b_ghost_equation", "c.b row of (S,Gamma) = b Sigma_cbc + (d b) G_Kc: the ghost equation"
        if row == ("c", "cbs"):
            return "ST4b_unreachable", "c.cbar*: unreachable (XVI-P P22 a_closure: S0 quadratic part h.h, h.b, b.b, b.cbs)"
        return "zero", "auxiliary/nonminimal field on no interaction vertex (checked): no 1PI loop entry"
    table = {("h", "h"): "hh", ("phi", "phi"): "phiphi", ("c", "cb"): "cbc", ("c", "hs"): "Kc", ("c", "h"): "ST4b_ch",
             ("h",): "onepoint_h"}
    if row in table:
        return table[row], None
    if row == ("c",):
        return "zero", ("gh1 one-point defect c^mu(p=0): no 1PI one-loop graph has a single external ghost (the only ghost "
                        "vertex V_cb_c_h would need a second internal c to close the cbar line) and no Lorentz-covariant "
                        "constant vector exists")
    return "unclassified", None


def main(bulk_graphs_json):
    bg = jl(bulk_graphs_json)
    vc = vertex_checks()
    R = rows()
    A = jl(PV / "P15_batchA.json")
    C = jl(PV / "P16_batchC.json")
```

```
    D13 = jl(PV / "P15_D13.json")
    D19 = jl(PV / "P19_defect.json")
    E22 = jl(PV / "P22_evanescent.json")
    S23 = jl(PV / "P23_O1_scope.json")
    ST4 = jl(XO / "ledgers" / "symmetry_defect.json")["ST4_split"] if (XO / "ledgers" / "symmetry_defect.json").is_file() else None
    ev_ct = E22["EV_CT"]
    # ---------------- ST4a rows (gh 0)
    st4a = {}
    st4a["hh"] = {"graphs": ["D01", "D02", "D03", "D11", "D12"],
                  "values": {"D01_D03": "XVI-P P16_batchC (evaluated)", "D02": "0 derived (P15_batchA)",
                             "D12": bg["tadpoles"]["D12"]["value"] + " ; XVI-P P15_batchA: " + A["D12_slot"]["status"] if
isinstance(A.get("D12_slot"), dict) and "status" in A["D12_slot"] else bg["tadpoles"]["D12"]["value"],
                             "D11": "XVI-P P18_D11 (matching M1-M3; evanescent EV_M2)"},
                  "counterterm": "p14r 4D h.h MS-bar directions + p22 evanescent h.h operators (sum coefficients pinned in P22)",
                  "outcome": "cancellable", "evidence_flags": {"batchC_evaluated": bool(C.get("batchC_evaluated")),
                                                               "batchA_evaluated": bool(A.get("batchA_evaluated"))}}
    d06 = bg["D06"]
    pole = {k: sp.sympify(v["UV_pole"]) for k, v in d06["laurent"].items()}
    ccb_se_ct = sp.sympify(ev_ct.get("coefficient_ccb_SE", E22.get("EV_CT", {}).get("coefficient_ccb_SE", "nan")))
    st4a["cbc"] = {"graphs": ["D06"], "D06_equals_XVI_P_Sigma": d06["equals_XVI_P_P15_exactly"],
                   "poles": {k: str(v) for k, v in pole.items()},
                   "counterterm": {"4D (delta p^4, p^rho p_nu p^2)": {k: str(-v) for k, v in pole.items() if not k.startswith("SE")},
                                   "evanescent SE (p22 ghost-equation partner)": str(ccb_se_ct)},
                   "SE_pole_plus_p22_CT_zero": sp.simplify(pole.get("SE_Gt^rho_nu_p4", sp.nan) + ccb_se_ct) == 0,
                   "outcome": "cancellable"}
    d4, d5 = bg["D04"], bg["tadpoles"]["D05a"]
    st4a["phiphi"] = {"graphs": ["D04", "D05a"], "D04": d4.get("value"), "D04_routes_agree": d4.get("routes_agree"),
                      "D05a": d5["value"], "total_pole": "0", "counterterm": "z_phiphi = 0 (none needed; the declared phi phi "
                      "domain {phi Box^2 phi} is untouched)", "outcome": "zero" if d4.get("value") == "0" and not d5["logarithmic_member"] else
"OPEN"}
    st4a["Kc"] = {"graphs": ["D13"], "4D_poles_in_p14r_cK_basis": D13.get("UV_pole_in_p14r_cK_basis") is not None,
                  "E2_pole": D13["result"]["E2"]["UV_pole"], "p22_Kc_E2_CT": ev_ct.get("coefficient_Kc_E2"),
                  "E2_pole_plus_CT_zero": sp.simplify(sp.sympify(D13["result"]["E2"]["UV_pole"]) + sp.sympify(ev_ct.get("coefficient_Kc_E2"))) == 0,
                  "outcome": "cancellable"}
    st4a["onepoint_h"] = {"graphs": ["D14a", "D14b", "D14c"], "value": "0 derived (P15_batchA)", "outcome": "zero"}
    # ---------------- ST4b rows (gh 1)
    st4b = {"ST4b_ch": {"source": "XVI-P P19_defect: projected defect ZERO exactly in d; counterterm solve outcome "
                                  + D19["counterterm_solve"]["outcome"], "projected_identity_verified": D19["projected_identity_verified"],
                        "outcome": "zero" if D19["counterterm_solve"]["outcome"] == "ZERO" else "check"},
            "ST4b_ghost_equation": {"source": "XVI-P P15_D13 ghost_equation (64 components, exact in d) + P22 a_closure c.b "
                                              "(closed; exact in the gt sector); Sigma there == D06 here exactly",
                                    "ghost_equation_agree": D13["ghost_equation"]["agree"],
                                    "c_b_closed": E22["EV_CT"].get("a_closure", {}).get("c.b", {}).get("closed") if
isinstance(E22["EV_CT"].get("a_closure"), dict) else None,
                                    "D06_is_that_Sigma": d06["equals_XVI_P_P15_exactly"], "outcome": "zero"},
            "ST4b_unreachable": {"source": "P22 a_closure c.cbs", "outcome": "zero"}}
    # ---------------- assemble all rows
    table = {}
    for grp, lst in R.items():
        for row in lst:
            cls, why = classify(row, vc)
            ent = {"group": grp, "class": cls}
            if why:
                ent["reason"] = why
            if cls in st4a:
                ent["outcome"] = st4a[cls]["outcome"]
            elif cls in st4b:
                ent["outcome"] = st4b[cls]["outcome"]
            elif cls == "zero":
                ent["outcome"] = "zero"
            else:
                ent["outcome"] = "UNCLASSIFIED"
            table[".".join(row)] = ent
    unclassified = [k for k, v in table.items() if v["outcome"] in ("UNCLASSIFIED", "OPEN", "check")]
    # ---------------- full sigma_0 closure
    fc = D19.get("full_closure_mutation_detector", {})
    closure = {"XVI_P_full_closure_all_gh1_sectors": "P19_defect full_closure_mutation_detector + renormalised_defect_zero",
               "renormalised_defect_zero": D19.get("renormalised_defect_zero"),
               "mutation_detector_cases_detected": {k: v.get("detected") for k, v in fc.get("cases", {}).items()},
               "phiphi_CT": "0 (D04 + D05a = 0); sigma_0 rank on the phi.phi direction is 0 (XVI-P P19 D_cap_ker_sigma0), so the "
                            "direction is sigma_0-closed for any coefficient; NO free-EOM equivalence is used (XVI-O O.6 domain item 4)",
               "phiphi_sigma0_rank": D19["D_cap_ker_sigma0"]["sector_ranks"]["phi.phi"]["sigma0_rank"],
               "phi_odd_rows": "identically zero (parity)", "closed": True}
    # ---------------- explicit bulk counterterms in the declared p14r domain (11 directions) + Q02 phi.phi
    dom = jl(PV / "P14_counterterm_domain.json")["kappa2"]["sectors"]
    g1 = D19["gamma1_local_data_domain_coordinates"]
    labels = g1["labels"]
    ct = {}
    for lab, pl in zip(labels, g1["pole"]):
        ct[lab] = -sp.sympify(pl)
    cbc_routeG = [sp.sympify(v) for v in g1["c_cb_ghost_self_energy (route-G (c,cb) orientation, p20)"]["pole"]]
    cbc_P15 = [sp.sympify(v) for v in g1["c_cb_ghost_self_energy (P15 (cb,c) orientation)"]["pole"]]
    ct["c.cb#0"], ct["c.cb#1"] = -cbc_routeG[0], -cbc_routeG[1]
    d06_poles = [pole["S1_delta^rho_nu_p4"], pole["S2_p^rho_p_nu_p2"]]
```

```
    phiphi_pole = sp.Integer(0) if st4a["phiphi"]["outcome"] == "zero" else None
    ct["phi.phi#0"] = -phiphi_pole if phiphi_pole is not None else None
    def in_Qi(v):
        v = sp.nsimplify(v)
        re, im = v.as_real_imag()
        return v is not None and re.is_rational and im.is_rational
    counterterms = {"convention": "Ghat coordinates on the p14r kappa^2 densities (G = kappa^2 N_eps Ghat MB); MS-bar CT = -pole; "
                                  "c.cb in the route-G (c,cb) orientation used by the p20 full closure",
                    "coefficients": {k: str(v) for k, v in ct.items()},
                    "domain_sectors_and_sizes": {k: len(v["structures"]) for k, v in dom.items()},
                    "every_label_in_domain": all(lab.split("#")[0] in dom and int(lab.split("#")[1]) < len(dom[lab.split("#")[0]]["structures"]) for
lab in ct),
                    "n_directions": len(ct), "all_in_Q_i": all(v is not None and in_Qi(v) for v in ct.values()),
                    "cbc_from_D06_equals_P15_orientation_poles": [sp.simplify(a - b) == 0 for a, b in zip(d06_poles, cbc_P15)],
                    "cbc_orientation_ratio_routeG_over_P15": [str(sp.simplify(a / b)) for a, b in zip(cbc_routeG, cbc_P15)],
                    "phiphi": "0 from Q02 D04 + D05a (in P19 the phi.phi coordinate 0 was a placeholder 'not an input'; now earned)",
                    "evanescent_p22": {"Kc_E2": ev_ct.get("coefficient_Kc_E2"), "ccb_SE": ev_ct.get("coefficient_ccb_SE"),
                                       "hh_sum": ev_ct.get("coefficients_hh_counterterm_sum"),
                                       "role": "outside the p14r 4D domain (disclosed p22 contract revision); subtract the "
                                               "evanescent poles only; NOT needed for the identity (P19 ch solve x = 0; "
                                               "evanescent poles decouple from the ch slots)"}}
    counterterms["ok"] = counterterms["every_label_in_domain"] and counterterms["all_in_Q_i"] and
all(counterterms["cbc_from_D06_equals_P15_orientation_poles"]) \
        and counterterms["n_directions"] == 11
    closure.update({"full_closure_with_complete_MSbar_CT": D19["counterterm_solve"].get("full_closure_reading"),
                    "relative_factor_derived_equals_P15": D19["counterterm_solve"].get("K1_ghost_closure_convention", {}).get("derived_equals_P15"),
                    "sigma0_ranks": D19["D_cap_ker_sigma0"]["sector_ranks"], "cohomology_dim_D": D19["D_cap_ker_sigma0"]["cohomology_dim"]})
    obstruction = [k for k, v in table.items() if v["outcome"] == "obstruction"]
    outcome = "obstruction" if obstruction else ("OPEN" if unclassified else
                                                 ("cancellable" if any(v["outcome"] == "cancellable" for v in table.values()) else "zero"))
    return {"XVI_O_ST4_definition_verbatim": ST4, "bulk_counterterms_explicit": counterterms,
            "independent_graph_enumeration": enumerate_graphs(), "vertex_checks": vc, "rows": table, "n_rows": len(table),
            "unclassified_rows": unclassified, "ST4a": st4a, "ST4b": st4b, "sigma0_closure": closure,
            "bulk_identity_outcome": outcome, "residual": None if outcome != "obstruction" else obstruction,
            "E2_enumeration_XVI_P_p23": S23.get("power_counting", S23).get("frozen_E2_match"),
            "bulk_counterterms": {"hh": "p14r MS-bar h.h + p22 evanescent (pinned)", "Kc": "p14r c.K' + p22 E2 (-1/16)",
                                  "cbc": {"4D": {k: str(-v) for k, v in pole.items() if not k.startswith("SE")}, "SE": str(ccb_se_ct)},
                                  "phiphi": "0", "onepoint": "0"}}


def o1_walk(bulk, O2_status, d12_reconciled, d04_scrutiny=None):
    """Walk XVI-O's O.1 closure_check VERBATIM (checklist, closure reasons, enlarge and downgrade rules).  CLOSED only
    if every item is earned; the label is CLOSED_PENDING_INDEPENDENT_REVIEW until the Q02 independent rebuild passes."""
    tgt = jl(XO / "ledgers" / "O_quantum_target.json")
    cc = tgt["closure_check"]
    vc = bulk["vertex_checks"]
    E2 = bulk["E2_enumeration_XVI_P_p23"] or {}
    en = bulk["independent_graph_enumeration"]
    ct = bulk["bulk_counterterms_explicit"]
    sc = bulk["sigma0_closure"]
    st = {c["item"]: c["status"] for c in cc["checklist"]}
    d04_ok = bool(d04_scrutiny) and d04_scrutiny.get("declared_zero") is True and d04_scrutiny.get("all_mutations_nonzero") is True
    items = [
        {"item": "State retained quantum identity", "owner": "O.6", "XVI_O_status": st["State retained quantum identity"],
         "evidence": "XVI-O ST4a/ST4b and (S,Gamma_1) = A_1 (retained_quantum_identity.tex) stated on the bulk two-point "
                     "retained sector; 23 gh0/gh1 rows enumerated mechanically",
         "earned": bulk["XVI_O_ST4_definition_verbatim"] is not None and bulk["n_rows"] == 23},
        {"item": "List amplitudes constrained by identity", "owner": "O.4", "XVI_O_status": st["List amplitudes constrained by identity"],
         "evidence": "every gh0/gh1 one/two-field row classified with an individual justification; independent 1PI "
                     f"enumeration from the loop-capable vertices: E2 = {en['n_E2']}, E1 = {en['n_E1']} (== XVI-P 9 / 3); "
                     "antifield vertices c*cc, phi*c phi give no E <= 2 graph",
         "earned": not bulk["unclassified_rows"] and E2.get("equal") is True and en["n_E2"] == 9 and en["n_E1"] == 3
                   and vc["phi_parity_even_all"] and vc["aux_b_cbs_bs_on_no_interaction_vertex"]},
        {"item": "S_inv^{[2]} classical consistency", "owner": "O.2", "XVI_O_status": st["S_inv^{[2]} classical consistency"],
         "evidence": "XVI-Q Q01: 2(S0,S2)+(S1,S1)=0 Constructed ((a) ell_4, (b) explicit antibracket expansion, (c) "
                     "ghost/antifield); Q01 independent rebuild PASSED", "earned": O2_status == "Constructed"},
        {"item": "Evaluate inventory graphs", "owner": "O.5", "XVI_O_status": st["Evaluate inventory graphs"],
         "evidence": "all 9 E=2 + 3 E=1 graphs have exact values: D01, D02, D03, D11, D13, D14a-c (XVI-P, pinned); D04 "
                     "(Q02, 0 by engine routes 1+2 and independent route 4, with nonzero mutations, literature agreement); "
                     "D05a (Q02, derived 0); D06 (Q02 == XVI-P Sigma); D12 reconciled (derived 0)",
         "earned": all(bulk["ST4a"][r]["outcome"] in ("zero", "cancellable") for r in ("hh", "cbc", "phiphi", "Kc", "onepoint_h"))
                   and d12_reconciled and d04_ok},
        {"item": "Demonstrate identity remainder cancellation / CT solution", "owner": "O.6/O.7",
         "XVI_O_status": st["Demonstrate identity remainder cancellation / CT solution"],
         "evidence": "ST4a: 11-direction MS-bar CT in the declared p14r domain over Q(i) (phi.phi = 0 now earned); ST4b: "
                     "c.h defect ZERO exactly in d (P19, x = 0), c.b = ghost equation (Sigma == D06), c.cbar* unreachable, "
                     "c.phi parity; full sigma_0 closure of the complete MS-bar CT in every gh+1 sector (p20); cohomology "
                     "of the domain 0; no free-EOM equivalence used",
         "earned": bulk["bulk_identity_outcome"] in ("zero", "cancellable") and sc["closed"] and ct["ok"]
                   and sc.get("relative_factor_derived_equals_P15") is True},
        {"item": "Enlarge target if identity requires more legs/orders", "owner": "O.1",
         "XVI_O_status": st["Enlarge target if identity requires more legs/orders"],
         "evidence": "rule 1 evaluated below: not triggered", "earned": True}]
    reasons = {
```

```
        cc["reason"][0]: "ANSWERED: identity verified on the retained two-point sector (items 1, 5).",
        cc["reason"][1]: "ANSWERED: the quantum identity is the EA-route (S,Gamma_1) = A_1, not XVI-N's classical truncation.",
        cc["reason"][2]: "ANSWERED: XVI-M selected-image data are not used anywhere (graphs come from the action's vertices).",
        cc["reason"][3]: "ANSWERED: one-loop antifield channels derived from vertices: K'.c = D13 (evaluated); phi*'.c odd (0); "
                         "c*', cbar*', b* give no E <= 2 graph (independent enumeration); no (Phi, Phi*) assignment used.",
        cc["reason"][4]: "ANSWERED: completeness by mechanical enumeration of rows AND graphs (two independent enumerations).",
        cc["reason"][5]: "ANSWERED: kappa^2 master Constructed in Q01 (incl. grav ell_4).",
        cc["reason"][6]: "ANSWERED: ST4a local matching = explicit CT in the declared domain; ST4b on the retained two-point "
                         "projection verified (the FUNCTIONAL ST4b, all n-point, is NOT claimed).",
        cc["reason"][7]: "ANSWERED: the bulk sector (hh, phi phi, cbar c, K'.c, one-point) is evaluated, not only the D11 kernel."}
    rules = {
        tgt["closure_check"]["enlarge_rules"][0]: {
            "triggered": False,
            "argument": "At one loop the kappa^2 component of (S,Gamma_1) is (S_0, Gamma_1[kappa^2]) + (S_1, Gamma_1[kappa^1]) "
                        "(S_k: tree kappa^k terms incl. the seeds D07-D09). One-loop graphs with E external legs are kappa^E "
                        "(every vertex kappa^{legs-2}, p14r power counting; XVI-P p23): Gamma_1[kappa^1] is one-point only and "
                        "is 0 (D14a-c), Gamma_1[kappa^2] is two-point only (three-point starts at kappa^3; one-loop kappa^2 "
                        "three-point sector empty). Hence the identity at this order constrains no n >= 3 or higher-loop piece."},
        tgt["closure_check"]["enlarge_rules"][1]: {"triggered": False, "argument": "classical kappa^2 consistency holds (Q01)."},
        tgt["closure_check"]["enlarge_rules"][2]: {"respected": True,
                                                   "argument": "no XVI-M graph is used or identified with an action graph; "
                                                               "XVI_M pin unchanged; every graph is built from S_inv vertices."}}
    down = {
        tgt["closure_check"]["downgrade_rules"][0]: "not applicable: closure decided by the checks above",
        tgt["closure_check"]["downgrade_rules"][1]: "respected: no amplitude omitted; every row/graph individually justified"}
    allx = all(i["earned"] for i in items)
    return {"ledger_closure_check_verbatim": cc, "items": items, "closure_reasons_answered": reasons,
            "enlarge_rules": rules, "downgrade_rules": down, "all_earned": allx,
            "O1_closure": "CLOSED_PENDING_INDEPENDENT_REVIEW" if allx else "NOT_CLOSED",
            "O1_closure_confirmed": False,
            "confirmation_condition": "Q02 independent rebuild PASS (until then: pending independent review)",
            "remaining": [i["item"] for i in items if not i["earned"]]}


if __name__ == "__main__":
    b = main(sys.argv[1])
    print(json.dumps({k: b[k] for k in ("bulk_identity_outcome", "n_rows", "unclassified_rows")}, indent=1))
```

code/q02/d04_route4.py

```
#!/usr/bin/env python3
"""XVI-Q Q02 scrutiny of D04 = 0: an INDEPENDENT route (no XVI-P engine code) with covariant algebra and an own
Feynman-parameter evaluation with symbolic dimension D (a plain symbol: this file never assigns D a value), plus mutations.

Mechanism. Vertex (phi(p), phi(q), graviton k = -(p+q)):
  V_ab = p_a q_b + p_b q_a - alpha_v eta_ab p.q + lam [k_a k_b - eta_ab k^2]   (lam ~ non-minimal xi R phi^2 coupling)
propagator numerator P = 1/2(eta eta + eta eta) - c eta eta (+ g3 * Froeb's T3 = 4 l_(a eta_b)(c l_d)/l^2 gauge piece),
vertex-index trace eta.eta = n.  With alpha_v = 1, lam = 0, g3 = 0:
  V.P.V' = 2 p^2 q^2 + (n - 2)(p.q)^2 [1 - c (n - 2)],
so the (p.q)^2 (trace-mode) piece cancels iff c = 1/(n - 2): DRED (n = 4, c = 1/2) and CDR (n = D, c = 1/(D - 2)) both
give V.P.V' = 2 p^2 (1 - p)^2, which cancels the scalar propagator; the remainder p^2 int 1/l^2 is scaleless (omega = 2,
no logarithmic member).  D04 = 0 is therefore an exact statement of de Donder gauge + minimal coupling + m = 0, NOT a
property of scaleless integrals in general (the bubble with p != 0 is not scaleless).
Literature: Froeb, CQG 35 (2018) 055006 [arXiv:1710.00839], eq. inv2pf_termsumint_grav: graviton-loop self-energy with
the de Donder parts g1, g2 of the propagator in n dimensions; the integrand bracket is proportional to m^2 or xi, and
the text states it 'vanishes identically' for xi = 0, m = 0 'in accordance with' Kahya-Woodard 2007; for m = 0 the xi^2
terms vanish; gauge-dependent propagator pieces (T3..T5) contribute (cancelled there only by coordinate corrections)."""
from __future__ import annotations
import json
import sys

import sympy as sp

D = sp.Symbol("D")                       # own dimension symbol (never substituted)
PP, PL, LL = sp.symbols("pp pl ll")     # p.p, p.l, l.l
x = sp.Symbol("x")


# ---------------- vectors over the basis (p, l) and their dot product
def vec(cp, cl):
    return (sp.sympify(cp), sp.sympify(cl))


def dot(u, v):
    return sp.expand(u[0] * v[0] * PP + (u[0] * v[1] + u[1] * v[0]) * PL + u[1] * v[1] * LL)


def add(*vs):
    return (sum(v[0] for v in vs), sum(v[1] for v in vs))


def sc(a, v):
    return (a * v[0], a * v[1])
```

```
P_ = vec(1, 0)
L_ = vec(0, 1)


# ---------------- symmetric tensors: list of (coef, u, v) for coef * u v^T, plus coefficient of eta
class T:
    def __init__(self, outer, eta):
        self.outer, self.eta = outer, sp.sympify(eta)

    def dot(self, o, n):
        s = sum(a * b * dot(u, u2) * dot(v, v2) for a, u, v in self.outer for b, u2, v2 in o.outer)
        s += self.eta * o.trace(n) + o.eta * sum(a * dot(u, v) for a, u, v in self.outer) - self.eta * o.eta * n + self.eta * o.eta * n
        return sp.expand(s)

    def trace(self, n):
        return sp.expand(sum(a * dot(u, v) for a, u, v in self.outer) + n * self.eta)

    def contract_vec(self, w):
        """(T w)_a as a vector."""
        out = vec(0, 0)
        for a, u, v in self.outer:
            out = add(out, sc(a * dot(v, w), u))
        return add(out, sc(self.eta, w))


def vertex(p, q, alpha_v=1, lam=0):
    k = sc(-1, add(p, q))
    outer = [(1, p, q), (1, q, p)]
    eta = -alpha_v * dot(p, q)
    if lam:
        outer.append((lam, k, k))
        eta -= lam * dot(k, k)
    return T(outer, eta)


def numerator45(n, g4=0, g5=0):
    """Froeb T4 = eta_ab l_c l_d/l^2 + l_a l_b eta_cd/l^2 (coefficient g4 ~ (2+beta)) -> numerator over one extra l^2;
    T5 = l_a l_b l_c l_d / l^4 (coefficient g5) -> numerator over (l^2)^2."""
    q = add(L_, sc(-1, P_))
    VA, VB = vertex(P_, q), vertex(sc(-1, P_), sc(-1, q))
    lVl_A, lVl_B = dot(L_, VA.contract_vec(L_)), dot(L_, VB.contract_vec(L_))
    N4 = sp.expand(g4 * (VA.trace(n) * lVl_B + VB.trace(n) * lVl_A))
    N5 = sp.expand(g5 * lVl_A * lVl_B)
    return N4, N5


def numerator(n, c, alpha_v=1, lam=0, g3=0):
    """N = V(p, q) . P . V(-p, -q), q = l - p, graviton momentum l; returns (N_deDonder, N_T3) with N_T3 to be divided
    by an extra l^2 (Froeb T3 = 4 l_(mu eta_nu)(rho l_sigma)/l^2)."""
    q = add(L_, sc(-1, P_))
    VA = vertex(P_, q, alpha_v, lam)
    VB = vertex(sc(-1, P_), sc(-1, q), alpha_v, lam)
    N0 = sp.expand(VA.dot(VB, n) - c * VA.trace(n) * VB.trace(n))
    N3 = sp.expand(g3 * 4 * dot(VA.contract_vec(L_), VB.contract_vec(L_))) if g3 else sp.Integer(0)
    return N0, N3


# ---------------- own Feynman-parameter integration (Euclidean form; ratios to the master bubble)
def kint(j, Nexp):
    """int d^Dk (k^2)^j/(k^2+Delta)^N  /  [pi^{D/2}] = G(j+D/2) G(N-j-D/2)/(G(D/2) G(N)) Delta^{D/2+j-N}"""
    return sp.gamma(j + D / 2) * sp.gamma(Nexp - j - D / 2) / (sp.gamma(D / 2) * sp.gamma(Nexp))


def bubble(num, a, b):
    """int num(l,p)/((l^2)^a ((l-p)^2)^b), normalised by the master int 1/(l^2 (l-p)^2); returns rational(D) * pp^power.
    l = k + x p, (1-x) l^2 + x (l-p)^2 = k^2 + x(1-x) pp; symmetric integration in D dims."""
    num = sp.expand(num)
    if num == 0:
        return sp.Integer(0)
    kk, kp = sp.symbols("kk kp")
    # substitute l.l = kk + 2 x kp + x^2 pp, p.l = kp + x pp
    e = sp.expand(num.subs({LL: kk + 2 * x * kp + x ** 2 * PP, PL: kp + x * PP}, simultaneous=True))
    Ptot = a + b
    pref = sp.gamma(Ptot) / (sp.gamma(a) * sp.gamma(b))
    tot = 0
    for (ik, ip), cf in sp.Poly(e, kk, kp).terms():
        if ip % 2:
            continue
        m = ip // 2
        # <(k.p)^{2m}> = (2m-1)!! (k^2)^m pp^m / prod_{i<m}(D + 2i)
        ang = sp.factorial2(2 * m - 1) * PP ** m / sp.Mul(*[D + 2 * i for i in range(m)]) if m else 1
        j = ik + m
```

```
        # x-integral of x^{b-1} (1-x)^{a-1} * cf(x) * [x(1-x)]^{D/2+j-Ptot}
        for (ex,), cx in sp.Poly(sp.expand(cf), x).terms():
            beta = sp.gamma(ex + b + D / 2 + j - Ptot) * sp.gamma(a + D / 2 + j - Ptot) / sp.gamma(ex + a + b + D + 2 * j - 2 * Ptot)
            tot += pref * cx * ang * kint(j, Ptot) * beta * PP ** (j - Ptot + 2)   # pp power relative to the master
    master = kint(0, 2) * sp.gamma(D / 2 - 1) ** 2 / sp.gamma(D - 2)
    return sp.factor(sp.simplify(sp.gammasimp(sp.expand(tot) / master)))


def D04(n, c, alpha_v=1, lam=0, g3=0, g4=0, g5=0):
    N0, N3 = numerator(n, c, alpha_v, lam, g3)
    N4, N5 = numerator45(n, g4, g5)
    return sp.factor(sp.simplify(bubble(N0, 1, 1) + (bubble(N3, 2, 1) if N3 != 0 else 0)
                                 + (bubble(N4, 2, 1) if N4 != 0 else 0) + (bubble(N5, 3, 1) if N5 != 0 else 0)))


def main():
    xi = sp.Symbol("xi")
    out = {"mechanism_numerator_general_c_n": str(sp.factor(numerator(sp.Symbol("n"), sp.Symbol("c"))[0])),
           "mechanism_reading": "V.P.V' = 2 p^2 (1-p)^2 + (n-2)(p.q)^2 [1 - c(n-2)]: trace-mode piece cancels iff c = 1/(n-2)"}
    N_dred = numerator(4, sp.Rational(1, 2))[0]
    out["N_DRED_equals_2p2(1-p)^2"] = sp.expand(N_dred - 2 * PP * (LL - 2 * PL + PP)) == 0
    cases = {
        "DRED (n=4, c=1/2) -- declared": (4, sp.Rational(1, 2), 1, 0, 0),
        "CDR (n=D, c=1/(D-2))": (D, 1 / (D - 2), 1, 0, 0),
        "MUT mixed scheme (n=4, c=1/(D-2))": (4, 1 / (D - 2), 1, 0, 0),
        "MUT vertex trace normalisation alpha_v=1/2": (4, sp.Rational(1, 2), sp.Rational(1, 2), 0, 0),
        "GAUGE T3 piece g3=1 (expected 0: l^a V_ab = p_b q^2 + q_b p^2, every term carries q^2 -> scaleless)": (4, sp.Rational(1, 2), 1, 0, 1, 0, 0),
        "MUT non-de Donder gauge T4 piece (beta != -2), g4=1": (4, sp.Rational(1, 2), 1, 0, 0, 1, 0),
        "MUT non-de Donder gauge T5 piece, g5=1": (4, sp.Rational(1, 2), 1, 0, 0, 0, 1),
        "MUT non-minimal coupling lam=xi (CDR)": (D, 1 / (D - 2), 1, xi, 0),
    }
    res = {}
    for k, args in cases.items():
        v = D04(*args)
        res[k] = {"value_over_master": str(v), "zero": v == 0}
    out["cases"] = res
    out["T3_zero_by_Ward_identity"] = res[next(k for k in res if k.startswith("GAUGE T3"))]["zero"]
    v_xi = sp.sympify(res["MUT non-minimal coupling lam=xi (CDR)"]["value_over_master"], locals={"D": D, "pp": PP, "xi": xi})
    out["xi_dependence_linear_only (Froeb: m=0 xi^2 terms vanish)"] = sp.expand(v_xi).coeff(xi, 2) == 0 and sp.expand(v_xi).coeff(xi, 1) != 0
    out["declared_zero"] = res["DRED (n=4, c=1/2) -- declared"]["zero"] and res["CDR (n=D, c=1/(D-2))"]["zero"]
    out["all_mutations_nonzero"] = all(not r["zero"] for k, r in res.items() if k.startswith("MUT"))
    out["literature"] = {"reference": "M. B. Froeb, Class. Quantum Grav. 35 (2018) 055006, arXiv:1710.00839 (flat space, general gauge, n dims)",
                         "statement": "graviton-loop scalar self-energy from the de Donder propagator parts vanishes identically for "
                                      "xi = 0, m = 0 (in accordance with Kahya & Woodard 2007); m = 0 xi^2 terms vanish; "
                                      "gauge-parameter pieces contribute (gauge-dependent 1PI)",
                         "agreement": "AGREES: declared case 0; xi enters linearly only; non-de Donder piece nonzero"}
    return out


if __name__ == "__main__":
    r = main()
    print(json.dumps(r, indent=1))
```

code/q02/q02_certs.py

```
#!/usr/bin/env python3
"""XVI-Q Q02: runs the bulk graphs (bulk_graphs.py) and the bulk identity / 0.1 walk (bulk_identity.py) and writes
certificates Q02_bulk_graphs.json, Q02_D12_reconciliation.json, Q02_bulk_identity.json, Q02_01.json."""
from __future__ import annotations
import json
import subprocess
import sys
from pathlib import Path

Q = Path(__file__).resolve().parents[2]
sys.path.insert(0, str(Q / "code" / "q02"))
import bulk_identity as BI  # noqa: E402

CERT = Q / "certificates"


def w(name, obj):
    (CERT / f"{name}.json").write_text(json.dumps({"landmark": "XVI-Q", "release": "Q02", **obj}, indent=1, default=str) + "\n")


def main():
    bg_path = Q / "build" / "q02_bulk_graphs.json"
    r = subprocess.run([sys.executable, str(Q / "code" / "q02" / "bulk_graphs.py"), str(bg_path)], capture_output=True, text=True)
    if r.returncode != 0:
        print(r.stdout[-2000:], r.stderr[-3000:])
        raise SystemExit("bulk_graphs failed")
    bg = json.loads(bg_path.read_text())
    w("Q02_bulk_graphs", bg)
```

```
    A = json.loads((Q / "provenance/XVI-P/certificates/P15_batchA.json").read_text())
    s23 = (Q / "provenance/XVI-P/certificates/P23_O1_scope.json").read_text()
    rec = {"graph": "D12_hh_scalar_tadpole",
           "XVI_O": "inventory 'provisional_listed', sym_factor 1/2 'Unverified', never evaluated; XVI-O ledgers do NOT label "
                    "D12 'Missing' (searched: O_quantum_target, amplitude_status, symmetry_defect, diagram_inventory)",
           "XVI_P_p14r": A["D12_slot"]["previous_status"], "XVI_P_p15": A["D12_slot"]["status"],
           "XVI_P_p15_derivation": A["D12_slot"]["derivation"],
           "XVI_P_p23_stale_line_present": "D12 Missing never zero per P14" in s23,
           "XVI_Q_Q02_independent": bg["tadpoles"]["D12"],
           "reconciliation": ("'Missing' was the p14r status (integrand only, no value) and survives only as stale text in XVI-P "
                              "P23_O1_scope's 0.5 evidence line (self-contradictory with P15, already flagged by the p23 review). "
                              "D12 was EVALUATED in XVI-P p15: vanishing_under_declared_prescription by two routes with IR-mass "
                              "and cutoff analysis; Q02 confirms independently on the engine integrand: every member "
                              "int (l^2)^j / l^2 with omega in {2, 4}, no logarithmic member, so neither a UV nor an IR pole: DR "
                              "value 0 DERIVED (not by convention; Missing is never set to zero). Recorded as erratum E6; XVI-O "
                              "and XVI-P untouched."),
           "status": "Evaluated_vanishing_under_declared_prescription",
           "consistent": A["D12_slot"]["status"] == "vanishing_under_declared_prescription"
           and not bg["tadpoles"]["D12"]["logarithmic_member"]}
    w("Q02_D12_reconciliation", rec)
    sys.path.insert(0, str(Q / "code" / "q02"))
    import d04_route4 as R4
    scr = R4.main()
    scr["engine_routes"] = {k: bg["D04"][k] for k in ("route1_A", "route2_IBP_A", "routes_agree", "N_factorised",
                                                      "N_proportional_to_p2_times_scalar_denominator")}
    scr["engine_mutation_drop_trace"] = bg["mutations"].get("D04_drop_trace_part_value")
    scr["E7"] = ("symmetry factor of D04 is 1 (distinct-species internal lines; no automorphism with labelled external legs). "
                 "XVI-O's '1/2' was Unverified and never used for a value; XVI-P never evaluated D04 (excluded from ch by "
                 "phi-parity) and its D11 (two identical phi lines) keeps S_f = 1/2 correctly: NO earlier value changes.")
    scr["D05a_D12"] = ("genuine one-propagator tadpoles: no external momentum flows through the loop line, so the integral "
                       "is scaleless; omega = 2 (D05a) / 2, 4 (D12) power divergences, no logarithmic member -> no 1/eps on "
                       "either side; 0 is not a UV/IR pole cancellation. (Unlike D04, whose bubble is NOT scaleless.)")
    w("Q02_D04_scrutiny", scr)
    bulk = BI.main(bg_path)
    w("Q02_bulk_identity", bulk)
    o2 = json.loads((CERT / "Q01_O2.json").read_text())["O2_kappa2_master_status"]
    walk = BI.o1_walk(bulk, o2, rec["consistent"], scr)
    walk["scope"] = ("one-loop kappa^2 bulk two-point sector (hh, phi phi, cbar c, K'.c, one-point) of S_inv, m = 0, frozen gauge "
                     "fermion, under the XVI-P declared prescription (DRED with opaque d, MS-bar, Feynman +i0, p22 evanescent "
                     "counterterms; external momenta in QdS); insertion frozen (XVI-N massless obstruction untouched)")
    walk["not_implied"] = ["bulk_symmetry_restored (all orders / all n-point): false",
                           "finite renormalised amplitudes with non-QdS (epsilon-scalar) external components: the p22 "
                           "Q4S-trace O(eps)x pole remnant stays disclosed (identity unaffected: projected Q4S sum 0)",
                           "three-point / higher orders (P.6), prepared S_D, insertion"]
    w("Q02_O1", walk)
    print(json.dumps({"d04_declared_zero": scr["declared_zero"], "bulk": bulk["bulk_identity_outcome"], "rows": bulk["n_rows"], "unclassified":
bulk["unclassified_rows"],
                      "O1": walk["O1_closure"], "remaining": walk["remaining"]}))


if __name__ == "__main__":
    main()
```

code/q02/release_q02.py

```
#!/usr/bin/env python3
"""XVI-Q Q02 release gate (fails closed; on reject no RELEASE_STATUS.json is left).
(1) the expected-values file code/q02/q02_expected.json is itself pinned (EXPECTED_SHA256 below);
(2) EVERY certified value: each Q01_*/Q02_* certificate (run-specific 'seconds' removed) must equal its pinned copy
    in the expected file -- every number, string and flag, path by path;
(3) semantic checks independent of the pins: per-file intake pins (each True, count == imports.json), the O.2(c)
    ghost/antifield cells (exact cell set, all True, kappa^0..kappa^2, read from the build input P13_master_kappa2 too),
    D04/D05a/D06/D12 values, bulk rows, O.1 walk consistency, hard stops, conventions files pinned."""
from __future__ import annotations
import hashlib
import json
import sys
from pathlib import Path

Q = Path(__file__).resolve().parents[2]
CERT = Q / "certificates"
EXPECTED_FILE = Q / "code" / "q02" / "q02_expected.json"
EXPECTED_SHA256 = "5b50d16935d3ff9a8ab8a9eb51ddcaf7d3bbb46870ac6c9208a80e7e0e1e36ec"
STATUS = (Q / "RELEASE_STATUS.json", CERT / "RELEASE_STATUS.json")
if "--layer" in sys.argv:   # Q03: run as an earlier-layer gate; RELEASE_STATUS is owned by the Q03 gate
    STATUS = ()
CELLS = {"af0_grav_Noether", "af0_scalar_Noether", "af0_ghost_gf_transformed", "af1_invariant", "af1_transformed",
         "af2_invariant", "af2_transformed"}
HARD_STOPS = {"bulk_symmetry_restored": False, "insertion": "frozen", "XV_B_opened": False, "XIV_J_opened": False,
              "Route_B_discharged": False, "Missing_never_zero": True}


def sha(p):
```

```
    return hashlib.sha256(Path(p).read_bytes()).hexdigest()


def strip(o):
    if isinstance(o, dict):
        return {k: strip(v) for k, v in o.items() if k != "seconds"}
    if isinstance(o, list):
        return [strip(v) for v in o]
    return o


def diff_paths(a, b, path=""):
    if isinstance(a, dict) and isinstance(b, dict):
        out = []
        for k in sorted(set(a) | set(b)):
            if k not in a or k not in b:
                out.append(f"{path}/{k} (missing)")
            else:
                out += diff_paths(a[k], b[k], f"{path}/{k}")
        return out
    if isinstance(a, list) and isinstance(b, list):
        if len(a) != len(b):
            return [f"{path} (length)"]
        return sum((diff_paths(x, y, f"{path}[{i}]") for i, (x, y) in enumerate(zip(a, b))), [])
    return [] if a == b else [path]


def jl(n):
    f = CERT / f"{n}.json"
    return json.loads(f.read_text()) if f.is_file() else None


def checks():
    c = {}
    c["expected_file_pinned"] = EXPECTED_FILE.is_file() and sha(EXPECTED_FILE) == EXPECTED_SHA256
    exp = json.loads(EXPECTED_FILE.read_text()) if EXPECTED_FILE.is_file() else {}
    bad = {}
    for name, ref in exp.get("certificates", {}).items():
        got = jl(name)
        d = ["(certificate missing)"] if got is None else diff_paths(strip(got), ref)
        if d:
            bad[name] = d[:20]
    c["every_certified_value_matches_pinned"] = bool(exp.get("certificates")) and not bad
    c["_value_mismatches"] = bad
    c["conventions_files_pinned"] = all((Q / f).is_file() and sha(Q / f) == s for f, s in exp.get("file_sha256", {}).items()) \
        and len(exp.get("file_sha256", {})) >= 4
    it = jl("Q01_intake") or {}
    imp = json.loads((Q / "intake" / "imports.json").read_text())
    c["per_file_pins_each_true"] = isinstance(it.get("pins"), dict) and len(it["pins"]) == len(imp["files"]) \
        and all(v is True for v in it["pins"].values()) and it.get("n_pins") == len(imp["files"])
    o2 = jl("Q01_O2") or {}
    tbl = o2.get("items", {}).get("c_ghost_antifield", {}).get("kappa2_table", {})
    c["O2c_cells_exact_and_true"] = set(tbl) == CELLS and all(v is True for v in tbl.values())
    m = Q / "build" / "xvip_run" / "certificates" / "P13_master_kappa2.json"
    if m.is_file():
        t = json.loads(m.read_text())["table_by_kappa_and_antifield_number"]
        c["O2c_build_input_all_orders"] = set(t) == {"kappa0", "kappa1", "kappa2"} and all(
            set(t[k]) == CELLS and all(v is True for v in t[k].values()) for k in t)
    else:
        c["O2c_build_input_all_orders"] = False
    c["O2_constructed"] = o2.get("O2_kappa2_master_status") == "Constructed"
    bg = jl("Q02_bulk_graphs") or {}
    d4, d6, tp = bg.get("D04", {}), bg.get("D06", {}), bg.get("tadpoles", {})
    c["D04_zero_two_routes"] = d4.get("value") == "0" and d4.get("routes_agree") is True and d4.get("route1_A") == "0" \
        and d4.get("route2_IBP_A") == "0" and d4.get("N_proportional_to_p2_times_scalar_denominator") is True
    c["D06_equals_XVI_P_and_exact"] = d6.get("equals_XVI_P_P15_exactly") is True and d6.get("fit_exact") is True and \
        {k: v["UV_pole"] for k, v in d6.get("laurent", {}).items()} == {"S1_delta^rho_nu_p4": "-3/8", "S2_p^rho_p_nu_p2": "1/4", "SE_Gt^rho_nu_p4":
"-1/8"}
    c["D05a_D12_no_log_member"] = all(tp.get(g, {}).get("logarithmic_member") is False for g in ("D05a", "D12"))
    c["mutations_detect"] = bg.get("mutations", {}).get("D04_drop_trace_part_changes_A") is True and \
        bg.get("mutations", {}).get("D06_S_f_half_changes_coefficients") is True
    rec = jl("Q02_D12_reconciliation") or {}
    c["D12_reconciled"] = rec.get("consistent") is True and rec.get("status") == "Evaluated_vanishing_under_declared_prescription"
    bi = jl("Q02_bulk_identity") or {}
    c["bulk_rows_all_classified"] = bi.get("n_rows") == 23 and bi.get("unclassified_rows") == [] and \
        all(v.get("outcome") in ("zero", "cancellable") for v in bi.get("rows", {}).values())
    c["bulk_outcome_and_closure"] = bi.get("bulk_identity_outcome") == "cancellable" and bi.get("sigma0_closure", {}).get("closed") is True \
        and bi.get("sigma0_closure", {}).get("renormalised_defect_zero") is True
    c["ST4a_pole_ct_cancel"] = bi.get("ST4a", {}).get("cbc", {}).get("SE_pole_plus_p22_CT_zero") is True and \
        bi.get("ST4a", {}).get("Kc", {}).get("E2_pole_plus_CT_zero") is True
    c["vertex_checks"] = bi.get("vertex_checks", {}).get("phi_parity_even_all") is True and \
        bi.get("vertex_checks", {}).get("aux_b_cbs_bs_on_no_interaction_vertex") is True
    o1 = jl("Q02_O1") or {}
    items = o1.get("items", [])
```

```
    c["O1_walk_consistent"] = len(items) == 6 and o1.get("O1_closure") == (
        "CLOSED_PENDING_INDEPENDENT_REVIEW" if all(i.get("earned") is True for i in items) else "NOT_CLOSED") \
        and o1.get("all_earned") == all(i.get("earned") is True for i in items) and o1.get("O1_closure_confirmed") is False
    c["O1_enlarge_rule1_not_triggered_and_XVIM_unused"] = list(o1.get("enlarge_rules", {}).values())[0].get("triggered") is False \
        and list(o1.get("enlarge_rules", {}).values())[2].get("respected") is True and len(o1.get("closure_reasons_answered", {})) == 8
    sc = jl("Q02_D04_scrutiny") or {}
    cs = sc.get("cases", {})
    c["D04_scrutiny"] = sc.get("declared_zero") is True and sc.get("all_mutations_nonzero") is True and \
        sc.get("T3_zero_by_Ward_identity") is True and sc.get("N_DRED_equals_2p2(1-p)^2") is True and \
        sc.get("xi_dependence_linear_only (Froeb: m=0 xi^2 terms vanish)") is True and \
        cs.get("MUT non-de Donder gauge T4 piece (beta != -2), g4=1", {}).get("value_over_master") == "pp**2*(D - 1)"
    ctx = bi.get("bulk_counterterms_explicit", {})
    c["bulk_CT_explicit_in_domain"] = ctx.get("ok") is True and ctx.get("n_directions") == 11 and \
        ctx.get("coefficients", {}).get("phi.phi#0") == "0" and ctx.get("coefficients", {}).get("c.cb#0") == "-3/8"
    en = bi.get("independent_graph_enumeration", {})
    c["independent_enumeration_9_3"] = en.get("n_E2") == 9 and en.get("n_E1") == 3
    g = jl("Q01_guard") or {}
    c["guard_18_forms_all_raise"] = g.get("n_forms") == 18 and g.get("missed") == [] and "apart_residue" in g.get("forms", [])
    er_file = json.loads((Q / "intake" / "errata_p23.json").read_text())["errata"]
    c["errata_file_equals_certified"] = er_file == it.get("errata")
    er = {e["id"]: e for e in er_file}
    c["errata_E1_E7_severities"] = list(er) == ["E1", "E2", "E3", "E4", "E5", "E6", "E7"] and er["E2"]["severity"] == "low" \
        and "p23_conventions" in er["E4"]["correction"]
    reg = json.loads((Q / "registration" / "local_registration.json").read_text())
    c["registration_corrected"] = "XVI-A..XVI-I" in reg["how_found"] and "XVI-O has no registration flag" in reg["how_found"]
    c["xvin_label_note"] = "r12" in imp["files"]["provenance/XVI-N/RELEASE_STATUS_r10.json"].get("label_note", "")
    q1 = jl("Q01_release") or {}
    c["Q01_layer_accepted"] = q1.get("publication") == "accepted"
    return c, o1, bi


def main():
    try:
        c, o1, bi = checks()
    except Exception as exc:  # noqa: BLE001
        c, o1, bi = {"error": f"{type(exc).__name__}: {exc}"}, {}, {}
    ok = all(v is True for k, v in c.items() if not k.startswith("_"))
    for s in STATUS:
        s.unlink(missing_ok=True)
    closure = o1.get("O1_closure", "NOT_CLOSED") if ok else "NOT_CLOSED"
    rec = {"landmark": "XVI-Q", "release": "Q02", "version_label": "XVI-Q Q02", "date": "2026-10-10",
           "execution_pass": ok, "publication": "accepted" if ok else "REJECTED", "value_checks": c,
           "O1_closure": closure, "O1_closure_confirmed": False,
           "O1_closure_note": "pending independent review: becomes confirmed only after the Q02 independent rebuild passes",
           "O1_closure_scope": o1.get("scope"), "O1_not_implied": o1.get("not_implied"),
           "O1_checklist": [{k: i[k] for k in ("item", "owner", "earned", "evidence")} for i in o1.get("items", [])],
           "bulk_identity_outcome": bi.get("bulk_identity_outcome"), "bulk_symmetry_restored": False, "symmetry_restored": False,
           "flags": {"bulk_O5_graphs_evaluated": "earned" if ok else "not_earned",
                     "D12_reconciled": "earned" if ok else "not_earned",
                     "bulk_identity_verified_two_point_kappa2": "earned" if ok else "not_earned",
                     "bulk_counterterms_solved": "earned" if ok else "not_earned",
                     "sigma0_closure_bulk": "earned" if ok else "not_earned",
                     "O2_kappa2_master_constructed": "earned (Q01)", "d_guard_q02_verified": "earned" if ok else "not_earned",
                     "author_rebuild_passed": "see reproducibility", "independent_rebuild_passed": "pending",
                     "independent_rebuild_passed_q01": True},
           "hard_stops": HARD_STOPS, "XVI_Q_registered": "locally; not published"}
    cr = CERT / "Q02_release.json"
    if ok:
        txt = json.dumps(rec, indent=1) + "\n"
        for s in STATUS:
            s.write_text(txt)
        cr.write_text(txt)
        (CERT / "Q02_release_rejected.json").unlink(missing_ok=True)
        (Q / "ledgers").mkdir(exist_ok=True)
        (Q / "ledgers" / "Q02_milestones.json").write_text(json.dumps(
            {"landmark": "XVI-Q", "release": "Q02", "O1_closure": closure, "O1_checklist": rec["O1_checklist"],
             "flags": rec["flags"], "hard_stops": HARD_STOPS}, indent=1) + "\n")
    else:
        cr.unlink(missing_ok=True)
        (CERT / "Q02_release_rejected.json").write_text(json.dumps(rec, indent=1, default=str) + "\n")
    print(json.dumps({"publication": rec["publication"], "O1_closure": closure,
                      "failed": [k for k, v in c.items() if not k.startswith("_") and v is not True],
                      "mismatches": c.get("_value_mismatches")}, default=str)[:1500])
    return 0 if ok else 1


if __name__ == "__main__":
    raise SystemExit(main())
```

code/q03/q03_certs.py

```
#!/usr/bin/env python3
"""XVI-Q Q03 correction layer (no new science).  Writes
```

```
  certificates/Q03_O1_record.json -- the O.1 closure record: CLOSED (scoped), confirmed, bound to the Q02 independent
      review (report sha256 + reviewed source zip/tree); item-5 evidence with the ENLARGED counterterm domain (p14r 11 4D
      directions + p22 evanescent operators); prescription statement (XVI-P DRED, not XVI-O's o70-sync D11 kernel
      prescription; closure reason 8 NOT marked answered); scope verbatim; what it does not imply; caveats.
  certificates/Q03_guard_ring.json -- the PolyRing/FracField-with-d-generator bypass forms (Q02 review), each must raise.
Q02 certificates are a frozen layer and are not rewritten."""
from __future__ import annotations
import hashlib, json, sys  # noqa: E401
from pathlib import Path

Q = Path(__file__).resolve().parents[2]
CERT = Q / "certificates"
IND = Q / "reproducibility" / "independent_rebuild_report_q02.json"
BOUND = {"report_sha256": "e4b54702bc768899f22a00a289de0c0aeb2bf9317b87a28c4a462f05ae4bf495",
         "reviewed_zip_sha256": "378651a3a279cde0a8fd67d4e39ac5833d5a94ff00f10f62a001be8ab2b244dc",
         "reviewed_tree_sha256": "4b551b9801cd14b8f2b81775942d535ab282a75c1557148a94a91a9dad88a810",
         "decision": "CONFIRM CLOSED (scoped)"}
NOT_IMPLIED = [
    "the functional / all-n-point ST4b identity (only the retained two-point projection is verified)",
    "bulk symmetry restoration (bulk_symmetry_restored stays false)",
    "any change to XVI-O's own O.1 ledger: XVI-O is read-only and its O.1 stays NOT_CLOSED there",
    "the three-point sector / P.6 (kappa^3) and higher orders or loops",
    "the insertion (frozen; XVI-N massless obstruction untouched)",
    "finite renormalised amplitudes with non-QdS (epsilon-scalar) external components (p22 Q4S-trace O(eps)x pole remnant stays disclosed)"]
CAVEATS = [
    "same-box reviews: Q01/Q02 independent rebuilds ran on the author's machine (shared Python/sympy pin and TeX Live); not an independent machine",
    "D01/D03 pole values were never recomputed from independent Feynman rules in any review; cross-checked only via the exact cancellation of the c.h
defect in d",
    "full sigma_0 closure and the c.h defect were not re-derived in the Q02 review; they rely on the earlier p19-p22 independent checks on byte-
identical inputs",
    "D06 accepted via exact equality with XVI-P P15 Sigma (ghost equation), not via the reviewer's own Feynman rules"]


def jl(p):
    return json.loads(Path(p).read_text())


def guard_ring():
    sys.path.insert(0, str(Q / "code" / "q01"))
    import qguard as G
    import sympy as sp
    from sympy import QQ
    from sympy.polys.rings import ring, sring
    from sympy.polys.fields import field
    d, X = G.d, sp.Symbol("X")
    R, D_, x = ring([d, X], QQ)
    F, Df, xf = field([d, X], QQ)
    forms = {"ring_evaluate": lambda: (D_**2 - D_ + x).evaluate(D_, 4), "ring_subs": lambda: (D_**2 - D_).subs(D_, 4),
             "ring_call": lambda: (D_**2 - D_)(4), "ring_compose": lambda: (D_**2).compose(D_, R(4)),
             "field_evaluate": lambda: ((Df - 1) / (Df - 4 + xf)).evaluate(Df, 5),
             "field_subs": lambda: ((Df - 1) / Df).subs(Df, 4), "field_call": lambda: ((Df - 1) / Df)(4),
             "sring_evaluate": lambda: (lambda Rp: Rp[1].evaluate(Rp[0].gens[0], 4))(sring(d**2 - d, d))}
    res = {}
    for k, f in forms.items():
        try:
            f()
            res[k] = "BYPASS"
        except G.PrematureD4Substitution:
            res[k] = "raises"
    R2, x2 = ring([X], QQ)
    ok = {"d_free_ring_evaluate": str((x2**2).evaluate(x2, 3)), "cancel": str(sp.cancel((d**2 - 16) / (d - 4))),
          "factor": str(sp.factor(d**2 - d)), "together": str(sp.together(1 / d + 1 / X))}
    return {"forms": res, "n_forms": len(res), "missed": [k for k, v in res.items() if v != "raises"],
            "legitimate_algebra": ok, "entry_points": G.ENTRY_POINTS,
            "residual_disclosed": "raw dense-list evaluation (densetools dup_eval/dmp_eval_in on lists carrying no symbol "
                                  "information) cannot be attributed to d and is not guarded; no package code uses it"}


def record():
    o1 = jl(CERT / "Q02_O1.json")
    bi = jl(CERT / "Q02_bulk_identity.json")
    ct = bi["bulk_counterterms_explicit"]
    ev = ct["evanescent_p22"]
    rep = jl(IND)
    v = rep["O1_verdict"]
    rep_sha = hashlib.sha256(IND.read_bytes()).hexdigest()
    binding_ok = rep_sha == BOUND["report_sha256"] and v["decision"] == BOUND["decision"] and \
        rep["source"]["zip_sha256"] == BOUND["reviewed_zip_sha256"] and rep["source"]["tree_sha256"] == BOUND["reviewed_tree_sha256"]
    ev_ops = ["K'.c E2 (evanescent)", "cbar.c SE (evanescent)"] + [f"h.h evanescent: {k}" for k in sorted(ev["hh_sum"])]
    domain = {"p14r_4D_directions": sorted(ct["coefficients"]), "n_p14r_4D": ct["n_directions"],
              "p22_evanescent_operators": ev_ops, "n_p22_evanescent": len(ev_ops),
              "n_enlarged_domain": ct["n_directions"] + len(ev_ops),
              "p22_evanescent_coefficients": {"Kc_E2": ev["Kc_E2"], "ccb_SE": ev["ccb_SE"], "hh_sum": ev["hh_sum"]},
              "statement": "The counterterm domain used for O.1 item 5 is the ENLARGED domain: the 11 p14r 4D directions "
```

```
                        "(over Q(i)) PLUS the p22 evanescent operators (disclosed p22 contract revision, outside p14r). "
                        "With the declared QdS external momenta the local divergent part contains evanescent poles that "
                        "are cancelled only by the p22 evanescent operators; 'ST4a in the declared p14r domain' alone "
                        "holds only for 4D externals. sigma_0 closure including the evanescent operators: verified "
                        "(p20/p22 independent reviews)."}
    items = []
    for i in o1["items"]:
        j = {k: i[k] for k in ("item", "owner", "XVI_O_status", "evidence", "earned")}
        if i["item"] == "Demonstrate identity remainder cancellation / CT solution":
            j["evidence"] = ("ST4a: MS-bar CT in the ENLARGED domain = 11 p14r 4D directions over Q(i) (phi.phi = 0 earned) "
                             "PLUS the p22 evanescent operators (K'.c E2, cbar.c SE, 8 h.h evanescent structures), which alone "
                             "cancel the evanescent poles with d-dimensional (QdS) externals; ST4b: c.h defect ZERO exactly in d "
                             "(P19, x = 0), c.b = ghost equation (Sigma == D06), c.cbar* unreachable, c.phi parity; full "
                             "sigma_0 closure of the complete MS-bar CT incl. the evanescent operators in every gh+1 sector "
                             "(p20/p22); cohomology of the domain 0; no free-EOM equivalence used")
            j["evidence_corrected_in"] = "Q03 (QO2 review finding ST4a_domain_wording, medium)"
        items.append(j)
    reasons = {}
    for k, val in o1["closure_reasons_answered"].items():
        reasons[k] = {"status": "ANSWERED", "answer": val.removeprefix("ANSWERED: ")}
    r7, r8 = list(o1["closure_reasons_answered"])[6], list(o1["closure_reasons_answered"])[7]
    reasons[r7]["answer"] += " Local matching uses the ENLARGED domain (p14r 4D + p22 evanescent)."
    reasons[r8] = {"status": "NOT_ANSWERED_IN_XVI_O_TERMS (superseded prescription)",
                   "answer": "XVI-O's o70-sync D11 matter-loop kernel counterterm prescription is NOT the one used and is "
                             "not completed: both XVI-O kernel continuations fail on evanescent external legs (p22 finding). "
                             "O.1 is closed under XVI-P's declared DRED prescription (QdS loop momenta, Q4S vertex indices, "
                             "MS-bar, Feynman +i0, p22 evanescent counterterm contract revision). The bulk sector (hh, phi phi, "
                             "cbar c, K'.c, one-point) is evaluated, not only the D11 kernel; this does not complete XVI-O's "
                             "kernel prescription."}
    allx = all(i["earned"] for i in items)
    closed = allx and binding_ok
    return {"landmark": "XVI-Q", "release": "Q03",
            "O1_closure": "CLOSED (scoped)" if closed else "NOT_CLOSED", "O1_closure_confirmed": closed,
            "confirmation": {**BOUND, "report_file": "reproducibility/independent_rebuild_report_q02.json",
                             "report_sha256_now": rep_sha, "binding_ok": binding_ok,
                             "reviewer_conditions_for_the_label": v["conditions_for_the_label"],
                             "conditions_discharged_by": ["counterterm_domain (enlarged)", "prescription"]},
            "scope_verbatim_from_independent_review": v["scope"],
            "prescription": {"used": "XVI-P declared prescription: DRED (QdS loop momenta, Q4S vertex indices, opaque d via "
                                     "qguard), MS-bar, Feynman +i0, p22 evanescent counterterms; m = 0; frozen gauge fermion",
                             "not_used": "XVI-O o70-sync D11 kernel counterterm prescription (closure reason 8): superseded "
                                         "on evanescent legs, both kernel continuations fail there (p22); reason 8 is NOT "
                                         "marked answered"},
            "counterterm_domain": domain, "items": items, "all_earned": allx, "closure_reasons": reasons,
            "enlarge_rules": o1["enlarge_rules"], "downgrade_rules": o1["downgrade_rules"],
            "not_implied": NOT_IMPLIED, "caveats": CAVEATS,
            "XVI_O_ledger": "read-only; O.1 there stays NOT_CLOSED (unchanged)"}


def main():
    (CERT / "Q03_O1_record.json").write_text(json.dumps(record(), indent=1, sort_keys=False) + "\n")
    (CERT / "Q03_guard_ring.json").write_text(json.dumps(guard_ring(), indent=1) + "\n")
    print("written Q03_O1_record.json, Q03_guard_ring.json")


if __name__ == "__main__":
    main()
```

code/q03/release_q03.py

```
#!/usr/bin/env python3
"""XVI-Q Q03 release gate (fails closed; on reject no RELEASE_STATUS.json is left).
(1) code/q03/q03_expected.json is pinned (EXPECTED_SHA256) and EVERY Q01/Q02/Q03 certificate equals its pinned copy;
(2) RE-HASH at gate time: every pinned provenance file in intake/imports.json (no unpinned provenance file allowed),
    the bound Q02 independent report, and every pinned code/conventions file;
(3) every file the gate reads is either in the source set (tools/source_set.py) or a build output it re-checks;
(4) semantic checks of the O.1 record (enlarged domain, prescription, reason 8 not answered, binding, scope, not-implied,
    caveats) and of the ring/field guard forms; the Q01 and Q02 layer gates accepted."""
from __future__ import annotations
import hashlib, json, sys  # noqa: E401
from pathlib import Path

Q = Path(__file__).resolve().parents[2]
sys.path.insert(0, str(Q / "tools"))
import source_set as SS  # noqa: E402
CERT = Q / "certificates"
EXPECTED_FILE = Q / "code" / "q03" / "q03_expected.json"
EXPECTED_SHA256 = "44cdd33648bd8ba42267fbd70f28a15d6c32dd988b64be8170204760e23aa9b0"
STATUS = (Q / "RELEASE_STATUS.json", CERT / "RELEASE_STATUS.json")
if "--layer" in sys.argv:   # Q04: earlier-layer gate; RELEASE_STATUS is owned by the Q04 gate
    STATUS = ()
GENERATED = ("certificates/", "ledgers/", "build/")       # build outputs: checked by value / re-hash, not source
HARD_STOPS = {"bulk_symmetry_restored": False, "insertion": "frozen", "XV_B_opened": False, "XIV_J_opened": False,
```

```
            "Route_B_discharged": False, "Missing_never_zero": True}
READS: set[str] = set()


def rd(rel):
    READS.add(rel)
    return (Q / rel).read_bytes()


def jr(rel):
    return json.loads(rd(rel))


def sha_b(b):
    return hashlib.sha256(b).hexdigest()


def strip(o):
    if isinstance(o, dict):
        return {k: strip(v) for k, v in o.items() if k != "seconds"}
    if isinstance(o, list):
        return [strip(v) for v in o]
    return o


def checks():
    c = {}
    eb = rd("code/q03/q03_expected.json") if EXPECTED_FILE.is_file() else b""
    c["expected_file_pinned"] = bool(eb) and sha_b(eb) == EXPECTED_SHA256
    exp = json.loads(eb) if eb else {}
    bad = []
    for name, ref in exp.get("certificates", {}).items():
        f = f"certificates/{name}.json"
        if not (Q / f).is_file() or strip(jr(f)) != ref:
            bad.append(name)
    c["every_certified_value_matches_pinned"] = len(exp.get("certificates", {})) >= 12 and not bad
    c["_value_mismatches"] = bad
    badf = [f for f, s in exp.get("file_sha256", {}).items() if not (Q / f).is_file() or sha_b(rd(f)) != s]
    c["code_and_conventions_rehashed"] = len(exp.get("file_sha256", {})) >= 8 and not badf
    c["_file_mismatches"] = badf
    imp = jr("intake/imports.json")
    badp = [f for f, m in imp["files"].items() if not (Q / f).is_file() or sha_b(rd(f)) != m["sha256"]]
    unpinned = sorted(p.relative_to(Q).as_posix() for p in (Q / "provenance").rglob("*") if p.is_file()
                      and p.relative_to(Q).as_posix() not in imp["files"])
    c["provenance_rehashed_all_pinned"] = len(imp["files"]) == 33 and not badp and not unpinned
    c["_provenance_mismatches"] = badp + [f"unpinned:{u}" for u in unpinned]
    rec = jr("certificates/Q03_01_record.json")
    ind = "reproducibility/independent_rebuild_report_q02.json"
    rb = rd(ind)
    cf = rec["confirmation"]
    c["bound_independent_report_rehashed"] = sha_b(rb) == cf["report_sha256"] == "e4b54702bc768899f22a00a289de0c0aeb2bf9317b87a28c4a462f05ae4bf495" \
        and json.loads(rb)["01_verdict"]["decision"] == "CONFIRM CLOSED (scoped)" and cf["binding_ok"] is True \
        and json.loads(rb)["source"]["tree_sha256"] == cf["reviewed_tree_sha256"]
    dom = rec["counterterm_domain"]
    c["item5_enlarged_domain"] = dom["n_p14r_4D"] == 11 and dom["n_p22_evanescent"] == 10 and dom["n_enlarged_domain"] == 21 \
        and dom["p22_evanescent_coefficients"]["Kc_E2"] == "-1/16" and dom["p22_evanescent_coefficients"]["ccb_SE"] == "1/8" \
        and any("ENLARGED domain" in i["evidence"] and "p22 evanescent" in i["evidence"] for i in rec["items"]
                if i["item"] == "Demonstrate identity remainder cancellation / CT solution")
    r = rec["closure_reasons"]
    r8 = list(r.values())[7]
    c["reason8_not_answered_prescription_stated"] = len(r) == 8 and not r8["status"].startswith("ANSWERED") \
        and "XVI-P" in rec["prescription"]["used"] and "o70-sync" in rec["prescription"]["not_used"] \
        and all(v["status"] == "ANSWERED" for v in list(r.values())[:7])
    c["scope_verbatim"] = rec["scope_verbatim_from_independent_review"] == json.loads(rb)["01_verdict"]["scope"]
    ni = " | ".join(rec["not_implied"])
    c["not_implied_complete"] = all(s in ni for s in ("ST4b", "bulk symmetry restoration", "XVI-O", "NOT_CLOSED", "P.6", "insertion"))
    cv = " | ".join(rec["caveats"])
    c["caveats_recorded"] = all(s in cv for s in ("same-box", "D01/D03", "sigma_0 closure", "c.h defect"))
    c["01_closed_scoped_consistent"] = rec["01_closure"] == "CLOSED (scoped)" and rec["01_closure_confirmed"] is True \
        and rec["all_earned"] is True and len(rec["items"]) == 6 and all(i["earned"] is True for i in rec["items"])
    g = jr("certificates/Q03_guard_ring.json")
    c["guard_ring_field_forms_all_raise"] = g["n_forms"] == 8 and g["missed"] == []
    g1 = jr("certificates/Q01_guard.json")
    c["guard_18_forms_still_raise"] = g1["n_forms"] == 18 and g1["missed"] == []
    c["Q01_layer_accepted"] = jr("certificates/Q01_release.json").get("publication") == "accepted"
    q2 = jr("certificates/Q02_release.json")
    c["Q02_layer_accepted"] = q2.get("publication") == "accepted" and q2.get("bulk_symmetry_restored") is False
    for f in ("intake/errata_p23.json", "registration/local_registration.json", "code/q02/q02_expected.json"):
        rd(f)
    src = set(SS.source_files(Q))
    outside = sorted(f for f in READS if f not in src and not f.startswith(GENERATED))
    c["every_gate_read_in_source_set"] = not outside and len(READS) > 40
    c["_reads_outside_source_set"] = outside
    return c, rec
```

```
def main():
    try:
        c, rec = checks()
    except Exception as exc:  # noqa: BLE001
        c, rec = {"error": f"{type(exc).__name__}: {exc}"}, {}
    ok = all(v is True for k, v in c.items() if not k.startswith("_"))
    for s in STATUS:
        s.unlink(missing_ok=True)
    closure = rec.get("O1_closure", "NOT_CLOSED") if ok else "NOT_CLOSED"
    out = {"landmark": "XVI-Q", "release": "Q03", "version_label": "XVI-Q Q03", "date": "2026-10-10",
           "kind": "correction release (no new science)", "execution_pass": ok,
           "publication": "accepted" if ok else "REJECTED", "value_checks": c,
           "O1_closure": closure, "O1_closure_confirmed": bool(ok and rec.get("O1_closure_confirmed")),
           "O1_confirmation": rec.get("confirmation"), "O1_closure_scope": rec.get("scope_verbatim_from_independent_review"),
           "O1_prescription": rec.get("prescription"), "O1_counterterm_domain": rec.get("counterterm_domain", {}).get("statement"),
           "O1_not_implied": rec.get("not_implied"), "O1_caveats": rec.get("caveats"),
           "O1_checklist": [{k: i[k] for k in ("item", "owner", "earned", "evidence")} for i in rec.get("items", [])],
           "XVI_0_ledger_O1": "NOT_CLOSED (read-only, unchanged)", "bulk_symmetry_restored": False, "symmetry_restored": False,
           "flags": {"O1_closed_scoped": "earned + independently confirmed (Q02 review)" if ok else "not_earned",
                     "O2_kappa2_master_constructed": "earned (Q01)", "bulk_identity_verified_two_point_kappa2": "earned (Q02)",
                     "d_guard_q03_ring_field": "earned" if ok else "not_earned",
                     "independent_rebuild_passed_q01": True, "independent_rebuild_passed_q02": True,
                     "independent_rebuild_passed_q03": "pending", "author_rebuild_passed": "see reproducibility"},
           "hard_stops": HARD_STOPS, "XVI_Q_registered": "locally; not published"}
    cr = CERT / "Q03_release.json"
    if ok:
        txt = json.dumps(out, indent=1) + "\n"
        for s in STATUS:
            s.write_text(txt)
        cr.write_text(txt)
        (CERT / "Q03_release_rejected.json").unlink(missing_ok=True)
        (Q / "ledgers" / "Q03_milestones.json").write_text(json.dumps(
            {"landmark": "XVI-Q", "release": "Q03", "O1_closure": closure, "O1_closure_confirmed": out["O1_closure_confirmed"],
             "flags": out["flags"], "hard_stops": HARD_STOPS}, indent=1) + "\n")
    else:
        cr.unlink(missing_ok=True)
        (CERT / "Q03_release_rejected.json").write_text(json.dumps(out, indent=1, default=str) + "\n")
    print(json.dumps({"publication": out["publication"], "O1_closure": closure,
                      "failed": [k for k, v in c.items() if not k.startswith("_") and v is not True],
                      "mm": {k: v for k, v in c.items() if k.startswith("_") and v}}, default=str)[:1500])
    return 0 if ok else 1


if __name__ == "__main__":
    raise SystemExit(main())
```

code/q04/guard_scan.py

```
#!/usr/bin/env python3
"""XVI-Q Q04 d-guard: the 'leave sympy symbolics' bypass class (Q03 review finding guard_rename_class, low).

Runtime probes (recorded, NOT closable at runtime): once an expression is turned into Python data -- a pure-Python walk
over its terms with d replaced by 4, a str()/srepr() of it renamed and re-parsed, or raw dense coefficient lists
(densetools dup_eval/dmp_eval_in) -- d's value can be computed by code that never touches a sympy entry point.  No
runtime guard can see that.  It is therefore closed by POLICY + STATIC SCAN of the package's own code (code/q0*/*.py):
none of the escape primitives may appear there.  The gate requires the scan to be clean."""
from __future__ import annotations
import ast, sys  # noqa: E401
from pathlib import Path

Q = Path(__file__).resolve().parents[2]
FORBIDDEN_CALLS = {"srepr", "parse_expr", "dup_eval", "dmp_eval", "dmp_eval_in", "dmp_eval_tail", "dup_eval_in",
                   "as_coefficients_dict", "lambdify", "evalf", "N"}
SELF = {"code/q04/guard_scan.py", "code/q01/guard_probe.py"}   # probe files: call the primitives to show they raise / bypass


def scan():
    hits = []
    for p in sorted((Q / "code").glob("q0*/*.py")):
        rel = p.relative_to(Q).as_posix()
        if rel in SELF:
            continue
        tree = ast.parse(p.read_text())
        for n in ast.walk(tree):
            if isinstance(n, ast.Call):
                f = n.func
                name = f.attr if isinstance(f, ast.Attribute) else f.id if isinstance(f, ast.Name) else None
                if name in FORBIDDEN_CALLS:
                    hits.append(f"{rel}:{n.lineno}: {name}()")
                # str(...).replace(...) / repr(...).replace(...)  and  sympify(str(...)) / S(str(...))
                if name == "replace" and isinstance(f, ast.Attribute) and isinstance(f.value, ast.Call) and \
```

```
                    isinstance(f.value.func, ast.Name) and f.value.func.id in ("str", "repr"):
                hits.append(f"{rel}:{n.lineno}: str(...).replace")
            if name in ("sympify", "S") and n.args and isinstance(n.args[0], ast.Call) and \
                    isinstance(n.args[0].func, ast.Name) and n.args[0].func.id in ("str", "repr"):
                hits.append(f"{rel}:{n.lineno}: sympify(str(...))")
    return hits


def probes():
    """demonstrate the class on a probe expression (honest disclosure: these DO bypass the runtime guard)."""
    sys.path.insert(0, str(Q / "code" / "q01"))
    import qguard as G
    import sympy as sp
    d = G.d
    e = (d ** 2 - 16) / (d - 4) + 3 * d
    num, den = sp.fraction(sp.together(e))
    from fractions import Fraction

    def walk(x):                                    # pure-Python evaluation (Fractions), d -> 4: never re-enters sympy
        if isinstance(x, sp.Symbol) and x.name == d.name:     # identify d by NAME ('x == d' itself is refused by p22)
            return Fraction(4)
        if x.is_Integer or x.is_Rational:
            return Fraction(int(x.p), int(x.q))
        vals = [walk(a) for a in x.args]
        if x.is_Add:
            return sum(vals, Fraction(0))
        if x.is_Mul:
            out = Fraction(1)
            for v in vals:
                out *= v
            return out
        if x.is_Pow:
            return vals[0] ** int(vals[1])
        raise TypeError(type(x))
    res = {}
    try:
        _ = (d ** 2).args[0] == d
        res["python_term_walk_eq_d"] = "BYPASS"
    except G.PrematureD4Substitution:
        res["python_term_walk_eq_d"] = "raises (p22: equality test against d refused)"
    try:
        res["python_term_walk"] = f"BYPASS (value {walk(num)})"
    except G.PrematureD4Substitution:
        res["python_term_walk"] = "raises"
    try:
        v = sp.sympify(str(num).replace(str(d), "4"))
        res["str_rename_reparse"] = f"BYPASS (value {v})"
    except G.PrematureD4Substitution:
        res["str_rename_reparse"] = "raises"
    res["dense_list_eval"] = "BYPASS by construction (lists carry no symbol); disclosed since Q03"
    return res


def report():
    h = scan()
    return {"class": "leave-sympy (pure-Python term walk, str/srepr rename + re-parse, raw dense lists)",
            "runtime_closable": False,
            "why": "the computation happens outside every sympy entry point; a runtime guard cannot observe it",
            "probes": probes(), "policy": "package code (code/q0*/*.py) must not use any escape primitive",
            "forbidden": sorted(FORBIDDEN_CALLS) + ["str(...).replace", "sympify(str(...))"],
            "scan_hits": h, "scan_clean": not h,
            "n_files_scanned": len([p for p in (Q / "code").glob("q0*/*.py")]) - len(SELF)}


if __name__ == "__main__":
    import json
    print(json.dumps(report(), indent=1))
```

code/q04/p6_inventory.py

```
#!/usr/bin/env python3
"""XVI-Q Q04 P.6 entry (one-loop kappa^3): independent graph enumeration incl. kappa^3 vertices, power counting,
kappa^3 vertex derivation (V_hhh_phi_phi exact, with an independent determinant/inverse route and a mutation control),
kappa^3 identity ledger (which (S_k, Gamma_1^[3-k]) enter; gh1 three-field rows) and the kappa^3 three-point local
counterterm-domain dimensions over Q(i) (4D vs evanescent), for the hhh and h phi phi sectors."""
from __future__ import annotations
import itertools, json, sys  # noqa: E401
from pathlib import Path
import sympy as sp

Q = Path(__file__).resolve().parents[2]
PV = Q / "provenance" / "XVI-P" / "certificates"
sys.path.insert(0, str(Q / "code" / "q02"))
```

```
from bulk_identity import FIELDS  # noqa: E402

PAIRS = {("h", "h"), ("phi", "phi"), ("c", "cb"), ("cb", "c")}
KAPPA3_VERTICES = {"V5_grav": ["h"] * 5, "V_hhh_phi_phi": ["h", "h", "h", "phi", "phi"]}
EXACT_IN_KAPPA = {
    "V_cb_c_h": "ghost action = cbar . d(s h) with Psi linear in h (XVI-P conventions PSI) and s h = (1/2) L_c(eta + kappa h) "
                "affine in h (GHOST_SCALING): no cbar c h^n vertex for n >= 2",
    "V_K_c_h": "K s h, s h affine in h: no K c h^n, n >= 2", "V_cs_c_c": "c* s c, s c = kappa a c.dc: exact at kappa^1",
    "V_phis_c_phi": "phi* s phi, s phi = kappa a c.dphi: exact at kappa^1",
    "gauge_fixing": "Psi linear in h -> b/h sector quadratic, no vertices"}


def jl(p):
    return json.loads(Path(p).read_text())


def vertices():
    V = dict(jl(PV / "P14_counterterm_domain.json")["loop_capable_vertices"])
    V.update(KAPPA3_VERTICES)
    return V


def kappa(legs):
    return len(legs) - 2


def enumerate_one_loop(V, nmax=3):
    """all one-loop 1PI graphs = cycles of n vertices, each contributing an (in, out) pair of loop legs joined by
    propagating pairs.  Key: (sorted vertices, sorted line types, sorted externals)."""
    out = {}
    names = sorted(V)
    for n in range(1, nmax + 1):
        for combo in itertools.product(names, repeat=n):
            legsets = [V[v] for v in combo]
            for choice in itertools.product(*[list(itertools.permutations(range(len(l)), 2)) for l in legsets]):
                ok, lines = True, []
                for i in range(n):
                    a = legsets[i][choice[i][1]]                      # out-leg of vertex i
                    b = legsets[(i + 1) % n][choice[(i + 1) % n][0]]  # in-leg of the next vertex
                    if (a, b) not in PAIRS:
                        ok = False
                        break
                    lines.append("-".join(sorted((a, b))))
                if not ok:
                    continue
                ext = sorted(l for i in range(n) for k, l in enumerate(legsets[i]) if k not in choice[i])
                key = (tuple(sorted(combo)), tuple(sorted(lines)), tuple(ext))
                out[key] = sum(kappa(V[v]) for v in combo)
    return out


def inventory():
    V = vertices()
    g = enumerate_one_loop(V)
    by_E = {}
    for (vs, ls, ext), k in g.items():
        by_E.setdefault(len(ext), []).append({"vertices": list(vs), "lines": list(ls), "external": list(ext), "kappa": k})
    kappa_eq_E = all(x["kappa"] == E for E, lst in by_E.items() for x in lst)
    E3 = sorted(by_E.get(3, []), key=lambda x: (x["external"], x["vertices"], x["lines"]))
    tad = [x for x in E3 if len(x["vertices"]) == 1]
    loops = [x for x in E3 if len(x["vertices"]) > 1]
    p23 = jl(PV / "P23_O1_scope.json")["P6_entry_kappa3_three_point"]
    p23_keys = sorted((tuple(sorted(x["vertices"])), tuple(sorted("-".join(sorted(l.split("-"))) for l in x["loop_lines"])),
                       tuple(sorted(x["external"].split(",")))) for x in p23["graphs"])
    mine_keys = sorted((tuple(x["vertices"]), tuple(x["lines"]), tuple(x["external"])) for x in loops)
    by_ext = {}
    for x in E3:
        by_ext[",".join(x["external"])] = by_ext.get(",".join(x["external"]), 0) + 1
    for x in tad:
        x["status"] = "derived 0 (scaleless): massless self-contraction, the loop momentum flows through ONE propagator " \
                      "and the vertex is a polynomial in l and external momenta -> int d^dl l^{2k} / l^2 = 0 (XVI-P rule S0, no log member)"
    return {"vertices_used": V, "kappa3_vertices_new": sorted(KAPPA3_VERTICES), "exact_in_kappa_vertices": EXACT_IN_KAPPA,
            "power_counting": {"rule": "one loop: I = V, sum_v (legs_v - 2) = (2I + E) - 2V = E  => kappa power = E",
                               "verified_on_all_enumerated_graphs": kappa_eq_E,
                               "kappa3_one_point_one_loop": "empty (kappa^E with E = 1 is kappa^1)",
                               "kappa3_two_point_one_loop": "empty (E = 2 is kappa^2)",
                               "completeness": "each vertex of a one-loop E = 3 graph carries >= 1 external leg (3-leg vertices: "
                                               "exactly 1) -> at most 3 vertices; enumerated n = 1..3"},
            "n_by_E": {E: len(v) for E, v in sorted(by_E.items())},
            "E3_loops": loops, "E3_tadpoles": tad, "n_E3_loops": len(loops), "n_E3_tadpoles": len(tad),
            "by_external": by_ext,
            "equals_XVI_P_P23_15": mine_keys == p23_keys, "XVI_P_n": len(p23_keys)}
```

```
# ---------------- kappa^3 vertex V_hhh_phi_phi: cubic term of X = sqrt(-g) g^{mn}
def X3_formula(h, eta):
    """X3 = -h^3 + (1/2) t1 h^2 + (t1^2/8 - t2/4)(-h) + (t3/6 - t1 t2/8 + t1^3/48) eta^{-1}  (h with indices raised by eta)."""
    hu = eta.inv() * h * eta.inv()                      # h^{mn}
    m = eta.inv() * h                                   # h^m_n (mixed)
    t1, t2, t3 = m.trace(), (m * m).trace(), (m * m * m).trace()
    return sp.expand(-(m * m * m) * eta.inv() + sp.Rational(1, 2) * t1 * (m * m) * eta.inv()
                     - (t1 ** 2 / 8 - t2 / 4) * hu + (t3 / 6 - t1 * t2 / 8 + t1 ** 3 / 48) * eta.inv())


def vertex_hhhphiphi_check(mutate=False):
    """independent route: X(t) = sqrt(-det(eta + t h)) (eta + t h)^{-1}, exact series in t, t^3 coefficient, on two
    random rational symmetric h (4x4, signature +---); compared with X3_formula."""
    eta = sp.diag(1, -1, -1, -1)
    tt = sp.Symbol("tt")
    res = []
    for seed in (3, 11):
        import random
        rnd = random.Random(seed)
        h = sp.zeros(4, 4)
        for i in range(4):
            for j in range(i, 4):
                h[i, j] = h[j, i] = sp.Rational(rnd.randint(-7, 7), rnd.randint(1, 5))
        g = eta + tt * h
        det = sp.expand(-g.det())
        sq = sp.series(sp.sqrt(det), tt, 0, 4).removeO()
        ginv = sum((((-tt) ** k) * (eta.inv() * h) ** k for k in range(4)), sp.zeros(4, 4)) * eta.inv()
        X = (sq * ginv).applyfunc(lambda e: sp.expand(e).coeff(tt, 3))
        F = X3_formula(h, eta)
        if mutate:
            F = F + sp.Rational(1, 48) * (eta.inv() * h).trace() ** 3 * eta.inv()
        res.append(sp.simplify(X - F) == sp.zeros(4, 4))
    return all(res)


# ---------------- kappa^3 identity ledger
def identity_rows():
    names = list(FIELDS)
    rows = []
    for tri in itertools.combinations_with_replacement(names, 3):
        gh = sum(FIELDS[x][0] for x in tri)
        if gh != 1:
            continue
        if sum(FIELDS[x][1] for x in tri) % 2:
            rows.append({"row": ".".join(tri), "class": "zero", "why": "phi-parity odd (every vertex even in {phi, phi*})"})
            continue
        if {"b", "cbs", "bs"} & set(tri):
            rows.append({"row": ".".join(tri), "class": "aux", "why": "contains b / cbar* / b*: no interaction vertex "
                         "(ghost-equation / nonminimal rows; not a loop entry)"})
            continue
        rows.append({"row": ".".join(tri), "class": "needs_inputs"})
    needs = {
        "c.h.h": {"S0_Gamma3": ["Gamma_hhh (5 graphs)", "Gamma_K c h (2)"], "S1_Gamma2": ["Gamma_hh (Q02: D01, D02, D03, D11, D12)",
                  "Gamma_K c (D13)"], "matter_subsector": "scalar-loop part = matter Ward identity (evaluated, Q04)"},
        "c.phi.phi": {"S0_Gamma3": ["Gamma_h phi phi (4 graphs)", "Gamma_phi* c phi (1)"], "S1_Gamma2": ["Gamma_phi phi = 0 (Q02)",
                      "Gamma_hh not entering"]},
        "c.c.cb": {"S0_Gamma3": ["Gamma_cbar c h (2)", "Gamma_c c c* (1)"], "S1_Gamma2": ["Gamma_cbar c (D06)"]},
        "c.c.hs": {"S0_Gamma3": ["Gamma_K c h (2)", "Gamma_c c c* (1)"], "S1_Gamma2": ["Gamma_K c (D13)"]},
    }
    needs = {".".join(sorted(k.split("."))): v for k, v in needs.items()}
    for r in rows:
        if r["class"] == "needs_inputs":
            r["inputs"] = needs.get(".".join(sorted(r["row"].split("."))), {"S0_Gamma3": [], "S1_Gamma2": [], "note": "no E = 3 kappa^3 graph with
these "
                                                                                                                    "external legs (enumeration) -> row zero"})
            if ".".join(sorted(r["row"].split("."))) not in needs:
                r["class"] = "zero_no_graph"
    return {"identity": "(S, Gamma_1) = A_1 at kappa^3, one loop: (S_0, Gamma_1^[3]) + (S_1, Gamma_1^[2]) + (S_2, Gamma_1^[1]) + (S_3, Gamma_1^[0])",
            "terms": {"(S_0, Gamma_1^[3])": "three-point Gamma_1 with the linear BRST (s0 h = d c, s0 c = 0, s0 phi = 0, antifield "
                                            "equations of S_0) -- needs the P.6 graphs",
                      "(S_1, Gamma_1^[2])": "two-point Gamma_1 (Q02, evaluated) with the quadratic BRST (c.dh, h dc, c.dc, c.dphi)",
                      "(S_2, Gamma_1^[1])": "0: Gamma_1^[1] is one-point only and vanishes (D14a-c)",
                      "(S_3, Gamma_1^[0])": "0: field independent"},
            "rows": rows, "n_rows": len(rows)}


# ---------------- kappa^3 local counterterm domain dimensions (structural; before BRST reduction)
def ct_domain_dims():
    """local three-point polynomials, momentum degree 4 (one-loop kappa^3 counterterms have 4 derivatives: kappa^3 x
    mass^4 for a 3-field operator), multilinear (degree 2) in each graviton polarisation a_i, Bose symmetric, momentum
    conserved (p3 = -p1-p2); invariants H (hat), T (tilde, evanescent) as in uvpole.py.  Dimension over Q(i) of the
    symmetric span, and of its 4D part (T-free) -- evanescent = total - 4D."""
    p = ["p1", "p2"]
    def mono_space(pol, sym_group):
```

```
        syms = []
        vecs = pol + p
        for i, x in enumerate(vecs):
            for y in vecs[i:]:
                if x in p and y in p:
                    syms.append((("H", x, y), {x: 0, y: 0}))
                else:
                    syms.append((("H", x, y), None))
                    if x in pol and y in pol:
                        syms.append((("T", x, y), None))
        # monomials: product of invariants with degree 2 in each pol vector and momentum degree 4
        out = set()
        def rec(i, cur, degp, degm):
            if all(degp[a] == 2 for a in pol) and degm == 4:
                out.add(tuple(sorted(cur)))
                return
            if i == len(syms):
                return
            (k, x, y), _ = syms[i]
            for n in range(0, 5):
                nd = dict(degp)
                ok = True
                m = 0
                for z in (x, y):
                    if z in pol:
                        nd[z] += n
                    else:
                        m += n
                if any(nd[a] > 2 for a in pol) or degm + m > 4:
                    break
                rec(i + 1, cur + [(k, x, y)] * n, nd, degm + m)
        rec(0, [], {a: 0 for a in pol}, 0)
        return sorted(out)
    res = {}
    for name, pol in (("hhh", ["a1", "a2", "a3"]), ("h_phi_phi", ["a1"])):
        mons = mono_space(pol, None)
        res[name] = {"n_monomials_before_symmetry": len(mons),
                     "n_with_T (evanescent)": sum(1 for m in mons if any(k == "T" for k, _, _ in m))}
    res["status"] = ("structural count only (momentum conservation used through p1, p2; Bose symmetrisation and the "
                     "BRST/sigma_0 reduction to the invariant domain NOT done in Q04)")
    return res
```

code/q04/q04_certs.py

```
#!/usr/bin/env python3
"""XVI-Q Q04 certificates (P.6 entry, one-loop kappa^3; honest partial):
  Q04_P6_inventory.json  independent enumeration (15 loop graphs == XVI-P P23 + 3 kappa^3 tadpoles), power counting,
                         kappa^3 vertices (V_hhh_phi_phi exact: two routes + mutation; V5_grav enters only a scaleless tadpole)
  Q04_P6_identity.json   kappa^3 identity ledger: terms, gh1 three-field rows, inputs per row, CT-domain structural counts
  Q04_P6_eval.json       own UV-pole engine validated against XVI-P D11 (all 14 structures incl. evanescent) + mutation;
                         scalar-loop hhh graphs G14/G15 (UV pole residue, exact); matter Ward identity at three points
                         (pole residue) + mutations; graph table with every other graph explicitly Missing
  Q04_P6_status.json     outcome per stage; defect/CT/closure status"""
from __future__ import annotations
import hashlib, json, sys, time  # noqa: E401
from concurrent.futures import ProcessPoolExecutor
from pathlib import Path

Q = Path(__file__).resolve().parents[2]
sys.path.insert(0, str(Q / "code" / "q04"))
CERT = Q / "certificates"


def _sub4(e, why):
    import dred
    import sympy as sp
    return sp.expand(dred.substitute_d(e, 4, why))


def task_d11(trace_coef="1/2"):
    import sympy as sp
    import uvpole as U
    U.TRACE_COEF = sp.Rational(trace_coef)
    p, l = U.vec("p1"), U.vec("l")
    e1, e2 = U.rank1("a1"), U.rank1("a2")
    G2 = U.loop_pole(U.V_hphiphi(e1, l, -(l + p)) * U.V_hphiphi(e2, l + p, -l), [U.V(), p], 4, sp.Rational(1, 2))
    mine = _sub4(G2, "Q04 D11 validation: MS-bar residue")
    r = json.loads((Q / "provenance/XVI-P/certificates/P18_D11.json").read_text())["result"]
    H, T = U.H, U.T
    idx = {"m": 1, "n": 1, "r": 2, "s": 2}
    tot = 0
    for k, v in r.items():
        parts = k.split(" ")
```

```
        e = int(parts[-1][-1]) if parts[-1].startswith("(p.p)") else 0
        parts = [f for f in parts if not f.startswith("(p.p)")]
        val = H("p1", "p1") ** e
        orbit = set()
        for s1 in (0, 1):
            for s2 in (0, 1):
                mp = {"m": "nm"[s1], "n": "mn"[s1], "r": "sr"[s2], "s": "rs"[s2]}
                ren = []
                for f in parts:
                    nm, ix = f.split("_")
                    nix = "".join(mp[c] for c in ix)
                    ren.append(nm + "_" + ("".join(sorted(nix)) if nm in ("g4", "gt") else nix))
                orbit.add(tuple(sorted(ren)))
        for f in parts:
            nm, ix = f.split("_")
            a, b = (f"a{idx[ix[0]]}", f"a{idx[ix[1]]}") if len(ix) == 2 else (f"a{idx[ix]}", None)
            val *= {"g4": lambda: H(a, b) + T(a, b), "gt": lambda: T(a, b), "p": lambda: H(a, "p1")}[nm]()
        tot += len(orbit) * sp.sympify(v["UV_pole"]) * val
    ratio = sp.cancel(mine / sp.expand(tot))
    return {"trace_coef": trace_coef, "ratio_mine_over_XVI_P": str(ratio), "constant_ratio": not ratio.free_symbols,
            "n_structures": len(r), "gt_reading": "tilde (evanescent Q(4-d)S metric); g4 = hat + tilde; XVI-P 'upper' "
            "structures weighted by their (m<->n, r<->s) orbit size"}


def task_ward(lie_sign=1, drop_bubbles=False):
    import sympy as sp
    import scalar_loop3 as S
    w, g3 = S.ward(drop_bubbles=drop_bubbles, lie_sign=lie_sign)
    w4 = _sub4(w, "Q04 matter Ward identity: pole residue")
    Ts = [s for s in w.free_symbols if s.name.startswith("T_")]
    return {"lie_sign": lie_sign, "drop_bubbles": drop_bubbles,
            "residue_d4_zero": w4 == 0, "residue_d4_4D_part_zero": sp.expand(w4.subs({s: 0 for s in Ts})) == 0,
            "n_terms_residue_d4": 0 if w4 == 0 else len(sp.Add.make_args(w4)),
            "coefficient_exact_in_d_zero": w == 0, "n_terms_generic_d": len(sp.Add.make_args(w)) if w != 0 else 0}


def task_gamma3():
    import sympy as sp
    import scalar_loop3 as S
    import uvpole as U
    p1, p2 = U.vec("p1"), U.vec("p2")
    g3, parts = S.gamma3([(U.rank1("a1"), p1), (U.rank1("a2"), p2), (U.rank1("a3"), -(p1 + p2))])
    g4 = _sub4(g3, "Q04 scalar-loop hhh: MS-bar residue")
    Ts = [s for s in g4.free_symbols if s.name.startswith("T_")]
    terms = sorted(str(x) for x in sp.Add.make_args(g4))
    per = {k: len(sp.Add.make_args(_sub4(v, "Q04 per-graph residue"))) for k, v in parts.items()}
    return {"residue_d4_n_terms": len(terms), "residue_d4_sha256": hashlib.sha256("\n".join(terms).encode()).hexdigest(),
            "residue_d4_n_evanescent_terms": sum(1 for x in sp.Add.make_args(g4) if x.free_symbols & set(Ts)),
            "residue_d4": str(g4), "per_part_n_terms": per,
            "bose_symmetric_a1_a2": _bose(g4)}


def _bose(g4):
    """swap (a1, p1) <-> (a2, p2) (leg 3 fixed, p3 = -p1-p2 invariant), re-canonicalising H/T symbol names."""
    import sympy as sp
    sw = {"a1": "a2", "a2": "a1", "p1": "p2", "p2": "p1"}
    rep = {}
    for sym in g4.free_symbols:
        k, x, y = sym.name.split("_")
        x, y = sorted((sw.get(x, x), sw.get(y, y)))
        rep[sym] = sp.Symbol(f"{k}_{x}_{y}")
    return sp.expand(g4 - g4.xreplace(rep)) == 0


def main():
    t0 = time.time()
    import p6_inventory as P
    inv = P.inventory()
    inv["V_hhh_phi_phi"] = {"formula": P.X3_formula.__doc__.strip(), "two_routes_agree": P.vertex_hhhphiphi_check(),
                            "mutation_tr_h_cubed_detected": not P.vertex_hhhphiphi_check(mutate=True),
                            "role": "enters only the scaleless tadpoles (E3 tadpoles 2, 3)"}
    inv["V5_grav"] = {"status": "NOT constructed", "why_not_needed": "enters only the self-contracted tadpole (hhh), which "
                      "is scaleless for ANY polynomial vertex; no other one-loop E = 3 graph contains it"}
    (CERT / "Q04_P6_inventory.json").write_text(json.dumps(inv, indent=1) + "\n")
    ident = P.identity_rows()
    ident["counterterm_domain_kappa3"] = P.ct_domain_dims()
    (CERT / "Q04_P6_identity.json").write_text(json.dumps(ident, indent=1) + "\n")
    with ProcessPoolExecutor(max_workers=6) as ex:
        f = {"d11": ex.submit(task_d11), "d11_mut": ex.submit(task_d11, "1"), "ward": ex.submit(task_ward),
             "ward_mut_lie": ex.submit(task_ward, -1), "ward_mut_bub": ex.submit(task_ward, 1, True),
             "g3": ex.submit(task_gamma3)}
        R = {k: v.result() for k, v in f.items()}
    p23 = json.loads((Q / "provenance/XVI-P/certificates/P23_01_scope.json").read_text())["P6_entry_kappa3_three_point"]["graphs"]
    table = []
```

```
    for i, g in enumerate(p23, 1):
        evald = g["vertices"] in (["V_h_phi_phi"] * 3, ["V_h_phi_phi", "V_hh_phi_phi"]) and g["external"] == "h,h,h"
        table.append({"id": f"P6-G{i:02d}", "vertices": g["vertices"], "lines": g["loop_lines"], "external": g["external"],
                      "status": "UV pole evaluated (exact residue, own rules, validated vs D11; Ward-checked); finite part Missing"
                      if evald else "Missing (not evaluated in Q04)"})
    for j, tp in enumerate(inv["E3_tadpoles"], 1):
        table.append({"id": f"P6-T{j}", "vertices": tp["vertices"], "lines": tp["lines"], "external": ",".join(tp["external"]),
                      "status": "derived 0 (scaleless, no log member)"})
    ev = {"engine": "code/q04/uvpole.py (own Feynman rules, symbolic d via qguard, DRED hat/tilde split, IR-rearranged "
                    "Taylor UV pole; units U = i/(16 pi^2 eps); 'i Gamma' convention)",
          "validation_D11": R["d11"], "validation_mutation_trace_coef_1": R["d11_mut"],
          "scalar_loop_hhh": R["g3"],
          "matter_ward_three_point": {"identity": "Gamma3(d0h, e1, e2) + Gamma2(d1h[xi,e1], e2) + Gamma2(d1h[xi,e2], e1) = 0 "
                                                  "(diffeomorphism invariance of the scalar-loop W[g]); row c.h.h, matter subsector",
                                      "declared": R["ward"], "mutation_wrong_Lie_sign": R["ward_mut_lie"],
                                      "mutation_drop_X2_bubbles": R["ward_mut_bub"],
                                      "reading": "the MS-bar pole residue satisfies the identity exactly, incl. the evanescent "
                                                 "(tilde) structures; the d-dependent coefficient differs by O(d-4) x U, i.e. "
                                                 "only finite (non-pole) terms, which Q04 does not evaluate"},
          "graph_table": table, "n_evaluated_pole": sum(1 for x in table if x["status"].startswith("UV pole")),
          "n_missing": sum(1 for x in table if x["status"].startswith("Missing")),
          "n_derived_zero": sum(1 for x in table if x["status"].startswith("derived 0"))}
    (CERT / "Q04_P6_eval.json").write_text(json.dumps(ev, indent=1) + "\n")
    ok_val = R["d11"]["constant_ratio"] and R["d11"]["ratio_mine_over_XVI_P"] == "I" and not R["d11_mut"]["constant_ratio"]
    ok_ward = R["ward"]["residue_d4_zero"] and not R["ward_mut_lie"]["residue_d4_zero"] and not R["ward_mut_bub"]["residue_d4_zero"]
    st = {"landmark": "XVI-Q", "release": "Q04", "P6_outcome": "Partial",
          "stages": {"inventory": "Constructed" if inv["equals_XVI_P_P23_15"] and inv["n_E3_tadpoles"] == 3 else "FAIL",
                     "kappa3_vertices": "Constructed (V_hhh_phi_phi); V5_grav not needed (scaleless only)",
                     "power_counting_kappa3_1pt_2pt_empty": inv["power_counting"]["verified_on_all_enumerated_graphs"],
                     "identity_ledger_rows": f"Constructed ({ident['n_rows']} gh1 three-field rows; "
                                             f"{sum(1 for r in ident['rows'] if r['class'] == 'needs_inputs')} need P.6 inputs)",
                     "counterterm_domain_kappa3": "Partial (structural counts; BRST/sigma_0 reduction not done)",
                     "engine_validated_vs_XVI_P_D11": ok_val,
                     "evaluation": f"Partial: {ev['n_evaluated_pole']} graphs at pole level, {ev['n_derived_zero']} tadpoles "
                                   f"derived 0, {ev['n_missing']} Missing",
                     "matter_subsector_ward_c_h_h_pole": "verified" if ok_ward else "FAIL",
                     "defect_assembly": "Missing (inputs Missing for every needs_inputs row)",
                     "counterterm_solve_kappa3": "Missing", "sigma0_closure_kappa3": "Missing"},
          "rows_status": {r["row"]: ("zero (" + r["why"][:40] + ")" if r["class"] == "zero" else
                                     "aux" if r["class"] == "aux" else "Missing inputs") for r in ident["rows"]},
          "O1_closure_unchanged": "CLOSED (scoped) -- kappa^2 two-point; P.6 is outside that scope",
          "bulk_symmetry_restored": False, "seconds": round(time.time() - t0, 1)}
    (CERT / "Q04_P6_status.json").write_text(json.dumps(st, indent=1) + "\n")
    import guard_scan
    (CERT / "Q04_guard_scan.json").write_text(json.dumps(guard_scan.report(), indent=1) + "\n")
    print(json.dumps({k: v for k, v in st["stages"].items()}, indent=0))


if __name__ == "__main__":
    main()
```

code/q04/release_q04.py

```
#!/usr/bin/env python3
"""XVI-Q Q04 release gate (fails closed; on reject no RELEASE_STATUS.json is left).
(1) code/q04/q04_expected.json pinned; EVERY Q01..Q04 certificate equals its pinned copy; pinned code files re-hashed;
(2) provenance re-hashed (all 33 pinned, no unpinned file); bound Q02 independent report re-hashed;
(3) semantic P.6 checks: inventory == XVI-P 15 + 3 tadpoles; kappa = E; V_hhh_phi_phi two routes + mutation; engine
    validated vs XVI-P D11 (ratio exactly I) + mutation; scalar-loop Ward residue 0 + both mutations nonzero; graph table
    counts; honesty: P6_outcome 'Partial', defect / CT / closure 'Missing' unless every graph is evaluated;
(4) Q01..Q03 layer gates accepted; O.1 record unchanged (CLOSED (scoped), confirmed); every gate read in the source set."""
from __future__ import annotations
import hashlib, json, sys  # noqa: E401
from pathlib import Path

Q = Path(__file__).resolve().parents[2]
sys.path.insert(0, str(Q / "tools"))
import source_set as SS  # noqa: E402
CERT = Q / "certificates"
EXPECTED_SHA256 = "9b4e38d5c3894455bd9b20801f6dbf2b1bbfa0c0a46b1e28b446601717514a15"
STATUS = (Q / "RELEASE_STATUS.json", CERT / "RELEASE_STATUS.json")
GENERATED = ("certificates/", "ledgers/", "build/")
HARD_STOPS = {"bulk_symmetry_restored": False, "insertion": "frozen", "XV_B_opened": False, "XIV_J_opened": False,
              "Route_B_discharged": False, "Missing_never_zero": True}
READS: set[str] = set()


def rd(rel):
    READS.add(rel)
    return (Q / rel).read_bytes()


def jr(rel):
```

```
    return json.loads(rd(rel))


def sha_b(b):
    return hashlib.sha256(b).hexdigest()


def strip(o):
    if isinstance(o, dict):
        return {k: strip(v) for k, v in o.items() if k != "seconds"}
    if isinstance(o, list):
        return [strip(v) for v in o]
    return o


def _live_scan_clean():
    sys.path.insert(0, str(Q / "code" / "q04"))
    import guard_scan
    READS.add("code/q04/guard_scan.py")
    return guard_scan.scan() == []


def checks():
    c = {}
    eb = rd("code/q04/q04_expected.json") if (Q / "code/q04/q04_expected.json").is_file() else b""
    c["expected_file_pinned"] = bool(eb) and sha_b(eb) == EXPECTED_SHA256
    exp = json.loads(eb) if eb else {}
    bad = [n for n, ref in exp.get("certificates", {}).items()
           if not (CERT / f"{n}.json").is_file() or strip(jr(f"certificates/{n}.json")) != ref]
    c["every_certified_value_matches_pinned"] = len(exp.get("certificates", {})) >= 18 and not bad
    c["_value_mismatches"] = bad
    badf = [f for f, s in exp.get("file_sha256", {}).items() if not (Q / f).is_file() or sha_b(rd(f)) != s]
    c["code_rehashed"] = len(exp.get("file_sha256", {})) >= 15 and not badf
    c["_file_mismatches"] = badf
    imp = jr("intake/imports.json")
    badp = [f for f, m in imp["files"].items() if not (Q / f).is_file() or sha_b(rd(f)) != m["sha256"]]
    unp = [p.relative_to(Q).as_posix() for p in (Q / "provenance").rglob("*") if p.is_file()
           and p.relative_to(Q).as_posix() not in imp["files"]]
    c["provenance_rehashed_all_pinned"] = len(imp["files"]) == 33 and not badp and not unp
    c["bound_q02_report_rehashed"] = sha_b(rd("reproducibility/independent_rebuild_report_q02.json")) == \
        "e4b54702bc768899f22a00a289de0c0aeb2bf9317b87a28c4a462f05ae4bf495"
    c["bound_q03_report_rehashed"] = sha_b(rd("reproducibility/independent_rebuild_report_q03.json")) ==
"3b73b81f854cb55e6f1fbe2a76f80d7ad8c6e7164915afef2d595c7b36656417"
    fs = exp.get("file_sha256", {})
    c["tools_source_set_pinned"] = all(f in fs for f in ("tools/source_set.py", "tools/build.py", "tools/rebuild_from_source.py"))
    gs = jr("certificates/Q04_guard_scan.json")
    c["guard_leave_sympy_class_disclosed_and_scan_clean"] = gs["scan_clean"] is True and gs["runtime_closable"] is False         and
gs["n_files_scanned"] >= 18 and _live_scan_clean() and set(gs["probes"]) >= {"python_term_walk", "str_rename_reparse", "dense_list_eval"}
    inv = jr("certificates/Q04_P6_inventory.json")
    c["inventory_15_plus_3_tadpoles"] = inv["equals_XVI_P_P23_15"] is True and inv["n_E3_loops"] == 15 and \
        inv["n_E3_tadpoles"] == 3 and inv["by_external"] == {"K,c,h": 2, "c,c,cs": 1, "c,cb,h": 2, "c,phi,phis": 1,
                                                             "h,h,h": 7, "h,phi,phi": 5}
    c["kappa_equals_E_and_1pt_2pt_counts"] = inv["power_counting"]["verified_on_all_enumerated_graphs"] is True and \
        inv["n_by_E"].get("1") == 3 and inv["n_by_E"].get("2") == 9
    v = inv["V_hhh_phi_phi"]
    c["vertex_hhhphiphi_two_routes_and_mutation"] = v["two_routes_agree"] is True and v["mutation_tr_h_cubed_detected"] is True
    ev = jr("certificates/Q04_P6_eval.json")
    c["engine_validated_vs_D11"] = ev["validation_D11"]["ratio_mine_over_XVI_P"] == "I" and \
        ev["validation_D11"]["constant_ratio"] is True and ev["validation_mutation_trace_coef_1"]["constant_ratio"] is False
    w = ev["matter_ward_three_point"]
    c["matter_ward_residue_zero_mutations_detect"] = w["declared"]["residue_d4_zero"] is True and \
        w["mutation_wrong_Lie_sign"]["residue_d4_zero"] is False and w["mutation_drop_X2_bubbles"]["residue_d4_zero"] is False
    c["scalar_hhh_bose_symmetric"] = ev["scalar_loop_hhh"]["bose_symmetric_a1_a2"] is True
    tb = ev["graph_table"]
    n_ev = sum(1 for x in tb if x["status"].startswith("UV pole"))
    n_miss = sum(1 for x in tb if x["status"].startswith("Missing"))
    n_zero = sum(1 for x in tb if x["status"].startswith("derived 0"))
    c["graph_table_counts"] = len(tb) == 18 and (n_ev, n_zero, n_miss) == (ev["n_evaluated_pole"], ev["n_derived_zero"], ev["n_missing"]) \
        and n_ev + n_zero + n_miss == 18 and n_zero == 3
    st = jr("certificates/Q04_P6_status.json")
    sg = st["stages"]
    honest = n_miss > 0 and st["P6_outcome"] == "Partial" and sg["defect_assembly"].startswith("Missing") and \
        sg["counterterm_solve_kappa3"] == "Missing" and sg["sigma0_closure_kappa3"] == "Missing"
    c["honest_partial_outcome"] = honest or (n_miss == 0 and st["P6_outcome"] != "Partial")
    c["no_missing_set_to_zero"] = all(not x["status"].startswith("derived 0") or x["id"].startswith("P6-T") for x in tb)
    c["bulk_symmetry_restored_false"] = st["bulk_symmetry_restored"] is False
    for n in ("Q01_release", "Q02_release", "Q03_release"):
        c[f"{n}_layer_accepted"] = jr(f"certificates/{n}.json").get("publication") == "accepted"
    q3 = jr("certificates/Q03_release.json")
    c["O1_record_unchanged"] = q3.get("O1_closure") == "CLOSED (scoped)" and q3.get("O1_closure_confirmed") is True
    src = set(SS.source_files(Q))
    out = sorted(f for f in READS if f not in src and not f.startswith(GENERATED))
    c["every_gate_read_in_source_set"] = not out and len(READS) > 40
    c["_reads_outside_source_set"] = out
```

```
    return c, st, q3, ev


def main():
    try:
        c, st, q3, ev = checks()
    except Exception as exc:  # noqa: BLE001
        c, st, q3, ev = {"error": f"{type(exc).__name__}: {exc}"}, {}, {}, {}
    ok = all(v is True for k, v in c.items() if not k.startswith("_"))
    for s in STATUS:
        s.unlink(missing_ok=True)
    out = {"landmark": "XVI-Q", "release": "Q04", "version_label": "XVI-Q Q04", "date": "2026-10-11",
           "kind": "P.6 entry (one-loop kappa^3 three-point): honest partial", "execution_pass": ok,
           "publication": "accepted" if ok else "REJECTED", "value_checks": c,
           "P6_outcome": st.get("P6_outcome") if ok else "NOT_EARNED", "P6_stages": st.get("stages"),
           "P6_rows": st.get("rows_status"),
           "P6_graphs": [{k: x[k] for k in ("id", "external", "status")} for x in ev.get("graph_table", [])],
           "O1_closure": q3.get("O1_closure") if ok else "see Q03", "O1_closure_confirmed": bool(ok and q3.get("O1_closure_confirmed")),
           "O1_closure_scope": q3.get("O1_closure_scope"), "O1_not_implied": q3.get("O1_not_implied"),
           "O1_caveats": q3.get("O1_caveats"), "XVI_O_ledger_O1": "NOT_CLOSED (read-only, unchanged)",
           "bulk_symmetry_restored": False, "symmetry_restored": False,
           "guard_leave_sympy_class": "disclosed; runtime-unclosable; enforced by static scan of package code (clean)",
           "flags": {"P6_inventory": "earned" if ok else "not_earned", "P6_identity_ledger": "earned" if ok else "not_earned",
                     "P6_engine_validated": "earned" if ok else "not_earned",
                     "P6_matter_subsector_ward_pole": "earned" if ok else "not_earned",
                     "P6_evaluated": "partial", "P6_counterterms": "Missing", "P6_sigma0_closure": "Missing",
                     "independent_rebuild_passed_q02": True, "independent_rebuild_passed_q03": True,
                     "independent_rebuild_passed_q04": "pending", "author_rebuild_passed": "see reproducibility"},
           "hard_stops": HARD_STOPS, "XVI_Q_registered": "locally; not published"}
    cr = CERT / "Q04_release.json"
    if ok:
        txt = json.dumps(out, indent=1) + "\n"
        for s in STATUS:
            s.write_text(txt)
        cr.write_text(txt)
        (CERT / "Q04_release_rejected.json").unlink(missing_ok=True)
        (Q / "ledgers" / "Q04_milestones.json").write_text(json.dumps(
            {"landmark": "XVI-Q", "release": "Q04", "P6_outcome": out["P6_outcome"], "flags": out["flags"],
             "hard_stops": HARD_STOPS}, indent=1) + "\n")
    else:
        cr.unlink(missing_ok=True)
        (CERT / "Q04_release_rejected.json").write_text(json.dumps(out, indent=1, default=str) + "\n")
    print(json.dumps({"publication": out["publication"], "P6": out["P6_outcome"],
                      "failed": [k for k, v in c.items() if not k.startswith("_") and v is not True],
                      "mm": {k: v for k, v in c.items() if k.startswith("_") and v}}, default=str)[:1500])
    return 0 if ok else 1


if __name__ == "__main__":
    raise SystemExit(main())
```

code/q04/scalar_loop3.py

```
#!/usr/bin/env python3
"""XVI-Q Q04: scalar-loop kappa^3 graviton three-point UV pole (P.6 graphs G13 = V_h_phi_phi^3 triangle, G14 =
V_h_phi_phi + V_hh_phi_phi bubble; the V_hhh_phi_phi phi-phi tadpole is scaleless) and the matter-loop diffeomorphism
Ward identity at three points:
   Gamma3(delta0 h(q), e1(p1), e2(p2)) + Gamma2(delta1 h[xi, e1](q+p1), e2(p2)) + Gamma2(delta1 h[xi, e2](q+p2), e1(p1)) = 0,
   delta0 h = -i (q (x) xi + xi (x) q),  delta1 h[xi, e](k) = -i [ (k.xi) e + q (x) (e.xi) + (e.xi) (x) q ]   (L_xi g)
(W[g] of the scalar loop is diffeomorphism invariant; g = eta + h, kappa = 1).  Pole in units of U (uvpole.py).
Conventions tie: Gamma2 here equals I x XVI-P P18 D11 pole on all 14 structures (validated in q04 certs)."""
from __future__ import annotations
import sys
from pathlib import Path
import sympy as sp

sys.path.insert(0, str(Path(__file__).resolve().parent))
import uvpole as U  # noqa: E402

HALF = sp.Rational(1, 2)


def gamma2(eA, eB, k):
    """i Gamma2 pole / U: A carries incoming momentum k, B carries -k.  Scalar bubble, symmetry 1/2."""
    l = U.vec("l")
    num = U.V_hphiphi(eA, l, -(l + k)) * U.V_hphiphi(eB, l + k, -l)
    return U.loop_pole(num, [U.V(), k], 4, HALF)


def gamma3(legs):
    """legs = [(eps, k)] x 3 (incoming momenta, sum 0).  Triangles: both cyclic orders, factor 1/2;
    bubbles: X2 vertex with a pair + X1 vertex with the third, factor 1/2."""
    l = U.vec("l")
```

```
    tot = 0
    parts = {}
    for order in ((0, 1, 2), (0, 2, 1)):
        (eA, kA), (eB, kB), (eC, kC) = (legs[i] for i in order)
        num = U.V_hphiphi(eA, l, -(l + kA)) * U.V_hphiphi(eB, l + kA, -(l + kA + kB)) * U.V_hphiphi(eC, l + kA + kB, -l)
        v = U.loop_pole(num, [U.V(), kA, kA + kB], 4, HALF)
        parts[f"triangle{order}"] = v
        tot += v
    for c in range(3):
        a, b = [i for i in range(3) if i != c]
        (eC, kC), (ea, _), (eb, _) = legs[c], legs[a], legs[b]
        num = U.V_hphiphi(eC, l, -(l + kC)) * U.V_hhphiphi(ea, eb, l + kC, -l)
        v = U.loop_pole(num, [U.V(), kC], 4, HALF)
        parts[f"bubble_X2{(a, b)}_X1{c}"] = v
        tot += v
    return sp.expand(tot), parts


def pv(*pairs):
    """polarisation-side vector WITHOUT the Taylor parameter (constant w.r.t. the loop expansion)."""
    return U.V(dict(pairs))


def ward(drop_bubbles=False, lie_sign=1):
    p1, p2 = U.vec("p1"), U.vec("p2")
    q = -(p1 + p2)
    qc = pv(("p1", -1), ("p2", -1))
    xi = pv(("xi", 1))
    e1, e2 = U.rank1("a1"), U.rank1("a2")
    d0 = U.Eps([(-2 * sp.I, qc, xi)])
    g3, parts = gamma3([(d0, q), (e1, p1), (e2, p2)])
    if drop_bubbles:
        g3 = sp.expand(sum(v for k, v in parts.items() if k.startswith("triangle")))

    def d1(a, kvec_c):
        ax = U.dot(U.vec(a), xi)                          # (e.xi) = (a.xi) a  (4S)
        return U.Eps([(-sp.I * U.dot(kvec_c, xi), U.vec(a), U.vec(a)), (-2 * sp.I * ax * lie_sign, qc, U.vec(a))])
    w = g3 + gamma2(d1("a1", pv(("p1", 1))), e2, -p2) + gamma2(d1("a2", pv(("p2", 1))), e1, -p1)
    return sp.expand(w), g3


if __name__ == "__main__":
    import time
    t0 = time.time()
    w, g3 = ward()
    print("ward residual (pole/U):", sp.factor(w) if w != 0 else 0, time.time() - t0)
```

code/q04/uvpole.py

```
#!/usr/bin/env python3
"""XVI-Q Q04 independent one-loop UV-pole engine (own Feynman rules; NOT XVI-P exports).

Prescription (XVI-P declared, DRED): loop momentum l in QdS; external momenta p_i in QdS; vertex (graviton/ghost)
indices in Q4S = QdS (+) Q(4-d)S.  Polarisation vectors a carry a hat part (QdS) and a tilde part (evanescent).
Invariants: H(x,y) = hat dot, T(x,y) = tilde dot (only between polarisation vectors); 4S dot = H + T; momenta are hat.
d is the XVI-P opaque dimension (qguard); it appears only through symmetric integration
    < l^{mu1} ... l^{mu2n} > = sym(ghat)^n (l^2)^n / (d (d+2) ... (d+2n-2)).
UV pole: Taylor expansion in the external momenta (scaling parameter t) to the superficial degree omega (exact
algebraic identity per term); the t^omega coefficient is homogeneous of l-degree -4 and its UV pole is
<...> * U,  U := pole of int d^dl/(2pi)^d (l^2+i0)^-2 = i/(16 pi^2 eps).  Terms of other l-degree are power divergent
(scaleless after IR rearrangement: pole proportional to a regulator mass, vanishing at M = 0) or UV finite.
Massless: no mass anywhere (XVI-O O.1 m = 0)."""
from __future__ import annotations
import itertools, sys  # noqa: E401
from functools import lru_cache
from pathlib import Path
import sympy as sp

Q = Path(__file__).resolve().parents[2]
sys.path.insert(0, str(Q / "code" / "q01"))
import qguard  # noqa: E402,F401   (installs the d guard)
d = qguard.d
t = sp.Symbol("t")
L = sp.Symbol("L")           # l.l
MOM = ("p1", "p2")           # independent external momenta (QdS)


def is_mom(n):
    return n == "l" or n in MOM


def H(x, y):
    x, y = sorted((x, y))
```

```
    return sp.Symbol(f"H_{x}_{y}")


def T(x, y):
    x, y = sorted((x, y))
    return sp.Symbol(f"T_{x}_{y}")


def S(x):                    # l.x
    return sp.Symbol(f"s_{x}")


class V(dict):
    """vector = {basis name: coefficient}; basis 'l', 'p1', 'p2' (hat) or polarisation names (hat+tilde)."""
    def __add__(self, o):
        r = V(self)
        for k, v in o.items():
            r[k] = r.get(k, 0) + v
        return r

    def __neg__(self):
        return V({k: -v for k, v in self.items()})

    def __sub__(self, o):
        return self + (-o)

    def __rmul__(self, c):
        return V({k: c * v for k, v in self.items()})


def vec(n, c=1):
    return V({n: (t * c if n in MOM else c)})      # momenta carry the Taylor parameter t


def bdot(x, y, hat_only=False):
    if x == "l" and y == "l":
        return L
    if x == "l" or y == "l":
        o = y if x == "l" else x
        return S(o)
    if is_mom(x) or is_mom(y) or hat_only:
        return H(x, y)
    return H(x, y) + T(x, y)


def dot(u, w, hat_only=False):
    return sp.expand(sum(cu * cw * bdot(a, b, hat_only) for a, cu in u.items() for b, cw in w.items()))


class Eps(list):
    """symmetric 4S tensor = sum c * (u (x) w + w (x) u)/2 ; items (c, u, w)."""
    def __call__(self, x, y):                      # eps(x, y)
        return sp.expand(sum(sp.Rational(1, 2) * c * (dot(u, x) * dot(w, y) + dot(w, x) * dot(u, y)) for c, u, w in self))

    def tr(self):
        return sp.expand(sum(c * dot(u, w) for c, u, w in self))

    def vdot(self, x):                             # (eps . x)_mu as a list of (coef, vector)
        return [(sp.Rational(1, 2) * c * dot(w, x), u) for c, u, w in self] + [(sp.Rational(1, 2) * c * dot(u, x), w) for c, u, w in self]


def rank1(a):
    return Eps([(1, vec(a), vec(a))])


def pair_contract(e1, e2, x, y):
    """(e1 . e2)(x, y) = e1_{x rho} e2^{rho}_y (4S contraction over rho)."""
    tot = 0
    for c1, v1 in e1.vdot(x):
        for c2, v2 in e2.vdot(y):
            tot += c1 * c2 * dot(v1, v2)
    return sp.expand(tot)


def tr2(e1, e2):
    """tr4(e1 e2)."""
    tot = 0
    for c1, u1, w1 in e1:
        for c2, u2, w2 in e2:
            tot += sp.Rational(1, 2) * c1 * c2 * (dot(u1, u2) * dot(w1, w2) + dot(u1, w2) * dot(w1, u2))
    return sp.expand(tot)


# ---------------- scalar sector:  L_phi = (1/2) sqrt(-g) g^{mn} d_m phi d_n phi,  g = eta + kappa h  (eta + - - -)
```

```
#   X^{mn} = sqrt(-g) g^{mn} = eta + kappa X1 + kappa^2 X2 + ... ; X1 = (1/2) tr h eta - h ; X2 = h.h - (1/2) tr h h + (tr h^2/8 - tr(h.h)/4) eta
TRACE_COEF = sp.Rational(1, 2)    # mutation hook (q04 certs): sqrt(-g) -> 1 + TRACE_COEF tr h


def X1(e, x, y):
    return sp.expand(TRACE_COEF * e.tr() * dot(x, y) - e(x, y))


def X2cross(e1, e2, x, y):
    """coefficient of t1 t2 in X2[t1 e1 + t2 e2](x, y)."""
    return sp.expand(pair_contract(e1, e2, x, y) + pair_contract(e2, e1, x, y)
                     - sp.Rational(1, 2) * (e1.tr() * e2(x, y) + e2.tr() * e1(x, y))
                     + (sp.Rational(1, 4) * e1.tr() * e2.tr() - sp.Rational(1, 2) * tr2(e1, e2)) * dot(x, y))


def X3(es, x, y):
    """fully polarised cubic term of sqrt(-g) g^{mn} (coefficient of t1 t2 t3) -- used for the V_hhh_phi_phi vertex."""
    raise NotImplementedError


def V_hphiphi(e, k1, k2):
    """vertex i * delta^3 S / (delta h delta phi delta phi), scalar momenta k1, k2 incoming: -i X1(k1, k2) (kappa^1)."""
    return -sp.I * X1(e, k1, k2)


def V_hhphiphi(e1, e2, k1, k2):
    return -sp.I * X2cross(e1, e2, k1, k2)


# ---------------- symmetric integration and UV pole
@lru_cache(maxsize=None)
def _avg_coeff(key):
    """<prod_j (l.v_j)^{k_j}> / (l^2)^{K/2} as a polynomial in hat dots; key = tuple((name, k_j))."""
    names = [n for n, _ in key]
    ks = [k for _, k in key]
    K = sum(ks)
    if K % 2:
        return sp.Integer(0)
    tt = sp.symbols(f"tt0:{len(names)}")
    x2 = sp.expand(sum(tt[i] * tt[j] * H(names[i], names[j]) for i in range(len(names)) for j in range(len(names))))
    P = sp.Poly(sp.expand(x2 ** (K // 2)), *tt)
    c = P.coeff_monomial(tuple(ks))
    num = sp.Integer(1)
    for k in ks:
        num *= sp.factorial(k)
    dfac = sp.Integer(1)
    for i in range(K // 2):
        dfac *= (d + 2 * i)
    return sp.expand(c * num / sp.factorial(K) * sp.factorial2(K - 1)) / dfac


def integrate_pole(expr):
    """expr: polynomial in s_x and L^{+-1} (homogeneous l-degree -4 terms selected); returns pole / U."""
    expr = sp.expand(expr)
    svars = sorted({s for s in expr.free_symbols if s.name.startswith("s_")}, key=lambda s: s.name)
    out = 0
    for term in sp.Add.make_args(expr):
        powd = term.as_powers_dict()
        kL = powd.get(L, 0)
        ks = [(s.name[2:], int(powd.get(s, 0))) for s in svars if powd.get(s, 0)]
        K = sum(k for _, k in ks)
        if K + 2 * kL != -4:
            continue
        coef = term / (L ** kL)
        for s in svars:
            coef = coef / s ** powd.get(s, 0)
        out += coef * (_avg_coeff(tuple(ks)) if ks else 1)
    return out


def taylor_denominators(dens, omega):
    """prod_j 1/(l+K_j)^2 expanded in t to order omega: sum_n (-(2 l.K + K^2)/L)^n / L."""
    tot = sp.Integer(1)
    for K in dens:
        lk = dot(vec("l"), K)
        kk = dot(K, K)
        x = sp.expand(2 * lk + kk)
        ser = sum((-x) ** n / L ** (n + 1) for n in range(omega + 1))
        tot = sp.expand(tot * ser)
        tot = trunc(tot, omega)
    return tot


def trunc(e, n):
```

```
    e = sp.expand(e)
    return sum(term for term in sp.Add.make_args(e) if sp.degree(term, t) <= n) if e != 0 else e


def loop_pole(numer, dens, omega, prefactor):
    """UV pole / U of prefactor * int numer * prod i/(l+K)^2 ; numer contains the vertex factors."""
    num = trunc(sp.expand(numer), omega)
    den = taylor_denominators(dens, omega)
    full = sp.expand(num * den)
    top = sp.expand(full).coeff(t, omega)
    return sp.expand(prefactor * sp.I ** len(dens) * integrate_pole(top))
```

tests/test_q01.py

```
"""XVI-Q Q01 tests: intake pins, 0.2 items (values), guard forms, gate value checks (fail closed, no stale status)."""
from __future__ import annotations
import json
import shutil
import subprocess
import sys
import tempfile
from pathlib import Path

import pytest

Q = Path(__file__).resolve().parents[1]
sys.path.insert(0, str(Q / "code" / "q01"))
import q01_conventions as QC  # noqa: E402

CERT = Q / "certificates"


def cj(n):
    return json.loads((CERT / f"{n}.json").read_text())


def test_intake_pins_and_errata():
    it = cj("Q01_intake")
    assert it["pins_ok"] and it["XVI_P_p23_zip_ok"] and it["XVI_P_p23_extracted_identical"] and it["O1_ledger_sha_ok"]
    assert [e["id"] for e in it["errata"]][:5] == ["E1", "E2", "E3", "E4", "E5"]
    assert "overclaim" in it["errata"][0]["correction"].lower()


def test_O2_items_constructed_with_values():
    o2 = cj("Q01_O2")
    assert o2["item_status"] == QC.EXPECTED["O2_items"] and o2["O2_kappa2_master_status"] == "Constructed"
    a = o2["items"]["a_ell4"]["detail"]
    assert a["n_terms_mine"] == a["n_terms_export"] == 3360 and a["full_polynomial_equal"] and a["normalisation_from_XVI_O_probe"] == "-2"
    assert o2["W_terms"] == QC.EXPECTED["W_terms"]
    assert "NOT quantum" in o2["scope"]


def test_O2b_live_independent_engine():
    import o2b_antibracket as AB
    r = AB.main(Q / "build" / "xvip_run" / "exports")
    assert all(r[k]["antibracket_af0_vanishes"] for k in ("kappa0", "kappa1", "kappa2"))
    assert r["kappa2"]["W_terms"] == 15428


def test_master_mutations_all_break():
    m = cj("Q01_O2")["master_mutations"]
    assert {k: v["n_residuals"] for k, v in m.items()} == QC.EXPECTED["master_mutation_residuals"]
    assert all(v["claim_status"] != "verified" for v in m.values())


FORMS = ["Matrix_Dummy", "Matrix_y", "Poly_shift", "apart_list", "apart_residue", "captured_rem", "compose", "evalf", "groebner", "lambdify", "limit",
         "polys_ns_rem", "reduced", "rem", "rep_eval", "resultant", "subs", "xreplace"]


def test_guard_probe_all_forms_raise():
    g = cj("Q01_guard")
    assert g["forms"] == sorted(FORMS) and g["missed"] == [] and g["n_exprs"] == 4
    assert g["algebra"]["authorised_value"] == "1/80" and g["algebra"]["domain_only_poly"] == "True"


def test_guard_live_p23_bypasses():
    code = ("import sys; sys.path.insert(0, %r)\n" % str(Q / "code" / "q01") +
            "from sympy.polys.polytools import rem as r0\nimport qguard, sympy as sp, sympy.polys as spp\n"
            "d = qguard.d; out = []\n"
            "for f in (lambda: spp.rem(d**2-1, d-4, d), lambda: r0(d**2-1, d-4, d), lambda: sp.reduced(d**2-1, [d-4], d),\n"
            "          lambda: sp.groebner([d-4, d**2-1], d), lambda: sp.Poly(d**2-1, d).rep.eval(4),\n"
            "          lambda: sp.Matrix([1/(d**2-1)]).subs(d, sp.Dummy()).subs(sp.Dummy(), 4)):\n"
            "    try:\n        f(); out.append('NOT')\n    except qguard.PrematureD4Substitution:\n        out.append('R')\n"
```

```
            "print(',',.join(out))")
    r = subprocess.run([sys.executable, "-c", code], capture_output=True, text=True, timeout=300)
    assert r.stdout.strip() == "R,R,R,R,R,R", r.stdout + r.stderr


def test_release_status_flags_and_hard_stops():
    s = json.loads((CERT / "Q01_release.json").read_text())      # Q01 layer record (RELEASE_STATUS is Q02's)
    assert s["publication"] == "accepted" and s["execution_pass"] is True and all(v is True for v in s["value_checks"].values())
    assert s["O1_closure"] == "NOT_CLOSED" and s["bulk_symmetry_restored"] is False
    assert s["flags"]["O2_kappa2_master_constructed"] == "earned" and s["flags"]["bulk_O5_evaluated"] == "not_earned"
    assert s["hard_stops"] == QC.HARD_STOPS


def _copy():
    d = Path(tempfile.mkdtemp(prefix="q01gate_"))
    for sub in ("code", "certificates"):
        shutil.copytree(Q / sub, d / sub, ignore=shutil.ignore_patterns("__pycache__"))
    for f in ("RELEASE_STATUS.json",):
        if (Q / f).exists():
            shutil.copy(Q / f, d / f)
    return d


def _edit(d, n, fn):
    f = d / "certificates" / f"{n}.json"
    c = json.loads(f.read_text())
    fn(c)
    f.write_text(json.dumps(c))


MUTS = {
    "ell4_terms": lambda d: _edit(d, "Q01_O2", lambda c: c["items"]["a_ell4"]["detail"].__setitem__("n_terms_mine", 3359)),
    "ell4_norm": lambda d: _edit(d, "Q01_O2", lambda c: c["items"]["a_ell4"]["detail"].__setitem__("normalisation_from_XVI_O_probe", "2")),
    "W2_terms": lambda d: _edit(d, "Q01_O2", lambda c: c["W_terms"].__setitem__("kappa2", 15427)),
    "Ec_nonzero": lambda d: _edit(d, "Q01_O2", lambda c:
c["items"]["b_antibracket_expansion"]["detail"]["unmutated"]["kappa2"]["E_c_W_terms"].__setitem__("0", 3)),
    "mut_count": lambda d: _edit(d, "Q01_O2", lambda c: c["master_mutations"]["corrupt_a2"].__setitem__("n_residuals", 0)),
    "status_c": lambda d: _edit(d, "Q01_O2", lambda c: c["item_status"].__setitem__("c_ghost_antifield", "Partial")),
    "export_sha": lambda d: _edit(d, "Q01_O2", lambda c: c["exports_sha256"].__setitem__("4", "0" * 64)),
    "guard_missed": lambda d: _edit(d, "Q01_guard", lambda c: c.__setitem__("missed", [["rem", "pow", "NOT_RAISED"]])),
    "pins": lambda d: _edit(d, "Q01_intake", lambda c: c.__setitem__("pins_ok", False)),
    "errata_dropped": lambda d: _edit(d, "Q01_intake", lambda c: c["errata"].pop(0)),
    "cert_missing": lambda d: (d / "certificates" / "Q01_O2.json").unlink(),
}


@pytest.mark.parametrize("name", sorted(MUTS))
def test_gate_rejects_value_edit_without_stale_status(name):
    d = _copy()
    try:
        MUTS[name](d)
        r = subprocess.run([sys.executable, str(d / "code" / "q01" / "release_q01.py")], capture_output=True, text=True)
        assert r.returncode == 1, r.stdout
        assert not (d / "certificates" / "Q01_release.json").exists()
    finally:
        shutil.rmtree(d, ignore_errors=True)


def test_gate_accepts_unmodified_copy():
    d = _copy()
    try:
        r = subprocess.run([sys.executable, str(d / "code" / "q01" / "release_q01.py")], capture_output=True, text=True)
        assert r.returncode == 0, r.stdout
    finally:
        shutil.rmtree(d, ignore_errors=True)
```

tests/test_q02.py

```
"""XVI-Q Q02 tests: bulk graphs, D12 reconciliation, bulk identity, O.1 walk, guard (apart), gate value checks."""
from __future__ import annotations
import json
import shutil
import subprocess
import sys
import tempfile
from pathlib import Path

import pytest

Q = Path(__file__).resolve().parents[1]
CERT = Q / "certificates"
```

```
def cj(n):
    return json.loads((CERT / f"{n}.json").read_text())


def test_D04_zero_by_two_routes_and_factorisation():
    d4 = cj("Q02_bulk_graphs")["D04"]
    assert d4["value"] == "0" and d4["routes_agree"] and d4["route1_A"] == "0" and d4["route2_IBP_A"] == "0"
    assert d4["N_proportional_to_p2_times_scalar_denominator"] is True


def test_D04_hand_contraction_live():
    import sympy as sp
    p, q, pq = sp.symbols("p2 q2 pq")
    # T.P.T' = 2(p.p')(q.q') + 2(p.q')(q.p') - 2(p.q)(p'.q'), p' = -p, q' = -q
    val = 2 * (-p) * (-q) + 2 * (-pq) * (-pq) - 2 * pq * pq
    assert sp.expand(val - 2 * p * q) == 0


def test_D06_equals_XVI_P_and_poles():
    d6 = cj("Q02_bulk_graphs")["D06"]
    assert d6["equals_XVI_P_P15_exactly"] and d6["fit_exact"]
    assert {k: v["UV_pole"] for k, v in d6["laurent"].items()} == {"S1_delta^rho_nu_p4": "-3/8", "S2_p^rho_p_nu_p2": "1/4",
                                                                   "SE_Gt^rho_nu_p4": "-1/8"}


def test_tadpoles_derived_no_log():
    tp = cj("Q02_bulk_graphs")["tadpoles"]
    assert tp["D05a"]["omegas"] == [2] and tp["D12"]["omegas"] == [2, 4]
    assert not tp["D05a"]["logarithmic_member"] and not tp["D12"]["logarithmic_member"]


def test_D12_reconciliation():
    r = cj("Q02_D12_reconciliation")
    assert r["consistent"] and r["XVI_P_p23_stale_line_present"] and "Missing" not in r["status"]


def test_bulk_identity_rows_and_closure():
    b = cj("Q02_bulk_identity")
    assert b["n_rows"] == 23 and b["unclassified_rows"] == [] and b["bulk_identity_outcome"] == "cancellable"
    assert b["rows"]["phi.phi"]["outcome"] == "zero" and b["rows"]["b.c"]["class"] == "ST4b_ghost_equation"
    assert b["sigma0_closure"]["closed"] and b["ST4a"]["cbc"]["SE_pole_plus_p22_CT_zero"]


def test_O1_walk_and_status():
    o = cj("Q02_O1")
    assert o["all_earned"] and o["O1_closure"] == "CLOSED_PENDING_INDEPENDENT_REVIEW" and len(o["items"]) == 6
    assert o["O1_closure_confirmed"] is False and len(o["closure_reasons_answered"]) == 8
    s = json.loads((Q / "certificates" / "Q02_release.json").read_text())   # Q02 layer record (RELEASE_STATUS is Q03's)
    assert s["O1_closure"] == "CLOSED_PENDING_INDEPENDENT_REVIEW" and s["O1_closure_confirmed"] is False
    assert s["bulk_symmetry_restored"] is False and "QdS" in s["O1_closure_scope"]


def test_D04_route4_live():
    sys.path.insert(0, str(Q / "code" / "q02"))
    import d04_route4 as R4
    r = R4.main()
    assert r["declared_zero"] and r["all_mutations_nonzero"] and r["T3_zero_by_Ward_identity"]
    assert r == {k: v for k, v in cj("Q02_D04_scrutiny").items() if k in r}


def test_enumeration_and_CTs():
    b = cj("Q02_bulk_identity")
    assert b["independent_graph_enumeration"]["n_E2"] == 9 and b["independent_graph_enumeration"]["n_E1"] == 3
    c = b["bulk_counterterms_explicit"]
    assert c["ok"] and c["coefficients"]["h.h#0"] == "49*I/480" and c["coefficients"]["phi.phi#0"] == "0"


def test_guard_apart_bypass_live():
    code = ("import sys; sys.path.insert(0, %r)\n" % str(Q / "code" / "q01") +
            "import qguard, sympy as sp\nd = qguard.d\n"
            "try:\n    sp.apart((d**2 - 1)/(d - 4), d); print('NOT')\nexcept qguard.PrematureD4Substitution:\n    print('R')\n")
    r = subprocess.run([sys.executable, "-c", code], capture_output=True, text=True, timeout=300)
    assert r.stdout.strip() == "R", r.stdout + r.stderr


def _copy():
    d = Path(tempfile.mkdtemp(prefix="q02gate_"))
    for sub in ("code", "certificates", "intake", "registration"):
        shutil.copytree(Q / sub, d / sub, ignore=shutil.ignore_patterns("__pycache__"))
    (d / "build" / "xvip_run" / "certificates").mkdir(parents=True)
    shutil.copy(Q / "build/xvip_run/certificates/P13_master_kappa2.json", d / "build/xvip_run/certificates/")
    return d
```

```
def _edit(d, n, fn):
    f = d / "certificates" / f"{n}.json"
    c = json.loads(f.read_text())
    fn(c)
    f.write_text(json.dumps(c))


def _jedit(path, fn):
    c = json.loads(path.read_text())
    fn(c)
    path.write_text(json.dumps(c))


MUTS = {
    "O2c_cell_false": lambda d: _edit(d, "Q01_O2", lambda c: c["items"]["c_ghost_antifield"]["kappa2_table"].__setitem__("af2_transformed", False)),
    "O2c_table_replaced": lambda d: _edit(d, "Q01_O2", lambda c: c["items"]["c_ghost_antifield"].__setitem__("kappa2_table", {"x": True})),
    "O2c_build_input_cell": lambda d: _jedit(d / "build/xvip_run/certificates/P13_master_kappa2.json",
                                             lambda c: c["table_by_kappa_and_antifield_number"]["kappa1"].__setitem__("af1_invariant", False)),
    "per_file_pin_false": lambda d: _edit(d, "Q01_intake", lambda c: c["pins"].__setitem__(next(iter(c["pins"])), False)),
    "expected_file_edit": lambda d: (d / "code/q02/q02_expected.json").write_text((d / "code/q02/q02_expected.json").read_text().replace("-3/8",
"-1/8", 1)),
    "D04_value": lambda d: _edit(d, "Q02_bulk_graphs", lambda c: c["D04"].__setitem__("route1_A", "1/2")),
    "D06_pole": lambda d: _edit(d, "Q02_bulk_graphs", lambda c: c["D06"]["laurent"]["S2_p^rho_p_nu_p2"].__setitem__("UV_pole", "1/3")),
    "D06_local_value": lambda d: _edit(d, "Q02_bulk_graphs", lambda c: c["D06"]["laurent"]["S1_delta^rho_nu_p4"].__setitem__("local", "-1/3")),
    "row_outcome": lambda d: _edit(d, "Q02_bulk_identity", lambda c: c["rows"]["phi.phi"].__setitem__("outcome", "obstruction")),
    "O1_item_unearned": lambda d: _edit(d, "Q02_O1", lambda c: c["items"][3].__setitem__("earned", False)),
    "errata_E2_severity": lambda d: _jedit(d / "intake/errata_p23.json", lambda c: c["errata"][1].__setitem__("severity", "info")),
    "registration_reverted": lambda d: _jedit(d / "registration/local_registration.json", lambda c: c.__setitem__("how_found", "old")),
    "hh_per_graph_ct_edit": lambda d: _edit(d, "Q02_bulk_identity", lambda c: c["bulk_counterterms"]["cbc"].__setitem__("SE", "1/4")),
    "D04_scrutiny_mutation_zeroed": lambda d: _edit(d, "Q02_D04_scrutiny", lambda c: c["cases"]["MUT non-de Donder gauge T4 piece (beta != -2),
g4=1"].__setitem__("value_over_master", "0")),
    "O1_label_confirmed_early": lambda d: _edit(d, "Q02_O1", lambda c: c.__setitem__("O1_closure_confirmed", True)),
    "CT_phiphi_edit": lambda d: _edit(d, "Q02_bulk_identity", lambda c: c["bulk_counterterms_explicit"]["coefficients"].__setitem__("phi.phi#0",
"1/2")),
    "conventions_edit": lambda d: (d / "code/q01/q01_conventions.py").write_text((d / "code/q01/q01_conventions.py").read_text() + "\n# edit\n"),
}


@pytest.mark.parametrize("name", sorted(MUTS))
def test_gate_rejects_without_stale_status(name):
    d = _copy()
    try:
        MUTS[name](d)
        r = subprocess.run([sys.executable, str(d / "code/q02/release_q02.py")], capture_output=True, text=True)
        assert r.returncode == 1, r.stdout
        assert not (d / "RELEASE_STATUS.json").exists() and not (d / "certificates/Q02_release.json").exists()
    finally:
        shutil.rmtree(d, ignore_errors=True)


def test_gate_accepts_unmodified_copy():
    d = _copy()
    try:
        r = subprocess.run([sys.executable, str(d / "code/q02/release_q02.py")], capture_output=True, text=True)
        assert r.returncode == 0, r.stdout
    finally:
        shutil.rmtree(d, ignore_errors=True)
```

tests/test_q03.py

```
"""XVI-Q Q03 tests: O.1 record content, ring/field guard (live), gate fail-closed on corrections/provenance edits."""
import json, shutil, subprocess, sys  # noqa: E401
from pathlib import Path
import pytest

Q = Path(__file__).resolve().parents[1]
sys.path.insert(0, str(Q / "code" / "q01"))


def jl(p):
    return json.loads((Q / p).read_text())


def test_record():
    r = jl("certificates/Q03_O1_record.json")
    assert r["O1_closure"] == "CLOSED (scoped)" and r["O1_closure_confirmed"] is True and r["confirmation"]["binding_ok"]
    assert r["counterterm_domain"]["n_enlarged_domain"] == 21
    assert not list(r["closure_reasons"].values())[7]["status"].startswith("ANSWERED")
    assert len(r["caveats"]) == 4 and len(r["not_implied"]) == 6


def test_ring_field_guard_live():
    import qguard as G
```

```
    import sympy as sp
    from sympy import QQ
    from sympy.polys.rings import ring
    from sympy.polys.fields import field
    R, D_, x = ring([G.d, sp.Symbol("X")], QQ)
    F, Df, _ = field([G.d, sp.Symbol("X")], QQ)
    for f in (lambda: (D_ ** 2).evaluate(D_, 4), lambda: (D_ - 1)(4), lambda: ((Df - 1) / Df).subs(Df, 4)):
        with pytest.raises(G.PrematureD4Substitution):
            f()
    assert sp.cancel((G.d ** 2 - 16) / (G.d - 4)) == G.d + 4


def _copy(tmp):
    dst = tmp / "XVI-Q"
    top = {"build", "dist", "archival", "manuscript"}
    shutil.copytree(Q, dst, ignore=lambda d, names: [n for n in names if n == "__pycache__" or (Path(d) == Q and n in top)])
    (dst / "build" / "xvip_run").mkdir(parents=True)
    return dst


def _gate(root):
    return subprocess.run([sys.executable, "code/q03/release_q03.py"], cwd=root, capture_output=True, text=True)


def _edit_json(root, rel, fn):
    p = root / rel
    o = json.loads(p.read_text())
    fn(o)
    p.write_text(json.dumps(o, indent=1) + "\n")


MUT = {
    "provenance_P19_edit": lambda r: (r / "provenance/XVI-P/certificates/P19_defect.json").write_text(
        (r / "provenance/XVI-P/certificates/P19_defect.json").read_text().replace("ZERO", "NONZERO", 1)),
    "unpinned_provenance_file": lambda r: (r / "provenance/XVI-O/extra.json").write_text("{}"),
    "reason8_marked_answered": lambda r: _edit_json(r, "certificates/Q03_O1_record.json", lambda o: list(
        o["closure_reasons"].values())[7].update(status="ANSWERED")),
    "domain_shrunk": lambda r: _edit_json(r, "certificates/Q03_O1_record.json", lambda o: o["counterterm_domain"].update(n_p22_evanescent=0)),
    "caveat_dropped": lambda r: _edit_json(r, "certificates/Q03_O1_record.json", lambda o: o["caveats"].pop(1)),
    "report_edited": lambda r: (r / "reproducibility/independent_rebuild_report_q02.json").write_text(
        (r / "reproducibility/independent_rebuild_report_q02.json").read_text() + " "),
    "qguard_edited": lambda r: (r / "code/q01/qguard.py").write_text((r / "code/q01/qguard.py").read_text() + "\n#x\n"),
    "expected_edited": lambda r: (r / "code/q03/q03_expected.json").write_text((r / "code/q03/q03_expected.json").read_text() + " "),
}


@pytest.mark.parametrize("name", sorted(MUT))
def test_gate_rejects(tmp_path, name):
    root = _copy(tmp_path)
    MUT[name](root)
    r = _gate(root)
    assert r.returncode == 1, r.stdout
    assert not (root / "RELEASE_STATUS.json").exists()


def test_gate_accepts_unmodified(tmp_path):
    r = _gate(_copy(tmp_path))
    assert r.returncode == 0, r.stdout
```

tests/test_q04.py

```
"""XVI-Q Q04 tests: engine validation (live, fast), inventory, gate fail-closed on honesty / value edits."""
import json, shutil, subprocess, sys  # noqa: E401
from pathlib import Path
import pytest

Q = Path(__file__).resolve().parents[1]
sys.path.insert(0, str(Q / "code" / "q04"))


def jl(p):
    return json.loads((Q / p).read_text())


def test_inventory_and_vertex():
    import p6_inventory as P
    i = P.inventory()
    assert i["equals_XVI_P_P23_15"] and i["n_E3_tadpoles"] == 3 and i["power_counting"]["verified_on_all_enumerated_graphs"]
    assert P.vertex_hhhphiphi_check() and not P.vertex_hhhphiphi_check(mutate=True)


def test_engine_d11_live():
    sys.path.insert(0, str(Q / "code" / "q04"))
```

```
    import q04_certs as C
    r = C.task_d11()
    assert r["ratio_mine_over_XVI_P"] == "I" and r["constant_ratio"]


def test_status_honest():
    s = jl("certificates/Q04_P6_status.json")
    assert s["P6_outcome"] == "Partial" and s["stages"]["counterterm_solve_kappa3"] == "Missing"
    e = jl("certificates/Q04_P6_eval.json")
    assert e["n_missing"] == 13 and e["n_evaluated_pole"] == 2 and e["n_derived_zero"] == 3


def _copy(tmp):
    dst = tmp / "XVI-Q"
    top = {"build", "dist", "archival", "manuscript"}
    shutil.copytree(Q, dst, ignore=lambda d, names: [n for n in names if n == "__pycache__" or (Path(d) == Q and n in top)])
    return dst


def _edit(root, rel, fn):
    p = root / rel
    o = json.loads(p.read_text())
    fn(o)
    p.write_text(json.dumps(o, indent=1) + "\n")


MUT = {
    "claim_closure": lambda r: _edit(r, "certificates/Q04_P6_status.json", lambda o: o["stages"].update(sigma0_closure_kappa3="closed")),
    "missing_to_zero": lambda r: _edit(r, "certificates/Q04_P6_eval.json", lambda o: o["graph_table"][0].update(status="derived 0")),
    "ward_flip": lambda r: _edit(r, "certificates/Q04_P6_eval.json", lambda o:
o["matter_ward_three_point"]["declared"].update(residue_d4_zero=False)),
    "d11_ratio": lambda r: _edit(r, "certificates/Q04_P6_eval.json", lambda o: o["validation_D11"].update(ratio_mine_over_XVI_P="2*I")),
    "outcome_full": lambda r: _edit(r, "certificates/Q04_P6_status.json", lambda o: o.update(P6_outcome="Constructed")),
    "code_edit": lambda r: (r / "code/q04/uvpole.py").write_text((r / "code/q04/uvpole.py").read_text() + "\n#x\n"),
}


@pytest.mark.parametrize("name", sorted(MUT))
def test_gate_rejects(tmp_path, name):
    root = _copy(tmp_path)
    MUT[name](root)
    r = subprocess.run([sys.executable, "code/q04/release_q04.py"], cwd=root, capture_output=True, text=True)
    assert r.returncode == 1, r.stdout
    assert not (root / "RELEASE_STATUS.json").exists()


def test_gate_accepts(tmp_path):
    r = subprocess.run([sys.executable, "code/q04/release_q04.py"], cwd=_copy(tmp_path), capture_output=True, text=True)
    assert r.returncode == 0, r.stdout


def test_guard_scan_and_pins():
    import guard_scan as g
    r = g.report()
    assert r["scan_clean"] and r["probes"]["python_term_walk"].startswith("BYPASS")
    exp = json.loads((Q / "code/q04/q04_expected.json").read_text())
    assert "tools/source_set.py" in exp["file_sha256"]


def test_gate_rejects_source_set_edit(tmp_path):
    root = _copy(tmp_path)
    (root / "tools/source_set.py").write_text((root / "tools/source_set.py").read_text() + "\n#x\n")
    r = subprocess.run([sys.executable, "code/q04/release_q04.py"], cwd=root, capture_output=True, text=True)
    assert r.returncode == 1


def test_gate_rejects_escape_primitive(tmp_path):
    root = _copy(tmp_path)
    (root / "code/q04/uvpole.py").write_text((root / "code/q04/uvpole.py").read_text() + "\n_x = __import__('sympy').srepr(1)\n")
    r = subprocess.run([sys.executable, "code/q04/release_q04.py"], cwd=root, capture_output=True, text=True)
    assert r.returncode == 1
```

tests/test_q05.py

```
#!/usr/bin/env python3
"""XVI-Q Q05 tests: hardened scan, gate fail-closed on value / honesty edits and on unpinned code files."""
import json, shutil, subprocess, sys  # noqa: E401
from pathlib import Path
import pytest

Q = Path(__file__).resolve().parents[1]
sys.path.insert(0, str(Q / "code" / "p6q05"))
```

```
def _copy(tmp):
    dst = tmp / "XVI-Q"
    top = {"build", "dist", "archival", "manuscript"}
    shutil.copytree(Q, dst, ignore=lambda d, names: [n for n in names if n == "__pycache__" or (Path(d) == Q and n in top)])
    return dst


def _edit(root, rel, fn):
    p = root / rel
    o = json.loads(p.read_text())
    fn(o)
    p.write_text(json.dumps(o, indent=1) + "\n")


def _gate(root):
    return subprocess.run([sys.executable, "code/p6q05/release_q05.py"], cwd=root, capture_output=True, text=True)


MUT = {
    "row_residual": lambda r: _edit(r, "certificates/Q05_P6_rows.json", lambda o: o["c.phi.phi"].update(residual="x")),
    "ghost_row_closed": lambda r: _edit(r, "certificates/Q05_P6_rows.json",
                                        lambda o: o["ghost_rows"]["c.c.K"].update(loop_residual_zero=True)),
    "claim_closure": lambda r: _edit(r, "certificates/Q05_P6_status.json",
                                     lambda o: o.update(sigma0_closure_kappa3="closed")),
    "missing_to_zero": lambda r: _edit(r, "certificates/Q05_P6_graphs.json", lambda o: o["missing"].pop("G01")),
    "D13_ratio": lambda r: _edit(r, "certificates/Q05_rules_validation.json",
                                 lambda o: o["validations"]["D13"].update(ratio="I")),
    "pole_value": lambda r: _edit(r, "certificates/Q05_P6_graphs.json",
                                  lambda o: o["graphs"]["G12"].update(pole_over_U_d4_residue="0")),
    "unpinned_code_file": lambda r: (r / "code/p6q05/sub").mkdir() or (r / "code/p6q05/sub/extra.py").write_text("x = 1\n"),
    "unpinned_tool": lambda r: (r / "tools/extra_tool.py").write_text("x = 1\n"),
    "escape_primitive": lambda r: (r / "code/p6q05/graphs.py").write_text((r / "code/p6q05/graphs.py").read_text()
                                                                          + "\nfrom sympy import lambdify as _f\n"),
}


@pytest.mark.parametrize("name", sorted(MUT))
def test_gate_rejects(tmp_path, name):
    root = _copy(tmp_path)
    MUT[name](root)
    r = _gate(root)
    assert r.returncode != 0
    assert not (root / "RELEASE_STATUS.json").exists()


def test_gate_accepts(tmp_path):
    r = _gate(_copy(tmp_path))
    assert r.returncode == 0, r.stdout


def test_scan2_flags_reviewer_forms():
    import guard_scan2 as g
    r = g.report()
    assert r["all_reviewer_evasions_flagged"] and r["scan_clean"] and "NOT a security boundary" in r["claim"]
```

tests/test_q06.py

```
#!/usr/bin/env python3
"""XVI-Q Q06 tests: gate fail-closed on value / honesty edits and on unpinned code files; fast path sanity."""
import json, shutil, subprocess, sys  # noqa: E401
from pathlib import Path
import pytest

Q = Path(__file__).resolve().parents[1]


def _copy(tmp):
    dst = tmp / "XVI-Q"
    top = {"build", "dist", "archival", "manuscript"}
    shutil.copytree(Q, dst, ignore=lambda d, names: [n for n in names if n == "__pycache__" or (Path(d) == Q and n in top)])
    return dst


def _edit(root, rel, fn):
    p = root / rel
    o = json.loads(p.read_text())
    fn(o)
    p.write_text(json.dumps(o, indent=1) + "\n")


def _gate(root):
    return subprocess.run([sys.executable, "code/p6q06/release_q06.py"], cwd=root, capture_output=True, text=True)
```

```
def _gr(o, t, **kw):
    o[t].update(**kw)


MUT = {
    "row_4D_residual": lambda r: _edit(r, "certificates/Q06_ghost_rows.json", lambda o: _gr(o, "c.c.K", residual_4D="x")),
    "evanescent_claimed_closed": lambda r: _edit(r, "certificates/Q06_ghost_rows.json", lambda o: o.update(status="CLOSED")),
    "K_sign_back_to_Q05": lambda r: _edit(r, "certificates/Q06_ghost_rows.json",
                                          lambda o: o["c.c.cbar"]["channel_coefficients"].update(K1="1")),
    "mutation_undetected": lambda r: _edit(r, "certificates/Q06_ghost_rows.json",
                                           lambda o: o["c.c.K"]["mutations_detected"].update(drop_c1=False)),
    "V4_gauge": lambda r: _edit(r, "certificates/Q06_V4_validation.json", lambda o: o.update(gauge_n3_V4_vs_V3_zero=False)),
    "G01_value": lambda r: _edit(r, "certificates/Q06_P6_hhh.json",
                                 lambda o: o["graphs"]["G01"].update(pole_over_U_d4_residue="0")),
    "claim_closure": lambda r: _edit(r, "certificates/Q06_P6_status.json", lambda o: o.update(sigma0_closure_kappa3="closed")),
    "hhc_claimed": lambda r: _edit(r, "certificates/Q06_P6_status.json", lambda o: o["stages"].update(row_h_h_c="closed")),
    "unpinned_code_file": lambda r: (r / "code/p6q06/extra.py").write_text("x = 1\n"),
    "q05_gate_edit": lambda r: (r / "code/p6q05/release_q05.py").write_text((r / "code/p6q05/release_q05.py").read_text() + "\n"),
}


@pytest.mark.parametrize("name", sorted(MUT))
def test_gate_rejects(tmp_path, name):
    root = _copy(tmp_path)
    MUT[name](root)
    r = _gate(root)
    assert r.returncode != 0
    assert not (root / "RELEASE_STATUS.json").exists()


def test_gate_accepts(tmp_path):
    r = _gate(_copy(tmp_path))
    assert r.returncode == 0, r.stdout


def test_fast_trunc_exact():
    sys.path.insert(0, str(Q / "code" / "p6q06"))
    import fastpole as FP
    import uvpole as U
    F = FP.Fast(["p1", "a1"])
    e = U.t ** 2 * U.H("a1", "p1") + U.t ** 5 * U.H("p1", "p1")
    assert F.back(F.trunc(F.to(e), 4)) == U.t ** 2 * U.H("a1", "p1")
```

tools/build.py

```
#!/usr/bin/env python3
"""XVI-Q Q04 full build (Q04 = P.6 entry layer: q04 certs, Q03 gate as layer gate, Q04 gate owns RELEASE_STATUS).  Q03: (Q03 = correction layer on
Q02: Q03 O.1 record + ring/field guard certs, Q02 gate run as layer gate,
Q03 gate owns RELEASE_STATUS).  Q02 description: (from scratch): clean -> intake/extract -> classical re-execution of XVI-P p23 stages ->
7 master mutations -> Q01 certificates (o2_q01) -> Q02 certificates (bulk graphs, bulk identity, O.1 walk) ->
Q01 layer gate -> Q02 gate (value-checks every certified number) -> tests -> PDFs (science/appendix/combined) ->
zips (source-only/review-focus/full) -> external manifest LAST -> OPTIONAL sync (Q02: default NONE; release opt-in: --sync <dir> or Q_SYNC_DIR)."""
from __future__ import annotations
import hashlib, json, os, shutil, subprocess, sys, time, zipfile  # noqa: E401
from concurrent.futures import ThreadPoolExecutor
from pathlib import Path

Q = Path(__file__).resolve().parents[1]
sys.path.insert(0, str(Q / "tools"))
import source_set as SS  # noqa: E402

PY = sys.executable
SYNC = (sys.argv[sys.argv.index("--sync") + 1] if "--sync" in sys.argv else os.environ.get("Q_SYNC_DIR", "none"))  # Q02: NO sync by default
RUN = Q / "build" / "xvip_run"
STAGES = ["code/classical/inherited.py", "code/classical/boundary_identity.py", "code/classical/quartic.py",
          "code/classical/brst_from_geometry.py", "code/classical/gauge_fixing_map.py", "code/classical/master_kappa2.py"]
MUTS = ["flip_sigma_c", "drop_cstar", "alter_gf_shift", "flip_odd_derivative_sign", "corrupt_a2", "corrupt_ell4",
        "drop_s1_transport"]
PDFS = ["XVI-Q-science", "XVI-Q-appendix", "XVI-Q"]


def sh(p):
    return hashlib.sha256(Path(p).read_bytes()).hexdigest()


def step(name, cmd, cwd=Q, env=None):
    t0 = time.time()
    r = subprocess.run(cmd, cwd=cwd, capture_output=True, text=True, env=env)
    print(f"[{name}] rc={r.returncode} {time.time() - t0:.1f}s {r.stdout.strip()[-300:]}", flush=True)
    if r.returncode != 0:
        print(r.stdout[-3000:], r.stderr[-3000:])
```

```
        raise SystemExit(f"stage {name} failed")
    return r


def clean():
    for d in ("certificates", "ledgers", "dist", "imports"):
        shutil.rmtree(Q / d, ignore_errors=True)
    if (Q / "build").is_dir():                      # keep build/logs (persistent logs), wipe everything else
        for x in (Q / "build").iterdir():
            if x.name != "logs":
                shutil.rmtree(x) if x.is_dir() else x.unlink()
    for f in ["RELEASE_STATUS.json", "RELEASE_MANIFEST_EXTERNAL.txt"] + [f"manuscript/{p}.pdf" for p in PDFS]:
        (Q / f).unlink(missing_ok=True)
    for d in ("certificates", "ledgers", "dist", "build", "imports"):
        (Q / d).mkdir(exist_ok=True)


def extract():
    z = zipfile.ZipFile(Q / "provenance/XVI-P/XVI-P-p23-source-only.zip")
    assert sh(Q / "provenance/XVI-P/XVI-P-p23-source-only.zip") == \
        "3f4a88db462923553272e8a41407c0c06aacbc431736efdfeda10c27aae8a177"
    z.extractall(Q / "imports")
    (Q / "imports" / "XVI-P").rename(Q / "imports" / "XVI-P-p23")
    shutil.copytree(Q / "imports" / "XVI-P-p23", RUN)
    for d in ("certificates", "ledgers", "exports", "build/p14"):
        (RUN / d).mkdir(parents=True, exist_ok=True)


def classical():
    for s in STAGES:
        step(Path(s).stem, [PY, s], cwd=RUN)


def mutations():
    def one(m):
        d = Q / "build" / f"mut_{m}"
        shutil.copytree(RUN, d)
        return m, subprocess.run([PY, "code/classical/master_kappa2.py"], cwd=d, capture_output=True, text=True,
                                 env=dict(os.environ, P13_MUTATIONS=m)).returncode
    with ThreadPoolExecutor(7) as ex:
        res = list(ex.map(one, MUTS))
    print("[master_mutations]", res, flush=True)


def tex_escape_path(s):
    return s.replace("_", r"\_")


def manuscript():
    ms = Q / "manuscript"
    status = json.loads((Q / "RELEASE_STATUS.json").read_text())
    pre = (r"\documentclass[10pt]{article}\usepackage[margin=2cm]{geometry}\usepackage{amsmath,amssymb}"
           r"\usepackage{fancyhdr}\pagestyle{fancy}\fancyfoot[C]{XVI-Q Q04 \textbar{} page \thepage}"
           r"\usepackage{hyperref}\begin{document}")
    title = (r"\title{Landmark XVI-Q --- Q04 (P.6 entry, honest partial)\\\large One-loop $\kappa^3$ three-point sector: "
             r"inventory (15 + 3 scaleless tadpoles), $\kappa^3$ identity ledger, validated UV-pole engine, scalar-loop hhh "
             r"evaluated with the matter Ward identity verified at the pole; 13 graphs Missing; $\kappa^3$ CT/closure Missing. "
             r"0.1 CLOSED (scoped, $\kappa^2$ two-point) unchanged}"
             r"\date{2026-10-11}\maketitle\thispagestyle{fancy}\tableofcontents\newpage")
    sci = r"\input{notes/Q04_science.tex}\newpage\section*{Q03 layer}\addcontentsline{toc}{section}{Q03
layer}\input{notes/Q03_science.tex}\newpage\section*{Q02 layer (science; 0.1 record superseded by Q03)}\addcontentsline{toc}{section}{Q02
layer}\input{notes/Q02_science.tex}\newpage\section*{Q01 layer (unchanged science)}\addcontentsline{toc}{section}{Q01
layer}\input{notes/Q01_science.tex}"
    app = [r"\section{Appendix: certificates, status, intake}"]
    for f in ["RELEASE_STATUS.json", "certificates/Q04_P6_status.json", "certificates/Q04_P6_inventory.json",
              "certificates/Q04_P6_identity.json", "certificates/Q04_P6_eval.json", "certificates/Q03_O1_record.json",
"certificates/Q03_guard_ring.json",
              "ledgers/Q03_milestones.json", "certificates/Q02_O1.json", "certificates/Q02_bulk_identity.json",
              "certificates/Q02_bulk_graphs.json", "certificates/Q02_D12_reconciliation.json", "certificates/Q02_D04_scrutiny.json",
"ledgers/Q02_milestones.json",
              "certificates/Q01_O2.json", "certificates/Q01_guard.json",
              "certificates/Q01_intake.json", "ledgers/Q01_milestones.json", "intake/errata_p23.json",
              "registration/local_registration.json"]:
        app += [r"\subsection*{" + tex_escape_path(f) + "}", r"{\scriptsize\begin{verbatim}",
                SS.wrap((Q / f).read_text()), r"\end {verbatim}}"]
    app += [r"\section{Appendix: source listings}"]
    for f in SS.source_files(Q):
        if f.endswith(".py"):
            app += [r"\subsection*{" + tex_escape_path(f) + "}", r"{\scriptsize\begin{verbatim}",
                    SS.wrap((Q / f).read_text()), r"\end {verbatim}}"]
    (ms / "body_appendix.tex").write_text("\n".join(app))
    docs = {"XVI-Q-science": sci, "XVI-Q-appendix": r"\input{body_appendix.tex}",
            "XVI-Q": sci + r"\newpage" + r"\input{body_appendix.tex}"}
    for name, body in docs.items():
```

```
        (ms / f"{name}.tex").write_text(pre + title + body + r"\end{document}")
        for _ in range(2):
            r = subprocess.run(["pdflatex", "-interaction=nonstopmode", "-halt-on-error", f"{name}.tex"], cwd=ms,
                               capture_output=True, text=True)
        if r.returncode != 0:
            print(r.stdout[-3000:])
            raise SystemExit(f"pdflatex {name} failed")
    for p in ms.glob("*.aux"):
        p.unlink()
    print("[pdfs]", {p: subprocess.run(["pdfinfo", f"manuscript/{p}.pdf"], cwd=Q, capture_output=True,
                                       text=True).stdout.split("Pages:")[1].split()[0] for p in PDFS}, flush=True)
    return status


def zips():
    dist = Q / "dist"
    SS.make_zip(dist / "XVI-Q-Q04-source-only.zip", Q)
    gen = ["RELEASE_STATUS.json"] + [p.relative_to(Q).as_posix() for p in sorted((Q / "certificates").glob("*.json"))] + \
          [p.relative_to(Q).as_posix() for p in sorted((Q / "ledgers").glob("*.json"))] + [f"manuscript/{p}.pdf" for p in PDFS]
    with zipfile.ZipFile(dist / "XVI-Q-review-focus.zip", "w", zipfile.ZIP_DEFLATED) as z:
        for f in gen + ["README.md", "intake/errata_p23.json", "registration/local_registration.json"]:
            z.write(Q / f, f"XVI-Q/{f}")
    with zipfile.ZipFile(dist / "XVI-Q-full.zip", "w", zipfile.ZIP_DEFLATED) as z:
        for f in sorted(set(SS.source_files(Q)) | set(gen)):
            z.write(Q / f, f"XVI-Q/{f}")


def manifest():
    files = [f"manuscript/{p}.pdf" for p in PDFS] + ["dist/XVI-Q-Q04-source-only.zip", "dist/XVI-Q-review-focus.zip",
                                                     "dist/XVI-Q-full.zip", "RELEASE_STATUS.json"] + \
            [p.relative_to(Q).as_posix() for p in sorted((Q / "certificates").glob("Q0*.json"))]
    lines = ["# XVI-Q Q04 external manifest (written LAST; PDFs never contain these hashes)"]
    for f in files:
        pages = ""
        if f.endswith(".pdf"):
            pages = " pages=" + subprocess.run(["pdfinfo", f], cwd=Q, capture_output=True, text=True).stdout.split("Pages:")[1].split()[0]
        lines.append(f"{sh(Q / f)}  {f}{pages}")
    lines.append(f"source_tree_sha256 {SS.tree_sha(Q)}")
    (Q / "RELEASE_MANIFEST_EXTERNAL.txt").write_text("\n".join(lines) + "\n")
    for f in [f"manuscript/{p}.pdf" for p in PDFS]:            # no manifest hash inside any PDF
        txt = subprocess.run(["pdftotext", f, "-"], cwd=Q, capture_output=True, text=True).stdout
        assert not any(l.split()[0] in txt for l in lines[1:-1]), f


def sync():
    if SYNC == "none":
        return
    top = Path(SYNC)
    for p in PDFS:
        shutil.copy(Q / "manuscript" / f"{p}.pdf", top / f"{p}.pdf")
    for z in ("XVI-Q-Q04-source-only.zip", "XVI-Q-review-focus.zip", "XVI-Q-full.zip"):
        shutil.copy(Q / "dist" / z, top / z)
    print(f"[sync] {top}", flush=True)


def main():
    clean()
    extract()
    classical()
    mutations()
    step("certificates", [PY, "code/q01/o2_q01.py"])
    step("certificates_q02", [PY, "code/q02/q02_certs.py"])
    step("gate_q01_layer", [PY, "code/q01/release_q01.py"])
    step("certificates_q03", [PY, "code/q03/q03_certs.py"])
    step("gate_q02_layer", [PY, "code/q02/release_q02.py", "--layer"])
    step("certificates_q04", [PY, "code/q04/q04_certs.py"])
    step("gate_q03_layer", [PY, "code/q03/release_q03.py", "--layer"])
    step("gate", [PY, "code/q04/release_q04.py"])
    step("tests", [PY, "-m", "pytest", "-q", "-p", "no:cacheprovider", "tests"])
    manuscript()
    zips()
    manifest()
    sync()
    print("BUILD_OK", flush=True)


if __name__ == "__main__":
    main()
```

tools/build_q05.py

```
#!/usr/bin/env python3
"""XVI-Q Q05 full build (from scratch).  Reuses the helpers of tools/build.py (pinned by the Q04 layer gate, unchanged):
```

```
clean -> extract -> classical -> mutations -> Q01..Q04 certificates -> Q01..Q03 layer gates -> Q04 gate (layer record)
-> Q05 certificates -> Q05 gate (owns RELEASE_STATUS) -> tests -> PDFs -> zips -> manifest LAST -> optional sync
(default none; release opt-in: --sync <dir> or Q_SYNC_DIR)."""
from __future__ import annotations
import hashlib, json, os, shutil, subprocess, sys, zipfile  # noqa: E401
from pathlib import Path

Q = Path(__file__).resolve().parents[1]
sys.path.insert(0, str(Q / "tools"))
import build as B  # noqa: E402
import source_set_q05 as SS  # noqa: E402

PY = sys.executable
SYNC = (sys.argv[sys.argv.index("--sync") + 1] if "--sync" in sys.argv else os.environ.get("Q_SYNC_DIR", "none"))
PDFS = ["XVI-Q-science", "XVI-Q-appendix", "XVI-Q"]
ZIP = "XVI-Q-Q05-source-only.zip"
NOTES = ["Q05", "Q04", "Q03", "Q02", "Q01"]
APPX = ["RELEASE_STATUS.json", "certificates/Q05_P6_status.json", "certificates/Q05_P6_rows.json",
        "certificates/Q05_rules_validation.json", "certificates/Q05_P6_graphs.json", "certificates/Q05_guard_scan.json",
        "ledgers/Q05_milestones.json", "certificates/Q04_release.json", "certificates/Q04_P6_status.json",
        "certificates/Q04_P6_inventory.json", "certificates/Q04_P6_identity.json", "certificates/Q04_P6_eval.json",
        "certificates/Q03_O1_record.json", "certificates/Q02_bulk_identity.json", "certificates/Q01_O2.json",
        "intake/errata_p23.json", "registration/local_registration.json"]


def sh(p):
    return hashlib.sha256(Path(p).read_bytes()).hexdigest()


def manuscript():
    ms = Q / "manuscript"
    pre = (r"\documentclass[10pt]{article}\usepackage[margin=2cm]{geometry}\usepackage{amsmath,amssymb}"
           r"\usepackage{fancyhdr}\pagestyle{fancy}\fancyfoot[C]{XVI-Q Q05 \textbar{} page \thepage}"
           r"\usepackage{hyperref}\begin{document}")
    title = (r"\title{Landmark XVI-Q --- Q05 (P.6 continued, honest partial)\\\large One-loop $\kappa^3$ three-point "
             r"sector: own gravity/ghost rules validated vs XVI-P (D01, D03, D06, D13), 11 more graphs at pole level, "
             r"row c$\phi\phi$ closed at the pole; ghost rows open; G01/G02 Missing; $\kappa^3$ $\sigma_0$ closure not "
             r"established. O.1 CLOSED (scoped, $\kappa^2$ two-point) unchanged}"
             r"\date{2026-10-11}\maketitle\thispagestyle{fancy}\tableofcontents\newpage")
    parts = [r"\input{notes/Q05_science.tex}"]
    for n in NOTES[1:]:
        parts.append(r"\newpage\section*{" + n + r" layer}\addcontentsline{toc}{section}{" + n + r" layer}\input{notes/"
                     + n + r"_science.tex}")
    sci = "".join(parts)
    app = [r"\section{Appendix: certificates, status, intake}"]
    for f in APPX:
        app += [r"\subsection*{" + B.tex_escape_path(f) + "}", r"{\scriptsize\begin{verbatim}",
                SS.wrap((Q / f).read_text()), r"\end {verbatim}}"]
    app += [r"\section{Appendix: source listings}"]
    for f in SS.source_files(Q):
        if f.endswith(".py"):
            app += [r"\subsection*{" + B.tex_escape_path(f) + "}", r"{\scriptsize\begin{verbatim}",
                    SS.wrap((Q / f).read_text()), r"\end {verbatim}}"]
    (ms / "body_appendix.tex").write_text("\n".join(app))
    docs = {"XVI-Q-science": sci, "XVI-Q-appendix": r"\input{body_appendix.tex}",
            "XVI-Q": sci + r"\newpage" + r"\input{body_appendix.tex}"}
    for name, body in docs.items():
        (ms / f"{name}.tex").write_text(pre + title + body + r"\end{document}")
        for _ in range(2):
            r = subprocess.run(["pdflatex", "-interaction=nonstopmode", "-halt-on-error", f"{name}.tex"], cwd=ms,
                               capture_output=True, text=True)
        if r.returncode != 0:
            print(r.stdout[-3000:])
            raise SystemExit(f"pdflatex {name} failed")
    for p in ms.glob("*.aux"):
        p.unlink()
    print("[pdfs]", {p: subprocess.run(["pdfinfo", f"manuscript/{p}.pdf"], cwd=Q, capture_output=True,
                                       text=True).stdout.split("Pages:")[1].split()[0] for p in PDFS}, flush=True)


def zips():
    dist = Q / "dist"
    for old in dist.glob("*.zip"):
        old.unlink()
    SS.make_zip(dist / ZIP, Q)
    gen = ["RELEASE_STATUS.json"] + [p.relative_to(Q).as_posix() for p in sorted((Q / "certificates").glob("*.json"))] + \
          [p.relative_to(Q).as_posix() for p in sorted((Q / "ledgers").glob("*.json"))] + [f"manuscript/{p}.pdf" for p in PDFS]
    with zipfile.ZipFile(dist / "XVI-Q-review-focus.zip", "w", zipfile.ZIP_DEFLATED) as z:
        for f in gen + ["README.md", "intake/errata_p23.json", "registration/local_registration.json"]:
            z.write(Q / f, f"XVI-Q/{f}")
    with zipfile.ZipFile(dist / "XVI-Q-full.zip", "w", zipfile.ZIP_DEFLATED) as z:
        for f in sorted(set(SS.source_files(Q)) | set(gen)):
            z.write(Q / f, f"XVI-Q/{f}")
```

```
def manifest():
    files = [f"manuscript/{p}.pdf" for p in PDFS] + [f"dist/{ZIP}", "dist/XVI-Q-review-focus.zip", "dist/XVI-Q-full.zip",
                                                     "RELEASE_STATUS.json"] + \
            [p.relative_to(Q).as_posix() for p in sorted((Q / "certificates").glob("Q0*.json"))]
    lines = ["# XVI-Q Q05 external manifest (written LAST; PDFs never contain these hashes)"]
    for f in files:
        pages = ""
        if f.endswith(".pdf"):
            pages = " pages=" + subprocess.run(["pdfinfo", f], cwd=Q, capture_output=True, text=True).stdout.split("Pages:")[1].split()[0]
        lines.append(f"{sh(Q / f)}  {f}{pages}")
    lines.append(f"source_tree_sha256 {SS.tree_sha(Q)}")
    (Q / "RELEASE_MANIFEST_EXTERNAL.txt").write_text("\n".join(lines) + "\n")
    for f in [f"manuscript/{p}.pdf" for p in PDFS]:
        txt = subprocess.run(["pdftotext", f, "-"], cwd=Q, capture_output=True, text=True).stdout
        assert not any(ln.split()[0] in txt for ln in lines[1:-1]), f


def sync():
    if SYNC == "none":
        return
    top = Path(SYNC)
    for p in PDFS:
        shutil.copy(Q / "manuscript" / f"{p}.pdf", top / f"{p}.pdf")
    for z in (ZIP, "XVI-Q-review-focus.zip", "XVI-Q-full.zip"):
        shutil.copy(Q / "dist" / z, top / z)
    print(f"[sync] {top}", flush=True)


def main():
    B.clean()
    B.extract()
    B.classical()
    B.mutations()
    st = B.step
    st("certificates", [PY, "code/q01/o2_q01.py"])
    st("certificates_q02", [PY, "code/q02/q02_certs.py"])
    st("gate_q01_layer", [PY, "code/q01/release_q01.py"])
    st("certificates_q03", [PY, "code/q03/q03_certs.py"])
    st("gate_q02_layer", [PY, "code/q02/release_q02.py", "--layer"])
    st("certificates_q04", [PY, "code/q04/q04_certs.py"])
    st("gate_q03_layer", [PY, "code/q03/release_q03.py", "--layer"])
    st("gate_q04_layer", [PY, "code/q04/release_q04.py"])
    st("certificates_q05", [PY, "code/p6q05/q05_certs.py"])
    st("gate", [PY, "code/p6q05/release_q05.py"])
    st("tests", [PY, "-m", "pytest", "-q", "-p", "no:cacheprovider", "tests"])
    manuscript()
    zips()
    manifest()
    sync()
    print("BUILD_OK", flush=True)


if __name__ == "__main__":
    main()
```

tools/build_q06.py

```
#!/usr/bin/env python3
"""XVI-Q Q06 full build (from scratch).  Reuses the helpers of tools/build.py (pinned by the Q04 layer gate, unchanged):
clean -> extract -> classical -> mutations -> Q01..Q04 certificates -> Q01..Q03 layer gates -> Q04 gate (layer record)
-> Q05 certificates -> Q05 layer gate -> Q06 certificates -> Q06 gate (owns RELEASE_STATUS) -> tests -> PDFs -> zips -> manifest LAST -> optional sync
(default none; release opt-in: --sync <dir> or Q_SYNC_DIR)."""
from __future__ import annotations
import hashlib, json, os, shutil, subprocess, sys, zipfile  # noqa: E401
from pathlib import Path

Q = Path(__file__).resolve().parents[1]
sys.path.insert(0, str(Q / "tools"))
import build as B  # noqa: E402
import source_set_q06 as SS  # noqa: E402

PY = sys.executable
SYNC = (sys.argv[sys.argv.index("--sync") + 1] if "--sync" in sys.argv else os.environ.get("Q_SYNC_DIR", "none"))
PDFS = ["XVI-Q-science", "XVI-Q-appendix", "XVI-Q"]
ZIP = "XVI-Q-Q06-source-only.zip"
NOTES = ["Q06", "Q05", "Q04", "Q03", "Q02", "Q01"]
APPX = ["RELEASE_STATUS.json", "certificates/Q06_P6_status.json", "certificates/Q06_ghost_rows.json",
        "certificates/Q06_G10_crosscheck.json", "certificates/Q06_V4_validation.json", "certificates/Q06_fastpath.json",
        "certificates/Q06_P6_hhh.json", "ledgers/Q06_milestones.json", "certificates/Q05_P6_status.json", "certificates/Q05_P6_rows.json",
        "certificates/Q05_rules_validation.json", "certificates/Q05_P6_graphs.json", "certificates/Q05_guard_scan.json",
        "ledgers/Q05_milestones.json", "certificates/Q04_release.json", "certificates/Q04_P6_status.json",
        "certificates/Q04_P6_inventory.json", "certificates/Q04_P6_identity.json", "certificates/Q04_P6_eval.json",
```

```
        "certificates/Q03_O1_record.json", "certificates/Q02_bulk_identity.json", "certificates/Q01_O2.json",
        "intake/errata_p23.json", "registration/local_registration.json"]


def sh(p):
    return hashlib.sha256(Path(p).read_bytes()).hexdigest()


def manuscript():
    ms = Q / "manuscript"
    pre = (r"\documentclass[10pt]{article}\usepackage[margin=2cm]{geometry}\usepackage{amsmath,amssymb}"
           r"\usepackage{fancyhdr}\pagestyle{fancy}\fancyfoot[C]{XVI-Q Q06 \textbar{} page \thepage}"
           r"\usepackage{hyperref}\begin{document}")
    title = (r"\title{Landmark XVI-Q --- Q06 (P.6 continued, honest partial)\\\large One-loop $\kappa^3$ three-point "
             r"sector: ghost rows c c $\bar c$ / c c K closed on the 4D part (Grassmann-sign fix), evanescent remainder "
             r"open; G01, G02 (own V4) evaluated at pole level; h h c row, $\kappa^3$ defect and $\sigma_0$ closure not "
             r"established. O.1 CLOSED (scoped, $\kappa^2$ two-point) unchanged}"
             r"\date{2026-10-11}\maketitle\thispagestyle{fancy}\tableofcontents\newpage")
    parts = [r"\input{notes/Q06_science.tex}"]
    for n in NOTES[1:]:
        parts.append(r"\newpage\section*{" + n + r" layer}\addcontentsline{toc}{section}{" + n + r" layer}\input{notes/"
                     + n + r"_science.tex}")
    sci = "".join(parts)
    app = [r"\section{Appendix: certificates, status, intake}"]
    for f in APPX:
        app += [r"\subsection*{" + B.tex_escape_path(f) + "}", r"{\scriptsize\begin{verbatim}",
                SS.wrap((Q / f).read_text()), r"\end {verbatim}}"]
    app += [r"\section{Appendix: source listings}"]
    for f in SS.source_files(Q):
        if f.endswith(".py"):
            app += [r"\subsection*{" + B.tex_escape_path(f) + "}", r"{\scriptsize\begin{verbatim}",
                    SS.wrap((Q / f).read_text()), r"\end {verbatim}}"]
    (ms / "body_appendix.tex").write_text("\n".join(app))
    docs = {"XVI-Q-science": sci, "XVI-Q-appendix": r"\input{body_appendix.tex}",
            "XVI-Q": sci + r"\newpage" + r"\input{body_appendix.tex}"}
    for name, body in docs.items():
        (ms / f"{name}.tex").write_text(pre + title + body + r"\end{document}")
        for _ in range(2):
            r = subprocess.run(["pdflatex", "-interaction=nonstopmode", "-halt-on-error", f"{name}.tex"], cwd=ms,
                               capture_output=True, text=True)
        if r.returncode != 0:
            print(r.stdout[-3000:])
            raise SystemExit(f"pdflatex {name} failed")
    for p in ms.glob("*.aux"):
        p.unlink()
    print("[pdfs]", {p: subprocess.run(["pdfinfo", f"manuscript/{p}.pdf"], cwd=Q, capture_output=True,
                                       text=True).stdout.split("Pages:")[1].split()[0] for p in PDFS}, flush=True)


def zips():
    dist = Q / "dist"
    for old in dist.glob("*.zip"):
        old.unlink()
    SS.make_zip(dist / ZIP, Q)
    gen = ["RELEASE_STATUS.json"] + [p.relative_to(Q).as_posix() for p in sorted((Q / "certificates").glob("*.json"))] + \
          [p.relative_to(Q).as_posix() for p in sorted((Q / "ledgers").glob("*.json"))] + [f"manuscript/{p}.pdf" for p in PDFS]
    with zipfile.ZipFile(dist / "XVI-Q-review-focus.zip", "w", zipfile.ZIP_DEFLATED) as z:
        for f in gen + ["README.md", "intake/errata_p23.json", "registration/local_registration.json"]:
            z.write(Q / f, f"XVI-Q/{f}")
    with zipfile.ZipFile(dist / "XVI-Q-full.zip", "w", zipfile.ZIP_DEFLATED) as z:
        for f in sorted(set(SS.source_files(Q)) | set(gen)):
            z.write(Q / f, f"XVI-Q/{f}")


def manifest():
    files = [f"manuscript/{p}.pdf" for p in PDFS] + [f"dist/{ZIP}", "dist/XVI-Q-review-focus.zip", "dist/XVI-Q-full.zip",
                                                     "RELEASE_STATUS.json"] + \
            [p.relative_to(Q).as_posix() for p in sorted((Q / "certificates").glob("Q0*.json"))]
    lines = ["# XVI-Q Q06 external manifest (written LAST; PDFs never contain these hashes)"]
    for f in files:
        pages = ""
        if f.endswith(".pdf"):
            pages = " pages=" + subprocess.run(["pdfinfo", f], cwd=Q, capture_output=True, text=True).stdout.split("Pages:")[1].split()[0]
        lines.append(f"{sh(Q / f)}  {f}{pages}")
    lines.append(f"source_tree_sha256 {SS.tree_sha(Q)}")
    (Q / "RELEASE_MANIFEST_EXTERNAL.txt").write_text("\n".join(lines) + "\n")
    for f in [f"manuscript/{p}.pdf" for p in PDFS]:
        txt = subprocess.run(["pdftotext", f, "-"], cwd=Q, capture_output=True, text=True).stdout
        assert not any(ln.split()[0] in txt for ln in lines[1:-1]), f


def sync():
    if SYNC == "none":
        return
```

```
    top = Path(SYNC)
    for p in PDFS:
        shutil.copy(Q / "manuscript" / f"{p}.pdf", top / f"{p}.pdf")
    for z in (ZIP, "XVI-Q-review-focus.zip", "XVI-Q-full.zip"):
        shutil.copy(Q / "dist" / z, top / z)
    print(f"[sync] {top}", flush=True)


def main():
    B.clean()
    B.extract()
    B.classical()
    B.mutations()
    st = B.step
    st("certificates", [PY, "code/q01/o2_q01.py"])
    st("certificates_q02", [PY, "code/q02/q02_certs.py"])
    st("gate_q01_layer", [PY, "code/q01/release_q01.py"])
    st("certificates_q03", [PY, "code/q03/q03_certs.py"])
    st("gate_q02_layer", [PY, "code/q02/release_q02.py", "--layer"])
    st("certificates_q04", [PY, "code/q04/q04_certs.py"])
    st("gate_q03_layer", [PY, "code/q03/release_q03.py", "--layer"])
    st("gate_q04_layer", [PY, "code/q04/release_q04.py"])
    st("certificates_q05", [PY, "code/p6q05/q05_certs.py"])
    st("gate_q05_layer", [PY, "code/p6q05/release_q05.py"])
    st("certificates_q06", [PY, "code/p6q06/q06_certs.py"])
    st("gate", [PY, "code/p6q06/release_q06.py"])
    st("tests", [PY, "-m", "pytest", "-q", "-p", "no:cacheprovider", "tests"])
    manuscript()
    zips()
    manifest()
    sync()
    print("BUILD_OK", flush=True)


if __name__ == "__main__":
    main()
```

tools/build_q06_q07.py

```
#!/usr/bin/env python3
"""XVI-Q Q07 full build (from scratch).  Reuses the helpers of tools/build.py (pinned by the Q04 layer gate, unchanged):
clean -> extract -> classical -> mutations -> Q01..Q04 certificates -> Q01..Q03 layer gates -> Q04 gate (layer record)
-> Q05 certificates -> Q05 layer gate -> Q06 certificates -> Q06 layer gate -> Q07 certificates -> Q07 gate (owns RELEASE_STATUS) -> tests -> PDFs ->
zips -> manifest LAST -> optional sync
(default none; release opt-in: --sync <dir> or Q_SYNC_DIR)."""
from __future__ import annotations
import hashlib, json, os, shutil, subprocess, sys, zipfile  # noqa: E401
from pathlib import Path

Q = Path(__file__).resolve().parents[1]
sys.path.insert(0, str(Q / "tools"))
import build as B  # noqa: E402
import source_set_q06_q07 as SS  # noqa: E402

PY = sys.executable
SYNC = (sys.argv[sys.argv.index("--sync") + 1] if "--sync" in sys.argv else os.environ.get("Q_SYNC_DIR", "none"))
PDFS = ["XVI-Q-science", "XVI-Q-appendix", "XVI-Q"]
ZIP = "XVI-Q-Q07-source-only.zip"
NOTES = ["Q07", "Q06", "Q05", "Q04", "Q03", "Q02", "Q01"]
APPX = ["RELEASE_STATUS.json", "certificates/Q07_P6_status.json", "certificates/Q07_hhc_row.json",
        "certificates/Q07_evanescent_CT.json", "ledgers/Q07_milestones.json", "certificates/Q06_P6_status.json", "certificates/Q06_ghost_rows.json",
        "certificates/Q06_G10_crosscheck.json", "certificates/Q06_V4_validation.json", "certificates/Q06_fastpath.json",
        "certificates/Q06_P6_hhh.json", "ledgers/Q06_milestones.json", "certificates/Q05_P6_status.json", "certificates/Q05_P6_rows.json",
        "certificates/Q05_rules_validation.json", "certificates/Q05_P6_graphs.json", "certificates/Q05_guard_scan.json",
        "ledgers/Q05_milestones.json", "certificates/Q04_release.json", "certificates/Q04_P6_status.json",
        "certificates/Q04_P6_inventory.json", "certificates/Q04_P6_identity.json", "certificates/Q04_P6_eval.json",
        "certificates/Q03_O1_record.json", "certificates/Q02_bulk_identity.json", "certificates/Q01_O2.json",
        "intake/errata_p23.json", "registration/local_registration.json"]


def sh(p):
    return hashlib.sha256(Path(p).read_bytes()).hexdigest()


def manuscript():
    ms = Q / "manuscript"
    pre = (r"\documentclass[10pt]{article}\usepackage[margin=2cm]{geometry}\usepackage{amsmath,amssymb}"
           r"\usepackage{fancyhdr}\pagestyle{fancy}\fancyfoot[C]{XVI-Q Q07 \textbar{} page \thepage}"
           r"\usepackage{hyperref}\begin{document}")
    title = (r"\title{Landmark XVI-Q --- Q07 (P.6 three-point rows, honest partial)\\\large One-loop $\kappa^3$: "
             r"h h c row assembled and closed on the 4D part; evanescent remainders of all three-point rows cancelled "
             r"by a local evanescent counterterm (existence, non-unique); pole-level $\sigma_0$ closure on the "
             r"three-point rows only. O.1 CLOSED (scoped, $\kappa^2$ two-point) unchanged}"
```

```
            r"\date{2026-10-11}\maketitle\thispagestyle{fancy}\tableofcontents\newpage")
    parts = [r"\input{notes/Q07_science.tex}"]
    for n in NOTES[1:]:
        parts.append(r"\newpage\section*{" + n + r" layer}\addcontentsline{toc}{section}{" + n + r" layer}\input{notes/"
                     + n + r"_science.tex}")
    sci = "".join(parts)
    app = [r"\section{Appendix: certificates, status, intake}"]
    for f in APPX:
        app += [r"\subsection*{" + B.tex_escape_path(f) + "}", r"{\scriptsize\begin{verbatim}",
                SS.wrap((Q / f).read_text()), r"\end {verbatim}}"]
    app += [r"\section{Appendix: source listings}"]
    for f in SS.source_files(Q):
        if f.endswith(".py"):
            app += [r"\subsection*{" + B.tex_escape_path(f) + "}", r"{\scriptsize\begin{verbatim}",
                    SS.wrap((Q / f).read_text()), r"\end {verbatim}}"]
    (ms / "body_appendix.tex").write_text("\n".join(app))
    docs = {"XVI-Q-science": sci, "XVI-Q-appendix": r"\input{body_appendix.tex}",
            "XVI-Q": sci + r"\newpage" + r"\input{body_appendix.tex}"}
    for name, body in docs.items():
        (ms / f"{name}.tex").write_text(pre + title + body + r"\end{document}")
        for _ in range(2):
            r = subprocess.run(["pdflatex", "-interaction=nonstopmode", "-halt-on-error", f"{name}.tex"], cwd=ms,
                               capture_output=True, text=True)
        if r.returncode != 0:
            print(r.stdout[-3000:])
            raise SystemExit(f"pdflatex {name} failed")
    for p in ms.glob("*.aux"):
        p.unlink()
    print("[pdfs]", {p: subprocess.run(["pdfinfo", f"manuscript/{p}.pdf"], cwd=Q, capture_output=True,
                                       text=True).stdout.split("Pages:")[1].split()[0] for p in PDFS}, flush=True)


def zips():
    dist = Q / "dist"
    for old in dist.glob("*.zip"):
        old.unlink()
    SS.make_zip(dist / ZIP, Q)
    gen = ["RELEASE_STATUS.json"] + [p.relative_to(Q).as_posix() for p in sorted((Q / "certificates").glob("*.json"))] + \
          [p.relative_to(Q).as_posix() for p in sorted((Q / "ledgers").glob("*.json"))] + [f"manuscript/{p}.pdf" for p in PDFS]
    with zipfile.ZipFile(dist / "XVI-Q-review-focus.zip", "w", zipfile.ZIP_DEFLATED) as z:
        for f in gen + ["README.md", "intake/errata_p23.json", "registration/local_registration.json"]:
            z.write(Q / f, f"XVI-Q/{f}")
    with zipfile.ZipFile(dist / "XVI-Q-full.zip", "w", zipfile.ZIP_DEFLATED) as z:
        for f in sorted(set(SS.source_files(Q)) | set(gen)):
            z.write(Q / f, f"XVI-Q/{f}")


def manifest():
    files = [f"manuscript/{p}.pdf" for p in PDFS] + [f"dist/{ZIP}", "dist/XVI-Q-review-focus.zip", "dist/XVI-Q-full.zip",
                                                     "RELEASE_STATUS.json"] + \
            [p.relative_to(Q).as_posix() for p in sorted((Q / "certificates").glob("Q0*.json"))]
    lines = ["# XVI-Q Q07 external manifest (written LAST; PDFs never contain these hashes)"]
    for f in files:
        pages = ""
        if f.endswith(".pdf"):
            pages = " pages=" + subprocess.run(["pdfinfo", f], cwd=Q, capture_output=True, text=True).stdout.split("Pages:")[1].split()[0]
        lines.append(f"{sh(Q / f)}  {f}{pages}")
    lines.append(f"source_tree_sha256 {SS.tree_sha(Q)}")
    (Q / "RELEASE_MANIFEST_EXTERNAL.txt").write_text("\n".join(lines) + "\n")
    for f in [f"manuscript/{p}.pdf" for p in PDFS]:
        txt = subprocess.run(["pdftotext", f, "-"], cwd=Q, capture_output=True, text=True).stdout
        assert not any(ln.split()[0] in txt for ln in lines[1:-1]), f


def sync():
    if SYNC == "none":
        return
    top = Path(SYNC)
    for p in PDFS:
        shutil.copy(Q / "manuscript" / f"{p}.pdf", top / f"{p}.pdf")
    for z in (ZIP, "XVI-Q-review-focus.zip", "XVI-Q-full.zip"):
        shutil.copy(Q / "dist" / z, top / z)
    print(f"[sync] {top}", flush=True)


def main():
    B.clean()
    B.extract()
    B.classical()
    B.mutations()
    st = B.step
    st("certificates", [PY, "code/q01/o2_q01.py"])
    st("certificates_q02", [PY, "code/q02/q02_certs.py"])
    st("gate_q01_layer", [PY, "code/q01/release_q01.py"])
```

```
    st("certificates_q03", [PY, "code/q03/q03_certs.py"])
    st("gate_q02_layer", [PY, "code/q02/release_q02.py", "--layer"])
    st("certificates_q04", [PY, "code/q04/q04_certs.py"])
    st("gate_q03_layer", [PY, "code/q03/release_q03.py", "--layer"])
    st("gate_q04_layer", [PY, "code/q04/release_q04.py"])
    st("certificates_q05", [PY, "code/p6q05/q05_certs.py"])
    st("gate_q05_layer", [PY, "code/p6q05/release_q05.py"])
    st("certificates_q06", [PY, "code/p6q06/q06_certs.py"])
    st("gate_q06_layer", [PY, "code/p6q06/release_q06.py"])
    st("certificates_q07", [PY, "code/p6q06/q07/q07_certs.py"])
    st("gate", [PY, "code/p6q06/q07/release_q07.py"])
    st("tests", [PY, "-m", "pytest", "-q", "-p", "no:cacheprovider", "tests", "code/p6q06/q07/test_q07.py"])
    manuscript()
    zips()
    manifest()
    sync()
    print("BUILD_OK", flush=True)


if __name__ == "__main__":
    main()
```

tools/build_q06_q07_q08.py

```
#!/usr/bin/env python3
"""XVI-Q Q08 full build (from scratch).  Reuses the helpers of tools/build.py (pinned by the Q04 layer gate, unchanged):
clean -> extract -> classical -> mutations -> Q01..Q04 certificates -> Q01..Q03 layer gates -> Q04 gate (layer record)
-> Q05 certificates -> Q05 layer gate -> Q06 certificates -> Q06 layer gate -> Q07 certificates -> Q07 layer gate -> Q08 certificates -> Q08 gate
(owns RELEASE_STATUS) -> tests -> PDFs -> zips -> manifest LAST -> optional sync
(default none; release opt-in: --sync <dir> or Q_SYNC_DIR)."""
from __future__ import annotations
import hashlib, json, os, shutil, subprocess, sys, zipfile  # noqa: E401
from pathlib import Path

Q = Path(__file__).resolve().parents[1]
sys.path.insert(0, str(Q / "tools"))
import build as B  # noqa: E402
import source_set_q06_q07_q08 as SS  # noqa: E402

PY = sys.executable
SYNC = (sys.argv[sys.argv.index("--sync") + 1] if "--sync" in sys.argv else os.environ.get("Q_SYNC_DIR", "none"))
PDFS = ["XVI-Q-science", "XVI-Q-appendix", "XVI-Q"]
ZIP = "XVI-Q-Q08-source-only.zip"
NOTES = ["Q08", "Q07", "Q06", "Q05", "Q04", "Q03", "Q02", "Q01"]
APPX = ["RELEASE_STATUS.json", "certificates/Q08_P6_status.json", "certificates/Q08_ghost_equation.json",
        "certificates/Q08_free_directions.json", "ledgers/Q08_milestones.json", "certificates/Q07_P6_status.json", "certificates/Q07_hhc_row.json",
        "certificates/Q07_evanescent_CT.json", "ledgers/Q07_milestones.json", "certificates/Q06_P6_status.json", "certificates/Q06_ghost_rows.json",
        "certificates/Q06_G10_crosscheck.json", "certificates/Q06_V4_validation.json", "certificates/Q06_fastpath.json",
        "certificates/Q06_P6_hhh.json", "ledgers/Q06_milestones.json", "certificates/Q05_P6_status.json", "certificates/Q05_P6_rows.json",
        "certificates/Q05_rules_validation.json", "certificates/Q05_P6_graphs.json", "certificates/Q05_guard_scan.json",
        "ledgers/Q05_milestones.json", "certificates/Q04_release.json", "certificates/Q04_P6_status.json",
        "certificates/Q04_P6_inventory.json", "certificates/Q04_P6_identity.json", "certificates/Q04_P6_eval.json",
        "certificates/Q03_O1_record.json", "certificates/Q02_bulk_identity.json", "certificates/Q01_O2.json",
        "intake/errata_p23.json", "registration/local_registration.json"]


def sh(p):
    return hashlib.sha256(Path(p).read_bytes()).hexdigest()


def manuscript():
    ms = Q / "manuscript"
    pre = (r"\documentclass[10pt]{article}\usepackage[margin=2cm]{geometry}\usepackage{amsmath,amssymb}"
           r"\usepackage{fancyhdr}\pagestyle{fancy}\fancyfoot[C]{XVI-Q Q08 \textbar{} page \thepage}"
           r"\usepackage{hyperref}\begin{document}")
    title = (r"\title{Landmark XVI-Q --- Q08 (P.6 evanescent counterterm structure, honest partial)\\\large "
             r"One-loop $\kappa^3$: the local evanescent counterterm X can be chosen ghost-equation compatible; "
             r"free directions 152 to 104, partly BRST-trivial (fitted families); operator basis, physical ambiguity "
             r"and p22 consistency open. O.1 CLOSED (scoped, $\kappa^2$ two-point) unchanged}"
             r"\date{2026-10-11}\maketitle\thispagestyle{fancy}\tableofcontents\newpage")
    parts = [r"\input{notes/Q08_science.tex}"]
    for n in NOTES[1:]:
        parts.append(r"\newpage\section*{" + n + r" layer}\addcontentsline{toc}{section}{" + n + r" layer}\input{notes/"
                     + n + r"_science.tex}")
    sci = "".join(parts)
    app = [r"\section{Appendix: certificates, status, intake}"]
    for f in APPX:
        app += [r"\subsection*{" + B.tex_escape_path(f) + "}", r"{\scriptsize\begin{verbatim}",
                SS.wrap((Q / f).read_text()), r"\end {verbatim}}"]
    app += [r"\section{Appendix: source listings}"]
    for f in SS.source_files(Q):
        if f.endswith(".py"):
            app += [r"\subsection*{" + B.tex_escape_path(f) + "}", r"{\scriptsize\begin{verbatim}",
```

```
                    SS.wrap((Q / f).read_text()), r"\end {verbatim}}"]
    (ms / "body_appendix.tex").write_text("\n".join(app))
    docs = {"XVI-Q-science": sci, "XVI-Q-appendix": r"\input{body_appendix.tex}",
            "XVI-Q": sci + r"\newpage" + r"\input{body_appendix.tex}"}
    for name, body in docs.items():
        (ms / f"{name}.tex").write_text(pre + title + body + r"\end{document}")
        for _ in range(2):
            r = subprocess.run(["pdflatex", "-interaction=nonstopmode", "-halt-on-error", f"{name}.tex"], cwd=ms,
                               capture_output=True, text=True)
        if r.returncode != 0:
            print(r.stdout[-3000:])
            raise SystemExit(f"pdflatex {name} failed")
    for p in ms.glob("*.aux"):
        p.unlink()
    print("[pdfs]", {p: subprocess.run(["pdfinfo", f"manuscript/{p}.pdf"], cwd=Q, capture_output=True,
                                       text=True).stdout.split("Pages:")[1].split()[0] for p in PDFS}, flush=True)


def zips():
    dist = Q / "dist"
    for old in dist.glob("*.zip"):
        old.unlink()
    SS.make_zip(dist / ZIP, Q)
    gen = ["RELEASE_STATUS.json"] + [p.relative_to(Q).as_posix() for p in sorted((Q / "certificates").glob("*.json"))] + \
          [p.relative_to(Q).as_posix() for p in sorted((Q / "ledgers").glob("*.json"))] + [f"manuscript/{p}.pdf" for p in PDFS]
    with zipfile.ZipFile(dist / "XVI-Q-review-focus.zip", "w", zipfile.ZIP_DEFLATED) as z:
        for f in gen + ["README.md", "intake/errata_p23.json", "registration/local_registration.json"]:
            z.write(Q / f, f"XVI-Q/{f}")
    with zipfile.ZipFile(dist / "XVI-Q-full.zip", "w", zipfile.ZIP_DEFLATED) as z:
        for f in sorted(set(SS.source_files(Q)) | set(gen)):
            z.write(Q / f, f"XVI-Q/{f}")


def manifest():
    files = [f"manuscript/{p}.pdf" for p in PDFS] + [f"dist/{ZIP}", "dist/XVI-Q-review-focus.zip", "dist/XVI-Q-full.zip",
                                                     "RELEASE_STATUS.json"] + \
            [p.relative_to(Q).as_posix() for p in sorted((Q / "certificates").glob("Q0*.json"))]
    lines = ["# XVI-Q Q08 external manifest (written LAST; PDFs never contain these hashes)"]
    for f in files:
        pages = ""
        if f.endswith(".pdf"):
            pages = " pages=" + subprocess.run(["pdfinfo", f], cwd=Q, capture_output=True, text=True).stdout.split("Pages:")[1].split()[0]
        lines.append(f"{sh(Q / f)}  {f}{pages}")
    lines.append(f"source_tree_sha256 {SS.tree_sha(Q)}")
    (Q / "RELEASE_MANIFEST_EXTERNAL.txt").write_text("\n".join(lines) + "\n")
    for f in [f"manuscript/{p}.pdf" for p in PDFS]:
        txt = subprocess.run(["pdftotext", f, "-"], cwd=Q, capture_output=True, text=True).stdout
        assert not any(ln.split()[0] in txt for ln in lines[1:-1]), f


def sync():
    if SYNC == "none":
        return
    top = Path(SYNC)
    for p in PDFS:
        shutil.copy(Q / "manuscript" / f"{p}.pdf", top / f"{p}.pdf")
    for z in (ZIP, "XVI-Q-review-focus.zip", "XVI-Q-full.zip"):
        shutil.copy(Q / "dist" / z, top / z)
    print(f"[sync] {top}", flush=True)


def main():
    B.clean()
    B.extract()
    B.classical()
    B.mutations()
    st = B.step
    st("certificates", [PY, "code/q01/o2_q01.py"])
    st("certificates_q02", [PY, "code/q02/q02_certs.py"])
    st("gate_q01_layer", [PY, "code/q01/release_q01.py"])
    st("certificates_q03", [PY, "code/q03/q03_certs.py"])
    st("gate_q02_layer", [PY, "code/q02/release_q02.py", "--layer"])
    st("certificates_q04", [PY, "code/q04/q04_certs.py"])
    st("gate_q03_layer", [PY, "code/q03/release_q03.py", "--layer"])
    st("gate_q04_layer", [PY, "code/q04/release_q04.py"])
    st("certificates_q05", [PY, "code/p6q05/q05_certs.py"])
    st("gate_q05_layer", [PY, "code/p6q05/release_q05.py"])
    st("certificates_q06", [PY, "code/p6q06/q06_certs.py"])
    st("gate_q06_layer", [PY, "code/p6q06/release_q06.py"])
    st("certificates_q07", [PY, "code/p6q06/q07/q07_certs.py"])
    st("gate_q07_layer", [PY, "code/p6q06/q07/release_q07.py"])
    st("certificates_q08", [PY, "code/p6q06/q07/q08/q08_certs.py"])
    st("gate", [PY, "code/p6q06/q07/q08/release_q08.py"])
    st("tests", [PY, "-m", "pytest", "-q", "-p", "no:cacheprovider", "tests", "code/p6q06/q07/test_q07.py", "code/p6q06/q07/q08/test_q08.py"])
```

```
    manuscript()
    zips()
    manifest()
    sync()
    print("BUILD_OK", flush=True)


if __name__ == "__main__":
    main()
```

tools/build_q06_q07_q08_q09.py

```
#!/usr/bin/env python3
"""XVI-Q Q09 full build (from scratch).  Reuses the helpers of tools/build.py (pinned by the Q04 layer gate, unchanged):
clean -> extract -> classical -> mutations -> Q01..Q04 certificates -> Q01..Q03 layer gates -> Q04 gate (layer record)
-> Q05 certificates -> Q05 layer gate -> Q06 certificates -> Q06 layer gate -> Q07 certificates -> Q07 layer gate -> Q08 certificates -> Q08 layer
gate -> Q09 certificates -> Q09 gate (owns RELEASE_STATUS) -> tests -> PDFs -> zips -> manifest LAST -> optional sync
(default none; release opt-in: --sync <dir> or Q_SYNC_DIR)."""
from __future__ import annotations
import hashlib, json, os, shutil, subprocess, sys, zipfile  # noqa: E401
from pathlib import Path

Q = Path(__file__).resolve().parents[1]
sys.path.insert(0, str(Q / "tools"))
import build as B  # noqa: E402
import source_set_q06_q07_q08_q09 as SS  # noqa: E402

PY = sys.executable
SYNC = (sys.argv[sys.argv.index("--sync") + 1] if "--sync" in sys.argv else os.environ.get("Q_SYNC_DIR", "none"))
PDFS = ["XVI-Q-science", "XVI-Q-appendix", "XVI-Q"]
ZIP = "XVI-Q-Q09-source-only.zip"
NOTES = ["Q09", "Q08", "Q07", "Q06", "Q05", "Q04", "Q03", "Q02", "Q01"]
APPX = ["RELEASE_STATUS.json", "certificates/Q09_P6_status.json", "certificates/Q09_trivial_families.json",
        "certificates/Q09_guard_scan3.json", "certificates/Q09_q04_report_binding.json", "ledgers/Q09_milestones.json",
"certificates/Q08_P6_status.json", "certificates/Q08_ghost_equation.json",
        "certificates/Q08_free_directions.json", "ledgers/Q08_milestones.json", "certificates/Q07_P6_status.json", "certificates/Q07_hhc_row.json",
        "certificates/Q07_evanescent_CT.json", "ledgers/Q07_milestones.json", "certificates/Q06_P6_status.json", "certificates/Q06_ghost_rows.json",
        "certificates/Q06_G10_crosscheck.json", "certificates/Q06_V4_validation.json", "certificates/Q06_fastpath.json",
        "certificates/Q06_P6_hhh.json", "ledgers/Q06_milestones.json", "certificates/Q05_P6_status.json", "certificates/Q05_P6_rows.json",
        "certificates/Q05_rules_validation.json", "certificates/Q05_P6_graphs.json", "certificates/Q05_guard_scan.json",
        "ledgers/Q05_milestones.json", "certificates/Q04_release.json", "certificates/Q04_P6_status.json",
        "certificates/Q04_P6_inventory.json", "certificates/Q04_P6_identity.json", "certificates/Q04_P6_eval.json",
        "certificates/Q03_O1_record.json", "certificates/Q02_bulk_identity.json", "certificates/Q01_O2.json",
        "intake/errata_p23.json", "registration/local_registration.json"]


def sh(p):
    return hashlib.sha256(Path(p).read_bytes()).hexdigest()


def manuscript():
    ms = Q / "manuscript"
    pre = (r"\documentclass[10pt]{article}\usepackage[margin=2cm]{geometry}\usepackage{amsmath,amssymb}"
           r"\usepackage{fancyhdr}\pagestyle{fancy}\fancyfoot[C]{XVI-Q Q09 \textbar{} page \thepage}"
           r"\usepackage{hyperref}\begin{document}")
    title = (r"\title{Landmark XVI-Q --- Q09 (P.6 evanescent counterterm: review fixes and derived trivial "
             r"directions, honest partial)\\\large One-loop $\kappa^3$: evanescent three-point defect "
             r"$\sigma_0$-exact at pole level, X non-unique (ghost-equation compatible); derived BRST-trivial "
             r"normalisations partly verified; operator basis and p22 consistency open. O.1 CLOSED (scoped, "
             r"$\kappa^2$ two-point) unchanged}"
             r"\date{2026-10-11}\maketitle\thispagestyle{fancy}\tableofcontents\newpage")
    parts = [r"\input{notes/Q09_science.tex}"]
    for n in NOTES[1:]:
        parts.append(r"\newpage\section*{" + n + r" layer}\addcontentsline{toc}{section}{" + n + r" layer}\input{notes/"
                     + n + r"_science.tex}")
    sci = "".join(parts)
    app = [r"\section{Appendix: certificates, status, intake}"]
    for f in APPX:
        app += [r"\subsection*{" + B.tex_escape_path(f) + "}", r"{\scriptsize\begin{verbatim}",
                SS.wrap((Q / f).read_text()), r"\end {verbatim}}"]
    app += [r"\section{Appendix: source listings}"]
    for f in SS.source_files(Q):
        if f.endswith(".py"):
            app += [r"\subsection*{" + B.tex_escape_path(f) + "}", r"{\scriptsize\begin{verbatim}",
                    SS.wrap((Q / f).read_text()), r"\end {verbatim}}"]
    (ms / "body_appendix.tex").write_text("\n".join(app))
    docs = {"XVI-Q-science": sci, "XVI-Q-appendix": r"\input{body_appendix.tex}",
            "XVI-Q": sci + r"\newpage" + r"\input{body_appendix.tex}"}
    for name, body in docs.items():
        (ms / f"{name}.tex").write_text(pre + title + body + r"\end{document}")
        for _ in range(2):
            r = subprocess.run(["pdflatex", "-interaction=nonstopmode", "-halt-on-error", f"{name}.tex"], cwd=ms,
                               capture_output=True, text=True)
```

```
        if r.returncode != 0:
            print(r.stdout[-3000:])
            raise SystemExit(f"pdflatex {name} failed")
    for p in ms.glob("*.aux"):
        p.unlink()
    print("[pdfs]", {p: subprocess.run(["pdfinfo", f"manuscript/{p}.pdf"], cwd=Q, capture_output=True,
                                       text=True).stdout.split("Pages:")[1].split()[0] for p in PDFS}, flush=True)


def zips():
    dist = Q / "dist"
    for old in dist.glob("*.zip"):
        old.unlink()
    SS.make_zip(dist / ZIP, Q)
    gen = ["RELEASE_STATUS.json"] + [p.relative_to(Q).as_posix() for p in sorted((Q / "certificates").glob("*.json"))] + \
          [p.relative_to(Q).as_posix() for p in sorted((Q / "ledgers").glob("*.json"))] + [f"manuscript/{p}.pdf" for p in PDFS]
    with zipfile.ZipFile(dist / "XVI-Q-review-focus.zip", "w", zipfile.ZIP_DEFLATED) as z:
        for f in gen + ["README.md", "intake/errata_p23.json", "registration/local_registration.json"]:
            z.write(Q / f, f"XVI-Q/{f}")
    with zipfile.ZipFile(dist / "XVI-Q-full.zip", "w", zipfile.ZIP_DEFLATED) as z:
        for f in sorted(set(SS.source_files(Q)) | set(gen)):
            z.write(Q / f, f"XVI-Q/{f}")


def manifest():
    files = [f"manuscript/{p}.pdf" for p in PDFS] + [f"dist/{ZIP}", "dist/XVI-Q-review-focus.zip", "dist/XVI-Q-full.zip",
                                                     "RELEASE_STATUS.json"] + \
            [p.relative_to(Q).as_posix() for p in sorted((Q / "certificates").glob("Q0*.json"))]
    lines = ["# XVI-Q Q09 external manifest (written LAST; PDFs never contain these hashes)"]
    for f in files:
        pages = ""
        if f.endswith(".pdf"):
            pages = " pages=" + subprocess.run(["pdfinfo", f], cwd=Q, capture_output=True, text=True).stdout.split("Pages:")[1].split()[0]
        lines.append(f"{sh(Q / f)}  {f}{pages}")
    lines.append(f"source_tree_sha256 {SS.tree_sha(Q)}")
    (Q / "RELEASE_MANIFEST_EXTERNAL.txt").write_text("\n".join(lines) + "\n")
    for f in [f"manuscript/{p}.pdf" for p in PDFS]:
        txt = subprocess.run(["pdftotext", f, "-"], cwd=Q, capture_output=True, text=True).stdout
        assert not any(ln.split()[0] in txt for ln in lines[1:-1]), f


def sync():
    if SYNC == "none":
        return
    top = Path(SYNC)
    for p in PDFS:
        shutil.copy(Q / "manuscript" / f"{p}.pdf", top / f"{p}.pdf")
    for z in (ZIP, "XVI-Q-review-focus.zip", "XVI-Q-full.zip"):
        shutil.copy(Q / "dist" / z, top / z)
    print(f"[sync] {top}", flush=True)


def main():
    B.clean()
    B.extract()
    B.classical()
    B.mutations()
    st = B.step
    st("certificates", [PY, "code/q01/o2_q01.py"])
    st("certificates_q02", [PY, "code/q02/q02_certs.py"])
    st("gate_q01_layer", [PY, "code/q01/release_q01.py"])
    st("certificates_q03", [PY, "code/q03/q03_certs.py"])
    st("gate_q02_layer", [PY, "code/q02/release_q02.py", "--layer"])
    st("certificates_q04", [PY, "code/q04/q04_certs.py"])
    st("gate_q03_layer", [PY, "code/q03/release_q03.py", "--layer"])
    st("gate_q04_layer", [PY, "code/q04/release_q04.py"])
    st("certificates_q05", [PY, "code/p6q05/q05_certs.py"])
    st("gate_q05_layer", [PY, "code/p6q05/release_q05.py"])
    st("certificates_q06", [PY, "code/p6q06/q06_certs.py"])
    st("gate_q06_layer", [PY, "code/p6q06/release_q06.py"])
    st("certificates_q07", [PY, "code/p6q06/q07/q07_certs.py"])
    st("gate_q07_layer", [PY, "code/p6q06/q07/release_q07.py"])
    st("certificates_q08", [PY, "code/p6q06/q07/q08/q08_certs.py"])
    st("gate_q08_layer", [PY, "code/p6q06/q07/q08/release_q08.py"])
    st("certificates_q09", [PY, "code/p6q06/q07/q08/q09/q09_certs.py"])
    st("gate", [PY, "code/p6q06/q07/q08/q09/release_q09.py"])
    st("tests", [PY, "-m", "pytest", "-q", "-p", "no:cacheprovider", "tests", "code/p6q06/q07/test_q07.py", "code/p6q06/q07/q08/test_q08.py",
"code/p6q06/q07/q08/q09/test_q09.py"])
    manuscript()
    zips()
    manifest()
    sync()
    print("BUILD_OK", flush=True)
```

```
if __name__ == "__main__":
    main()
```

tools/build_q06_q07_q08_q09_q10.py

```
#!/usr/bin/env python3
"""XVI-Q Q10 full build (from scratch).  Reuses the helpers of tools/build.py (pinned by the Q04 layer gate, unchanged):
clean -> extract -> classical -> mutations -> Q01..Q04 certificates -> Q01..Q03 layer gates -> Q04 gate (layer record)
-> Q05 certificates -> Q05 layer gate -> Q06 certificates -> Q06 layer gate -> Q07 certificates -> Q07 layer gate -> Q08 certificates -> Q08 layer
gate -> Q09 certificates -> Q09 layer gate -> Q10 certificates -> Q10 gate (owns RELEASE_STATUS) -> tests -> PDFs -> zips -> manifest LAST -> optional
sync
(default none; release opt-in: --sync <dir> or Q_SYNC_DIR)."""
from __future__ import annotations
import hashlib, json, os, shutil, subprocess, sys, zipfile  # noqa: E401
from pathlib import Path

Q = Path(__file__).resolve().parents[1]
sys.path.insert(0, str(Q / "tools"))
import build as B  # noqa: E402
import source_set_q06_q07_q08_q09_q10 as SS  # noqa: E402

PY = sys.executable
SYNC = (sys.argv[sys.argv.index("--sync") + 1] if "--sync" in sys.argv else os.environ.get("Q_SYNC_DIR", "none"))
PDFS = ["XVI-Q-science", "XVI-Q-appendix", "XVI-Q"]
ZIP = "XVI-Q-Q10-source-only.zip"
NOTES = ["Q10", "Q09", "Q08", "Q07", "Q06", "Q05", "Q04", "Q03", "Q02", "Q01"]
APPX = ["RELEASE_STATUS.json", "certificates/Q10_P6_status.json", "certificates/Q10_family_anatomy.json", "certificates/Q10_erratum_Q08_counts.json",
"certificates/Q10_new_manuscripts.json", "ledgers/Q10_milestones.json", "certificates/Q09_P6_status.json", "certificates/Q09_trivial_families.json",
        "certificates/Q09_guard_scan3.json", "certificates/Q09_q04_report_binding.json", "ledgers/Q09_milestones.json",
"certificates/Q08_P6_status.json", "certificates/Q08_ghost_equation.json",
        "certificates/Q08_free_directions.json", "ledgers/Q08_milestones.json", "certificates/Q07_P6_status.json", "certificates/Q07_hhc_row.json",
        "certificates/Q07_evanescent_CT.json", "ledgers/Q07_milestones.json", "certificates/Q06_P6_status.json", "certificates/Q06_ghost_rows.json",
        "certificates/Q06_G10_crosscheck.json", "certificates/Q06_V4_validation.json", "certificates/Q06_fastpath.json",
        "certificates/Q06_P6_hhh.json", "ledgers/Q06_milestones.json", "certificates/Q05_P6_status.json", "certificates/Q05_P6_rows.json",
        "certificates/Q05_rules_validation.json", "certificates/Q05_P6_graphs.json", "certificates/Q05_guard_scan.json",
        "ledgers/Q05_milestones.json", "certificates/Q04_release.json", "certificates/Q04_P6_status.json",
        "certificates/Q04_P6_inventory.json", "certificates/Q04_P6_identity.json", "certificates/Q04_P6_eval.json",
        "certificates/Q03_O1_record.json", "certificates/Q02_bulk_identity.json", "certificates/Q01_O2.json",
        "intake/errata_p23.json", "registration/local_registration.json"]


def sh(p):
    return hashlib.sha256(Path(p).read_bytes()).hexdigest()


def manuscript():
    ms = Q / "manuscript"
    pre = (r"\documentclass[10pt]{article}\usepackage[margin=2cm]{geometry}\usepackage{amsmath,amssymb}"
           r"\usepackage{fancyhdr}\pagestyle{fancy}\fancyfoot[C]{XVI-Q Q10 \textbar{} page \thepage}"
           r"\usepackage{hyperref}\begin{document}")
    title = (r"\title{Landmark XVI-Q --- Q10 (P.6 evanescent counterterm: erratum on Q08 trivial counts, anatomy of the "
             r"family vectors, honest partial)\\\large One-loop $\kappa^3$: evanescent three-point defect "
             r"$\sigma_0$-exact at pole level, X non-unique (ghost-equation compatible); BRST-trivial classification, "
             r"operator basis and p22 consistency open. O.1 CLOSED (scoped, $\kappa^2$ two-point) unchanged}"
             r"\date{2026-10-12}\maketitle\thispagestyle{fancy}\tableofcontents\newpage")
    parts = [r"\input{notes/Q10_science.tex}"]
    for n in NOTES[1:]:
        parts.append(r"\newpage\section*{" + n + r" layer}\addcontentsline{toc}{section}{" + n + r" layer}\input{notes/"
                     + n + r"_science.tex}")
    sci = "".join(parts)
    app = [r"\section{Appendix: certificates, status, intake}"]
    for f in APPX:
        app += [r"\subsection*{" + B.tex_escape_path(f) + "}", r"{\scriptsize\begin{verbatim}",
                SS.wrap((Q / f).read_text()), r"\end {verbatim}}"]
    app += [r"\section{Appendix: source listings}"]
    for f in SS.source_files(Q):
        if f.endswith(".py"):
            app += [r"\subsection*{" + B.tex_escape_path(f) + "}", r"{\scriptsize\begin{verbatim}",
                    SS.wrap((Q / f).read_text()), r"\end {verbatim}}"]
    (ms / "body_appendix.tex").write_text("\n".join(app))
    docs = {"XVI-Q-science": sci, "XVI-Q-appendix": r"\input{body_appendix.tex}",
            "XVI-Q": sci + r"\newpage" + r"\input{body_appendix.tex}"}
    for name, body in docs.items():
        (ms / f"{name}.tex").write_text(pre + title + body + r"\end{document}")
        for _ in range(2):
            r = subprocess.run(["pdflatex", "-interaction=nonstopmode", "-halt-on-error", f"{name}.tex"], cwd=ms,
                               capture_output=True, text=True)
        if r.returncode != 0:
            print(r.stdout[-3000:])
            raise SystemExit(f"pdflatex {name} failed")
    for p in ms.glob("*.aux"):
        p.unlink()
```

```
    print("[pdfs]", {p: subprocess.run(["pdfinfo", f"manuscript/{p}.pdf"], cwd=Q, capture_output=True,
                                       text=True).stdout.split("Pages:")[1].split()[0] for p in PDFS}, flush=True)


def zips():
    dist = Q / "dist"
    for old in dist.glob("*.zip"):
        old.unlink()
    SS.make_zip(dist / ZIP, Q)
    gen = ["RELEASE_STATUS.json"] + [p.relative_to(Q).as_posix() for p in sorted((Q / "certificates").glob("*.json"))] + \
          [p.relative_to(Q).as_posix() for p in sorted((Q / "ledgers").glob("*.json"))] + [f"manuscript/{p}.pdf" for p in PDFS]
    with zipfile.ZipFile(dist / "XVI-Q-review-focus.zip", "w", zipfile.ZIP_DEFLATED) as z:
        for f in gen + ["README.md", "intake/errata_p23.json", "intake/new_manuscripts_q10.json", "registration/local_registration.json"]:
            z.write(Q / f, f"XVI-Q/{f}")
    with zipfile.ZipFile(dist / "XVI-Q-full.zip", "w", zipfile.ZIP_DEFLATED) as z:
        for f in sorted(set(SS.source_files(Q)) | set(gen)):
            z.write(Q / f, f"XVI-Q/{f}")


def manifest():
    files = [f"manuscript/{p}.pdf" for p in PDFS] + [f"dist/{ZIP}", "dist/XVI-Q-review-focus.zip", "dist/XVI-Q-full.zip",
                                                     "RELEASE_STATUS.json"] + \
            [p.relative_to(Q).as_posix() for p in sorted((Q / "certificates").glob("Q*.json"))]
    lines = ["# XVI-Q Q10 external manifest (written LAST; PDFs never contain these hashes)"]
    for f in files:
        pages = ""
        if f.endswith(".pdf"):
            pages = " pages=" + subprocess.run(["pdfinfo", f], cwd=Q, capture_output=True, text=True).stdout.split("Pages:")[1].split()[0]
        lines.append(f"{sh(Q / f)}  {f}{pages}")
    lines.append(f"source_tree_sha256 {SS.tree_sha(Q)}")
    (Q / "RELEASE_MANIFEST_EXTERNAL.txt").write_text("\n".join(lines) + "\n")
    for f in [f"manuscript/{p}.pdf" for p in PDFS]:
        txt = subprocess.run(["pdftotext", f, "-"], cwd=Q, capture_output=True, text=True).stdout
        assert not any(ln.split()[0] in txt for ln in lines[1:-1]), f


def sync():
    if SYNC == "none":
        return
    top = Path(SYNC)
    for p in PDFS:
        shutil.copy(Q / "manuscript" / f"{p}.pdf", top / f"{p}.pdf")
    for z in (ZIP, "XVI-Q-review-focus.zip", "XVI-Q-full.zip"):
        shutil.copy(Q / "dist" / z, top / z)
    print(f"[sync] {top}", flush=True)


def main():
    B.clean()
    B.extract()
    B.classical()
    B.mutations()
    st = B.step
    st("certificates", [PY, "code/q01/o2_q01.py"])
    st("certificates_q02", [PY, "code/q02/q02_certs.py"])
    st("gate_q01_layer", [PY, "code/q01/release_q01.py"])
    st("certificates_q03", [PY, "code/q03/q03_certs.py"])
    st("gate_q02_layer", [PY, "code/q02/release_q02.py", "--layer"])
    st("certificates_q04", [PY, "code/q04/q04_certs.py"])
    st("gate_q03_layer", [PY, "code/q03/release_q03.py", "--layer"])
    st("gate_q04_layer", [PY, "code/q04/release_q04.py"])
    st("certificates_q05", [PY, "code/p6q05/q05_certs.py"])
    st("gate_q05_layer", [PY, "code/p6q05/release_q05.py"])
    st("certificates_q06", [PY, "code/p6q06/q06_certs.py"])
    st("gate_q06_layer", [PY, "code/p6q06/release_q06.py"])
    st("certificates_q07", [PY, "code/p6q06/q07/q07_certs.py"])
    st("gate_q07_layer", [PY, "code/p6q06/q07/release_q07.py"])
    st("certificates_q08", [PY, "code/p6q06/q07/q08/q08_certs.py"])
    st("gate_q08_layer", [PY, "code/p6q06/q07/q08/release_q08.py"])
    st("certificates_q09", [PY, "code/p6q06/q07/q08/q09/q09_certs.py"])
    st("gate_q09_layer", [PY, "code/p6q06/q07/q08/q09/release_q09.py"])
    st("certificates_q10", [PY, "code/p6q06/q07/q08/q09/q10/q10_certs.py"])
    st("gate", [PY, "code/p6q06/q07/q08/q09/q10/release_q10.py"])
    st("tests", [PY, "-m", "pytest", "-q", "-p", "no:cacheprovider", "tests", "code/p6q06/q07/test_q07.py", "code/p6q06/q07/q08/test_q08.py",
"code/p6q06/q07/q08/q09/test_q09.py", "code/p6q06/q07/q08/q09/q10/test_q10.py"])
    manuscript()
    zips()
    manifest()
    sync()
    print("BUILD_OK", flush=True)


if __name__ == "__main__":
    main()
```

tools/build_q06_q07_q08_q09_q10_q11.py

```
#!/usr/bin/env python3
"""XVI-Q Q11 full build (from scratch).  Reuses the helpers of tools/build.py (pinned by the Q04 layer gate, unchanged):
clean -> extract -> classical -> mutations -> Q01..Q04 certificates -> Q01..Q03 layer gates -> Q04 gate (layer record)
-> Q05 certificates -> Q05 layer gate -> Q06 certificates -> Q06 layer gate -> Q07 certificates -> Q07 layer gate -> Q08 certificates -> Q08 layer
gate -> Q09 certificates -> Q09 layer gate -> Q10 certificates -> Q10 layer gate -> Q11 certificates -> Q11 gate (owns RELEASE_STATUS) -> tests ->
PDFs -> zips -> manifest LAST -> optional sync
(default none; release opt-in: --sync <dir> or Q_SYNC_DIR)."""
from __future__ import annotations
import hashlib, json, os, shutil, subprocess, sys, zipfile  # noqa: E401
from pathlib import Path

Q = Path(__file__).resolve().parents[1]
sys.path.insert(0, str(Q / "tools"))
import build as B  # noqa: E402
import source_set_q06_q07_q08_q09_q10_q11 as SS  # noqa: E402

PY = sys.executable
SYNC = (sys.argv[sys.argv.index("--sync") + 1] if "--sync" in sys.argv else os.environ.get("Q_SYNC_DIR", "none"))
PDFS = ["XVI-Q-science", "XVI-Q-appendix", "XVI-Q"]
ZIP = "XVI-Q-Q11-source-only.zip"
NOTES = ["Q11", "Q10", "Q09", "Q08", "Q07", "Q06", "Q05", "Q04", "Q03", "Q02", "Q01"]
APPX = ["RELEASE_STATUS.json", "certificates/Q11_sigma_images.json", "certificates/Q11_classification.json", "certificates/Q11_P6_status.json",
"ledgers/Q11_milestones.json", "certificates/Q10_P6_status.json", "certificates/Q10_family_anatomy.json", "certificates/Q10_erratum_Q08_counts.json",
"certificates/Q10_new_manuscripts.json", "ledgers/Q10_milestones.json", "certificates/Q09_P6_status.json", "certificates/Q09_trivial_families.json",
        "certificates/Q09_guard_scan3.json", "certificates/Q09_q04_report_binding.json", "ledgers/Q09_milestones.json",
"certificates/Q08_P6_status.json", "certificates/Q08_ghost_equation.json",
        "certificates/Q08_free_directions.json", "ledgers/Q08_milestones.json", "certificates/Q07_P6_status.json", "certificates/Q07_hhc_row.json",
        "certificates/Q07_evanescent_CT.json", "ledgers/Q07_milestones.json", "certificates/Q06_P6_status.json", "certificates/Q06_ghost_rows.json",
        "certificates/Q06_G10_crosscheck.json", "certificates/Q06_V4_validation.json", "certificates/Q06_fastpath.json",
        "certificates/Q06_P6_hhh.json", "ledgers/Q06_milestones.json", "certificates/Q05_P6_status.json", "certificates/Q05_P6_rows.json",
        "certificates/Q05_rules_validation.json", "certificates/Q05_P6_graphs.json", "certificates/Q05_guard_scan.json",
        "ledgers/Q05_milestones.json", "certificates/Q04_release.json", "certificates/Q04_P6_status.json",
        "certificates/Q04_P6_inventory.json", "certificates/Q04_P6_identity.json", "certificates/Q04_P6_eval.json",
        "certificates/Q03_O1_record.json", "certificates/Q02_bulk_identity.json", "certificates/Q01_O2.json",
        "intake/errata_p23.json", "registration/local_registration.json"]


def sh(p):
    return hashlib.sha256(Path(p).read_bytes()).hexdigest()


def manuscript():
    ms = Q / "manuscript"
    pre = (r"\documentclass[10pt]{article}\usepackage[margin=2cm]{geometry}\usepackage{amsmath,amssymb}"
           r"\usepackage{fancyhdr}\pagestyle{fancy}\fancyfoot[C]{XVI-Q Q11 \textbar{} page \thepage}"
           r"\usepackage{hyperref}\begin{document}")
    title = (r"\title{Landmark XVI-Q --- Q11 (P.6 evanescent counterterm: BRST-trivial directions as sigma-images in the "
             r"gauge-fixed BV variables, honest partial)\\\large One-loop $\kappa^3$: evanescent three-point defect "
             r"$\sigma_0$-exact at pole level, X non-unique (ghost-equation compatible); trivial span conditional, 43 directions unclassified, "
             r"operator basis and p22 consistency open. O.1 CLOSED (scoped, $\kappa^2$ two-point) unchanged}"
             r"\date{2026-10-12}\maketitle\thispagestyle{fancy}\tableofcontents\newpage")
    parts = [r"\input{notes/Q11_science.tex}"]
    for n in NOTES[1:]:
        parts.append(r"\newpage\section*{" + n + r" layer}\addcontentsline{toc}{section}{" + n + r" layer}\input{notes/"
                     + n + r"_science.tex}")
    sci = "".join(parts)
    app = [r"\section{Appendix: certificates, status, intake}"]
    for f in APPX:
        app += [r"\subsection*{" + B.tex_escape_path(f) + "}", r"{\scriptsize\begin{verbatim}",
                SS.wrap((Q / f).read_text()), r"\end {verbatim}}"]
    app += [r"\section{Appendix: source listings}"]
    for f in SS.source_files(Q):
        if f.endswith(".py"):
            app += [r"\subsection*{" + B.tex_escape_path(f) + "}", r"{\scriptsize\begin{verbatim}",
                    SS.wrap((Q / f).read_text()), r"\end {verbatim}}"]
    (ms / "body_appendix.tex").write_text("\n".join(app))
    docs = {"XVI-Q-science": sci, "XVI-Q-appendix": r"\input{body_appendix.tex}",
            "XVI-Q": sci + r"\newpage" + r"\input{body_appendix.tex}"}
    for name, body in docs.items():
        (ms / f"{name}.tex").write_text(pre + title + body + r"\end{document}")
        for _ in range(2):
            r = subprocess.run(["pdflatex", "-interaction=nonstopmode", "-halt-on-error", f"{name}.tex"], cwd=ms,
                               capture_output=True, text=True)
        if r.returncode != 0:
            print(r.stdout[-3000:])
            raise SystemExit(f"pdflatex {name} failed")
    for p in ms.glob("*.aux"):
        p.unlink()
    print("[pdfs]", {p: subprocess.run(["pdfinfo", f"manuscript/{p}.pdf"], cwd=Q, capture_output=True,
                                       text=True).stdout.split("Pages:")[1].split()[0] for p in PDFS}, flush=True)


def zips():
```

```
    dist = Q / "dist"
    for old in dist.glob("*.zip"):
        old.unlink()
    SS.make_zip(dist / ZIP, Q)
    gen = ["RELEASE_STATUS.json"] + [p.relative_to(Q).as_posix() for p in sorted((Q / "certificates").glob("*.json"))] + \
          [p.relative_to(Q).as_posix() for p in sorted((Q / "ledgers").glob("*.json"))] + [f"manuscript/{p}.pdf" for p in PDFS]
    with zipfile.ZipFile(dist / "XVI-Q-review-focus.zip", "w", zipfile.ZIP_DEFLATED) as z:
        for f in gen + ["README.md", "intake/errata_p23.json", "intake/new_manuscripts_q10.json", "registration/local_registration.json"]:
            z.write(Q / f, f"XVI-Q/{f}")
    with zipfile.ZipFile(dist / "XVI-Q-full.zip", "w", zipfile.ZIP_DEFLATED) as z:
        for f in sorted(set(SS.source_files(Q)) | set(gen)):
            z.write(Q / f, f"XVI-Q/{f}")


def manifest():
    files = [f"manuscript/{p}.pdf" for p in PDFS] + [f"dist/{ZIP}", "dist/XVI-Q-review-focus.zip", "dist/XVI-Q-full.zip",
                                                     "RELEASE_STATUS.json"] + \
            [p.relative_to(Q).as_posix() for p in sorted((Q / "certificates").glob("Q*.json"))]
    lines = ["# XVI-Q Q11 external manifest (written LAST; PDFs never contain these hashes)"]
    for f in files:
        pages = ""
        if f.endswith(".pdf"):
            pages = " pages=" + subprocess.run(["pdfinfo", f], cwd=Q, capture_output=True, text=True).stdout.split("Pages:")[1].split()[0]
        lines.append(f"{sh(Q / f)}  {f}{pages}")
    lines.append(f"source_tree_sha256 {SS.tree_sha(Q)}")
    (Q / "RELEASE_MANIFEST_EXTERNAL.txt").write_text("\n".join(lines) + "\n")
    for f in [f"manuscript/{p}.pdf" for p in PDFS]:
        txt = subprocess.run(["pdftotext", f, "-"], cwd=Q, capture_output=True, text=True).stdout
        assert not any(ln.split()[0] in txt for ln in lines[1:-1]), f


def sync():
    if SYNC == "none":
        return
    top = Path(SYNC)
    for p in PDFS:
        shutil.copy(Q / "manuscript" / f"{p}.pdf", top / f"{p}.pdf")
    for z in (ZIP, "XVI-Q-review-focus.zip", "XVI-Q-full.zip"):
        shutil.copy(Q / "dist" / z, top / z)
    print(f"[sync] {top}", flush=True)


def main():
    B.clean()
    B.extract()
    B.classical()
    B.mutations()
    st = B.step
    st("certificates", [PY, "code/q01/o2_q01.py"])
    st("certificates_q02", [PY, "code/q02/q02_certs.py"])
    st("gate_q01_layer", [PY, "code/q01/release_q01.py"])
    st("certificates_q03", [PY, "code/q03/q03_certs.py"])
    st("gate_q02_layer", [PY, "code/q02/release_q02.py", "--layer"])
    st("certificates_q04", [PY, "code/q04/q04_certs.py"])
    st("gate_q03_layer", [PY, "code/q03/release_q03.py", "--layer"])
    st("gate_q04_layer", [PY, "code/q04/release_q04.py"])
    st("certificates_q05", [PY, "code/p6q05/q05_certs.py"])
    st("gate_q05_layer", [PY, "code/p6q05/release_q05.py"])
    st("certificates_q06", [PY, "code/p6q06/q06_certs.py"])
    st("gate_q06_layer", [PY, "code/p6q06/release_q06.py"])
    st("certificates_q07", [PY, "code/p6q06/q07/q07_certs.py"])
    st("gate_q07_layer", [PY, "code/p6q06/q07/release_q07.py"])
    st("certificates_q08", [PY, "code/p6q06/q07/q08/q08_certs.py"])
    st("gate_q08_layer", [PY, "code/p6q06/q07/q08/release_q08.py"])
    st("certificates_q09", [PY, "code/p6q06/q07/q08/q09/q09_certs.py"])
    st("gate_q09_layer", [PY, "code/p6q06/q07/q08/q09/release_q09.py"])
    st("certificates_q10", [PY, "code/p6q06/q07/q08/q09/q10/q10_certs.py"])
    st("gate_q10_layer", [PY, "code/p6q06/q07/q08/q09/q10/release_q10.py"])
    st("certificates_q11", [PY, "code/p6q06/q07/q08/q09/q10/q11/q11_certs.py"])
    st("gate", [PY, "code/p6q06/q07/q08/q09/q10/q11/release_q11.py"])
    st("tests", [PY, "-m", "pytest", "-q", "-p", "no:cacheprovider", "tests", "code/p6q06/q07/test_q07.py", "code/p6q06/q07/q08/test_q08.py",
"code/p6q06/q07/q08/q09/test_q09.py", "code/p6q06/q07/q08/q09/q10/test_q10.py", "code/p6q06/q07/q08/q09/q10/q11/test_q11.py"])
    manuscript()
    zips()
    manifest()
    sync()
    print("BUILD_OK", flush=True)


if __name__ == "__main__":
    main()
```

tools/make_expected_q02.py

```
#!/usr/bin/env python3
"""AUTHOR tool (run once at freeze): write code/q02/q02_expected.json = pinned copies of every Q01_*/Q02_* certificate
('seconds' removed) + sha256 of the conventions/gate files, then pin its sha256 into release_q02.EXPECTED_SHA256."""
import hashlib, json, re, sys  # noqa: E401
from pathlib import Path

Q = Path(__file__).resolve().parents[1]
sys.path.insert(0, str(Q / "code" / "q02"))
import release_q02 as R  # noqa: E402
NAMES = ["Q01_intake", "Q01_O2", "Q01_guard", "Q01_release", "Q02_bulk_graphs", "Q02_D12_reconciliation",
         "Q02_bulk_identity", "Q02_O1", "Q02_D04_scrutiny"]
FILES = ["code/q01/q01_conventions.py", "code/q01/release_q01.py", "code/q01/qguard.py", "code/q02/bulk_graphs.py",
         "code/q02/bulk_identity.py", "code/q02/d04_route4.py", "intake/imports.json"]
exp = {"certificates": {n: R.strip(json.loads((Q / "certificates" / f"{n}.json").read_text())) for n in NAMES},
       "file_sha256": {f: hashlib.sha256((Q / f).read_bytes()).hexdigest() for f in FILES}}
(Q / "code/q02/q02_expected.json").write_text(json.dumps(exp, indent=1, sort_keys=True) + "\n")
s = hashlib.sha256((Q / "code/q02/q02_expected.json").read_bytes()).hexdigest()
g = Q / "code/q02/release_q02.py"
g.write_text(re.sub(r'EXPECTED_SHA256 = "[^"]*"', f'EXPECTED_SHA256 = "{s}"', g.read_text()))
print(s)
```

tools/make_expected_q03.py

```
#!/usr/bin/env python3
"""AUTHOR tool (run at freeze, after make_expected_q02): write code/q03/q03_expected.json = pinned copies of every
Q01/Q02/Q03 certificate ('seconds' removed) + sha256 of code/conventions files; pin its sha into release_q03."""
import hashlib, json, re, sys  # noqa: E401
from pathlib import Path

Q = Path(__file__).resolve().parents[1]
sys.path.insert(0, str(Q / "code" / "q03"))
import release_q03 as R  # noqa: E402
NAMES = ["Q01_intake", "Q01_O2", "Q01_guard", "Q01_release", "Q02_bulk_graphs", "Q02_D12_reconciliation",
         "Q02_bulk_identity", "Q02_O1", "Q02_D04_scrutiny", "Q02_release", "Q03_O1_record", "Q03_guard_ring"]
FILES = ["code/q01/q01_conventions.py", "code/q01/release_q01.py", "code/q01/qguard.py", "code/q02/bulk_graphs.py",
         "code/q02/bulk_identity.py", "code/q02/d04_route4.py", "code/q02/release_q02.py", "code/q02/q02_expected.json",
         "code/q03/q03_certs.py", "intake/imports.json", "intake/errata_p23.json", "registration/local_registration.json"]
exp = {"certificates": {n: R.strip(json.loads((Q / "certificates" / f"{n}.json").read_text())) for n in NAMES},
       "file_sha256": {f: hashlib.sha256((Q / f).read_bytes()).hexdigest() for f in FILES}}
(Q / "code/q03/q03_expected.json").write_text(json.dumps(exp, indent=1, sort_keys=True) + "\n")
s = hashlib.sha256((Q / "code/q03/q03_expected.json").read_bytes()).hexdigest()
g = Q / "code/q03/release_q03.py"
g.write_text(re.sub(r'EXPECTED_SHA256 = "[^"]*"', f'EXPECTED_SHA256 = "{s}"', g.read_text()))
print(s)
```

tools/make_expected_q04.py

```
#!/usr/bin/env python3
"""AUTHOR tool (freeze): code/q04/q04_expected.json = pinned copies of every Q01..Q04 certificate ('seconds' removed)
+ sha256 of code files; pins its sha into release_q04.EXPECTED_SHA256."""
import hashlib, json, re, sys  # noqa: E401
from pathlib import Path

Q = Path(__file__).resolve().parents[1]
sys.path.insert(0, str(Q / "code" / "q04"))
import release_q04 as R  # noqa: E402
NAMES = ["Q01_intake", "Q01_O2", "Q01_guard", "Q01_release", "Q02_bulk_graphs", "Q02_D12_reconciliation",
         "Q02_bulk_identity", "Q02_O1", "Q02_D04_scrutiny", "Q02_release", "Q03_O1_record", "Q03_guard_ring",
         "Q03_release", "Q04_P6_inventory", "Q04_P6_identity", "Q04_P6_eval", "Q04_P6_status", "Q04_guard_scan"]
FILES = sorted({p.relative_to(Q).as_posix() for p in (Q / "code").rglob("*.py") if "__pycache__" not in p.parts
                and p.name != "release_q04.py"} | {"code/q02/q02_expected.json", "code/q03/q03_expected.json",
                                                   "intake/imports.json", "intake/errata_p23.json",
                                                   "reproducibility/independent_rebuild_report_q03.json"}
               | {p.relative_to(Q).as_posix() for p in (Q / "tools").glob("*.py") if p.name != "make_expected_q04.py"})
exp = {"certificates": {n: R.strip(json.loads((Q / "certificates" / f"{n}.json").read_text())) for n in NAMES},
       "file_sha256": {f: hashlib.sha256((Q / f).read_bytes()).hexdigest() for f in FILES}}
(Q / "code/q04/q04_expected.json").write_text(json.dumps(exp, indent=1, sort_keys=True) + "\n")
s = hashlib.sha256((Q / "code/q04/q04_expected.json").read_bytes()).hexdigest()
g = Q / "code/q04/release_q04.py"
g.write_text(re.sub(r'EXPECTED_SHA256 = "[^"]*"', f'EXPECTED_SHA256 = "{s}"', g.read_text()))
print(s)
```

tools/make_expected_q05.py

```
#!/usr/bin/env python3
"""AUTHOR tool (freeze): code/p6q05/q05_expected.json = pinned copies of every Q01..Q05 certificate ('seconds' removed)
+ sha256 of EVERY code/**/*.py, tools/*.py, tests/*.py (except the Q05 gate itself) and the intake / expected / Q04 report
files; pins its sha into release_q05.EXPECTED_SHA256."""
import hashlib, json, re, sys  # noqa: E401
```

```
from pathlib import Path

Q = Path(__file__).resolve().parents[1]
sys.path.insert(0, str(Q / "code" / "p6q05"))
import release_q05 as R  # noqa: E402
NAMES = ["Q01_intake", "Q01_O2", "Q01_guard", "Q01_release", "Q02_bulk_graphs", "Q02_D12_reconciliation",
         "Q02_bulk_identity", "Q02_O1", "Q02_D04_scrutiny", "Q02_release", "Q03_O1_record", "Q03_guard_ring",
         "Q03_release", "Q04_P6_inventory", "Q04_P6_identity", "Q04_P6_eval", "Q04_P6_status", "Q04_guard_scan",
         "Q04_release", "Q05_rules_validation", "Q05_P6_graphs", "Q05_P6_rows", "Q05_P6_status", "Q05_guard_scan"]
FILES = sorted(R.code_files() | {"code/q02/q02_expected.json", "code/q03/q03_expected.json", "code/q04/q04_expected.json",
                                 "intake/imports.json", "intake/errata_p23.json",
                                 "reproducibility/independent_rebuild_report_q04.json"})
exp = {"certificates": {n: R.strip(json.loads((Q / "certificates" / f"{n}.json").read_text())) for n in NAMES},
       "file_sha256": {f: hashlib.sha256((Q / f).read_bytes()).hexdigest() for f in FILES}}
(Q / "code/p6q05/q05_expected.json").write_text(json.dumps(exp, indent=1, sort_keys=True) + "\n")
s = hashlib.sha256((Q / "code/p6q05/q05_expected.json").read_bytes()).hexdigest()
g = Q / "code/p6q05/release_q05.py"
g.write_text(re.sub(r'EXPECTED_SHA256 = "[^"]*"', f'EXPECTED_SHA256 = "{s}"', g.read_text()))
print(s)
```

tools/make_expected_q06.py

```
#!/usr/bin/env python3
"""AUTHOR tool (freeze): code/p6q06/q06_expected.json = pinned copies of every Q01..Q06 certificate ('seconds' removed)
+ sha256 of EVERY code/**/*.py, tools/*.py, tests/*.py (except the Q06 gate itself) and the intake / expected / Q04
report files; pins its sha into release_q06.EXPECTED_SHA256."""
import hashlib, json, re, sys  # noqa: E401
from pathlib import Path

Q = Path(__file__).resolve().parents[1]
sys.path.insert(0, str(Q / "code" / "p6q06"))
import release_q06 as R  # noqa: E402
NAMES = ["Q01_intake", "Q01_O2", "Q01_guard", "Q01_release", "Q02_bulk_graphs", "Q02_D12_reconciliation",
         "Q02_bulk_identity", "Q02_O1", "Q02_D04_scrutiny", "Q02_release", "Q03_O1_record", "Q03_guard_ring",
         "Q03_release", "Q04_P6_inventory", "Q04_P6_identity", "Q04_P6_eval", "Q04_P6_status", "Q04_guard_scan",
         "Q04_release", "Q05_rules_validation", "Q05_P6_graphs", "Q05_P6_rows", "Q05_P6_status", "Q05_guard_scan",
         "Q05_release"] + R.Q06_CERTS
FILES = sorted(R.code_files() | {"code/q02/q02_expected.json", "code/q03/q03_expected.json", "code/q04/q04_expected.json",
                                 "code/p6q05/q05_expected.json", "intake/imports.json", "intake/errata_p23.json",
                                 "reproducibility/independent_rebuild_report_q04.json"})
exp = {"certificates": {n: R.strip(json.loads((Q / "certificates" / f"{n}.json").read_text())) for n in NAMES},
       "file_sha256": {f: hashlib.sha256((Q / f).read_bytes()).hexdigest() for f in FILES}}
(Q / "code/p6q06/q06_expected.json").write_text(json.dumps(exp, indent=1, sort_keys=True) + "\n")
s = hashlib.sha256((Q / "code/p6q06/q06_expected.json").read_bytes()).hexdigest()
g = Q / "code/p6q06/release_q06.py"
g.write_text(re.sub(r'EXPECTED_SHA256 = "[^"]*"', f'EXPECTED_SHA256 = "{s}"', g.read_text()))
print(s)
```

tools/make_expected_q06_q07.py

```
#!/usr/bin/env python3
"""AUTHOR tool (freeze): code/p6q06/q07/q07_expected.json = pinned copies of every Q01..Q07 certificate ('seconds'
removed) + sha256 of EVERY code/**/*.py, tools/*.py, tests/*.py (except the Q07 gate itself) and the intake /
expected files; pins its sha into release_q07.EXPECTED_SHA256."""
import hashlib, json, re, sys  # noqa: E401
from pathlib import Path

Q = Path(__file__).resolve().parents[1]
sys.path.insert(0, str(Q / "code" / "p6q06" / "q07"))
sys.path.insert(0, str(Q / "code" / "p6q06"))
import release_q07 as R  # noqa: E402
import release_q06 as R6  # noqa: E402
NAMES = ["Q01_intake", "Q01_O2", "Q01_guard", "Q01_release", "Q02_bulk_graphs", "Q02_D12_reconciliation",
         "Q02_bulk_identity", "Q02_O1", "Q02_D04_scrutiny", "Q02_release", "Q03_O1_record", "Q03_guard_ring",
         "Q03_release", "Q04_P6_inventory", "Q04_P6_identity", "Q04_P6_eval", "Q04_P6_status", "Q04_guard_scan",
         "Q04_release", "Q05_rules_validation", "Q05_P6_graphs", "Q05_P6_rows", "Q05_P6_status", "Q05_guard_scan",
         "Q05_release"] + R6.Q06_CERTS + ["Q06_release"] + R.Q07_CERTS
FILES = sorted(R.code_files() | {"code/q02/q02_expected.json", "code/q03/q03_expected.json", "code/q04/q04_expected.json",
                                 "code/p6q05/q05_expected.json", "code/p6q06/q06_expected.json", "intake/imports.json",
                                 "intake/errata_p23.json"})
exp = {"certificates": {n: R.strip(json.loads((Q / "certificates" / f"{n}.json").read_text())) for n in NAMES},
       "file_sha256": {f: hashlib.sha256((Q / f).read_bytes()).hexdigest() for f in FILES}}
p = Q / "code/p6q06/q07/q07_expected.json"
p.write_text(json.dumps(exp, indent=1, sort_keys=True) + "\n")
s = hashlib.sha256(p.read_bytes()).hexdigest()
g = Q / "code/p6q06/q07/release_q07.py"
g.write_text(re.sub(r'EXPECTED_SHA256 = "[^"]*"', f'EXPECTED_SHA256 = "{s}"', g.read_text()))
print(s)
```

tools/make_expected_q06_q07_q08.py

```
#!/usr/bin/env python3
"""AUTHOR tool (freeze): code/p6q06/q07/q07_expected.json = pinned copies of every Q01..Q08 certificate ('seconds'
removed) + sha256 of EVERY code/**/*.py, tools/*.py, tests/*.py (except the Q08 gate itself) and the intake /
expected files; pins its sha into release_q08.EXPECTED_SHA256."""
import hashlib, json, re, sys  # noqa: E401
from pathlib import Path

Q = Path(__file__).resolve().parents[1]
sys.path.insert(0, str(Q / "code" / "p6q06" / "q07" / "q08"))
sys.path.insert(0, str(Q / "code" / "p6q06" / "q07"))
sys.path.insert(0, str(Q / "code" / "p6q06"))
import release_q08 as R  # noqa: E402
import release_q07 as R7  # noqa: E402
import release_q06 as R6  # noqa: E402
NAMES = ["Q01_intake", "Q01_O2", "Q01_guard", "Q01_release", "Q02_bulk_graphs", "Q02_D12_reconciliation",
         "Q02_bulk_identity", "Q02_O1", "Q02_D04_scrutiny", "Q02_release", "Q03_O1_record", "Q03_guard_ring",
         "Q03_release", "Q04_P6_inventory", "Q04_P6_identity", "Q04_P6_eval", "Q04_P6_status", "Q04_guard_scan",
         "Q04_release", "Q05_rules_validation", "Q05_P6_graphs", "Q05_P6_rows", "Q05_P6_status", "Q05_guard_scan",
         "Q05_release"] + R6.Q06_CERTS + ["Q06_release"] + R.Q07_CERTS + R.Q08_CERTS
FILES = sorted(R.code_files() | {"code/q02/q02_expected.json", "code/q03/q03_expected.json", "code/q04/q04_expected.json",
                                 "code/p6q05/q05_expected.json", "code/p6q06/q06_expected.json", "code/p6q06/q07/q07_expected.json",
"intake/imports.json",
                                 "intake/errata_p23.json"})
exp = {"certificates": {n: R.strip(json.loads((Q / "certificates" / f"{n}.json").read_text())) for n in NAMES},
       "file_sha256": {f: hashlib.sha256((Q / f).read_bytes()).hexdigest() for f in FILES}}
p = Q / "code/p6q06/q07/q08/q08_expected.json"
p.write_text(json.dumps(exp, indent=1, sort_keys=True) + "\n")
s = hashlib.sha256(p.read_bytes()).hexdigest()
g = Q / "code/p6q06/q07/q08/release_q08.py"
g.write_text(re.sub(r'EXPECTED_SHA256 = "[^"]*"', f'EXPECTED_SHA256 = "{s}"', g.read_text()))
print(s)
```

tools/make_expected_q06_q07_q08_q09.py

```
#!/usr/bin/env python3
"""AUTHOR tool (freeze): code/p6q06/q07/q07_expected.json = pinned copies of every Q01..Q09 certificate ('seconds'
removed) + sha256 of EVERY code/**/*.py, tools/*.py, tests/*.py (except the Q09 gate itself) and the intake /
expected files; pins its sha into release_q09.EXPECTED_SHA256."""
import hashlib, json, re, sys  # noqa: E401
from pathlib import Path

Q = Path(__file__).resolve().parents[1]
sys.path.insert(0, str(Q / "code" / "p6q06" / "q07" / "q08" / "q09"))
sys.path.insert(0, str(Q / "code" / "p6q06" / "q07" / "q08"))
sys.path.insert(0, str(Q / "code" / "p6q06" / "q07"))
sys.path.insert(0, str(Q / "code" / "p6q06"))
import release_q09 as R  # noqa: E402
import release_q07 as R7  # noqa: E402
import release_q06 as R6  # noqa: E402
NAMES = ["Q01_intake", "Q01_O2", "Q01_guard", "Q01_release", "Q02_bulk_graphs", "Q02_D12_reconciliation",
         "Q02_bulk_identity", "Q02_O1", "Q02_D04_scrutiny", "Q02_release", "Q03_O1_record", "Q03_guard_ring",
         "Q03_release", "Q04_P6_inventory", "Q04_P6_identity", "Q04_P6_eval", "Q04_P6_status", "Q04_guard_scan",
         "Q04_release", "Q05_rules_validation", "Q05_P6_graphs", "Q05_P6_rows", "Q05_P6_status", "Q05_guard_scan",
         "Q05_release"] + R6.Q06_CERTS + ["Q06_release"] + R.Q07_CERTS + R.Q08_CERTS + R.Q09_CERTS
FILES = sorted(R.code_files() | {"code/q02/q02_expected.json", "code/q03/q03_expected.json", "code/q04/q04_expected.json",
                                 "code/p6q05/q05_expected.json", "code/p6q06/q06_expected.json", "code/p6q06/q07/q07_expected.json",
"code/p6q06/q07/q08/q08_expected.json", "code/p6q06/q07/q08/q09/guard_scan3_rules.json", "intake/imports.json",
                                 "intake/errata_p23.json"})
exp = {"certificates": {n: R.strip(json.loads((Q / "certificates" / f"{n}.json").read_text())) for n in NAMES},
       "file_sha256": {f: hashlib.sha256((Q / f).read_bytes()).hexdigest() for f in FILES}}
p = Q / "code/p6q06/q07/q08/q09/q09_expected.json"
p.write_text(json.dumps(exp, indent=1, sort_keys=True) + "\n")
s = hashlib.sha256(p.read_bytes()).hexdigest()
g = Q / "code/p6q06/q07/q08/q09/release_q09.py"
g.write_text(re.sub(r'EXPECTED_SHA256 = "[^"]*"', f'EXPECTED_SHA256 = "{s}"', g.read_text()))
print(s)
```

tools/make_expected_q06_q07_q08_q09_q10.py

```
#!/usr/bin/env python3
"""AUTHOR tool (freeze): code/p6q06/q07/q08/q09/q10/q10_expected.json = pinned copies of every Q01..Q10 certificate ('seconds'
removed) + sha256 of EVERY code/**/*.py, tools/*.py, tests/*.py (except the Q10 gate itself) and the intake /
expected files; pins its sha into release_q10.EXPECTED_SHA256."""
import hashlib, json, re, sys  # noqa: E401
from pathlib import Path

Q = Path(__file__).resolve().parents[1]
sys.path.insert(0, str(Q / "code" / "p6q06" / "q07" / "q08" / "q09"))
sys.path.insert(0, str(Q / "code" / "p6q06" / "q07" / "q08" / "q09" / "q10"))
sys.path.insert(0, str(Q / "code" / "p6q06" / "q07" / "q08"))
```

```
sys.path.insert(0, str(Q / "code" / "p6q06" / "q07"))
sys.path.insert(0, str(Q / "code" / "p6q06"))
import release_q10 as R  # noqa: E402
import release_q07 as R7  # noqa: E402
import release_q06 as R6  # noqa: E402
NAMES = ["Q01_intake", "Q01_O2", "Q01_guard", "Q01_release", "Q02_bulk_graphs", "Q02_D12_reconciliation",
         "Q02_bulk_identity", "Q02_O1", "Q02_D04_scrutiny", "Q02_release", "Q03_O1_record", "Q03_guard_ring",
         "Q03_release", "Q04_P6_inventory", "Q04_P6_identity", "Q04_P6_eval", "Q04_P6_status", "Q04_guard_scan",
         "Q04_release", "Q05_rules_validation", "Q05_P6_graphs", "Q05_P6_rows", "Q05_P6_status", "Q05_guard_scan",
         "Q05_release"] + R6.Q06_CERTS + ["Q06_release"] + R.Q07_CERTS + R.Q08_CERTS + R.Q09_CERTS + ["Q09_release"] + R.Q10_CERTS
FILES = sorted(R.code_files() | {"code/q02/q02_expected.json", "code/q03/q03_expected.json", "code/q04/q04_expected.json",
                                 "code/p6q05/q05_expected.json", "code/p6q06/q06_expected.json", "code/p6q06/q07/q07_expected.json",
"code/p6q06/q07/q08/q08_expected.json", "code/p6q06/q07/q08/q09/guard_scan3_rules.json", "code/p6q06/q07/q08/q09/q09_expected.json",
"intake/new_manuscripts_q10.json", "intake/imports.json",
                                 "intake/errata_p23.json"})
exp = {"certificates": {n: R.strip(json.loads((Q / "certificates" / f"{n}.json").read_text())) for n in NAMES},
       "file_sha256": {f: hashlib.sha256((Q / f).read_bytes()).hexdigest() for f in FILES}}
p = Q / "code/p6q06/q07/q08/q09/q10/q10_expected.json"
p.write_text(json.dumps(exp, indent=1, sort_keys=True) + "\n")
s = hashlib.sha256(p.read_bytes()).hexdigest()
g = Q / "code/p6q06/q07/q08/q09/q10/release_q10.py"
g.write_text(re.sub(r'EXPECTED_SHA256 = "[^"]*"', f'EXPECTED_SHA256 = "{s}"', g.read_text()))
print(s)
```

tools/make_expected_q06_q07_q08_q09_q10_q11.py

```
#!/usr/bin/env python3
"""AUTHOR tool (freeze): code/p6q06/q07/q08/q09/q10/q11/q11_expected.json = pinned copies of every Q01..Q11 certificate ('seconds'
removed) + sha256 of EVERY code/**/*.py, tools/*.py, tests/*.py (except the Q11 gate itself) and the intake /
expected files; pins its sha into release_q11.EXPECTED_SHA256."""
import hashlib, json, re, sys  # noqa: E401
from pathlib import Path

Q = Path(__file__).resolve().parents[1]
sys.path.insert(0, str(Q / "code" / "p6q06" / "q07" / "q08" / "q09"))
sys.path.insert(0, str(Q / "code" / "p6q06" / "q07" / "q08" / "q09" / "q10"))
sys.path.insert(0, str(Q / "code" / "p6q06" / "q07" / "q08" / "q09" / "q10" / "q11"))
sys.path.insert(0, str(Q / "code" / "p6q06" / "q07" / "q08"))
sys.path.insert(0, str(Q / "code" / "p6q06" / "q07"))
sys.path.insert(0, str(Q / "code" / "p6q06"))
import release_q11 as R  # noqa: E402
import release_q07 as R7  # noqa: E402
import release_q06 as R6  # noqa: E402
NAMES = ["Q01_intake", "Q01_O2", "Q01_guard", "Q01_release", "Q02_bulk_graphs", "Q02_D12_reconciliation",
         "Q02_bulk_identity", "Q02_O1", "Q02_D04_scrutiny", "Q02_release", "Q03_O1_record", "Q03_guard_ring",
         "Q03_release", "Q04_P6_inventory", "Q04_P6_identity", "Q04_P6_eval", "Q04_P6_status", "Q04_guard_scan",
         "Q04_release", "Q05_rules_validation", "Q05_P6_graphs", "Q05_P6_rows", "Q05_P6_status", "Q05_guard_scan",
         "Q05_release"] + R6.Q06_CERTS + ["Q06_release"] + R.Q07_CERTS + R.Q08_CERTS + R.Q09_CERTS + ["Q09_release"] + R.Q10_CERTS + ["Q10_release"] +
R.Q11_CERTS
FILES = sorted(R.code_files() | {"code/q02/q02_expected.json", "code/q03/q03_expected.json", "code/q04/q04_expected.json",
                                 "code/p6q05/q05_expected.json", "code/p6q06/q06_expected.json", "code/p6q06/q07/q07_expected.json",
"code/p6q06/q07/q08/q08_expected.json", "code/p6q06/q07/q08/q09/guard_scan3_rules.json", "code/p6q06/q07/q08/q09/q09_expected.json",
"code/p6q06/q07/q08/q09/q10/q10_expected.json", "intake/new_manuscripts_q10.json", "intake/imports.json",
                                 "intake/errata_p23.json"})
exp = {"certificates": {n: R.strip(json.loads((Q / "certificates" / f"{n}.json").read_text())) for n in NAMES},
       "file_sha256": {f: hashlib.sha256((Q / f).read_bytes()).hexdigest() for f in FILES}}
p = Q / "code/p6q06/q07/q08/q09/q10/q11/q11_expected.json"
p.write_text(json.dumps(exp, indent=1, sort_keys=True) + "\n")
s = hashlib.sha256(p.read_bytes()).hexdigest()
g = Q / "code/p6q06/q07/q08/q09/q10/q11/release_q11.py"
g.write_text(re.sub(r'EXPECTED_SHA256 = "[^"]*"', f'EXPECTED_SHA256 = "{s}"', g.read_text()))
print(s)
```

tools/rebuild_from_source.py

```
#!/usr/bin/env python3
"""XVI-Q Q04 rebuild entrypoint.  --write-reference (author, built tree): reproducibility/reference_values_q04.json =
content sha256 (run-specific keys 'seconds' removed) of every Q01/Q02 certificate + RELEASE_STATUS + export shas.
Default: copy the source-only zip's tree to a temp dir, run tools/build.py there (no sync), compare with the reference."""
from __future__ import annotations
import hashlib, json, os, shutil, subprocess, sys, tempfile, zipfile  # noqa: E401
from pathlib import Path

Q = Path(__file__).resolve().parents[1]
sys.path.insert(0, str(Q / "tools"))
import source_set as SS  # noqa: E402
REF = "reproducibility/reference_values_q04.json"
CERTS = ["certificates/Q01_intake.json", "certificates/Q01_O2.json", "certificates/Q01_guard.json",
         "certificates/Q01_release.json", "RELEASE_STATUS.json", "ledgers/Q01_milestones.json",
         "certificates/Q02_bulk_graphs.json", "certificates/Q02_D12_reconciliation.json", "certificates/Q02_bulk_identity.json",
         "certificates/Q02_O1.json", "certificates/Q02_D04_scrutiny.json", "certificates/Q02_release.json", "ledgers/Q02_milestones.json",
         "certificates/Q03_O1_record.json", "certificates/Q03_guard_ring.json", "certificates/Q03_release.json", "ledgers/Q03_milestones.json",
```

```
         "certificates/Q04_P6_inventory.json", "certificates/Q04_P6_identity.json", "certificates/Q04_P6_eval.json",
         "certificates/Q04_P6_status.json", "certificates/Q04_release.json", "ledgers/Q04_milestones.json"]


def strip(o):
    if isinstance(o, dict):
        return {k: strip(v) for k, v in o.items() if k != "seconds"}
    if isinstance(o, list):
        return [strip(v) for v in o]
    return o


def values(root):
    v = {c: hashlib.sha256(json.dumps(strip(json.loads((root / c).read_text())), sort_keys=True).encode()).hexdigest()
         for c in CERTS}
    v.update({f"export_ell{k}": hashlib.sha256((root / f"build/xvip_run/exports/ell{k}_generating_polynomial.srepr")
                                               .read_bytes()).hexdigest() for k in (2, 3, 4)})
    return v


def main():
    if "--write-reference" in sys.argv:
        (Q / "reproducibility").mkdir(exist_ok=True)
        (Q / REF).write_text(json.dumps({"tree_sha256_without_reference": SS.tree_sha(Q, exclude=(REF,)),
                                         "values": values(Q)}, indent=1) + "\n")
        print("written", REF)
        return 0
    ref = json.loads((Q / REF).read_text())
    with tempfile.TemporaryDirectory(prefix="q04src_") as td:
        zp = Path(td) / "src.zip"
        SS.make_zip(zp, Q)
        zipfile.ZipFile(zp).extractall(td)
        root = Path(td) / "XVI-Q"
        r = subprocess.run([sys.executable, "tools/build.py"], cwd=root, capture_output=True, text=True,
                           env=dict(os.environ, Q_SYNC_DIR="none"))
        (Q / "build").mkdir(exist_ok=True)
        (Q / "build" / "rebuild_build.log").write_text(r.stdout + r.stderr)
        built = r.returncode == 0 and "BUILD_OK" in r.stdout
        got = values(root) if built else {}
        diff = {k: (v, got.get(k)) for k, v in ref["values"].items() if got.get(k) != v}
        v_files = {f: hashlib.sha256((Q / f).read_bytes()).hexdigest() for f in ('code/q02/release_q02.py', 'code/q02/q02_expected.json',
'code/q03/release_q03.py', 'code/q03/q03_expected.json', 'code/q04/release_q04.py', 'code/q04/q04_expected.json')}
        rep = {"landmark": "XVI-Q", "release": "Q04", "zip_sha256": hashlib.sha256(zp.read_bytes()).hexdigest(),
               "source_tree_sha256": SS.tree_sha(root), "tree_matches_reference": SS.tree_sha(root, exclude=(REF,))
               == ref["tree_sha256_without_reference"], "full_build_ok": built, "compared": len(ref["values"]),
               "equal": len(ref["values"]) - len(diff), "differences": diff, "gate_and_expected_sha256": v_files}
        rep["pass"] = bool(built and not diff and rep["tree_matches_reference"])
    out = Path(sys.argv[sys.argv.index("--report") + 1]) if "--report" in sys.argv else Q / "build" / "rebuild_report.json"
    out.write_text(json.dumps(rep, indent=1) + "\n")
    print(json.dumps(rep, indent=1))
    return 0 if rep["pass"] else 1


if __name__ == "__main__":
    raise SystemExit(main())
```

tools/rebuild_q05.py

```
#!/usr/bin/env python3
"""XVI-Q Q05 rebuild entrypoint.  --write-reference (author, built tree): reproducibility/reference_values_q05.json =
content sha256 (run-specific keys 'seconds' removed) of every Q01/Q02 certificate + RELEASE_STATUS + export shas.
Default: copy the source-only zip's tree to a temp dir, run tools/build.py there (no sync), compare with the reference."""
from __future__ import annotations
import hashlib, json, os, shutil, subprocess, sys, tempfile, zipfile  # noqa: E401
from pathlib import Path

Q = Path(__file__).resolve().parents[1]
sys.path.insert(0, str(Q / "tools"))
import source_set_q05 as SS  # noqa: E402
REF = "reproducibility/reference_values_q05.json"
CERTS = ["certificates/Q01_intake.json", "certificates/Q01_O2.json", "certificates/Q01_guard.json",
         "certificates/Q01_release.json", "RELEASE_STATUS.json", "ledgers/Q01_milestones.json",
         "certificates/Q02_bulk_graphs.json", "certificates/Q02_D12_reconciliation.json", "certificates/Q02_bulk_identity.json",
         "certificates/Q02_O1.json", "certificates/Q02_D04_scrutiny.json", "certificates/Q02_release.json", "ledgers/Q02_milestones.json",
         "certificates/Q03_O1_record.json", "certificates/Q03_guard_ring.json", "certificates/Q03_release.json", "ledgers/Q03_milestones.json",
         "certificates/Q04_P6_inventory.json", "certificates/Q04_P6_identity.json", "certificates/Q04_P6_eval.json",
         "certificates/Q04_P6_status.json", "certificates/Q04_release.json", "ledgers/Q04_milestones.json",
         "certificates/Q04_guard_scan.json", "certificates/Q05_rules_validation.json", "certificates/Q05_P6_graphs.json",
         "certificates/Q05_P6_rows.json", "certificates/Q05_P6_status.json", "certificates/Q05_guard_scan.json",
         "certificates/Q05_release.json", "ledgers/Q05_milestones.json"]


def strip(o):
```

```
    if isinstance(o, dict):
        return {k: strip(v) for k, v in o.items() if k != "seconds"}
    if isinstance(o, list):
        return [strip(v) for v in o]
    return o


def values(root):
    v = {c: hashlib.sha256(json.dumps(strip(json.loads((root / c).read_text())), sort_keys=True).encode()).hexdigest()
         for c in CERTS}
    v.update({f"export_ell{k}": hashlib.sha256((root / f"build/xvip_run/exports/ell{k}_generating_polynomial.srepr")
                                               .read_bytes()).hexdigest() for k in (2, 3, 4)})
    return v


def main():
    if "--write-reference" in sys.argv:
        (Q / "reproducibility").mkdir(exist_ok=True)
        (Q / REF).write_text(json.dumps({"tree_sha256_without_reference": SS.tree_sha(Q, exclude=(REF,)),
                                         "values": values(Q)}, indent=1) + "\n")
        print("written", REF)
        return 0
    ref = json.loads((Q / REF).read_text())
    with tempfile.TemporaryDirectory(prefix="q05src_") as td:
        zp = Path(td) / "src.zip"
        SS.make_zip(zp, Q)
        zipfile.ZipFile(zp).extractall(td)
        root = Path(td) / "XVI-Q"
        r = subprocess.run([sys.executable, "tools/build_q05.py"], cwd=root, capture_output=True, text=True,
                           env=dict(os.environ, Q_SYNC_DIR="none"))
        (Q / "build").mkdir(exist_ok=True)
        (Q / "build" / "rebuild_build.log").write_text(r.stdout + r.stderr)
        built = r.returncode == 0 and "BUILD_OK" in r.stdout
        got = values(root) if built else {}
        diff = {k: (v, got.get(k)) for k, v in ref["values"].items() if got.get(k) != v}
        v_files = {f: hashlib.sha256((Q / f).read_bytes()).hexdigest() for f in ('code/q02/release_q02.py', 'code/q02/q02_expected.json',
'code/q03/release_q03.py', 'code/q03/q03_expected.json', 'code/q04/release_q04.py', 'code/q04/q04_expected.json', 'code/p6q05/release_q05.py',
'code/p6q05/q05_expected.json')}
        rep = {"landmark": "XVI-Q", "release": "Q05", "zip_sha256": hashlib.sha256(zp.read_bytes()).hexdigest(),
               "source_tree_sha256": SS.tree_sha(root), "tree_matches_reference": SS.tree_sha(root, exclude=(REF,))
               == ref["tree_sha256_without_reference"], "full_build_ok": built, "compared": len(ref["values"]),
               "equal": len(ref["values"]) - len(diff), "differences": diff, "gate_and_expected_sha256": v_files}
        rep["pass"] = bool(built and not diff and rep["tree_matches_reference"])
    out = Path(sys.argv[sys.argv.index("--report") + 1]) if "--report" in sys.argv else Q / "build" / "rebuild_report.json"
    out.write_text(json.dumps(rep, indent=1) + "\n")
    print(json.dumps(rep, indent=1))
    return 0 if rep["pass"] else 1


if __name__ == "__main__":
    raise SystemExit(main())
```

tools/rebuild_q06.py

```
#!/usr/bin/env python3
"""XVI-Q Q06 rebuild entrypoint.  --write-reference (author, built tree): reproducibility/reference_values_q06.json =
content sha256 (run-specific keys 'seconds' removed) of every Q01/Q02 certificate + RELEASE_STATUS + export shas.
Default: copy the source-only zip's tree to a temp dir, run tools/build.py there (no sync), compare with the reference."""
from __future__ import annotations
import hashlib, json, os, shutil, subprocess, sys, tempfile, zipfile  # noqa: E401
from pathlib import Path

Q = Path(__file__).resolve().parents[1]
sys.path.insert(0, str(Q / "tools"))
import source_set_q06 as SS  # noqa: E402
REF = "reproducibility/reference_values_q06.json"
CERTS = ["certificates/Q01_intake.json", "certificates/Q01_O2.json", "certificates/Q01_guard.json",
         "certificates/Q01_release.json", "RELEASE_STATUS.json", "ledgers/Q01_milestones.json",
         "certificates/Q02_bulk_graphs.json", "certificates/Q02_D12_reconciliation.json", "certificates/Q02_bulk_identity.json",
         "certificates/Q02_O1.json", "certificates/Q02_D04_scrutiny.json", "certificates/Q02_release.json", "ledgers/Q02_milestones.json",
         "certificates/Q03_O1_record.json", "certificates/Q03_guard_ring.json", "certificates/Q03_release.json", "ledgers/Q03_milestones.json",
         "certificates/Q04_P6_inventory.json", "certificates/Q04_P6_identity.json", "certificates/Q04_P6_eval.json",
         "certificates/Q04_P6_status.json", "certificates/Q04_release.json", "ledgers/Q04_milestones.json",
         "certificates/Q04_guard_scan.json", "certificates/Q05_rules_validation.json", "certificates/Q05_P6_graphs.json",
         "certificates/Q05_P6_rows.json", "certificates/Q05_P6_status.json", "certificates/Q05_guard_scan.json",
         "certificates/Q05_release.json", "ledgers/Q05_milestones.json",
         "certificates/Q06_ghost_rows.json", "certificates/Q06_G10_crosscheck.json", "certificates/Q06_V4_validation.json",
         "certificates/Q06_fastpath.json", "certificates/Q06_P6_hhh.json", "certificates/Q06_P6_status.json",
         "certificates/Q06_release.json", "ledgers/Q06_milestones.json"]


def strip(o):
    if isinstance(o, dict):
```

```
        return {k: strip(v) for k, v in o.items() if k != "seconds"}
    if isinstance(o, list):
        return [strip(v) for v in o]
    return o


def values(root):
    v = {c: hashlib.sha256(json.dumps(strip(json.loads((root / c).read_text())), sort_keys=True).encode()).hexdigest()
         for c in CERTS}
    v.update({f"export_ell{k}": hashlib.sha256((root / f"build/xvip_run/exports/ell{k}_generating_polynomial.srepr")
                                               .read_bytes()).hexdigest() for k in (2, 3, 4)})
    return v


def main():
    if "--write-reference" in sys.argv:
        (Q / "reproducibility").mkdir(exist_ok=True)
        (Q / REF).write_text(json.dumps({"tree_sha256_without_reference": SS.tree_sha(Q, exclude=(REF,)),
                                         "values": values(Q)}, indent=1) + "\n")
        print("written", REF)
        return 0
    ref = json.loads((Q / REF).read_text())
    with tempfile.TemporaryDirectory(prefix="q06src_") as td:
        zp = Path(td) / "src.zip"
        SS.make_zip(zp, Q)
        zipfile.ZipFile(zp).extractall(td)
        root = Path(td) / "XVI-Q"
        r = subprocess.run([sys.executable, "tools/build_q06.py"], cwd=root, capture_output=True, text=True,
                           env=dict(os.environ, Q_SYNC_DIR="none"))
        (Q / "build").mkdir(exist_ok=True)
        (Q / "build" / "rebuild_build.log").write_text(r.stdout + r.stderr)
        built = r.returncode == 0 and "BUILD_OK" in r.stdout
        got = values(root) if built else {}
        diff = {k: (v, got.get(k)) for k, v in ref["values"].items() if got.get(k) != v}
        v_files = {f: hashlib.sha256((Q / f).read_bytes()).hexdigest() for f in ('code/q02/release_q02.py', 'code/q02/q02_expected.json',
'code/q03/release_q03.py', 'code/q03/q03_expected.json', 'code/q04/release_q04.py', 'code/q04/q04_expected.json', 'code/p6q05/release_q05.py',
'code/p6q05/q05_expected.json', 'code/p6q06/release_q06.py', 'code/p6q06/q06_expected.json')}
        rep = {"landmark": "XVI-Q", "release": "Q06", "zip_sha256": hashlib.sha256(zp.read_bytes()).hexdigest(),
               "source_tree_sha256": SS.tree_sha(root), "tree_matches_reference": SS.tree_sha(root, exclude=(REF,))
               == ref["tree_sha256_without_reference"], "full_build_ok": built, "compared": len(ref["values"]),
               "equal": len(ref["values"]) - len(diff), "differences": diff, "gate_and_expected_sha256": v_files}
        rep["pass"] = bool(built and not diff and rep["tree_matches_reference"])
    out = Path(sys.argv[sys.argv.index("--report") + 1]) if "--report" in sys.argv else Q / "build" / "rebuild_report.json"
    out.write_text(json.dumps(rep, indent=1) + "\n")
    print(json.dumps(rep, indent=1))
    return 0 if rep["pass"] else 1


if __name__ == "__main__":
    raise SystemExit(main())
```

tools/rebuild_q06_q07.py

```
#!/usr/bin/env python3
"""XVI-Q Q07 rebuild entrypoint.  --write-reference (author, built tree): reproducibility/reference_values_q07.json =
content sha256 (run-specific keys 'seconds' removed) of every Q01/Q02 certificate + RELEASE_STATUS + export shas.
Default: copy the source-only zip's tree to a temp dir, run tools/build.py there (no sync), compare with the reference."""
from __future__ import annotations
import hashlib, json, os, shutil, subprocess, sys, tempfile, zipfile  # noqa: E401
from pathlib import Path

Q = Path(__file__).resolve().parents[1]
sys.path.insert(0, str(Q / "tools"))
import source_set_q06_q07 as SS  # noqa: E402
REF = "reproducibility/reference_values_q07.json"
CERTS = ["certificates/Q01_intake.json", "certificates/Q01_O2.json", "certificates/Q01_guard.json",
         "certificates/Q01_release.json", "RELEASE_STATUS.json", "ledgers/Q01_milestones.json",
         "certificates/Q02_bulk_graphs.json", "certificates/Q02_D12_reconciliation.json", "certificates/Q02_bulk_identity.json",
         "certificates/Q02_O1.json", "certificates/Q02_D04_scrutiny.json", "certificates/Q02_release.json", "ledgers/Q02_milestones.json",
         "certificates/Q03_O1_record.json", "certificates/Q03_guard_ring.json", "certificates/Q03_release.json", "ledgers/Q03_milestones.json",
         "certificates/Q04_P6_inventory.json", "certificates/Q04_P6_identity.json", "certificates/Q04_P6_eval.json",
         "certificates/Q04_P6_status.json", "certificates/Q04_release.json", "ledgers/Q04_milestones.json",
         "certificates/Q04_guard_scan.json", "certificates/Q05_rules_validation.json", "certificates/Q05_P6_graphs.json",
         "certificates/Q05_P6_rows.json", "certificates/Q05_P6_status.json", "certificates/Q05_guard_scan.json",
         "certificates/Q05_release.json", "ledgers/Q05_milestones.json",
         "certificates/Q06_ghost_rows.json", "certificates/Q06_G10_crosscheck.json", "certificates/Q06_V4_validation.json",
         "certificates/Q06_fastpath.json", "certificates/Q06_P6_hhh.json", "certificates/Q06_P6_status.json",
         "certificates/Q06_release.json", "certificates/Q07_hhc_row.json", "certificates/Q07_evanescent_CT.json",
         "certificates/Q07_P6_status.json", "certificates/Q07_release.json", "ledgers/Q07_milestones.json"]


def strip(o):
    if isinstance(o, dict):
```

```
        return {k: strip(v) for k, v in o.items() if k != "seconds"}
    if isinstance(o, list):
        return [strip(v) for v in o]
    return o


def values(root):
    v = {c: hashlib.sha256(json.dumps(strip(json.loads((root / c).read_text())), sort_keys=True).encode()).hexdigest()
         for c in CERTS}
    v.update({f"export_ell{k}": hashlib.sha256((root / f"build/xvip_run/exports/ell{k}_generating_polynomial.srepr")
                                               .read_bytes()).hexdigest() for k in (2, 3, 4)})
    return v


def main():
    if "--write-reference" in sys.argv:
        (Q / "reproducibility").mkdir(exist_ok=True)
        (Q / REF).write_text(json.dumps({"tree_sha256_without_reference": SS.tree_sha(Q, exclude=(REF,)),
                                         "values": values(Q)}, indent=1) + "\n")
        print("written", REF)
        return 0
    ref = json.loads((Q / REF).read_text())
    with tempfile.TemporaryDirectory(prefix="q07src_") as td:
        zp = Path(td) / "src.zip"
        SS.make_zip(zp, Q)
        zipfile.ZipFile(zp).extractall(td)
        root = Path(td) / "XVI-Q"
        r = subprocess.run([sys.executable, "tools/build_q06_q07.py"], cwd=root, capture_output=True, text=True,
                           env=dict(os.environ, Q_SYNC_DIR="none"))
        (Q / "build").mkdir(exist_ok=True)
        (Q / "build" / "rebuild_build.log").write_text(r.stdout + r.stderr)
        built = r.returncode == 0 and "BUILD_OK" in r.stdout
        got = values(root) if built else {}
        diff = {k: (v, got.get(k)) for k, v in ref["values"].items() if got.get(k) != v}
        v_files = {f: hashlib.sha256((Q / f).read_bytes()).hexdigest() for f in ('code/q02/release_q02.py', 'code/q02/q02_expected.json',
'code/q03/release_q03.py', 'code/q03/q03_expected.json', 'code/q04/release_q04.py', 'code/q04/q04_expected.json', 'code/p6q05/release_q05.py',
'code/p6q05/q05_expected.json', 'code/p6q06/release_q06.py', 'code/p6q06/q06_expected.json', 'code/p6q06/q07/release_q07.py',
'code/p6q06/q07/q07_expected.json')}
        rep = {"landmark": "XVI-Q", "release": "Q07", "zip_sha256": hashlib.sha256(zp.read_bytes()).hexdigest(),
               "source_tree_sha256": SS.tree_sha(root), "tree_matches_reference": SS.tree_sha(root, exclude=(REF,))
               == ref["tree_sha256_without_reference"], "full_build_ok": built, "compared": len(ref["values"]),
               "equal": len(ref["values"]) - len(diff), "differences": diff, "gate_and_expected_sha256": v_files}
        rep["pass"] = bool(built and not diff and rep["tree_matches_reference"])
    out = Path(sys.argv[sys.argv.index("--report") + 1]) if "--report" in sys.argv else Q / "build" / "rebuild_report.json"
    out.write_text(json.dumps(rep, indent=1) + "\n")
    print(json.dumps(rep, indent=1))
    return 0 if rep["pass"] else 1


if __name__ == "__main__":
    raise SystemExit(main())
```

tools/rebuild_q06_q07_q08.py

```
#!/usr/bin/env python3
"""XVI-Q Q08 rebuild entrypoint.  --write-reference (author, built tree): reproducibility/reference_values_q08.json =
content sha256 (run-specific keys 'seconds' removed) of every Q01/Q02 certificate + RELEASE_STATUS + export shas.
Default: copy the source-only zip's tree to a temp dir, run tools/build.py there (no sync), compare with the reference."""
from __future__ import annotations
import hashlib, json, os, shutil, subprocess, sys, tempfile, zipfile  # noqa: E401
from pathlib import Path

Q = Path(__file__).resolve().parents[1]
sys.path.insert(0, str(Q / "tools"))
import source_set_q06_q07_q08 as SS  # noqa: E402
REF = "reproducibility/reference_values_q08.json"
CERTS = ["certificates/Q01_intake.json", "certificates/Q01_O2.json", "certificates/Q01_guard.json",
         "certificates/Q01_release.json", "RELEASE_STATUS.json", "ledgers/Q01_milestones.json",
         "certificates/Q02_bulk_graphs.json", "certificates/Q02_D12_reconciliation.json", "certificates/Q02_bulk_identity.json",
         "certificates/Q02_O1.json", "certificates/Q02_D04_scrutiny.json", "certificates/Q02_release.json", "ledgers/Q02_milestones.json",
         "certificates/Q03_O1_record.json", "certificates/Q03_guard_ring.json", "certificates/Q03_release.json", "ledgers/Q03_milestones.json",
         "certificates/Q04_P6_inventory.json", "certificates/Q04_P6_identity.json", "certificates/Q04_P6_eval.json",
         "certificates/Q04_P6_status.json", "certificates/Q04_release.json", "ledgers/Q04_milestones.json",
         "certificates/Q04_guard_scan.json", "certificates/Q05_rules_validation.json", "certificates/Q05_P6_graphs.json",
         "certificates/Q05_P6_rows.json", "certificates/Q05_P6_status.json", "certificates/Q05_guard_scan.json",
         "certificates/Q05_release.json", "ledgers/Q05_milestones.json",
         "certificates/Q06_ghost_rows.json", "certificates/Q06_G10_crosscheck.json", "certificates/Q06_V4_validation.json",
         "certificates/Q06_fastpath.json", "certificates/Q06_P6_hhh.json", "certificates/Q06_P6_status.json",
         "certificates/Q06_release.json", "certificates/Q07_hhc_row.json", "certificates/Q07_evanescent_CT.json",
         "certificates/Q07_P6_status.json", "certificates/Q07_release.json", "ledgers/Q07_milestones.json", "certificates/Q08_ghost_equation.json",
         "certificates/Q08_free_directions.json", "certificates/Q08_P6_status.json", "certificates/Q08_release.json", "ledgers/Q08_milestones.json"]
```

```
def strip(o):
    if isinstance(o, dict):
        return {k: strip(v) for k, v in o.items() if k != "seconds"}
    if isinstance(o, list):
        return [strip(v) for v in o]
    return o


def values(root):
    v = {c: hashlib.sha256(json.dumps(strip(json.loads((root / c).read_text())), sort_keys=True).encode()).hexdigest()
         for c in CERTS}
    v.update({f"export_ell{k}": hashlib.sha256((root / f"build/xvip_run/exports/ell{k}_generating_polynomial.srepr")
                                               .read_bytes()).hexdigest() for k in (2, 3, 4)})
    return v


def main():
    if "--write-reference" in sys.argv:
        (Q / "reproducibility").mkdir(exist_ok=True)
        (Q / REF).write_text(json.dumps({"tree_sha256_without_reference": SS.tree_sha(Q, exclude=(REF,)),
                                         "values": values(Q)}, indent=1) + "\n")
        print("written", REF)
        return 0
    ref = json.loads((Q / REF).read_text())
    with tempfile.TemporaryDirectory(prefix="q08src_") as td:
        zp = Path(td) / "src.zip"
        SS.make_zip(zp, Q)
        zipfile.ZipFile(zp).extractall(td)
        root = Path(td) / "XVI-Q"
        r = subprocess.run([sys.executable, "tools/build_q06_q07_q08.py"], cwd=root, capture_output=True, text=True,
                           env=dict(os.environ, Q_SYNC_DIR="none"))
        (Q / "build").mkdir(exist_ok=True)
        (Q / "build" / "rebuild_build.log").write_text(r.stdout + r.stderr)
        built = r.returncode == 0 and "BUILD_OK" in r.stdout
        got = values(root) if built else {}
        diff = {k: (v, got.get(k)) for k, v in ref["values"].items() if got.get(k) != v}
        v_files = {f: hashlib.sha256((Q / f).read_bytes()).hexdigest() for f in ('code/q02/release_q02.py', 'code/q02/q02_expected.json',
'code/q03/release_q03.py', 'code/q03/q03_expected.json', 'code/q04/release_q04.py', 'code/q04/q04_expected.json', 'code/p6q05/release_q05.py',
'code/p6q05/q05_expected.json', 'code/p6q06/release_q06.py', 'code/p6q06/q06_expected.json', 'code/p6q06/q07/release_q07.py',
'code/p6q06/q07/q07_expected.json', 'code/p6q06/q07/q08/release_q08.py', 'code/p6q06/q07/q08/q08_expected.json')}
        rep = {"landmark": "XVI-Q", "release": "Q08", "zip_sha256": hashlib.sha256(zp.read_bytes()).hexdigest(),
               "source_tree_sha256": SS.tree_sha(root), "tree_matches_reference": SS.tree_sha(root, exclude=(REF,))
               == ref["tree_sha256_without_reference"], "full_build_ok": built, "compared": len(ref["values"]),
               "equal": len(ref["values"]) - len(diff), "differences": diff, "gate_and_expected_sha256": v_files}
        rep["pass"] = bool(built and not diff and rep["tree_matches_reference"])
    out = Path(sys.argv[sys.argv.index("--report") + 1]) if "--report" in sys.argv else Q / "build" / "rebuild_report.json"
    out.write_text(json.dumps(rep, indent=1) + "\n")
    print(json.dumps(rep, indent=1))
    return 0 if rep["pass"] else 1


if __name__ == "__main__":
    raise SystemExit(main())
```

tools/rebuild_q06_q07_q08_q09.py

```
#!/usr/bin/env python3
"""XVI-Q Q09 rebuild entrypoint.  --write-reference (author, built tree): reproducibility/reference_values_q09.json =
content sha256 (run-specific keys 'seconds' removed) of every Q01/Q02 certificate + RELEASE_STATUS + export shas.
Default: copy the source-only zip's tree to a temp dir, run tools/build.py there (no sync), compare with the reference."""
from __future__ import annotations
import hashlib, json, os, shutil, subprocess, sys, tempfile, zipfile  # noqa: E401
from pathlib import Path

Q = Path(__file__).resolve().parents[1]
sys.path.insert(0, str(Q / "tools"))
import source_set_q06_q07_q08_q09 as SS  # noqa: E402
REF = "reproducibility/reference_values_q09.json"
CERTS = ["certificates/Q01_intake.json", "certificates/Q01_O2.json", "certificates/Q01_guard.json",
         "certificates/Q01_release.json", "RELEASE_STATUS.json", "ledgers/Q01_milestones.json",
         "certificates/Q02_bulk_graphs.json", "certificates/Q02_D12_reconciliation.json", "certificates/Q02_bulk_identity.json",
         "certificates/Q02_O1.json", "certificates/Q02_D04_scrutiny.json", "certificates/Q02_release.json", "ledgers/Q02_milestones.json",
         "certificates/Q03_O1_record.json", "certificates/Q03_guard_ring.json", "certificates/Q03_release.json", "ledgers/Q03_milestones.json",
         "certificates/Q04_P6_inventory.json", "certificates/Q04_P6_identity.json", "certificates/Q04_P6_eval.json",
         "certificates/Q04_P6_status.json", "certificates/Q04_release.json", "ledgers/Q04_milestones.json",
         "certificates/Q04_guard_scan.json", "certificates/Q05_rules_validation.json", "certificates/Q05_P6_graphs.json",
         "certificates/Q05_P6_rows.json", "certificates/Q05_P6_status.json", "certificates/Q05_guard_scan.json",
         "certificates/Q05_release.json", "ledgers/Q05_milestones.json",
         "certificates/Q06_ghost_rows.json", "certificates/Q06_G10_crosscheck.json", "certificates/Q06_V4_validation.json",
         "certificates/Q06_fastpath.json", "certificates/Q06_P6_hhh.json", "certificates/Q06_P6_status.json",
         "certificates/Q06_release.json", "certificates/Q07_hhc_row.json", "certificates/Q07_evanescent_CT.json",
         "certificates/Q07_P6_status.json", "certificates/Q07_release.json", "ledgers/Q07_milestones.json", "certificates/Q08_ghost_equation.json",
         "certificates/Q08_free_directions.json", "certificates/Q08_P6_status.json", "certificates/Q08_release.json", "ledgers/Q08_milestones.json",
```

```
"certificates/Q09_guard_scan3.json",
         "certificates/Q09_q04_report_binding.json", "certificates/Q09_trivial_families.json", "certificates/Q09_P6_status.json",
"certificates/Q09_release.json", "ledgers/Q09_milestones.json"]


def strip(o):
    if isinstance(o, dict):
        return {k: strip(v) for k, v in o.items() if k != "seconds"}
    if isinstance(o, list):
        return [strip(v) for v in o]
    return o


def values(root):
    v = {c: hashlib.sha256(json.dumps(strip(json.loads((root / c).read_text())), sort_keys=True).encode()).hexdigest()
         for c in CERTS}
    v.update({f"export_ell{k}": hashlib.sha256((root / f"build/xvip_run/exports/ell{k}_generating_polynomial.srepr")
                                               .read_bytes()).hexdigest() for k in (2, 3, 4)})
    return v


def main():
    if "--write-reference" in sys.argv:
        (Q / "reproducibility").mkdir(exist_ok=True)
        (Q / REF).write_text(json.dumps({"tree_sha256_without_reference": SS.tree_sha(Q, exclude=(REF,)),
                                         "values": values(Q)}, indent=1) + "\n")
        print("written", REF)
        return 0
    ref = json.loads((Q / REF).read_text())
    with tempfile.TemporaryDirectory(prefix="q09src_") as td:
        zp = Path(td) / "src.zip"
        SS.make_zip(zp, Q)
        zipfile.ZipFile(zp).extractall(td)
        root = Path(td) / "XVI-Q"
        r = subprocess.run([sys.executable, "tools/build_q06_q07_q08_q09.py"], cwd=root, capture_output=True, text=True,
                           env=dict(os.environ, Q_SYNC_DIR="none"))
        (Q / "build").mkdir(exist_ok=True)
        (Q / "build" / "rebuild_build.log").write_text(r.stdout + r.stderr)
        built = r.returncode == 0 and "BUILD_OK" in r.stdout
        got = values(root) if built else {}
        diff = {k: (v, got.get(k)) for k, v in ref["values"].items() if got.get(k) != v}
        v_files = {f: hashlib.sha256((Q / f).read_bytes()).hexdigest() for f in ('code/q02/release_q02.py', 'code/q02/q02_expected.json',
'code/q03/release_q03.py', 'code/q03/q03_expected.json', 'code/q04/release_q04.py', 'code/q04/q04_expected.json', 'code/p6q05/release_q05.py',
'code/p6q05/q05_expected.json', 'code/p6q06/release_q06.py', 'code/p6q06/q06_expected.json', 'code/p6q06/q07/release_q07.py',
'code/p6q06/q07/q07_expected.json', 'code/p6q06/q07/q08/release_q08.py', 'code/p6q06/q07/q08/q08_expected.json',
'code/p6q06/q07/q08/q09/release_q09.py', 'code/p6q06/q07/q08/q09/q09_expected.json')}
        rep = {"landmark": "XVI-Q", "release": "Q09", "zip_sha256": hashlib.sha256(zp.read_bytes()).hexdigest(),
               "source_tree_sha256": SS.tree_sha(root), "tree_matches_reference": SS.tree_sha(root, exclude=(REF,))
               == ref["tree_sha256_without_reference"], "full_build_ok": built, "compared": len(ref["values"]),
               "equal": len(ref["values"]) - len(diff), "differences": diff, "gate_and_expected_sha256": v_files}
        rep["pass"] = bool(built and not diff and rep["tree_matches_reference"])
    out = Path(sys.argv[sys.argv.index("--report") + 1]) if "--report" in sys.argv else Q / "build" / "rebuild_report.json"
    out.write_text(json.dumps(rep, indent=1) + "\n")
    print(json.dumps(rep, indent=1))
    return 0 if rep["pass"] else 1


if __name__ == "__main__":
    raise SystemExit(main())
```

tools/rebuild_q06_q07_q08_q09_q10.py

```
#!/usr/bin/env python3
"""XVI-Q Q10 rebuild entrypoint.  --write-reference (author, built tree): reproducibility/reference_values_q10.json =
content sha256 (run-specific keys 'seconds' removed) of every Q01/Q02 certificate + RELEASE_STATUS + export shas.
Default: copy the source-only zip's tree to a temp dir, run tools/build.py there (no sync), compare with the reference."""
from __future__ import annotations
import hashlib, json, os, shutil, subprocess, sys, tempfile, zipfile  # noqa: E401
from pathlib import Path

Q = Path(__file__).resolve().parents[1]
sys.path.insert(0, str(Q / "tools"))
import source_set_q06_q07_q08_q09_q10 as SS  # noqa: E402
REF = "reproducibility/reference_values_q10.json"
CERTS = ["certificates/Q01_intake.json", "certificates/Q01_O2.json", "certificates/Q01_guard.json",
         "certificates/Q01_release.json", "RELEASE_STATUS.json", "ledgers/Q01_milestones.json",
         "certificates/Q02_bulk_graphs.json", "certificates/Q02_D12_reconciliation.json", "certificates/Q02_bulk_identity.json",
         "certificates/Q02_O1.json", "certificates/Q02_D04_scrutiny.json", "certificates/Q02_release.json", "ledgers/Q02_milestones.json",
         "certificates/Q03_O1_record.json", "certificates/Q03_guard_ring.json", "certificates/Q03_release.json", "ledgers/Q03_milestones.json",
         "certificates/Q04_P6_inventory.json", "certificates/Q04_P6_identity.json", "certificates/Q04_P6_eval.json",
         "certificates/Q04_P6_status.json", "certificates/Q04_release.json", "ledgers/Q04_milestones.json",
         "certificates/Q04_guard_scan.json", "certificates/Q05_rules_validation.json", "certificates/Q05_P6_graphs.json",
         "certificates/Q05_P6_rows.json", "certificates/Q05_P6_status.json", "certificates/Q05_guard_scan.json",
```

```
        "certificates/Q05_release.json", "ledgers/Q05_milestones.json",
        "certificates/Q06_ghost_rows.json", "certificates/Q06_G10_crosscheck.json", "certificates/Q06_V4_validation.json",
        "certificates/Q06_fastpath.json", "certificates/Q06_P6_hhh.json", "certificates/Q06_P6_status.json",
        "certificates/Q06_release.json", "certificates/Q07_hhc_row.json", "certificates/Q07_evanescent_CT.json",
        "certificates/Q07_P6_status.json", "certificates/Q07_release.json", "ledgers/Q07_milestones.json", "certificates/Q08_ghost_equation.json",
        "certificates/Q08_free_directions.json", "certificates/Q08_P6_status.json", "certificates/Q08_release.json", "ledgers/Q08_milestones.json",
"certificates/Q09_guard_scan3.json",
        "certificates/Q09_q04_report_binding.json", "certificates/Q09_trivial_families.json", "certificates/Q09_P6_status.json",
"certificates/Q09_release.json", "ledgers/Q09_milestones.json", "certificates/Q10_family_anatomy.json", "certificates/Q10_erratum_Q08_counts.json",
"certificates/Q10_new_manuscripts.json", "certificates/Q10_P6_status.json", "certificates/Q10_release.json", "ledgers/Q10_milestones.json"]


def strip(o):
    if isinstance(o, dict):
        return {k: strip(v) for k, v in o.items() if k != "seconds"}
    if isinstance(o, list):
        return [strip(v) for v in o]
    return o


def values(root):
    v = {c: hashlib.sha256(json.dumps(strip(json.loads((root / c).read_text())), sort_keys=True).encode()).hexdigest()
         for c in CERTS}
    v.update({f"export_ell{k}": hashlib.sha256((root / f"build/xvip_run/exports/ell{k}_generating_polynomial.srepr")
                                               .read_bytes()).hexdigest() for k in (2, 3, 4)})
    return v


def main():
    if "--write-reference" in sys.argv:
        (Q / "reproducibility").mkdir(exist_ok=True)
        (Q / REF).write_text(json.dumps({"tree_sha256_without_reference": SS.tree_sha(Q, exclude=(REF,)),
                                         "values": values(Q)}, indent=1) + "\n")
        print("written", REF)
        return 0
    ref = json.loads((Q / REF).read_text())
    with tempfile.TemporaryDirectory(prefix="q10src_") as td:
        zp = Path(td) / "src.zip"
        SS.make_zip(zp, Q)
        zipfile.ZipFile(zp).extractall(td)
        root = Path(td) / "XVI-Q"
        r = subprocess.run([sys.executable, "tools/build_q06_q07_q08_q09_q10.py"], cwd=root, capture_output=True, text=True,
                           env=dict(os.environ, Q_SYNC_DIR="none"))
        (Q / "build").mkdir(exist_ok=True)
        (Q / "build" / "rebuild_build.log").write_text(r.stdout + r.stderr)
        built = r.returncode == 0 and "BUILD_OK" in r.stdout
        got = values(root) if built else {}
        diff = {k: (v, got.get(k)) for k, v in ref["values"].items() if got.get(k) != v}
        v_files = {f: hashlib.sha256((Q / f).read_bytes()).hexdigest() for f in ('code/q02/release_q02.py', 'code/q02/q02_expected.json',
'code/q03/release_q03.py', 'code/q03/q03_expected.json', 'code/q04/release_q04.py', 'code/q04/q04_expected.json', 'code/p6q05/release_q05.py',
'code/p6q05/q05_expected.json', 'code/p6q06/release_q06.py', 'code/p6q06/q06_expected.json', 'code/p6q06/q07/release_q07.py',
'code/p6q06/q07/q07_expected.json', 'code/p6q06/q07/q08/release_q08.py', 'code/p6q06/q07/q08/q08_expected.json',
'code/p6q06/q07/q08/q09/release_q09.py', 'code/p6q06/q07/q08/q09/q09_expected.json', 'code/p6q06/q07/q08/q09/q10/release_q10.py',
'code/p6q06/q07/q08/q09/q10/q10_expected.json')}
        rep = {"landmark": "XVI-Q", "release": "Q10", "zip_sha256": hashlib.sha256(zp.read_bytes()).hexdigest(),
               "source_tree_sha256": SS.tree_sha(root), "tree_matches_reference": SS.tree_sha(root, exclude=(REF,))
               == ref["tree_sha256_without_reference"], "full_build_ok": built, "compared": len(ref["values"]),
               "equal": len(ref["values"]) - len(diff), "differences": diff, "gate_and_expected_sha256": v_files}
        rep["pass"] = bool(built and not diff and rep["tree_matches_reference"])
    out = Path(sys.argv[sys.argv.index("--report") + 1]) if "--report" in sys.argv else Q / "build" / "rebuild_report.json"
    out.write_text(json.dumps(rep, indent=1) + "\n")
    print(json.dumps(rep, indent=1))
    return 0 if rep["pass"] else 1


if __name__ == "__main__":
    raise SystemExit(main())
```

tools/rebuild_q06_q07_q08_q09_q10_q11.py

```
#!/usr/bin/env python3
"""XVI-Q Q11 rebuild entrypoint.  --write-reference (author, built tree): reproducibility/reference_values_q11.json =
content sha256 (run-specific keys 'seconds' removed) of every Q01/Q02 certificate + RELEASE_STATUS + export shas.
Default: copy the source-only zip's tree to a temp dir, run tools/build.py there (no sync), compare with the reference."""
from __future__ import annotations
import hashlib, json, os, shutil, subprocess, sys, tempfile, zipfile  # noqa: E401
from pathlib import Path

Q = Path(__file__).resolve().parents[1]
sys.path.insert(0, str(Q / "tools"))
import source_set_q06_q07_q08_q09_q10_q11 as SS  # noqa: E402
REF = "reproducibility/reference_values_q11.json"
CERTS = ["certificates/Q01_intake.json", "certificates/Q01_O2.json", "certificates/Q01_guard.json",
```

```
        "certificates/Q01_release.json", "RELEASE_STATUS.json", "ledgers/Q01_milestones.json",
        "certificates/Q02_bulk_graphs.json", "certificates/Q02_D12_reconciliation.json", "certificates/Q02_bulk_identity.json",
        "certificates/Q02_O1.json", "certificates/Q02_D04_scrutiny.json", "certificates/Q02_release.json", "ledgers/Q02_milestones.json",
        "certificates/Q03_O1_record.json", "certificates/Q03_guard_ring.json", "certificates/Q03_release.json", "ledgers/Q03_milestones.json",
        "certificates/Q04_P6_inventory.json", "certificates/Q04_P6_identity.json", "certificates/Q04_P6_eval.json",
        "certificates/Q04_P6_status.json", "certificates/Q04_release.json", "ledgers/Q04_milestones.json",
        "certificates/Q04_guard_scan.json", "certificates/Q05_rules_validation.json", "certificates/Q05_P6_graphs.json",
        "certificates/Q05_P6_rows.json", "certificates/Q05_P6_status.json", "certificates/Q05_guard_scan.json",
        "certificates/Q05_release.json", "ledgers/Q05_milestones.json",
        "certificates/Q06_ghost_rows.json", "certificates/Q06_G10_crosscheck.json", "certificates/Q06_V4_validation.json",
        "certificates/Q06_fastpath.json", "certificates/Q06_P6_hhh.json", "certificates/Q06_P6_status.json",
        "certificates/Q06_release.json", "certificates/Q07_hhc_row.json", "certificates/Q07_evanescent_CT.json",
        "certificates/Q07_P6_status.json", "certificates/Q07_release.json", "ledgers/Q07_milestones.json", "certificates/Q08_ghost_equation.json",
        "certificates/Q08_free_directions.json", "certificates/Q08_P6_status.json", "certificates/Q08_release.json", "ledgers/Q08_milestones.json",
"certificates/Q09_guard_scan3.json",
        "certificates/Q09_q04_report_binding.json", "certificates/Q09_trivial_families.json", "certificates/Q09_P6_status.json",
"certificates/Q09_release.json", "ledgers/Q09_milestones.json", "certificates/Q10_family_anatomy.json", "certificates/Q10_erratum_Q08_counts.json",
"certificates/Q10_new_manuscripts.json", "certificates/Q10_P6_status.json", "certificates/Q10_release.json", "ledgers/Q10_milestones.json",
"certificates/Q11_sigma_images.json", "certificates/Q11_classification.json", "certificates/Q11_P6_status.json", "certificates/Q11_release.json",
"ledgers/Q11_milestones.json"]


def strip(o):
    if isinstance(o, dict):
        return {k: strip(v) for k, v in o.items() if k != "seconds"}
    if isinstance(o, list):
        return [strip(v) for v in o]
    return o


def values(root):
    v = {c: hashlib.sha256(json.dumps(strip(json.loads((root / c).read_text())), sort_keys=True).encode()).hexdigest()
         for c in CERTS}
    v.update({f"export_ell{k}": hashlib.sha256((root / f"build/xvip_run/exports/ell{k}_generating_polynomial.srepr")
                                               .read_bytes()).hexdigest() for k in (2, 3, 4)})
    return v


def main():
    if "--write-reference" in sys.argv:
        (Q / "reproducibility").mkdir(exist_ok=True)
        (Q / REF).write_text(json.dumps({"tree_sha256_without_reference": SS.tree_sha(Q, exclude=(REF,)),
                                         "values": values(Q)}, indent=1) + "\n")
        print("written", REF)
        return 0
    ref = json.loads((Q / REF).read_text())
    with tempfile.TemporaryDirectory(prefix="q11src_") as td:
        zp = Path(td) / "src.zip"
        SS.make_zip(zp, Q)
        zipfile.ZipFile(zp).extractall(td)
        root = Path(td) / "XVI-Q"
        r = subprocess.run([sys.executable, "tools/build_q06_q07_q08_q09_q10_q11.py"], cwd=root, capture_output=True, text=True,
                           env=dict(os.environ, Q_SYNC_DIR="none"))
        (Q / "build").mkdir(exist_ok=True)
        (Q / "build" / "rebuild_build.log").write_text(r.stdout + r.stderr)
        built = r.returncode == 0 and "BUILD_OK" in r.stdout
        got = values(root) if built else {}
        diff = {k: (v, got.get(k)) for k, v in ref["values"].items() if got.get(k) != v}
        v_files = {f: hashlib.sha256((Q / f).read_bytes()).hexdigest() for f in ('code/q02/release_q02.py', 'code/q02/q02_expected.json',
'code/q03/release_q03.py', 'code/q03/q03_expected.json', 'code/q04/release_q04.py', 'code/q04/q04_expected.json', 'code/p6q05/release_q05.py',
'code/p6q05/q05_expected.json', 'code/p6q06/release_q06.py', 'code/p6q06/q06_expected.json', 'code/p6q06/q07/release_q07.py',
'code/p6q06/q07/q07_expected.json', 'code/p6q06/q07/q08/release_q08.py', 'code/p6q06/q07/q08/q08_expected.json',
'code/p6q06/q07/q08/q09/release_q09.py', 'code/p6q06/q07/q08/q09/q09_expected.json', 'code/p6q06/q07/q08/q09/q10/release_q10.py',
'code/p6q06/q07/q08/q09/q10/q10_expected.json', 'code/p6q06/q07/q08/q09/q10/q11/release_q11.py', 'code/p6q06/q07/q08/q09/q10/q11/q11_expected.json')}
        rep = {"landmark": "XVI-Q", "release": "Q11", "zip_sha256": hashlib.sha256(zp.read_bytes()).hexdigest(),
               "source_tree_sha256": SS.tree_sha(root), "tree_matches_reference": SS.tree_sha(root, exclude=(REF,))
               == ref["tree_sha256_without_reference"], "full_build_ok": built, "compared": len(ref["values"]),
               "equal": len(ref["values"]) - len(diff), "differences": diff, "gate_and_expected_sha256": v_files}
        rep["pass"] = bool(built and not diff and rep["tree_matches_reference"])
    out = Path(sys.argv[sys.argv.index("--report") + 1]) if "--report" in sys.argv else Q / "build" / "rebuild_report.json"
    out.write_text(json.dumps(rep, indent=1) + "\n")
    print(json.dumps(rep, indent=1))
    return 0 if rep["pass"] else 1


if __name__ == "__main__":
    raise SystemExit(main())
```

tools/source_set.py

```
"""XVI-Q source set (what the source-only zip contains) and helpers."""
from __future__ import annotations
import hashlib, textwrap, zipfile  # noqa: E401
```

```
from pathlib import Path

GLOBS = ("README.md", "REBUILD_INSTRUCTIONS.md", "registration/*.json", "intake/*.json", "provenance/**/*",
         "code/**/*.py", "code/*/q0*_expected.json", "reproducibility/independent_rebuild_report_q02.json",
"reproducibility/independent_rebuild_report_q03.json", "tests/*.py", "tools/*.py", "manuscript/notes/*.tex",
"reproducibility/reference_values_q04.json")


def source_files(root):
    root = Path(root)
    out = set()
    for g in GLOBS:
        out |= {p.relative_to(root).as_posix() for p in root.glob(g) if p.is_file() and "__pycache__" not in p.parts}
    return sorted(out)


def tree_sha(root, exclude=()):
    root = Path(root)
    lines = [f"{r}\t{hashlib.sha256((root / r).read_bytes()).hexdigest()}" for r in source_files(root) if r not in exclude]
    return hashlib.sha256(("\n".join(lines) + "\n").encode()).hexdigest()


def make_zip(out, root):
    with zipfile.ZipFile(out, "w", zipfile.ZIP_DEFLATED) as z:
        for r in source_files(root):
            zi = zipfile.ZipInfo(f"XVI-Q/{r}", date_time=(2026, 10, 10, 0, 0, 0))
            zi.compress_type = zipfile.ZIP_DEFLATED
            z.writestr(zi, (Path(root) / r).read_bytes())


def wrap(s, w=150):
    s = s.replace("\\end {verbatim}", "\\end {verbatim}")   # a listed file must not close the listing
    s = "".join(ch if ord(ch) < 128 else f"<U+{ord(ch):04X}>" for ch in s)   # Q02: pdflatex verbatim is ASCII-only
    return "\n".join(textwrap.fill(l, w, break_long_words=True, replace_whitespace=False) if len(l) > w else l
                     for l in s.splitlines())
```

tools/source_set_q05.py

```
"""XVI-Q Q05 source set: Q04 set + the Q04 independent report + Q05 reference values (tools/source_set.py is pinned by
the Q04 layer gate and left unchanged)."""
from __future__ import annotations
import sys
from pathlib import Path

sys.path.insert(0, str(Path(__file__).resolve().parent))
import source_set as S0  # noqa: E402

GLOBS = tuple(S0.GLOBS) + ("reproducibility/independent_rebuild_report_q04.json", "reproducibility/reference_values_q05.json")
wrap = S0.wrap


def source_files(root):
    root = Path(root)
    out = set()
    for g in GLOBS:
        out |= {p.relative_to(root).as_posix() for p in root.glob(g) if p.is_file() and "__pycache__" not in p.parts}
    return sorted(out)


def tree_sha(root, exclude=()):
    import hashlib
    root = Path(root)
    lines = [f"{r}\t{hashlib.sha256((root / r).read_bytes()).hexdigest()}" for r in source_files(root) if r not in exclude]
    return hashlib.sha256(("\n".join(lines) + "\n").encode()).hexdigest()


def make_zip(out, root):
    import zipfile
    with zipfile.ZipFile(out, "w", zipfile.ZIP_DEFLATED) as z:
        for r in source_files(root):
            zi = zipfile.ZipInfo(f"XVI-Q/{r}", date_time=(2026, 10, 11, 0, 0, 0))
            zi.compress_type = zipfile.ZIP_DEFLATED
            z.writestr(zi, (Path(root) / r).read_bytes())
```

tools/source_set_q06.py

```
"""XVI-Q Q06 source set: Q05 set + Q06 reference values (tools/source_set.py is pinned by
the Q04 layer gate and left unchanged)."""
from __future__ import annotations
import sys
from pathlib import Path
```

```
sys.path.insert(0, str(Path(__file__).resolve().parent))
import source_set as S0  # noqa: E402

GLOBS = tuple(S0.GLOBS) + ("reproducibility/independent_rebuild_report_q04.json", "reproducibility/reference_values_q05.json",
                           "reproducibility/reference_values_q06.json")
wrap = S0.wrap


def source_files(root):
    root = Path(root)
    out = set()
    for g in GLOBS:
        out |= {p.relative_to(root).as_posix() for p in root.glob(g) if p.is_file() and "__pycache__" not in p.parts}
    return sorted(out)


def tree_sha(root, exclude=()):
    import hashlib
    root = Path(root)
    lines = [f"{r}\t{hashlib.sha256((root / r).read_bytes()).hexdigest()}" for r in source_files(root) if r not in exclude]
    return hashlib.sha256(("\n".join(lines) + "\n").encode()).hexdigest()


def make_zip(out, root):
    import zipfile
    with zipfile.ZipFile(out, "w", zipfile.ZIP_DEFLATED) as z:
        for r in source_files(root):
            zi = zipfile.ZipInfo(f"XVI-Q/{r}", date_time=(2026, 10, 11, 0, 0, 0))
            zi.compress_type = zipfile.ZIP_DEFLATED
            z.writestr(zi, (Path(root) / r).read_bytes())
```

tools/source_set_q06_q07.py

```
"""XVI-Q Q07 source set: Q06 set + Q07 reference values + Q07 expected (tools/source_set.py is pinned by
the Q04 layer gate and left unchanged)."""
from __future__ import annotations
import sys
from pathlib import Path

sys.path.insert(0, str(Path(__file__).resolve().parent))
import source_set_q06 as S0  # noqa: E402

GLOBS = tuple(S0.GLOBS) + ("reproducibility/reference_values_q07.json",
                           "code/p6q06/q07/q07_expected.json")
wrap = S0.wrap


def source_files(root):
    root = Path(root)
    out = set()
    for g in GLOBS:
        out |= {p.relative_to(root).as_posix() for p in root.glob(g) if p.is_file() and "__pycache__" not in p.parts}
    return sorted(out)


def tree_sha(root, exclude=()):
    import hashlib
    root = Path(root)
    lines = [f"{r}\t{hashlib.sha256((root / r).read_bytes()).hexdigest()}" for r in source_files(root) if r not in exclude]
    return hashlib.sha256(("\n".join(lines) + "\n").encode()).hexdigest()


def make_zip(out, root):
    import zipfile
    with zipfile.ZipFile(out, "w", zipfile.ZIP_DEFLATED) as z:
        for r in source_files(root):
            zi = zipfile.ZipInfo(f"XVI-Q/{r}", date_time=(2026, 10, 11, 0, 0, 0))
            zi.compress_type = zipfile.ZIP_DEFLATED
            z.writestr(zi, (Path(root) / r).read_bytes())
```

tools/source_set_q06_q07_q08.py

```
"""XVI-Q Q08 source set: Q07 set + Q08 reference values + Q08 expected (tools/source_set.py is pinned by
the Q04 layer gate and left unchanged)."""
from __future__ import annotations
import sys
from pathlib import Path

sys.path.insert(0, str(Path(__file__).resolve().parent))
import source_set_q06_q07 as S0  # noqa: E402

GLOBS = tuple(S0.GLOBS) + ("reproducibility/reference_values_q08.json",
```

```
                                "code/p6q06/q07/q08/q08_expected.json")
wrap = S0.wrap


def source_files(root):
    root = Path(root)
    out = set()
    for g in GLOBS:
        out |= {p.relative_to(root).as_posix() for p in root.glob(g) if p.is_file() and "__pycache__" not in p.parts}
    return sorted(out)


def tree_sha(root, exclude=()):
    import hashlib
    root = Path(root)
    lines = [f"{r}\t{hashlib.sha256((root / r).read_bytes()).hexdigest()}" for r in source_files(root) if r not in exclude]
    return hashlib.sha256(("\n".join(lines) + "\n").encode()).hexdigest()


def make_zip(out, root):
    import zipfile
    with zipfile.ZipFile(out, "w", zipfile.ZIP_DEFLATED) as z:
        for r in source_files(root):
            zi = zipfile.ZipInfo(f"XVI-Q/{r}", date_time=(2026, 10, 11, 0, 0, 0))
            zi.compress_type = zipfile.ZIP_DEFLATED
            z.writestr(zi, (Path(root) / r).read_bytes())
```

tools/source_set_q06_q07_q08_q09.py

```
"""XVI-Q Q09 source set: Q08 set + Q09 reference values, expected, scan rules (tools/source_set.py is pinned by
the Q04 layer gate and left unchanged)."""
from __future__ import annotations
import sys
from pathlib import Path

sys.path.insert(0, str(Path(__file__).resolve().parent))
import source_set_q06_q07_q08 as S0  # noqa: E402

GLOBS = tuple(S0.GLOBS) + ("reproducibility/reference_values_q09.json", "code/p6q06/q07/q08/q09/guard_scan3_rules.json",
                           "code/p6q06/q07/q08/q09/q09_expected.json")
wrap = S0.wrap


def source_files(root):
    root = Path(root)
    out = set()
    for g in GLOBS:
        out |= {p.relative_to(root).as_posix() for p in root.glob(g) if p.is_file() and "__pycache__" not in p.parts}
    return sorted(out)


def tree_sha(root, exclude=()):
    import hashlib
    root = Path(root)
    lines = [f"{r}\t{hashlib.sha256((root / r).read_bytes()).hexdigest()}" for r in source_files(root) if r not in exclude]
    return hashlib.sha256(("\n".join(lines) + "\n").encode()).hexdigest()


def make_zip(out, root):
    import zipfile
    with zipfile.ZipFile(out, "w", zipfile.ZIP_DEFLATED) as z:
        for r in source_files(root):
            zi = zipfile.ZipInfo(f"XVI-Q/{r}", date_time=(2026, 10, 11, 0, 0, 0))
            zi.compress_type = zipfile.ZIP_DEFLATED
            z.writestr(zi, (Path(root) / r).read_bytes())
```

tools/source_set_q06_q07_q08_q09_q10.py

```
"""XVI-Q Q10 source set: Q09 set + Q10 reference values, expected, new-manuscript intake (tools/source_set.py is pinned by
the Q04 layer gate and left unchanged)."""
from __future__ import annotations
import sys
from pathlib import Path

sys.path.insert(0, str(Path(__file__).resolve().parent))
import source_set_q06_q07_q08_q09 as S0  # noqa: E402

GLOBS = tuple(S0.GLOBS) + ("reproducibility/reference_values_q10.json", "code/p6q06/q07/q08/q09/q10/q10_expected.json",
                           "intake/new_manuscripts_q10.json")
wrap = S0.wrap
```

```
def source_files(root):
    root = Path(root)
    out = set()
    for g in GLOBS:
        out |= {p.relative_to(root).as_posix() for p in root.glob(g) if p.is_file() and "__pycache__" not in p.parts}
    return sorted(out)


def tree_sha(root, exclude=()):
    import hashlib
    root = Path(root)
    lines = [f"{r}\t{hashlib.sha256((root / r).read_bytes()).hexdigest()}" for r in source_files(root) if r not in exclude]
    return hashlib.sha256(("\n".join(lines) + "\n").encode()).hexdigest()


def make_zip(out, root):
    import zipfile
    with zipfile.ZipFile(out, "w", zipfile.ZIP_DEFLATED) as z:
        for r in source_files(root):
            zi = zipfile.ZipInfo(f"XVI-Q/{r}", date_time=(2026, 10, 11, 0, 0, 0))
            zi.compress_type = zipfile.ZIP_DEFLATED
            z.writestr(zi, (Path(root) / r).read_bytes())
```

tools/source_set_q06_q07_q08_q09_q10_q11.py

```
"""XVI-Q Q11 source set: Q10 set + Q11 reference values, expected (tools/source_set.py is pinned by
the Q04 layer gate and left unchanged)."""
from __future__ import annotations
import sys
from pathlib import Path

sys.path.insert(0, str(Path(__file__).resolve().parent))
import source_set_q06_q07_q08_q09_q10 as S0  # noqa: E402

GLOBS = tuple(S0.GLOBS) + ("reproducibility/reference_values_q11.json", "code/p6q06/q07/q08/q09/q10/q11/q11_expected.json")
wrap = S0.wrap


def source_files(root):
    root = Path(root)
    out = set()
    for g in GLOBS:
        out |= {p.relative_to(root).as_posix() for p in root.glob(g) if p.is_file() and "__pycache__" not in p.parts}
    return sorted(out)


def tree_sha(root, exclude=()):
    import hashlib
    root = Path(root)
    lines = [f"{r}\t{hashlib.sha256((root / r).read_bytes()).hexdigest()}" for r in source_files(root) if r not in exclude]
    return hashlib.sha256(("\n".join(lines) + "\n").encode()).hexdigest()


def make_zip(out, root):
    import zipfile
    with zipfile.ZipFile(out, "w", zipfile.ZIP_DEFLATED) as z:
        for r in source_files(root):
            zi = zipfile.ZipInfo(f"XVI-Q/{r}", date_time=(2026, 10, 11, 0, 0, 0))
            zi.compress_type = zipfile.ZIP_DEFLATED
            z.writestr(zi, (Path(root) / r).read_bytes())
```